



ABAP RESTful Application Programming Model

Generated on: 2026-07-06 18:30:38 GMT+0000

ABAP Cloud | ABAP Cross-Product

Public

Original content: https://help.sap.com/docs/ABAP_Cloud/f055b8bf582d4f34b91da667bc1fcce6?locale=en-US&state=PRODUCTION&version=sap_cross_product_abap

Warning

This document has been generated from SAP Help Portal and is an incomplete version of the official SAP product documentation. The information included in custom documentation may not reflect the arrangement of topics in SAP Help Portal, and may be missing important aspects and/or correlations to other topics. For this reason, it is not for production use.

For more information, please visit <https://help.sap.com/docs/disclaimer>.

Developing Transactional Apps with Exclusive Draft Capabilities

This chapter is a walk-through tutorial that guides you through the main implementation steps to develop an application with exclusive draft support. Draft-enabled applications provide the end user with the highest flexibility when working with the app.

From a technical perspective, the outcome of these chapters is a business service that is developed based on the implementation type **managed with draft** of the ABAP RESTful Application Programming Model using UUID keys. For more information about draft business objects, see [Draft](#). The business object (BO) is exposed as a UI service. In this example scenario, the Fiori Elements App Preview is used to illustrate the examples. Yet, the same business object can also be consumed as a Web API.

Introduction

The standard use case for creating a managed transactional app with draft capabilities is the following:

You want to create a completely new transactional app for a Fiori Elements UI following the greenfield approach. That means, there is no legacy business application logic involved. You create everything from scratch. Following from that, you can benefit from the lightweight BO standard implementation of the managed implementation type, in which the RAP managed runtime framework assumes an extensive part of the standard implementation with out-of-the-box support for **create**, **update**, and **delete**. In addition, you may need to provide the end user with capabilities to store changed data in the backend at any time and proceed at a later point in time, or to recover such data from a different device, even if the application client has terminated unexpectedly. This use case is supported by the draft concept of the ABAP RESTful Application Programming Model. Draft-enabled business objects persist the state of the transactional buffer after every transaction on a designated draft database table. This allows the end user to stop and continue work processes at any point in time, even with inconsistent data. For more detailed information about the draft concept, see [Draft](#).

The example scenario uses a data model with UUID-key layout. In managed scenarios, UUIDs are beneficial as they can be drawn automatically by the RAP managed framework upon creating new instances. However, the draft scenario can also be developed with semantic primary keys.

Prerequisites

Developing the scenario that is described in the subsequent chapters requires the following:

- You fulfill the requirements described in [Prerequisites](#).
- You have installed ABAP Development Tools (ADT).

SAP recommends to use the latest version of the client installation. The ADT download is available on the update site <https://tools.hana.ondemand.com/>.

- To recreate the demo scenario, the **ABAP Flight Reference Scenario** must be available in your ABAP system. You can download the complete reference scenario from [GitHub: Downloading the ABAP Flight Reference Scenario](#).

Development Flow

The example development scenario is designed to support two approaches. On the one hand, it teaches you how to build a managed business object with exclusive draft from scratch. On the other hand, if you just want to add draft capabilities to your existing BO, you can leave out the first steps to create a managed business object and start directly with [Draft-Enabling the Managed Business Object](#).

The development of an application with draft capabilities based on a managed business object requires the following activities:

1. Providing the CDS Data Model based on UUID keys

The first chapter provides you with information on how to create suitable database tables and CDS view entities for the managed business object. The example scenario guides you to build a three-node business object, which illustrates how root and child entities are handled in the draft scenario.

See [Data Model and Business Object Structure](#).

2. Developing Transactional Behavior for the Managed Business Object

In this section, you will equip the data model with transactional behavior. The business object framework for managed implementation type provides you with out-of-the-box support for standard implementation tasks. Other business-specific logic must be implemented using actions, determinations, and validations.

See [Developing Transactional Behavior](#).

3. Exposing the Managed Business Object for a UI service

Consuming the managed business object via a UI requires defining the projection layer, where you define all service-specific features, like UI annotations for UI configuration, search capabilities or value helps. The projected BO and all its related entities must then be defined in the service definition and then bound to a protocol. This UI service remains stable even when you add draft capabilities later on. Whereas the underlying BO changes, the BO projection filters only the non-draft features. The steps will be explained in this section.

See [Exposing the Managed Business Object for a UI Business Service](#).

4. Adding Exclusive Draft Capabilities to the Managed Business Object

This section explains how you draft-enable the existing business object with exclusive draft. On the one hand, draft-enabling requires the revision of all existing business logic. Sometimes the BO provider needs to distinguish if draft instances or active instances are processed. On the other hand, there are some draft features that you need to implement to add draft qualities to the business service.

See [Draft-Enabling the Managed Business Object](#).

5. Exposing the Draft-Enabled Business Object for a UI service

To enable the consumption of the draft-enabled BO via a Fiori Elements UI, the projection layer must be defined. It reuses all the draft capabilities that are defined in the underlying layer. In addition, the service must be defined and bound to a protocol.

See [Exposing the Draft Business Object for a UI Business Service](#).

Related Information

[Exclusive Draft Handling](#)

Building a Managed Business Object with UUID Keys

The first part of the managed development scenario with draft describes setting up a managed business object from scratch.

The assumption in this scenario is that you do not have any data or legacy coding that you want to include in the new business object. Everything is created from scratch. Therefore the first step is to create database table to store the data that the business object consumes. We use a simplified approach to have one database table for each business object entity. In real-world scenario, this might rarely be the case, but this demo scenario mainly demonstrates how you handle the transactional behavior with draft for the business object and does not focus on database structures for business objects.

The key structure of the data model in this scenario is based on UUID keys. That means, every BO entity instance is uniquely identifiable by one single unique key.

Based on these database tables, the data model and structure for the business object is defined via CDS view entities. For more information about the basic structure of a business object, see [Data Modeling and Behavior](#).

In a second step, the second component of a business object, the behavior, is defined and implemented for the business object. For more information about the business object behavior, see [Business Object](#)

1. [Data Model and Business Object Structure](#)
2. [Developing Transactional Behavior](#)

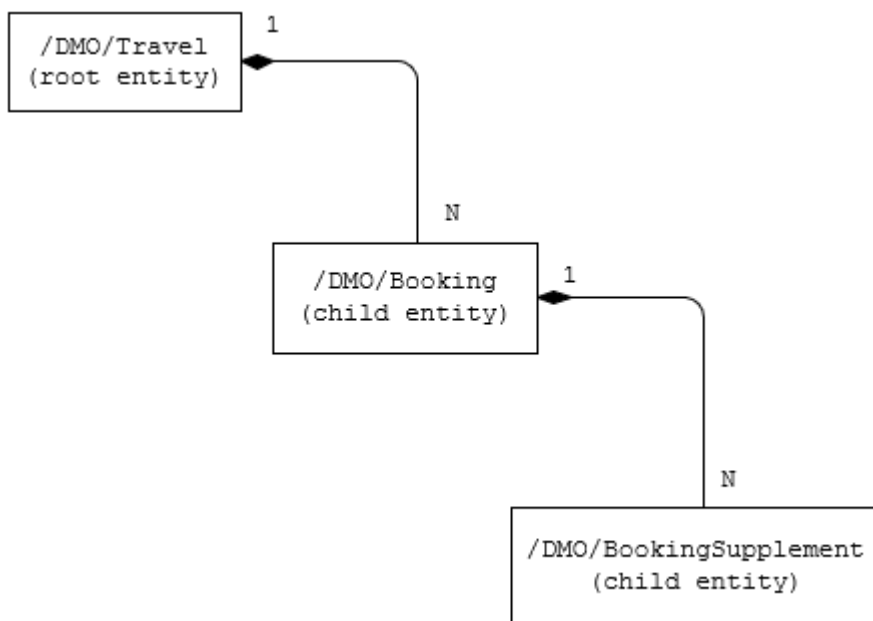
Data Model and Business Object Structure

The draft scenario uses the same data model as the other development scenarios. The final app processes travel data, primarily flight data, to create, update, and delete travel instances.

The travel business object consists of three entities that are structured in a hierarchy tree. Every entity in the BO-composition tree is modeled with a CDS view entity. The following entities are used for the transactional data model:

- **Travel:** The root entity comprises general travel data, like a trip's start and end dates, and the relevant customer and agency data.
- **Booking:** The booking entity depicts which bookings are relevant for a certain travel. In the current data model, only flights are available as bookings.
- **BookingSupplement:** The booking supplement entity presents additional bookable supplement for a certain booking. These are meals and beverages for the flights.

The BO-entities are related to each other in a composition structure. They maintain a dependency relationship, which means that a child entity cannot exist without its parent entity. In addition, a parent entity can have one to many child entities. The following diagram illustrates this relationship.



Editable Entities of the Business Object

The root entity is the head of the business object. You address a BO with the name of the root entity. It is also the root entity that receives special capabilities during runtime. In managed scenarios, only root entities can be lock masters. That means, if one of the

entities that are part of the composition tree is requested for lock, it is the lock master instance and its child instances that are locked for the request. In addition, in the draft scenario, the lock master, in this case the root entity, receives an additional field for optimistic concurrency control in the transition from draft to active data. This field is called `total ETag`. For more information about the locking mechanism and the total ETag, see [Concurrency Control](#).

The composition tree includes child entities that are also part of the business object. They are connected via compositions, bidirectional connections by which the relationship to the root entity is defined.

Whereas the BO-entities are built up from scratch, as you create both, the database table and the related CDS entity, there are additional entities for master data that are reused from the demo content of the ABAP Flight Reference Scenario. These entities are not part of the transactional BO-structure. Most of these entities are used primarily as value help provider views for individual input fields on the UI.

- ▼  /DMO/FLIGHT_REUSE (51) *Flight Reference Scenario: Reused Entities*
 - >  /DMO/FLIGHT_REUSE_SUPPLEMENT (27) *Flight Reference Scenario: Supplement*
 - >  Authorizations (2)
 - ▼  Core Data Services (12)
 - ▼  Data Definitions (12)
 - >  /DMO/I_AGENCY *Travel View - CDS Data Model*
 - >  /DMO/I_AIRPORT *Airport View - CDS Data Model*
 - >  /DMO/I_BOOKING_STATUS_VH_TEXT *Overall Status Value Help*
 - >  /DMO/I_BOOKING_STATUS_VH *Overall Status Value Help*
 - >  /DMO/I_CARRIER *Carrier View - CDS Data Model*
 - >  /DMO/I_CONNECTION *Connection View - CDS Data Model*
 - >  /DMO/I_CUSTOMER *Customer View - CDS Data Model*
 - >  /DMO/I_FLIGHT *Flight View - CDS Data Model*
 - >  /DMO/I_OVERALL_STATUS_VH_TEXT *Overall Status Value Help*
 - >  /DMO/I_OVERALL_STATUS_VH *Overall Status Value Help*

Non-Editable Entities Reused in the Data Model

Additional entities for currencies and countries are generally available in your system and are included in out data model using associations. These are `I_Currency` and `I_Country`.

Tasks for the Developer

1. [Creating Persistent Database Tables](#)
2. [Creating CDS View Entities](#)

Creating Persistent Database Tables

For the managed scenario with draft, create new database tables for the business object data model that match the transactional data structure of travel, booking and booking supplement.

Context and Procedure

In scenarios without existing legacy code, the first step is to create persistent database tables that store data.

Create these tables by using the creation wizard for creating database tables in ADT.

i Note

The draft scenario in the ABAP Flight Reference Scenario uses the suffix `_D`. For the persistent database tables, we use the prefix `A_` to indicate the active persistence. For detailed information, see [Naming Conventions for Development Objects](#).

When creating database tables for a managed business object with draft that use UUID key layout, the following considerations are relevant:

- The RAP managed runtime engine requires administrative data fields for internal processing of the BO entities. Especially, a field that denotes the timestamp when the instance was last changed is important for the ETag master handling on each entity. The RAP framework offers reuse data elements to define administrative fields on the database, see [RAP Reuse Data Elements](#). If the administrative fields on local instances are annotated in CDS with the annotation `@Semantics.systemDateTime.localInstanceLastChangedAt: true`, they are automatically filled by the RAP managed runtime engine. For more information, see [Optimistic Concurrency Control](#).
- In a UUID scenario, every entity needs a field for the UUID key. In addition, child entities must have a field for their parent's UUID. Otherwise, you cannot define associations for the composition relationship. In addition, any child entity that has no parent-child relationship with the lock master entity must have a field with the lock master's UUID. Otherwise you cannot define associations to the lock master entity, which is important for lock and ETag handling.
- Apart from the UUID key, each database table also has a semantic ID field. These fields are no technical keys, but are used to semantically distinguish the entity instance in the business object.

Table /DMO/A_TRAVEL_D: Persistent Table for Managing Travel Data

This tables defines general travel data, data that describe the basic information of a trip, such as the travel ID, customer and agency ID, and dates. Since this scenario is based on a UUID key layout, the table also entails a field with a UUID data type. In addition, the fields for standard administration data, such as the respective user or the time of creation are added to the table. In managed business objects, these fields can be filled automatically by the RAP managed runtime framework if the corresponding `@Semantics` annotations are added to the fields in the CDS view entity. Administrative fields, in particular the field that determines when instance data is changed, are important for concurrency control.

i Expand the following code sample to view the source code of /DMO/A_TRAVEL_D.

Sample Code

```
@EndUserText.label : 'Active Travel Persistence for Draft Reference Scenario'
@AbapCatalog.enhancement.category : #NOT_EXTENSIBLE
@AbapCatalog.tableCategory : #TRANSPARENT
@AbapCatalog.deliveryClass : #A
@AbapCatalog.dataMaintenance : #RESTRICTED
define table /DMO/A_TRAVEL_D {
  key client          : abap.clnt not null;
  key travel_uuid     : sysuuid_x16 not null;
  travel_id           : /dmo/travel_id not null;
  agency_id           : /dmo/agency_id;
  customer_id         : /dmo/customer_id;
  begin_date          : /dmo/begin_date;
  end_date            : /dmo/end_date;
  @Semantics.amount.currencyCode : '/dmo/a_travel_d.currency_code'
  booking_fee         : /dmo/booking_fee;
  @Semantics.amount.currencyCode : '/dmo/a_travel_d.currency_code'
  total_price         : /dmo/total_price;
  currency_code       : /dmo/currency_code;
  description         : /dmo/description;
  overall_status      : /dmo/overall_status;
  @AbapCatalog.anonymizedWhenDelivered : true
  local_created_by    : abp_creation_user;
  local_created_at    : abp_creation_tstmpl;
  @AbapCatalog.anonymizedWhenDelivered : true
  local_last_changed_by : abp_locinst_lastchange_user;
```

```

    local_last_changed_at : abp_locinst_lastchange_tstmpl;
    last_changed_at       : abp_lastchange_tstmpl;

}

```

Table /DMO/A_BOOKING_D: Persistent Table for Managing Booking Data

This table defines the booking data for a specific travel instance. It contains general booking data, such as the booking ID, booking date, and general flight data. To identify a booking instance uniquely, the table must contain a UUID field for the booking. In UUID scenarios, the parent UUID acts as a foreign-key field in the database for the child entity. For the ETag master handling, the booking database table receives a field to store the timestamp when the instance was updated.

i Expand the following code sample to view the source code of /DMO/A_BOOKING_D.

Sample Code

```

@EndUserText.label : 'Active Booking Persistence for Draft Reference Scenario'
@AbapCatalog.enhancement.category : #NOT_EXTENSIBLE
@AbapCatalog.tableCategory : #TRANSPARENT
@AbapCatalog.deliveryClass : #A
@AbapCatalog.dataMaintenance : #RESTRICTED
define table /DMO/A_BOOKING_D{
    key client          : abap.clnt not null;
    key booking_uuid    : sysuuid_x16 not null;
    parent_uuid         : sysuuid_x16;
    booking_id          : /dmo/booking_id not null;
    booking_date        : /dmo/booking_date;
    customer_id         : /dmo/customer_id;
    carrier_id          : /dmo/carrier_id;
    connection_id       : /dmo/connection_id;
    flight_date         : /dmo/flight_date;
    @Semantics.amount.currencyCode : '/dmo/a_booking_d.currency_code'
    flight_price        : /dmo/flight_price;
    currency_code       : /dmo/currency_code;
    booking_status      : /dmo/booking_status;
    local_last_changed_at : abp_locinst_lastchange_tstmpl;

}

```

Table /DMO/A_BKSUPPL_D: Persistent Table for Managing Booking Supplement Data

This table defines the supplement for a specific booking instance. It contains general booking supplement data, such as the booking supplement ID and price information. To identify a booking supplement instance uniquely, the table must contain a UUID field for the booking supplement. In UUID scenarios, the parent UUID and the lock master UUID (in our case the root UUID) act as a foreign-key field in the database. For the ETag master handling, the booking supplement database table receives a field to store the timestamp when the instance was updated.

i Expand the following code sample to view the source code of /DMO/A_BKSUPPL_D.

Sample Code

```

@EndUserText.label : 'Active BookingSupplement Persistence for Draft Ref. Scenario'
@AbapCatalog.enhancement.category : #NOT_EXTENSIBLE
@AbapCatalog.tableCategory : #TRANSPARENT
@AbapCatalog.deliveryClass : #A
@AbapCatalog.dataMaintenance : #RESTRICTED
define table /DMO/A_BKSUPPL_D{
    key client          : abap.clnt not null;
    key booksuppl_uuid  : sysuuid_x16 not null;
    root_uuid           : sysuuid_x16;

```

```

parent_uuid      : sysuuid_x16;
booking_supplement_id : /dmo/booking_supplement_id not null;
supplement_id    : /dmo/supplement_id;
@Semantics.amount.currencyCode : '/dmo/a_bksuppl_d.currency_code'
price            : /dmo/supplement_price;
currency_code    : /dmo/currency_code;
local_last_changed_at : abp_locinst_lastchange_tstmpl;
}

```

Creating CDS View Entities

For the managed scenario with draft, create data definitions for CDS view entities to build the structure for the business object.

Context and Procedure

Previously, you have created persistent database table for the travel business object. The structure of the managed business object must now be defined by CDS data modeling.

Create data definitions for CDS view entities that represent the three transactional entities of the business object: **Travel**, **Booking**, and **Booking Supplement**.

i Note

CDS view entities are the successor DDIC-based CDS views. CDS view entities offer many advantages, such as optimization and simplification of syntax checks, and improved performance during view activations.

Select the template **Define Root View Entity** for the travel entity.

Select the template **Define View Entity with To-Parent Association** for the booking and the booking supplement entity.

i Note

The draft scenario in the ABAP Flight Reference Scenario uses the suffix `_D`. Since CDS view entities are interface views, they are prefixed with `R_` accordance with the VDM (virtual data model) naming convention. For detailed information, see [Naming Conventions for Development Objects](#).

When using CDS view entities for a managed business object with draft that use UUID key layout, the following considerations are relevant:

- For a business object, you must define a root entity that represents the head of the business object and provides the name with which the BO is addressed.
- The composition structure of the managed business object must be defined in the CDS view entities by compositions to the child entity and associations to the parent entity. In addition, an association to the lock and authorization master entity (in our scenario, the root entity) must be available from all entities in the business object.
- The RAP managed runtime framework fills the administrative fields automatically if they are annotated with the corresponding `@Semantics` annotation. These fields are only necessary on the root entity of the business object. In particular, the field that is annotated with `@Semantics.systemDateTime.localInstanceLastChangedAt: true` is updated whenever the instance is changed. In our scenario, this field is used for ETag comparison.

Data Definition /DMO/R_Travel_D: Travel Root View

The data source for the travel root view is the database table `/DMO/A_TRAVEL_D`. All fields from the database table are used for the data model.

To retrieve master data for value helps from other entities, define the association to the CDS entities

`/DMO/I_Agency`, `/DMO/I_Customer`, `I_Currency`, `/DMO/I_Overall_Status_VH` and `I_Currency`. The former are part of the ABAP Flight Reference Scenario, which you can download. The CDS view `I_Currency` is generally available in your development system.

Annotate the price fields with the annotations `@Semantics.amount.currencyCode` to define which field is used as currency.

i Note

In contrast to DDIC-based CDS views, in CDS view entities, you do not need to specify the currency code field as a currency code if the field is already specified in the database table.

Use semantics annotations for the administrative fields. The ETag field `local_instance_last_changed_at` receives the annotation `@Semantics.systemDateTime.localInstanceLastChangedAt: true`.

As soon as the entity of the composition structure exists, define the compositional relationship to the child entity `/DMO/R_Booking_D` with the keyword `composition`.

i Expand the following code sample to view the source code of `/DMO/R_Travel_D`.

≡ Sample Code

```
@EndUserText.label: 'Travel View Entity for Draft RefScen'
@AccessControl.authorizationCheck: #NOT_REQUIRED

define root view entity /DMO/R_Travel_D
as select from /DMO/A_TRAVEL_D

    composition [0..*] of /DMO/R_Booking_D as _Booking

    association [0..1] to /DMO/I_Agency as _Agency      on $projection.AgencyID = _Agency.AgencyID
    association [0..1] to /DMO/I_Customer as _Customer  on $projection.CustomerID = _Customer.CustomerID
    association [1..1] to /DMO/I_Overall_Status_VH as _OverallStatus on $projection.OverallStatus = _OverallStatus.OverallStatus
    association [0..1] to I_Currency as _Currency      on $projection.CurrencyCode = _Currency.CurrencyCode

{ ///DMO/A_TRAVEL_D
    key travel_uuid          as TravelUUID,

        travel_id           as TravelID,
        agency_id           as AgencyID,
        customer_id         as CustomerID,
        begin_date           as BeginDate,
        end_date             as EndDate,
        @Semantics.amount.currencyCode: 'CurrencyCode'
        booking_fee         as BookingFee,
        @Semantics.amount.currencyCode: 'CurrencyCode'
        total_price         as TotalPrice,
        currency_code       as CurrencyCode,
        description         as Description,
        overall_status      as OverallStatus,

        @Semantics.user.createdBy: true
        local_created_by    as LocalCreatedBy,
        @Semantics.systemDateTime.createdAt: true
        local_created_at    as LocalCreatedAt,
        @Semantics.user.localInstanceLastChangedBy: true
        local_last_changed_by as LocalLastChangedBy,
        //local ETag field --> OData ETag
        @Semantics.systemDateTime.localInstanceLastChangedAt: true
        local_last_changed_at as LocalLastChangedAt,
```

```

//total ETag field
@Semantics.systemDateTime.lastChangedAt: true
last_changed_at      as LastChangedAt,

//Associations
_booking,

_agency,
_customer,
_overallStatus,
_currency
}

```

Data Definition /DMO/R_Booking_D: Booking View

The data source for the booking view is the database table /DMO/A_BOOKING_D. All fields from the database table are used for the data model.

To retrieve master data for value helps from other entities, define the association to the CDS entities /DMO/I_Customer, /DMO/I_Carrier, /DMO/I_Connection and /DMO/I_Booking_Status_VH. These CDS views are part of the ABAP Flight Reference Scenario, which you can download.

Annotate the price fields with the annotations @Semantics.amount.currencyCode to define which field is used as currency.

i Note

In contrast to CDS views, in CDS view entities, you do not need to specify the currency code field as a currency code if the field is already specified in the database table.

Use semantics annotations for the administrative field. The local ETag field local_instance_last_changed_at receives the annotation @Semantics.systemDateTime.localInstanceLastChangedAt: true.

As soon as the entities of the composition structure exist, define the compositional relationship to the child entity /DMO/R_BookingSupplement_D with the keyword composition, and define the compositional relationship to the parent entity /DMO/R_Travel_D with the keyword association to parent.

i Expand the following code sample to view the source code of /DMO/R_Booking_D.

≡ Sample Code

```

@EndUserText.label: 'Booking View Entity for Draft RefScen'
@AccessControl.authorizationCheck: #NOT_REQUIRED

define view entity /DMO/R_Booking_D as select from /DMO/A_BOOKING_D

    association          to parent /DMO/R_Travel_D as _Travel          on $projection.TravelUUID = _
    composition [0..*] of /DMO/R_BookingSupplement_D as _BookingSupplement

    association [1..1] to /DMO/I_Customer          as _Customer          on $projection.CustomerID
    association [1..1] to /DMO/I_Carrier            as _Carrier            on $projection.AirlineID
    association [1..1] to /DMO/I_Connection        as _Connection        on $projection.AirlineID
                                         and $projection.ConnectionID
    association [1..1] to /DMO/I_Booking_Status_VH as _BookingStatus on $projection.BookingSta

{ ///DMO/A_BOOKING_D
    key booking_uuid          as BookingUUID,
      parent_uuid            as TravelUUID,

      booking_id              as BookingID,
      booking_date            as BookingDate,
      customer_id             as CustomerID,

```

```

        carrier_id          as AirlineID,
        connection_id       as ConnectionID,
        flight_date         as FlightDate,
        @Semantics.amount.currencyCode: 'CurrencyCode'
        flight_price        as FlightPrice,
        currency_code       as CurrencyCode,
        booking_status      as BookingStatus,

        //local ETag field --> OData ETag
        @Semantics.systemDateTime.localInstanceLastChangedAt: true
        local_last_changed_at as LocalLastChangedAt,

        //Associations
        _Travel,
        _BookingSupplement,

        _Customer,
        _Carrier,
        _Connection,
        _BookingStatus
    }

```

Data Definition /DMO/R_BookingSupplement_D: Booking Supplement View

The data source for the booking view is the database table /DMO/A_BKSUPPL_D. All fields from the database table are used for the data model.

To retrieve master data for value helps and texts from other entities, define the association to the CDS entities /DMO/I_Supplement and /DMO/I_SupplementText. These CDS views are part of the ABAP Flight Reference Scenario, which you can download.

Annotate the price fields with the annotations @Semantics.amount.currencyCode to define which field is used as currency.

i Note

In contrast to CDS views, in CDS view entities, you do not need to specify the currency code field as a currency code if the field is already specified in the database table.

Use semantics annotations for the administrative field. The local ETag field local_instance_last_changed_at receives the annotation @Semantics.systemDateTime.localInstanceLastChangedAt: true.

As soon as the entities of the composition structure exist, define the compositional relationship to the parent entity /DMO/R_Booking_D with the keyword association to parent, and define the compositional relationship to the root entity /DMO/R_Travel_D.

i Expand the following code sample to view the source code of /DMO/R_BookingSupplement_D.

Sample Code

```

@EndUserText.label: 'BookSuppl View Entity fro Draft RefScen'
@AccessControl.authorizationCheck: #NOT_REQUIRED

define view entity /DMO/R_BookingSupplement_D
as select from /DMO/A_BKSUPPL_D

    association          to parent /DMO/R_Booking_D as _Booking          on $projection.BookingUUID
    association [1..1] to /DMO/R_Travel_D as _Travel          on $projection.TravelUUID = _Travel

    association [1..1] to /DMO/I_Supplement          as _Product          on $projection.SupplementID
    association [1..*] to /DMO/I_SupplementText      as _SupplementText on $projection.SupplementID

```

```

{ ///DMO/A_BKSUPPL_D
  key booksuppl_uuid      as BookSupplUUID,
    root_uuid             as TravelUUID,
    parent_uuid           as BookingUUID,

    booking_supplement_id as BookingSupplementID,
    supplement_id         as SupplementID,
    @Semantics.amount.currencyCode: 'CurrencyCode'
    price                 as BookSupplPrice,
    currency_code         as CurrencyCode,

    //local ETag field --> OData ETag
    @Semantics.systemDateTime.localInstanceLastChangedAt: true
    local_last_changed_at as LocalLastChangedAt,

    //Associations
    _Travel,
    _Booking,

    _Product,
    _SupplementText
}

```

Developing Transactional Behavior

This topic describes the behavior that is used for the business object in the managed scenario with draft.

The managed business object comprises the following behavior. The keywords in brackets refer to the technical features for this behavior.

1. Root Entity **Travel**:

- a. You can create new instances of the travel entity.

On creating travel instances, the following behavior is implemented:

- The UUID key for new instances is automatically assigned (managed internal numbering).
- The travel UUID, the travel ID, the overall status, the total travel price, and administrative fields can't be provided by the consumer (read-only fields).
- The semantic travel ID is calculated (determination on save).
- The overall status for new travel instances is set to initial (determination on modify).
- The administrative fields are filled automatically (internal draft processing of the RAP runtime framework).
- The total travel price is calculated based on the booking fee of new travels (determination on modify, internal action).
- The customer ID, the agency ID, the begin and the end date must be provided on creating new travel instances (mandatory fields).
- All modify operations are checked for authorization against the user profile (global authorization).
- The travel instance can only be modified depending on the country code of the agency that is used for the respective travel. (instance authorization)
- The agency ID is checked for existence, validity, and authorization (validation on save).
- The customer ID is checked for existence and validity (validation on save).
- The travel dates are checked for consistency (validation on save).

b. You can update instances of the travel entity.

On updating travel instances, the following behavior is implemented:

- The travel ID, the overall status, the total travel price, and administrative fields cannot be modified BO-externally (read-only fields).
- You can only update the booking fee if the overall status is not set to accepted ('A') (dynamic field control).
- If the customer ID, the agency ID or the travel dates are changed, they are checked for existence and consistency (validations on save).
- You can set the overall status to accepted ('A') by an action, if the travel does not already have this status (dynamic action control).
- You can set the overall status to rejected ('X') by an action, if the travel does not already have this status (dynamic action control).
- You can deduct a discount for the travel with a parameter action if the overall status is not already accepted ('A') (action with parameter).

c. You can delete instances of the travel entity. Due to compositional relationship, all instances of child entities are deleted as well.

2. Child Entity **Booking**

a. You can create new booking instances for travel instances.

On creating booking instances, the following behavior is implemented:

- The UUID key for new instances is automatically assigned (managed internal numbering).
- The travel UUID, the booking ID, the booking date, and administrative fields cannot be provided by the consumer (read-only fields).
- The semantic booking ID is calculated (determination on save).
- The booking date is set automatically (determination on save).
- The administrative fields are filled automatically (internal draft processing of the RAP runtime framework).
- The customer ID, the airline ID, the connection ID, and the flight date must be provided on creating new booking instances (mandatory fields).
- The customer ID is checked for existence and validity (validation on save).
- The airline ID is checked for existence and compatibility with the connection ID and the flight date (validation on save).
- The connection ID is checked for existence and compatibility with the airline ID and the flight date (validation on save).
- The flight date is checked for compatibility with the airline ID and the connection ID (validation on save).

b. You can update instances of the booking entity.

On updating booking instances, the following behavior is implemented:

- The booking UUID, travel UUID, the booking ID, the booking date, and administrative fields cannot be modified BO-externally (read-only fields).
- If the customer ID, the airline ID, the connection ID, or the flight date are changed, they are checked for existence and consistency (validations on save).

- c. You can delete instances of the booking entity. Due to compositional relationship, all instances of the child entity are deleted as well.

3. Child entity **BookingSupplement**

- a. You can create new booking supplement instances for booking instances.

On creating booking supplement instances, the following behavior is implemented:

- The UUID key for new instances is automatically assigned (managed internal numbering).
- The travel UUID, the booking UUID, the booking supplement UUID, the booking supplement ID, and administrative fields cannot be provided by the consumer (read-only fields).
- The semantic booking supplement ID is calculated (determination on save).
- The administrative fields are filled automatically (internal draft processing of the RAP runtime framework).
- The supplement ID must be provided on creating new booking supplement instances (mandatory field).
- The supplement ID is checked for existence and validity (validation on save).

- b. You can update instances of the booking supplement entity.

On updating booking supplement instances, the following behavior is implemented:

- The travel UUID, the booking UUID, the booking supplement UUID, the booking supplement ID, and administrative fields be modified BO-externally (read-only fields).
- If the supplement ID is changed, it is checked for existence and consistency (validation on save).

- c. You can delete instances of the booking supplement entity.

Defining Standard Behavior

For the managed scenario with draft, create a behavior definition to define the behavior for the business object.

Context and Procedure

The behavior definition is the development object that defines the implementation type behavior for the business object. When using the implementation type **managed**, for some features, you only need to define behavior characteristics and standard operations in the behavior definition. The RAP managed runtime framework provides a generic solution for

- `strict mode`
- `create`
- `update`
- `delete`
- `create-by-association`
- `lock handling`
- `ETag handling`.

For more information, see [Defining Elementary Behavior for Ready-to-Run Business Object](#).

Create the behavior definition by using the creation wizard in ADT. By using the context menu on the root view (/DMO/R_Travel_D), you can directly create a behavior definition for the travel data model. In the wizard, choose the implementation **Managed**.

For the demo scenario with a managed business object with draft, the following considerations are relevant.

- The behavior of each entity is implemented in a separate behavior class pool. Hence, the implementation class must be created for each entity separately. For more information, see [Best Practices for Modularization and Performance](#).
- The managed implementation type requires the specification of `lock master` or `lock dependent` on each entity. For more information, see [Pessimistic Concurrency Control \(Locking\)](#).
- The strict mode ensures RAP best practices and up-to-date syntax for the behavior definition and behavior implementation. For conceptual background, see [Strict Mode](#).
- It is recommended to use authorization control to protect your business object against unauthorized access. Define `authorization master` or `authorization dependent by _Assoc` on each entity. The authorization behavior must then be implemented in the corresponding methods in the behavior pool. For conceptual background, see [Authorization Control](#). For detailed information about the implementation, see [Developing Authorization](#).
- Define the authorization context with reference to the authorization object /DMO/TRVL. Define a full authorization context by `define own authorization context by privileged mode`. In addition define privileged access.
- In managed business objects, it is best practice to define a local ETag master on each entity. With an ETag master on every entity, you ensure that the ETag check is done for every entity independently. For more information, see [Optimistic Concurrency Control](#).
- The RAP managed runtime framework is able to automatically draw primary key values in UUID scenarios. You use this functionality by defining early managed numbering in the behavior definition. Setting the primary key field to `read only` defines strict internal numbering. An external BO consumer is not allowed to provide the primary key values in this case. For more detailed information, see [Automatically Drawing Primary Key Values in Managed BOs](#).
- If different names are used on the database table and in the CDS data model, you need to define the mapping for these fields. This is done via the mapping operator in the behavior definition.

Behavior Definition /DMO/R_Travel_D

The template for the **managed** implementation type provides the behavior definition for every entity in the composition tree of the root entity. In addition, it specifies the data source of the respective entities as persistent table. The behavior definition proposes to include the behavior characteristics `lock`, `authorization` and `etag`. The standard operations `create`, `update` and `delete`, as well as the association `_Booking {create}` are included in the template for each entity. The create operation in child entities is not supported in RAP scenarios. Instances of child entities can only be created by create-by-association operation.

Define the root entity as `lock master`. The current version of the ABAP RESTful Application Programming Model only supports the specification of `lock master` on the root entity. Define the booking entity and the booking supplement entity as `lock dependent`. Lock dependents require an association to the lock master entity. In the CDS data model, you have specified the association to the travel entity `_Travel`. To use the association for the lock dependent, define the association as transaction-enabled in the behavior definition. For more information, see [Lock Dependent](#).

As the `TravelUUID` is used for the lock dependent relationship, it must be set to `readonly` in the `Booking` and `BookingSupplement` entity behavior.

Define the root entity as `authorization master`. The current version of the ABAP RESTful Application Programming Model only supports the specification of `authorization master` on the root entity. Define the booking entity and the booking supplement

entity as authorization dependent. Just like lock dependents, authorization dependents require an association to the authorization master entity. For more information, see [Authorization Dependent](#).

Define an ETag master field on each entity. Use the field `LocalLastChangedAt` that is part of the entity's data model. Since this field is annotated with the annotation `@Semantics.systemDateTime.localInstanceLastChangedAt: true`, it is updated automatically by the RAP managed runtime framework. It is recommended that you specify the respective field as read-only in the behavior definition, using the field characteristic `readonly`.

Use early managed numbering for the primary key fields on every entity and set the fields to `read only`, so that the UUID values are drawn strictly internally.

Map the field names in the CDS data definition to the fields on the database table.

For each entity, define an alias and an implementation class. The behavior definition editor provides a quickfix to directly create the behavior class with the name that you specify in the behavior definition. The only behavior that needs implementation is authorization control. The corresponding methods for global and instance authorization control are generated in the local types of the ABAP behavior pool.

i *Expand the following code sample to view the source code of `/DMO/R_Travel_D`.*

Sample Code

```
managed;
strict(2);

with privileged mode disabling NoCheckWhenPrivileged;
define authorization context NoCheckWhenPrivileged { '/DMO/TRVL'; }
define own authorization context by privileged mode;

define behavior for /DMO/R_Travel_D alias Travel
implementation in class /dmo/bp_i_travel_d unique
persistent table /DMO/A_TRAVEL_D
lock master
authorization master ( instance,global )
etag master LocalLastChangedAt
{
  create;
  update;
  delete;
  association _Booking { create; }

  field ( numbering : managed, readonly ) TravelUUID;
  field ( readonly ) LocalLastChangedAt;

  mapping for /DMO/A_TRAVEL_D
  { AgencyID = agency_id;
    BeginDate = begin_date;
    BookingFee = booking_fee;
    CurrencyCode = currency_code;
    CustomerID = customer_id;
    Description = description;
    EndDate = end_date;
    LocalCreatedAt = local_created_at;
    LocalCreatedBy = local_created_by;
    LocalLastChangedAt = local_last_changed_at;
    LocalLastChangedBy = local_last_changed_by;
    OverallStatus = overall_status;
    TotalPrice = total_price;
    TravelID = travel_id;
    TravelUUID = travel_uuid; }
}

define behavior for /DMO/R_Booking_D alias Booking
implementation in class /dmo/bp_i_booking_d unique
persistent table /DMO/A_BOOKING_D
lock dependent by _Travel
```



```

authorization dependent by _Travel
etag master LocalLastChangedAt
{
  update;
  delete;
  association _BookingSupplement { create; }

  association _Travel { }

  field ( readonly ) TravelUUID, LocalLastChangedAt;
  field ( numbering : managed, readonly ) BookingUUID;

  mapping for /DMO/A_BOOKING_D
  { AirlineID = carrier_id;
    BookingDate = booking_date;
    BookingID = booking_id;
    BookingStatus = booking_status;
    BookingUUID = booking_uuid;
    ConnectionID = connection_id;
    CurrencyCode = currency_code;
    CustomerID = customer_id;
    FlightDate = flight_date;
    FlightPrice = flight_price;
    LocalLastChangedAt = local_last_changed_at;
    TravelUUID = parent_uuid; }

}

define behavior for /DMO/R_BookingSupplement_D alias BookingSupplement
implementation in class /dmo/bp_i_bookingsupplement_d unique
persistent table /DMO/A_BKSUPPL_D
lock dependent by _Travel
authorization dependent by _Travel
etag master LocalLastChangedAt
{
  update;
  delete;

  association _Travel { }

  field ( readonly ) TravelUUID, LocalLastChangedAt;
  field ( numbering : managed, readonly ) BookSupplUUID;

  mapping for /DMO/A_BKSUPPL_D
  { BookSupplPrice = price;
    BookSupplUUID = booksuppl_uuid;
    BookingSupplementID = booking_supplement_id;
    BookingUUID = parent_uuid;
    CurrencyCode = currency_code;
    LocalLastChangedAt = local_last_changed_at;
    SupplementID = supplement_id;
    TravelUUID = root_uuid; }

}

```

Developing Actions

For the managed scenario with draft, add actions for nonstandard modify operations.

Context and Procedure

Actions are used to manifest business-logic-specific work-flows in one operation. You can implement simple status changes or a complete creation work-flow in one operation. For the UI, you can define action buttons that execute the action directly when the consumer chooses the button. For more detailed information, see [Actions](#).

For this scenario, define and implement the following actions:

- For the **travel entity**, the following actions are described:

- **acceptTravel**

The action sets the travel status to **Accepted (A)**.

Technically speaking, this action is an **instance action** with return parameter `$self`. The value of the field **OverallStatus** is changed by executing a modify request to update this field with the value A.

- **rejectTravel**

The action sets the travel status to **Rejected (X)**.

Technically speaking, this action is an **instance action** with return parameter `$self`. The value of the field **OverallStatus** is changed by executing a modify request to update this field with the value X.

- **deductDiscount**

The action calculates a new travel price by deducting a discount for the **BookingFee**. The consumer can freely choose the percentage that is deducted. The value must be passed together with the action request. On the Fiori UI, the UI provides a pop-up window for the consumer to fill in the requested value. The action parameter is modeled with an abstract entity.

Technically speaking, the action is an **instance action** with an importing action parameter and returns itself. The parameter is modeled with an abstract entity using a simple type for its parameter field. The value of the field **BookingFee** is changed by executing a modify request to update this field with the calculated value.

- **reCalcTotalPrice**

The action calculates the total price for one travel instance. It adds up the prices of all bookings, including their supplements, and the booking fee of the travel instance. If different currencies are used, the prices are converted to the currency of the travel instance.

Technically speaking, the action is an **internal instance action**. This action is invoked by determinations that are triggered when one of the involved fields is changed: **BookingFee** (travel entity), **FlightPrice** (booking entity), and **Price** (booking supplement entity).

For a detailed description on how to implement the described behavior for the relevant entity, check the implementation steps in the following topics.

[Implementing Actions for the Travel Entity](#)

For the managed scenario with draft, define and implement actions on the travel entity.

Implementing Actions for the Travel Entity

For the managed scenario with draft, define and implement actions on the travel entity.

Actions **acceptTravel** and **rejectTravel**

Both actions change the status of a travel instance. They are implemented exactly in the same way: Define the actions in the behavior definition and implement them in the behavior implementation class for **Travel**.

Definition

Define instance actions with result `$self`. The field **OverallStatus** is set to **read only**, so the consumer can only change the status via the actions **acceptTravel** and **rejectTravel**.

≡ Sample Code

```
define behavior for /DMO/R_Travel_D alias Travel
...
{ ...
  field ( readonly ) OverallStatus;

  action acceptTravel result [1] $self;
  action rejectTravel result [1] $self;
  ... }
```

Via a quick fix, you can generate the method declaration in the behavior pool directly from the behavior definition editor.

Implementation

1. Define constants that are available for the travel status in the private section of the local types.

≡ Sample Code

```
CLASS lhc_travel DEFINITION INHERITING FROM cl_abap_behavior_handler.

  PRIVATE SECTION.
    CONSTANTS:
      BEGIN OF travel_status,
        open      TYPE c LENGTH 1 VALUE 'O', "Open
        accepted  TYPE c LENGTH 1 VALUE 'A', "Accepted
        rejected  TYPE c LENGTH 1 VALUE 'X', "Rejected
      END OF travel_status.

    ...
  ENDCLASS.
```

2. Update the field `OverallStatus` with a modify request for all instances with the given keys. Provide the value `accepted` for accepting the travel, and `rejected` for rejecting the travel.
3. The actions return `$self`. That means, they must return the updated instances in the result parameter. For that reason, read the updated instances from the buffer and fill an internal table.
4. Pass the internal table to the result parameter.

i *Expand the following code sample to view the source code of Action **acceptTravel**.*

≡ Sample Code

```
METHOD acceptTravel.

  "Modify travel instance
  MODIFY ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
  ENTITY Travel
    UPDATE FIELDS ( OverallStatus )
    WITH VALUE #( FOR key IN keys ( %tky          = key-%tky
                                     OverallStatus = travel_status-accepted ) ).

  "Read changed data for action result
  READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
  ENTITY Travel
    ALL FIELDS WITH
    CORRESPONDING #( keys )
  RESULT DATA(travels).

  result = VALUE #( FOR travel IN travels ( %tky    = travel-%tky
```

```
%param = travel ) ).
```

```
ENDMETHOD.
```

i Expand the following code sample to view the source code of Action **rejectTravel**.

Sample Code

```
METHOD rejectTravel.

"Modify travel instance
MODIFY ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
ENTITY Travel
  UPDATE FIELDS ( OverallStatus )
  WITH VALUE #( FOR key IN keys ( %tky          = key-%tky
                                OverallStatus = travel_status-rejected ) ).

"Read changed data for action result
READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
ENTITY Travel
  ALL FIELDS WITH
  CORRESPONDING #( keys )
  RESULT DATA(travels).

result = VALUE #( FOR travel IN travels ( %tky   = travel-%tky
                                          %param = travel ) ).

ENDMETHOD.
```

Action deductDiscount

Define the action in the behavior definition and implement it in the behavior implementation class for [Travel](#).

deductDiscount requires an importing parameter for entering the discount percentage. The value must be passed together with the action request. The action parameter is modeled with an abstract entity which uses a simple type for its parameter field.

Creating a Simple Type for the Parameter Field in the Abstract Entity

1. Create a CDS type without enumeration.
2. Define a simple type for the abstract entity field with the data type `abap.int1`.

Sample Code

```
@EndUserText.label: 'Discount Percentage'
define type /DMO/BT_DiscountPercentage: abap.int1
```

Creating an Abstract Entity for the Action Input Parameter

1. Create a data definition for a CDS abstract entity.
2. Define an element for the discount percentage which uses the simple type defined above.

Sample Code

```
@EndUserText.label: 'Abstract Entity for Deducting Discount'
define abstract entity /DMO/A_TRAVEL_DISCOUNT
{
```

```
discount_percent : /DMO/BT_DiscountPercentage;
}
```

Define the action in the behavior definition and implement it in the behavior implementation class for **Travel**.

Definition

Define an instance action with an action parameter `/DMO/A_TRAVEL_DISCOUNT` and the result `$self`.

For the `deductDiscount` action, a default values function is defined which provides default values for the input parameter once the action is triggered on the Fiori UI. For information on how to define and implement this default values function, see [Defaulting Input Parameters for Operations](#).

For conceptual information on default values functions, see [Operation Defaulting](#).

≡, Sample Code

```
define behavior for /DMO/R_Travel_D alias Travel
...
{ ...
  action deductDiscount parameter /DMO/A_TRAVEL_DISCOUNT result [1] $self { default function Ge
... }
```

Via a quick fix, you can generate the method declaration in the behavior pool directly from the behavior definition editor.

Implementation

i Note

The input parameter of parameter actions is passed to the action method as the component `%pa ram` in the importing parameter.

Calculate the new total price for the travel instance by deducting the provided discount percentages. The following steps guide you through the implementation.

1. Define an internal table and pass the entries of the importing parameter `keys`. Loop at the internal table for the keys with parameter values that do not match the preconditions for percentage value:
 - a. It must not be initial.
 - b. It must be less than 100.
 - c. It must be greater or equal than 0.
2. Append the transactional key to the failed table and append the corresponding message to the reported table if there are keys that do not match the preconditions. Delete the entries in the internal table that do not match the preconditions.
3. If there are still entries in the internal table that match the preconditions, read all travel instances with the imported `keys` into an internal table `travels` for the remaining entries.
4. Loop over the internal table and calculate the new reduced booking fee and append it to an internal table for update.
5. Execute a modify request to update the field `BookingFee` for the instances in the buffer.
6. To fill the action result, read the involved travel instances and fill the result correspondingly.

i Expand the following code sample to view the source code of Action **deductDiscount**.

≡ Sample Code

```

METHOD deductDiscount.
  DATA travels_for_update TYPE TABLE FOR UPDATE /DMO/R_Travel_D.
  DATA(keys_with_valid_discount) = keys.

  LOOP AT keys_with_valid_discount ASSIGNING FIELD-SYMBOL(<key_with_valid_discount>) WHERE %pa
                                         OR %param-discount_percent > 100
                                         OR %param-discount_percent <= 0.

    APPEND VALUE #( %tky                                = <key_with_valid_discount>-%tky ) TO failed-tr

    APPEND VALUE #( %tky                                = <key_with_valid_discount>-%tky
                    %msg                                = NEW /dmo/cm_flight_messages(
                                                            textid = /dmo/cm_flight_messages=>discoi
                                                            severity = if_abap_behv_message=>severity
                    %element-TotalPrice                = if_abap_behv=>mk-on
                    %op-%action-deductDiscount          = if_abap_behv=>mk-on
                    ) TO reported-travel.

  DELETE keys_with_valid_discount.
ENDLOOP.

CHECK keys_with_valid_discount IS NOT INITIAL.

"get total price
READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
  ENTITY Travel
    FIELDS ( BookingFee )
    WITH CORRESPONDING #( keys_with_valid_discount )
  RESULT DATA(travels).

LOOP AT travels ASSIGNING FIELD-SYMBOL(<travel>).
  DATA percentage TYPE decfloat16.
  DATA(discount_percent) = keys_with_valid_discount[ KEY id %tky = <travel>-%tky ]-%param-d
  percentage = discount_percent / 100 .
  DATA(reduced_fee) = <travel>-BookingFee * ( 1 - percentage ) .

  APPEND VALUE #( %tky                                = <travel>-%tky
                  BookingFee = reduced_fee
                  ) TO travels_for_update.
ENDLOOP.

"update total price with reduced price
MODIFY ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
  ENTITY Travel
    UPDATE FIELDS ( BookingFee )
    WITH travels_for_update.

"Read changed data for action result
READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
  ENTITY Travel
    ALL FIELDS WITH
    CORRESPONDING #( travels )
  RESULT DATA(travels_with_discount).

result = VALUE #( FOR travel IN travels_with_discount ( %tky    = travel-%tky
                                                         %param = travel ) ).

ENDMETHOD.

```

Action reCalcTotalPrice

The action `reCalcTotal` price is called by determinations if changes on prices in the travel entity itself or in the child entities are executed. The total price on the travel entity is then recalculated.

Definition

Define the internal action in the behavior definition and implement it in the behavior implementation class for **Travel**.

→ Remember

Internal actions can only be called from BO internal consumers.

In our demo scenario, the action is called from determinations on all three BO entities, whenever a price field or a currency code field is changed.

≡ Sample Code

```
define behavior for /DM0/R_Travel_D alias Travel
...
{ ...
  internal action reCalcTotalPrice;
  ... }
```

Via a quick fix, you can generate the method declaration in the behavior pool directly from the behavior definition editor.

Implementation

To determine the total price of a travel, all prices must be converted to the travel currency and then summed up. This is done for each BO entity separately. The following steps guide you through the implementation.

1. Define an internal standard table `amounts_per_currencycode` to process amounts and the related currency code.
2. Read all the instances with the imported keys into an internal table `travels`.
3. Delete instances with empty `CurrencyCode`. For such instances, the total price cannot be calculated.
4. Read all bookings for the returned travels into an internal table `bookings`. Return the link table to receive the associated bookings for each travel.
5. Read all booking supplements for the returned bookings into an internal table `bookingsupplements`. Return the link table to receive the associated bookingsupplements for each affected booking.
6. Loop at `travels` and start filling the amounts per currency with the corresponding values from the travel instance.
7. Loop at `booking_links` where the travel ID matches the respective travel and add the price to the travel price in the table `amounts_per_currencycode` if it has the same currency code. The ABAP statement `COLLECT` is useful in this case, as it sums the amounts with the same currency code and appends a new line if the currency codes differ.
8. Loop at `bookingsupplement_links` where the booking ID matches the respective booking and add the price to the travel price in the table `amounts_per_currencycode` if it has the same currency code.
9. Delete the entries in this table where the `currency_code` is not filled.
10. Loop at the table `amounts_per_currencycode` and convert the amounts with currency codes other than the travel currency code into the currency code of the travel instance.
11. Sum up the converted amounts for one travel.
12. Modify the travel entity with the new total price for all requested travel instances.

i *Expand the following code sample to view the source code of Action `reCalcTotalPrice`.*

≡, Sample Code

```

METHOD reCalcTotalPrice.
  TYPES: BEGIN OF ty_amount_per_currencycode,
           amount          TYPE /dmo/total_price,
           currency_code TYPE /dmo/currency_code,
         END OF ty_amount_per_currencycode.

  DATA: amounts_per_currencycode TYPE STANDARD TABLE OF ty_amount_per_currencycode.

  " Read all relevant travel instances.
  READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
    ENTITY Travel
      FIELDS ( BookingFee CurrencyCode )
      WITH CORRESPONDING #( keys )
    RESULT DATA(travels).

  DELETE travels WHERE CurrencyCode IS INITIAL.

  " Read all associated bookings and add them to the total price.
  READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
    ENTITY Travel BY \_Booking
      FIELDS ( FlightPrice CurrencyCode )
      WITH CORRESPONDING #( travels )
    LINK DATA(booking_links)
    RESULT DATA(bookings).

  " Read all associated booking supplements and add them to the total price.
  READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
    ENTITY Booking BY \_BookingSupplement
      FIELDS ( BookSupplPrice CurrencyCode )
      WITH CORRESPONDING #( bookings )
    LINK DATA(bookingsupplement_links)
    RESULT DATA(bookingsupplements).

  LOOP AT travels ASSIGNING FIELD-SYMBOL(<travel>).
    " Set the start for the calculation by adding the booking fee.
    amounts_per_currencycode = VALUE #( ( amount          = <travel>-bookingfee
                                           currency_code = <travel>-currencycode ) ).

    LOOP AT booking_links INTO DATA(booking_link) USING KEY id WHERE source-%tky = <travel>-%t
      " Short dump occurs if link table does not match read table, which must never happen
      DATA(booking) = bookings[ KEY id %tky = booking_link-target-%tky ].
      COLLECT VALUE ty_amount_per_currencycode( amount          = booking-flightprice
                                                  currency_code = booking-currencycode ) INTO am

    LOOP AT bookingsupplement_links INTO DATA(bookingsupplement_link) USING KEY id WHERE sou
      DATA(bookingsupplement) = bookingsupplements[ KEY id %tky = bookingsupplement_link-ta
      COLLECT VALUE ty_amount_per_currencycode( amount          = bookingsupplement-booksupplp
                                                  currency_code = bookingsupplement-currencyco

    ENDLLOOP.
  ENDLLOOP.

  DELETE amounts_per_currencycode WHERE currency_code IS INITIAL.

  CLEAR <travel>-TotalPrice.
  LOOP AT amounts_per_currencycode INTO DATA(amount_per_currencycode).
    " If needed do a Currency Conversion
    IF amount_per_currencycode-currency_code = <travel>-CurrencyCode.
      <travel>-TotalPrice += amount_per_currencycode-amount.
    ELSE.
      /DMO/CL_FLIGHT_AMDP=>convert_currency(
        EXPORTING
          iv_amount          = amount_per_currencycode-amount
          iv_currency_code_source = amount_per_currencycode-currency_code
          iv_currency_code_target = <travel>-CurrencyCode
          iv_exchange_rate_date = cl_abap_context_info=>get_system_date( )
        IMPORTING
          ev_amount          = DATA(total_booking_price_per_curr)
      ).
      <travel>-TotalPrice += total_booking_price_per_curr.
    ENDIF.
  ENDLLOOP.

```


ENDLOOP.

```
" write back the modified total_price of travels
MODIFY ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
ENTITY travel
  UPDATE FIELDS ( TotalPrice )
  WITH CORRESPONDING #( travels ).
```

ENDMETHOD.

Developing Determinations

For the managed scenario with draft, add determinations to calculate values implicitly.

Context and Procedure

With determinations, you can generalize the calculation of values in the business logic of a managed BO. For more detailed information, see [Determinations](#).

i Note

You can assign determinations to determine actions that are called by RAP side effects. This enables immediate feedback after the user changes UI fields or field groups. For more detailed information, see [RAP Side Effects](#).

For this demo scenario, define and implement determinations for all three BO entities.

- For the **travel entity**, the following determinations are described:

- **setTravelNumber**

The semantic ID for the travel entity is drawn by the determination when a new instance is created. Since the semantic `TravelID` should not be provided by the client, the field must be set to `read only`.

i Note

In this demo scenario, we use a simplified approach to determine new readable IDs for the BO instances. To ensure that gap-free and non-duplicate IDs are assigned, use a number range object.

Technically speaking, this determination is a `determination on save` with the trigger operation `create`.

- **setStatusToOpen**

The travel is set to **Open** when a new instance is created.

Technically speaking, this determination is a `determination on modify` with the trigger operation `create`.

- **calculateTotalPrice**

The determination adds the prices of the travel (`BookingFee`), the booking (`FlightPrice`), and the booking supplement entity (`Price`). The sum of these values is the total price of the travel. The determination is triggered whenever one of the fields or the corresponding currency field is changed, and when a travel instance is created. Since the recalculation should be triggered whenever one of the mentioned fields is changed, the calculation of the total price is outsourced to an action. This action is triggered by a determination on each entity.

i Note

You can only define trigger fields for a determination from the same entity the determination is assigned to. A determination that is defined for the travel entity cannot have trigger fields from the booking entity.

Technically speaking, this determination is a **determination on modify** with the trigger operation **create** and with the field triggers **BookingFee** and **CurrencyCode**.

- For the **booking entity**, the following determinations are described:

- **setBookingNumber**

The semantic ID for the booking entity is drawn by the determination when a new instance is created. Since the semantic **BookingID** should not be provided by the client, the field must be set to **read only**.

Technically speaking, this is a **determination on save** with the trigger operation **create**.

- **setBookingDate**

The booking date is set when the booking instance is saved to the database. The booking date is only set internally by the determination and must not be changed after the instance is saved. Therefore, set the field **BookingDate** to **read only**. #

Technically speaking, this is **determination on save** with the trigger operation **create**.

- **calculateTotalPrice**

See description above. Since the recalculation must also be triggered when the related fields of the booking entity are changed, the determination must be defined and implemented again.

Technically speaking, this is a **determination on modify** with the field triggers **FlightPrice** and **CurrencyCode**.

- For the **booking supplement entity**, the following determinations are described:

- **setBookSupplNumber**

The semantic ID for the booking supplement entity is drawn by the determination when a new instance is created. Since the semantic **BookingSupplementID** should not be provided by the client, the field must be set to **read only**.

Technically speaking, this is a **determination on save** with the trigger operation **create**.

- **calculateTotalPrice**

See description above. Since the recalculation must also be triggered when the related fields of the booking supplement entity are changed, the determination must be defined and implemented again.

Technically speaking, this is a **determination on modify** with the trigger operation **create** and the field triggers **BookSupplPrice** and **CurrencyCode**.

For a detailed description on how to implement the described behavior for the relevant entity, check the implementation steps in the following topics.

[Determining Values for the Travel Entity](#)

For the managed scenario with draft, define and implement determinations on the travel entity for implicit value calculation.

[Determining Values for the Booking Entity](#)

For the managed scenario with draft, define and implement determinations on the booking entity for implicit value calculation.

[Determining Values for the Booking Supplement Entity](#)

Determining Values for the Travel Entity

For the managed scenario with draft, define and implement determinations on the travel entity for implicit value calculation.

Determination `setTravelNumber`

Define the determination in the behavior definition and implement it in the behavior implementation class for `Travel`.

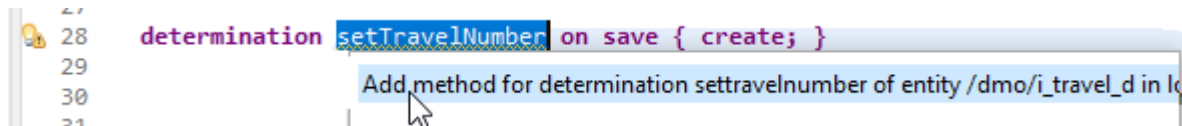
Definition

Define a determination on modify with trigger operation `create`. Since the travel ID must not be changed externally, define the field `TravelID` as `readonly`.

Sample Code

```
define behavior for /DMO/R_Travel_D alias Travel
...
{ ...
  field ( readonly ) TravelID;
  determination setTravelNumber on save { create; }
... }
```

Via a quick fix, you can generate the method declaration in the behavior pool directly from the behavior definition editor.



Quick Fix for Generating Method Declaration in the Behavior Pool

Implementation

Determine the number for a new travel instance by selecting the highest available number from the database and add 1. The following steps guide you through the implementation.

1. Read all the travel instances with the imported keys into an internal table and delete the instances that already have a travel number (`TravelID`). Continue to work only with instances that do not have a travel number yet. If the determination is executed several times, the travel numbers must not be calculated again. This is particularly important in order to stick to the rule of idempotence for determinations.

→ Remember

The determination result must not change if the determination is executed several times under the same conditions (idempotence). See [Rules for Determinations](#).

2. Select the maximum travel number from the database table `/DMO/A_TRAVEL_D`.
3. Update the field `TravelID` of all involved instances with a modify request and assign a new `TravelID` by adding 1 to the max travel from before. Write messages in the request's reported table.

i Note

Since the determination is only executed on save, it is secured that adding 1 to the maximum number in the persisted table returns unique results.

i Expand the following code sample to view the source code of Determination `setTravelNumber`.

Sample Code

```
METHOD setTravelNumber.
  "Ensure idempotence
  READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
    ENTITY Travel
      FIELDS ( TravelID )
      WITH CORRESPONDING #( keys )
      RESULT DATA(travels).

  DELETE travels WHERE TravelID IS NOT INITIAL.
  CHECK travels IS NOT INITIAL.

  "Get max travelID
  SELECT SINGLE FROM /DMO/A_TRAVEL_D FIELDS MAX( travel_id ) INTO @DATA(max_travelid).

  "update involved instances
  MODIFY ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
    ENTITY Travel
      UPDATE FIELDS ( TravelID )
      WITH VALUE #( FOR travel IN travels INDEX INTO i (
                    %tky      = travel-%tky
                    TravelID   = max_travelid + i ) ).

ENDMETHOD.
```

Determination `setStatusToOpen`

Define the determination in the behavior definition and implement it in the behavior implementation class for [Travel](#).

Definition

Define a determination on modify with trigger operation `create`. The overall status of the travel is only changed by the actions `rejectTravel` and `acceptTravel`, see [Accept and Reject Travel](#). Therefore the field is read only for the external consumer.

Sample Code

```
define behavior for /DMO/R_Travel_D alias Travel
...
{ ...
  field ( readonly ) TravelID; OverallStatus;
  determination setStatusToOpen on modify { create; }
... }
```

Via a quick fix, you can generate the method declaration in the behavior pool directly from the behavior definition editor.

Implementation

Set the status to [Open](#) when new instances are created The following steps guide you through the implementation.

1. Read all the travel instances with the imported keys into an internal table and delete the instances that already have an entry in the status field (OverallStatus). Continue to work only with instances that do not have a status yet. If the

determination is executed after the status value is set by the consumer, the status must not be set to open.

2. For those instances, that have an initial value in `OverallStatus` update the field `OverallStatus` with open by executing a modify request for all instances with the imported keys. The constant open was defined before in the private section of the local types. Write messages in the reported table of the modify request.

i Note

Idempotence is not an issue in this determination. Even if the determination is executed several times and an instance already has the status **Open**, the outcome of the determination does not change with the modify request execution.

i Expand the following code sample to view the source code of Determination `setStatusToOpen`.

Sample Code

```
METHOD setStatusToOpen.

  READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
  ENTITY Travel
    FIELDS ( OverallStatus )
    WITH CORRESPONDING #( keys )
  RESULT DATA(travels).

  "If Status is already set, do nothing
  DELETE travels WHERE OverallStatus IS NOT INITIAL.
  CHECK travels IS NOT INITIAL.

  MODIFY ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
  ENTITY Travel
    UPDATE FIELDS ( OverallStatus )
    WITH VALUE #( FOR travel IN travels ( %tky          = travel-%tky
                                              OverallStatus = travel_status-open ) ).

ENDMETHOD.
```

Determination `calculateTotalPrice`

Define the determination in the behavior definition and implement it in the behavior implementation class for **Travel**.

i Note

The actual calculation of the total price is done by the action `recalcTotalPrice`. The determination just executes the action. See [Action `recalcTotalPrice`](#).

Definition

Define a determination on modify with operation trigger `create` and field triggers `BookingFee` and `CurrencyCode` and define the field `TotalPrice` as `read only`, as the value must not be defined externally.

Sample Code

```
define behavior for /DMO/R_Travel_D alias Travel
...
{ ...
  field ( readonly ) TravelID; OverallStatus; TotalPrice;
  determination calculateTotalPrice on modify { create; field BookingFee, CurrencyCode; }
```

```
... }
```

Via a quick fix, you can generate the method declaration in the behavior pool directly from the behavior definition editor.

Implementation

Execute the action `reCalcPrice` when the trigger fields are changed. The following steps guide you through the implementation.

1. Execute a modify request for action execution for all instances with the imported `keys`. Write messages in the reported table of the modify request.

i *Expand the following code sample to view the source code of Determination `calculateTotalPrice`.*

Sample Code

```
METHOD calculateTotalPrice.

    MODIFY ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
    ENTITY Travel
    EXECUTE reCalcTotalPrice
    FROM CORRESPONDING #( keys ).

ENDMETHOD.
```

Determining Values for the Booking Entity

For the managed scenario with draft, define and implement determinations on the booking entity for implicit value calculation.

Determination `setBookingNumber`

Define the determination in the behavior definition and implement it in the behavior implementation class for [Travel](#).

Definition

Define a determination on save with trigger operation `create`. Since the booking ID must not be changed externally, define the field `BookingID` as `read only`.

Sample Code

```
define behavior for /DMO/R_Booking_D alias Booking
...
{ ...
field ( readonly ) BookingID;
determination setBookingNumber on save { create; }
... }
```

Via a quick fix, you can generate the method declaration in the behavior pool directly from the behavior definition editor.

Implementation

Determine the number for a new booking instance by looping at all bookings for one travel and determine the greatest ID. Add 1 for every new booking instance. The following steps guide you through the implementation.

1. Read all the corresponding travel instances for the incoming booking keys into an internal table `travels`. If multiple bookings of the travel are requested, the travel is returned only once.
2. Read all bookings for all affected travels into an internal table `bookings`. Return the link table to receive the associated bookings for each travel.
3. Loop at `travels` and define a variable for the maximum booking ID and set the value to '0000' (initial value).
4. Loop at `booking_links` where the travel key matches the affected travel. Compare the booking ID with the maximum booking ID to determine the maximum booking ID available for the travel instance.
5. Provide a booking ID for all bookings of this travel that have none. Only the booking instances that have no booking ID assigned are taken into account. This is particularly important in order to stick to the rule of idempotence for determinations.

→ Remember

The determination result must not change if the determination is executed several times under the same conditions (idempotence). See [Rules for Determinations](#).

Write the new booking IDs into a new internal table typed as table for update.

6. Update the booking entity with the entries of the internal update table.

i Expand the following code sample to view the source code of Determination `setBookingNumber`.

≡ Sample Code

```
METHOD setBookingNumber.
  DATA:
    max_bookingid   TYPE /dmo/booking_id,
    bookings_update TYPE TABLE FOR UPDATE /DMO/R_Travel_D\\Booking,
    booking         TYPE STRUCTURE FOR READ RESULT /DMO/R_Booking_D.

  "Read all travels for the requested bookings
  " If multiple bookings of the same travel are requested, the travel is returned only once.
  READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
    ENTITY Booking BY \_Travel
      FIELDS ( TravelUUID )
      WITH CORRESPONDING #( keys )
    RESULT DATA(travels).

  " Read all bookings for all affected travels
  READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
    ENTITY Travel BY \_Booking
      FIELDS ( BookingID )
      WITH CORRESPONDING #( travels )
    LINK DATA(booking_links)
    RESULT DATA(bookings).

  " Process all affected travels.
  LOOP AT travels INTO DATA(travel).

    " find max used bookingID in all bookings of this travel
    max_bookingid = '0000'.
    LOOP AT booking_links INTO DATA(booking_link) USING KEY id WHERE source-%tky = travel-%tky
      " Short dump occurs if link table does not match read table, which must never happen
      booking = bookings[ KEY id %tky = booking_link-target-%tky ].
      IF booking-BookingID > max_bookingid.
        max_bookingid = booking-BookingID.
      ENDIF.
    ENLOOP.
```

```

"Provide a booking ID for all bookings of this travel that have none.
LOOP AT booking_links INTO booking_link USING KEY id WHERE source-%tky = travel-%tky.
" Short dump occurs if link table does not match read table, which must never happen
booking = bookings[ KEY id %tky = booking_link-target-%tky ].
IF booking-BookingID IS INITIAL.
    max_bookingid += 1.
    APPEND VALUE #( %tky          = booking-%tky
                    BookingID = max_bookingid
                    ) TO bookings_update.
ENDIF.
ENDLOOP.
ENDLOOP.

" Provide a booking ID for all bookings that have none.
MODIFY ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
ENTITY booking
    UPDATE FIELDS ( BookingID )
    WITH bookings_update.

ENDMETHOD.

```

Determination setBookingDate

Define the determination in the behavior definition and implement it in the behavior implementation class for [Booking](#).

Definition

Define a determination on save with trigger operation `create`. Once the booking date is set, when a booking instance is saved to the database, the date should not be changed afterward. Therefore the field `BookingDate` is read only for the external consumer.

Sample Code

```

define behavior for /DMO/R_Booking_D alias Booking
...
{ ...
field ( readonly ) TravelUUID, BookingID, BookingDate;
determination setBookingDate on save { create; }
... }

```

Via a quick fix, you can generate the method declaration in the behavior pool directly from the behavior definition editor.

Implementation

Set the system date for the booking date. The following steps guide you through the implementation.

1. Read all the booking instances with the imported keys into an internal table and delete the instances that already have a booking date (`BookingDate`). Continue to work only with instances that do not have a booking date yet. If the determination is executed several times, the booking date must not be calculated again. This is particularly important in order to stick to the rule of idempotence for determinations.

→ Remember

The determination result must not change if the determination is executed several times under the same conditions (idempotence). See [Rules for Determinations](#).

2. Assign the system date `sy-datum` to `BookingDate` for every incoming booking instance .
3. Update the field `BookingDate` of all involved instances with a modify request.

i Expand the following code sample to view the source code of Determination **setBookingDate**.

≡ Sample Code

```
METHOD setBookingDate.

  READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
    ENTITY Booking
      FIELDS ( BookingDate )
      WITH CORRESPONDING #( keys )
      RESULT DATA(bookings).

  DELETE bookings WHERE BookingDate IS NOT INITIAL.
  CHECK bookings IS NOT INITIAL.

  LOOP AT bookings ASSIGNING FIELD-SYMBOL(<booking>).
    <booking>-BookingDate = cl_abap_context_info=>get_system_date( ).
  ENDLLOOP.

  MODIFY ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
    ENTITY Booking
      UPDATE FIELDS ( BookingDate )
      WITH CORRESPONDING #( bookings ).

ENDMETHOD.
```

Determination **calculateTotalPrice**

Define the determination in the behavior definition and implement it in the behavior implementation class for **Booking**.

i Note

The actual calculation of the total price is done by the action **reCalcPrice**. The determination just executes the action. See [Action **reCalcTotalPrice**](#).

Definition

Define a determination on modify with operation trigger create and field triggers **FlightPrice** and **CurrencyCode**.

≡ Sample Code

```
define behavior for /DMO/R_Booking_D alias Booking
...
{ ...
determination calculateTotalPrice on modify { create; field FlightPrice, CurrencyCode; }

... }
```

Via a quick fix, you can generate the method declaration in the behavior pool directly from the behavior definition editor.

Implementation

Execute the action **reCalcTotalPrice** when the trigger fields are changed. The following steps guide you through the implementation.

1. To get the **TravelUUID**, read the travel instance by a read by association into an internal table. The key of the root instance is needed for the action execution.

2. Execute a modify request for action execution for all affected travel instances.

i Note

The response parameters `failed` and `reported` are not retrieved in this EML call, since the action `reCalcTotalPrice` does not return these parameters. It is assumed that the action cannot fail as it is only called internally and processed by the managed BO provider.

The response parameter `result` is not retrieved as the action doesn't define this parameter.

i Expand the following code sample to view the source code of Determination `calculateTotalPrice`.

≡, Sample Code

```
METHOD calculateTotalPrice.

" Read all parent UUIDs
READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
  ENTITY Booking BY \_Travel
  FIELDS ( TravelUUID )
  WITH CORRESPONDING #( keys )
  RESULT DATA(travels).

" Trigger Re-Calculation on Root Node
MODIFY ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
  ENTITY Travel
  EXECUTE reCalcTotalPrice
  FROM CORRESPONDING #( travels ).

ENDMETHOD.
```

Determining Values for the Booking Supplement Entity

For the managed scenario with draft, define and implement determinations on the booking supplement entity for implicit value calculation.

Determination `setBookSupplNumber`

Define the determination in the behavior definition and implement it in the behavior implementation class for [BookingSupplement](#).

Definition

Define a determination on save with trigger operation `create`. Since the booking supplement ID must not be changed externally, define the field `BookingSupplementID` as `read only`.

≡, Sample Code

```
define behavior for /DMO/R_BookingSupplement_D alias BookingSupplement
...
{ ...
  field ( readonly ) TravelUUID, BookingSupplementID;
  determination setBookSupplNumber on save { create; }
... }
```

Via a quick fix, you can generate the method declaration in the behavior pool directly from the behavior definition editor.

Implementation

The implementation for determining new booking supplement numbers is the same as the calculation for the booking. For a detailed step-by-step description, see [Determination setBookingNumber](#).

i Expand the following code sample to view the source code of **Determination setBookSupplNumber**.

Sample Code

```
METHOD setBookSupplNumber.
  DATA:
    max_bookingsupplementid TYPE /dmo/booking_supplement_id,
    bookingsupplements_update TYPE TABLE FOR UPDATE /DMO/R_Travel_D\\BookingSupplement,
    bookingsupplement TYPE STRUCTURE FOR READ RESULT /DMO/R_BookingSupplement_D.

  "Read all bookings for the requested booking supplements
  " If multiple booking supplements of the same booking are requested, the booking is returned
  READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
    ENTITY BookingSupplement BY \_Booking
    FIELDS ( BookingUUID )
    WITH CORRESPONDING #( keys )
    RESULT DATA(bookings).

  " Read all booking supplements for the affected bookings
  READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
    ENTITY Booking BY \_BookingSupplement
    FIELDS ( BookingSupplementID )
    WITH CORRESPONDING #( bookings )
    LINK DATA(bookingsupplement_links)
    RESULT DATA(bookingsupplements).

  " Process all affected bookings.
  LOOP AT bookings INTO DATA(booking).

    " find max used booking supplement ID in all booking supplements of this booking
    max_bookingsupplementid = '00'.
    LOOP AT bookingsupplement_links INTO DATA(bookingsupplement_link) USING KEY id WHERE source
      bookingsupplement = bookingsupplements[ KEY id %tky = bookingsupplement_link-target-%tky
      IF bookingsupplement-BookingSupplementID > max_bookingsupplementid.
        max_bookingsupplementid = bookingsupplement-BookingSupplementID.
      ENDIF.
    ENDLOOP.

    "Provide a booking supplement ID for all booking supplements of this booking that have non
    LOOP AT bookingsupplement_links INTO bookingsupplement_link USING KEY id WHERE source-%tky
      bookingsupplement = bookingsupplements[ KEY id %tky = bookingsupplement_link-target-%tky
      IF bookingsupplement-BookingSupplementID IS INITIAL.
        max_bookingsupplementid += 1.
        APPEND VALUE #( %tky = bookingsupplement-%tky
          bookingsupplementid = max_bookingsupplementid
        ) TO bookingsupplements_update.
      ENDIF.
    ENDLOOP.
  ENDLOOP.

  " Provide a booking supplement ID for all booking supplements that have none.
  MODIFY ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
    ENTITY BookingSupplement
    UPDATE FIELDS ( BookingSupplementID ) WITH bookingsupplements_update.

ENDMETHOD.
```

Determination calculateTotalPrice

Define the determination in the behavior definition and implement it in the behavior implementation class for **BookingSupplement**.

i Note

The actual calculation of the total price is done by the action `reCalcPrice`. The determination just executes the action. See [Action `reCalcTotalPrice`](#).

Definition

Define a determination on modify with operation trigger `create` and field triggers `BookSupplPrice` and `CurrencyCode`.

Sample Code

```
define behavior for /DMO/R_BookingSupplement_D alias BookingSupplement
...
{ ...
  determination calculateTotalPrice on modify { create; field BookSupplPrice, CurrencyCode; }
... }
```

Via a quick fix, you can generate the method declaration in the behavior pool directly from the behavior definition editor.

Implementation

Execute the action `reCalcTotalPrice` when the trigger fields are changed. The steps are exactly the same as in [Determination `calculateTotalPrice`](#).

i *Expand the following code sample to view the source code of Determination `calculateTotalPrice`.*

Sample Code

```
METHOD calculateTotalPrice.
  " Read all parent UUIDs
  READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
    ENTITY BookingSupplement BY \_Travel
      FIELDS ( TravelUUID )
      WITH CORRESPONDING #( keys )
    RESULT DATA(travels).

  " Trigger Re-Calculation on Root Node
  MODIFY ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
    ENTITY Travel
      EXECUTE reCalcTotalPrice
      FROM CORRESPONDING #( travels ).

ENDMETHOD.
```

Developing Feature Control

For the managed scenario with draft, enable and disable functionality with feature control.

Context and Procedure

Feature control is used to define the availability of behavior components, like fields, actions, and operations. You can define if these components are always available for the client or only under certain circumstances (dynamic feature control). For more detailed information, see [Feature Control](#).

For this demo scenario, define and implement field, operation and action control.

- Define **field control**.

- **field (readonly)**

Fields that are filled by the RAP managed runtime framework (internally) are set to read only as the consumer is not allowed to change these. Marking these fields as read only in the behavior definition makes also has an effect on the UI. The end user cannot provide values for these fields. In the travel scenario, the fields that are set to read only are the administrative fields and fields that are filled internally, for example by determinations. These fields are:

- On the travel entity:

`TravelID, OverallStatus, TotalPrice, LocalCreatedAt, LocalCreatedBy, LocalLastChangedAt, LocalLastChangedBy, LastChangedAt`

- On the booking entity:

`TravelUUID, BookingID, BookingDate, LocalLastChangedAt`

- On the booking supplement entity:

`TravelUUID, BookingUUID, BookingSupplementID, LocalLastChangedAt`

The key fields whose values are set by the RAP managed runtime framework via managed numbering must also be set to read only. They also receive the attribute numbering: `managed`. These fields are:

- On the travel entity: `TravelUUID`

- On the booking entity: `BookingUUID`

- On the booking supplement entity: `BookSupplUUID`

- **field (mandatory)**

Fields whose values are checked by validations are set to mandatory. These fields are:

- On the travel entity:

`CustomerID, AgencyID, BeginDate, EndDate`

- On the booking entity:

`CustomerID, AirlineID, ConnectionID, FlightDate`

- On the booking supplement entity:

`SupplementID`

- **field (feature: instance)**

Fields whose logic depends on other circumstances are controlled by dynamic feature control. In this scenario, the `BookingFee` (on the travel entity) is set to read only if the instance's status is **Accepted (A)**. The feature control condition must be implemented in the behavior class.

- Define **operation control**.

- **create (features : instance)**

Use dynamic operation control for the associations that are enabled for create. In this demo scenario, you can only create new booking instance for a travel if the overall status is not rejected ('X'). The feature control condition must be implemented in the behavior class.

- On the travel entity:

```
association _Booking { create (features : instance); }
```

- Define **action control**.

- action (features: instance)

Dynamic operation control is used for actions that are enabled depending on other states of the instance. These actions are

- On the travel entity:

The action `acceptTravel` can only be executed, if the overall status is not already set to **Accepted (A)**:
 action (features: instance) acceptTravel result [1] \$self; ,

The action `rejectTravel` can only be executed, if the overall status is not already set **Rejected (X)**:
 action (features: instance) rejectTravel result [1] \$self; on the travel entity.

The action `deductDiscount` can only be executed, if the overall status is not already set **Accepted (A)**:
 action (features: instance) deductDiscount result [1] \$self; on the travel entity.

For a detailed description on how to implement the described behavior for the relevant entity, check the implementation steps in the following topics.

[Adding Feature Control for the Travel Entity](#)

For the managed scenario with draft, enable and disable functionality with feature control on the travel entity.

[Adding Feature Control for the Booking Entity](#)

For the managed scenario with draft, enable and disable functionality with feature control on the booking entity.

[Adding Feature Control for the Booking Supplement Entity](#)

For the managed scenario with draft, enable and disable functionality with feature control on the booking supplement entity.

Adding Feature Control for the Travel Entity

For the managed scenario with draft, enable and disable functionality with feature control on the travel entity.

Define the static and dynamic feature control in the behavior definition and implement dynamic feature control in the behavior implementation class for **Travel**.

Definition

Define field, action, and operation control. For dynamic feature control, add (feature: instance) to the feature control definition.

Sample Code

```
define behavior for /DMO/R_Travel_D alias Travel
...
{ ...
  association _Booking { create ( features : instance ); }
```

```

field ( numbering : managed, readonly ) TravelUUID;
field ( readonly ) TravelID, OverallStatus, TotalPrice, LocalCreatedAt, LocalCreatedBy, LocalL
field ( mandatory ) CustomerID, AgencyID, BeginDate, EndDate, CurrencyCode;
field ( features : instance ) BookingFee;

action ( features : instance ) acceptTravel result [1] $self;
action ( features : instance ) rejectTravel result [1] $self;
action ( features : instance ) deductDiscount parameter /DMO/A_TRAVEL_DISCOUNT result [1] $se

... }

```

Via a quick fix, you can generate the method declaration for the feature control implementation in the behavior pool directly from the behavior definition editor.

Dynamic Feature Control Implementation for the Travel Entity

Dynamic feature control must be implemented in the behavior implementation in the method `FOR FEATURES`. You can use the quick fix on one of the dynamic feature control features in the behavior definition for the method declaration. The following steps guide you through the implementation.

1. Read all the instances with the imported keys into an internal table `travels`. The fields that are relevant to decide whether features are available must be included in the read result. For the travel entity, only the field `OverallStatus` is relevant.
2. Fill the result table of the method with the respective feature. Use the transaction key `%tky` to identify the travel instance. The importing parameter `keys` contains the `%field`, `%action`, `%assoc` component to determine what kind of feature is controlled. Use a condition statement to define the circumstance and the outcome of each dynamically controlled feature. You can work with the constants, defined in the private section of the behavior class, to refer to the values of the overall status.

i Expand the following code sample to view the source code of `GET_INSTANCE_FEATURES`.

Sample Code

```

METHOD get_instance_features.

  READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
    ENTITY Travel
      FIELDS ( OverallStatus )
      WITH CORRESPONDING #( keys )
    RESULT DATA(travels)
    FAILED failed.

  result = VALUE #( FOR ls_travel IN travels
                    ( %tky                                = ls_travel-%tky
                      %field-BookingFee                    = COND #( WHEN ls_travel-OverallStatus = trav
                                                                    THEN if_abap_behv=>fc-f-read_only
                                                                    ELSE if_abap_behv=>fc-f-unrestricte
                      %action-acceptTravel                  = COND #( WHEN ls_travel-OverallStatus = trav
                                                                    THEN if_abap_behv=>fc-o-disabled
                                                                    ELSE if_abap_behv=>fc-o-enabled )
                      %action-rejectTravel                  = COND #( WHEN ls_travel-OverallStatus = trav
                                                                    THEN if_abap_behv=>fc-o-disabled
                                                                    ELSE if_abap_behv=>fc-o-enabled )
                      %action-deductDiscount                = COND #( WHEN ls_travel-OverallStatus = trav
                                                                    THEN if_abap_behv=>fc-o-disabled

```

```

                                ELSE if_abap_behv=>fc-o-enabled )
                                = COND #( WHEN ls_travel-OverallStatus = trav
                                THEN if_abap_behv=>fc-o-disabled
                                ELSE if_abap_behv=>fc-o-enabled )
                                ) ).

ENDMETHOD..

```

Adding Feature Control for the Booking Entity

For the managed scenario with draft, enable and disable functionality with feature control on the booking entity.

Define static feature control in the behavior definition. For the **Booking** entity, there is no dynamic feature control.

Definition

Define field control.

Sample Code

```

define behavior for /DMO/R_Booking_D alias Booking
...
{...

    field ( numbering : managed, readonly ) BookingUUID;
    field ( readonly ) TravelUUID, BookingID, BookingDate, LocalLastChangedAt;
    field ( mandatory ) CustomerID, AirlineID, ConnectionID, FlightDate, CurrencyCode;

... }

```

Adding Feature Control for the Booking Supplement Entity

For the managed scenario with draft, enable and disable functionality with feature control on the booking supplement entity.

Define the static feature control in the behavior definition. For the **BookingSupplement**, there is no dynamic feature control.

Definition

Define field control.

Sample Code

```

define behavior for /DMO/R_BookingSupplement_D alias BookingSupplement
...
{ ...
    field ( numbering : managed, readonly ) BookSupplUUID;
    field ( readonly ) TravelUUID, BookingUUID, BookingSupplementID, LocalLastChangedAt;
    field ( mandatory ) SupplementID, CurrencyCode;

... }

```


Developing Authorization

For the managed scenario with draft, add authorization control to prevent unauthorized modifying access to the business object.

Context and Procedure

With authorization control, you define which users are allowed to make changes on BO instances. The RAP model differentiates between general authorization to use operations (global authorization), and instance-specific authorization, which assigns authorization to use operations depending on values of an instance in question (instance authorization). For more detailed information, see [Authorization Control](#).

In this reference scenario, authorization for each standard operation is considered separately. That means, one user group might be able to create new instances, but is not allowed to update this instance or vice versa. Actions, however, use the same authorization as the operation update. The authorization control can be delegated for these operations.

i Note

In the ABAP Flight Reference Scenario, the authorization objects aren't configured in the back end and thus would prevent any operation for any user. Hence, the code example provides a simulated authorization check without using authorization objects in addition to the check via the authorization object. Don't use the simulation code in productive applications.

For this scenario, define and implement the following authorization control:

- Define and implement **global authorization**.

Global authorization is checked for all standard operations and actions. The authorization for the operating user is checked with authorization objects for each modifying operation separately, which means there can be a different authorization for the create, update and delete operation. All actions are treated like the update operation. In this example implementation, a simulation is provided to allow all users to execute all operations.

For a detailed description on how to implement global authorization control, see [Implementing Global Authorization](#).

- Define and implement **instance authorization**.

Instance authorization is checked for all instance-related operations. Instance authorization is checked with an authorization object against an authorization field. In this example scenario, users are only authorized to modify travel instances that have an agency with one specific country code. A simulation implementation is provided to allow users to modify instances only, if the country code of the related agency is US. However, you can use any other country code as well.

i Note

In the dedicated authorization methods, it's only possible to check instance authorization against the state of the instance before the current LUW. In other words, authorization is checked against the persistent state of the instance, if there's one. For new instances, the same authorization as for global authorization is applied. To check authorization against incoming values, you need to implement a precheck.

For a detailed description on how to implement instance authorization control, see [Implementing Instance Authorization](#).

- Define and implement **prechecks**.

Prechecks check against incoming values. In this example, the precheck implementation checks that only BOs with AgencyIDs with the `country_code` US can be created or updated.

For a detailed description on how to implement prechecks, see [Implementing Prechecks](#).

i Note

Authorization for authorization-dependent entities is delegated to the authorization master entity. That means, modify requests on authorization-dependent entities are treated like the update operation on the authorization master entity.

For a detailed description on how to implement the described behavior for the relevant entity, check the implementation steps in the following topics.

[Implementing Global Authorization](#)

For the managed scenario with draft, authorize users to generally modify travel instances.

[Implementing Instance Authorization](#)

For the managed scenario with draft, authorize users to modify travel instances based on an authorization field.

[Implementing Prechecks](#)

For the managed scenario with draft, add prechecks to check authorization against incoming values to prevent values from reaching the transactional buffer.

Implementing Global Authorization

For the managed scenario with draft, authorize users to generally modify travel instances.

Define global authorization in the behavior definition and implement it in the behavior implementation class for **Travel**. For conceptual information, see [Authorization Control](#).

i Note

In the ABAP Flight Reference Scenario, the authorization objects aren't configured in the back end and thus would prevent any operation for any user. Hence, the code example provides a simulated authorization check without using authorization objects in addition to the check via the authorization object. Don't use the simulation code in productive applications.

Definition

If not already done so, define `authorization master (global)` for the travel entity and `authorization dependent by _Travel` for the booking and the booking supplement entity in the behavior definition.

In addition, delegate the authorization control for all actions, except for the internal and draft actions, and the create-by-association to the update operation. The syntax `authorization: update` is used.

≡ Sample Code

```
...
define behavior for /DMO/R_Travel_D alias Travel
implementation in class /DMO/BP_TRAVEL_D unique
persistent table /DMO/A_TRAVEL_D
...
authorization master ( global )
{...
    association _Booking { create ( features : instance, authorization: update ) ; }
...

    action ( features : instance, authorization: update ) acceptTravel result [1] $self;
    action ( features : instance, authorization: update ) rejectTravel result [1] $self;
    action ( features : instance, authorization: update ) deductDiscount parameter /DMO/A_TRAVEL_
```

```

    internal action reCalcTotalPrice;

...}

define behavior for /DMO/R_Booking_D alias Booking
implementation in class /DMO/BP_BOOKING_D unique
....
authorization dependent by _Travel
{...
    association _BookingSupplement { create ( authorization: update ); }
...}

define behavior for /DMO/R_BookingSupplement_D alias BookingSupplement
implementation in class /DMO/BP_BOOKINGSUPPLEMENT_D unique
...
authorization dependent by _Travel
{
...
}

```

Via a quick fix, you can generate the method declaration for the authorization control implementation in the behavior pool directly from the behavior definition editor.

Implementation

Global authorization control must be implemented in the behavior implementation in the method `FOR_GLOBAL_AUTHORIZATION`. You can use the quick fix in the behavior definition for the method declaration. The method imports the parameter `requested_authorizations`, which indicates the operations for which authorization is requested. In the implementation, you have to fill the parameter `result`, which must be filled with the authorization result for each requested operation. The method also has the changing parameter `reported`, which is to be filled if you want to return messages for unauthorized cases. For more information, about implementation details, see [Authorization Implementation](#).

The authorization concept in this guide assumes that there can be different authorizations for creating, updating, and deleting instances. All actions of the travel BO are considered with the same authorization as the update operation. With this approach, it is possible that one user is allowed to create a new travel instance, but is not allowed to use actions. Actions and create-by-association are defined with the addition `authorization: update` and thus must not be treated separately in the authorization implementation.

As the authorization for create, update, and delete is the same for global and instance authorization (except for the relevant authorization field for instance authorization), it is best practice to outsource the actual authorization check in separate methods that can be called by all authorization checks that are needed in the BO implementation.

The following steps guide you through the implementation.

Methods for Authorization Checks

1. For each operation, create a method to check the authorization for it.
2. We use the authorization object `/DMO/TRVL` to check the authorization for each operation.

For more information, about using and configuring authorization objects, see [Authorization Basics](#).

🏠 For more information, about using and configuring authorization objects, see [User and Role Administration of Application Server ABAP](#)

In each method, call the authorization object and check for the relevant activity.

Authorization Fields in Authorization Object

Operation	Activity Field
create	Field 01 (Add or Create)
update + all actions	Field 02 (Change)
delete	Field 06 (Delete)

The authorization object checks against the authorization field `/DMO/CNTRY`. For global authorization, this is not relevant as the authorization for executing the operations is independent of the state of the respective instance, and thus independent of the value of the authorization field. For that reason, you can use `'DUMMY'` for the authorization field.

The implementation considers the authorization for each modify operation separately. There can be a different authorization for creating, updating, and deleting instances on the travel BO.

3. Evaluate the authorization for the respective operation based on `sy-subrc` and fill the response parameter of the method.
4. As the authorization which is checked by the authorization object is not configured in the backend, we provide a simulation implementation to allow all users for all operations. Do not use this piece of code in productive applications.

i Expand the following code sample to view the source code of the methods `is_create_granted`, `is_update_granted`, and `is_delete_granted`.

Sample Code

```

CLASS lhc_travel DEFINITION INHERITING FROM cl_abap_behavior_handler.

    PRIVATE SECTION.

        ...

        METHODS is_create_granted
            RETURNING VALUE(create_granted) TYPE abap_bool.
        METHODS is_update_granted
            RETURNING VALUE(update_granted) TYPE abap_bool.
        METHODS is_delete_granted
            RETURNING VALUE(delete_granted) TYPE abap_bool.

    ENDCLASS.

CLASS lhc_travel IMPLEMENTATION.
    METHOD is_create_granted.
        AUTHORITY-CHECK OBJECT '/DMO/TRVL'
            ID '/DMO/CNTRY' DUMMY
            ID 'ACTVT'      FIELD '01'.
        create_granted = COND #( WHEN sy-subrc = 0 THEN abap_true ELSE abap_false ).

        "Simulation for full authorization
        "(not to be used in productive code)
        create_granted = abap_true.
    ENDIF.

    ENDMETHOD.

```

```

METHOD is_update_granted.
  AUTHORITY-CHECK OBJECT '/DMO/TRVL'
    ID '/DMO/CNTRY' DUMMY
    ID 'ACTVT'      FIELD '02'.
  update_granted = COND #( WHEN sy-subrc = 0 THEN abap_true ELSE abap_false ).

  "Simulation for full authorization
  "(not to be used in productive code)
  update_granted = abap_true.
ENDIF.

ENDMETHOD.

METHOD is_delete_granted.
  AUTHORITY-CHECK OBJECT '/DMO/TRVL'
    ID '/DMO/CNTRY' DUMMY
    ID 'ACTVT'      FIELD '06'.
  delete_granted = COND #( WHEN sy-subrc = 0 THEN abap_true ELSE abap_false ).

  "Simulation for full authorization
  "(not to be used in productive code)
  delete_granted = abap_true.
ENDIF.

ENDMETHOD.

ENDCLASS.

```

Global Authorization Implementation

1. For each operation, check if the authorization is requested. In this example, all actions are treated like the update operation. The draft action Edit must be delegated manually in the implementation. Authorizations for all other actions are delegated in the behavior definition.
2. Fill the reported parameter with an adequate message, if the user is unauthorized for the respective operation.

i Expand the following code sample to view the source code of the method `get_global_authorizations`.

Sample Code

```

CLASS lhc_travel DEFINITION INHERITING FROM cl_abap_behavior_handler.

  PRIVATE SECTION.

    ...

    METHODS get_global_authorizations FOR GLOBAL AUTHORIZATION
      IMPORTING REQUEST requested_authorizations FOR travel RESULT result.

ENDCLASS.

CLASS lhc_travel IMPLEMENTATION.
  METHOD get_global_authorizations.

```

```

IF requested_authorizations-%create EQ if_abap_behv=>mk-on.
  IF is_create_granted( ) = abap_true.
    result-%create = if_abap_behv=>auth-allowed.
  ELSE.
    result-%create = if_abap_behv=>auth-unauthorized.
    APPEND VALUE #( %msg      = NEW /dmo/cm_flight_messages(
                      textid    = /dmo/cm_flight_messages=>not_authorized
                      severity  = if_abap_behv_message=>severity-error )
                  %global = if_abap_behv=>mk-on
                  ) TO reported-travel.

  ENDIF.
ENDIF.

"Edit is treated like update
IF requested_authorizations-%update                        = if_abap_behv=>mk-on OR
   requested_authorizations-%action-Edit                  = if_abap_behv=>mk-on.

  IF is_update_granted( ) = abap_true.
    result-%update                        = if_abap_behv=>auth-allowed.
    result-%action-Edit                  = if_abap_behv=>auth-allowed.
  ELSE.
    result-%update                        = if_abap_behv=>auth-unauthorized.
    result-%action-Edit                  = if_abap_behv=>auth-unauthorized.

    APPEND VALUE #( %msg      = NEW /dmo/cm_flight_messages(
                      textid    = /dmo/cm_flight_messages=>not_authorized
                      severity  = if_abap_behv_message=>severity-error )
                  %global = if_abap_behv=>mk-on
                  ) TO reported-travel.

  ENDIF.
ENDIF.

IF requested_authorizations-%delete = if_abap_behv=>mk-on.
  IF is_delete_granted( ) = abap_true.
    result-%delete = if_abap_behv=>auth-allowed.
  ELSE.
    result-%delete = if_abap_behv=>auth-unauthorized.
    APPEND VALUE #( %msg      = NEW /dmo/cm_flight_messages(
                      textid    = /dmo/cm_flight_messages=>not_authorized
                      severity  = if_abap_behv_message=>severity-error )
                  %global = if_abap_behv=>mk-on
                  ) TO reported-travel.

  ENDIF.
ENDIF.

ENDMETHOD.

ENDCLASS.

```

Implementing Instance Authorization

For the managed scenario with draft, authorize users to modify travel instances based on an authorization field.

Define instance authorization in the behavior definition and implement it in the behavior implementation class for **Travel**. For conceptual information, see [Authorization Control](#).

i Note

In the ABAP Flight Reference Scenario, the authorization objects aren't configured in the back end and thus would prevent any operation for any user. Hence, the code example provides a simulated authorization check without using authorization objects in addition to the check via the authorization object. Don't use the simulation code in productive applications.

Definition

If not already done so, define `authorization master (instance)` for the travel entity and `authorization dependent by _Travel` for the booking and the booking supplement entity in the behavior definition.

If not already done, delegate the authorization control for all actions, except for the internal and draft actions, and the create-by-association to the update operation. The syntax `authorization: update` is used.

≡ Sample Code

```
...
define behavior for /DMO/R_Travel_D alias Travel
implementation in class /DMO/BP_TRAVEL_D unique
persistent table /DMO/A_TRAVEL_D
...
authorization master ( global, instance )
{...
  association _Booking { create ( features : instance, authorization: update ) ; }
...

  action ( features : instance, authorization: update ) acceptTravel result [1] $self;
  action ( features : instance, authorization: update ) rejectTravel result [1] $self;
  action ( features : instance, authorization: update ) deductDiscount parameter /DMO/A_TRAVEL_
    internal action reCalcTotalPrice;

...}

define behavior for /DMO/R_Booking_D alias Booking
implementation in class /DMO/BP_BOOKING_D unique
....
authorization dependent by _Travel
{...
  association _BookingSupplement { create ( authorization: update ); }
...}

define behavior for /DMO/R_BookingSupplement_D alias BookingSupplement
implementation in class /DMO/BP_BOOKINGSUPPLEMENT_D unique
...
authorization dependent by _Travel
```

```
{
...
}
```

Via a quick fix, you can generate the method declaration for the authorization control implementation in the behavior pool directly from the behavior definition editor.

Implementation

Instance authorization control must be implemented in the behavior implementation in the method `FOR_INSTANCE_AUTHORIZATION`. You can use the quick fix in the behavior definition for the method declaration. The method imports the parameter `requested_authorizations`, which indicates the operations for which authorization is requested. Like other instance-specific implementations, the method imports the key of the instances, for which authorization is requested. In the implementation, you have to fill the parameter `result`, which must be filled with the authorization result for each requested operation. The method also has the changing parameter `reported`, which is to be filled if you want to return messages for unauthorized cases. For more information, about implementation details, see [Authorization Implementation](#).

The authorization concept in this guide assumes that there can be different authorizations for creating, updating, and deleting instances. All actions of the travel BO are considered with the same authorization as the update operation. With this approach, it is possible that one user is allowed to create a new travel instance, but is not allowed to use actions.

As the authorization for create, update, and delete is the same for global and instance authorization (except for the relevant authorization field for instance authorization), it is best practice to outsource the actual authorization check in separate methods that can be called for all authorization checks that are needed in the BO implementation. The implementation for these methods is explained in detail in [Methods for Authorization Checks](#). Since the instance authorization check is based on a field value, the methods must be adapted to also import a parameter.

The following steps guide you through the implementation.

Methods for Authorization Checks

1. In the methods `is_update_granted` and `is_delete_granted`, add an optional importing parameter to pass the country code of the used agency ID. (The method `is_create_granted` is not used for instance authorization checks as the create operation is not instance specific.)
2. In the method implementation for the corresponding method, differentiate between calls from global and instance authorization by adding an if-block, in which the authorization object is called when the authorization field `country_code` is supplied.
3. Evaluate the authorization for the respective operation based on `sy-subrc` and fill the response parameter of the method.
4. As the authorization which is checked by the authorization object is not configured in the backend, we provide a simulation implementation to allow all users for all operations. Do not use this piece of code in productive applications.
5. To simulate the authorization check against the agency's country code, we provide a simulation implementation to only allow modification on instances that have an agency with country code `US`. You can change this country code by any other.

i Expand the following code sample to view the source code of the methods `is_update_granted`, and `is_delete_granted`.

Sample Code

```
CLASS lhc_travel DEFINITION INHERITING FROM cl_abap_behavior_handler.

PRIVATE SECTION.
```



```

...
METHODS is_update_granted
  IMPORTING country_code          TYPE land1 OPTIONAL
  RETURNING VALUE(update_granted) TYPE abap_bool.
METHODS is_delete_granted
  IMPORTING country_code          TYPE land1 OPTIONAL
  RETURNING VALUE(delete_granted) TYPE abap_bool.

ENDCLASS.

METHOD is_update_granted.
  "For instance auth
  IF country_code IS SUPPLIED.
    AUTHORITY-CHECK OBJECT '/DMO/TRVL'
      ID '/DMO/CNTRY' FIELD country_code
      ID 'ACTVT'      FIELD '02'.
    update_granted = COND #( WHEN sy-subrc = 0 THEN abap_true ELSE abap_false ).

    "Simulation for full authorization
    "(not to be used in productive code)
    update_granted = abap_true.

    " simulation of auth check for demo,
    " auth granted for country_code US, else not
    * CASE country_code.
    *   WHEN 'US'.
    *     create_granted = abap_true.
    *   WHEN OTHERS.
    *     create_granted = abap_false.
    *   ENDCASE.

    "For global auth
  ELSE.
    AUTHORITY-CHECK OBJECT '/DMO/TRVL'
      ID '/DMO/CNTRY' DUMMY
      ID 'ACTVT'      FIELD '02'.
    update_granted = COND #( WHEN sy-subrc = 0 THEN abap_true ELSE abap_false ).

    "Simulation for full authorization
    "(not to be used in productive code)
    update_granted = abap_true.
  ENDIF.

ENDMETHOD.

METHOD is_delete_granted.
  "For instance auth
  IF country_code IS SUPPLIED.
    AUTHORITY-CHECK OBJECT '/DMO/TRVL'
      ID '/DMO/CNTRY' FIELD country_code
      ID 'ACTVT'      FIELD '06'.
    delete_granted = COND #( WHEN sy-subrc = 0 THEN abap_true ELSE abap_false ).

```

```

"Simulation for full authorization
"(not to be used in productive code)
delete_granted = abap_true.

" simulation of auth check for demo,
" auth granted for country_code US, else not
* CASE country_code.
*   WHEN 'US'.
*     create_granted = abap_true.
*   WHEN OTHERS.
*     create_granted = abap_false.
*   ENDCASE.

"For global auth
ELSE.
  AUTHORITY-CHECK OBJECT '/DMO/TRVL '
    ID '/DMO/CNTRY' DUMMY
    ID 'ACTVT'      FIELD '06'.
  delete_granted = COND #( WHEN sy-subrc = 0 THEN abap_true ELSE abap_false ).

"Simulation for full authorization
"(not to be used in productive code)
delete_granted = abap_true.
ENDIF.

ENDMETHOD.

...

ENDCLASS.

```

Instance Authorization Implementation

In this scenario, the instance authorization is based on the value of the agency's country code that is used in the travel instance. With instance authorization you can only check authorization against the persistent state of the instance. That means, you have to provide the value of the country code as it is on the persistent table (and not the value, which is supplied by the request). In other words, if an update request changes the value of the agency to a value that is not in the authorization scope for this user, the request is not denied during the transaction phase (as the value on the persistent table is one, for which the user is authorized). The request is only denied during the save sequence if a precheck checks the authorization of the incoming value. See [Implementing Prechecks](#).

For instances that don't exist on the persistent database table yet (because they have just been created in the current LUW), authorization is always granted. This also becomes relevant in draft scenarios for edit-drafts.

The following steps guide you through the implementation steps for instance authorization.

1. Read all the instances with the imported keys into an internal table `travels`. As the `AgencyID` is relevant for the authorization, it must be included in the read result.
2. To get the agency's country code of the persistent version of the imported instance keys, select from the persistent table for travels `/DMO/A_TRAVEL_D` with an inner join on the agency table `/DMO/AGENCY`.
3. Loop at the `travels` table to assign the authorization for each instance separately. There are two cases to consider:

- For instances that have a persistent state on the database table /DMO/A_TRAVEL_D, delegate the authorization check to the relevant method for update or delete and pass the retrieved value for the corresponding country code of the agency. To check if a persistent instance exists, check if an instance with the current travel UUID exists on the select result. If sy - subrc returns 0, the read was successful.

If the corresponding authorization check methods deny authorization for the respective operation, fill the reported parameter with an adequate message.

- For instances that do not have a persistent state on the database table /DMO/A_TRAVEL_D, check the authorization for create, as update or delete operations on newly created instances are considered as part of the create operation as long as the changes are not committed.

4. Fill the result parameter result with the determined authorization for each operation.

i Expand the following code sample to view the source code of the method `get_instance_authorizations`.

Sample Code

```

CLASS lhc_travel DEFINITION INHERITING FROM cl_abap_behavior_handler.

    PRIVATE SECTION.

    ...

    METHODS get_instance_authorizations FOR INSTANCE AUTHORIZATION
        IMPORTING keys REQUEST requested_authorizations FOR travel RESULT result.

ENDCLASS.

CLASS lhc_travel IMPLEMENTATION.
    METHOD get_instance_authorizations.

        DATA: update_requested TYPE abap_bool,
              delete_requested TYPE abap_bool,
              update_granted    TYPE abap_bool,
              delete_granted    TYPE abap_bool.

        READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
            ENTITY Travel
                FIELDS ( AgencyID )
                WITH CORRESPONDING #( keys )
            RESULT DATA(travels)
            FAILED failed.

        CHECK travels IS NOT INITIAL.

        "Select country_code and agency of corresponding persistent travel instance
        "authorization only checked against instance that have active persistence
        SELECT FROM /DMO/A_TRAVEL_D AS travel INNER JOIN /DMO/AGENCY AS agency
            ON travel~agency_id = agency~agency_id
            FIELDS travel~travel_uuid , travel~agency_id, agency~country_code
            FOR ALL ENTRIES IN @travels WHERE travel_uuid EQ @travels-TravelUUID
            INTO TABLE @DATA(travel_agency_country).

        "edit is treated like update

```

```

update_requested = COND #( WHEN requested_authorizations-%update           = if_at
                             requested_authorizations-%action-Edit        = if_at
                             THEN abap_true ELSE abap_false ).

delete_requested = COND #( WHEN requested_authorizations-%delete          = if_at
                             THEN abap_true ELSE abap_false ).

LOOP AT travels INTO DATA(travel).
  "get country_code of agency in corresponding instance on persistent table
  READ TABLE travel_agency_country WITH KEY travel_uuid = travel-TravelUUID
    ASSIGNING FIELD-SYMBOL(<travel_agency_country_code>).

  "Auth check for active instances that have before image on persistent table
  IF sy-subrc = 0.

    "check auth for update
    IF update_requested = abap_true.
      update_granted = is_update_granted( <travel_agency_country_code>-country_code ).
      IF update_granted = abap_false.
        APPEND VALUE #( %tky = travel-%tky
                        %msg = NEW /dmo/cm_flight_messages(
                                textid      = /dmo/cm_flight_messages=
                                agency_id   = travel-AgencyID
                                severity    = if_abap_behv_message=>seve
                                %element-AgencyID = if_abap_behv=>mk-on
                                ) TO reported-travel.
        ENDIF.
      ENDIF.

    "check auth for delete
    IF delete_requested = abap_true.
      delete_granted = is_delete_granted( <travel_agency_country_code>-country_code ).
      IF delete_granted = abap_false.
        APPEND VALUE #( %tky = travel-%tky
                        %msg = NEW /dmo/cm_flight_messages(
                                textid      = /dmo/cm_flight_messages=>not_authorized_f
                                agency_id   = travel-AgencyID
                                severity    = if_abap_behv_message=>severity-error )
                                %element-AgencyID = if_abap_behv=>mk-on
                                ) TO reported-travel.
        ENDIF.
      ENDIF.

    " operations on draft instances and on active instances that have no persistent bel
    " create authorization is checked, for newly created instances
  ELSE.
    update_granted = delete_granted = is_create_granted( ).
    IF update_granted = abap_false.
      APPEND VALUE #( %tky = travel-%tky
                      %msg = NEW /dmo/cm_flight_messages(
                              textid      = /dmo/cm_flight_messages=>not_authorized
                              severity    = if_abap_behv_message=>severity-error )
                              %element-AgencyID = if_abap_behv=>mk-on

```

```

        ) TO reported-travel.

    ENDIF.
ENDIF.

APPEND VALUE #( LET upd_auth = COND #( WHEN update_granted = abap_true THEN if_abap_t
                                         ELSE if_abap_behv=>auth-unauthorized )
               del_auth = COND #( WHEN delete_granted = abap_true THEN if_abap_t
                                   ELSE if_abap_behv=>auth-unauthorized )

               IN
               %tky = travel-%tky
               %update           = upd_auth
               %action-Edit      = upd_auth

               %delete           = del_auth
               ) TO result.

ENDLOOP.

ENDMETHOD.

ENDCLASS.

```

Implementing Prechecks

For the managed scenario with draft, add prechecks to check authorization against incoming values to prevent values from reaching the transactional buffer.

Define prechecks in the behavior definition and implement it in the behavior implementation class for Travel. For conceptual information, see [Operation Precheck](#).

Definition

Define the `create` and `update` with the addition (`precheck`). With the addition of `precheck`, the behavior implementation for `precheck` is called, before the main operation is executed. To get an overview of the runtime behavior for `create` and `update` with `precheck` and other standard operations, see [Standard Operations](#). The `precheck` operations receive all entities to be created and returns `failed` and `reported` as changing parameters if the check against the incoming values returns a negative result. For more information about the response parameters, see [Response Parameters](#).

Additionally, add `with additional implementation` to the draft action `Resume`. This is required to trigger a check against the `AgencyID` field in case of an `edit draft`. This additional check is only required for RAP BOs with draft capabilities.

i *Expand the following code sample to view the source code of the behavior definition for `/DMO/R_Travel_D`.*

Sample Code

```

...
define behavior for /DMO/R_Travel_D alias Travel
implementation in class /dmo/bp_travel_d unique
persistent table /dmo/a_travel_d
draft table /dmo/d_travel_d
lock master

```

```

total etag LastChangedAt
authorization master ( global, instance )
etag master LocalLastChangedAt
{
  create ( precheck );
  update ( precheck );
  ...
  draft action Resume with additional implementation;
}

```

Via a quick fix, you can generate the method declaration for the prechecks in the behavior pool directly from the behavior definition editor.

Implementation

Prechecks

The authorization to create or save an updated instance with a provided agency ID is checked based on the value of the agency's country/region code. It's only possible to save newly created or updated instance with a certain value of country_code. The demo scenario uses US, which can be changed to any other country code. The precheck for the create and update operation checks whether this condition is fulfilled before a create is triggered. Since both operation prechecks check against the same incoming value, a helper method `precheck_auth` was created to streamline the implementation. Both precheck implementations delegate the actual check against the agencyID country code to the helper method. Both methods return failed and reported as changing parameters.

i Expand the following code sample to view the source code of the methods `precheck_create` **FOR CREATE** and `precheck_update` **FOR UPDATE**.

☰ Sample Code

```

METHOD precheck_create.
  precheck_auth(
    EXPORTING
      entities_create = entities
    CHANGING
      failed          = failed-travel
      reported        = reported-travel
  ).
ENDMETHOD.

METHOD precheck_update.
  precheck_auth(
    EXPORTING
      entities_update = entities
    CHANGING
      failed          = failed-travel
      reported        = reported-travel
  ).
ENDMETHOD.

```

Method `precheck_auth`

The method `precheck_auth` first differentiates whether it is called from the precheck method `FOR CREATE` or `FOR UPDATE` based on its importing parameters and the `operation` variable. An assert ensures that the method can only be called from one of the precheck implementations and not both at the same time. A `select` retrieves the corresponding `country_code` for the `AgencyID` from the database table. The implementation loops over all entities that were imported and delegates the check against the US value for the `country_code` based on the operation type: If the `operation` variable is flagged as `create`, the method `is_create_granted` is called and received the retrieved `country_code` as importing parameter. In case `operation` is flagged as `update`, the method `is_update_granted` is called and check the `country_code`. If any other value than `US` is set, the boolean value `is_modify_granted` is set to `abap_false`, meaning that the authorization is denied. In this case, the response parameters `failed` and `reported` are filled.

i Note

In the ABAP Flight Reference Scenario, the authorization objects aren't configured in the back end and thus would prevent any operation for any user. Hence, the code example provides a simulated authorization check without using authorization objects in addition to the check via the authorization object. Don't use the simulation code in productive applications.

i Expand the following code sample to view the source code of the method `precheck_auth`.

≡ Sample Code

```
METHOD precheck_auth.
  DATA:
    entities          TYPE t_entities_update,
    operation          TYPE if_abap_behv=>t_char01,
    agencies           TYPE SORTED TABLE OF /dmo/agency WITH UNIQUE KEY agency_id,
    is_modify_granted TYPE abap_bool.

  " Either entities_create or entities_update is provided. NOT both and at least one.
  ASSERT NOT ( entities_create IS INITIAL EQUIV entities_update IS INITIAL ).

  IF entities_create IS NOT INITIAL.
    entities = CORRESPONDING #( entities_create MAPPING %cid_ref = %cid ).
    operation = if_abap_behv=>op-m-create.
  ELSE.
    entities = entities_update.
    operation = if_abap_behv=>op-m-update.
  ENDIF.

  DELETE entities WHERE %control-AgencyID = if_abap_behv=>mk-off.

  agencies = CORRESPONDING #( entities DISCARDING DUPLICATES MAPPING agency_id = AgencyID EXCE
  CHECK agencies IS NOT INITIAL.
  SELECT FROM /dmo/agency FIELDS agency_id, country_code
          FOR ALL ENTRIES IN @agencies
          WHERE agency_id = @agencies-agency_id
  INTO TABLE @DATA(agency_country_codes).

  LOOP AT entities INTO DATA(entity).
    is_modify_granted = abap_false.

    READ TABLE agency_country_codes WITH KEY agency_id = entity-AgencyID
      ASSIGNING FIELD-SYMBOL(<agency_country_code>).
```

```

    "If invalid or initial AgencyID -> validateAgency
    CHECK sy-subrc = 0.
    CASE operation.
        WHEN if_abap_behv=>op-m-create. is_modify_granted = is_create_granted( <agency_country_c
        WHEN if_abap_behv=>op-m-update. is_modify_granted = is_update_granted( <agency_country_c
    ENDCASE.
    IF is_modify_granted = abap_false.
        APPEND VALUE #(
            %cid      = COND #( WHEN operation = if_abap_behv=>op-m-create THEN ent
            %tky      = entity-%tky
        ) TO failed.

        APPEND VALUE #(
            %cid      = COND #( WHEN operation = if_abap_behv=>op-m-create THEN ent
            %tky      = entity-%tky
            %msg      = NEW /dmo/cm_flight_messages(
                textid   = /dmo/cm_flight_messages=>not_author
                agency_id = entity-AgencyID
                severity  = if_abap_behv_message=>severity-erro
            %element-AgencyID = if_abap_behv=>mk-on
        ) TO reported.
    ENDIF.
ENDLOOP.

ENDMETHOD.

```

Resume with Additional Implementation

The draft capabilities of the BO require an additional check implementation for an `edit draft` use case. If a draft has expired and the `AgencyID` was changed and the user comes back to work on the instance, the precheck implementation must be triggered again to ensure the consistency of the authority check against the `country_code` value. The implementation read the `AgencyID` of the travel. Then the `%control-AgencyID` is set to true to trigger the precheck implementation. An `edit draft` scenario is treated as `update` in case of the precheck, so the helper method `precheck_auth` is called with the exporting parameter `entities_update`. The helper method treats this call as an `update` in its implementation.

i *Expand the following code sample to view the source code of the method **resume**.*

Sample Code

```

METHOD resume.
    DATA:
        entities_update TYPE t_entities_update.

    READ ENTITIES OF /dmo/r_travel_d IN LOCAL MODE
    ENTITY Travel
        FIELDS ( AgencyID )
        WITH VALUE #(
            FOR key IN keys
                %is_draft = if_abap_behv=>mk-on
                ( %key = key-%key )
        )
    RESULT DATA(travels).

```



```
" Set %control-AgencyID (if set) to true, so that the precheck_auth checks the permissions.
entities_update = CORRESPONDING #( travels CHANGING CONTROL ).
```

```
IF entities_update IS NOT INITIAL.
  precheck_auth(
    EXPORTING
      entities_update = entities_update
    CHANGING
      failed          = failed-travel
      reported        = reported-travel
  ).
ENDIF.
ENDMETHOD.
```

Developing Validations

For the managed scenario with draft, add validations to check the values provided by the client.

Context and Procedure

Validations are used to check whether provided values by a client are consistent. They give direct feedback (messages) before the BO instance is saved to the database. For more detailed information, see [Validations](#).

i Note

You can assign validations to determine actions that are called by RAP side effects. This enables immediate feedback after the user changes UI fields or field groups. For more detailed information, see [Action Definition](#).

As validations verify the state of an instance, state messages are used to return messages.

i Note

State messages reflect the business object state. Since validations check for data inconsistencies, they return state messages. State messages are stored on the database and can be retrieved together with the BO instance they relate to. In draft scenarios, state messages are used to describe data inconsistencies. As state messages are persisted, they must be invalidated in the implementation. For more information, refer to [State Messages](#).

For messages that relate to a change of a BO instance, transition messages are used. Every modify request is a change of state. If a modify requests fails, transition messages are used to describe and explain the failure. Transition messages are not saved on a database. They disappear automatically once the failed operation is rolled back. For more information, refer to [Transition Messages](#).

For the travel demo scenario, define and implement validation for all three BO entities:

- For the **travel entity**, the following validations are described:
 - `validateCustomer`

The provided customer ID is checked against the entries in `/DMO/CUSTOMER`. Saving instances with invalid or initial values for the customer ID is rejected. In such a case, the validation returns failed keys and a message.

The validation is triggered on `create` and on every update of the trigger field `CustomerID`.

- **validateAgency**

The provided agency ID is checked against the entries in **/DMO/AGENCY**. Saving instances with invalid or initial values for the agency ID is rejected. In such a case, the validation returns failed keys and a message.

The validation is triggered on **create** and on every update of the trigger field **AgencyID**.

- **validateDates**

The provided starting date is checked against the system date. It cannot be earlier than the system date. In addition, the starting and the end date are compared against it each other. The end date cannot be before the start date. If one of the conditions is not fulfilled, the saving is rejected and the validation returns failed keys and a message.

The validation is triggered on **create** and on every update of the trigger fields **BeginDate** and **EndDate**.

- **validateCurrencyCode**

The provided currency code is checked against the entries in **I_Currency**. Saving instances with invalid or initial values for the currency code is rejected. In such a case, the validation returns failed keys and a message

The validation is triggered on **create** and on every update of the trigger field **CurrencyCode**.

- **validateBookingFee**

The validation checks whether the provided booking fee is less than 0. If this is the case, the saving is rejected and the validation returns failed keys and a message.

The validation is triggered on **create** and on every update of the trigger field **BookingFee**.

- For the **booking entity**, the following validations are described:

- **validateCustomer**

See [validateCustomer](#) of travel entity.

- **validateConnection**

The provided values for **AirlineID**, **ConnectionID** and **FlightDate** are checked against the flight data in **/DMO/FLIGHT**. Only existing flights with valid connection and airline IDs can be added to the booking instance. If the condition is not fulfilled, the validation returns failed keys and a message.

The validation is triggered on **create** and every update of the trigger fields **AirlineID**, **ConnectionID**, and **FlightDate**.

- **validateCurrencyCode**

The provided currency code is checked against the entries in **I_Currency**. Saving instances with invalid or initial values for the currency code is rejected. In such a case, the validation returns failed keys and a message

The validation is triggered on **create** and on every update of the trigger field **CurrencyCode**.

- **validateFlightPrice**

The validation checks whether the provided flight price is less than 0. If this is the case, the saving is rejected and the validation returns failed keys and a message.

The validation is triggered on **create** and on every update of the trigger field **FlightPrice**.

- **validateStatus**

The validation checks whether the provided booking status matches one of the predefined values. If this is not the case, the saving is rejected and the validation returns failed keys and a message.

The validation is triggered on create and on every update of the trigger field BookingStatus.

- For the **booking supplement entity**, the following validation is described:

- **validateSupplement**

The provided supplement ID is checked against the entries in /DMO/SUPPLEMENT. Saving instances with invalid or initial values for the supplement ID is rejected. In such a case, the validation returns failed keys and a message.

The validation is triggered on create and on every update of the trigger field SupplementID.

- **validateCurrencyCode**

The provided currency code is checked against the entries in I_Currency. Saving instances with invalid or initial values for the currency code is rejected. In such a case, the validation returns failed keys and a message

The validation is triggered on create and on every update of the trigger field CurrencyCode.

- **validateSupplementPrice**

The validation checks whether the entered value is less than 0. In such a case, the validation returns failed keys and a message.

The validation is triggered on create and on every update of the trigger field BookSupplPrice.

For a detailed description on how to implement the described behavior for the relevant entity, check the implementation steps in the following topics.

[Validating Values for the Travel Entity](#)

For the managed scenario with draft, define and implement validations on the travel entity for value consistency checks.

[Validating Values for the Booking Entity](#)

For the managed scenario with draft, define and implement validations on the booking entity for value consistency checks.

[Validating Values for the Booking Supplement Entity](#)

For the managed scenario with draft, define and implement validations on the booking supplement entity for value consistency checks.

Validating Values for the Travel Entity

For the managed scenario with draft, define and implement validations on the travel entity for value consistency checks.

Validation **validateCustomer**

Define the validation in the behavior definition and implement it in the behavior implementation class for **Travel**.

Definition

Define a validation on save with trigger operation create and trigger field CustomerID. Since there must always be a customer assigned to a certain travel, define the field CustomerID as mandatory.

≡ Sample Code

```
define behavior for /DMO/R_Travel_D alias Travel
...
{ ...
  field ( mandatory ) CustomerID;
```

```
validation validateCustomer on save { create; field CustomerID; }

... }
```

Via a quick fix, you can generate the method declaration in the behavior pool directly from the behavior definition editor.

Implementation

Validate the customer ID by checking if the provided value is the ID of an entry in the customer database table. Raise adequate messages for the consumer if the value is initial or not valid. The following steps guide you through the implementation.

1. Read all the instances with the imported keys into an internal table `travels`. This table is the basis to check whether messages must be raised.
2. Copy the entries to another internal table and delete all instances with an initial customer ID. If the resulting internal table is not initial, select the entries of `/DMO/CUSTOMER` with the given customer ID. If there is no corresponding entry in this database table, the provided customer ID is not valid.
3. Loop over the internal table `travels`. To avoid duplicate state messages for the consumer, append an empty message to the reported table to clear the state area.
4. If the customer ID is initial, write the transactional key to the failed table and append the corresponding message to the state area `VALIDATE_CUSTOMER`. The RAP runtime framework provides the method `new_message`, with which you can easily retrieve messages from message classes provide the respective parameters. The reported table includes the component `%element`. Here you can specify which CDS element is responsible for the state inconsistency. The Fiori Elements UI, evaluates this component and highlights the corresponding input field.
5. If there is no entry in the customer master data table, write the transactional key to the failed table and append the corresponding message to the state area `VALIDATE_CUSTOMER`.

The messages for the managed scenario with draft are stored in the message class `/dmo/cm_flight` that is part of the ABAP Flight Reference Scenario.

i Expand the following code sample to view the source code of Validation `validateCustomer`.

Sample Code

```
METHOD validateCustomer.

  READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
    ENTITY Travel
      FIELDS ( CustomerID )
      WITH CORRESPONDING #( keys )
    RESULT DATA(travels).

  DATA customers TYPE SORTED TABLE OF /DMO/CUSTOMER WITH UNIQUE KEY customer_id.

  " Optimization of DB select: extract distinct non-initial customer IDs
  customers = CORRESPONDING #( travels DISCARDING DUPLICATES MAPPING customer_id = CustomerID
  DELETE customers WHERE customer_id IS INITIAL.

  IF customers IS NOT INITIAL.
    " Check if customer ID exists
    SELECT FROM /DMO/CUSTOMER FIELDS customer_id
      FOR ALL ENTRIES IN @customers
      WHERE customer_id = @customers-customer_id
    INTO TABLE @DATA(valid_customers).
  ENDIF.

  " Raise message for non existing customer id
  LOOP AT travels INTO DATA(travel).
```

```

APPEND VALUE #( %tky                = travel-%tky
                 %state_area        = 'VALIDATE_CUSTOMER'
                 ) TO reported-travel.

IF travel-CustomerID IS INITIAL.
  APPEND VALUE #( %tky = travel-%tky ) TO failed-travel.

  APPEND VALUE #( %tky                = travel-%tky
                 %state_area        = 'VALIDATE_CUSTOMER'
                 %msg               = NEW /dmo/cm_flight_messages(
                                     textid   = /dmo/cm_flight_messa
                                     severity = if_abap_behv_message=
                                     %element-CustomerID = if_abap_behv=>mk-on
                                   ) TO reported-travel.

ELSEIF travel-CustomerID IS NOT INITIAL AND NOT line_exists( valid_customers[ customer_id
  APPEND VALUE #( %tky = travel-%tky ) TO failed-travel.

  APPEND VALUE #( %tky                = travel-%tky
                 %state_area        = 'VALIDATE_CUSTOMER'
                 %msg               = NEW /dmo/cm_flight_messages(
                                     customer_id = travel-customerid
                                     textid      = /dmo/cm_flight_me
                                     severity    = if_abap_behv_messa
                                     %element-CustomerID = if_abap_behv=>mk-on
                                   ) TO reported-travel.

ENDIF.

ENDLOOP.
ENDMETHOD.

```

Validation `validateAgency`

Define the validation in the behavior definition and implement it in the behavior implementation class for [Travel](#).

Definition

Define a validation on save with trigger operation `create` and trigger field `AgencyID`. Since there must always be an agency assigned to a certain travel, define the field `AgencyID` as mandatory.

Sample Code

```

define behavior for /DMO/R_Travel_D alias Travel
...
{ ...
  field ( mandatory ) CustomerID, AgencyID;
  validation validateAgency on save { create; field AgencyID; }

... }

```

Via a quick fix, you can generate the method declaration in the behavior pool directly from the behavior definition editor.

Implementation

Validate the agency ID by checking if the provided value is the ID of an entry in the agency database table. Raise adequate messages for the consumer if the value is initial or not valid. The implementation steps are exactly the same as for the validation `ValidateCustomer`.

i *Expand the following code sample to view the source code of Validation `validateAgency`.*

≡, Sample Code

```

METHOD validateAgency.
  DATA: modification_granted TYPE abap_boolean,
        agency_country_code  TYPE land1.

  READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
    ENTITY Travel
      FIELDS ( AgencyID TravelID )
      WITH CORRESPONDING #( keys )
      RESULT DATA(travels).

  DATA agencies TYPE SORTED TABLE OF /dmo/agency WITH UNIQUE KEY agency_id.

  " Optimization of DB select: extract distinct non-initial agency IDs
  agencies = CORRESPONDING #( travels DISCARDING DUPLICATES MAPPING agency_id = AgencyID EXCEPT
  DELETE agencies WHERE agency_id IS INITIAL.

  IF agencies IS NOT INITIAL.
    " Check if Agency ID exists
    SELECT FROM /dmo/agency FIELDS agency_id, country_code
      FOR ALL ENTRIES IN @agencies
      WHERE agency_id = @agencies-agency_id
    INTO TABLE @DATA(valid_agencies).
  ENDIF.

  " Raise message for non existing Agency id
  LOOP AT travels INTO DATA(travel).
    APPEND VALUE #( %tky          = travel-%tky
                   %state_area   = 'VALIDATE_AGENCY'
                   ) TO reported-travel.

    IF travel-AgencyID IS INITIAL.
      APPEND VALUE #( %tky = travel-%tky ) TO failed-travel.

      APPEND VALUE #( %tky          = travel-%tky
                     %state_area   = 'VALIDATE_AGENCY'
                     %msg          = NEW /dmo/cm_flight_messages(
                                     textid   = /dmo/cm_flight_messages=>en
                                     severity = if_abap_behv_message=>severity
                                     %element-AgencyID = if_abap_behv=>mk-on
                                   ) TO reported-travel.

    ELSEIF travel-AgencyID IS NOT INITIAL AND NOT line_exists( valid_agencies[ agency_id = travel-AgencyID ] )
      APPEND VALUE #( %tky = travel-%tky ) TO failed-travel.

      APPEND VALUE #( %tky          = travel-%tky
                     %state_area   = 'VALIDATE_AGENCY'
                     %msg          = NEW /dmo/cm_flight_messages(
                                     agency_id = travel-agencyid
                                     textid   = /dmo/cm_flight_messages=>en
                                     severity = if_abap_behv_message=>severity
                                     %element-AgencyID = if_abap_behv=>mk-on
                                   ) TO reported-travel.

    ENDIF.
  ENDLOOP.
ENDMETHOD.

```

Validation validateDates

Define the validation in the behavior definition and implement it in the behavior implementation class for [Travel](#).

Definition

Define a validation on save with trigger operation `create` and trigger fields `BeginDate` and `EndDate`. Since the travel dates are an essential part of the travel data, define the fields `BeginDate` and `EndDate` as mandatory.

Sample Code

```
define behavior for /DMO/R_Travel_D alias Travel
...
{ ...
  field ( mandatory ) CustomerID, AgencyID, BeginDate, EndDate;
  validation validateDates on save { create; field BeginDate, EndDate; }

... }
```

Implementation

Validate the travel dates by comparing the value of the `BeginDate` with the `EndDate`. The end date must not be before the begin date and the begin date must be after the system date. Raise adequate messages for the consumer if the provided values are not valid. The following steps guide you through the implementation. The messages for the managed scenario with draft are stored in the message class `/DMO/CM_FLIGHT` that is part of the ABAP Flight Reference Scenario.

1. Read all the instances with the imported keys into an internal table `travels`. This table is the basis to check whether messages must be raised.
2. Loop over the internal table `travels`. To avoid duplicate state messages for the consumer, append an empty message to the reported table to clear the state area.
3. The travel date must fulfill four conditions:
 - a. Begin date must not be initial.
 - b. End date must not be initial.
 - c. Begin date must not be before system date.
 - d. End date must not be before begin date.

For all of the situations, check with an `if`-loop and append the transactional key to the failed table. In addition, append the corresponding message to the state area `VALIDATE_DATES`. The RAP runtime framework provides the method `new message`, with which you can easily retrieve messages from message classes provide the respective parameters. The reported table includes the component `%element`. Here you can specify which CDS element is responsible for the state inconsistency. The Fiori Elements UI, evaluates this component and highlights the corresponding input field.

 **Expand the following code sample to view the source code of Validation `validateDates`.**

Sample Code

```
METHOD validateDates.

    READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
    ENTITY Travel
      FIELDS ( BeginDate EndDate TravelID )
      WITH CORRESPONDING #( keys )
    RESULT DATA(travels).

    LOOP AT travels INTO DATA(travel).

      APPEND VALUE #( %tky          = travel-%tky
                     %state_area = 'VALIDATE_DATES' ) TO reported-travel.
```

```

IF travel-BeginDate IS INITIAL.
  APPEND VALUE #( %tky = travel-%tky ) TO failed-travel.

  APPEND VALUE #( %tky          = travel-%tky
                  %state_area   = 'VALIDATE_DATES'
                  %msg          = NEW /dmo/cm_flight_messages(
                                textid   = /dmo/cm_flight_message=
                                severity = if_abap_behv_message=
                                %element-BeginDate = if_abap_behv=>mk-on ) TO reported-travel.
ENDIF.
IF travel-EndDate IS INITIAL.
  APPEND VALUE #( %tky = travel-%tky ) TO failed-travel.

  APPEND VALUE #( %tky          = travel-%tky
                  %state_area   = 'VALIDATE_DATES'
                  %msg          = NEW /dmo/cm_flight_messages(
                                textid   = /dmo/cm_flight_message=
                                severity = if_abap_behv_message=
                                %element-EndDate   = if_abap_behv=>mk-on ) TO reported-travel.
ENDIF.
IF travel-EndDate < travel-BeginDate AND travel-BeginDate IS NOT INITIAL
  AND travel-EndDate IS NOT INITIAL.
  APPEND VALUE #( %tky = travel-%tky ) TO failed-travel.

  APPEND VALUE #( %tky          = travel-%tky
                  %state_area   = 'VALIDATE_DATES'
                  %msg          = NEW /dmo/cm_flight_messages(
                                textid   = /dmo/cm_flight_message=
                                begin_date = travel-BeginDate
                                end_date   = travel-EndDate
                                severity   = if_abap_behv_message=
                                %element-BeginDate = if_abap_behv=>mk-on
                                %element-EndDate   = if_abap_behv=>mk-on ) TO reported-travel.
ENDIF.
IF travel-BeginDate < cl_abap_context_info=>get_system_date( ) AND travel-BeginDate IS NOT
  APPEND VALUE #( %tky          = travel-%tky ) TO failed-travel.

  APPEND VALUE #( %tky          = travel-%tky
                  %state_area   = 'VALIDATE_DATES'
                  %msg          = NEW /dmo/cm_flight_messages(
                                begin_date = travel-BeginDate
                                textid     = /dmo/cm_flight_message=
                                severity    = if_abap_behv_message=
                                %element-BeginDate = if_abap_behv=>mk-on ) TO reported-travel.
ENDIF.

ENDLOOP.

ENDMETHOD.

```

Validation validateCurrencyCode

Define the validation in the behavior definition and implement it in the behavior implementation class for [Travel](#).

Definition

Define a validation on save with trigger operation create and trigger field CurrencyCode. Since the currency code must always be provided for a booking, define the field CurrencyCode as mandatory.

Sample Code

```

define behavior for /DMO/R_Travel_D alias Travel
...
{ ...
  field ( mandatory ) CurrencyCode;
  validation validateCurrencyCode on save { create; field CurrencyCode; }
}

```



```
... }
```

Via a quick fix, you can generate the method declaration in the behavior pool directly from the behavior definition editor.

Implementation

Validate the currency code by checking if the provided value is the ID of an entry in the currency database table. Raise adequate messages for the consumer if the value is initial or not valid. The following steps guide you through the implementation.

1. Read all the instances with the imported keys into an internal table `travels`. This table is the basis to check whether messages must be raised.
2. Copy the entries to another internal table and delete all instances with an initial currency code. If the resulting internal table is not initial, select the entries of `I_Currency` with the given currency code. If there is no corresponding entry in this database table, the provided currency code is not valid.
3. Loop over the internal table `travels`. To avoid duplicate state messages for the consumer, append an empty message to the reported table to clear the state area.
4. If the currency code is initial, write the transactional key to the failed table and append the corresponding message to the state area `VALIDATE_CURRENCYCODE`. The RAP runtime framework provides the method `new_message`, with which you can easily retrieve messages from message classes provide the respective parameters. The reported table includes the component `%element`. Here you can specify which CDS element is responsible for the state inconsistency. The Fiori Elements UI, evaluates this component and highlights the corresponding input field.
5. If there is no entry in the currency code master data table, write the transactional key to the failed table and append the corresponding message to the state area `VALIDATE_CURRENCYCODE`.

The messages for the managed scenario with draft are stored in the message class `/dmo/cm_flight_messages` that is part of the ABAP Flight Reference Scenario.

i *Expand the following code sample to view the source code of Validation `validateCurrencyCode`.*

Sample Code

```
METHOD validateCustomer.

  READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
    ENTITY Travel
      FIELDS ( CustomerID )
      WITH CORRESPONDING #( keys )
    RESULT DATA(travels).

  DATA: currencies TYPE SORTED TABLE OF I_Currency WITH UNIQUE KEY currency.

  currencies = CORRESPONDING #( travels DISCARDING DUPLICATES MAPPING currency = currencycode
  DELETE currencies WHERE currency IS INITIAL.

  IF currencies IS NOT INITIAL.
    SELECT FROM I_Currency FIELDS currency
      FOR ALL ENTRIES IN @currencies
      WHERE currency = @currencies-currency
      INTO TABLE @DATA(currency_db).
  ENDIF.

  LOOP AT travels INTO DATA(travel).
    APPEND VALUE #( %tky = travel-%tky
                   %state_area = 'VALIDATE_CURRENCYCODE'
                   ) TO reported-travel.
    IF travel-currencycode IS INITIAL.
```

```

" Raise message for empty Currency
APPEND VALUE #( %tky                = travel-%tky ) TO failed-travel.
APPEND VALUE #( %tky                = travel-%tky
                 %state_area        = 'VALIDATE_CURRENCYCODE'
                 %msg                = NEW /dmo/cm_flight_messages(
                                     textid      = /dmo/cm_flight_messages=>cu
                                     severity    = if_abap_behv_message=>severi
                                     %element-currencycode = if_abap_behv=>mk-on
                                     ) TO reported-travel.
ELSEIF NOT line_exists( currency_db[ currency = travel-currencycode ] ).
" Raise message for not existing Currency
APPEND VALUE #( %tky                = travel-%tky ) TO failed-travel.
APPEND VALUE #( %tky                = travel-%tky
                 %state_area        = 'VALIDATE_CURRENCYCODE'
                 %msg                = NEW /dmo/cm_flight_messages(
                                     textid      = /dmo/cm_flight_message:
                                     severity    = if_abap_behv_message=>se
                                     currency_code = travel-currencycode )
                                     %element-currencycode = if_abap_behv=>mk-on
                                     ) TO reported-travel.

ENDIF.
ENDLOOP.
ENDMETHOD.

```

Validation validateBookingFee

Define the validation in the behavior definition and implement it in the behavior implementation class for [Travel](#).

Definition

Define a validation on save with trigger operation create and trigger field BookingFee.

Sample Code

```

define behavior for /DMO/R_Travel_D alias Travel
...
{ ...
  validation validateBookingFee    on save { create; field BookingFee; }

... }

```

Via a quick fix, you can generate the method declaration in the behavior pool directly from the behavior definition editor.

Implementation

Validate the booking fee by checking if the entered value is less than 0. Raise an adequate message for the consumer if the value is not valid. The following steps guide you through the implementation.

1. Read all the instances with the imported keys into an internal table `travels`. This table is the basis to check whether messages must be raised.
2. Loop over the internal table `travels`. To avoid duplicate state messages for the consumer, append an empty message to the reported table to clear the state area.
3. If the booking fee is less than 0, write the transactional key to the failed table and append a corresponding message along with the state area `VALIDATE_BOOKINGFEE` to the reported table for that instance. The reported table includes the component `%element`. Here you can specify which CDS element is responsible for the state inconsistency. The Fiori Elements UI evaluates this component and highlights the corresponding input field.

The messages for the managed scenario with draft are stored in the message class `/dmo/cm_flight_messages` that is part of the ABAP Flight Reference Scenario.

i Expand the following code sample to view the source code of Validation **validateBookingFee**.

Sample Code

```
METHOD validateBookingFee.

    READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
        ENTITY travel
            FIELDS ( BookingFee )
            WITH CORRESPONDING #( keys )
    RESULT DATA(travels).

    LOOP AT travels INTO DATA(travel).
        APPEND VALUE #( %tky          = travel-%tky
                        %state_area   = 'VALIDATE_BOOKINGFEE'
                        ) TO reported-travel.
        " Raise message for booking fee < 0
        IF travel-BookingFee < 0.
            APPEND VALUE #( %tky          = travel-%tky ) TO failed-travel.
            APPEND VALUE #( %tky          = travel-%tky
                            %state_area   = 'VALIDATE_BOOKINGFEE'
                            %msg          = NEW /dmo/cm_flight_messages(
                                                textid      = /dmo/cm_flight_messages=>boo
                                                severity    = if_abap_behv_message=>severi
                                                )
                            %element-BookingFee = if_abap_behv=>mk-on
                        ) TO reported-travel.
        ENDIF.
    ENDLOOP.
ENDMETHOD.
```

Validating Values for the Booking Entity

For the managed scenario with draft, define and implement validations on the booking entity for value consistency checks.

Validation **validateCustomer**

Define the validation in the behavior definition and implement it in the behavior implementation class for **Booking**.

Definition

Define a validation on save with trigger operation `create` and trigger field `CustomerID`. Since there must always be a customer assigned to a certain travel, define the field `CustomerID` is mandatory.

Sample Code

```
define behavior for /DMO/R_Booking_D alias Booking
...
{ ...
    field ( mandatory ) CustomerID;
    validation validateCustomer on save { create; field CustomerID; }
... }
```

Via a quick fix, you can generate the method declaration in the behavior pool directly from the behavior definition editor.

Implementation

The validation for the customer ID field is done similarly to the validation for the customer ID in the travel entity. See [Validation validateCustomer](#).

To retrieve the correct messages of validations in child entities and to display them on the UI, you have to define a path expression to the root entity. To fill the path expression with the correct keys, you have to read the keys of the parent entity first. This is done by a read by association. You do not need the full result of the read by association, but just the link table, which contains the key mappings of source and target entity.

i *Expand the following code sample to view the source code of Validation **validateCustomer**.*

Sample Code

```
METHOD validateCurrencyCode.
  READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
    ENTITY booking
      FIELDS ( currencycode )
      WITH CORRESPONDING #( keys )
    RESULT DATA(bookings).

  READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
    ENTITY Booking BY \_Travel
      FROM CORRESPONDING #( bookings )
    LINK DATA(travel_booking_links).

DATA: currencies TYPE SORTED TABLE OF I_Currency WITH UNIQUE KEY currency.

currencies = CORRESPONDING #( bookings DISCARDING DUPLICATES MAPPING currency = currencycode
DELETE currencies WHERE currency IS INITIAL.

IF currencies IS NOT INITIAL.
  SELECT FROM I_Currency FIELDS currency
    FOR ALL ENTRIES IN @currencies
    WHERE currency = @currencies-currency
    INTO TABLE @DATA(currency_db).
ENDIF.

LOOP AT bookings INTO DATA(booking).
  APPEND VALUE #( %tky = booking-%tky
                  %state_area = 'VALIDATE_CURRENCYCODE'
                  ) TO reported-booking.
  IF booking-currencycode IS INITIAL.
    " Raise message for empty Currency
    APPEND VALUE #( %tky = booking-%tky ) TO failed-booking.
    APPEND VALUE #( %tky = booking-%tky
                  %state_area = 'VALIDATE_CURRENCYCODE'
                  %msg = NEW /dmo/cm_flight_messages(
                        textid = /dmo/cm_flight_messages=>cu
                        severity = if_abap_behv_message=>severi
                        %path = VALUE #( travel-%tky = travel_booking_links[ KE
                        %element-currencycode = if_abap_behv=>mk-on
                        ) TO reported-booking.
  ELSEIF NOT line_exists( currency_db[ currency = booking-currencycode ] ).
    " Raise message for not existing Currency
    APPEND VALUE #( %tky = booking-%tky ) TO failed-booking.
    APPEND VALUE #( %tky = booking-%tky
                  %state_area = 'VALIDATE_CURRENCYCODE'
                  %msg = NEW /dmo/cm_flight_messages(
                        textid = /dmo/cm_flight_messages=>cu
                        severity = if_abap_behv_message=>se
                        currency_code = booking-currencycode )
                        %path = VALUE #( travel-%tky = travel_booking_links[ KE
                        %element-currencycode = if_abap_behv=>mk-on
                        ) TO reported-booking.
  ENDIF.
```

Validation validateConnection

Define the validation in the behavior definition and implement it in the behavior implementation class for [Booking](#).

Definition

Define a validation on save with trigger operation `create` and trigger fields `AirlineID`, `ConnectionID`, and `FlightDate`. Since a booking instance must always entail this flight information, define these fields as mandatory.

Sample Code

```
define behavior for /DMO/R_Booking_D alias Booking
...
{ ...
  field ( mandatory ) CustomerID, AirlineID, ConnectionID, FlightDate;
  validation validateConnection on save { create; field AirlineID, ConnectionID, FlightDate; }
... }
```

Via a quick fix, you can generate the method declaration in the behavior pool directly from the behavior definition editor.

Implementation

Validate the flight-related fields by checking if the provided values match an entry in the master database table `/DMO/FLIGHT`. Raise adequate messages for the consumer if the values are initial or not valid. The following steps guide you through the implementation.

1. Read all the instances with the imported keys into an internal table `bookings`. This table is the basis to check whether messages must be raised.
2. Read the corresponding travel instances via a read by association. The parent keys are needed for the path expression `%path` in the reported table. It represents the path the path to the root entity and is important to display the messages correctly on the UI.
3. Loop over the internal table `bookings`. To avoid duplicate state messages for the consumer, append an empty message to the reported table to clear the state area.
4. If the airline ID is initial, write the transactional key to the failed table and append the corresponding message to the state area `VALIDATE_CONNECTION`.

Messages are created with the `/dmo/cm_flight_messages` message exception class, with which you can easily retrieve messages from message classes and provide the respective parameters.

The reported table contains the component `%path`, in which you have to maintain the keys of all parent instances, including the root entity. Use the key mappings in the link table to fill the path expression. Only if all parent keys are maintained is the message displayed correctly on the UI.

The reported table includes the component `%element`. Here you can specify which CDS element is responsible for the state inconsistency. The Fiori Elements UI, evaluates this component and highlights the corresponding input field.

5. Repeat the step for the values in `ConnectionID` and `FlightDate`. Like `AirlineID`, they must not be initial.

6. If all related fields are not empty you can check if the values match an entry in the flight master data table. If the select finds a corresponding row (=sy-subrc <> 0), write the transactional key to the failed table and append the corresponding message to the state area `VALIDATE_CONNECTION`.

i Expand the following code sample to view the source code of Validation `validateConnection`.

Sample Code

METHOD `validateConnection`.

```

READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
  ENTITY Booking
    FIELDS ( BookingID AirlineID ConnectionID FlightDate )
    WITH CORRESPONDING #( keys )
    RESULT DATA(bookings).

READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
  ENTITY Booking BY \_Travel
  FROM CORRESPONDING #( bookings )
  LINK DATA(travel_booking_links).

LOOP AT bookings ASSIGNING FIELD-SYMBOL(<booking>).
  "overwrite state area with empty message to avoid duplicate messages
  APPEND VALUE #( %tky          = <booking>-%tky
                  %state_area   = 'VALIDATE_CONNECTION' ) TO reported-booking.

  " Raise message for non existing airline ID
  IF <booking>-AirlineID IS INITIAL.
    APPEND VALUE #( %tky = <booking>-%tky ) TO failed-booking.

    APPEND VALUE #( %tky          = <booking>-%tky
                  %state_area   = 'VALIDATE_CONNECTION'
                  %msg          = NEW /dmo/cm_flight_messages(
                                textid = /dmo/cm_flight_message
                                severity = if_abap_behv_message=
                                %path    = VALUE #( travel-%tky = travel_booking_links[ KEY id
                                %element-AirlineID = if_abap_behv=>mk-on
                                ) TO reported-booking.
                  ) TO reported-booking.
  ENDIF.

  " Raise message for non existing connection ID
  IF <booking>-ConnectionID IS INITIAL.
    APPEND VALUE #( %tky = <booking>-%tky ) TO failed-booking.

    APPEND VALUE #( %tky          = <booking>-%tky
                  %state_area   = 'VALIDATE_CONNECTION'
                  %msg          = NEW /dmo/cm_flight_messages(
                                textid = /dmo/cm_flight_message
                                severity = if_abap_behv_message=
                                %path    = VALUE #( travel-%tky = travel_booking_links[ KEY i
                                %element-ConnectionID = if_abap_behv=>mk-on
                                ) TO reported-booking.
                  ) TO reported-booking.
  ENDIF.

  " Raise message for non existing flight date
  IF <booking>-FlightDate IS INITIAL.
    APPEND VALUE #( %tky = <booking>-%tky ) TO failed-booking.

    APPEND VALUE #( %tky          = <booking>-%tky
                  %state_area   = 'VALIDATE_CONNECTION'
                  %msg          = NEW /dmo/cm_flight_messages(
                                textid = /dmo/cm_flight_message
                                severity = if_abap_behv_message=
                                %path    = VALUE #( travel-%tky = travel_booking_links[ KEY i
                                %element-FlightDate = if_abap_behv=>mk-on
                                ) TO reported-booking.
                  ) TO reported-booking.
  ENDIF.

  " check if flight connection exists
  IF <booking>-AirlineID IS NOT INITIAL AND
    <booking>-ConnectionID IS NOT INITIAL AND
    <booking>-FlightDate IS NOT INITIAL.

    SELECT SINGLE Carrier_ID, Connection_ID, Flight_Date FROM /DMO/FLIGHT WHERE carrier_

```

```

AND connection_id = @<booking>-C
AND flight_date   = @<booking>-F
INTO @DATA(flight).

```

```

IF sy-subrc <> 0.
  APPEND VALUE #( %tky = <booking>-%tky ) TO failed-booking.

  APPEND VALUE #( %tky           = <booking>-%tky
                  %state_area    = 'VALIDATE_CONNECTION'
                  %msg           = NEW /dmo/cm_flight_messages(
                                textid      = /dmo/cm_flight_me
                                carrier_id   = <booking>-AirlineI
                                flight_date  = <booking>-FlightDa
                                severity     = if_abap_behv_messa
                                %path        = VALUE #( travel-%tky = travel_booking_links[
                  %element-FlightDate      = if_abap_behv=>mk-on
                  %element-AirlineID       = if_abap_behv=>mk-on
                  %element-ConnectionID    = if_abap_behv=>mk-on
                  ) TO reported-booking.

ENDIF.

ENDIF.

ENDLOOP.

ENDMETHOD.

```

Validation `validateCurrencyCode`

Define the validation in the behavior definition and implement it in the behavior implementation class for [Booking](#).

Definition

Define a validation on save with trigger operation `create` and trigger field `CurrencyCode`. Since there must always be a currency code assigned to a booking price, define the field `CurrencyCode` as mandatory.

≡ Sample Code

```

define behavior for /DMO/R_Booking_D alias Booking
...
{ ...
  field ( mandatory ) CurrencyCode;
  validation validateCurrencyCode on save { create; field CurrencyCode; }

... }

```

Via a quick fix, you can generate the method declaration in the behavior pool directly from the behavior definition editor.

Implementation

The validation for the currency code field is done similarly to the validation for the currency code in the travel entity. See [Validation `validateCurrencyCode`](#).

To retrieve the correct messages of validations in child entities and to display them on the UI, you have to define a path expression to the root entity. To fill the path expression with the correct keys, you have to read the keys of the parent entity first. This is done by a read by association. You do not need the full result of the read by association, but just the link table, which contains the key mappings of source and target entity.

i Expand the following code sample to view the source code of Validation **validateCustomer**.

Sample Code

```

METHOD validateCurrencyCode.
    READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
    ENTITY booking
        FIELDS ( currencycode )
        WITH CORRESPONDING #( keys )
    RESULT DATA(bookings).

DATA: currencies TYPE SORTED TABLE OF I_Currency WITH UNIQUE KEY currency.

currencies = CORRESPONDING #( bookings DISCARDING DUPLICATES MAPPING currency = currencycode
DELETE currencies WHERE currency IS INITIAL.

IF currencies IS NOT INITIAL.
    SELECT FROM I_Currency FIELDS currency
    FOR ALL ENTRIES IN @currencies
    WHERE currency = @currencies-currency
    INTO TABLE @DATA(currency_db).
ENDIF.

LOOP AT bookings INTO DATA(booking).
    APPEND VALUE #( %tky                = booking-%tky
                    %state_area         = 'VALIDATE_CURRENCYCODE'
                    ) TO reported-booking.
    IF booking-currencycode IS INITIAL.
        " Raise message for empty Currency
        APPEND VALUE #( %tky                = booking-%tky ) TO failed-booking.
        APPEND VALUE #( %tky                = booking-%tky
                        %state_area         = 'VALIDATE_CURRENCYCODE'
                        %msg                = NEW /dmo/cm_flight_messages(
                                                textid      = /dmo/cm_flight_messages=>cu
                                                severity    = if_abap_behv_message=>severi
                                                %element-currencycode = if_abap_behv=>mk-on
                                                ) TO reported-booking.
    ELSEIF NOT line_exists( currency_db[ currency = booking-currencycode ] ).
        " Raise message for not existing Currency
        APPEND VALUE #( %tky                = booking-%tky ) TO failed-booking.
        APPEND VALUE #( %tky                = booking-%tky
                        %state_area         = 'VALIDATE_CURRENCYCODE'
                        %msg                = NEW /dmo/cm_flight_messages(
                                                textid      = /dmo/cm_flight_messages=>cu
                                                severity    = if_abap_behv_message=>se
                                                currency_code = booking-currencycode )
                                                %element-currencycode = if_abap_behv=>mk-on
                                                ) TO reported-booking.
    ENDIF.
ENDLOOP.
ENDMETHOD.

```

Validation **validateFlightPrice**

Define the validation in the behavior definition and implement it in the behavior implementation class for **Booking**.

Definition

Define a validation on save with trigger operation create and trigger field FlightPrice.

Sample Code

```

define behavior for /DMO/R_Booking_D alias Booking
    ...

```



```

{ ...
  validation validateFlightPrice on save { create; field FlightPrice; }
... }

```

Via a quick fix, you can generate the method declaration in the behavior pool directly from the behavior definition editor.

Implementation

Validate the flight price by checking if the entered value is less than 0. Raise an adequate message for the consumer if the value is not valid. The following steps guide you through the implementation.

1. Read all the instances with the imported keys into an internal table `bookings`. This table is the basis to check whether messages must be raised.
2. Read the keys of the corresponding parent instances using the association `_Travel` and store the result in the `link` response. This information will be needed for linking booking instances to travel instances in the reported response.
3. Loop over the internal table `bookings`. To avoid duplicate state messages for the consumer, append an empty message to the reported table to clear the state area.
4. If the flight price is less than 0, write the transactional key to the failed table and append a corresponding message along with the state area `VALIDATE_FLIGHTPRICE` to the reported table for that instance. The reported table includes the component `%element`. Here you can specify which CDS element is responsible for the state inconsistency. The Fiori Elements UI evaluates this component and highlights the corresponding input field. Use the `travel_booking_links` table populated above to provide the keys of the corresponding travel instances for each booking instance.

The messages for the managed scenario with draft are stored in the message class `/dmo/cm_flight_messages` that is part of the ABAP Flight Reference Scenario.

i Expand the following code sample to view the source code of Validation **`validateFlightPrice`**.

Sample Code

```

METHOD validateFlightPrice.

  READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
    ENTITY booking
      FIELDS ( FlightPrice )
      WITH CORRESPONDING #( keys )
    RESULT DATA(bookings).

  READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
    ENTITY Booking BY \_Travel
    FROM CORRESPONDING #( bookings )
    LINK DATA(travel_booking_links).

  LOOP AT bookings INTO DATA(booking).
    APPEND VALUE #( %tky          = booking-%tky
                   %state_area   = 'VALIDATE_FLIGHTPRICE'
                   ) TO reported-booking.
    " Raise message for flight price < 0
    IF booking-FlightPrice < 0.
      APPEND VALUE #( %tky          = booking-%tky ) TO failed-booking.
      APPEND VALUE #( %tky          = booking-%tky
                     %state_area   = 'VALIDATE_FLIGHTPRICE'
                     %msg          = NEW /dmo/cm_flight_messages(
                                     textid      = /dmo/cm_flight_messages=>fl
                                     severity    = if_abap_behv_message=>sever
                                     %element-FlightPrice = if_abap_behv=>mk-on
                                     %path          = VALUE #( travel-%tky = travel_booking_links[ KEY
                                     ) TO reported-booking.
    
```

```
ENDIF.
ENDLOOP.
```

```
ENDMETHOD.
```

Validation **validateStatus**

Define the validation in the behavior definition and implement it in the behavior implementation class for **Booking**.

Definition

Define a validation on save with trigger operation **create** and trigger field **BookingStatus**.

Sample Code

```
define behavior for /DMO/R_Booking_D alias Booking
...
{ ...
  validation validateStatus on save { create; field BookingStatus; }

... }
```

Via a quick fix, you can generate the method declaration in the behavior pool directly from the behavior definition editor.

Implementation

Validate the booking status by checking if it matches one of the predefined values. Raise an adequate message for the consumer if the value is not valid. The following steps guide you through the implementation.

1. Read all the instances with the imported **keys** into an internal table **bookings**. This table is the basis to check whether messages must be raised.
2. Read the keys of the corresponding parent instances using the association **_Travel** and store the result in the **link** response. This information will be needed for linking booking instances to travel instances in the reported response.
3. Loop over the internal table **bookings**. To avoid duplicate state messages for the consumer, append an empty message to the reported table to clear the state area.
4. If the booking status differs from the predefined statuses, write the transactional key to the failed table and append a corresponding message along with the state area **VALIDATE_STATUS** to the reported table for that instance. The reported table includes the component **%element**. Here you can specify which CDS element is responsible for the state inconsistency. The Fiori Elements UI evaluates this component and highlights the corresponding input field. Use the **travel_booking_links** table populated above to provide the keys of the corresponding travel instances for each booking instance.

The messages for the managed scenario with draft are stored in the message class **/dmo/cm_flight_messages** that is part of the ABAP Flight Reference Scenario.

i *Expand the following code sample to view the source code of Validation **validateStatus**.*

Sample Code

```
METHOD validateStatus.
```

```
  READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
```

```

ENTITY booking
FIELDS ( BookingStatus )
WITH CORRESPONDING #( keys )
RESULT DATA(bookings).

READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
ENTITY Booking BY \_Travel
FROM CORRESPONDING #( bookings )
LINK DATA(travel_booking_links).

LOOP AT bookings INTO DATA(booking).
APPEND VALUE #( %tky          = booking-%tky
                %state_area   = 'VALIDATE_STATUS'
                ) TO reported-booking.
CASE booking-BookingStatus.
WHEN 'N'. " New
WHEN 'X'. " Canceled
WHEN 'B'. " Booked

WHEN OTHERS.
  APPEND VALUE #( %tky = booking-%tky ) TO failed-booking.
  APPEND VALUE #( %tky = booking-%tky
                  %state_area   = 'VALIDATE_STATUS'
                  %msg          = NEW /dmo/cm_flight_messages(
                                textid   = /dmo/cm_flight_messages=>st
                                status    = booking-BookingStatus
                                severity = if_abap_behv_message=>sever
                                %element-BookingStatus = if_abap_behv=>mk-on
                                %path              = VALUE #( travel-%tky = travel_booking_links[
  ) TO reported-booking.
ENDCASE.
ENDLOOP.
ENDMETHOD.

```

Validating Values for the Booking Supplement Entity

For the managed scenario with draft, define and implement validations on the booking supplement entity for value consistency checks.

Validation `validateSupplement`

Define the validation in the behavior definition and implement it in the behavior implementation class for [BookingSupplement](#).

Definition

Define a validation on save with trigger operation `create` and trigger field `SupplementID`. Since a booking supplement instance always needs a supplement, define the field `SupplementID` as mandatory.

Sample Code

```

define behavior for /DMO/R_BookingSupplement_D alias BookingSupplement
...
{ ...
  field ( mandatory ) SupplementID
  validation validateSupplement on save { create; field SupplementID; }

... }

```

Via a quick fix, you can generate the method declaration in the behavior pool directly from the behavior definition editor.

Implementation

Validate the supplement ID by checking if the provided value is an entry in the supplement database table. Raise adequate messages for the consumer if the value is initial or not valid. The implementation steps are exactly the same as for the validation `ValidateCustomer`. For a detailed step-by-step description, see [Validation validateCustomer](#).

To fill the `%path` component in the reported table, you have to provide the keys of all parent instances. Therefore you have to execute a read by association to every entity in the business object structure. In this case, to `Booking` and to `BookingSupplement`.

i Expand the following code sample to view the source code of Validation `validateSupplement`.

Sample Code

```
METHOD validateSupplement.

  READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
    ENTITY BookingSupplement
      FIELDS ( SupplementID )
      WITH CORRESPONDING #( keys )
    RESULT DATA(bookingsupplements)
    FAILED DATA(read_failed).

  failed = CORRESPONDING #( DEEP read_failed ).

  READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
    ENTITY BookingSupplement BY \_Booking
    FROM CORRESPONDING #( bookingsupplements )
    LINK DATA(booksuppl_booking_links).

  READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
    ENTITY BookingSupplement BY \_Travel
    FROM CORRESPONDING #( bookingsupplements )
    LINK DATA(booksuppl_travel_links).

  DATA supplements TYPE SORTED TABLE OF /DMO/SUPPLEMENT WITH UNIQUE KEY supplement_id.

  " Optimization of DB select: extract distinct non-initial customer IDs
  supplements = CORRESPONDING #( bookingsupplements DISCARDING DUPLICATES MAPPING supplement_id
  DELETE supplements WHERE supplement_id IS INITIAL.

  IF supplements IS NOT INITIAL.
    " Check if customer ID exists
    SELECT FROM /DMO/SUPPLEMENT FIELDS supplement_id
      FOR ALL ENTRIES IN @supplements
      WHERE supplement_id = @supplements-supplement_id
    INTO TABLE @DATA(valid_supplements).
  ENDIF.

  LOOP AT bookingsupplements ASSIGNING FIELD-SYMBOL(<bookingsupplement>).

    APPEND VALUE #( %tky      = <bookingsupplement>-%tky
      %state_area = 'VALIDATE_SUPPLEMENT'
    ) TO reported-bookingsupplement.

  IF <bookingsupplement>-SupplementID IS INITIAL.
    APPEND VALUE #( %tky = <bookingsupplement>-%tky ) TO failed-bookingsupplement.

    APPEND VALUE #( %tky      = <bookingsupplement>-%tky
      %state_area = 'VALIDATE_SUPPLEMENT'
      %msg        = NEW /dmo/cm_flight_messages(
        textid = /dmo/cm_flight_message
        severity = if_abap_behv_message=
      %path      = VALUE #( booking-%tky = booksuppl_booking_links[
        travel-%tky = booksuppl_travel_links[
      %element-SupplementID = if_abap_behv=>mk-on
```

) T0 reported-bookingsupplement.

```
ELSEIF <bookingsupplement>-SupplementID IS NOT INITIAL AND NOT line_exists( valid_suppleme
  APPEND VALUE #( %tky = <bookingsupplement>-%tky ) T0 failed-bookingsupplement.
```

```
  APPEND VALUE #( %tky          = <bookingsupplement>-%tky
                  %state_area   = 'VALIDATE_SUPPLEMENT'
                  %msg          = NEW /dmo/cm_flight_messages(
                                textid = /dmo/cm_flight_message
                                severity = if_abap_behv_message=
                  %path          = VALUE #( booking-%tky = booksuppl_booking_links[
                                travel-%tky = booksuppl_travel_links[
                  %element-SupplementID = if_abap_behv=>mk-on
                  ) T0 reported-bookingsupplement.
```

```
ENDIF.
```

```
ENDLOOP.
```

```
ENDMETHOD.
```

Validation **validateCurrencyCode**

Define the validation in the behavior definition and implement it in the behavior implementation class for [BookingSupplement](#).

Definition

Define a validation on save with trigger operation `create` and trigger field `CurrencyCode`. Since there must always be currency code, define the field `CurrencyCode` as mandatory.

Sample Code

```
define behavior for /DMO/R_BookingSupplement_D alias BookingSupplement
...
{ ...
  field ( mandatory ) SupplementID
  validation validateSupplement on save { create; field SupplementID; }
... }
```

Via a quick fix, you can generate the method declaration in the behavior pool directly from the behavior definition editor.

Implementation

Validate the currency code by checking if the provided value is an entry in I. Raise adequate messages for the consumer if the value is initial or not valid. The implementation steps are exactly the same as for the validation `ValidateCustomer`. For a detailed step-by-step description, see [Validation validateCustomer](#).

To fill the `%path` component in the reported table, you have to provide the keys of all parent instances. Therefore you have to execute a read by association to every entity in the business object structure. In this case, to [Booking](#) and to [BookingSupplement](#).

i *Expand the following code sample to view the source code of Validation **validateSupplement**.*

Sample Code

```
METHOD validateCurrencyCode.
  READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
  ENTITY BookingSupplement
```

```

        FIELDS ( currencycode )
        WITH CORRESPONDING #( keys )
        RESULT DATA(booking_supplements).

READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
  ENTITY BookingSupplement BY \_Booking
  FROM CORRESPONDING #( booking_supplements )
  LINK DATA(booksuppl_booking_links).

READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
  ENTITY BookingSupplement BY \_Travel
  FROM CORRESPONDING #( booking_supplements )
  LINK DATA(booksuppl_travel_links).

DATA: currencies TYPE SORTED TABLE OF I_Currency WITH UNIQUE KEY currency.

currencies = CORRESPONDING #( booking_supplements DISCARDING DUPLICATES MAPPING currency = c
DELETE currencies WHERE currency IS INITIAL.

IF currencies IS NOT INITIAL.
  SELECT FROM I_Currency FIELDS currency
  FOR ALL ENTRIES IN @currencies
  WHERE currency = @currencies-currency
  INTO TABLE @DATA(currency_db).
ENDIF.

LOOP AT booking_supplements INTO DATA(booking_supplement).
  APPEND VALUE #( %tky          = booking_supplement-%tky
                  %state_area   = 'VALIDATE_CURRENCYCODE'
                  ) TO reported-bookingsupplement.
  IF booking_supplement-currencycode IS INITIAL.
    " Raise message for empty Currency
    APPEND VALUE #( %tky          = booking_supplement-%tky ) TO failed-bookingsupp
    APPEND VALUE #( %tky          = booking_supplement-%tky
                  %state_area   = 'VALIDATE_CURRENCYCODE'
                  %msg          = NEW /dmo/cm_flight_messages(
                                textid      = /dmo/cm_flight_messages=>cu
                                severity    = if_abap_behv_message=>severi
                                %path       = VALUE #( booking-%tky = booksuppl_booking_links[
                                                        travel-%tky = booksuppl_travel_links[
                                                        %element-currencycode = if_abap_behv=>mk-on
                                                        ) TO reported-bookingsupplement.
    ELSEIF NOT line_exists( currency_db[ currency = booking_supplement-currencycode ] ).
      " Raise message for not existing Currency
      APPEND VALUE #( %tky          = booking_supplement-%tky ) TO failed-bookingsupp
      APPEND VALUE #( %tky          = booking_supplement-%tky
                    %state_area   = 'VALIDATE_CURRENCYCODE'
                    %msg          = NEW /dmo/cm_flight_messages(
                                textid      = /dmo/cm_flight_messages=>cu
                                severity    = if_abap_behv_message=>severi
                                %path       = VALUE #( booking-%tky = booksuppl_booking_links[
                                                        travel-%tky = booksuppl_travel_links[
                                                        %element-currencycode = if_abap_behv=>mk-on
                                                        ) TO reported-bookingsupplement.
  ENDIF.
ENDLOOP.
ENDMETHOD.

```

Validation validatePrice

Define the validation in the behavior definition and implement it in the behavior implementation class for **BookingSupplement**.

Definition

Define a validation on save with trigger operation create and trigger field BookSupplPrice.

Sample Code

```

define behavior for /DMO/R_BookingSupplement_D alias BookingSupplement
...
{ ...
  validation validatePrice on save { create; field BookSupplPrice; }

... }

```

Via a quick fix, you can generate the method declaration in the behavior pool directly from the behavior definition editor.

Implementation

Validate the price by checking if the entered value is less than 0. Raise an adequate message for the consumer if the value is not valid. The following steps guide you through the implementation.

1. Read all the instances with the imported keys into an internal table `booking_supplements`. This table is the basis to check whether messages must be raised.
2. Read the keys of the corresponding parent and root instances using the associations `_Booking` and `_Travel` and store the result in the `link` response respectively. This information will be needed for linking supplement instances to booking and travel instances in the reported response.
3. Loop over the internal table `booking_supplements`. To avoid duplicate state messages for the consumer, append an empty message to the reported table to clear the state area.
4. If the flight price is less than 0, write the transactional key to the failed table and append a corresponding message along with the state area `VALIDATE_PRICE` to the reported table for that instance. The reported table includes the component `%element`. Here you can specify which CDS element is responsible for the state inconsistency. The Fiori Elements UI evaluates this component and highlights the corresponding input field. Use the tables `booksuppl_booking_links` and `booksuppl_travel_links` populated above to provide the keys of the corresponding booking and travel instances for each supplement instance.

The messages for the managed scenario with draft are stored in the message class `/dmo/cm_flight_messages` that is part of the ABAP Flight Reference Scenario.

i Expand the following code sample to view the source code of Validation `validatePrice`.

Sample Code

```

METHOD validatePrice.

  READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
    ENTITY BookingSupplement
      FIELDS ( BookSupplPrice )
      WITH CORRESPONDING #( keys )
    RESULT DATA(booking_supplements).

  READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
    ENTITY BookingSupplement BY \_Booking
      FROM CORRESPONDING #( booking_supplements )
    LINK DATA(booksuppl_booking_links).

  READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
    ENTITY BookingSupplement BY \_Travel
      FROM CORRESPONDING #( booking_supplements )
    LINK DATA(booksuppl_travel_links).

  LOOP AT booking_supplements INTO DATA(booking_supplement).
    APPEND VALUE #( %tky          = booking_supplement-%tky
                    %state_area   = 'VALIDATE_PRICE'
                    ) TO reported-bookingsupplement.

```

```

" Raise message for Supplement Price < 0
IF booking_supplement-BookSupplPrice < 0.
  APPEND VALUE #( %tky                                = booking_supplement-%tky ) TO failed-bookingsupp
  APPEND VALUE #( %tky                                = booking_supplement-%tky
                  %state_area                          = 'VALIDATE_PRICE'
                  %msg                                  = NEW /dmo/cm_flight_messages(
                                                            textid      = /dmo/cm_flight_messages=>sup
                                                            severity    = if_abap_behv_message=>severi
                  %path                                = VALUE #( booking-%tky = booksuppl_booking_links[
                                                            travel-%tky  = booksuppl_travel_links[
                  %element-BookSupplPrice = if_abap_behv=>mk-on
                  ) TO reported-bookingsupplement.
ENDIF.
ENDLOOP.

ENDMETHOD.

```

Developing RAP Side Effects

For the managed scenario with draft, add RAP side effects to model interdependencies between BO properties for a reload of the affected properties.

Context and Procedure

With RAP side effects, you can trigger data, permission, or message updates based on data changes in UI scenarios with draft-enabled BOs. For more detailed information, see [RAP Side Effects](#).

For this demo scenario, define RAP side effects on all three BO entities. Side effects are defined in the base behavior definition in the entity behavior body.

- For the **travel entity**, the following RAP side effects are relevant:
 - Whenever the field **BookingFee** is changed, the recalculation of the field **TotalPrice** must be triggered to adjust the value with the new amount for the booking fee field.
 - Whenever the field **AgencyID** is changed, the user must receive feedback if the used value is valid for this field. This is validated by the validation **validateAgency**. To be able to define a trigger point for the validation, it must firstly be defined in a determine action.
 - Whenever the field **CustomerID** is changed, the user must receive feedback if the used value is valid for this field. This is validated by the validation **validateCustomer**. To be able to define a trigger point for the validation, it must firstly be defined in a determine action.
 - Whenever the fields **BeginDate** and **EndDate** are changed, the user must receive feedback if the used values are valid for these fields. This is validated by the validation **validateDates**. To be able to define a trigger point for the validation, it must firstly be defined in a determine action.
- For the **booking entity**, the following RAP side effects are described:
 - Whenever the field **FlightPrice** is changed, the recalculation of the field **TotalPrice** of the travel entity must be triggered to adjust the value with the new amount of the flight price field.
- For the **booking supplement entity**, the following RAP side effects are described.
 - Whenever the field **BookSupplPrice** is changed, the recalculation of the field **TotalPrice** of the travel entity must be triggered to adjust the value with the new amount for the booking supplement price field.

- Whenever the field `BookSupplPrice` is changed, the validation `validatePrice` is executed which checks whether the entered price is less than 0.

For a detailed description on how to implement the described behavior for the relevant entity, check the implementation steps in the following topics.

[Defining RAP Side Effects for the Travel Entity](#)

For the managed scenario with draft, define RAP side effects on the travel entity for immediate feedback on the Fiori Elements UI.

[Defining RAP Side Effects for the Booking Entity](#)

For the managed scenario with draft, define RAP side effects on the booking entity for immediate feedback on the Fiori Elements UI.

[Defining RAP Side Effects for the Booking Supplement Entity](#)

For the managed scenario with draft, define RAP side effects on the booking entity for immediate feedback on the Fiori Elements UI.

Defining RAP Side Effects for the Travel Entity

For the managed scenario with draft, define RAP side effects on the travel entity for immediate feedback on the Fiori Elements UI.

Side effects are defined in the base behavior definition. To be exposed for a UI service, they must be reused in the projection behavior definition like any other BO behavior, see [Projecting the Draft BO Behavior](#).

RAP Side Effects for Total Price Recalculation

The RAP side effect triggers the recalculation of the `Total Price` on every change of the `Booking Fee`.

Define the RAP side effect with field trigger and field target in the behavior body of the behavior definition.

Sample Code

```
define behavior for /DMO/R_Travel_D alias Travel
...
  side effects
  { field BookingFee affects field TotalPrice; }
```

Whenever the `Booking Fee` is changed, the `Total Price` is reloaded and the recalculated total price is displayed on the UI. The recalculation in the backend is triggered by the field change.

RAP Side Effect to Trigger the Validation of the AgencyID

The RAP side effect triggers the validation that checks if the `AgencyID` is valid and reloads the respective messages. To be able to define a trigger point for the RAP side effect, it must firstly be defined in a determine action. The determine action has the same authorization as update. Therefore (authorization: update) is defined.

i Note

The keyword `side effects` can only appear once in the behavior definition body. You have to define all RAP side effects for one entity together.

Sample Code

```

define behavior for /DMO/R_Travel_D alias Travel
...
{ ...
    determine action ( authorization : update ) validateAgencyID { validation validateAgency; }

    side effects { field BookingFee affects field TotalPrice;
                  determine action ( authorization : update ) validateAgencyID executed on field
    ... }

```

Whenever the AgencyID is changed the determine action is triggered, which calls the validation and the messages are reloaded. The determine action has the same authorization as update. Therefore (authorization: update) is defined.

RAP Side Effect to Trigger the Validation of the CustomerID

The RAP side effect triggers the validation that checks if the CustomerID is valid and reloads the respective messages. To be able to define a trigger point for the RAP side effect, it must firstly be defined in a determine action. The determine action has the same authorization as update. Therefore (authorization: update) is defined.

Note

The keyword `side effects` can only appear once in the behavior definition body. You have to define all side effects for one entity together.

Sample Code

```

define behavior for /DMO/R_Travel_D alias Travel
...
{ ...
    determine action ( authorization : update ) validateAgencyID { validation validateAgency; }
    determine action ( authorization : update ) validateCustomerID { validation validateCustomer;

    side effects { field BookingFee affects field TotalPrice;
                  determine action ( authorization : update ) validateAgencyID   executed on field
                  determine action ( authorization : update ) validateCustomerID executed on field
    }
    ... }

```

Whenever the CustomerID is changed the determine action is triggered, which calls the validation and the messages are reloaded.

RAP Side Effect to Trigger the Validation of the Dates

The RAP side effect triggers the validation that checks if the BeginDate and the EndDate is valid and reloads the respective messages. To be able to define a trigger point for the RAP side effect, it must firstly be defined in a determine action.

Note

The keyword `side effects` can only appear once in the behavior definition body. You have to define all RAP side effects for one entity together.

≡ Sample Code

```

define behavior for /DMO/R_Travel_D alias Travel
...
{ ...
  determine action ( authorization : update ) validateAgencyID { validation validateAgency; }
  determine action ( authorization : update ) validateCustomerID { validation validateCustomer; }
  determine action ( authorization : update ) validateDateRange { validation validateDates; }

  side effects { field BookingFee affects field TotalPrice;
    determine action ( authorization : update ) validateAgencyID executed on field
    determine action ( authorization : update ) validateCustomerID executed on field
    determine action ( authorization : update ) validateDateRange executed on field

  }
... }

```

Whenever the **BeginDate** or the **EndDate** is changed the determine action is triggered, which calls the validation and the messages are reloaded.

RAP Side Effect to Trigger the Validation of the Booking Fee

The RAP side effect triggers the validation that checks if the **BookingFee** is valid and reloads the respective messages. To be able to define a trigger point for the RAP side effect, it must firstly be defined in a determine action. The determine action has the same authorization as update. Therefore (authorization: update) is defined.

i Note

The keyword **side effects** can only appear once in the behavior definition body. You have to define all RAP side effects for one entity together.

≡ Sample Code

```

define behavior for /DMO/R_Travel_D alias Travel
...
{ ...
  determine action ( authorization : update ) validateAgencyID { validation validateAgency; }
  determine action ( authorization : update ) validateCustomerID { validation validateCustomer; }
  determine action ( authorization : update ) validateDateRange { validation validateDates; }
  determine action ( authorization : update ) validateBookingCharge {

  side effects { field BookingFee affects field TotalPrice;
    determine action ( authorization : update ) validateAgencyID executed on field
    determine action ( authorization : update ) validateCustomerID executed on field
    determine action ( authorization : update ) validateDateRange executed on field
    determine action ( authorization : update ) validateBookingCharge executed on field
  }
... }

```

```
}
... }
```

Whenever the field `BookingFee` is changed the `determine` action is triggered, which calls the validation and the messages are reloaded.

Defining RAP Side Effects for the Booking Entity

For the managed scenario with draft, define RAP side effects on the booking entity for immediate feedback on the Fiori Elements UI.

RAP side effects are defined in the base behavior definition. To be exposed for a UI service, they must be reused in the projection behavior definition like any other BO behavior, see [Projecting the Draft BO Behavior](#).

RAP Side Effects for Total Price Recalculation

The RAP side effect triggers the recalculation of the `Total Price` on every change of the `Flight Price`.

Define the RAP side effect with field trigger and field target in the behavior body of the behavior definition on the booking entity.

Sample Code

```
define behavior for /DMO/R_Booking_D alias Travel
...
{ ...
  side effects { field FlightPrice affects field _Travel.TotalPrice; }
... }
```

Whenever the `Flight Price` is changed on the booking entity, the `Total Price` on the travel entity is reloaded and the recalculated total price is displayed on the UI. The recalculation in the backend is triggered by the field change

RAP Side Effect to Trigger the Validation of the `flight price`

The RAP side effect triggers the validation that checks if the `flight price` is valid and reloads the respective messages. To be able to define a trigger point for the RAP side effect, it must firstly be defined in a `determine` action. The `determine` action has the same authorization as `update`. Therefore (authorization: update) is defined.

i Note

The keyword `side effects` can only appear once in the behavior definition body. You have to define all RAP side effects for one entity together.

Sample Code

```
define behavior for /DMO/R_Booking_D alias Travel
...
{ ...
  determine action ( authorization : update ) validateFlightCosts { validation validateFlightPri

  side effects
```

```

{ field FlightPrice affects field _Travel.TotalPrice;
  determine action ( authorization : update ) validateFlightCosts executed on field FlightPric
... }

```

Whenever the `FlightPrice` is changed the `determine` action is triggered, which calls the validation and the messages are reloaded.

Defining RAP Side Effects for the Booking Supplement Entity

For the managed scenario with draft, define RAP side effects on the booking entity for immediate feedback on the Fiori Elements UI.

RAP side effects are defined in the base behavior definition. To be exposed for a UI service, they must be reused in the projection behavior definition like any other BO behavior, see [Projecting the Draft BO Behavior](#).

RAP Side Effect for Total Price Recalculation

The RAP side effect triggers the recalculation of the `Total Price` on every change of the `BookSupplPrice`.

Define the RAP side effect with field trigger and field target in the behavior body of the behavior definition on the booking booking supplements.

Sample Code

```

define behavior for /DM0/R_BookingSupplement_D alias BookingSupplement
...
{ ...
  side effects { field BookSupplPrice affects field _Travel.TotalPrice; }
... }

```

Whenever the `BookSupplPrice` is changed on the booking supplement entity, the `Total Price` on the travel entity is reloaded and the recalculated total price is displayed on the UI. The recalculation in the backend is triggered by the field change

RAP Side Effect to Trigger the Validation of the `BookSupplPrice`

The RAP side effect triggers the validation that checks if the `BookSupplPrice` is valid and reloads the respective messages. To be able to define a trigger point for the RAP side effect, it must firstly be defined in a `determine` action. The `determine` action has the same authorization as `update`. There fore (`authorization: update`) is defined.

i Note

The keyword `side effects` can only appear once in the behavior definition body. You have to define all RAP side effects for one entity together.

Sample Code

```

define behavior for /DM0/R_BookingSupplement_D alias BookingSupplement
...
{ ...
  determine action ( authorization : update ) validateSupplementPrice { validation validatePrice

```

```

side effects
{ field BookSupplPrice affects field _Travel.TotalPrice;
  determine action ( authorization : update ) validateSupplementPrice executed on field BookSupp
... }

```

Whenever the BookSupplPrice is changed the determine action is triggered, which calls the validation and the messages are reloaded.

Exposing the Managed Business Object for a UI Business Service

In the previous chapters, you have created a managed business object from scratch. To expose it as a business service the following steps are relevant:

1. [Projecting the Managed BO for Non-Draft Exposure](#)
2. [Defining and Publishing a UI Service](#)

Projecting the Managed BO for Non-Draft Exposure

For the managed scenario with draft, create a projection layer for the managed BO.

Context and Procedure

Whereas the business object that we have created so far is service agnostic, the business object projection is an approach to define a subset of this business object as service-specific for the UI use case. In this scenario, we want to publish a UI service, so UI-specifics are defined in the BO projection. For more information about the BO projection layer, see [Business Object Projection](#).

For the travel demo scenario, define the data model and the behavior projection:

- For the **data model projection**, the following steps are relevant:

- Create **CDS projection views**.

In the CDS projection views define UI-service-specific features, such as search capabilities, text associations, and value helps.

- Create **metadata extensions**.

Metadata extensions are primarily used to define the annotations that are relevant for the UI design. They are outsourced in metadata extension to obtain an easy approach for layering.

For a detailed description on how to define data model projection features, see [Projecting the BO Data Model](#).

- For the **behavior projection**, the following steps are relevant:

- Create a **projection behavior definition**.

For a detailed description on behavior projection features, see [Projecting the BO Behavior](#).

Projecting the BO Data Model

For the managed scenario with draft, create and implement CDS projection views and metadata extensions for the BO entities.

Context and Procedure

Create data definition for projection views for all three BO entities **Travel**, **Booking**, and **Booking Supplement**. In ADT, use the context menu on the underlying CDS view entity to open the wizard for creating CDS data definition.

Select the template **Define Projection View**.

i Note

The draft scenario in the ABAP Flight Reference Scenario uses the suffix `_D`. Since the CDS projection views only represent the active UI service, we use the marker `_A` before the suffix to distinguish the active projection layer from the projection layer that includes draft. In addition, projection views carry the prefix `C_` as they represent the consumption layer. For more information, see [Naming Conventions for Development Objects](#).

For each projection view, create a metadata extension. In ADT, right-click on the data definition to open the creation wizard for metadata extensions.

i Note

According to the virtual data model (VDM) guidelines, the metadata extension use the same name as the related CDS view.

For more information about the syntax in projection views, see [Define View Entity as Projection \(ABAP Keyword Documentation\)](#).

Projection View `/DMO/C_Travel_A_D`

The projection view is a projection on `/DMO/R_Travel_D`. Include the keyword `root` in the projection view.

Use all elements of the underlying BO travel entity. You do not need to use the administrative data in the projection view, except for the element representing the ETag master.

Enable the usage of a metadata extension with the annotation `@Metadata.allowExtension:true`.

Enable search capabilities for the travel entity, with the `@Search` annotations on the entity header and on designated elements. For more information, see [Enabling Text and Fuzzy Searches in SAP Fiori Apps](#).

To indicate that `TravelID` is the semantic identifier of the BO-entity, use the annotation `@ObjectModel.semanticKey: [ 'TravelID' ]` in the header of the projection view.

To retrieve the long texts of the ID elements `CustomerID`, `AgencyID`, and `OverallStatus` denormalize the text elements from the associated text provider views into the projection view. For more information, see [Getting Language-Dependent Text in Projection Views](#).

Define value helps for the elements `CustomerID`, `AgencyID`, `CurrencyCode`, and `OverallStatus`. For more information, see [Adding Value Help](#).

As soon as all projection views of the composition structure of the business object are available, redirect the composition to child to the projection view of the booking entity.

i Expand the following code sample to view the source code of Projection View `/DMO/C_Travel_A_D`.

≡ Sample Code

```
@EndUserText.label: 'Travel Projection View for Draft RefScen'
@AccessControl.authorizationCheck: #NOT_REQUIRED
```

```

@Metadata.allowExtensions: true
@Search.searchable: true

define root view entity /DMO/C_Travel_A_D
  provider contract transactional_query
  as projection on /DMO/R_Travel_D

{
  key TravelUUID,

  @Search.defaultSearchElement: true
  TravelID,

  @Search.defaultSearchElement: true
  @ObjectModel.text.element: ['AgencyName']
  @Consumption.valueHelpDefinition: [{ entity : {name: '/DMO/I_Agency', element: 'AgencyID'
  AgencyID,
  _Agency.Name          as AgencyName,

  @Search.defaultSearchElement: true
  @ObjectModel.text.element: ['CustomerName']
  @Consumption.valueHelpDefinition: [{ entity : {name: '/DMO/I_Customer', element: 'CustomerID',
  CustomerID,
  _Customer.LastName as CustomerName,

  BeginDate,

  EndDate,

  BookingFee,

  TotalPrice,

  @Consumption.valueHelpDefinition: [{entity: {name: 'I_Currency', element: 'Currency' }}]
  CurrencyCode,

  Description,

  @ObjectModel.text.element: ['OverallStatusText']
  @Consumption.valueHelpDefinition: [{ entity: {name: '/DMO/I_Overall_Status_VH', element:
  OverallStatus,
  _OverallStatus._Text.Text as OverallStatusText : localized,

  LocalLastChangedAt,
  /* Associations */
  _Agency,
  _Booking : redirected to composition child /DMO/C_Booking_A_D,
  _Currency,
  _OverallStatus,
  _Customer
}

```

Metadata Extension /DMO/C_Travel_A_D

Define the metadata extension as extension of the CDS projection view /DMO/C_Travel_A_D.

Define the layer #CORE.

Use all the @UI annotations to design the user interface for the travel app. For more information, see [Defining CDS Annotations for Metadata-Driven UIs](#).

To trigger actions via action buttons on the UI, use @UI annotations for data action on both the list report page and the object page. It is not important which element is annotated with this annotation, the action button always appears on the upper right corner on the UI. To enable multi-inline editing on the list report page, annotate the action Deduct_Discount with

@UI.lineItem.invocationGrouping: #CHANGE_SET. The action is then processed for all selected instances in once change set.

i Expand the following code sample to view the source code of Metadata Extension /DMO/C_Travel_A_D.

Sample Code

```
@Metadata.layer: #CORE

@UI: { headerInfo: { typeName: 'Travel',
                    typeNamePlural: 'Travels',
                    title: { type: #STANDARD, value: 'TravelID' } },
      presentationVariant: [{ sortOrder: [{ by: 'TravelID', direction: #DESC }] }] }

annotate entity /DMO/C_Travel_A_D
with
{
  @UI.facet: [{ id:      'Travel',
                purpose:  #STANDARD,
                type:     #IDENTIFICATION_REFERENCE,
                label:     'Travel',
                position:  10 },
              { id:      'Booking',
                purpose:  #STANDARD,
                type:     #LINEITEM_REFERENCE,
                label:     'Booking',
                position:  20,
                targetElement: '_Booking' }]

  @UI.hidden: true
  TravelUUID;

  @UI: { lineItem:      [ { position: 10, importance: #HIGH },
                        { type: #FOR_ACTION, dataAction: 'acceptTravel', label: 'Accept Trave
                        { type: #FOR_ACTION, dataAction: 'rejectTravel', label: 'Reject Trave
                        { type: #FOR_ACTION, dataAction: 'deductDiscount', label: 'Deduct Dis
      identification: [ { position: 10, label: 'Travel ID' },
                        { type: #FOR_ACTION, dataAction: 'acceptTravel', label: 'Accept Trave
                        { type: #FOR_ACTION, dataAction: 'rejectTravel', label: 'Reject Trave
                        { type: #FOR_ACTION, dataAction: 'deductDiscount', label: 'Deduct Dis
      selectionField: [ { position: 10 } ] }
  TravelID;

  @UI: { lineItem:      [ { position: 20, importance: #HIGH } ] ,
      identification: [ { position: 20 } ],
      selectionField: [ { position: 20 } ] }
  AgencyID;

  @UI: { lineItem:      [ { position: 30, importance: #HIGH } ] ,
      identification: [ { position: 30 } ],
      selectionField: [ { position: 30 } ] }
  CustomerID;

  @UI: { lineItem:      [ { position: 40, importance: #MEDIUM } ] ,
      identification: [ { position: 40 } ] }
  BeginDate;

  @UI: { lineItem:      [ { position: 50, importance: #MEDIUM } ] ,
      identification: [ { position: 50 } ] }
  EndDate;

  @UI: { lineItem:      [ { position: 60, importance: #MEDIUM } ] ,
      identification: [ { position: 60 } ] }
  BookingFee;

  @UI: { lineItem:      [ { position: 70, importance: #MEDIUM } ] ,
      identification: [ { position: 70 } ] }
  TotalPrice;

  // CurrencyCode;

  @UI: { lineItem:      [ { position: 90, importance: #MEDIUM } ] ,
      identification: [ { position: 90 } ] }
```

Description;

```
@UI: { lineItem:      [ { position: 99, importance: #HIGH } ] ,
      identification: [ { position: 99, label: 'Status' } ],
      selectionField: [ { position: 40 } ] ,
      textArrangement: #TEXT_ONLY }
OverallStatus;
@UI.hidden: true
OverallStatusText;

@UI.hidden: true
LocalLastChangedAt;

}
```

Projection View /DMO/C_Booking_A_D

The projection view is a projection on /DMO/R_Booking_D.

Use all elements of the underlying BO travel entity, in particular the element representing the ETag master.

Enable the usage of a metadata extension with the annotation `@Metadata.allowExtension:true`.

Enable search capabilities for the booking entity, with the `@Search` annotations. For more information, see [Enabling Text and Fuzzy Searches in SAP Fiori Apps](#).

To retrieve the long texts of the ID elements CustomerID, CarrierID, and BookingStatus denormalize the text elements from the associated text provider views into the projection view. For more information, see [Getting Language-Dependent Text in Projection Views](#).

Define value helps for the elements CustomerID, AirlineID, ConnectionID, FlightDate, CurrencyCode, and BookingStatus. For more information, see [Adding Value Help](#).

As soon as all projection views of the composition structure of the business object are available, redirect the composition to child to the projection view of the booking entity and the association to the root entity to the respective projection view.

i Expand the following code sample to view the source code of Projection View /DMO/C_Booking_A_D.

Sample Code

```
@EndUserText.label: 'Booking Proj View for Draft RefScen'
@AccessControl.authorizationCheck: #NOT_REQUIRED

@Metadata.allowExtensions: true
@Search.searchable: true
define view entity /DMO/C_Booking_A_D
  as projection on /DMO/R_Booking_D
{
  key BookingUUID,

  TravelUUID,

  @Search.defaultSearchElement: true
  BookingID,

  BookingDate,

  @ObjectModel.text.element: ['CustomerName']
  @Search.defaultSearchElement: true
  @Consumption.valueHelpDefinition: [{entity: {name: '/DMO/I_Customer', element: 'CustomerI
  CustomerID,
  _Customer.LastName as CustomerName,
```

```

@ObjectModel.text.element: ['CarrierName']
@Consumption.valueHelpDefinition: [{entity: {name: '/DMO/I_Carrier', element: 'AirlineID'
AirlineID,
_Carrier.Name      as CarrierName,

@Consumption.valueHelpDefinition: [ {entity: {name: '/DMO/I_Flight', element: 'Connection
additionalBinding: [ { localElement: 'FlightDate', element: 'FlightDate'}
{ localElement: 'AirlineID', element: 'AirlineID'},
{ localElement: 'FlightPrice', element: 'Price', usag
{ localElement: 'CurrencyCode', element: 'CurrencyCode

ConnectionID,

FlightDate,

@Consumption.valueHelpDefinition: [ {entity: {name: '/DMO/I_Flight', element: 'Connection
additionalBinding: [ { localElement: 'FlightDate', element: 'FlightDate'}
{ localElement: 'AirlineID', element: 'AirlineID'},
{ localElement: 'FlightPrice', element: 'Price', usag
{ localElement: 'CurrencyCode', element: 'CurrencyCode

FlightPrice,

@Consumption.valueHelpDefinition: [{entity: {name: 'I_Currency', element: 'Currency' }}]
CurrencyCode,

@ObjectModel.text.element: ['BookingStatusText']
@Consumption.valueHelpDefinition: [{entity: {name: '/DMO/I_Booking_Status_VH', element:
BookingStatus,
_BookingStatus._Text.Text as BookingStatusText: localized,

LocalLastChangedAt,

/* Associations */
_BookingSupplement: redirected to composition child /DMO/C_BookingSupplement_A_D,
_BookingStatus,
_Carrier,
_Connection,
_Customer,
_Travel: redirected to parent /DMO/C_Travel_A_D
}

```

Metadata Extension /DMO/C_Booking_A_D

Define the metadata extension as extension of the CDS projection view /DMO/C_Booking_A_D.

Define the layer #CORE.

Use all the @UI annotations to design the user interface for the travel app. For more information, see [Defining CDS Annotations for Metadata-Driven UIs](#).

i Expand the following code sample to view the source code of Metadata Extension /DMO/C_Booking_A_D.

Sample Code

```

@Metadata.layer: #CORE

@UI: { headerInfo: { typeName: 'Booking',
                    typeNamePlural: 'Bookings',
                    title: { type: #STANDARD, value: 'BookingID' } } }

annotate entity /DMO/C_Booking_A_D with
{
  @UI.facet: [ { id:
                purpose:
                type:
                label:
                'Booking',
                #STANDARD,
                #IDENTIFICATION_REFERENCE,
                'Booking',

```

```

        position:      10 },
    { id:              'BookingSupplement',
      purpose:         #STANDARD,
      type:            #LINEITEM_REFERENCE,
      label:           'Booking Supplement',
      position:        20,
      targetElement:   '_BookingSupplement'} ]
@UI.hidden: true
BookingUUID;

@UI.hidden: true
TravelUUID;

@UI: { lineItem:      [ { position: 20, importance: #HIGH } ],
      identification: [ { position: 20 } ] }
BookingID;

@UI: { lineItem:      [ { position: 30, importance: #HIGH } ],
      identification: [ { position: 30 } ] }
BookingDate;

@UI: { lineItem:      [ { position: 40, importance: #HIGH } ],
      identification: [ { position: 40 } ] }
CustomerID;

@UI: { lineItem:      [ { position: 50, importance: #HIGH } ],
      identification: [ { position: 50 } ] }
AirlineID;

@UI: { lineItem:      [ { position: 60, importance: #HIGH } ],
      identification: [ { position: 60 } ] }
ConnectionID;

@UI: { lineItem:      [ { position: 70, importance: #HIGH } ],
      identification: [ { position: 70 } ] }
FlightDate;

@UI: { lineItem:      [ { position: 80, importance: #HIGH } ],
      identification: [ { position: 80 } ] }
FlightPrice;

// CurrencyCode;

@UI: { lineItem:      [ { position: 90, importance: #HIGH } ],
      identification: [ { position: 90 } ],
      textArrangement: #TEXT_ONLY }
BookingStatus;

@UI.hidden: true
BookingStatusText;

@UI.hidden: true
LocalLastChangedAt;
}

```

Projection View /DM0/C_BookingSupplement_A_D

The projection view is a projection on /DM0/R_BookingSupplement_D.

Use all elements of the underlying BO travel entity, in particular the element representing the ETag master.

Enable the usage of a metadata extension with the annotation `@Metadata.allowExtension: true`.

Enable search capabilities for the travel entity, with the `@Search` annotations. For more information, see [Enabling Text and Fuzzy Searches in SAP Fiori Apps](#).

To retrieve the long texts of the ID element `SupplementID`, denormalize the text element from the associated text provider view into the projection view. For more information, see [Getting Language-Dependent Text in Projection Views](#).

Define value helps for the elements `SupplementID` and `CurrencyCode`. For more information, see [Adding Value Help](#).

As soon as all projection views of the composition structure of the business object are available, redirect the association to the parent to the projection view of the booking entity and the association to the root entity to the respective projection view.

i Expand the following code sample to view the source code of Projection View `/DMO/C_BookingSupplement_A_D`.

Sample Code

```
@EndUserText.label: 'BookingSuppl Proj View for Draft RefScen'
@AccessControl.authorizationCheck: #NOT_REQUIRED

@Metadata.allowExtensions: true
@Search.searchable: true

define view entity /DMO/C_BookingSupplement_A_D
  as projection on /DMO/R_BookingSupplement_D
{
  key BookSupplUUID,

  TravelUUID,

  BookingUUID,

  @Search.defaultSearchElement: true
  BookingSupplementID,

  @ObjectModel.text.element: ['SupplementDescription']
  @Consumption.valueHelpDefinition: [ {entity: {name: '/DMO/I_Supplement', element: 'Supple
    additionalBinding: [ { localElement: 'BookSupplPrice', element: 'Price', u
      { localElement: 'CurrencyCode', element: 'CurrencyCode

  SupplementID,
  _SupplementText.Description as SupplementDescription : localized,

  BookSupplPrice,

  @Consumption.valueHelpDefinition: [{entity: {name: 'I_Currency', element: 'Currency' }}]
  CurrencyCode,

  LocalLastChangedAt,

  /* Associations */
  _Booking : redirected to parent /DMO/C_Booking_A_D,
  _Product,
  _SupplementText,
  _Travel : redirected to /DMO/C_Travel_A_D
}
```

Metadata Extension `/DMO/C_BookingSupplement_A_D`

Define the metadata extension as extension of the CDS projection view `/DMO/C_BookingSupplement_A_D`.

Define the layer `#CORE`.

Use all the `@UI` annotations to design the user interface for the travel app. For more information, see [Defining CDS Annotations for Metadata-Driven UIs](#).

i Expand the following code sample to view the source code of Metadata Extension `/DMO/C_BookingSupplement_A_D`.

Sample Code

```
@Metadata.layer: #CORE

@UI: { headerInfo: { typeName:      'Booking Supplement',
                      typeNamePlural: 'Booking Supplements',
                      title:         { type: #STANDARD,
                                      label: 'Booking Supplement',
                                      value: 'BookingSupplementID' } } }

annotate entity /DM0/C_BookingSupplement_A_D
with
{
  @UI.facet: [ { id:          'BookingSupplement',
                  purpose:    #STANDARD,
                  type:       #IDENTIFICATION_REFERENCE,
                  label:      'Booking Supplement',
                  position:    10 } ]

  @UI.hidden:true
  BookSupplUUID;
  @UI.hidden:true
  BookingUUID;
  @UI.hidden:true
  TravelUUID;

  @UI: { lineItem:      [ { position: 10, importance: #HIGH } ],
        identification: [ { position: 10 } ] }
  BookingSupplementID;

  @UI: { lineItem:      [ { position: 20, importance: #HIGH } ],
        identification: [ { position: 20 } ] }
  SupplementID;

  @UI: { lineItem:      [ { position: 30, importance: #HIGH } ],
        identification: [ { position: 30 } ] }
  BookSupplPrice;

  // CurrencyCode;

  @UI.hidden: true
  LocalLastChangedAt;
}
```

Projecting the BO Behavior

For the managed scenario with draft, create a projection behavior definition for defining the projection behavior.

Context and Procedure

Create a projection behavior definition for the travel business object. In ADT, use the context menu of the root projection view to open the wizard for creating behavior definitions. The implementation type is prefilled with **Projection**.

Projection Behavior Definition /DM0/C_Travel_A_D

Use the complete behavior that was defined in the underlying behavior definition for the non-draft BO projection.

i *Expand the following code sample to view the source code of Projection Behavior Definition /DM0/C_Travel_A_D.*

Sample Code

```
projection;
strict(2);
```

```
use side effects;
```

```
define behavior for /DMO/C_Travel_A_D alias Travel
use etag
{
  use create;
  use update;
  use delete;

  use action acceptTravel;
  use action rejectTravel;
  use action deductDiscount;
  use function GetDefaultsFordeductDiscount;

  use association _Booking { create; }
}
```

```
define behavior for /DMO/C_Booking_A_D alias Booking
use etag
{
  use update;
  use delete;

  use association _BookingSupplement { create; }
  use association _Travel;
}
```

```
define behavior for /DMO/C_BookingSupplement_A_D alias BookingSupplement
use etag
{
  use update;
  use delete;

  use association _Travel;
}
```

Defining and Publishing a UI Service

For the managed scenario with draft, define a UI service and bind it to a protocol.

Context and Procedure

To address the managed BO from the ABAP-external world, it must be exposed as a business service to OData. In the service definition, you define which artifacts are included in the service. The service binding defines the protocol with which the service is addressed. In addition, it provides the means to activate the service locally. The Fiori Elements App Preview can be used as soon as the service is activated.

Create a service definition for the non-draft business service.

Based on the service definition, create a service binding. You can choose between OData V2 and V4 protocol to expose a UI service.

i Note

The draft scenario in the ABAP Flight Reference Scenario uses the suffix `_D`. To distinguish the active-only service from the service with draft, we use the addition `_A` before the suffix. For more information, see [Naming Conventions for Development Objects](#).

Service Definition `/DMO/UI_TRAVEL_A_D`

Include the projection views of the business object and all related CDS entities that are relevant for UI consumption into the service definition.

i *Expand the following code sample to view the source code of Service Definition /DMO/UI_TRAVEL_A_D.*

Sample Code

```
@EndUserText.label: 'Travel Draft Scenario(active projection)'
@ObjectModel.leadingEntity.name: '/DMO/C_Travel_A_D'
define service /DMO/UI_TRAVEL_A_D {
  expose /DMO/C_Travel_A_D as Travel;
  expose /DMO/C_Booking_A_D as Booking;
  expose /DMO/C_BookingSupplement_A_D as BookingSupplement;
  expose /DMO/I_Supplement_StdVH as Supplement;
  expose /DMO/I_SupplementCategory_VH as SupplementCategory;
  expose /DMO/I_Customer_StdVH as Passenger;
  expose /DMO/I_Agency_StdVH as TravelAgency;
  expose /DMO/I_Carrier_StdVH as Airline;
  expose /DMO/I_Connection_StdVH as FlightConnection;
  expose /DMO/I_Flight_StdVH as Flight;
  expose /DMO/I_Airport_StdVH as Airport;
  expose /DMO/I_Overall_Status_VH as OverallStatus;
  expose /DMO/I_Booking_Status_VH as BookingStatus;
  expose I_CurrencyStdVH as Currency;
  expose I_CountryVH as Country;
}
```

Service Binding /DMO/UI_TRAVEL_A_D_02

Expose the service as a UI service with OData protocol V2 and publish it locally. You can then use the preview to test the service.

Draft-Enabling the Managed Business Object

You can now draft-enable the managed business object with exclusive draft capabilities with a few additions in the behavior definition.

In draft business objects, the implementation of the business logic must be able to handle request for draft and for active instances. Above all, the implementation must be able to differentiate those requests.

The business object receives the information if a request is aimed at draft or active instances via the draft indicator `%IS_DRAFT`. This draft indicator is a component of the primary key in all incoming requests in draft scenarios. The transactional key `%tky` includes the draft indicator. So in draft scenarios, it is best practice to use `%tky` for referring to the primary key, as the indicator for drafts or non-drafts is already included there. In non-draft scenarios, the transactional key `%tky` comprises just the primary key information.

If you consequently use the derived type `%tky` to refer to the key of instances in your implementation, draft-enabling is just done in the behavior definition with a few additional syntax elements. The draft indicator `%is_draft` is added as a key component to `%tky` in draft scenarios. This means, the implementation receives the information if the request is aimed at draft or active instances. So, you do not have to adjust your implementation in draft scenarios.

The following steps are necessary to draft-enable a managed business object: [Adding Draft Capabilities to the Managed Business Object](#)

Related Information

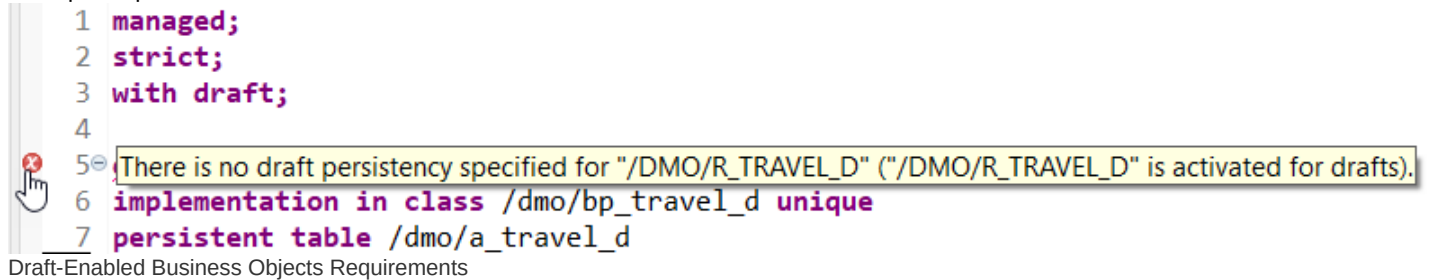
[Exclusive Draft Handling](#)

Adding Draft Capabilities to the Managed Business Object

For the managed scenario with draft, add draft behavior to the behavior definition.

Context and Procedure

In the previous sections of this development guide, you have developed a running business service with a managed business object. Draft capabilities are included to the business service by adding the syntax element `with draft` to the behavior definition. As soon as you use this keyword element in the behavior definition, you are guided through the follow-up steps in the behavior definition by tool support. Syntax warnings and quick fixes are available to adapt the managed business object to the draft prerequisites.



i Note

Business objects with implementation type `unmanaged` are draft-enabled in the same way.

Adaptions in the Behavior Definition

Follow these steps to draft-enable the `travel` business object:

1. Add the syntax element `with draft` to the header of the behavior definition.

i Note

You receive a syntax error that there is no draft persistence available for the draft BO.

2. Create a draft database table for the `Travel` entity by using the quick fix in the behavior definition:

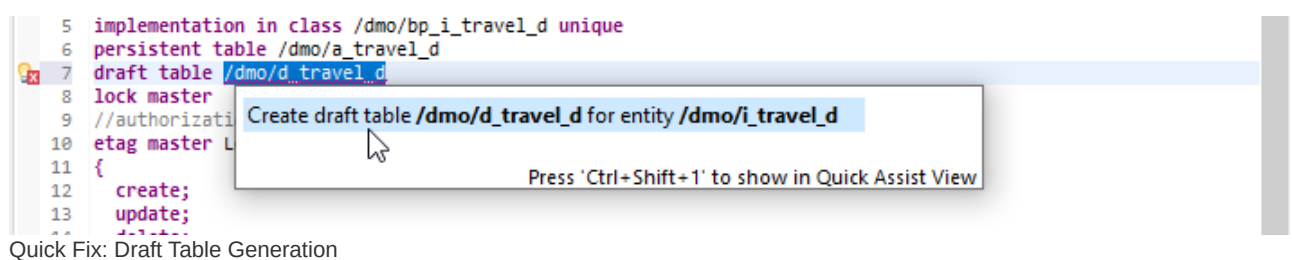
- o Define the draft database table in the behavior definition by using the keyword `draft table` and specify the name for the draft database table.

In the example scenario, we use `/DMO/D_TRAVEL_D`.

i Note

The draft scenario in the ABAP Flight Reference Scenario uses the suffix `_D`. For the draft database tables, we use the prefix `D_` to indicate the draft persistence. For detailed information, see [Naming Conventions for Development Objects](#).

- o Use the quick fix to create a database table with the specified name.



The wizard for creating database tables opens. When you finish the database table generation wizard, the ADT tooling support automatically generates the table elements corresponding to the active persistence. In addition, the draft admin structure include %admin is included in the database table.

For more information about the draft database table, see [Draft Database Table](#).

- o Add the annotation @AbapCatalog.anonymizedWhenDelivered : true to the elements that store user information. (Only relevant for the travel draft table.)

i Expand the following code sample to view the source code of Database Table /DMO/D_TRAVEL_D.

≡ Sample Code

```
@EndUserText.label : 'Draft table for entity /DMO/R_Travel_D'
@AbapCatalog.enhancement.category : #EXTENSIBLE_ANY
@AbapCatalog.tableCategory : #TRANSPARENT
@AbapCatalog.deliveryClass : #A
@AbapCatalog.dataMaintenance : #RESTRICTED
define table /DMO/D_TRAVEL_D {
  key mandt          : mandt not null;
  key traveluuid     : sysuuid_x16 not null;
  travelid           : /dmo/travel_id;
  agencyid           : /dmo/agency_id;
  customerid         : /dmo/customer_id;
  begindate           : /dmo/begin_date;
  enddate            : /dmo/end_date;
  @Semantics.amount.currencyCode : '/dmo/d_travel_d.currencycode'
  bookingfee         : /dmo/booking_fee;
  @Semantics.amount.currencyCode : '/dmo/d_travel_d.currencycode'
  totalprice         : /dmo/total_price;
  currencycode       : /dmo/currency_code;
  description         : /dmo/description;
  overallstatus      : /dmo/overall_status;
  @AbapCatalog.anonymizedWhenDelivered : true
  localcreatedby     : abp_creation_user;
  localcreatedat      : abp_creation_tstmpl;
  @AbapCatalog.anonymizedWhenDelivered : true
  locallastchangedby : abp_locinst_lastchange_user;
  locallastchangedat  : abp_locinst_lastchange_tstmpl;
  "%admin"           : include sych_bdl_draft_admin_inc;
}
```

3. Repeat the creation of draft database tables for the [Booking](#) entity and the [BookingSupplement](#) entity.

In the example scenario, we use the names /DMO/D_BOOKING_D and /DMO/D_BKSUPPL_D.

4. Define a total ETag field for the draft business object.

The total ETag is used to enable optimistic concurrency checks during the transition from draft to active data.

- o Add a total ETag field to the persistent database table of the [Travel](#) entity.

i Expand the following code sample to view the source code of Database Table /DMO/A_TRAVEL_D.

≡ Sample Code

```
@EndUserText.label : 'Active Travel Persistence for Draft Reference Scenario'
@AbapCatalog.enhancementCategory : #NOT_EXTENSIBLE
@AbapCatalog.tableCategory : #TRANSPARENT
@AbapCatalog.deliveryClass : #A
@AbapCatalog.dataMaintenance : #RESTRICTED
define table /DMO/A_TRAVEL_D {
  key client          : abap.clnt not null;
  key travel_uuid     : sysuuid_x16 not null;
  travel_id           : /dmo/travel_id not null;
  ...
}
```

```

@AbapCatalog.anonymizedWhenDelivered : true
local_created_by      : abp_creation_user;
local_created_at      : abp_creation_tstmpl;
@AbapCatalog.anonymizedWhenDelivered : true
local_last_changed_by : abp_locinst_lastchange_user;
local_last_changed_at : abp_locinst_lastchange_tstmpl;
last_changed_at       : abp_lastchange_tstmpl;
}

```

- o Add the element in the CDS view entity /DMO/R_Travel_D and annotate it with the annotation @Semantics.systemDateTime.lastChangedAt: true to allow the RAP runtime framework to update it automatically.

i Expand the following code sample to view the source code of CDS view entity /DMO/R_Travel_D.

Sample Code

```

@EndUserText.label: 'Travel View Entity for Draft RefScen'
@AccessControl.authorizationCheck: #NOT_REQUIRED

define root view entity /DMO/R_Travel_D
  as select from /DMO/A_TRAVEL_D

  composition [0..*] of /DMO/R_Booking_D as _Booking

  association [0..1] to /DMO/I_Agency as _Agency on $projection.AgencyID = _Ag
  association [0..1] to /DMO/I_Customer as _Customer on $projection.CustomerID = _
  association [0..1] to I_Currency as _Currency on $projection.CurrencyCode = _

{ ///DMO/A_TRAVEL_D
  key travel_uuid as TravelUUID,

  travel_id as TravelID,
  agency_id as AgencyID,
  customer_id as CustomerID,
  begin_date as BeginDate,
  end_date as EndDate,
  @Semantics.amount.currencyCode: 'CurrencyCode'
  booking_fee as BookingFee,
  @Semantics.amount.currencyCode: 'CurrencyCode'
  total_price as TotalPrice,
  currency_code as CurrencyCode,
  description as Description,
  overall_status as OverallStatus,

  @Semantics.user.createdBy: true
  local_created_by as LocalCreatedBy,
  @Semantics.systemDateTime.createdAt: true
  local_created_at as LocalCreatedAt,
  @Semantics.user.localInstanceLastChangedBy: true
  local_last_changed_by as LocalLastChangedBy,
  //local ETag field --> OData ETag
  @Semantics.systemDateTime.localInstanceLastChangedAt: true
  local_last_changed_at as LocalLastChangedAt,

  //total ETag field
  @Semantics.systemDateTime.lastChangedAt: true
  last_changed_at as LastChangedAt,

  //Associations
  _Booking,
  _Agency,
  _Customer,
  _Currency
}

```

i Note

The total ETag field does not need to be included in the projection view. The ETag is only used for internal processing of draft data and does not need to be exposed for the business service.

- o Add the total ETag field to the mapping prescription in the behavior definition for the [Travel](#) entity.

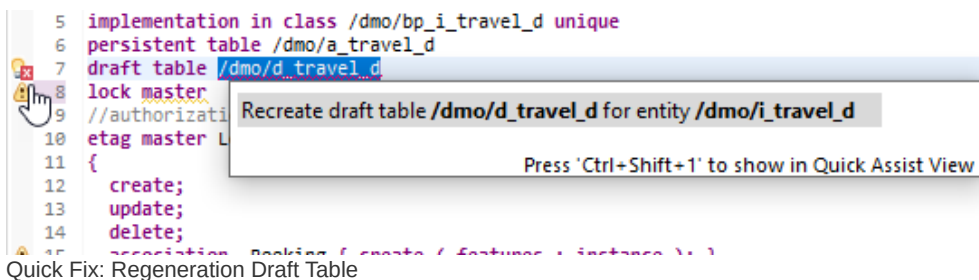
i Expand the following code sample to view the source code of Behavior Definition/DMO/R_Travel_D.

Sample Code

```
define behavior for /DMO/R_Travel_D alias Travel
implementation in class /DMO/BP_TRAVEL_D unique
...

mapping for /DMO/A_TRAVEL_D
{ AgencyID = agency_id;
  BeginDate = begin_date;
  BookingFee = booking_fee;
  CurrencyCode = currency_code;
  CustomerID = customer_id;
  Description = description;
  EndDate = end_date;
  LastChangedAt = last_changed_at;
  LocalCreatedAt = local_created_at;
  LocalCreatedBy = local_created_by;
  LocalLastChangedAt = local_last_changed_at;
  LocalLastChangedBy = local_last_changed_by;
  OverallStatus = overall_status;
  TotalPrice = total_price;
  TravelID = travel_id;
  TravelUUID = travel_uuid; }
}
```

- o Regenerate the draft database table /DMO/D_TRAVEL_D to include the total ETag field. The behavior definition provides a quick fix for the regeneration.



- o Define the element LastChangedAt as total ETag in the behavior definition by using the keyword `total etag`.

i Note

The definition of the total ETag is only possible directly after the lock master definition in the behavior definition.

i Note

It is recommended that you specify the ETag field as read-only using the field characteristic `readonly` in the behavior definition.

Sample Code

```
managed;
strict( 2 );
with draft;

define behavior for /DMO/R_Travel_D alias Travel
```

```

implementation in class /DMO/BP_I_TRAVEL_D unique
persistent table /DMO/A_TRAVEL_D
draft table /DMO/D_TRAVEL_D
lock master
total etag LastChangedAt
...
{
...
field ( readonly ) ...LastChangedAt;
...
}

```

5. Explicitly draft-enable the compositions within the draft BO.

As soon as you draft-enable a business object by adding `with draft` to the behavior definition, all BO-internal associations are automatically draft-enabled. To make this behavior explicit, the behavior prompts you to specify the compositions within a draft BO with `with draft`.

For more information, see [Draft-Enabled Associations](#).

6. Define the validations and determinations for the draft determine action Prepare.

The determinations and validations that are defined in the `Prepare` action are called before and during the `Activate` action to transfer draft to active instances. For more information, see [Preparing Draft Instances for Activation](#).

Use the `Activate` action in its optimized variant to increase the performance by skipping determination and validation executions that are not required. Note that the best practices for [Determination and Validation Modelling](#) need to be considered to make use of this optimized variant.

- o Define the `Prepare` action in the behavior definition.
- o Include all validations that are defined for the travel BO in the `Prepare`

i Note

It is not generally necessary to include all validations in the `Prepare` action, but only those that you want to check during the transition from draft to active.

≡ Sample Code

```

managed;
strict( 2 );
with draft;

define behavior for /DMO/R_Travel_D alias Travel
...

draft determine action Prepare
{
    validation validateAgency;
    validation validateCustomer;
    validation validateDates;
    validation validateCurrencyCode;
    validation validateBookingFee;
    validation Booking~validateCustomer;
    validation Booking~validateConnection;
    validation Booking~validateCurrencyCode;
    validation Booking~validateFlightPrice;
    validation Booking~validateStatus;
}

```

```
validation Bookingsupplement~validateSupplement;
validation Bookingsupplement~validateCurrencyCode;
validation Bookingsupplement~validatePrice; }
```

7. *Expand the following code sample to view the source code of the entire Behavior Definition /DMO/R_Travel_D.*

Sample Code

```
managed;
strict( 2 );
with draft;

define behavior for /DMO/R_Travel_D alias Travel
implementation in class /DMO/BP_TRAVEL_D unique
persistent table /DMO/A_TRAVEL_D
draft table /DMO/D_TRAVEL_D
lock master
total etag LastChangedAt
//authorization master ( instance )
etag master LocalLastChangedAt
{
  create;
  update;
  delete;
  association _Booking { create ( features : instance ); with draft; }

  field ( numbering : managed, readonly ) TravelUUID;
  field ( readonly ) TravelID, OverallStatus, TotalPrice, LocalCreatedAt, LocalCreatedBy, LocalL
  field ( mandatory ) CustomerID, AgencyID, BeginDate, EndDate;
  field ( features : instance ) BookingFee;

  action ( features : instance ) acceptTravel result [1] $self;
  action ( features : instance ) rejectTravel result [1] $self;
  action ( features : instance, authorization : update ) deductDiscount parameter /DMO/A_TRAVEL
  internal action reCalcTotalPrice;

  determination setTravelNumber on save { create; }
  determination setStatusToNew on modify { create; }
  determination calculateTotalPrice on modify { create; field BookingFee, CurrencyCode; }

  validation validateCustomer on save { create; field CustomerID; }
  validation validateAgency on save { create; field AgencyID; }
  validation validateDates on save { create; field BeginDate, EndDate; }

  determine action validationAgency { validation validateAgency; }
  determine action validationCustomer { validation validateCustomer; }
  determine action validationDates { validation validateDates; }

  side effects
  { field BookingFee affects field TotalPrice;
    determine action validationAgency executed on field AgencyID affects messages;
    determine action validationCustomer executed on field CustomerID affects messages;
    determine action validationDates executed on field BeginDate, field EndDate affects messages

  draft action resume;
  draft action Edit;
  draft action Activate optimized;
  draft action Discard;

  draft determine action Prepare
  {
    validation validateAgency;
    validation validateCustomer;
    validation validateDates;
    validation Booking ~ validateCustomer;
    validation Booking ~ validateConnection;
    validation Bookingsupplement ~ validateSupplement; }
```

```

mapping for /DMO/A_TRAVEL_D
{
  AgencyID = agency_id;
  BeginDate = begin_date;
  BookingFee = booking_fee;
  CurrencyCode = currency_code;
  CustomerID = customer_id;
  Description = description;
  EndDate = end_date;
  LastChangedAt = last_changed_at;
  LocalCreatedAt = local_created_at;
  LocalCreatedBy = local_created_by;
  LocalLastChangedAt = local_last_changed_at;
  LocalLastChangedBy = local_last_changed_by;
  OverallStatus = overall_status;
  TotalPrice = total_price;
  TravelID = travel_id;
  TravelUUID = travel_uuid; }
}

define behavior for /DMO/R_Booking_D alias Booking
implementation in class /DMO/BP_BOOKING_D unique
persistent table /DMO/A_BOOKING_D
draft table /DMO/D_BOOKING_D
lock dependent by _Travel
//authorization dependent by <association>
etag master LocalLastChangedAt
{
  update;
  delete;
  association _BookingSupplement { create ; with draft; }

  association _Travel { with draft; }

  field ( numbering : managed, readonly ) BookingUUID;
  field ( readonly ) TravelUUID, BookingID, BookingDate, LocalLastChangedAt;
  field ( mandatory ) CustomerID, AirlineID, ConnectionID, FlightDate;

  determination setBookingNumber on save { create; }
  determination setBookingDate on save { create; }
  determination calculateTotalPrice on modify { create; field FlightPrice, CurrencyCode; }

  validation validateCustomer on save { create; field CustomerID; }
  validation validateConnection on save { create; field AirlineID, ConnectionID, FlightDate; }

  side effects
  { field FlightPrice affects field _Travel.TotalPrice; }

mapping for /DMO/A_BOOKING_D
{
  AirlineID = carrier_id;
  BookingDate = booking_date;
  BookingID = booking_id;
  BookingStatus = booking_status;
  BookingUUID = booking_uuid;
  ConnectionID = connection_id;
  CurrencyCode = currency_code;
  CustomerID = customer_id;
  FlightDate = flight_date;
  FlightPrice = flight_price;
  LocalLastChangedAt = local_last_changed_at;
  TravelUUID = parent_uuid; }
}

define behavior for /DMO/R_BookingSupplement_D alias BookingSupplement
implementation in class /DMO/BP_BOOKINGSUPPLEMENT_D unique
persistent table /DMO/A_BKSUPPL_D
draft table /DMO/D_BKSUPPL_D
lock dependent by _Travel
//authorization dependent by <association>
etag master LocalLastChangedAt
{
  update;
  delete;

```

```

association _Travel { with draft; }
association _Booking { with draft; }

field ( numbering : managed, readonly ) BookSupplUUID;
field ( readonly ) TravelUUID, BookingUUID, BookingSupplementID, LocalLastChangedAt;
field ( mandatory ) SupplementID;

determination setBookSupplNumber on save { create; }
determination calculateTotalPrice on modify { create; field CurrencyCode, BookSupplPrice; }

validation validateSupplement on save { create; field SupplementID; }

side effects
{ field BookSupplPrice affects entity _Travel; }

mapping for /DMO/A_BKSUPPL_D
{ BookSupplPrice = price;
  BookSupplUUID = booksuppl_uuid;
  BookingSupplementID = booking_supplement_id;
  BookingUUID = parent_uuid;
  CurrencyCode = currency_code;
  LocalLastChangedAt = local_last_changed_at;
  SupplementID = supplement_id;
  TravelUUID = root_uuid; }
}

```

i Note

Draft-enabled business objects require a feature control method in the behavior pool. If you do not use dynamic feature control in your business logic, include the method declaration and an empty method implementation. In Draft BOs, feature control is called for every request to determine whether draft or active instances are requested.

Adaptions in the Behavior Pool

If you have consequently used %tky to refer to the key fields of instances, you shouldn't have to make changes in the behavior implementation in general.

For authorization, it is different. When draft-enabling a business object, draft actions are implicitly added to the BO behavior. There are two draft actions, which need to be considered in the authorization implementation as they also call the authorization methods during their runtime, see [Runtime Edit Action](#) and [Runtime Activate Action](#).

In this demo example, authorization for actions is handled exactly like the update operation. This can also be applied for the draft actions `Edit` and `Activate`.

Add the relevant draft actions to the action mapping in the behavior implementation for global and instance authorization.

i Expand the following code sample to view the source code of the method `get_global_authorizations`.

≡ Sample Code

```

METHOD get_global_authorizations.
  IF requested_authorizations-%create EQ if_abap_behv=>mk-on.
    IF is_create_granted( ) = abap_true.
      result-%create = if_abap_behv=>auth-allowed.
    ELSE.
      result-%create = if_abap_behv=>auth-unauthorized.
      APPEND VALUE #( %msg      = NEW /dmo/cm_flight_messages(
                          textid   = /dmo/cm_flight_messages=>not_authorized
                          severity = if_abap_behv_message=>severity-error )
                    %global = if_abap_behv=>mk-on ) TO reported-travel.
    ENDIF.

```


ENDIF.

"Actions are treated like update

```

IF requested_authorizations-%update          = if_abap_behv=>mk-on OR
  requested_authorizations-%action-Edit       = if_abap_behv=>mk-on OR
  requested_authorizations-%action-Prepare    = if_abap_behv=>mk-on OR
  requested_authorizations-%action-acceptTravel = if_abap_behv=>mk-on OR
  requested_authorizations-%action-rejectTravel = if_abap_behv=>mk-on OR
  requested_authorizations-%action-deductDiscount = if_abap_behv=>mk-on.

```

IF is_update_granted() = abap_true.

```

  result-%update          = if_abap_behv=>auth-allowed.
  result-%action-Edit     = if_abap_behv=>auth-allowed.
  result-%action-Prepare  = if_abap_behv=>auth-allowed.
  result-%action-acceptTravel = if_abap_behv=>auth-allowed.
  result-%action-rejectTravel = if_abap_behv=>auth-allowed.
  result-%action-deductDiscount = if_abap_behv=>auth-allowed.

```

ELSE.

```

  result-%update          = if_abap_behv=>auth-unauthorized.
  result-%action-Edit     = if_abap_behv=>auth-unauthorized.
  result-%action-Prepare  = if_abap_behv=>auth-unauthorized.
  result-%action-acceptTravel = if_abap_behv=>auth-unauthorized.
  result-%action-rejectTravel = if_abap_behv=>auth-unauthorized.
  result-%action-deductDiscount = if_abap_behv=>auth-unauthorized.

```

```

  APPEND VALUE #( %msg      = NEW /dmo/cm_flight_messages(
                        textid   = /dmo/cm_flight_messages=>not_authorized
                        severity = if_abap_behv_message=>severity-error )
                %global = if_abap_behv=>mk-on ) TO reported-travel.

```

ENDIF.

ENDIF.

IF requested_authorizations-%delete = if_abap_behv=>mk-on.

IF is_delete_granted() = abap_true.

result-%delete = if_abap_behv=>auth-allowed.

ELSE.

result-%delete = if_abap_behv=>auth-unauthorized.

```

  APPEND VALUE #( %msg      = NEW /dmo/cm_flight_messages(
                        textid   = /dmo/cm_flight_messages=>not_authorized
                        severity = if_abap_behv_message=>severity-error )
                %global = if_abap_behv=>mk-on ) TO reported-travel.

```

ENDIF.

ENDIF.

ENDMETHOD.

i Expand the following code sample to view the source code of the method `get_instance_authorizations`.

Sample Code

METHOD get_instance_authorizations.

```

DATA: update_requested TYPE abap_bool,
      delete_requested TYPE abap_bool,
      update_granted   TYPE abap_bool,
      delete_granted   TYPE abap_bool.

```

READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE

ENTITY Travel

FIELDS (AgencyID)

WITH CORRESPONDING #(keys)

RESULT DATA(travels)

FAILED failed.

CHECK travels IS NOT INITIAL.

"Select country_code and agency of corresponding persistent travel instance

"authorization only checked against instance that have active persistence

SELECT FROM /DMO/A_TRAVEL_D AS travel INNER JOIN /DMO/AGENCY AS agency

```

ON travel~agency_id = agency~agency_id
FIELDS travel~travel_uuid , travel~agency_id, agency~country_code
FOR ALL ENTRIES IN @travels WHERE travel_uuid EQ @travels-TravelUUID
INTO TABLE @DATA(travel_agency_country).

```

"all actions are treated like update

```

update_requested = COND #( WHEN requested_authorizations-%update                = if_abap_be
                             requested_authorizations-%assoc-Booking           = if_abap_be
                             requested_authorizations-%action-Edit              = if_abap_be
                             requested_authorizations-%action-Prepare           = if_abap_be
                             requested_authorizations-%action-acceptTravel      = if_abap_be
                             requested_authorizations-%action-deductDiscount    = if_abap_be
                             requested_authorizations-%action-rejectTravel      = if_abap_be
                             THEN abap_true ELSE abap_false ).

```

```

delete_requested = COND #( WHEN requested_authorizations-%delete                = if_abap_be
                             THEN abap_true ELSE abap_false ).

```

LOOP AT travels INTO DATA(travel).

```

"get country_code of agency in corresponding instance on persistent table
READ TABLE travel_agency_country WITH KEY travel_uuid = travel-TravelUUID
  ASSIGNING FIELD-SYMBOL(<travel_agency_country_code>).

```

"Auth check for active instances that have before image on persistent table
IF sy-subrc = 0.

"check auth for update

IF update_requested = abap_true.

update_granted = is_update_granted(<travel_agency_country_code>-country_code).

IF update_granted = abap_false.

APPEND VALUE #(%tky = travel-%tky

%msg = NEW /dmo/cm_flight_messages(

textid = /dmo/cm_flight_messages=>not_authorized_for_age

agency_id = travel-AgencyID

severity = if_abap_behv_message=>severity-error)

%element-AgencyID = if_abap_behv=>mk-on) T0 reported-travel.

ENDIF.

ENDIF.

"check auth for delete

IF delete_requested = abap_true.

delete_granted = is_delete_granted(<travel_agency_country_code>-country_code).

IF delete_granted = abap_false.

APPEND VALUE #(%tky = travel-%tky

%msg = NEW /dmo/cm_flight_messages(

textid = /dmo/cm_flight_messages=>not_authorized_for_age

agency_id = travel-AgencyID

severity = if_abap_behv_message=>severity-error)

%element-AgencyID = if_abap_behv=>mk-on) T0 reported-travel.

ENDIF.

ENDIF.

" operations on draft instances and on active instances that have no persistent before ima
" create authorization is checked, for newly created instances

ELSE.

update_granted = delete_granted = is_create_granted().

IF update_granted = abap_false.

APPEND VALUE #(%tky = travel-%tky

%msg = NEW /dmo/cm_flight_messages(

textid = /dmo/cm_flight_messages=>not_authorized

severity = if_abap_behv_message=>severity-error)

%element-AgencyID = if_abap_behv=>mk-on) T0 reported-travel.

ENDIF.

ENDIF.

```

APPEND VALUE #( LET upd_auth = COND #( WHEN update_granted = abap_true THEN if_abap_behv=>
                                         ELSE if_abap_behv=>auth-unauthorized )
                del_auth = COND #( WHEN delete_granted = abap_true THEN if_abap_behv=>
                                         ELSE if_abap_behv=>auth-unauthorized )

```

IN

%tky = travel-%tky

%update = upd_auth

```

        %assoc-_Booking          = upd_auth
        %action-Edit             = upd_auth
        %action-Prepare          = upd_auth
        %action-acceptTravel     = upd_auth
        %action-deductDiscount   = upd_auth
        %action-rejectTravel     = upd_auth

        %delete                  = del_auth
    ) TO result.

ENDLOOP.

ENDMETHOD.

```

Result

As soon as you activate the behavior definition, the business object is draft-enabled. In general, that means:

- Transactional requests are processed by the draft runtime,
- The importing structures for all methods in the behavior pool receive an additional component, the draft indicator `%is_draft`. In particular, the transactional key `%tky` determines whether the operation is executed on a draft or on an active instance.
- The draft actions are implicitly available and applied during the draft life cycle.
- The existing UI service does not change as long as you do not include the draft capabilities in the projection behavior definition. It still behaves like a non-draft business service. The underlying draft BO processes every request with the draft indicator `is_draft` set to false.

For more information, see [Draft Runtime](#).

The conversion to a draft business object sometimes requires follow-up activities of existing implementations in the business logic. In general, scan the existing code thoroughly to detect cases where you need to differentiate between active and draft processing.

If the transactional key `%tky` is used for referring to the key components, no adaptation is necessary for these implementations. The `%is_draft` is automatically added as key component to the `%tky` structure.

Related Information

[Exclusive Draft Handling](#)

Exposing the Draft Business Object for a UI Business Service

In the previous chapters, you have created a draft business object by draft-enabling a managed business object. Since the managed business object projection without draft capabilities remains untouched, it continues to operate only on active instances.

If you want to have a business service with draft capabilities, you can either add the draft features to the existing business object projection or create a new projection for the draft service. This is done in this demo scenario. In the end, you have two parallel business services, one for active only and one that includes draft features. To expose the draft business as a business service the following steps are relevant:

- [Projecting the Draft BO for Draft Exposure](#)
- [Defining and Publishing a UI Service with Draft Capabilities](#)

Projecting the Draft BO for Draft Exposure

For the managed scenario with draft, create a projection layer for the draft BO.

Context and Procedure

In the first part of this development guide, you have created a non-draft business service based on a managed business object. The business object projection includes all elements in the data model and the behavior features that are relevant for the UI service. This projection remains stable, even when you add draft capabilities to the basic business object. The projection layer filters out the elements and behavior features that are draft-specific as they are not included in the projection.

To expose the draft business object for a UI service, it is necessary that you define a business object projection that also includes the draft specific features. For more information about the BO projection layer, see [Business Object Projection](#).

For the travel demo scenario, define the data model and the behavior projection for the draft projection:

- For the **data model projection**, the following steps are relevant:

- Create **CDS projection views**.

In the CDS projection views define UI-service-specific features, such as search capabilities, text associations, and value helps.

- Create **metadata extensions**.

Metadata extensions are primarily used to define the annotations that are relevant for the UI design. They are outsourced in metadata extension to obtain an easy approach for layering.

For a detailed description on how to define data model projection features, see [Projecting the BO Data Model](#).

- For the **behavior projection**, the following steps are relevant:

- Create a **projection behavior definition**.

For a detailed description on behavior projection features, see [Projecting the BO Behavior](#).

Projecting the Draft BO Data Model

For the managed scenario with draft, create and implement CDS projection views and metadata extensions for the draft BO entities.

Context and Procedure

Create data definition for projection views for all three BO entities **Travel**, **Booking**, and **Booking Supplement**. In ADT, use the context menu on the underlying CDS view entity to open the wizard for creating CDS data definition.

Select the template **Define Projection View**.

i Note

The draft scenario in the ABAP Flight Reference Scenario uses the suffix `_D`. Since the new CDS projection views represent the UI service with draft capabilities, we use the marker `_D` before the suffix to distinguish the active projection layer from the projection layer that includes draft. In addition, projection views carry the prefix `C_` as they represent the consumption layer. For more information, see [Naming Conventions for Development Objects](#).

For each projection view, create a metadata extension. In ADT, right-click on the data definition to open the creation wizard for metadata extensions.

i Note

According to the virtual data model (VDM) guidelines, the metadata extension use the same name as the related CDS view.

For more information about the syntax in projection views, see [Define View Entity as Projection \(ABAP Keyword Documentation\)](#).

Projection View /DMO/C_Travel_D_D

The projection view is a projection on /DMO/R_Travel_D. Include the keyword `root` in the projection view.

Use all elements of the underlying BO travel entity. You do not need to use the administrative data in the projection view, except for the element representing the ETag master.

Enable the usage of a metadata extension with the annotation `@Metadata.allowExtension:true`.

Enable search capabilities for the travel entity, with the `@Search` annotations on the entity header and on designated elements. For more information, see [Enabling Text and Fuzzy Searches in SAP Fiori Apps](#).

For the OData V4 consumption of a draft business object via a Fiori Elements UI, the annotation `@ObjectModel.semanticKey: ['SemanticIdentifier']` must be used to ensure that draft indications on the instance are displayed.

To retrieve the long texts of the ID elements `CustomerID`, `AgencyID`, and `OverallStatus` denormalize the text elements from the associated text provider view into the projection view. For more information, see [Getting Language-Dependent Text in Projection Views](#).

Define value helps for the elements `CustomerID`, `AgencyID`, `CurrencyCode` and `OverallStatus`. For more information, see [Adding Value Help](#).

In contrast to the ETag master field, which is necessary to ensure optimistic concurrency control in the OData consumption, you do not need to include the total ETag field in the draft business service projection. The total ETag is only necessary for internal draft processing.

As soon as all projection views of the composition structure of the business object are available, redirect the composition to child to the projection view of the booking entity.

i Expand the following code sample to view the source code of Projection View /DMO/C_Travel_D_D.

Sample Code

```
@EndUserText.label: 'Travel Projection View with Draft'
@AccessControl.authorizationCheck: #NOT_REQUIRED

@Metadata.allowExtensions: true
@Search.searchable: true
@ObjectModel.semanticKey: ['TravelID']

define root view entity /DMO/C_Travel_D_D
  provider contract transactional_query
  as projection on /DMO/R_Travel_D

{
  key TravelUUID,
```

```

@Search.defaultSearchElement: true
TravelID,

@Search.defaultSearchElement: true
@ObjectModel.text.element: ['AgencyName']
@Consumption.valueHelpDefinition: [{ entity : {name: '/DM0/I_Agency_StdVH', element: 'Agen
AgencyID,
_Agency.Name          as AgencyName,

@Search.defaultSearchElement: true
@ObjectModel.text.element: ['CustomerName']
@Consumption.valueHelpDefinition: [{entity: {name: '/DM0/I_Customer_StdVH', element: 'Cust
CustomerID,
_Customer.LastName as CustomerName,

BeginDate,

EndDate,

BookingFee,

TotalPrice,

@Consumption.valueHelpDefinition: [{entity: {name: 'I_CurrencyStdVH', element: 'Currency'
CurrencyCode,

Description,

@ObjectModel.text.element: ['OverallStatusText']
@Consumption.valueHelpDefinition: [{ entity : {name: '/DM0/I_Overall_Status_VH', element:
OverallStatus,
_OverallStatus._Text.Text as OverallStatusText: localized,

LocalLastChangedAt,
/* Associations */
_Agency,
_Booking : redirected to composition child /DM0/C_Booking_D_D,
_Currency,
_Customer,
_OverallStatus
}

```

Metadata Extension /DM0/C_Travel_D_D

Define the metadata extension as extension of the CDS projection view /DM0/C_Travel_A_D.

Define the layer #CORE.

Use all the @UI annotations to design the user interface for the travel app. For more information, see [Defining CDS Annotations for Metadata-Driven UIs](#).

i Expand the following code sample to view the source code of Metadata Extension /DM0/C_Travel_D_D.

Sample Code

```

@Metadata.layer: #CORE

@UI: { headerInfo: { typeName: 'Travel',
                    typeNamePlural: 'Travels',
                    title: { type: #STANDARD, value: 'TravelID' } },
  presentationVariant: [{ sortOrder: [{ by: 'TravelID', direction: #DESC }], visualizations

annotate entity /DM0/C_Travel_D_D with
{
  @UI.facet: [{ id:          'Travel',

```

```

        purpose:      #STANDARD,
        type:         #IDENTIFICATION_REFERENCE,
        label:        'Travel',
        position:     10 },
    { id:             'Booking',
      purpose:       #STANDARD,
      type:          #LINEITEM_REFERENCE,
      label:         'Booking',
      position:      20,
      targetElement: '_Booking'}}]
@UI.hidden: true
TravelUUID;

@UI: { lineItem:      [ { position: 10, importance: #HIGH },
                       { type: #FOR_ACTION, dataAction: 'acceptTravel', label: 'Accept Trave
                       { type: #FOR_ACTION, dataAction: 'rejectTravel', label: 'Reject Trave
                       { type: #FOR_ACTION, dataAction: 'deductDiscount', label: 'Deduct Dis
      identification: [ { position: 10, label: 'Travel ID' },
                       { type: #FOR_ACTION, dataAction: 'acceptTravel', label: 'Accept Trave
                       { type: #FOR_ACTION, dataAction: 'rejectTravel', label: 'Reject Trave
                       { type: #FOR_ACTION, dataAction: 'deductDiscount', label: 'Deduct Dis
      selectionField: [ { position: 10 } ] }
TravelID;

@UI: { lineItem:      [ { position: 20, importance: #HIGH } ] ,
      identification: [ { position: 20 } ] ,
      selectionField: [ { position: 20 } ] }
AgencyID;

@UI: { lineItem:      [ { position: 30, importance: #HIGH } ] ,
      identification: [ { position: 30 } ] ,
      selectionField: [ { position: 30 } ] }
CustomerID;

@UI: { lineItem:      [ { position: 40, importance: #MEDIUM } ] ,
      identification: [ { position: 40 } ] }
BeginDate;

@UI: { lineItem:      [ { position: 50, importance: #MEDIUM } ] ,
      identification: [ { position: 50 } ] }
EndDate;

@UI: { lineItem:      [ { position: 60, importance: #MEDIUM } ] ,
      identification: [ { position: 60 } ] }
BookingFee;

@UI: { lineItem:      [ { position: 70, importance: #MEDIUM } ] ,
      identification: [ { position: 70 } ] }
TotalPrice;

// CurrencyCode;

@UI: { lineItem:      [ { position: 90, importance: #MEDIUM } ] ,
      identification: [ { position: 90 } ] }
Description;

@UI: { lineItem:      [ { position: 99, importance: #HIGH } ] ,
      identification: [ { position: 99, label: 'Status' } ] ,
      selectionField: [{ position: 40 }],
      textArrangement: #TEXT_ONLY }
OverallStatus;

@UI.hidden: true
OverallStatusText;

@UI.hidden: true
LocalLastChangedAt;
}

```

Projection View /DM0/C_Booking_D_D

The projection view is a projection on /DMO/R_Booking_D.

Use all elements of the underlying BO travel entity, in particular the element representing the ETag master.

Enable the usage of a metadata extension with the annotation @Metadata.allowExtension:true.

Enable search capabilities for the booking entity, with the @Search annotations. For more information, see [Enabling Text and Fuzzy Searches in SAP Fiori Apps](#).

To retrieve the long texts of the ID elements CustomerID and CarrierID, denormalize the text elements from the associated text provider views into the projection view. For more information, see [Getting Language-Dependent Text in Projection Views](#).

Define value helps for the elements CustomerID, AirlineID, ConnectionID, FlightDate, and CurrencyCode. For more information, see [Adding Value Help](#).

As soon as all projection views of the composition structure of the business object are available, redirect the composition to child to the projection view of the booking entity and the association to the root entity to the respective projection view.

i Expand the following code sample to view the source code of Projection View /DMO/C_Booking_D_D.

≡ Sample Code

```
@EndUserText.label: 'Booking Proj View with Draft'
@AccessControl.authorizationCheck: #NOT_REQUIRED

@Metadata.allowExtensions: true
@Search.searchable: true
@ObjectModel.semanticKey: ['BookingID']

define view entity /DMO/C_Booking_D_D
  as projection on /DMO/R_Booking_D
{
  key BookingUUID,

  TravelUUID,

  @Search.defaultSearchElement: true
  BookingID,

  BookingDate,

  @ObjectModel.text.element: ['CustomerName']
  @Search.defaultSearchElement: true
  @Consumption.valueHelpDefinition: [{entity: {name: '/DMO/I_Customer_StdVH', element: 'Cust
CustomerID,
_Customer.LastName as CustomerName,

  @ObjectModel.text.element: ['CarrierName']
  @Consumption.valueHelpDefinition: [
    { entity: {name: '/DMO/I_Flight_StdVH', element: 'AirlineID'},
      additionalBinding: [ { localElement: 'FlightDate', element: 'FlightDate', usage:
                          { localElement: 'ConnectionID', element: 'ConnectionID', usage:
                          { localElement: 'FlightPrice', element: 'Price', usage:
                          { localElement: 'CurrencyCode', element: 'CurrencyCode', usage:
                          qualifier: 'FlightVH1',
                          label: 'Flights' }
    ]
    AirlineID,
    _Carrier.Name      as CarrierName,

  @Consumption.valueHelpDefinition: [
    { entity: {name: '/DMO/I_Flight_StdVH', element: 'ConnectionID'},
      additionalBinding: [ { localElement: 'FlightDate', element: 'FlightDate', usage:
                          { localElement: 'AirlineID', element: 'AirlineID', usage:
                          { localElement: 'FlightPrice', element: 'Price', usage:
                          { localElement: 'CurrencyCode', element: 'CurrencyCode', usage:
                          qualifier: 'FlightVH2',
```



```

        label: 'Flights' }
    ]
    ConnectionID,

    @Consumption.valueHelpDefinition: [
        { entity: {name: '/DM0/I_Flight_StdVH', element: 'FlightDate'},
          additionalBinding: [ { _localElement: 'AirlineID', element: 'AirlineID', usage:
                                { localElement: 'ConnectionID', element: 'ConnectionID', usage:
                                  { localElement: 'FlightPrice', element: 'Price', usage:
                                    { localElement: 'CurrencyCode', element: 'CurrencyCode', usage:
                                      label: 'Flights',
                                      useForValidation: true }
                                }
                              }
                            }
        ]
    ]
    FlightDate,

    @Consumption.valueHelpDefinition: [
        { entity: {name: '/DM0/I_Flight_StdVH', element: 'Price'},
          additionalBinding: [ { _localElement: 'FlightDate', element: 'FlightDate', usage:
                                { localElement: 'AirlineID', element: 'AirlineID', usage:
                                  { localElement: 'ConnectionID', element: 'ConnectionID', usage:
                                    { localElement: 'CurrencyCode', element: 'CurrencyCode', usage:
                                      label: 'Flights',
                                      useForValidation: true }
                                }
                              }
                            }
        ]
    ]
    FlightPrice,

    @Consumption.valueHelpDefinition: [{entity: {name: 'I_CurrencyStdVH', element: 'Currency'
    CurrencyCode,

    @ObjectModel.text.element: ['BookingStatusText']
    @Consumption.valueHelpDefinition: [{entity: {name: '/DM0/I_Booking_Status_VH', element: 'B
    BookingStatus,
    _BookingStatus._Text.Text as BookingStatusText: localized,

    LocalLastChangedAt,

    /* Associations */
    _BookingSupplement: redirected to composition child /DM0/C_BookingSupplement_D_D,
    _BookingStatus,
    _Carrier,
    _Connection,
    _Customer,
    _Travel: redirected to parent /DM0/C_Travel_D_D
}

```

Metadata Extension /DM0/C_Booking_D_D

Define the metadata extension as extension of the CDS projection view /DM0/C_Booking_D_D.

Define the layer #CORE.

Use all the @UI annotations to design the user interface for the travel app. For more information, see [Defining CDS Annotations for Metadata-Driven UIs](#).

i Expand the following code sample to view the source code of Metadata Extension View /DM0/C_Booking_D_D.

Sample Code

```

@Metadata.layer: #CORE

@UI: { headerInfo: { typeName: 'Booking',
                    typeNamePlural: 'Bookings',
                    title: { type: #STANDARD, value: 'BookingID' } } }

annotate entity /DM0/C_Booking_D_D with

```

```

{
  @UI.facet: [ { id:          'Booking',
                 purpose:     #STANDARD,
                 type:        #IDENTIFICATION_REFERENCE,
                 label:       'Booking',
                 position:    10 },
               { id:          'BookingSupplement',
                 purpose:     #STANDARD,
                 type:        #LINEITEM_REFERENCE,
                 label:       'Booking Supplement',
                 position:    20,
                 targetElement: '_BookingSupplement' } ]

  @UI.hidden: true
  BookingUUID;

  @UI.hidden: true
  TravelUUID;

  @UI: { lineItem:      [ { position: 20, importance: #HIGH } ],
        identification: [ { position: 20 } ] }
  BookingID;

  @UI: { lineItem:      [ { position: 30, importance: #HIGH } ],
        identification: [ { position: 30 } ] }
  BookingDate;

  @UI: { lineItem:      [ { position: 40, importance: #HIGH } ],
        identification: [ { position: 40 } ] }
  CustomerID;

  @UI: { lineItem:      [ { position: 50, importance: #HIGH } ],
        identification: [ { position: 50 } ] }
  AirlineID;

  @UI: { lineItem:      [ { position: 60, importance: #HIGH } ],
        identification: [ { position: 60 } ] }
  ConnectionID;

  @UI: { lineItem:      [ { position: 70, importance: #HIGH } ],
        identification: [ { position: 70 } ] }
  FlightDate;

  @UI: { lineItem:      [ { position: 80, importance: #HIGH } ],
        identification: [ { position: 80 } ] }
  FlightPrice;

  // CurrencyCode;

  @UI: { lineItem:      [ { position: 90, importance: #HIGH } ],
        identification: [ { position: 90 } ],
        textArrangement: #TEXT_ONLY }
  BookingStatus;

  @UI.hidden: true
  BookingStatusText;

  @UI.hidden: true
  LocalLastChangedAt;
}

```

Projection View /DMO/C_BookingSupplement_D_D

The projection view is a projection on /DMO/R_BookingSupplement_D.

Use all elements of the underlying BO travel entity, in particular the element representing the ETag master.

Enable the usage of a metadata extension with the annotation `@Metadata.allowExtension:true`.

Enable search capabilities for the travel entity, with the @Search annotations. For more information, see [Enabling Text and Fuzzy Searches in SAP Fiori Apps](#).

To retrieve the long texts of the ID element SupplementID, denormalize the text element from the associated text provider view into the projection view. For more information, see [Getting Language-Dependent Text in Projection Views](#).

Define value helps for the elements SupplementID and CurrencyCode. For more information, see [Adding Value Help](#).

As soon as all projection views of the composition structure of the business object are available, redirect the association to the parent to the projection view of the booking entity and the association to the root entity to the respective projection view.

i Expand the following code sample to view the source code of Projection View /DMO/C_BookingSupplement_D_D.

≡ Sample Code

```
@EndUserText.label: 'BookingSuppl Proj View for Draft RefScen'
@AccessControl.authorizationCheck: #NOT_REQUIRED

@Metadata.allowExtensions: true
@Search.searchable: true
@ObjectModel.semanticKey: ['BookingSupplementID']

define view entity /DMO/C_BookingSupplement_D_D
  as projection on /DMO/R_BookingSupplement_D
{
  key BookSupplUUID,

  TravelUUID,

  BookingUUID,

  @Search.defaultSearchElement: true
  BookingSupplementID,

  @ObjectModel.text.element: ['SupplementDescription']
  @Consumption.valueHelpDefinition: [
    { entity: {name: '/DMO/I_Supplement_StdVH', element: 'SupplementID' },
      additionalBinding: [ { localElement: 'BookSupplPrice', element: 'Price', us
                           { localElement: 'CurrencyCode', element: 'CurrencyCode', us
                           useForValidation: true }
    ]
  SupplementID,
  _SupplementText.Description as SupplementDescription : localized,

  BookSupplPrice,

  @Consumption.valueHelpDefinition: [{entity: {name: 'I_CurrencyStdVH', element: 'Currency'
  CurrencyCode,

  LocalLastChangedAt,

  /* Associations */
  _Booking : redirected to parent /DMO/C_Booking_D_D,
  _Product,
  _SupplementText,
  _Travel : redirected to /DMO/C_Travel_D_D
}
```

Metadata Extension /DMO/C_BookingSupplement_D_D

Define the metadata extension as extension of the CDS projection view /DMO/C_BookingSupplement_D_D.

Define the layer #CORE.

Use all the @UI annotations to design the user interface for the travel app. For more information, see [Defining CDS Annotations for Metadata-Driven UIs](#).

i Expand the following code sample to view the source code of Projection View /DMO/C_BookingSupplement_D_D.

Sample Code

```
@Metadata.layer: #CORE

@UI: { headerInfo: { typeName:      'Booking Supplement',
                      typeNamePlural: 'Booking Supplements',
                      title:         { type: #STANDARD,
                                       label: 'Booking Supplement',
                                       value: 'BookingSupplementID' } } }

annotate entity /DMO/C_BookingSupplement_D_D
with
{
  @UI.facet: [ { id:          'BookingSupplement',
                  purpose:    #STANDARD,
                  type:       #IDENTIFICATION_REFERENCE,
                  label:      'Booking Supplement',
                  position:    10 } ]

  @UI.hidden:true
  BookSupplUUID;
  @UI.hidden:true
  BookingUUID;
  @UI.hidden:true
  TravelUUID;

  @UI: { lineItem:      [ { position: 10, importance: #HIGH } ],
        identification: [ { position: 10 } ] }
  BookingSupplementID;

  @UI: { lineItem:      [ { position: 20, importance: #HIGH } ],
        identification: [ { position: 20 } ] }
  SupplementID;

  @UI: { lineItem:      [ { position: 30, importance: #HIGH } ],
        identification: [ { position: 30 } ] }
  BookSupplPrice;

  // CurrencyCode;

  @UI.hidden: true
  LocalLastChangedAt;
}
```

Projecting the Draft BO Behavior

For the managed scenario with draft, create a projection behavior definition for defining the projection behavior.

Projection Behavior Definition /DMO/C_Travel_D_D

Use the complete behavior that was defined in the underlying behavior definition for the draft BO projection. (The template does not include everything, some features must be added manually.) Create a projection behavior definition for the travel business object. In ADT, use the context menu of the root projection view to open the wizard for creating behavior definitions. The implementation type is prefilled with

i Expand the following code sample to view the source codeCreate a projection behavior definition for the travel business object. In ADT, use of Projection Behavior Definition .

≡, Sample Code

```

projection;
strict(2);
use draft;
use side effects;

define behavior for /DMO/C_Travel_D_D alias Travel
use etag
{
    use create;
    use update;
    use delete;

    use action acceptTravel;
    use action rejectTravel;
    use action deductDiscount;
    use function GetDefaultsFor deductDiscount;

    use action validateAgencyID;
    use action validateCustomerID;
    use action validateDateRange;
    use action validateBookingCharge;

    use action Edit;
    use action Activate;
    use action Discard;
    use action Prepare;
    use action Resume;

    use association _Booking { create; with draft; }
}

define behavior for /DMO/C_Booking_D_D alias Booking
use etag
{
    use update;
    use delete;

    use action validateFlightCosts;

    use association _BookingSupplement { create; with draft; }
    use association _Travel { with draft; }
}

define behavior for /DMO/C_BookingSupplement_D_D alias BookingSupplement
use etag
{
    use update;
    use delete;

    use action validateSupplementPrice;

    use association _Travel { with draft; }
    use association _Booking { with draft; }
}

```

Defining and Publishing a UI Service with Draft Capabilities

For the managed scenario with draft, define a UI service and bind it to a protocol.

Context and Procedure

To address the draft BO from the ABAP-external world, it must be exposed as a business service to OData. In the service definition, you define which artifacts are included in the service. The service binding defines the protocol with which the service is addressed.

In addition, it provides the means to activate the service locally. The Fiori Elements App Preview can be used as soon as the service is activated.

Create a service definition for the draft business service.

Based on the service definition, create a service binding.

UI services of draft business objects with Fiori Elements are only supported with OData V4. Fiori Elements cannot handle the full scope of draft services that are exposed with OData V2.

i Note

The draft scenario in the ABAP Flight Reference Scenario uses the suffix `_D`. To distinguish the draft service from the service with draft, we use the addition `_D` before the suffix. For more information, see [Naming Conventions for Development Objects](#).

Service Definition /DMO/UI_TRAVEL_D_D

Include the projection views of the business object and all related CDS entities that are relevant for UI consumption into the service definition.

i *Expand the following code sample to view the source code of Projection View /DMO/UI_TRAVEL_D_D.*

Sample Code

```
@EndUserText.label: 'Travel Draft Scenario (draft projection)'
@ObjectModel.leadingEntity.name: '/DMO/C_Travel_D_D'
define service /DMO/UI_TRAVEL_D_D {
  expose /DMO/C_Travel_D_D as Travel;
  expose /DMO/C_Booking_D_D as Booking;
  expose /DMO/C_BookingSupplement_D_D as BookingSupplement;
  expose /DMO/I_Supplement_StdVH as Supplement;
  expose /DMO/I_SupplementCategory_VH as SupplementCategory;
  expose /DMO/I_Customer_StdVH as Passenger;
  expose /DMO/I_Agency_StdVH as TravelAgency;
  expose /DMO/I_Carrier_StdVH as Airline;
  expose /DMO/I_Connection_StdVH as FlightConnection;
  expose /DMO/I_Flight_StdVH as Flight;
  expose /DMO/I_Airport_StdVH as Airport;
  expose /DMO/I_Overall_Status_VH as OverallStatus;
  expose /DMO/I_Booking_Status_VH as BookingStatus;
  expose I_CurrencyStdVH as Currency;
  expose I_CountryVH as Country;
}
```

Service Binding

Expose the service as a UI service with OData protocol V4 and publish it locally. You can then use the preview to test the service.

Developing Transactional Apps with Collaborative Draft Capabilities

With the collaborative draft capability, you can invite users to edit the same draft instance simultaneously.

Prerequisites

The implementation prerequisites for the collaborative draft are the same as for the exclusive draft. For more information, see [Draft-Enabling the Managed Business Object](#).

You can also generate the necessary artifacts for a draft-enabled business object (BO) from scratch with the **OData UI Service from Scratch** wizard. For more information, see [OData UI Service from Scratch](#).

Role Authorizations for SAP S/4HANA Cloud Public Edition and SAP BTP ABAP environment

The new business catalog `SAP_CORE_BC_RAP_DRAFT_PC` needs to be assigned, which is part of the business role template `SAP_BR_EMPLOYEE`. Therefore, you initially have to ask your administrator to either generate an additional role based on the role template `SAP_BR_EMPLOYEE` or let the administrator assign the business catalogue to an existing role which is assigned to developers on your system.

Role Authorizations for SAP S/4HANA Cloud Private Edition

You need to define the necessary developer roles yourself, by adding the authorization object `S_DRAFT` with the authorization fields `DRAFTUSER` and `ACTVT`. Make sure to define the following values for the respective authorization fields:

- `DRAFTUSER= *`
- `ACTVT= W4`

Context

The collaborative draft capability for a BO is defined in the behavior definition. You can build collaborative draft BOs from scratch or you can add collaborative capabilities to a BO with already implemented exclusive draft capabilities in both implementation types `managed` or `unmanaged`, so multiple users can edit the same draft instance at the same time. A business object can only either be draft-enabled with an exclusive draft or with a collaborative draft. The draft capability applies to the entire BO. It's not possible to implement draft capabilities for individual entities within a BO. For more information about the draft concept, refer to [Draft](#) and [Draft Handling](#).

This topic explains how you implement the collaborative draft in the context of the ABAP RESTful Application Programming Model (RAP), and how to invite users to a draft instance of your BO on the SAP Fiori UI. The following code examples are based on a managed BO, which you can download and import into your development environment. For more information, refer to [Downloading the ABAP Flight Reference Scenario](#).

Procedure

To enable collaborative draft capabilities to your BO, proceed as follows:

1. In the header of the behavior definition of the base layer, add the syntax element with `collaborative draft`.

i Note

If you use an already draft-enabled BO with exclusive draft, change the syntax element with `draft` to with `collaborative draft`.

☰ Sample Code

```

managed implementation in class /DMO/BP_R_TravelTP_CD unique;
strict ( 2 );
with collaborative draft;
extensible;
{
...
}

```

i Note

Initially, a syntax error occurs because you haven't added the draft action `Share` yet.

2. For collaborative draft, a draft query view is mandatory for the `authorization master` entity. Draft query views allow you to define `READ` access limitations for draft data based on the data control language (DCL).

Add a draft query view to the draft table declaration in the base behavior definition with the aid of the keyword `query`:

Sample Code

```

define behavior for /DMO/R_TravelTP_CD alias Travel
persistent table /dmo/a_travel_cd
extensible
draft table /dmo/d_travel_cd query /DMO/R_TravelTPDraft_CD
...

```

You can now create the draft query view with a quick fix. It creates a new root view entity as your draft query view with your draft database table as the referenced object.

i Expand the following code sample to view the source code of Draft Query View `/DMO/R_TRAVELTPDRAFT_CD`.

Sample Code

```

@AccessControl.authorizationCheck: #MANDATORY
@Metadata.allowExtensions: true
@EndUserText.label: 'Draft Query View /DMO/D_TRAVEL_CD'
define view entity /DMO/R_TRAVELTPDRAFT_CD
  as select from /dmo/d_travel_cd
{
  key traveluuid as TravelUuid,
  travelid as TravelId,
  begindate as BeginDate,
  enddate as EndDate,
  bookingfee as BookingFee,
  currencycode as CurrencyCode,
  description as Description,
  localcreatedby as LocalCreatedBy,
  localcreatedat as LocalCreatedAt,
  locallastchangedby as LocalLastChangedBy,
  locallastchangedat as LocalLastChangedAt,
  lastchangedat as LastChangedAt,
  draftentitycreationdatetime as draftentitycreationdatetime,
  draftentitylastchangeddatetime as draftentitylastchangeddatetime,
  draftadministrativeatauid as draftadministrativeatauid,
  draftentityoperationcode as draftentityoperationcode,
  hasactiveentity as hasactiveentity,
  draftfieldchanges as draftfieldchanges
}

```

If you don't use the offered quick fix, make sure that you don't use the annotation

`@Metadata.ignorePropagatedAnnotations` at all or set its value to `false`. If you leave the value set to `true`, be aware that you have to maintain your annotations in the draft query view yourself.

In this scenario, the draft query view selects from the draft database table `/DMO/D_TRAVEL_CD` of the root entity `/DMO/R_TravelTP_CD`. The draft query view must have a DCL that inherits its access conditions from the corresponding DCL of the root entity.

For more information on draft query views, see [Draft Query Views](#).

i Note

A data control language (DCL) is mandatory if your active business object view entity is protected by a DCL.

3. Create a new DCL for your draft query view. Make sure your DCL inherits the conditions from the root view entity of your BO. In this scenario, it inherits the conditions from entity `/DMO/R_TravelTP_CD`.

i Expand the following code sample to view the source code of the Access Control for the Draft Query View `/DMO/R_TRAVELTPDRAFT_CD`.

Sample Code

```
@EndUserText.label: 'Mapping role for /DMO/R_TravelTPDraft_CD'
@MappingRole: true
define role /DMO/R_TravelTPDraft_CD {
  grant
  select
  on
    /DMO/R_TravelTPDraft_CD
  where
    inheriting conditions from entity /DMO/R_TravelTP_CD;
}
```

4. Add draft action `Share` to the behavior definition of the base layer.

Sample Code

```
define behavior for /DMO/R_TravelTP_CD alias Travel
...
{
  ...
  draft action Activate optimized;
  draft action Discard;
  draft action Edit;
  draft action Resume;
  draft determine action Prepare;
  draft action Share;
}
```

The resulting base behavior definition looks like this:

i Expand the following code sample to view the source code of Behavior Definition `/DMO/R_TRAVELTP_CD`.

Sample Code

```
managed implementation in class /DMO/BP_R_TravelTP_CD unique;
strict ( 2 );
with collaborative draft;
extensible;
```

```

define behavior for /DMO/R_TravelTP_CD alias Travel
persistent table /dmo/a_travel_cd
extensible
draft table /dmo/d_travel_cd query /DMO/R_TravelTPDraft_CD
etag master LocalLastChangedAt
lock master total etag LastChangedAt
authorization master ( global, instance )
{
    field ( readonly )
    TravelUuid,
    LocalCreatedBy,
    LocalCreatedAt,
    LocalLastChangedBy,
    LocalLastChangedAt,
    LastChangedAt;

    field ( numbering : managed )
    TravelUuid;

    field ( readonly : update, mandatory : create )
    TravelId;

    create ( authorization : global );
    update;
    delete;

    draft action Activate optimized;
    draft action Discard;
    draft action Edit;
    draft action Resume;
    draft determine action Prepare;
    draft action Share;

    association _Booking { create; with draft; }

    mapping for /dmo/a_travel_cd corresponding extensible
    {
        BeginDate          = begin_date;
        BookingFee          = booking_fee;
        CurrencyCode        = currency_code;
        Description         = description;
        EndDate             = end_date;
        LastChangedAt       = last_changed_at;
        LocalCreatedAt      = local_created_at;
        LocalCreatedBy      = local_created_by;
        LocalLastChangedAt  = local_last_changed_at;
        LocalLastChangedBy  = local_last_changed_by;
        TravelId            = travel_id;
        TravelUUID          = travel_uuid;
    }
}

define behavior for /DMO/R_BookingTP_CD alias Booking

```

```

persistent table /dmo/a_book_cd
extensible
draft table /dmo/d_book_cd
etag dependent by _Travel
lock dependent by _Travel
authorization dependent by _Travel
{
  field ( readonly )
  BookingUuid,
  TravelUuid;

  field ( numbering : managed )
  BookingUuid;

  field ( readonly : update, mandatory : create )
  BookingId;

  update;
  delete;

  association _Travel { with draft; }

  mapping for /dmo/a_book_cd corresponding extensible
  {
    BookingDate = booking_date;
    BookingId   = booking_id;
    BookingUUID = booking_uuid;
    CurrencyCode = currency_code;
    FlightDate  = flight_date;
    FlightPrice = flight_price;
    TravelUUID  = parent_uuid;
  }
}

```

5. In the header of the behavior definition of the projection layer, add the syntax element `use collaborative draft`. In case you have used an interface layer, you must also use the statement there.

i Note

If you use an already draft-enabled BO with exclusive draft, change the syntax element `use draft` to `use collaborative draft`.

Sample Code

```

projection implementation in class /DMO/BP_C_TravelTP_CD unique;
strict ( 2 );
extensible;
use collaborative draft;
use side effects;
{
  ...
}

```

i Note

Initially, a warning occurs because you haven't declared the usage of the draft action `Share` yet.

6. Add the statement `use action Share` to the projection layer of the behavior definition. In case you have used an interface layer, you must also use the statement there.

Sample Code

```
define behavior for /DMO/C_TravelTP_CD alias Travel
...
{
...
use action Activate;
use action Discard;
use action Edit;
use action Resume;
use action Prepare;
use action Share;
}
```

The resulting projection behavior definition looks like this:

i *Expand the following code sample to view the source code of Behavior Definition /DMO/C_TRAVELTP_CD.*

Sample Code

```
projection implementation in class /DMO/BP_C_TravelTP_CD unique;
strict ( 2 );
extensible;
use collaborative draft;
use side effects;

define behavior for /DMO/C_TravelTP_CD alias Travel
extensible
use etag
{
    use create;
    use update;
    use delete;

    use action Activate;
    use action Discard;
    use action Edit;
    use action Resume;
    use action Prepare;
    use action Share;

    use association _Booking { create; with draft; }
}

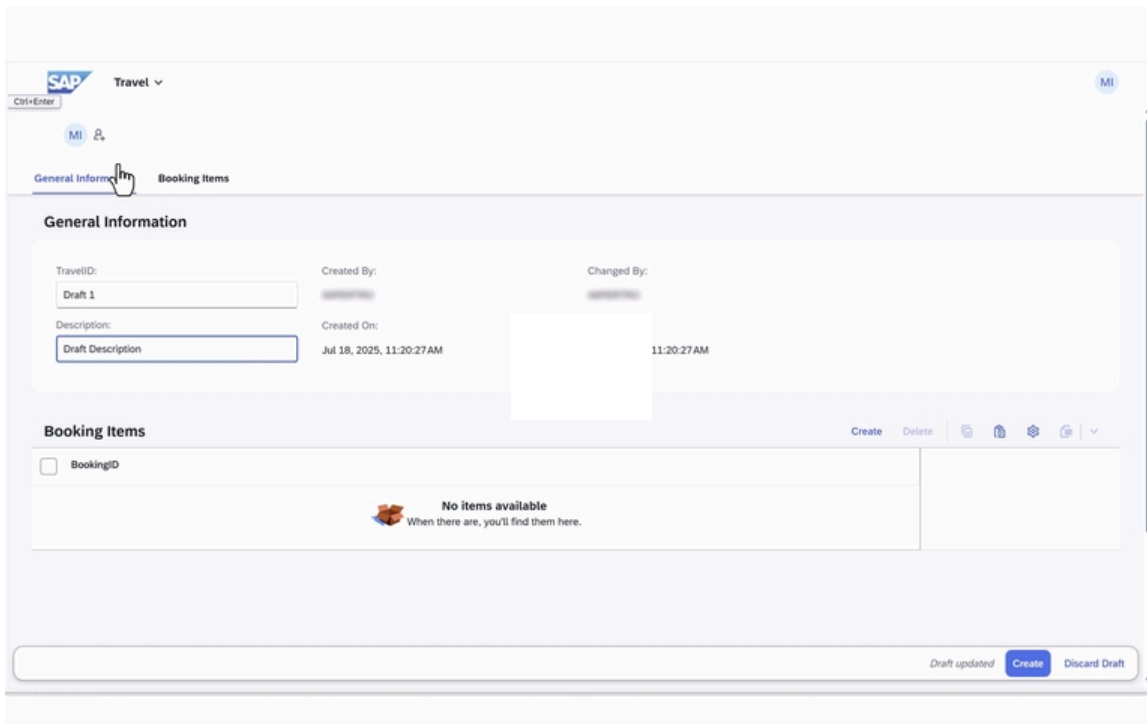
define behavior for /DMO/C_BookingTP_CD alias Booking
extensible
use etag
{
    use update;
    use delete;

    use association _Travel { with draft; }
}
```

Result

As soon as you activate all your artifacts, the collaborative draft capability is enabled for your BO. A business user who has created a draft instance can now invite other business users to the draft instance with the aid of a user list. The business user can access the user list by navigating from the list report into the draft instance and clicking the add user icon.

Video of a User Invitation with the Aid of the User List



[Open this video in a new window](#)

Related Information

[Collaborative Draft Handling](#)

Developing Cross-BO Scenarios with Draft Scope

Using cross-BO draft enablement, you can edit multiple related business objects as a single logical unit of work (LUW), ensuring consistency across related draft instances. In this topic, you will learn how to add a draft scope to two separate, draft enabled BOs.

Prerequisites

- You need two independent, draft enabled BOs. The two BOs should be separate from each other and fully functional. For more information on how to develop a draft enabled BO, see: [Developing Transactional Apps with Exclusive Draft Capabilities](#).
- In a cross-BO scenario, one of the BOs must hold the keys of the other BO as the foreign key. For more information, see: [Foreign Key Relationship](#).

Procedure

In the ABAP flight reference scenario, you can find the package /DM0/FLIGHT_XB0. It includes a cross BO scenario with the two independent, draft enabled BOs Agency and Travel.

For the Travel BO, we show two different use cases in this cross-BO scenario, using two separate consumption layers:

- /DM0/C_TRAVELTP_XB0 is the conventional consumption layer. In this case, the Travel BO is independent from the Agency BO.

- /DMO/C_TRAVELTP_XBO_DRAFTDP is defined as an additional draft scope enabled consumption layer. In this case, Travel is the **dependent draft** BO, with Agency as the **principal draft** BO

→ Tip

The database table /DMO/A_AGENCY_XBO uses `agency_uuid` as the key field, which is automatically filled with a UUID when an instance is created. The field `agency_id` serves as a semantic (human readable) id field.

Similarly, the database table /DMO/A_TRVL_XBO uses the field `travel_uuid` as key field, with the field `travel_id` as the semantic id field. Moreover, it holds the keys of the Agency BO instances in the field `agency_uuid`

1. In the Agency data definition on the R-layer (/DMO/R_AGENCYTP_XBO), create and expose a to-many association (`_Travel`) from Agency to Travel, using the AgencyUUID as key.

Sample Code

```
define root view entity /DMO/R_AGENCYTP_XBO
as select from /dmo/a_agency_xbo
association to many /DMO/R_TRAVELTP_XBO as _Travel on $projection.AgencyUUID = _Travel.AgencyUUID
{
  key agency_uuid as AgencyUUID,
  ...
  /* Associations */
  _Travel
}
```

2. Likewise, in the Travel data definition (/DMO/R_TRAVELTP_XBO), create and expose the association (`_Agency`) of many-to-one from Travel to Agency.

→ Tip

If you use the RAP BO generator, you might have to add the foreign key field `agencyUUID` to the Travel BO manually.

Sample Code

```
define root view entity /DMO/R_TRAVELTP_XBO
as select from /dmo/a_trvl_xbo
association to one /DMO/R_AGENCYTP_XBO as _Agency on $projection.AgencyUUID = _Agency.AgencyUUID
{
  key travel_uuid as TravelUUID,
  travel_id as TravelId,
  agency_uuid as AgencyUuid,
  ...
  /* Associations */
  _Agency
}
```

3. In the behavior definition of the dependent draft BO (/DMO/R_TRAVELTP_XBO), do the following:

- Add the statement with `cross associations;` to the header section of the BDEF.
- Declare the association `_Agency`, including the additions with `draft`, `link action`, `unlink action` and `inverse function`.
- If necessary, add a draft query view. For more information, see: [Draft Query Views](#)

i Note

For more information on cross-BO associations, refer to [Cross-BO Associations](#).

Sample Code

```
managed implementation in class /dmo/bp_r_traveltp_xbo unique;
strict ( 2 );
extensible;
with draft;

with cross associations;

define behavior for /DMO/R_TRAVELTP_XBO alias Travel
persistent table /dmo/a_trvl_xbo
extensible
draft table /dmo/d_trvl_xbo query /DMO/R_TravelDraft_XBO

...

association _Agency {
  with draft;
  link action linkAction;
  unlink action unlinkAction;
  inverse function inverseFunction;
}
```

4. In the behavior definition of the principal draft BO (/DMO/R_AGENCYTP_XBO), declare the association `_Travel` as a reverse association of `_Agency`, including the additions `create` and `with draft`.

→ Tip

It is recommended to add the statement “with cross associations” in the behavior definition header for every managed BO in a cross-BO scenario.

Sample Code

```
...
with cross associations;
...
association _Travel using /DMO/R_TRAVELTP_XBO \_Agency {
  create;
  with draft;
}
```

5. Create a projection view for the principal draft BO on the C-layer (/DMO/C_AGENCYTP_XBO). Expose the association to the dependent draft BO (`_Travel`) and redirect it to the projection view of the dependent draft BO, which you are going to create in the next step. For this reason, don't activate the principal draft BO projection view yet.

Sample Code

```
define root view entity /DMO/C_AGENCYTP_XBO
  provider contract transactional_query
  as projection on /DMO/R_AGENCYTP_XBO
{
  key AgencyUUID,

  ...

  /* Associations */
  _Travel: redirected to /DMO/C_TRAVELTP_XBO_DRAFTDP
}
```

6. Create a projection view for the dependent draft BO on the C-layer /DMO/C_TRAVELTP_XB0_DRAFTDP. Expose the association to the principal draft BO (_Agency) and redirect it to the projection view of the principal draft BO, which you have created in the previous step.

Sample Code

```
define root view entity /DMO/C_TRAVELTP_XB0
  provider contract transactional_query
  as projection on /DMO/R_TRAVELTP_XB0
{
  key TravelUUID,

  ...

  /* Associations */
  _Agency: redirected to /DMO/C_AGENCYTP_XB0
}
```

7. Activate both projection views.

8. In the C-layer BDEF of the dependent draft BO (/DMO/C_TRAVELTP_XB0_DRAFTDP), add `use draft as dependent;` to the header section.

In the behavior body, use the association to the principal draft BO, and give it draft capability (`use association _Agency { with draft; }`).

→ Tip

If existing, remove the statement `use draft;` from the header section, as well as existing draft actions.

Sample Code

```
projection;
strict ( 2 );
extensible;
use side effects;
use draft as dependent;
//use draft;

define behavior for /DMO/C_TRAVELTP_XB0_DRAFTDP alias Travel
extensible
use etag
{
  use create;
  use update;
  use delete;

  // use action Activate;
  // use action Discard;
  // use action Edit;
  // use action Resume;
  // use action Prepare;

  use association _Agency { with draft; }
}
```

9. In the C-layer BDEF of the principal draft BO (/DMO/C_AGENCYTP_XB0), use the association to the dependent draft BO (_Travel), and give it the additions `create;` and `with dependent draft;`.

→ Tip

Make sure that the C-layer BDEF of the principal draft BO has use `draft`; in the header and all necessary draft actions.

Sample Code

```
projection;
strict ( 2 );
extensible;
use side effects;
use draft;

define behavior for /DMO/C_AGENCYTP_XBO alias Agency
extensible
use etag
{
    use create;
    use update;
    use delete;

    use action Activate;
    use action Discard;
    use action Edit;
    use action Resume;
    use action Prepare;

    use association _Travel {
        create;
        with dependent draft;
    }
}
```

10. In the metadata extension of the projection view of the principal draft BO `/DMO/C_AGENCYTP_XBO`, add the `@UI.facet` annotation for the association directing to the dependent draft BO (`_Travel`). This ensures that the items of the dependent draft BO are displayed on the object page of the principal draft BO.

Sample Code

```
annotate view /DMO/C_AGENCYTP_XBO with

{
    @UI.facet: [ ...
        { id: 'Travel',
          purpose: #STANDARD,
          type: #LINEITEM_REFERENCE,
          label: 'Travel',
          position: 40,
          targetElement: '_Travel' } ]
}
```

11. In the service definition of the principal draft BO (`/DMO/UI_AGENCYTP_04_XBO`), expose the dependent draft BO as well.

Sample Code

```
define service /DMO/UI_AGENCYTP_04_XBO
    provider contracts odata_v4_ui {
        expose /DMO/C_AGENCYTP_XBO as Agency;
        expose /DMO/C_TRAVELTP_XBO_DRAFTDP as Travel;
    }
```

Result

When an instance of the principal draft BO (Agency) is set to edit mode and thus set to draft, all related instances of the dependent draft BO (Travel) are set to draft as well. Moreover, when an instance of the dependent draft BO is set to draft, all of its

Related Information

[Draft Scope Definition in Cross-BO Scenarios](#)

Developing Apps with Hierarchical Data Structures

The following topics explain how to use CDS hierarchies in the context of the ABAP RESTful Application Programming Model (RAP), and expose the hierarchical structures as treeviews on the SAP Fiori UI.

ABAP CDS provides means to query hierarchical data stored in database tables. A typical example of a hierarchical data set connected by parent-child relationships is the organizational structure of a company. The root node of this hierarchy is the CEO, because the CEO doesn't have a parent node, that is, a superior employee. Each employee is represented by a hierarchy node, and the hierarchy edges represent the relationships between the employees, that is, manager – subordinate relationships or peer relationships of employees at the same level.

The SAP HANA database provides built-in functions for handling hierarchical data. ABAP SQL and ABAP CDS wrap these built-in functions so that you can use them directly in ABAP without any additional implementation. The SAP HANA hierarchy functions are described in [Hierarchy Functions](#).

You can implement treeviews with the following two capabilities:

Treeviews with Read-Only Capabilities

You can display the hierarchy representation on the UI. For conceptual information on treeviews, see [Treeviews](#).

To implement read-only treeviews, see [Treeviews with Read-Only Capabilities](#).

Editable Treeviews with Draft Capabilities

You can display and edit the hierarchy representation on the UI directly by using, for example, drag and drop or cut and paste. For conceptual information on treeviews, see [Treeviews](#).

To implement editable treeviews in your managed with draft business object, see [Editable Treeviews: Development Process in Overview](#).

CDS Hierarchy Generation Wizard

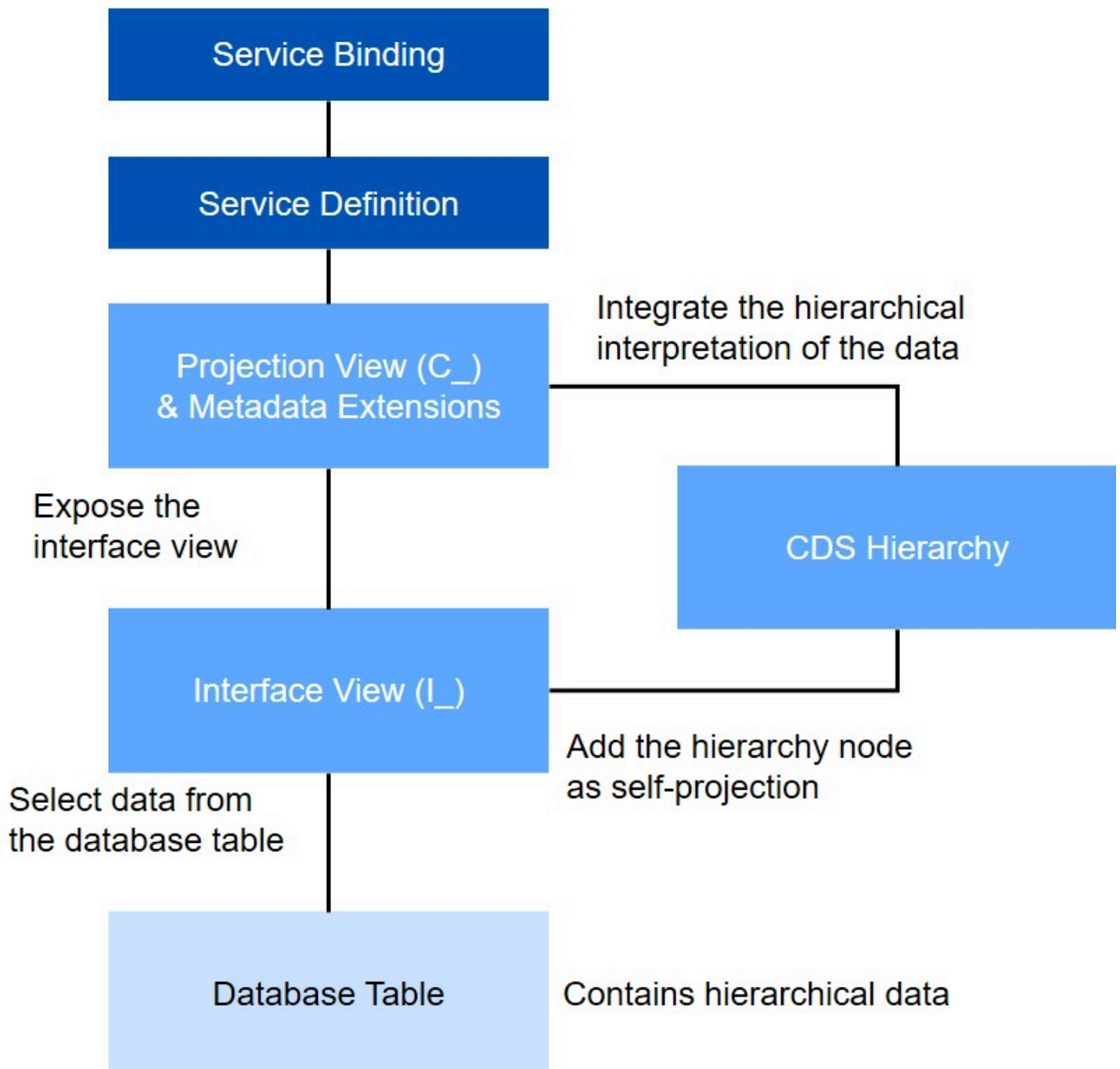
The CDS Hierarchy Generation Wizard simplifies the creation of hierarchically-structured data in ABAP CDS by generating all necessary objects and allowing customization of the hierarchy's structure and properties. For more information, see [CDS Hierarchy Generation Wizard](#).

Treeviews with Read-Only Capabilities

In read-only mode, the following artifacts work together to define the hierarchical structure of the data and expose it to the UI.

1. Database table (/DMO/employee_hr): This is the underlying database table where the data is physically stored.
2. Interface view (/DMO/I_Employee_HR): This is the first layer, referring to the underlying database table. It contains a self-association to that same CDS view.

3. CDS hierarchy (/DMO/I_EmployeeHN_HR): This artifact adds the hierarchical interpretation to the data exposed by the interface view.
4. Projection view (/DMO/C_Employee_HR): The last entity is essentially what the front-end or application layer interacts with. It puts together the interface view with the hierarchical interpretation of the data using the annotation `@odata.hierarchy.recursiveHierarchy` that refers back to the hierarchy definition.
5. To complete the business object, you also need to create a service definition and a service binding to expose the selected data and its hierarchy to the UI.



Creating the Database Table

In this example, provided with the ABAP Flight Reference Scenario, we use the table `/dmo/employee_hr`. In this example table, each employee can also be a manager. Therefore, the employee ID (node ID) can also be a manager ID (parent node ID).

EMPLOYEE	FIRST_NAME	LAST_NAME	MANAGER
00000001	Friedrich	Meier	00000000
00000002	Johannes	Rahn	00000001
00000003	Lee	Neubasler	00000001
00000004	Holm	Illner	00000003
00000005	Max	Mustermann	00000004
00000006	Christoph	Mustermann	00000004
00000007	Max	Matthaeus	00000006
00000008	Thomas	Barth	00000006
00000009	Sophie	Rahn	00000006
00000010	Thomas	Mustermann	00000006

The column EMPLOYEE indicates the employee id for each employee. The column MANAGER indicates which employee is the manager for the respective person. As an example, employees 007-010 report to employee 006, Christoph Mustermann. Christoph Mustermann reports to employee 004, which is Holm Illner.

Employee 001, Friedrich Meier, is the head of the agency and thus has no manager above him. Therefore, his manager is indicated as zero.

Database Table /DMO/EMPLOYEE_HR

i Expand the following code sample to view the source code of database table /dmo/employee_hr.

Sample Code

```
@EndUserText.label : 'Hierarchy: Read Only: Employee'
@AbapCatalog.enhancement.category : #NOT_EXTENSIBLE
@AbapCatalog.tableCategory : #TRANSPARENT
@AbapCatalog.deliveryClass : #A
@AbapCatalog.dataMaintenance : #RESTRICTED
define table /dmo/employee_hr {

    key client      : abap.clnt not null;
    key employee    : /dmo/employee_id not null;
    first_name      : /dmo/first_name;
    last_name       : /dmo/last_name;
    @EndUserText.label : 'Salary'
    @Semantics.amount.currencyCode : '/dmo/employee_hr.salary_currency'
    salary          : abap.curr(21,2);
    salary_currency : /dmo/currency_code;
    manager         : /dmo/employee_id;

}
```

Creating the Interface View

The data definition /DMO/I_Employee_HR is the interface view for table /DMO/employee_hr.

In this view, you need to add a self-association to the parent node of that same CDS view. This adds the hierarchical logic to the existing data, indicating that an employee ID can also serve as the manager ID for another employee. Since one manager can have several employees, the cardinality is defined as **many to one**.

Sample Code

```
define view entity /DMO/I_Employee_HR
as select from /dmo/employee_hr
association of many to one /DMO/I_Employee_HR
as _Manager on $projection.Manager = _Manager.Employee
```

Interface View /DMO/I_Employee_HR

 *Expand the following code sample to view the source code of data definition /DMO/I_Employee_HR.*

Sample Code

```
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'Hierarchy: Read Only: Employee'

define view entity /DMO/I_Employee_HR
as select from /dmo/employee_hr
association of many to one /DMO/I_Employee_HR as _Manager on $projection.Manager = _Manager.Employee
{
  key employee      as Employee,
    first_name      as FirstName,
    last_name       as LastName,

    @Semantics.amount.currencyCode: 'SalaryCurrency'
    salary          as Salary,
    salary_currency as SalaryCurrency,

    @EndUserText.label: 'Manager'
    manager         as Manager,

    _Manager
}
```

Defining the Hierarchy Interpretation

Define how the hierarchical data is interpreted with a parent-child hierarchy in the artifact /DMO/I_EmployeeHN_HR:

Sample Code

```
define hierarchy /DMO/I_EmployeeHN_HR as parent child hierarchy
```

Refer to the hierarchical data /DMO/I_Employee_HR as a source:

Sample Code

```
source /DMO/I_Employee_HR child to parent association _Manager
```

The hierarchy typically starts where there is no manager above, which is the CEO. Therefore, the root node of the hierarchy is where the manager ID is zero (initial).

Sample Code

```
start where Manager is initial
```

In this example, siblings are employees who report to the same manager. You can sort siblings by their last name, for example:

Sample Code

```
siblings order by LastName ascending
```

Hierarchy /DMO/I_EmployeeHN_HR

i *Expand the following code sample to view the source code of data definition /DMO/I_EmployeeHN_HR.*

Sample Code

```
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'Hierarchy: Read Only: Employee Hierarchy'
```

```
define hierarchy /DMO/I_EmployeeHN_HR
as parent child hierarchy(
  source /DMO/I_Employee_HR
  child to parent association _Manager
  start where
    Manager is initial
  siblings order by
    LastName ascending
)
{
  key Employee,
    Manager
}
```

Creating the Projection View

In the projection view, the exposed data from the interface view gets connected with the hierarchical interpretation of the data from the hierarchy entity.

To connect the hierarchy, add the following annotation, pointing to the hierarchy entity:

```
@odata.hierarchy.recursiveHierarchy:[{ entity.name: '/DMO/I_EmployeeHN_HR' }]
```

Define the projection view as a projection on the interface view:

Sample Code

```
define view entity /DMO/C_Employee_HR
  as select from /DMO/I_Employee_HR

  association of many to one /DMO/C_Employee_HR as _Manager
  on $projection.Manager = _Manager.Employee
```

Consumption View /DMO/C_Employee_HR

 *Expand the following code sample to view the source code of data definition /DMO/C_Employee_HR.*

Sample Code

```
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'Hierarchy: Read Only: Employee'

@Metadata.allowExtensions: true
@Search.searchable: true

@OData.hierarchy.recursiveHierarchy:[{ entity.name: '/DMO/I_EmployeeHN_HR' }]

define view entity /DMO/C_Employee_HR
  as select from /DMO/I_Employee_HR

  association of many to one /DMO/C_Employee_HR as _Manager on $projection.Manager = _Manager.Em
{
  key Employee,

    @Search: {
      defaultSearchElement: true,
      fuzzinessThreshold: 0.87
    }
  FirstName,

    @Search: {
      defaultSearchElement: true,
      fuzzinessThreshold: 0.87
    }
  LastName,

  Salary,
  SalaryCurrency,
  Manager,
  /* Associations */
  _Manager
}
```

Creating the Business Services

To complete your RAP business object, add a service definition. In our example, this is `/DMO/UI_Employee_HR`.

i *Expand the following code sample to view the source code of data definition `/DMO/UI_Employee_HR`.*

Sample Code

```
@EndUserText.label: 'Hierarchy: Read Only: Employee'
@ObjectModel.leadingEntity.name: '/DMO/C_Employee_HR'
define service /DMO/UI_Employee_HR {
  expose /DMO/C_Employee_HR as Employee;
}
```

Moreover, you need to set up a Service Binding. In our example, this is `/DMO/UI_Employee_HR_04`.

For more information on how to create business services, see [Business Service](#).

Displaying the Treeview on the SAP Fiori UI

Create a metadata extension for the CDS projection view `/DMO/C_Employee_HR` for the display on the SAP Fiori UI.

Sample Code

```
@Metadata.layer: #CORE

@UI: { headerInfo: {
  typeName: 'Employee',
  typeNamePlural: 'Employees',
  title: { type: #STANDARD, value: 'Employee' }
},
presentationVariant: [{
  sortOrder: [{ by: 'Employee', direction: #ASC }],
  visualizations: [{type: #AS_LINEITEM}]
}]
}
annotate view /DMO/C_Employee_HR
with
{
  @UI: {
    facet: [
      { id: 'Employee',
        purpose: #STANDARD,
        type: #IDENTIFICATION_REFERENCE,
        label: 'Employee',
        position: 10
      }
    ]
  }

  @UI: {
    lineItem: [{ position: 20 }],
    identification: [{ position: 20 }]
  }
}
```


Employee;

```
@UI: {
  selectionField: [{ position: 30 }],
  lineItem:       [{ position: 30 }],
  identification: [{ position: 30 }]
}
```

FirstName;

```
@UI: {
  selectionField: [{ position: 30 }],
  lineItem:       [{ position: 40 }],
  identification: [{ position: 40 }]
}
```

LastName;

```
@UI: {
  lineItem:       [{ position: 50 }],
  identification: [{ position: 50 }]
}
```

Salary;

```
@UI: {
  lineItem:       [{ position: 70 }],
  identification: [{ position: 70 }]
}
```

Manager;

}

When exposed to the SAP Fiori UI, the hierarchical structure is displayed as a collapsible treeview.

Employees (39,595)						
☐	Employee	≡	First Name	Last Name	Manager	
☐	▼ 1		Friedrich	Meier		>
☐	2		Johannes	Rahn	1	>
☐	▼ 3		Lee	Neubasler	1	>
☐	▼ 4		Holm	Illner	3	>
☐	5		Max	Mustermann	4	>
☐	▼ 6		Christoph	Mustermann	4	>
☐	7		Max	Matthaeus	6	>
☐	8		Thomas	Barth	6	>
☐	9		Sophie	Rahn	6	>
☐	10		Thomas	Mustermann	6	>

Editable Treeviews with Draft Capabilities

In this chapter, you will be guided through all steps necessary to implement editable treeviews based on the implementation type managed with draft of the ABAP RESTful Application Programming Model (RAP).

Why and When to Use Editable Treeviews

In the context of the RAP, a treeview visualizes data modeled by a CDS hierarchy, for example, by using a SAP Fiori Elements treetable. If you have the business need to display or edit hierarchical relationships, such as those between employees and their respective managers, you can use RAP to efficiently create treeviews and to display them on your SAP Fiori User Interface (UI).

Example

A Use Case of Editable Treeviews

You can use editable treeviews to maintain the hierarchical relationships between employees and their respective managers in a company. You can simply make changes to your editable treeviews on the UI by:

- reassigning a hierarchy node to another parent node by using drag and drop or cut and paste.
- moving a hierarchy node to a dedicated position among its siblings.
- deleting a hierarchy node including its children.
- editing inline data on a tree node directly on the user interface. For example, in an HR hierarchy rendered as a treeview, you can change the names and other employee data directly on the UI.

For conceptual information of treeviews, see [Treeviews](#).

For more information on treeview features, see [Treeview Features](#).

Architecture Overview

For an architecture overview, see [Architecture Overview: Editable Treeview with Draft](#).

Essential to the implementation of editable treeviews are hierarchy directories, see [Hierarchy Directories](#) for details.

Development Guide

To implement editable treeviews in your managed with draft business object, see [Editable Treeviews: Development Process in Overview](#).

Related Information

[Treeviews](#)

[Treeviews: Architecture Overview](#)

[Editable Treeviews: Development Process in Overview](#)

[Treeview Features](#)

[Hierarchy Directories](#)

Editable Treeviews: Development Process in Overview

In this chapter, we are using the ABAP Flight Reference Scenario to demonstrate how you can implement editable treeviews in your managed with draft business object. The ABAP Flight Reference Scenario contains demo content that you can refer to and use to build your own sample implementations.

Development Process in Overview

To implement editable treeviews in your managed with draft business object, you need to perform the following activities:

1. [Creating the Database Tables](#)
2. [Creating the Interface View](#)
3. [Creating the Root View Entity for the Directory](#)
4. [Creating the View Entity for the Directory as Composition Child](#)
5. [Defining the Structure of the Hierarchy](#)
6. [Creating the Behavior Definition for the Directory](#)
7. [Defining the Behavior for the Hierarchy](#)
8. [Creating the Projection Layer and the Metadata Extension](#)
9. [Finalizing the Business Service](#)

Prerequisites

Developing the scenario that is described in the subsequent chapters requires the following:

- You fulfill the requirements described in [Prerequisites](#).
- You have installed ABAP Development Tools (ADT).

SAP recommends to use the latest version of the client installation. The ADT download is available on the update site <https://tools.hana.ondemand.com/>.

- To recreate the demo scenario, the **ABAP Flight Reference Scenario** must be available in your ABAP system. You can download the complete reference scenario from **GitHub**: [Downloading the ABAP Flight Reference Scenario](#).

→ Remember

The namespace `/DM0/` is reserved for the demo content. Apart from the downloaded ABAP Flight Scenario, do not use the namespace `/DM0/` and do not create any development objects in the downloaded packages.

You can access the development objects in `/DM0/` from your own namespace.

However, if you want to recreate all development objects of this demo content, make sure that you use different names from this documentation.

Related Information

[Managed Implementation Type](#)

Creating the Database Tables

As a first step, you have to create your database tables. In this demonstration, we are using the two active database tables /dmo/agency_a_hd and /dmo/employ_a_hd from the package /DMO/FLIGHT_HIERARCHY_DRAFT, which can be found after [Downloading the ABAP Flight Reference Scenario](#).

Below is a brief description of the two database tables.

Database Table /dmo/agency_a_hd

The /dmo/agency_a_hd database table lists out all agencies with its own unique agency id.

AGENCY	NAME	STREET	POSTAL_CODE	CITY	COUNTRY_CODE
004005	Your Choice	Gustav-Jung-Str. 425	90455	Nuernberg	DE
004006	Bella Italia	Via Marconi 123	00139	Roma	IT
004007	Hot Socks Travel	224 Balnagask Rd	8053	Sydney	AU
004008	Burns Nuclear	14 Science Park Drive	118228	Singapore	SG
004009	Honauer Reisen GmbH	Baumgarten 8	4212	Neumarkt	AT
004011	Voyager Enterprises	Gustavslundsvaegen 151	70563	Stockholm	SE
004012	Ben McCloskey Ltd.	74 Court Oak Rd	B17 9TN	Birmingham	GB
004013	Pillepalle Trips	Gorki Park 4	8008	Zuerich	CH
004014	Kangeroos	Lancaster drive 435	20001	London	GB
004015	Bavarian Castle	Pilnizerstr. 241	01069	Dresden	DE

Table 1. List View of Database Table /dmo/agency_a_hd.

Expand the following code sample to view the source code of database table /DMO/AGENCY_A_HD.

Sample Code

```
@EndUserText.label : 'Hierarchy: Draft: Agency'
@AbapCatalog.enhancement.category : #NOT_EXTENSIBLE
@AbapCatalog.tableCategory : #TRANSPARENT
@AbapCatalog.deliveryClass : #A
@AbapCatalog.dataMaintenance : #RESTRICTED
define table /dmo/agency_a_hd {

    key client          : abap.clnt not null;
    key agency          : /dmo/agency_id not null;
    name                : /dmo/agency_name;
    street              : /dmo/street;
    postal_code         : /dmo/postal_code;
    city                : /dmo/city;
    country_code        : land1;
    phone_number        : /dmo/phone_number;
    email_address       : /dmo/email_address;
    web_address         : /dmo/web_address;
    @AbapCatalog.anonymizedWhenDelivered : true
    local_created_by    : abp_creation_user;
    local_created_at    : abp_creation_tstmpl;
    @AbapCatalog.anonymizedWhenDelivered : true
    local_last_changed_by : abp_locinst_lastchange_user;
    local_last_changed_at : abp_locinst_lastchange_tstmpl;
    last_changed_at     : abp_lastchange_tstmpl;
```

}

Database Table /dmo/employ_a_hd

The column AGENCY in the below /dmo/employ_a_hd database table shows the agency id for each of its respective employee. Under one agency, there are many employees. Some of the employees could also have the role of a manager. In the below screenshot, all employees shown are working in the same agency (004001) and they all have the same manager (00000096), so they are on the same hierarchy level. If you want to, you can assign each of them with a sibling order number, so that they would appear in a specific order on a list. For example, Kurt Leisert is assigned with the sibling order number 007, whereas Peter Illner with 008, which is the subsequent number. As a result, Peter will always appear after Kurt when you sort the SIBLING_ORDER_NUMBER in an ascending order.

AGENCY	EMPLOYEE	FIRST_NAME	LAST_NAME	MANAGER	SIBLING_ORDER_NUMBER
004001	00000098	Sophie	Heller	00000096	002
004001	00000099	Pierre	Zimmermann	00000096	003
004001	00000100	Theresia	Prinz	00000096	004
004001	00000101	Allen	Jacqmain	00000096	005
004001	00000102	Ruth	Kramer	00000096	006
004001	00000103	Kurt	Leisert	00000096	007
004001	00000104	Peter	Illner	00000096	008
004001	00000105	Roland	Benjamin	00000096	009
004001	00000106	Mathilde	Buehler	00000096	010
004001	00000107	Chantal	Hoffen	00000096	011
004001	00000108	Friedrich	Wohl	00000096	012
004001	00000109	August	Hansmann	00000096	013
004001	00000110	August	Kreiss	00000096	014

Table 2. List View of Database Table /dmo/employ_a_hd.

Expand the following code sample to view the source code of database table /DMO/EMPLOY_A_HD.

Sample Code

```
@EndUserText.label : 'Hierarchy: Draft: Employee'
@AbapCatalog.enhancement.category : #NOT_EXTENSIBLE
@AbapCatalog.tableCategory : #TRANSPARENT
@AbapCatalog.deliveryClass : #A
@AbapCatalog.dataMaintenance : #RESTRICTED
define table /dmo/employ_a_hd {

    key client          : abap.clnt not null;
    key agency          : /dmo/agency_id not null;
    key employee        : /dmo/employee_id not null;
    first_name          : /dmo/first_name;
    last_name           : /dmo/last_name;
    @Semantics.amount.currencyCode : '/dmo/employ_a_hd.salary_currency'
    salary              : /dmo/salary;
    salary_currency      : /dmo/currency_code;
    manager             : /dmo/employee_id;
    sibling_order_number : /dmo/sibling_order_number;
    @AbapCatalog.anonymizedWhenDelivered : true
    local_created_by    : abp_creation_user;
```

```

    local_created_at      : abp_creation_tstmpl;
    @AbapCatalog.anonymizedWhenDelivered : true
    local_last_changed_by : abp_locinst_lastchange_user;
    local_last_changed_at : abp_locinst_lastchange_tstmpl;

}

```

Creating the Interface View

In our demonstration, we are using these two interface views `/DMO/I_Employee_HD` and `/DMO/I_Agency_HD`.

Interface view `/DMO/I_Agency_HD`

The data definition `/DMO/I_Agency_HD` is the interface view for table `/dmo/agency_a_hd`.

The association shown below has the cardinality of exact one to many. It implies that in each single agency, there can be many employees.

Sample Code

```
association of exact one to many /DMO/I_Employee_HD as _Employee on $projection.Agency = _Employ
```

Expand the following code sample to view the source code of data definition `/DMO/I_Agency_HD`.

Sample Code

```

@AccessControl.authorizationCheck: #NOT_REQUIRED
@endUserText.label: 'Hierarchy: Draft: Agency'

@Search.searchable: true

define root view entity /DMO/I_Agency_HD
  as select from /dmo/agency_a_hd as Agency

  association of exact one to many /DMO/I_Employee_HD      as _Employee on $projection.Agency = _E

  association of one      to one I_Country      as _Country on $projection.CountryCode

{
  @ObjectModel.text.element: ['Name']
  key Agency.agency          as Agency,

  @Search.defaultSearchElement: true
  @Search.fuzzinessThreshold: 0.8
  @Semantics.text: true
  @Semantics.organization.name: true
  Agency.name              as Name,

  @Semantics.address.street: true

```

```

Agency.street                as Street,

@Semantics.address.zipCode: true
Agency.postal_code          as PostalCode,

@Search.defaultSearchElement: true
@Semantics.address.city: true
Agency.city                 as City,

@Consumption.valueHelpDefinition: [{entity: { name: 'I_CountryVH', element: 'Country' } }]
@Semantics.address.country: true
Agency.country_code         as CountryCode,


@Semantics.telephone.type: [#WORK]
Agency.phone_number         as PhoneNumber,


@Semantics.eMail.address: true
Agency.email_address        as EMailAddress,

Agency.web_address          as WebAddress,


@Semantics.user.createdBy: true
Agency.local_created_by     as LocalCreatedBy,


@Semantics.systemDateTime.createdAt: true
Agency.local_created_at     as LocalCreatedAt,


@Semantics.user.lastChangedBy: true
Agency.local_last_changed_by as LocalLastChangedBy,


@Semantics.systemDateTime.localInstanceLastChangedAt: true
Agency.local_last_changed_at as LocalLastChangedAt,


@Semantics.systemDateTime.lastChangedAt: true
Agency.last_changed_at      as LastChangedAt,


/* Associations */
_Employee,
_Country
}

```

Interface view /DMO/I_Employee_HD

The data definition /DMO/I_Employee_HD is the interface view for table /dmo/employ_a_hd. It selects data from the existing database table /dmo/employ_a_hd.

In this view, there are several associations. The first association specifies a many-to-one relationship between employees and agencies. It indicates that many employees can be associated with one agency.

Sample Code

```
association of many to one /DMO/I_Agency_HD as _Agency on $projection.Agency = _Agency.Agen
```

The second association specifies also a many-to-one relationship but with the same entity view (/DMO/I_Employee_HD) itself, indicating that an employee can have a manager, who is also an employee. This adds the hierarchical logic to the existing data, indicating that an employee ID can also serve as the manager ID for another employee. Since many employees are associated with one manager, the cardinality of the below association is defined as many to one.

Sample Code

```
association of many to one /DMO/I_Employee_HD as _Manager on $projection.Agency = _Manager.Age
and $projection.Manager = _Manager.E
```

The _Employee association enables you to identify all employees reporting to the current manager.

Sample Code

```
association of many to many /DMO/I_Employee_HD as _Employee on $projection.Agency = _Employee
and $projection.Employee = _Employ
```

Expand the following code sample to view the source code of data definiton /DMO/I_Employee_HD.

Sample Code

```
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'Hierarchy: Draft: Employee'
```

```
define view entity /DMO/I_Employee_HD
as select from /dmo/employ_a_hd
```

```
association of many to one /DMO/I_Agency_HD as _Agency on $projection.Agency = _Agency.A
association of many to one /DMO/I_Employee_HD as _Manager on $projection.Agency = _Manager
and $projection.Manager = _Manager
association of many to many /DMO/I_Employee_HD as _Employee on $projection.Agency = _Employ
and $projection.Employee = _Employ
association to one I_Currency as _Currency on $projection.SalaryCurrency = _
{
  key agency as Agency,
  key employee as Employee,

  @Semantics.name.givenName: true
  first_name as FirstName,
  @Semantics.name.familyName: true
  last_name as LastName,

  @Semantics.amount.currencyCode: 'SalaryCurrency'
  salary as Salary,
  salary_currency as SalaryCurrency,

  @EndUserText.label: 'Manager'
  manager as Manager,

  sibling_order_number as SiblingOrderNumber,

  local_created_by as LocalCreatedBy,
  local_created_at as LocalCreatedAt,
  local_last_changed_by as LocalLastChangedBy,
  local_last_changed_at as LocalLastChangedAt,

  _Agency,
  _Manager,
```



```

    _Employee,
    _Currency
}

```

Creating the Root View Entity for the Directory

Create the transactional root view entity `/DMO/R_AgencyTP_HD` based on the interface view `/DMO/I_AGENCY_HD`.

To express the parent-child relationship between Agency and Employees, you have to define the composition. For more information on composition, see [Compositions](#). In our scenario, since under one agency, there are at least one or more hierarchical relationships between the employees, the cardinality of the composition is defined as exact one to many. This **exact one to many** specifies that one entry in the root view entity (`/DMO/R_AgencyTP_HD`) is associated with one or more entries in the child entity (`/DMO/R_EmployeeTP_HD`).

Sample Code

```
composition of exact one to many /DMO/R_EmployeeTP_HD as _Employee
```

Expand the following code sample to view the source code of data definition `/DMO/R_AgencyTP_HD`.

Sample Code

```

@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'Hierarchy: Draft: Agency'

define root view entity /DMO/R_AgencyTP_HD
  as select from /DMO/I_Agency_HD

  composition of exact one to many /DMO/R_EmployeeTP_HD as _Employee
{
  key Agency,
    Name,
    Street,
    PostalCode,
    City,
    CountryCode,
    PhoneNumber,
    EMailAddress,
    WebAddress,
    LocalCreatedBy,
    LocalCreatedAt,
    LocalLastChangedBy,
    LocalLastChangedAt,
    LastChangedAt,
    /* Associations */
    _Country,
    _Employee
}

```

Creating the View Entity for the Directory as Composition Child

Create the view entity `/DMO/R_EmployeeTP_HD` with the interface view `/DMO/I_Employee_HD` for the directory as compositional child. Here, you must define the to-parent association.

In our example, the association `_Agency` is used as the composition parent of the transactional hierarchy directory. The association `_Agency` connects each employee to their parent agency.

Sample Code

```
association          to parent /DM0/R_AgencyTP_HD as _Agency    on  $projection.Agency    = _Agency
```

Since several employees can be assigned to the same manager, the cardinality of the below association is defined as many to one.

Sample Code

```
association of many to one /DM0/R_EmployeeTP_HD as _Manager on $projection.Agency = _Manager
                        and $projection.Manager = _Manager
```

The third association has a many-to-many relationship. The `_Employee` association is the reverse association of the `_Manager` association. It carries precisely the mirrored association condition.

Sample Code

```
association of one to many /DM0/R_EmployeeTP_HD as _Employee on $projection.Agency = _Employee
                        and $projection.Employee = _Employee
```

i *Expand the following code sample to view the source code of data definition `/DM0/R_EmployeeTP_HD`.*

Sample Code

```
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'Hierarchy: Draft: Employee'
```

```
define view entity /DM0/R_EmployeeTP_HD
as select from /DM0/I_Employee_HD
```

```
association          to parent /DM0/R_AgencyTP_HD as _Agency    on  $projection.Agency = _Agency
association of many to one /DM0/R_EmployeeTP_HD as _Manager on $projection.Agency = _Manager
                        and $projection.Manager = _Manager
association of one to many /DM0/R_EmployeeTP_HD as _Employee on $projection.Agency = _Employee
                        and $projection.Employee = _Employee
association of one to one I_Currency          as _Currency on  $projection.SalaryCurrency =
```

```
{
```

```
key Agency,
key Employee,
  FirstName,
  LastName,
  @Semantics.amount.currencyCode: 'SalaryCurrency'
  Salary,
  SalaryCurrency,
  Manager,
  SiblingOrderNumber,
  LocalCreatedBy,
  LocalCreatedAt,
  LocalLastChangedBy,
  LocalLastChangedAt,
  /* Associations */
  _Agency,
  _Manager,
  _Employee,
  _Currency
```

```
}
```

Defining the Structure of the Hierarchy

Define the structure of the hierarchy in the data definition /DMO/I_EmployeeHN_HD.

In order for editable treeviews to work, hierarchies with directories are the only technically possible implementation. If you create an editable treeview without a directory, you will only get a read-only treeview whose tree nodes can be edited on their object pages, but none of the features to change the tree itself, such as drag and drop, and so on.

Whenever the hierarchy has a directory, the CDS syntax requires a filter condition which either uses a literal or a parameter. In our example, since the selection of the directory entry is variable, we are using a parameter.

Use the below parameter to allow the hierarchy to be filtered based on the agency ID.

Sample Code

```
define hierarchy /DMO/I_EmployeeHN_HD
  with parameters
    P_Agency : /dmo/agency_id
```

The parent-child hierarchy makes reference to the associations defined in the data definition DMO/I_Employee_HD.

Sample Code

```
as parent child hierarchy(
  source /DMO/I_Employee_HD
```

The child to parent association shown below indicates that the relationship is defined by the _Manager association. This association connects employees to their managers. For more information on child to parent association, see [Hierarchy Definition](#).

Sample Code

```
child to parent association _Manager
```

! Restriction

The hierarchy-to parent association must be a pure foreign key relation, that is, this association must precisely bind the hierarchy-parent primary key. For example, binding a non-primary semantic key is not supported.

The hierarchy directory of this example is defined by the association _Agency. Here, the directory filter works on the level of agency instances. As a result, the hierarchies shown in the treeview always contain only the employees from one selected single agency instance. Employees from multiple agency instances would never be mixed in one treeview. For more information, see [Hierarchy Directories](#).

Sample Code

```
directory _Agency filter by
  Agency = $parameters.P_Agency
```

In the HR hierarchy used here as an example, the treeview starts where there is no manager above. Therefore, the root node of the treeview is where the manager id is initial.

Sample Code

```
start where
    Manager is initial
```

You have the option to define the siblings order, for example, as ascending or descending. In our example below, it specifies that within the same hierarchical level, employees (siblings) should be ordered by the SiblingOrderNumber field in ascending order.

Sample Code

```
siblings order by
    SiblingOrderNumber ascending
```

You must include the key elements, and we also recommend you to include the foreign key elements of the hierarchy child-to-parent associations, but no further elements.

Sample Code

```
key Agency,
key Employee,
    Manager,
    SiblingOrderNumber
```

Expand the following code sample to view the source code of data definition /DMO/I_EmployeeHN_HD.

Sample Code

```
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'Hierarchy: Draft: Employee HierarchyNode'

define hierarchy /DMO/I_EmployeeHN_HD
with parameters
    P_Agency : /dmo/agency_id

as parent child hierarchy(
    source /DMO/I_Employee_HD

    child to parent association _Manager

    directory _Agency filter by
        Agency = $parameters.P_Agency

    start where
        Manager is initial

    siblings order by
        SiblingOrderNumber ascending
)
{
    key Agency,
    key Employee,
        Manager,
        SiblingOrderNumber
}
```

Related Information

Creating the Behavior Definition for the Directory

Next, create the behavior definition `/DMO/R_AgencyTP_HD` based on the root view entity `/DMO/R_AgencyTP_HD`.

This demonstration is based on the implementation type managed with draft. Therefore, here you have to use managed with draft.

≡ Sample Code

```
managed;
strict ( 2 );
with draft;
```

Expose the composition to the hierarchy entity with `create` if you want to allow insertion of hierarchy instances.

≡ Sample Code

```
association _Employee { create; with draft; }
```

Generating the Draft Database Tables

If not already done so, in the source code of behavior definition `/DMO/R_AgencyTP_HD`, you can use quick fix to generate the draft database tables `/dmo/agency_d_hd` and `/dmo/employ_d_hd`.

For further information on how to draft-enable your managed business object, see [Draft-Enabling the Managed Business Object](#).

→ Remember

Whenever you change the transactional model, you must regenerate the draft table to align them. The behavior definition offers a quick fix for regeneration, but it is the responsibility of the application developer to keep the draft table in sync with the transactional model. Bear in mind that everytime when a draft database table is regenerated, it might be necessary to adjust the database table for preserving the existing data.

Expand the following code sample to view the source code of behavior definition `/DMO/R_AGENCYTP_HD`.

≡ Sample Code

```
managed;
strict ( 2 );
with draft;

define behavior for /DMO/R_AgencyTP_HD alias Agency
implementation in class /dmo/bp_r_agencytp_hd unique
early numbering
persistent table /dmo/agency_a_hd
draft table /dmo/agency_d_hd
lock master
total etag LastChangedAt
authorization master ( none )
etag master LocalLastChangedAt
{
  create;
  update;
```

```

delete;

field ( readonly ) Agency, LocalCreatedBy, LocalCreatedAt, LocalLastChangedBy, LocalLastChange
field ( mandatory ) CountryCode, EMailAddress, Name;

association _Employee { create; with draft; }

draft action Resume;
draft action Edit;
draft action Activate optimized;
draft action Discard;

validation validateEMailAddress on save { create; field EMailAddress; }
validation validateCountryCode on save { create; field CountryCode; }
validation validateName on save { create; field Name; }

draft determine action Prepare
{
  validation validateEMailAddress;
  validation validateCountryCode;
  validation validateName;
  validation Employee~validationSalaryPositive;
  validation Employee~validationCurrencyExists;
  validation Employee~validationNameGiven;
}

mapping for /dmo/agency_a_hd
{
  Agency          = agency;
  Name             = name;
  Street           = street;
  PostalCode       = postal_code;
  City             = city;
  CountryCode      = country_code;
  PhoneNumber      = phone_number;
  EMailAddress     = email_address;
  WebAddress       = web_address;
  LocalCreatedBy   = local_created_by;
  LocalCreatedAt   = local_created_at;
  LocalLastChangedBy = local_last_changed_by;
  LocalLastChangedAt = local_last_changed_at;
  LastChangedAt    = last_changed_at;
}

[...]
```

Related Information

[Draft-Enabling the Managed Business Object](#)

Defining the Behavior for the Hierarchy

In the source code of behavior definition /DMO/R_AgencyTP_HD, declare the following actions:

The Link and Unlink Action

Declare a link and unlink action to enable the reassignments of child nodes to parent nodes. It allows you to make the reassignment, for example, by dragging an employee entry and dropping it off to a new manager on the UI.

☰ **Sample Code**

```

association _Manager
{
    with draft;
    link action linkManager;
    unlink action unlinkManager;
}

```

The Reorder Action

To enable moving a hierarchy node to a dedicated position among its siblings on the UI, you must declare a **reorder** action (either a managed or unmanaged one) in the behavior definition for the instance hierarchy.

For the managed reorder action, you must declare a **SiblingOrderNumber** field in the behavior definition **/DMO/R_AgencyTP_HD**. You must ensure that the value of this field matches that same single **SiblingOrderNumber** field in the CDS hierarchy **/DMO/I_EmployeeHN_HD**. Also pay attention that it must be a single field in the value of either **INT** or **NUMC**.

i Note

A newly created child node does not have a sibling order number by default. Sibling order numbers are only calculated by the RAP framework when the reorder action is performed.

When a node is assigned to another parent node, there can be duplicate sibling numbers until the reorder action is performed. To ensure that the sibling numbers are always unique and non-empty, you must implement a determination.

Moreover, for the managed reorder action, you must also declare an ascending association, which is the hierarchy-to-parent association and a corresponding reverse association as a descending association. Here, you have the option to add the declaration `{ with cascading delete; }`. With this declaration in place, whenever you delete a hierarchy node, the framework will also delete all its children.

Even though the cascading delete behavior is optional, the UI expects that the server cascades the deletion for subtrees and it doesn't behave consistently without the cascade as it doesn't send requests for multi-selected subtree nodes.

For more information, see [Treeview Features](#).

The Copy Action

You have the option to declare a copy action. The copy action determines how a hierarchy node can be duplicated in an editable treeview scenario. This action is useful when you want to create new hierarchy nodes based on an existing one in an editable treeview scenario. To implement a copy action, see [Implementing the Copy Action for the Hierarchy](#).

For conceptual information, see [Treeview Features](#).

≡ Sample Code

```

instance hierarchy /DMO/I_EmployeeHN_HD
{
    managed reorder action changeNextSibling;
    field ( hierarchy-index ) SiblingOrderNumber;
    ascending association _Manager;
    descending association _Employee { with cascading delete; }
    copy action copy;
}

```

Expand the following code sample to view the source code of behavior definition /DMO/R_AgencyTP_HD.

Sample Code

```
managed;
strict ( 2 );
with draft;

define behavior for /DMO/R_AgencyTP_HD alias Agency
[...]
```

```
{
  [...]
```

```
define behavior for /DMO/R_EmployeeTP_HD alias Employee
implementation in class /dmo/bp_r_employeetp_hd unique
early numbering
persistent table /dmo/employ_a_hd
draft table /dmo/employ_d_hd
lock dependent by _Agency
authorization dependent by _Agency
etag master LocalLastChangedAt
{
  update;
  delete;

  field ( readonly ) Agency, Employee, SiblingOrderNumber;
  field ( readonly ) Manager;

  association _Agency { with draft; }
  association _Manager
  {
    with draft;
    link action linkManager;
    unlink action unlinkManager;
  }

  association _Employee { with draft; }

  instance hierarchy /DMO/I_EmployeeHN_HD
  {
    managed reorder action changeNextSibling;
    field ( hierarchy-index ) SiblingOrderNumber;
    ascending association _Manager;
    descending association _Employee { with cascading delete; }
    copy action copy;
  }

  validation validationSalaryPositive on save { create; field Salary; }
  validation validationCurrencyExists on save { create; field SalaryCurrency; }
  validation validationNameGiven on save { create; field FirstName, LastName; }

  determine action validateSalaryPositive { validation validationSalaryPositive; }
  determine action validateCurrencyExists { validation validationCurrencyExists; }
  determine action validateNameGiven { validation validationNameGiven; }

  side effects
  {
    determine action validateSalaryPositive executed on field Salary affects messages;
    determine action validateCurrencyExists executed on field SalaryCurrency affects messages;
    determine action validateNameGiven executed on field FirstName, field LastName affects messa
  }

  mapping for /dmo/employ_a_hd
  {
    Agency          = agency;
    Employee        = employee;
    FirstName       = first_name;
    LastName        = last_name;
    Salary          = salary;
    SalaryCurrency  = salary_currency;
    Manager         = manager;
    SiblingOrderNumber = sibling_order_number;
    LocalCreatedBy  = local_created_by;
```



```

        LocalCreatedAt      = local_created_at;
        LocalLastChangedBy = local_last_changed_by;
        LocalLastChangedAt = local_last_changed_at;
    }

}

```

Related Information

[Treeview Features](#)

Implementing the Copy Action for the Hierarchy

The copy action determines how a hierarchy node can be duplicated in an editable treeview scenario. It is especially useful when you want to create new hierarchy nodes based on an existing one in an editable treeview scenario.

For conceptual information, see [Treeview Features](#).

For details on syntax, see [RAP - Copy Action \(ABAP Keyword Documentation\)](#).

Definition

In the source code of behavior definition /DMO/R_AgencyTP_HD, under instance hierarchy for employee, declare the copy action. Use the syntax `copy action copy;`.

Sample Code

```

instance hierarchy /DMO/I_EmployeeHN_HD
{
    managed reorder action changeNextSibling;
    field ( hierarchy-index ) SiblingOrderNumber;
    ascending association _Manager;
    descending association _Employee { with cascading delete; }
    copy action copy;
}

```

Via a quick fix, you can generate the method declaration for the copy action implementation in the behavior pool directly from the behavior definition editor.

Implementation

The following is an implementation example of a copy action. In this example, the implementation of the methods in the `lhc_hierarchy_operation` class handle hierarchical copy operations for employee data. Within this `lhc_hierarchy_operation` class, there are two methods, namely, `copy` and `_copy_recursive`.

Be aware that the `_copy_recursive` method makes use of a recursive copying logic to copy employee data, so that all the child nodes of an employee are also copied. However, a recursive copying logic is not a prerequisite for a copy action to take place. For instance, if you only want to copy the data of one single employee but not all the subordinates reporting to this person, you do not need to apply a recursive copying logic. You should always consider your own use case when deciding on how your implementation should look like.

The following steps guide you through the implementation.

1. Declare the `_copy_recursion` method to handle recursive operations. Import essential parameters for processing copy actions. Export relevant fields to categorize employees according to the operation's status.
2. Invoke the `_copy_recursion` method within the `copy` method.
3. Perform a read operation to get the information of employee instance to be copied.
4. Introduce a `LOOP AT` iteration over the keys of the instance to be copied and make sure to handle the case of invalid keys, such as, duplicates or other failures, by filling the parameters `failed` and `reported` accordingly.
5. Perform a read-by-association operation to gather data about employees and their links to agencies. Make sure to fill the `failed` parameter accordingly.
6. Loop across the table holding the directory links and specify the fields you want to copy.
7. If an employee has child nodes, make sure that you copy them as well.
8. Make sure to remove any children links in case `failed` is filled. Then, fill the `parents_copy_map` with values provided from the read-by-association.
9. Iterate this also for every child. Use recursive calls to copy child entities within hierarchy using `result_rba_children` and insert them into `parents_copy_map`.

Expand the following code sample to see the source code of the `copy` and the `_copy_recursive` methods.

Sample Code

```
CLASS lhc_hierarchy_operation DEFINITION FINAL INHERITING FROM cl_abap_behavior_handler.
  PRIVATE SECTION.
```

```
  TYPES:
```

```
    copy_keys      TYPE TABLE FOR ACTION IMPORT /DMO/R_EMPLOYEEETP_HD~copy,
    mapped         TYPE TABLE FOR MAPPED /DMO/R_EMPLOYEEETP_HD,
    failed         TYPE TABLE FOR FAILED /DMO/R_EMPLOYEEETP_HD,
    reported       TYPE TABLE FOR REPORTED EARLY /DMO/R_EMPLOYEEETP_HD,
    failed_agencies TYPE TABLE FOR FAILED /DMO/R_AGENCYTP_HD.
```

```
  TYPES BEGIN OF parent_copy_map.
```

```
    TYPES:
```

```
      agency      TYPE /DMO/R_AGENCYTP_HD-agency,
      employee    TYPE /DMO/R_EMPLOYEEETP_HD-employee,
      is_draft    TYPE abp_behv_flag,
      copy_employee TYPE /DMO/R_EMPLOYEEETP_HD-employee.
```

```
  TYPES END OF parent_copy_map.
```

```
  TYPES parents_copy_map TYPE SORTED TABLE OF parent_copy_map WITH UNIQUE KEY agency employee
```

```
  METHODS copy FOR MODIFY IMPORTING keys FOR ACTION employee~copy.
```

```
    "Declare the _copy_recursion method to handle recursive operations.
```

```
    "Import essential parameters for processing copy actions.
```

```
    "Export relevant fields to categorize employees according to the operation's status.
```

```
  METHODS _copy_recursion IMPORTING keys              TYPE copy_keys
                           parents_copy_map_param    TYPE parents_copy_map OPTIONAL
                           EXPORTING mapped_employees TYPE mapped
                           failed_employees          TYPE failed
                           reported_employees        TYPE reported.
```

ENDCLASS.

CLASS lhc_hierarchy_operation IMPLEMENTATION.

METHOD copy.

"Invoke the _copy_recursion method.

```
_copy_recursion( EXPORTING keys = keys
                  IMPORTING mapped_employees = mapped-employee
                          failed_employees = failed-employee
                          reported_employees = reported-employee ).
```

ENDMETHOD.

METHOD _copy_recursion.

CONSTANTS: my_cid_kid TYPE abp_behv_cid VALUE 'My%CID_KID' ##NO_TEXT.

DATA:

```
employees_cba    TYPE TABLE FOR CREATE /DMO/R_AGENCYTP_HD\_employee,
employees_rba    TYPE TABLE FOR READ IMPORT /DMO/R_EMPLOYEEETP_HD\_employee,
employee_rba     LIKE LINE OF employees_rba,
parents_copy_map TYPE parents_copy_map.
```

FIELD-SYMBOLS <control> TYPE x.

CLEAR: mapped_employees, failed_employees.

"Read to get the information of employee instance to be copied.

READ ENTITY IN LOCAL MODE /DMO/R_EMPLOYEEETP_HD

ALL FIELDS WITH CORRESPONDING #(keys)

RESULT DATA(result_sources)

REPORTED DATA(reported_source)

FAILED DATA(failed_source).

"Iterate over the keys of the instance to be copied and make sure to handle the case of invalid loop at keys INTO DATA(key).

READ TABLE keys ASSIGNING FIELD-SYMBOL(<dupl_key>) WITH KEY id COMPONENTS %tky = key-%tky

IF <dupl_key>-%cid <> key-%cid.

```
failed_source-employee = VALUE #( ( %fail    = VALUE #( cause = if_abap_behv=>cause
                                %cid    = key-%cid
                                Agency  = key-Agency
                                Employee = key-Employee ) ).
```

INSERT LINES OF failed_source-employee INTO TABLE failed_employees.

CLEAR failed_source.

APPEND VALUE #(%cid = key-%cid

%tky = key-%tky

%msg = new_message_with_text(severity = if_abap_behv_message=>severity

```

INSERT LINES OF reported_source-employee INTO TABLE reported_employees.
CLEAR reported_source.

```

```
ELSE.
```

```
READ TABLE result_sources ASSIGNING FIELD-SYMBOL(<source>) WITH KEY id COMPONENTS %tky =
```

```
IF sy-subrc <> 0.
```

```

failed_source-employee = VALUE #( ( %fail      = VALUE #( cause = if_abap_behv=>cause
                                   %cid       = key-%cid
                                   Agency     = key-Agency
                                   Employee   = key-Employee ) ).

```

```

INSERT LINES OF failed_source-employee INTO TABLE failed_employees.
CLEAR failed_source.

```

```

APPEND VALUE #( %cid      = key-%cid
                %tky      = key-%tky
                %msg      = new_message_with_text(
                severity = if_abap_behv_message=>severity-error ) ) TO reported_source

```

```

INSERT LINES OF reported_source-employee INTO TABLE reported_employees.
CLEAR reported_source.

```

```
ENDIF.
```

```
ENDIF.
```

```
ENDLOOP.
```

"Perform a read-by-association operation to gather data about employees and their links to a
 "Make sure to handle the filling of failed parameter accordingly.

```

READ ENTITY IN LOCAL MODE /DMO/R_EMPLOYEE_TP_HD
  BY \_agency
  FIELDS ( agency ) WITH CORRESPONDING #( result_sources )
LINK DATA(directory_links)
RESULT DATA(result_rba)
REPORTED DATA(reported_rba)
FAILED failed_source.

```

```
INSERT LINES OF failed_source-employee INTO TABLE failed_employees.
```

"Loop into the directory links and specify the fields you want to copy.
 LOOP AT directory_links ASSIGNING FIELD-SYMBOL(<directory_link>).

```
ASSIGN employees_cba[ KEY id %tky = <directory_link>-target-%tky ] TO FIELD-SYMBOL(<cba>)
```

```
IF sy-subrc <> 0.
```

```

  INSERT VALUE #( %tky = <directory_link>-target-%tky ) INTO TABLE employees_cba ASSIGNING
ENDIF.

```

```

ASSIGN result_sources[ KEY id %tky = <directory_link>-source-%tky ] TO FIELD-SYMBOL(<source>)
ASSERT sy-subrc = 0.

```

```

INSERT VALUE #( %data = CORRESPONDING #( <source_directory> ) ) INTO TABLE <cba>-%target .

IF parents_copy_map_param IS SUPPLIED.
    ASSIGN parents_copy_map_param[   agency    = <source_directory>-agency
                                   employee = <source_directory>-manager
                                   is_draft  = <source_directory>-%is_draft ] TO FIELD-SYMB

    ASSERT sy-subrc = 0.
    <cba_target>-manager = <parent_copy_map>-copy_employee.

ELSE.
    CLEAR <cba_target>-manager.
ENDIF.

employee_rba-%tky = <source_directory>-%tky.
<cba_target>-%cid = keys[ KEY id %tky = <source_directory>-%tky ]-%cid.
<cba_target>-%is_draft = <source_directory>-%is_draft.

" business logic: fields to be copied
<cba_target>-firstname      = <source_directory>-firstname.
<cba_target>-lastname       = <source_directory>-lastname.
<cba_target>-salary         = <source_directory>-salary.
<cba_target>-salarycurrency = <source_directory>-salarycurrency.
<cba_target>-siblingordernumber = <source_directory>-siblingordernumber.
" business logic end

ASSIGN <cba_target>-%control TO <control> CASTING.
CLEAR <control> WITH if_abap_behv=>mk-on IN BYTE MODE. "as long as there is no selective i
CLEAR: <cba_target>-%control-agency, <cba_target>-%control-employee.
INSERT employee_rba INTO TABLE employees_rba.

ENDLOOP.

READ ENTITY IN LOCAL MODE /DMO/R_EMPLOYEE_TP_HD
  BY \_employee FROM employees_rba
LINK DATA(children_links)
RESULT DATA(result_rba_children)
FAILED DATA(failed_rba_children)
REPORTED DATA(reported_rba_children).

"If an employee has child nodes, make sure to copy them as well.
MODIFY ENTITY IN LOCAL MODE /DMO/R_AGENCY_TP_HD
  CREATE BY \_employee FROM employees_cba
MAPPED DATA(mapped_create)
FAILED DATA(failed_create)
REPORTED DATA(reported_create).

"Make sure to remove any children links in case failed is filled.
"Then, fill the parents_copy_map with values provided from the read by association.
LOOP AT failed_create-employee ASSIGNING FIELD-SYMBOL(<failed>).
  DELETE children_links USING KEY draft WHERE source-%tky = <failed>-%tky.
ENDLOOP.

```

```

IF NOT parents_copy_map_param IS SUPPLIED.
    mapped_employees = mapped_create-employee.
ENDIF.

IF result_rba_children IS INITIAL.
    RETURN.
ENDIF.

LOOP AT employees_rba ASSIGNING FIELD-SYMBOL(<rba>).
    ASSIGN mapped_create-employee[ KEY cid COMPONENTS %cid = keys[ KEY id %tky = <rba>-%tky ] ]-
    '

    CHECK sy-subrc = 0.
    ASSERT <mapped>-agency      = <rba>-agency AND <mapped>-%is_draft = <rba>-%is_draft.

    INSERT VALUE #( agency      = <rba>-agency
                    employee    = <rba>-employee
                    is_draft    = <rba>-%is_draft
                    copy_employee = <mapped>-employee ) INTO TABLE parents_copy_map.
ENDLOOP.

"Iterate this also for every child. Use recursive calls to copy child entities within hierar
"using result_rba_children and insert them into parents_copy_map.
_copy_recursion( keys = VALUE #( FOR <child> IN result_rba_children INDEX INTO lv_index
                                ( %cid = my_cid_kid && lv_index %tky = <child>-%tky ) )
                parents_copy_map_param = parents_copy_map ).

ENDMETHOD.
ENDCLASS.

```

Related Information

[Treeview Features](#)

[Defining the Behavior for the Hierarchy](#)

Creating the Projection Layer and the Metadata Extension

Creating the Projection Layer

In the projection view, the exposed data from the base CDS view entity gets connected with the hierarchical interpretation of the data from the hierarchy entity.

To connect the hierarchy, add the following annotation in the data definition /DMO/C_EmployeeTP_HD :

```
@OData.hierarchy.recursiveHierarchy: [{ entity.name: '/DMO/I_EmployeeHN_HD' }]
```

Define the projection view as a projection on the root view entity (base CDS view entity) /DMO/R_EmployeeTP_HD:

Sample Code

```
define view entity /DMO/C_EmployeeTP_HD
  as projection on /DMO/R_EmployeeTP_HD
```

Define the projection view for view entity /DMO/R_AgencyTP_HD:

Sample Code

```
define root view entity /DMO/C_AgencyTP_HD
  provider contract transactional_query
  as projection on /DMO/R_AgencyTP_HD
```

i Expand the following code sample to view the source code of data definition /DMO/C_EmployeeTP_HD.

Sample Code

```
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'Hierarchy: Draft: Employee'

@Metadata.allowExtensions: true

@OData.hierarchy.recursiveHierarchy: [{ entity.name: '/DMO/I_EmployeeHN_HD' }]

@Search.searchable: true

define view entity /DMO/C_EmployeeTP_HD
  as projection on /DMO/R_EmployeeTP_HD

{
  @ObjectModel.text.element: ['AgencyName']
  key Agency,
  key Employee,

  _Agency.Name      as AgencyName,

  @Search.defaultSearchElement: true
  @Search.fuzzinessThreshold: 0.87
  FirstName,

  @Search.defaultSearchElement: true
  @Search.fuzzinessThreshold: 0.87
  LastName,
  Salary,
  @Consumption.valueHelpDefinition: [
    { entity: {
      name: 'I_CurrencyStdVH',
      element: 'Currency'
    },
    useForValidation: true
  }
]
  SalaryCurrency,
```

```

    @Consumption.valueHelpDefinition: [
      {
        entity: { name: '/DM0/I_Employee_ManagerVH_HD', element: 'Employee' },
        additionalBinding: [{ element: 'Agency', localElement: 'Agency', usage: #FILTER }]
      }
    ]
    Manager,
    _Manager.FirstName as ManagerFirstName,
    _Manager.LastName as ManagerLastName,
    SiblingOrderNumber,
    LocalLastChangedAt,

    /* Associations */
    _Agency : redirected to parent /DM0/C_AgencyTP_HD,
    _Manager : redirected to /DM0/C_EmployeeTP_HD,
    _Employee : redirected to /DM0/C_EmployeeTP_HD,
    _Currency

  }

```

Add projection in the behavior definition /DM0/C_AgencyTP_HD

Sample Code

```

projection;
strict ( 2 );
use draft;
use side effects;

```

Don't forget to reuse the reorder action with the syntax `use action changeNextSibling`. You must also specify the `link` and `unlink` action when you reuse the association `_Manager` with the syntax `use association _Manager`.

Sample Code

```

use action changeNextSibling;
use action copy;

use association _Agency { with draft; }
use association _Manager
{
  with draft;
  link action linkManager;
  unlink action unlinkManager;
}

```

Expand the following code sample to view the source code of behavior definition /DM0/C_AgencyTP_HD.

Sample Code


```

projection;
strict ( 2 );
use draft;
use side effects;

define behavior for /DMO/C_AgencyTP_HD alias Agency
use etag
{
    use create;
    use update;
    use delete;

    use action Resume;
    use action Edit;
    use action Activate;
    use action Discard;
    use action Prepare;

    use association _Employee { create; with draft; }
}

define behavior for /DMO/C_EmployeeTP_HD alias Employee
use etag
{
    use update;
    use delete;

    use action changeNextSibling;
    use action copy;

    use association _Agency { with draft; }
    use association _Manager
    {
        with draft;
        link action linkManager;
        unlink action unlinkManager;
    }
    use association _Employee { with draft; }
}

```

Creating the Metadata Extension

For the projection view, we are creating two metadata extensions /DMO/C_EmployeeTP_HD and /DMO/C_AgencyTP_HD for the display on the SAP Fiori UI.

Expand the following code sample to view the source code of metadata extension /DMO/C_EmployeeTP_HD.

Sample Code

```

@Metadata.layer: #CORE

@UI:
{
    headerInfo:
    {

```

```

        typeName: 'Employee',
        typeNamePlural: 'Employees',
        description: { type: #STANDARD, value: 'Employee' },
        title: { type: #AS_CONNECTED_FIELDS, valueQualifier: 'FullName' }
    },
    presentationVariant:
    [
        {
            sortOrder: [{ by: 'Employee', direction: #ASC }],
            visualizations: [{type: #AS_LINEITEM}]
        }
    ]
}

```

```

annotate view /DMO/C_EmployeeTP_HD with
{

```

```

@UI:
{
    facet:
    [
        {
            id: 'Employee',
            purpose: #STANDARD,
            type: #COLLECTION,
            label: 'Employee',
            position: 10
        },
        {
            id: 'General',
            targetQualifier: 'General_FG',
            parentId: 'Employee',
            purpose: #STANDARD,
            type: #FIELDGROUP_REFERENCE,
            label: 'General',
            position: 10
        },
        {
            id: 'Name',
            targetQualifier: 'Name_FG',
            parentId: 'Employee',
            purpose: #STANDARD,
            type: #FIELDGROUP_REFERENCE,
            label: 'Name',
            position: 20
        },
        {
            id: 'Salary',
            targetQualifier: 'Salary_FG',
            parentId: 'Employee',
            purpose: #STANDARD,
            type: #FIELDGROUP_REFERENCE,
            label: 'Salary Data',
            position: 30
        }
    ],
    hidden: true
}
Agency;

@UI: {
    lineItem: [{ position: 20 }],
    fieldGroup: [{ position: 20, qualifier: 'General_FG' }]
}
Employee;

@UI: {
    selectionField: [{ position: 30 }],
    lineItem: [{ position: 30 }],
    identification: [{ position: 30 }],
    fieldGroup: [{ position: 30, qualifier: 'Name_FG' }],
    connectedFields: [{
        qualifier: 'FullName',

```

```

        groupLabel: 'Full Name',
        name: 'First',
        template: '{First} {Last}'
    ]]
}
FirstName;

@UI: {
    lineItem: [{ position: 40 }],
    fieldGroup: [{ position: 40, qualifier: 'Name_FG' }],
    connectedFields: [{
        qualifier: 'FullName',
        name: 'Last'
    }]
}
LastName;

@UI: {
    lineItem: [{ position: 50 }],
    fieldGroup: [{ position: 50, qualifier: 'Salary_FG' }]
}
Salary;

@UI: {
    lineItem: [{ position: 70 }],
    fieldGroup: [{ position: 70, qualifier: 'General_FG', type: #AS_CONNECTED_FIELDS, valueQua
    connectedFields: [{
        qualifier: 'ManagerFullName',
        groupLabel: 'Manager',
        name: 'ID',
        template: '{ID} {First} {Last}'
    }]
}
Manager;

@UI: {
    connectedFields: [{
        qualifier: 'ManagerFullName',
        name: 'First'
    }]
}
ManagerFirstName;

@UI: {
    connectedFields: [{
        qualifier: 'ManagerFullName',
        name: 'Last'
    }]
}
ManagerLastName;

@UI: {
    lineItem: [{ position: 80 }],
    fieldGroup: [{ position: 80, qualifier: 'General_FG' }]
}
SiblingOrderNumber;

@UI.hidden: true
LocalLastChangedAt;
}

```

Expand the following code sample to view the source code of metadata extension /DMO/C_AgencyTP_HD.

Sample Code

```

@Metadata.layer: #CORE

@UI: { headerInfo: {
    typeName: 'Agency',

```

```

        typeNamePlural: 'Agencies',
        title: { type: #STANDARD, value: 'Name' },
        description: { type: #STANDARD, value: 'Agency' }
    },
    presentationVariant: [{
        sortOrder: [{ by: 'Agency', direction: #ASC }],
        visualizations: [{type: #AS_LINEITEM}]
    }]
}

```

annotate view /DM0/C_AgencyTP_HD with

```

{
    @UI.facet: [
        {
            id: 'Agency',
            purpose: #STANDARD,
            type: #COLLECTION,
            label: 'Agency',
            position: 10
        },
        {
            id: 'General',
            targetQualifier: 'General_FG',
            parentId: 'Agency',
            purpose: #STANDARD,
            type: #FIELDGROUP_REFERENCE,
            label: 'General',
            position: 10
        },
        {
            id: 'Address',
            targetQualifier: 'Address_FG',
            parentId: 'Agency',
            purpose: #STANDARD,
            type: #FIELDGROUP_REFERENCE,
            label: 'Address',
            position: 20
        },
        {
            id: 'Contact',
            targetQualifier: 'Contact_FG',
            parentId: 'Agency',
            purpose: #STANDARD,
            type: #FIELDGROUP_REFERENCE,
            label: 'Contact Data',
            position: 30
        },
        {
            id: 'Employee',
            purpose: #STANDARD,
            type: #LINEITEM_REFERENCE,
            label: 'Employee',

```

```

        position:      20,
        targetElement: '_Employee'
    }
]
@UI: { lineItem:      [ { position: 10, importance: #HIGH } ] ,
      selectionField: [ { position: 10 } ] }
@UI.textArrangement: #TEXT_SEPARATE
Agency;

@UI: { lineItem:      [ { position: 20, importance: #HIGH } ] ,
      fieldGroup:     [ { position: 20, qualifier: 'General_FG' } ],
      selectionField: [ { position: 20 } ] }
Name;

@UI: { fieldGroup:     [ { position: 30, qualifier: 'Address_FG' } ],
      identification: [ { position: 30 } ] }
Street;

@UI: { fieldGroup:     [ { position: 40, qualifier: 'Address_FG' } ],
      identification: [ { position: 40 } ] }
PostalCode;

@UI: { lineItem:      [ { position: 50, importance: #MEDIUM, type: #AS_FIELDGROUP, valueQuali
      identification: [ { position: 50 } ],
      fieldGroup:     [ { position: 50, qualifier: 'Address_FG' } ],
      selectionField: [ { position: 50 } ] }
City;

@UI: { identification: [ { position: 60 } ],
      fieldGroup:     [ { position: 60, qualifier: 'Address_FG' } ],
      selectionField: [ { position: 60 } ] }
@UI.textArrangement: #TEXT_FIRST
CountryCode;

@UI: { lineItem:      [ { position: 70, importance: #LOW, type: #AS_FIELDGROUP, valueQualifie
      fieldGroup:     [ { position: 70, qualifier: 'Contact_FG' } ],
      identification: [ { position: 70 } ] }
PhoneNumber;

@UI: { fieldGroup:     [ { position: 80, qualifier: 'Contact_FG' } ],
      identification: [ { position: 80 } ] }
EmailAddress;

@UI: { fieldGroup:     [ { position: 90, qualifier: 'Contact_FG' } ],
      identification: [ { position: 90 } ] }
WebAddress;

@UI.hidden: true
LocalLastChangedAt;
}

```

Finalizing the Business Service

To complete the service exposure, add a service definition. In our example, this is the service definition `/DMO/UI_Agency_HD`.

i Expand the following code sample to view the source code of service definition `/DMO/UI_Agency_HD`.

Sample Code

```
@EndUserText.label: 'Agency'
@ObjectModel.leadingEntity.name: '/DMO/C_AgencyTP_HD'
define service /DMO/UI_Agency_HD {
  expose /DMO/C_AgencyTP_HD as Agency;
  expose /DMO/C_EmployeeTP_HD as Employee;
  expose I_CurrencyStdVH as Currency;
}
```

Moreover, you need to set up a service binding. In our example, this is the service binding `/DMO/UI_Agency_HD_04`.

For more information on how to create business services, see [Business Service](#).

Finally, when exposed to the SAP Fiori UI, the hierarchical structure is displayed as a collapsible treeview.

Employees (39,593)						
Employee		First Name	Last Name		Salary	Manager
1	⌵	Friedrich	Meier	128,147.24	EUR	
2		Johannes	Rahn	59,133.76	EUR	1
3	⌵	Lee	Neubasler	90,975.40	EUR	1
4	⌵	Holm	Illner	79,575.07	EUR	3
168	⌵	Andreas	Martin	84,177.44	EUR	3
725	⌵	Felix	Benz	72,435.73	EUR	3
787	⌵	Georg	Benz	61,311.15	EUR	3
798	⌵	Anna	Babilon	76,002.62	EUR	3
799		Guenther	Neubasler	52,552.62	EUR	798
800	⌵	Johann	Wohl	69,110.67	EUR	798
815	⌵	Martin	Koslowski	66,392.76	EUR	798

Related Information

[Business Service](#)

Developing Transactional Apps with Multi-Inline-Edit Capabilities

The following section guides you through the implementation steps to create an application with multi-inline-edit capabilities.

Introduction

Multi-inline-edit capabilities are always necessary if you want the end user of a UI app to be able to edit, add, and delete all instances directly on a Fiori UI list report without navigating to the object page of one instance. It is always relevant for customizing and maintaining complete table data. Multi-inline-capabilities on the UI represent the same feature scope as transaction SM30 in SAP GUI.

The following preview displays a multi-inline-edit app for maintaining and administrating flight carrier information. All instances can be edited on the same screen in inline mode. New instances can be added or deleted via the create and delete button on the UI.

i *Expand the following item to view an animated GIF of Multi-Inline-Editing on a Fiori UI.*

Multi-Inline-Editing on a Fiori Elements UI

Multi-Inline-Editing on a Fiori UI

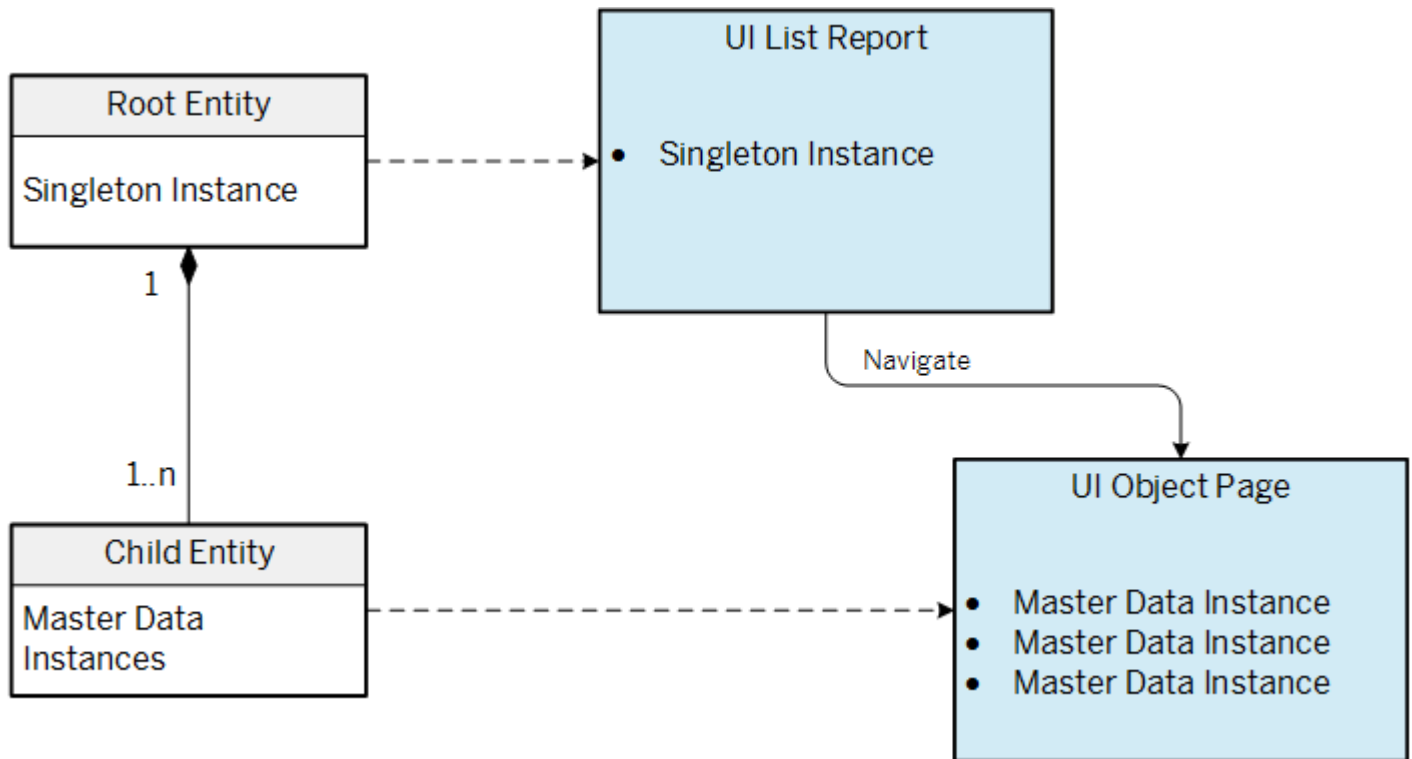
Technical Architecture of a Multi-Inline-Edit App

The technical requirements of a RAP BO that enables multi-inline-editing on a Fiori Elements UI app are defined by the structure of the Fiori Elements UI. A simple list reporting app consists of a list report page, on which all instances of the root view entity are displayed. From any instance, you can navigate to its object page, which displays all the instances of the child entity in a list, if the UI facet of the object page is modeled correctly. The list of child instances can always be edited in multi-inline-edit mode, which means by choosing **Edit** on the instance's object page, all fields of every child instance are editable at the same time. In this multi-inline-edit mode, you can change fields of different instances and then save the complete composition structure (root entity instance and all its child instances) at the same time. Since RAP BOs use a lock master/dependent mechanism, it is ensured that by choosing the **Edit** button, all instance of the child entity are locked.

The existing multi-inline-edit capabilities of the list of child instances are used to enable multi-inline-editing also for standalone master data entities. In this case, standalone refers to entities that are by default root entities in a RAP BO.

To enable multi-inline-editing also for default root view entities, RAP offers the solution to model a technical root entity with just one instance, from which you can navigate to its object page. This singleton pattern provides a single-entrance point to all master data instances. The technical root entity also provides the technical requirement of a lock master entity, which is essential for multi-inline-editing on the child entity. The master data entity is then modeled as a child entity in order to enable the standard multi-inline-edit capabilities of the object page of a UI. Like this, you can edit the master data of all instances at the same time.

In other words, for this scenario, you need to model a root entity with a single instance which has a composition to the master data entity that is editable as a list on the object page of the single root instance on the UI.



Multi-Inline-Scenario with Singleton Pattern in RAP BO

i Note

You cannot expose the same business object in a regular list report app and in a multi-inline-edit-enabled app at the same time. To expose a BO as a multi-inline-edit-scenario, a dummy root entity needs to be introduced and changes the common BO structure drastically.

Example Scenario

This example scenario develops an app to configure airline master data of the flight reference scenario. The master data contains the airline ID, the full name and the currency code that the airline uses. The data of all airlines shall be editable at the same time on one list.

The scenario is based on the database table `/dmo/carrier` and the interface view `/DMO/I_Carrier`. The transactional view `/DMO/I_Carrier_S` of the new business object selects from the interface view and thus, the new BO is built on it.

This development guide illustrates how a BO for a multi-inline-edit scenario is developed from scratch. It does not show development steps to migrate a regular business object into a business object for a multi-inline-edit scenario.

The example scenario can be downloaded into your system from Git Hub. You find the carrier multi-inline-edit scenario in the package `/DMO/FLIGHT_REAUSE_CARRIER`.

Prerequisites

Developing the scenario that is described in the subsequent chapters requires the following:

This is custom documentation. For more information, please visit [SAP Help Portal](#).

- You fulfill the requirements described in [Prerequisites](#).
- You have installed ABAP Development Tools (ADT).

SAP recommends to use the latest version of the client installation. The ADT download is available on the update site <https://tools.hana.ondemand.com/>.

- To recreate the demo scenario, the **ABAP Flight Reference Scenario** must be available in your ABAP system. You can download the complete reference scenario from **GitHub**: [Downloading the ABAP Flight Reference Scenario](#).

Development Steps

[Defining the Data Model and the Business Object Structure](#)

The multi-inline-edit scenario uses flight carrier master data of the ABAP Flight Reference Scenario. The final app processes carrier data in multi-inline-edit mode.

[Developing Transactional Behavior](#)

The multi-inline-edit scenario implements some standard behavior for draft BOs and additional specific behavior that is required to enable multi-inline-editing on the UI.

[Projecting the BO for UI Exposure](#)

For the multi-inline-edit scenario, create a business object projection for UI exposure.

[Defining and Publishing a UI Service with Multi-Inline-Edit Capabilities](#)

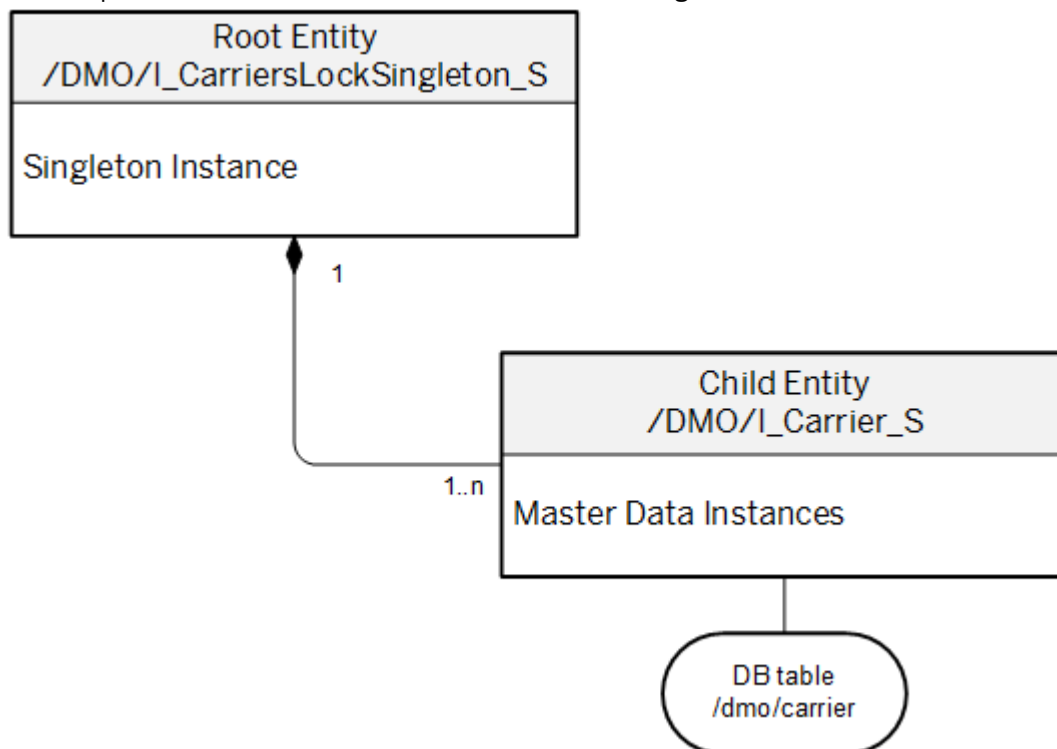
For the multi-inline-edit, define a UI service and bind it to a protocol.

Defining the Data Model and the Business Object Structure

The multi-inline-edit scenario uses flight carrier master data of the ABAP Flight Reference Scenario. The final app processes carrier data in multi-inline-edit mode.

The RAP business object in this scenario consists of two BO entities. The root entity represents the singleton entry point, whereas the first child entity comprises the general carrier data, like an ID, the full name and the currency code that is used by the airline.

The composition tree of the Carrier BO looks like the following.



Carrier Business Object with Singleton Root Entity

The root entity `/DM0/I_CarriersLockSingleton_S` is a technical artifact to ensure the single entrance point to the list of carrier data and the lock master for all carrier instances. It always retrieves only one dummy instance that has no relevance for any data of the BO. The real data substance of the BO lies in the child entity `/DM0/I_Carrier_S`. This CDS view selects from the database table `/dmo/carrier`, where the data is stored.

Tasks for the Developer

- [Creating the Singleton Root Entity](#)
- [Creating the Master Data Entity](#)

Creating the Singleton Root Entity

For the multi-inline-edit scenario, create a data definition for a CDS view entity that defines the singleton pattern for the BO.

Context

The multi-inline-edit scenario requires a singleton root entity with exactly one instance that represents the single entrance point for the carrier data list on the UI.

The singleton pattern is implemented in a RAP BO by one simple trick: You model a root entity that selects from an always existing CDS view and restrict the selection result with a where clause to ensure that there is always only one single result. Such a CDS view can be the released CDS view `I_Language`, which is always available in your SAP system. By restricting the selection result to the session language, you receive the required result. There is always only one session language, which is by default always available in the select list of the CDS view `I_Language`. The actual instance that is modeled in the root CDS view entity does not contain elements of the underlying CDS view `I_Language`, but defines a key element with a dummy value and other elements that are relevant for the draft processing. It is important to point out that the underlying CDS view `I_Language` does not have any relevance for the scenario. It just ensures the singleton mechanism for a RAP business object.

Procedure

1. Create a data definition for a CDS view entity to model the singleton pattern for the BO.

In ADT, use the wizard for creating CDS data definitions.

Select the template **Define Root View Entity** for the singleton entity and enter the name and a description for the view entity.

The CDS view entity `/DM0/I_CarriersLockSingleton_S` uses the CDS view `I_Language` as a data source. This CDS view is released for every system and can be used for any RAP BO. The language data is not used in the singleton entity.

2. Model the root view entity as a singleton entity to receive a single entrance point for the UI.

Restrict the selection list by the system language with a where clause. This ensures that there is always only one instance that is retrieved for requests on this entity.

Define a dummy field as key for the instance. You can use any hard-coded value.

3. Define a total ETag field to enable draft capabilities for the BO.

In this scenario, the total ETag in the root view is determined by the maximum value of all its child instances. The child carrier instances use the field `last_changed_at` as ETag fields. Whenever an instance is updated the field receives a new value and the total ETag is updated.

To get access to the data of the child instance, model a left outer join with the database table of the child instances `/dmo/carrier`. The join must use a dummy join condition as there is no field on the left side that matches any field on the right side of the join. By using a dummy join condition such as `on 0 = 0`, every instance (there is just one) receives the fields that are selected from the joined data source.

Add the maximum of the field `last_changed_at` of the carrier database table to the select list of the root view entity.

4. Define the composition structure of the carrier BO.

Once the CDS view entity of the child entity `/DMO/I_Carrier_S` exists, define the composition to is and use the association in the select list.

i Expand the following code sample to view the source code of CDS root view entity `/DMO/I_CarriersLockSingleton_S`.

Sample Code

```
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'Carrier Singleton Root View'
@Semantics.valueRange.maximum: '1'

define root view entity /DMO/I_CarriersLockSingleton_S
as select from I_Language
    left outer join /dmo/carrier as carr on 0 = 0
    composition [0..*] of /DMO/I_Carrier_S as _Airline
{
    key 1
        as CarrierSingletonID,
        max (carr.last_changed_at) as LastChangedAtMax,
        _Airline
}

where
    I_Language.Language = $session.system_language
```

Creating the Master Data Entity

For the multi-inline-edit scenario, create a data definition for a CDS view entity that selects the carrier master data.

Context

The multi-inline-edit scenario requires that the master data entity is modeled as a child entity of a root entity that enables a singleton mechanism.

Procedure

1. Create a data definition for a CDS view entity to model the carrier master data.

In ADT, use the wizard for creating CDS data definitions.

Select the template **Define View Entity with To-Parent Association** for the child entity and enter the name and a description for the view entity.

The CDS view entity `/DMO/I_Carrier_S` uses the database table `/dmo/carrier` as data source. All fields are selected.

2. Define the association to the parent view entity.

Include a dummy field with the same hard-coded value as the key field in the root view entity in the select list. This field is used as the on-condition of the association.

3. Use the `@Semantics` annotations that automatically update the administrative fields in the managed scenario.
4. Suppress search and text annotations that are inherits from the underlying CDS view by using the header annotation `@Metadata.ignorePropagatedAnnotations: true`.

i Expand the following code sample to view the source code of CDS view entity `/DMO/I_Carrier_S`.

Sample Code

```
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'Transactional Carrier View'
@Metadata.ignorePropagatedAnnotations: true

define view entity /DMO/I_Carrier_S
as select from /DMO/I_Carrier as Airline
association          to parent /DMO/I_CarriersLockSingleton_S as _CarrierSingleton on $projectio

{
  key AirlineID,
    1 as CarrierSingletonID,

    Name,

    CurrencyCode,

    @Semantics.user.createdBy: true
    LocalCreatedBy,
    @Semantics.systemDateTime.createdAt: true
    LocalCreatedAt,
    @Semantics.user.lastChangedBy: true
    LocalLastChangedBy,
    //local etag field --> odata etag
    @Semantics.systemDateTime.localInstanceLastChangedAt: true
    LocalLastChangedAt,

    //total etag field
    @Semantics.systemDateTime.lastChangedAt: true
    LastChangedAt,

    /* Associations */
    _CarrierSingleton
}
```

Developing Transactional Behavior

The multi-inline-edit scenario implements some standard behavior for draft BOs and additional specific behavior that is required to enable multi-inline-editing on the UI.

Context

The behavior for the singleton business object is defined in a behavior definition. Since the carrier business object is draft-enabled, there is some standard draft behavior that must be available for the draft BO. Other behavior results as a consequence of the special singleton pattern that is used in this BO.

Procedure

Create the behavior definition by using the creation wizard in ADT. By using the context menu on the root view (`/DMO/I_CarriersLockSingleton_S`), you can directly create a behavior definition for the carrier singleton data model. In the wizard, choose the implementation type **Managed**.

Standard Behavior in the Template

The template for the `managed` implementation types provides a suggestion for the behavior pool and the strict addition as best practice for RAP BOs. For every entity in the composition tree a behavior definition is added and the persistent table is specified. As the root entity uses a join on its data source, a persistent table is not found.

The standard operations `create`, `update` and `delete` are included for each entity. The create operation on child entities is not supported in RAP scenarios. Instances of child entities can only be created by the create-by-association operation, which is also included for parent entities.

The template also proposes to include the behavior characteristics `lock`, `authorization` and `etag`. The master for lock and authorization is defined on the root entity with dependents on the child entity. The strict mode requires that authorization is defined for the business object. With the addition `none`, no authorization checks are executed. This is a demo application, which is why the addition `none` is used. This approach is not recommended in real-world scenarios and violates best practices. For more information, see [Authorization Control](#). Since this carrier BO only has little behavior that must implemented, you can use one implementation class for both entities. The template proposes the name of the behavior pool that is used for both entities.

The root entity instance can never be modified. Therefore it does not need an ETag. The ETag master is defined on the child entity with the ETag field `LocalLastChangedAt`. It is recommended that you specify the respective ETag field as read-only in the behavior definition, using the field characteristic `readOnly`.

The child entity uses different field names in the CDS view than on the database table. Add a mapping definition for the carrier child entity for all fields.

Singleton-Specific Behavior

Since the root view entity is only designed as an entrance point to the app, it is expected that there is always only just one single instance. It is not intended that you can create, update, or delete instances on the root entity. Delete this modification behavior from the template on the root entity.

The implementation type `managed` requires the specification of a persistent table for each BO entity. For the root view entity, there is no persistent table, which leads to a syntax error in the behavior definition. Modification on the root view entity is not allowed in this scenario. Therefore, no persistent table is required. To solve the syntax issue in the behavior definition, define an `unmanaged save` for the root view entity with the syntax `with unmanaged save`. Since this scenario does not intend to save any modification of the root entity instances at all, the saver implementation method in the behavior pool remains empty. It is then called during the save sequence of the BO, but does not change anything for the single root view entity instance.

Set the fields `CarrierSingletonID` in the root entity to read-only. As no modification is allowed on the root entity, the field should not be editable on the UI.

Draft-Specific Behavior

To draft-enable the carrier business object, define `with draft` in the behavior definition header.

Draft-enablement requires that a draft database table is available for each entity. By adding `draft table` and a name for the draft table on each entity, you can make use of the behavior definition quick fix that triggers the creation of a draft table. The draft table for the root entity `/dmo/d_carr_lock` is just a dummy table that is not used, as no modification on the root entity instance is possible in this scenario. The draft table for the child entity `/dmo/d_carrier` is used for the draft life cycle management of carrier draft instances.

The draft-enablement also requires the definition of a total ETag field. The field `LastChangedAtMax` is calculated in the root view entity to be used as total ETag field.

Add the addition with `draft` to the associations `association _Airline {create;}` and `association _CarrierSingleton;` to explicitly define that the association is always followed with regards to the draft indicator. This is automatically implicitly enabled, but for transparency reasons, it is best practice to explicitly add it to the behavior definition. The field `CarrierSingletonID` is used in the on-condition for the to-parent association `_CarrierSingleton` in the child entity. It must be set to read-only to ensure that the connecting element between parent and child cannot be changed.

All master fields of the child entity are mandatory as no instance should exist without the `AirlineID`, `Name` or `CurrencyCode`. In addition, the key field `AirlineID` must be read-only on update.

Define all draft actions for the root entity.

i Expand the following code sample to view the source code of behavior definition `/DMO/I_CarriersLockSingleton_S`.

Sample Code

```
managed implementation in class /dmo/bp_i_carrierslocksngle_s unique;

strict(2);
with draft;

define behavior for /DMO/I_CarriersLockSingleton_S alias CarriersLockSingleton
with unmanaged save
draft table /dmo/d_carr_lock
lock master
total etag LastChangedAtMax
authorization master ( none )

{
  association _Airline { create; with draft; }

  field ( readonly ) CarrierSingletonID, LastChangedAtMax;

  draft action Edit;
  draft action Activate optimized;
  draft action Discard;
  draft action Resume;
  draft determine action Prepare
}

define behavior for /DMO/I_Carrier_S alias Carrier
persistent table /dmo/carrier
draft table /dmo/d_carrier
lock dependent by _CarrierSingleton
authorization dependent by _CarrierSingleton
etag master LocalLastChangedAt
{
  update;
  delete;

  field ( readonly ) CarrierSingletonID, LocalCreatedBy, LocalCreatedAt, LocalLastChangedBy, Loc
  field ( mandatory : create, readonly : update ) AirlineID;
  field ( mandatory ) Name, CurrencyCode;

  association _CarrierSingleton { with draft; }

  mapping for /dmo/carrier
  { AirlineID = carrier_id;
    CurrencyCode = currency_code;
    Name = name;
    LocalCreatedBy = local_created_by;
    LocalCreatedAt = local_created_at;
    LocalLastChangedBy = local_last_changed_by;
    LocalLastChangedAt = local_last_changed_at;
    LastChangedAt = last_changed_at;
  }
}
```

Developing Feature Control for the Carrier Entity

For the multi-inline-edit scenario, enable and disable functionality with feature control on the carrier entity.

Context

The carrier data that is used in this scenario to demonstrate multi-inline-edit capabilities on a Fiori UI is also used by other business objects as master data for travel instances. The booking entity in a travel business object has an association to the carrier interface view and validates if the carrier that is used in a booking instance exists on the database table for carrier data. This ensures that only existing airlines are used. Up to now, the carrier data remained stable as no other business object modified data on the database table. With the current scenario, this changes. When maintaining the carrier data, it must be ensured that no carrier instances can be deleted that is already used by another business object. This is enforced by defining feature control for the delete operation. The instance feature control checks if the airline is used in a flight connection that is maintained on the database table `/dmo/connection`. The delete operation is only enabled if the airline ID is not used in a connection.

Procedure

Define feature control for the delete operation on the carrier entity.

Sample Code

```
define behavior for /DMO/I_Carrier_S alias Carrier
...
{
  update;
  delete ( features : instance );
... }
```

Use the quick fix in the behavior definition to generate the method definition for the instance feature control handler in the behavior pool.

The implementation is done in the method `FOR INSTANCE FEATURES`.

Read all the instances with the imported keys into an internal table and use the `AirlineID` to select the corresponding `AirlineIDs` from the connection interface view `/DMO/I_Connection` to check if the airline IDs in the carrier BO are used in flight connections.

Disable the delete operation in the result table of the method if the respective `AirlineID` exists in a flight connection. If it doesn't exist, set the permission value to `enabled`.

Fill the reported table with an adequate information message that is returned in the response structure of the feature control method.

i *Expand the following code sample to view the source code of `GET_INSTANCE_FEATURES`.*

Sample Code

```
CLASS lhc_carrier DEFINITION INHERITING FROM cl_abap_behavior_handler.

  PRIVATE SECTION.

    ...
    METHODS get_instance_features FOR INSTANCE FEATURES
      IMPORTING keys REQUEST requested_features FOR Carrier RESULT result.
```

ENDCLASS.

CLASS lhc_carrier IMPLEMENTATION.
METHOD get_instance_features.

DATA: airline_ids TYPE SORTED TABLE OF /dmo/i_carrier WITH UNIQUE KEY AirlineID.

READ ENTITIES OF /DMO/I_CarriersLockSingleton_S IN LOCAL MODE
ENTITY Carrier
FIELDS (AirlineID)
WITH CORRESPONDING #(keys)
RESULT DATA(Carrier)
FAILED failed.

airline_ids = CORRESPONDING #(Carrier DISCARDING DUPLICATES MAPPING AirlineID = AirlineID E

IF airline_ids IS NOT INITIAL.
SELECT DISTINCT AirlineID
FROM /DMO/I_Connection
FOR ALL ENTRIES IN @airline_ids
WHERE AirlineID = @airline_ids-AirlineID
INTO TABLE @DATA(connections_db).
ENDIF.

LOOP AT Carrier ASSIGNING FIELD-SYMBOL(<carrier>).
" Delete is not allowed if any Connection exists
IF line_exists(connections_db[AirlineID = <carrier>-AirlineID]).
APPEND VALUE #(%tky = <carrier>-%tky
%delete = if_abap_behv=>fc-o-disabled) TO result.
APPEND VALUE #(%tky = <carrier>-%tky
%msg = NEW /dmo/cx_carriers_s(
textid = /dmo/cx_carriers_s=>airli
severity = if_abap_behv_message=>sev
airline_id = <carrier>-AirlineID)
%delete = if_abap_behv=>mk-on
%path-carrierslocksingleton = CORRESPONDING #(<carrier>)
) TO reported-carrier.
ELSE.
APPEND VALUE #(%tky = <carrier>-%tky
%delete = if_abap_behv=>fc-o-enabled) TO result.
ENDIF.
ENDLOOP.

ENDMETHOD.

ENDCLASS.

Developing Validations

For the multi-inline-edit scenario, define and implement validations for value consistency checks.

Context

Carrier instances are only completely usable if the values for each field are supplied. An airline without a name or currency code doesn't make any sense. It is therefore indispensable that the values are checked on creating new instances and on editing. The fields of carrier instances must not be empty. By defining the key field `AirlineID` to be mandatory on create and read-only on update, the RAP runtime engine ensures that the value is supplied when creating a new instance and that it does not get changed during the modification process. The other fields `Name` and `CurrencyCode` are the developer's responsibility. These fields must be checked by validations. Before saving a draft instance to the persistent table, the validations are executed and instances without all values are rejected.

The validations must have the trigger condition `create` and the corresponding field.

Procedure

Define validations to check the **Name** and the **CurrencyCode** when creating new carrier instances and editing these fields on the carrier entity.

Sample Code

```
define behavior for /DM0/I_Carrier_S alias Carrier
...
{
  validation validateName on save { create; field Name; }
  validation validateCurrencyCode on save { create; field CurrencyCode; }
... }
```

The messages of validations are only correctly presented on the UI, if the validations are defined for the draft determine action **Prepare** on the root entity.

Sample Code

```
define behavior for /DM0/I_CarrierLockSingleton_S alias Carrier
...
{
  draft determine action Prepare
  {
    validation Carrier ~ validateName;
    validation Carrier ~ validateCurrencyCode;
  }

... }
```

Use the quick fix in the behavior definition to generate the method definition for the validation handler in the behavior pool.

The implementation is done in the methods **FOR VALIDATE ON SAVE**.

Validation **ValidateName**

Read all the instances with the imported keys into an internal table. Pass the failed entries to the **failed** table of the validation.

Loop over the internal table **Carriers**. To avoid duplicate state messages for the consumer, append an empty message to the reported table to clear the state area. If the respective field is initial, write the instance identifier to the failed table and append a corresponding message to the respective state area.

i *Expand the following code sample to view the source code of **validateName**.*

Sample Code

```
CLASS lhc_carrier DEFINITION INHERITING FROM cl_abap_behavior_handler.

  PRIVATE SECTION.

    METHODS validateName FOR VALIDATE ON SAVE
      IMPORTING keys FOR Carrier~validateName.

ENDCLASS.
```

```

CLASS lhc_carrier IMPLEMENTATION.
  METHOD validateName.

```

```

  READ ENTITIES OF /DMO/I_CarriersLockSingleton_S IN LOCAL MODE
  ENTITY Carrier
    FIELDS ( CarrierSingletonID Name )
    WITH CORRESPONDING #( keys )
  RESULT DATA(Carrier).

```

```

  " Raise message for empty Airline Name
  LOOP AT Carrier ASSIGNING FIELD-SYMBOL(<carrier>).

```

```

    APPEND VALUE #( %tky      = <carrier>-%tky
                    %state_area = 'VALIDATE_NAME'
                    ) TO reported-carrier.

```

```

    IF <carrier>-name IS INITIAL.

```

```

      APPEND VALUE #( %tky      = <carrier>-%tky ) TO failed-carrier.
      APPEND VALUE #( %tky      = <carrier>-%tky
                      %msg      = NEW /dmo/cx_carriers_s(
                                  textid      = /dmo/cx_carriers_s=>name_r
                                  severity    = if_abap_behv_message=>seve
                                  airline_id = <carrier>-AirlineID )
                                  %element-name = if_abap_behv=>mk-on
                                  %state_area  = 'VALIDATE_NAME'
                                  %path-carrierslocksingleton = CORRESPONDING #( <carrier> )
                                ) TO reported-carrier.
    ENDIF.
  ENDLLOOP.

```

```

  ENDMETHOD.
ENDCLASS.

```

Validation **ValidateCurrencyCode**

In addition to checking if the provided value for the carrier instance is not initial, it is also relevant here to check whether the provided currency code actually exists.

Read all the instances with the imported keys into an internal table. Pass the failed entries to the `failed` table of the validation.

Copy the entries to another internal table and delete duplicates. If the resulting table is not initial, select the entries of `/DMO/I_Currency` with the given currency code. If there is no corresponding result entry, the provided currency code is not valid.

Loop over the internal table `Carriers`. To avoid duplicate state messages for the consumer, append an empty message to the reported table to clear the state area. If the respective field is initial, write the instance identifier to the failed table and append a corresponding message to the respective state area. If the select from `/DMO/I_Currency` does not return values for the provided currency code, write the instance identifier to the failed table and append a corresponding message to the respective state area.

i *Expand the following code sample to view the source code of **validateCurrencyCode**.*

Sample Code

```

CLASS lhc_carrier DEFINITION INHERITING FROM cl_abap_behavior_handler.

  PRIVATE SECTION.

    METHODS validatecurrencycode FOR VALIDATE ON SAVE
      IMPORTING keys FOR carrier~validatecurrencycode.

  ENDMETHOD.

CLASS lhc_carrier IMPLEMENTATION.
  METHOD validateCurrencyCode.

```

```

DATA: currency_codes TYPE SORTED TABLE OF I_Currency WITH UNIQUE KEY Currency.

READ ENTITIES OF /DMO/I_CarriersLockSingleton_S IN LOCAL MODE
  ENTITY Carrier
    FIELDS ( CarrierSingletonID CurrencyCode )
    WITH CORRESPONDING #( keys )
  RESULT DATA(Carrier).

currency_codes = CORRESPONDING #( Carrier DISCARDING DUPLICATES MAPPING Currency = CurrencyC

IF currency_codes IS NOT INITIAL.
  SELECT FROM I_Currency FIELDS Currency
    FOR ALL ENTRIES IN @currency_codes
    WHERE Currency = @currency_codes-Currency
    INTO TABLE @DATA(currency_codes_db).
ENDIF.

" Raise message for not existing Currency Codes
LOOP AT Carrier ASSIGNING FIELD-SYMBOL(<carrier>).

  APPEND VALUE #( %tky          = <carrier>-%tky
                  %state_area = 'VALIDATE_CURRENCY_CODE'
                  ) TO reported-carrier.

  IF <carrier>-CurrencyCode IS INITIAL.
    APPEND VALUE #( %tky          = <carrier>-%tky ) TO failed-carrier.
    APPEND VALUE #( %tky          = <carrier>-%tky
                    %msg          = NEW /dmo/cx_carriers_s(
                                      textid      = /dmo/cx_carriers_s=>
                                      severity     = if_abap_behv_message
                                      airline_id   = <carrier>-AirlineID
                                      = if_abap_behv=>mk-on
                                      = 'VALIDATE_CURRENCY_CODE'
                                      = CORRESPONDING #( <carrier> )
                                      %element-currencycode
                                      %state_area
                                      %path-carrierslocksingleton
                                      ) TO reported-carrier.

  ELSEIF NOT line_exists( currency_codes_db[ Currency = <carrier>-CurrencyCode ] ).
    APPEND VALUE #( %tky          = <carrier>-%tky ) TO failed-carrier.
    APPEND VALUE #( %tky          = <carrier>-%tky
                    %msg          = NEW /dmo/cx_carriers_s(
                                      textid      = /dmo/cx_carriers_
                                      severity     = if_abap_behv_mess
                                      currency_code = <carrier>-Currenc
                                      = if_abap_behv=>mk-on
                                      = 'VALIDATE_CURRENCY_CODE'
                                      = CORRESPONDING #( <carrier> )
                                      %element-currencycode
                                      %state_area
                                      %path-carrierslocksingleton
                                      ) TO reported-carrier.

  ENDIF.
ENDLOOP.

ENDMETHOD.

ENDCLASS.

```

Projecting the BO for UI Exposure

For the multi-inline-edit scenario, create a business object projection for UI exposure.

The business object with the singleton mechanism is aimed to be used in a UI scenario. It defines basic features that are essential for the UI use case of multi-inline-editing. To define the data model and its functionality that is needed for the use case of a carrier UI service, the business object must be projected. In this BO projection, you define value helps, search and UI layout specifics.

Tasks for the Developer

Create CDS projection views for the root and the child entity, see [Projecting the BO Data Model](#).

Create a CDS projection behavior definition to define the behavior that is to be used in the UI service, see [Projecting the BO Behavior](#).

Projecting the BO Data Model

For the multi-inline-edit scenario, create and define the data model for the BO projection.

Context

The multi-inline-edit scenario provides one projection use case that reuses all elements that are defined in the business object view entities to be exposed for a UI service. In the projection layer the data model is enriched with UI functionality that is implemented via CDS annotations or keywords in the projection view and manifested in the service metadata.

Procedure

Create a data definition for projection views for the two business object entities of the carrier BO. In ADT, use the context menu on the underlying CDS view entity to open the wizard for creating a CDS data definition and choose the template **Define Projection View**.

Projection View /DMO/C_CarrierLockSingleton_S

The projection view is a projection on /DMO/I_CarrierLockSingleton_S. Include the keyword root in the projection view.

Use all elements of the underlying BO travel entity.

Enable the usage of a metadata extension with the annotation `@Metadata.allowExtension:true`.

To indicate that `CarrierSingleton` is the semantic identifier of the BO entity, use the annotation `@ObjectModel.semanticKey: ['CarrierSingleton']` in the header of the projection view.

The `CarrierSingleton` is just used for the on-condition of the composition. It is not necessary on the UI representation. Use the annotation `@Consumption.hidden:true`.

As soon as all projection views of the composition structure of the business object are available, redirect the composition to child to the projection view of the carrier entity.

i *Expand the following code sample to view the source code of the projection view /DMO/C_CarrierLockSingleton_S.*

Sample Code

```
@EndUserText.label: 'Carrier Singleton Projection View'
@AccessControl.authorizationCheck: #NOT_REQUIRED
@Semantics.valueRange.maximum: '1'
@Metadata.allowExtensions: true
@ObjectModel.semanticKey: ['CarrierSingletonID']

define root view entity /DMO/C_CarriersLockSingleton_S
  as projection on /DMO/I_CarriersLockSingleton_S
  {
    key CarrierSingletonID,

    @Consumption.hidden: true
    LastChangedAtMax,
    /* Associations */
```

```

    _Airline : redirected to composition child /DM0/C_Carrier_S
}

```

Projection View /DM0/C_Carrier_S

The projection view is a projection on /DM0/I_Carrier_S.

Use all elements of the underlying BO travel entity. You do not need to use the administrative data in the projection view, except for the element representing the ETag master LocalLastChangedAt.

Enable the usage of a metadata extension with the annotation `@Metadata.allowExtension:true`.

Enable search capabilities for the travel entity, with the `@Search` annotations on the entity header and on designated elements. For more information, see [Enabling Text and Fuzzy Searches in SAP Fiori Apps](#)

To indicate that `CarrierID` is the semantic identifier of the BO entity, use the annotation `@ObjectModel.semanticKey: ['AirlineID']` in the header of the projection view.

The `CarrierSingletonID` is just used for the on-condition of the composition. It is not necessary on the UI representation. Use the annotation `@Consumption.hidden:true`.

Define a value help for the element `CurrencyCode`. For more information, see [Adding Value Help](#).

As soon as all projection views of the composition structure of the business object are available, redirect the association to the root entity to the respective projection view.

i *Expand the following code sample to view the source code of the projection view /DM0/C_Carrier_S.*

Sample Code

```

@EndUserText.label: 'Projection View Carrier'
@AccessControl.authorizationCheck: #NOT_REQUIRED

@Metadata.allowExtensions: true
@ObjectModel.semanticKey: ['AirlineID']

define view entity /DM0/C_Carrier_S
as projection on /DM0/I_Carrier_S
{
    key AirlineID,

    @Consumption.hidden: true
    CarrierSingletonID,

    Name,

    @Consumption.valueHelpDefinition: [{entity: {name: 'I_CurrencyStdVH', element: 'Currency'}
    CurrencyCode,

    LocalLastChangedAt,

    /* Associations */
    _CarrierSingleton : redirected to parent /DM0/C_CarriersLockSingleton_S
}

```

Adding UI Metadata for UI Consumption

For the multi-inline-edit scenario, define UI annotations in metadata extensions for the business object entities.

Context

The layout of a Fiori Elements UI can be mostly defined in the RAP BO with @UI annotations. These UI annotations are then exposed with the metadata of the UI service and are applied for defining a UI. It is best practice to maintain UI-relevant annotations in metadata extensions.

Procedure

Create a metadata extension for each projection view in the carrier BO. In ADT, right-click on the data definition to open the creation wizard for metadata extensions.

Metadata Extension /DMO/C_CARRIERSLOCKSSINGLETON_S

Define the metadata extension as extension of the CDS projection view /DMO/C_CARRIERSLOCKSSINGLETON_S.

Define the layer #CORE.

Since the root view of the carrier business object has just one instance, the UI layout for the list report page of a Fiori Elements has only limited layout requirements.

To specify the header texts for Fiori UIs, the annotation @UI.headerInfo is used at entity level. It specifies the list title as well as the title for the object page.

The main building blocks of the UI are specified as UI facets. Define a facet for the root view using the @UI.facet annotation and a navigation to the object page by defining the target elements.

For more information about UI layout configuration, see [Defining CDS Annotations for Metadata-Driven UIs](#).

i *Expand the following code sample to view the source code of the metadata extension /DMO/C_CarriersLockSingleton_S.*

Sample Code

```
@Metadata.layer: #CORE

@UI: { headerInfo: { typeName: 'Manage Carriers',
                    typeNamePlural: 'Carrier Singleton',
                    title: { type: #STANDARD, value: 'CarrierSingletonID' } } }

annotate entity /DMO/C_CarriersLockSingleton_S
with
{
  @UI.facet: [
    {
      purpose: #STANDARD,
      type: #LINEITEM_REFERENCE,
      label: 'Carrier Multi Inline Edit',
      position: 10,
      targetElement: '_Airline'
    }
  ]

  @UI.lineItem: [{ position: 10 }]
  CarrierSingletonID;
}
```

Metadata Extension /DMO/C_CARRIER_S

Define the metadata extension as extension of the CDS projection view /DMO/C_CARRIER_S.

Define the layer #CORE.

Like in the metadata extension for the root view, specify the header information, a facet as `identification_reference`, which indicates the list on the object page, and the position for the elements as line item and identification.

For more information about UI layout configuration, see [Defining CDS Annotations for Metadata-Driven UIs](#).

i *Expand the following code sample to view the source code of the metadata extension /DMO/C_Carrier_S.*

Sample Code

```
@Metadata.layer: #CORE

@UI: { headerInfo: { typeName:      'Carrier',
                      typeNamePlural: 'Carriers',
                      title:        { type: #STANDARD,
                                      label: 'Airline',
                                      value: 'AirlineID' } } }

annotate entity /DMO/C_Carrier_S
with
{
  @UI.facet: [{ type: #IDENTIFICATION_REFERENCE }]

  @UI.lineItem: [{ position: 10 }]
  @UI.identification: [{ position: 10 }]
  AirlineID;

  @UI.lineItem: [{ position: 20 }]
  @UI.identification: [{ position: 20 }]
  Name;

  @UI.lineItem: [{ position: 30 }]
  @UI.identification: [{ position: 30 }]
  CurrencyCode;
}
```

Projecting the BO Behavior

For the multi-inline-edit scenario, create a projection behavior definition for defining the projection behavior.

Context

In a behavior definition, you define the behavior characteristics and operations of the underlying behavior definition that are to be used in the service the BO is exposed to. For the carrier business object, all behavior that is defined in the base BO is used in the projection behavior definition.

Procedure

Create a projection behavior definition for the carrier business object. In ADT, use the context menu of the root projection view to open the wizard for creating behavior definitions. The implementation type is prefilled with Projection.

Use the complete behavior that was defined in the underlying behavior definition for the carrier BO.

i Expand the following code sample to view the source code of the projection behavior definition `/DMO/C_CarriersLockSingleton_S`.

Sample Code

```
projection;
strict(2);
use draft;

define behavior for /DMO/C_CarriersLockSingleton_S alias CarriersLockSingleton
{
    use action Activate;
    use action Discard;
    use action Edit;
    use action Prepare;
    use action Resume;

    use association _Airline { create; with draft; }
}

define behavior for /DMO/C_Carrier_S alias Carrier
use etag
{
    use update;
    use delete;

    use association _CarrierSingleton { with draft; }
}
```

Defining and Publishing a UI Service with Multi-Inline-Edit Capabilities

For the multi-inline-edit, define a UI service and bind it to a protocol.

Context

The RAP BO with the singleton mechanism is designed to be exposed as UI service for consumption via Fiori Elements UI with multi-inline-edit capabilities. For this to work, the carrier BO must be exposed as a business service to OData. In the service definition, you define which artifacts are included in the service. The service binding defines the protocol with which the service is addressed. In addition, it provides the means to activate the service locally. The Fiori Elements App Preview can be used as soon as the service is activated.

Procedure

Create a service definition for the business service.

Include the projection views of the business object and all related CDS entities that are relevant for UI consumption into the service definition.

i Expand the following code sample to view the source code of the service definition `/DMO/UI_Carriers_S`.

Sample Code

```
@EndUserText.label: 'Carrier Multi-Inline-Edit Scenario'
@ObjectModel.leadingEntity.name: '/DMO/C_CarriersLockSingleton_S'
define service /DMO/UI_Carriers_S {
    expose /DMO/C_CarriersLockSingleton_S as CarriersLockSingleton;
```



```
    expose /DM0/C_Carrier_S as Carrier;  
}
```

Based on the service definition, create a service binding.

Expose the service as a UI service with OData protocol V4 and publish it locally. You can then use the preview to test the service.

Developing a UI Service with Access to a Remote Service

Based on a remote OData service, you create a new service that enhances the remote service with additional information.

Introduction

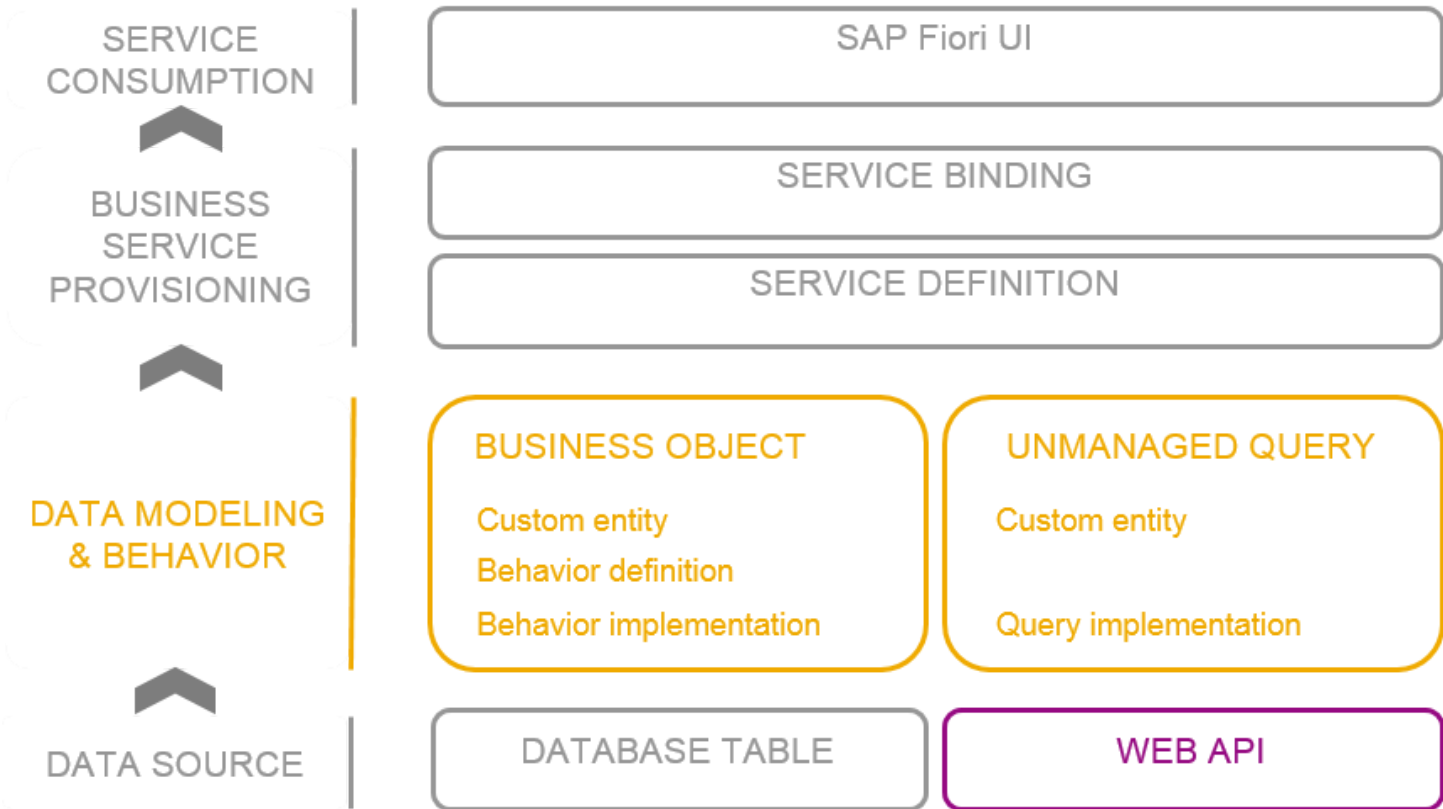
In this scenario, you develop an OData service that consumes a Web API. This Web API represents a remote service from which you retrieve data. This data is then enhanced with additional data. That means, the new OData service has two data sources. One is the remote service, the other source is a database table in the consuming development system. You build a new SAP Fiori UI application that consumes the remote service from the provisioning tenant, and enhances the remote service with persistent data of the consuming tenant.

The data model for the new service is defined in a CDS custom entity, which enables you to retrieve data from different sources. A CDS custom entity has an unmanaged query, which has to be implemented manually in a related ABAP class, the query implementation class. In this ABAP class, data from different sources, including from another system, can be retrieved.

With the help of the Service Consumption Model, you can import the OData metadata of the remote service into the consuming system and create a proxy class of the remote service. This gives you a local representation of the remote data model in your tenant, which helps you to consume the data of the remote service.

This guide also describes how you implement transactional behavior to modify the additional data on the local database table.

The following image gives an overview of the architecture of the service consumption scenario.



To be able to get data from a remote service, you build an OData client proxy in your implementation to forward and transform the requests for the remote service. The client proxy can only consume the remote service if a connection to the provisioning tenant is established. The configuration of such a destination is a precondition for developing this scenario.

Development Steps to Create a New Service to Consume a Remote Service

- Save the metadata file of the remote service locally.
- Create a proxy class with the Service Consumption Model wizard using the metadata file. The created proxy class will be used for consuming the remote service.
- Implement the query in an ABAP class using the code snippets generated by the Service Consumption Model.
- Implement the transactional behavior for the local fields.
- Create an OData service.

Prerequisites

Developing the scenario that is described in the subsequent chapters requires the following:

- You fulfill the requirements described in [Prerequisites](#).
- You have installed ABAP Development Tools (ADT).

SAP recommends to use the latest version of the client installation. The ADT download is available on the update site <https://tools.hana.ondemand.com/>.

- To recreate the demo scenario, the **ABAP Flight Reference Scenario** must be available in your ABAP system. You can download the complete reference scenario from **GitHub**: [Downloading the ABAP Flight Reference Scenario](#).
- You know the URL of the remote service that you want to consume or have access to the CSDL XML file of the service.

In this demo scenario, the service `/DMO/API_TRAVEL_U_V4` is used as remote service. You can download the service as part of the ABAP Flight Reference Scenario, see [Downloading the ABAP Flight Reference Scenario](#).

Prerequisites

Prerequisites in the remote system

- The remote OData service has been published in a communication scenario as an inbound service.

For more information, see [Consuming Services in the Context of API with Technical Users](#).

- A communication system with the same name as the communication scenario has been created. The communication system includes a user for inbound communication.
- The communication scenario and the communication system are used in a communication arrangement involving the inbound communication user.

Prerequisites in the local system

- A communication scenario with an outbound HTTP service has been created and published.

For more information, see [Consuming Services in the Context of API with Technical Users](#).

- A communication system with the same name as the communication scenario has been created. The communication system includes the host name of the remote system and the same user from above for outbound communication.
- The communication scenario and the communication system are used in a communication arrangement involving the outbound communication user.

For information on how to create the required systems and communication arrangements, see:

- [Creating a Communication System](#)
- [Creating a Communication Arrangement](#)

Scenario Description

Starting Point: Remote OData Service

In this development guide, we reuse the service that was built in [Develop Web APIs](#). We only use the root node from the remote service for our consumption and the transactional behavior is ignored. The new service in the consuming system does not implement operations to create, update, or delete data from the provisioning system. Instead, it retrieves the data from there and enriches the service with an additional field. This additional field, however, can be modified. Therefore, the new UI service contains transactional capabilities for this additional field.

A Web API service is the prototypical example for this consumption scenario. It contains all the required information, but does not contain any metadata that is superfluous in the consumption scenario (for example, UI-relevant annotations). However, it is also possible to consume an OData service that was published as a UI service.

The available entity types of the remote service can be viewed in the metadata document when extended.

i

```

▼<edmx:Edmx xmlns:edmx="http://docs.oasis-open.org/odata/ns/edmx" xmlns="http://docs.oasis-open.org/odata/ns/edm" Version="4.0">
  ▶<edmx:Reference Uri="/sap/opu/odata/IWFND/CATALOGSERVICE;v=2/Vocabularies(TechnicalName='%2FIWBEP%2FVOC_COMMUNICATION',Version='0001',SAP__Origin='LOCAL')/$value">
    ...
  </edmx:Reference>
  ▼<edmx:Reference Uri="/sap/opu/odata/IWFND/CATALOGSERVICE;v=2/Vocabularies(TechnicalName='%2FIWBEP%2FVOC_PERSONALDATA',Version='0001',SAP__Origin='LOCAL')/$value">
    <edmx:Include Namespace="com.sap.vocabularies.PersonalData.v1" Alias="PersonalData"/>
  </edmx:Reference>
  ▼<edmx:Reference Uri="/sap/opu/odata/IWFND/CATALOGSERVICE;v=2/Vocabularies(TechnicalName='%2FIWBEP%2FVOC_ANALYTICS',Version='0001',SAP__Origin='LOCAL')/$value">
    <edmx:Include Namespace="com.sap.vocabularies.Analytics.v1" Alias="Analytics"/>
  </edmx:Reference>
  ▼<edmx:Reference Uri="/sap/opu/odata/IWFND/CATALOGSERVICE;v=2/Vocabularies(TechnicalName='%2FIWBEP%2FVOC_COMMON',Version='0001',SAP__Origin='LOCAL')/$value">
    <edmx:Include Namespace="com.sap.vocabularies.Common.v1" Alias="SAP__common"/>
  </edmx:Reference>
  ▼<edmx:Reference Uri="/sap/opu/odata/IWFND/CATALOGSERVICE;v=2/Vocabularies(TechnicalName='%2FIWBEP%2FVOC_MEASURES',Version='0001',SAP__Origin='LOCAL')/$value">
    <edmx:Include Namespace="Org.OData.Measures.V1" Alias="SAP__measures"/>
  </edmx:Reference>
  ▼<edmx:Reference Uri="/sap/opu/odata/IWFND/CATALOGSERVICE;v=2/Vocabularies(TechnicalName='%2FIWBEP%2FVOC_CORE',Version='0001',SAP__Origin='LOCAL')/$value">
    <edmx:Include Namespace="Org.OData.Core.V1" Alias="SAP__core"/>
  </edmx:Reference>
  ▼<edmx:Reference Uri="/sap/opu/odata/IWFND/CATALOGSERVICE;v=2/Vocabularies(TechnicalName='%2FIWBEP%2FVOC_CAPABILITIES',Version='0001',SAP__Origin='LOCAL')/$value">
    <edmx:Include Namespace="Org.OData.Capabilities.V1" Alias="SAP__capabilities"/>
  </edmx:Reference>
  ▼<edmx:Reference Uri="/sap/opu/odata/IWFND/CATALOGSERVICE;v=2/Vocabularies(TechnicalName='%2FIWBEP%2FVOC_AGGREGATION',Version='0001',SAP__Origin='LOCAL')/$value">
    <edmx:Include Namespace="Org.OData.Aggregation.V1" Alias="SAP__aggregation"/>
  </edmx:Reference>
  ▼<edmx:Reference Uri="/sap/opu/odata/IWFND/CATALOGSERVICE;v=2/Vocabularies(TechnicalName='%2FIWBEP%2FVOC_VALIDATION',Version='0001',SAP__Origin='LOCAL')/$value">
    <edmx:Include Namespace="Org.OData.Validation.V1" Alias="SAP__validation"/>
  </edmx:Reference>
  ▼<edmx:Reference Uri="/sap/opu/odata/IWFND/CATALOGSERVICE;v=2/Vocabularies(TechnicalName='%2FIWBEP%2FVOC_CODELIST',Version='0001',SAP__Origin='LOCAL')/$value">
    <edmx:Include Namespace="com.sap.vocabularies.CodeList.v1" Alias="SAP__CodeList"/>
  </edmx:Reference>
  ▼<edmx:Reference Uri="/sap/opu/odata/IWFND/CATALOGSERVICE;v=2/Vocabularies(TechnicalName='%2FIWBEP%2FVOC_UI',Version='0001',SAP__Origin='LOCAL')/$value">
    <edmx:Include Namespace="com.sap.vocabularies.UI.v1" Alias="SAP__UI"/>
  </edmx:Reference>
  ▼<edmx:Reference Uri="/sap/opu/odata/IWFND/CATALOGSERVICE;v=2/Vocabularies(TechnicalName='%2FIWBEP%2FVOC_HTML5',Version='0001',SAP__Origin='LOCAL')/$value">
    <edmx:Include Namespace="com.sap.vocabularies.HTML5.v1" Alias="SAP__HTML5"/>
  </edmx:Reference>
  ▼<edmx:Reference Uri="/sap/opu/odata/IWFND/CATALOGSERVICE;v=2/Vocabularies(TechnicalName='%2FIWBEP%2FVOC_PDF',Version='0001',SAP__Origin='LOCAL')/$value">
    <edmx:Include Namespace="com.sap.vocabularies.PDF.v1" Alias="SAP__PDF"/>
  </edmx:Reference>
  ▼<edmx:Reference Uri="/sap/opu/odata/IWFND/CATALOGSERVICE;v=2/Vocabularies(TechnicalName='%2FIWBEP%2FVOC_SESSION',Version='0001',SAP__Origin='LOCAL')/$value">
    <edmx:Include Namespace="com.sap.vocabularies.Session.v1" Alias="SAP__session"/>
  </edmx:Reference>
  ▼<edmx:Reference Uri="/sap/opu/odata/IWFND/CATALOGSERVICE;v=2/Vocabularies(TechnicalName='%2FIWBEP%2FVOC_HIERARCHY',Version='0001',SAP__Origin='LOCAL')/$value">
    <edmx:Include Namespace="com.sap.vocabularies.Hierarchy.v1" Alias="SAP__hierarchy"/>
  </edmx:Reference>
  ▼<edmx:DataServices>
    ▼<Schema Namespace="com.sap.gateway.srvd_a2x.dmo.travel_u.v0001" Alias="SAP__self">
      <Annotation Term="SAP__core.SchemaVersion" String="1.0.0"/>
      ▼<EntityType Name="TravelAgencyType">
        ▼<Key>
          <PropertyRef Name="AgencyID"/>
        </Key>
        <Property Name="AgencyID" Type="Edm.String" Nullable="false" MaxLength="6"/>
        <Property Name="Name" Type="Edm.String" Nullable="false" MaxLength="80"/>
        <Property Name="Street" Type="Edm.String" Nullable="false" MaxLength="60"/>
        <Property Name="PostalCode" Type="Edm.String" Nullable="false" MaxLength="10"/>
        <Property Name="City" Type="Edm.String" Nullable="false" MaxLength="40"/>
        <Property Name="CountryCode" Type="Edm.String" Nullable="false" MaxLength="3"/>
        <Property Name="CountryCodeText" Type="Edm.String" Nullable="false" MaxLength="50"/>
        <Property Name="PhoneNumber" Type="Edm.String" Nullable="false" MaxLength="30"/>
        <Property Name="EmailAddress" Type="Edm.String" Nullable="false" MaxLength="256"/>
        <Property Name="WebAddress" Type="Edm.String" Nullable="false" MaxLength="256"/>
      </EntityType>
      ▼<EntityType Name="TravelType">
        ▼<Key>
          <PropertyRef Name="TravelID"/>
        </Key>
        <Property Name="TravelID" Type="Edm.String" Nullable="false" MaxLength="8"/>
        <Property Name="AgencyID" Type="Edm.String" Nullable="false" MaxLength="6"/>
        <Property Name="AgencyName" Type="Edm.String" Nullable="false" MaxLength="80"/>
        <Property Name="CustomerID" Type="Edm.String" Nullable="false" MaxLength="6"/>
        <Property Name="CustomerName" Type="Edm.String" Nullable="false" MaxLength="40"/>
        <Property Name="BeginDate" Type="Edm.Date"/>
      </EntityType>
    </Schema>
  </edmx:DataServices>
</edmx:Edmx>

```

Metadata of the Remote Service

End Point: New OData Service with Additional Field

We want to consume the remote service in the local system and extend it with an additional field for a percentage prize discount, related to the total prize of a travel. The end user will be able to maintain possible discounts for each trip in an SAP Fiori application. This discount field is persisted on a local database table and retrieved together with the data of the remote service. The end user UI does not show any difference between the persisted field and the fields that are retrieved remotely.

In our scenario, the **Discount Percentage** field is persisted in the database table **/DMO/TRVL_SCM_DC**, which consists of the discount field, the travel ID as the key, and administrative data to track changes.

In addition to the persistent fields, there is also a calculated field (**Total Price with Discount**) that displays the new total price including the possible discount. This field, however, is not persisted in the database, but is transient.

Procedure

To merge the locally and remotely retrieved data, the application developer must import the metadata of the remote service into the consuming system. A proxy class that mirrors the original data model of the remote service is generated by the Service

Consumption Model wizard. It is then possible to access the service within the local ABAP code. To build a new OData service based on the remote service including additional database fields, a CDS custom entity is used to represent the data model, including both local and remote fields.

The program logic of a custom entity is not managed by ABAP runtime frameworks, but has to be implemented manually. The implementation includes the query itself and every query option as well as the transactional behavior for the local fields. The query is implemented in a query implementation class and the transactional runtime behavior is defined via a behavior definition and implemented in the related behavior pool. To access the data from the remote service, an OData Client Proxy must be instantiated in the implementation classes.

To create the OData service, we expose the custom entity in a service definition and create a service binding to bind the service against a protocol. In our example case, this is a V2 UI service.

As a result, the existing remote read-only service and database fields with transactional capabilities are merged within the same SAP Fiori Elements app.

Related Information

[Developing Unmanaged Transactional Apps](#)

[Develop Web APIs](#)

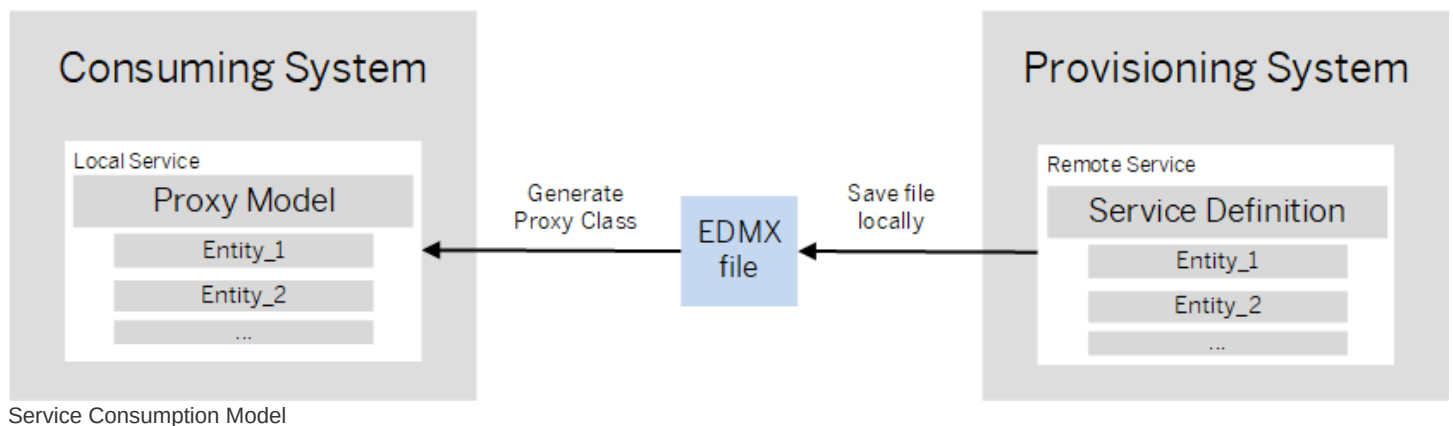
Preparing Access to the Remote OData Service

In order to consume the remote service in your local system, you need a local representation of the remote service.

Context

To be able to access the remote OData service, you need to generate a proxy model which mirrors the data model of the remote service. The Service Consumption Model generates this proxy model as an ABAP class.

To do so, the Service Consumption Model wizard requires a CSDL XML document of the external service as a basis for the proxy entities to be generated.



Getting the OData Service Metadata

Context

A CSDL XML file can be retrieved from any service if the service URL is known. The following procedure describes the steps for saving a CSDL XML document on your local machine.

Procedure

- 1. Open your browser.
- 2. Call the external OData service metadata document.

i Note

You get the metadata document by adding `/metadata` to the service URL.

- 3. Save the metadata as an **XML Document** on your local machine.

```
<?xml version='1.0' encoding='utf-8'>
<edmx:Edmx xmlns:edmx="http://docs.oasis-open.org/odata/ns/edmx" xmlns="http://docs.oasis-open.org/odata/ns/edm" Version="4.0">
  <edmx:Reference Uri="/sap/opu/odata/IWFND/CATALOGSERVICE;v=2/Vocabularies(TechnicalName='%2FIWBEP%2FVOC_COMMUNICATION',Version='0001',SAP__Origin='LOCAL')/$value">
    <edmx:Include Namespace="com.sap.vocabularies.Communication.v1" Alias="Communication"/>
  </edmx:Reference>
  <edmx:Reference Uri="/sap/opu/odata/IWFND/CATALOGSERVICE;v=2/Vocabularies(TechnicalName='%2FIWBEP%2FVOC_PERSONALDATA',Version='0001',SAP__Origin='LOCAL')/$value">
    <edmx:Include Namespace="com.sap.vocabularies.PersonalData.v1" Alias="PersonalData"/>
  </edmx:Reference>
  <edmx:Reference Uri="/sap/opu/odata/IWFND/CATALOGSERVICE;v=2/Vocabularies(TechnicalName='%2FIWBEP%2FVOC_ANALYTICS',Version='0001',SAP__Origin='LOCAL')/$value">
    <edmx:Include Namespace="com.sap.vocabularies.Analytics.v1" Alias="Analytics"/>
  </edmx:Reference>
  <edmx:Reference Uri="/sap/opu/odata/IWFND/CATALOGSERVICE;v=2/Vocabularies(TechnicalName='%2FIWBEP%2FVOC_COMMON',Version='0001',SAP__Origin='LOCAL')/$value">
    <edmx:Include Namespace="com.sap.vocabularies.Common.v1" Alias="SAP__common"/>
  </edmx:Reference>
  <edmx:Reference Uri="/sap/opu/odata/IWFND/CATALOGSERVICE;v=2/Vocabularies(TechnicalName='%2FIWBEP%2FVOC_MEASURES',Version='0001',SAP__Origin='LOCAL')/$value">
    <edmx:Include Namespace="Org.OData.Measures.V1" Alias="Measures"/>
  </edmx:Reference>
  <edmx:Reference Uri="/sap/opu/odata/IWFND/CATALOGSERVICE;v=2/Vocabularies(TechnicalName='%2FIWBEP%2FVOC_CORE',Version='0001',SAP__Origin='LOCAL')/$value">
    <edmx:Include Namespace="Org.OData.Core.V1" Alias="Core"/>
  </edmx:Reference>
  <edmx:Reference Uri="/sap/opu/odata/IWFND/CATALOGSERVICE;v=2/Vocabularies(TechnicalName='%2FIWBEP%2FVOC_CAPABILITIES',Version='0001',SAP__Origin='LOCAL')/$value">
    <edmx:Include Namespace="Org.OData.Capabilities.V1" Alias="Capabilities"/>
  </edmx:Reference>
  <edmx:Reference Uri="/sap/opu/odata/IWFND/CATALOGSERVICE;v=2/Vocabularies(TechnicalName='%2FIWBEP%2FVOC_AGGREGATION',Version='0001',SAP__Origin='LOCAL')/$value">
    <edmx:Include Namespace="Org.OData.Aggregation.V1" Alias="Aggregation"/>
  </edmx:Reference>
  <edmx:Reference Uri="/sap/opu/odata/IWFND/CATALOGSERVICE;v=2/Vocabularies(TechnicalName='%2FIWBEP%2FVOC_VALIDATION',Version='0001',SAP__Origin='LOCAL')/$value">
    <edmx:Include Namespace="Org.OData.Validation.V1" Alias="Validation"/>
  </edmx:Reference>
  <edmx:Reference Uri="/sap/opu/odata/IWFND/CATALOGSERVICE;v=2/Vocabularies(TechnicalName='%2FIWBEP%2FVOC_CODELIST',Version='0001',SAP__Origin='LOCAL')/$value">
    <edmx:Include Namespace="com.sap.vocabularies.CodeList.v1" Alias="CodeList"/>
  </edmx:Reference>
  <edmx:Reference Uri="/sap/opu/odata/IWFND/CATALOGSERVICE;v=2/Vocabularies(TechnicalName='%2FIWBEP%2FVOC_UI',Version='0001',SAP__Origin='LOCAL')/$value">
    <edmx:Include Namespace="com.sap.vocabularies.UI.v1" Alias="UI"/>
  </edmx:Reference>
  <edmx:Reference Uri="/sap/opu/odata/IWFND/CATALOGSERVICE;v=2/Vocabularies(TechnicalName='%2FIWBEP%2FVOC_HTML5',Version='0001',SAP__Origin='LOCAL')/$value">
    <edmx:Include Namespace="com.sap.vocabularies.HTML5.v1" Alias="SAP__HTML5"/>
  </edmx:Reference>
  <edmx:Reference Uri="/sap/opu/odata/IWFND/CATALOGSERVICE;v=2/Vocabularies(TechnicalName='%2FIWBEP%2FVOC_PDF',Version='0001',SAP__Origin='LOCAL')/$value">
    <edmx:Include Namespace="com.sap.vocabularies.PDF.v1" Alias="SAP__PDF"/>
  </edmx:Reference>
</edmx:Edmx>
```

Next Steps

You can now use this CSDL XML document in the Service Consumption Model wizard to generate the proxy model.

Generating the Proxy Model

Prerequisites

The metadata CSDL XML file of the service you want to consume is stored on your local machine.

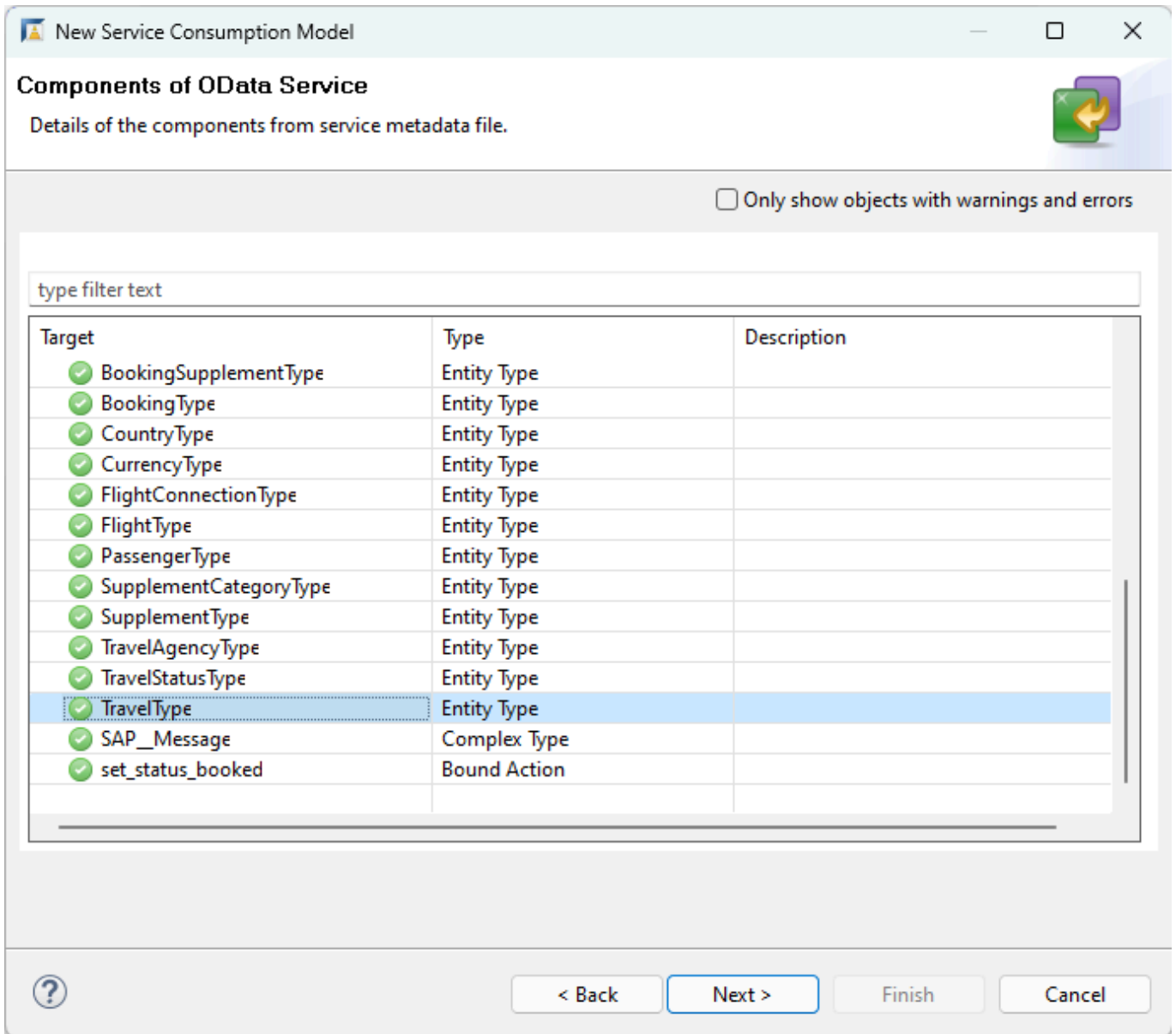
Context

The proxy model that represents the remote service in your system is required to access the remote service in your ABAP code. The Service Consumption Model wizard generates this proxy model as an ABAP class.

Procedure

- 1. In your ABAP package, open the context menu and choose **New** > **Other ABAP Repository Object** > **Business Services** > **Service Consumption Model** to launch the creation wizard.

2. In addition to the **Project** and **Package**, enter the name `/DMO/TRAVEL_SCM` and a **Description**. This artifact will serve as code snippet source for the implementation classes to be developed.
3. Choose **OData** as Remote Consumption Mode and click **Next**.
4. Browse your local machine for the **Service Metadata File** and provide the name `/DMO/CL_TRAVEL_SCM` for the proxy class to be generated.
5. Choose **Next**.
6. On the **Components of OData Service** screen, all entity types and sets to be generated are displayed.



7. Choose **Next** until you reach the **Select Transport Request** screen.
8. Assign a transport request and choose **Finish**.

The Service Consumption Model editor opens. This editor serves as a source for code snippets containing the names of the remote entities for data access to the remote service using an OData Client Proxy instance via an HTTP connection. Depending on the chosen entity and operation, the corresponding code is displayed.

Additionally, the proxy class `/DMO/CL_TRAVEL_SCM` has been created which contains the definitions of the remote entities. In this scenario, you will use the type `tys_travel_type` for the root node of the remote service. It is referenced in

sample declarations of variables contained in code snippets for the **Travel** node.

Service Consumption Model: /DMO/TRAVEL_SCM

General Information

View information on the Service Consumption Model

Consumption Type: OData

Model Class: [/DMO/CL_TRAVEL_SCM](#)

Service Entity Sets

View information on the service entity sets

type filter text

Element Name	Element Type
Airline	Entity Set
Airport	Entity Set
Booking	Entity Set
BookingSupplement	Entity Set
Country	Entity Set
Currency	Entity Set
Flight	Entity Set
FlightConnection	Entity Set
Passenger	Entity Set
Supplement	Entity Set
SupplementCategory	Entity Set
Travel	Entity Set
TravelAgency	Entity Set
TravelStatus	Entity Set

Service Entity Set Details

View information on selected service entity set

☒ ETag Supported

Code sample for entity set access

Operation: Read

[Copy to Clipboard](#)

```

DATA:
  ls_entity_key   TYPE /dmo/cl_travel_scm=>tys_travel_type,
  ls_business_data TYPE /dmo/cl_travel_scm=>tys_travel_type,
  lo_http_client  TYPE REF TO if_web_http_client,
  lo_resource     TYPE REF TO /iwbp/if_cp_resource_entity,
  lo_client_proxy TYPE REF TO /iwbp/if_cp_client_proxy,
  lo_request      TYPE REF TO /iwbp/if_cp_request_read,
  lo_response     TYPE REF TO /iwbp/if_cp_response_read.

TRY.
  " Create http client
  *DATA(lo_destination) = cl_http_destination_provider=>create_by_comm_arrangement(
  *      comm_scenario = '<Comm Scenario>'
  *      comm_system_id = '<Comm System Id>'
  *      service_id = '<Service Id>' ).
  *lo_http_client = cl_web_http_client_manager=>create_by_http_destination( lo_destination ).
  lo_client_proxy = /iwbp/cl_cp_factory_remote=>create_v4_remote_proxy(
    EXPORTING
      is_proxy_model_key = VALUE #( repository_id = 'DEFAULT'
                                     proxy_model_id = '/DMO/TRAVEL_SCM'
                                     proxy_model_version = '0001' )
      io_http_client = lo_http_client
      iv_relative_service_root = '<service_root>' ).

  ASSERT lo_http_client IS BOUND.

  " Set entity key
  ls_entity_key = VALUE #(
    travel_id = 'TravelId' ).

  " Navigate to the resource
  lo_resource = lo_client_proxy->create_resource_for_entity_set( 'TRAVEL' )->navigate_with_key( ls_entity_key ).

  " Execute the request and retrieve the business data
  lo_response = lo_resource->create_request_for_read( )->execute( ).
  lo_response->get_business_data( IMPORTING es_business_data = ls_business_data ).

```

In the generated proxy class, the type **tys_travel_type** is defined as follows:


```

368     TYPES:
369     "!" <p class="shorttext synchronized" lang="en">TravelType</p>
370     BEGIN OF tys_travel_type,
371     "!" <em>Value Control Structure</em>
372     value_controls TYPE tys_value_controls_2,
373     "!" <em>Key property</em> TravelID
374     travel_id      TYPE c LENGTH 8,
375     "!" AgencyID
376     agency_id      TYPE c LENGTH 6,
377     "!" AgencyName
378     agency_name    TYPE c LENGTH 80,
379     "!" CustomerID
380     customer_id    TYPE c LENGTH 6,
381     "!" CustomerName
382     customer_name  TYPE c LENGTH 40,
383     "!" BeginDate
384     begin_date     TYPE datn,
385     "!" EndDate
386     end_date       TYPE datn,
387     "!" BookingFee
388     booking_fee    TYPE decfloat16,
389     "!" TotalPrice
390     total_price    TYPE decfloat16,
391     "!" CurrencyCode
392     currency_code  TYPE c LENGTH 3,
393     "!" Memo
394     memo           TYPE c LENGTH 1024,
395     "!" Status
396     status         TYPE c LENGTH 1,
397     "!" StatusText
398     status_text    TYPE c LENGTH 1024,
399     "!" LastChangedAt
400     last_changed_at TYPE timestamp1,
401     "!" SAP__Messages
402     sap_messages   TYPE tyt_sap_message,
403     "!" odata.etag
404     etag           TYPE string,
405     END OF tys_travel_type,
406     "!" <p class="shorttext synchronized" lang="en">List of TravelType</p>

```

i Note

Element types, the semantics annotations, and the OData entity set name, which is referenced in the annotation, are taken from the service metadata document of the remote service. If an ETAG is maintained in the original service, an element for the ETAG is added. The text element for the elements that have a text association in the original service are also added to the type definition. We will not use them in this scenario.

Related Information

[Creating Service Consumption Model](#)

Creating a Database Table for the Persistent Fields

Context

The remote service is enriched with the discount field `discount_pct` which enables the user to maintain percentage discounts relative to the total prize of a trip. The calculated discount is stored in the transient field `TotalPriceWithDiscount`.

To create the database table for the discount field, follow the steps.

Procedure

1. In the **Project Explorer**, select the relevant **Package** node, open the context menu, and choose **►New ►Other ABAP Repository Object ►Dictionary**.

2. Select **Database Table** to launch the creation wizard.

The **Creation wizard** opens.

3. Enter the necessary information for the wizard to create the database table `/DM0/TRVL_SCM_DC`.

Once you have completed the wizard, the initially generated source code is displayed and ready for editing.

4. Define the fields for the database table.

```
@EndUserText.label : 'Travel Discount Information'
@AbapCatalog.enhancement.category : #NOT_EXTENSIBLE
@AbapCatalog.tableCategory : #TRANSPARENT
@AbapCatalog.deliveryClass : #A
@AbapCatalog.dataMaintenance : #RESTRICTED
define table /dmo/trvl_scm_dc {

    key client      : abap.clnt not null;
    key travel_id   : abap.numc(8) not null;
    discount_pct    : abap.dec(3,1);
    lastchangedat   : timestamp;

}
```

5. Save and activate.

Related Information

[Creating Database Tables](#)

Using a CDS Custom Entity for Data Modeling

Custom entities are used for data models whose runtime is implemented manually.

Context

This service consumption scenario retrieves its data partly from a remote service and partly from a local database table. In custom entities, you define the elements and type them to be used in the local OData service. Custom entities do not come with a `select` on the data source. The implementation of the logic to retrieve the data is implemented in an ABAP class that is referenced in an entity annotation.

For more information, see [Unmanaged Query](#).

Creating a CDS Custom Entity

Procedure

1. In your ABAP project, select the relevant package node in the **Project Explorer**. Open the context menu and choose **►New ►Other... ►Core Data Services ►Data Definition** to launch the creation wizard.

2. Enter the necessary information for the wizard to create the CDS custom entity. Choose **Define Custom Entity with Parameters** as the template.

A new data definition for a CDS custom entity is created and added to the **Core Data Services** folder of the corresponding package node. In our scenario, the name of the custom entity is `/DMO/R_TRAVEL_SCM_TP`.

3. Delete the `with parameters` statement. Parameters are not needed in the current scenario.

Results

Unlike CDS views, custom entities do not have a `select` statement to retrieve the data from a data source. The runtime of a custom entity must be implemented in a related ABAP class, which must be referenced using the annotation `@ObjectModel.query.implementedBy`.

Next Steps

The elements can now be defined in the editor. At least one element must be **key**. It is also necessary to determine a class for the query implementation in the annotation `@ObjectModel.query.implementedBy`.

Defining the Data Model in a CDS Custom Entity

Context

As described in [Scenario Description](#), the data for the new OData service is partly retrieved from a remote system and partly taken from a local database. The data model for the service to be developed is therefore composed from the remote data model and fields from the local database table. In addition, the data model needs to include transient fields that are calculated from the sources mentioned above.

i Note

Some fields are defined in camel case in the remote data model. The Service Consumption Model, however, uses underscores for these fields in the generated proxy class since ABAP is not case sensitive. In order to avoid that field names of the generated data model need to be mapped to field names of the custom entity, you need to use the field names from the generated data model in the custom entity. This example scenario shows by means of the remote field `Memo` how such a field name mapping can be implemented. This field is named `Description` in the custom entity.

Procedure

1. If you have not already done so, open the new data definition for the CDS custom entity in the editor.
2. Define the data model for the new travel service according to the definition of the type `tys_travel_type` contained in the proxy class `/DMO/CL_TRAVEL_SCM`. The components `value_controls`, `agency_name`, `customer_name`, `status_text` and `sap_messages` are not included in the custom entity. For the etag handling, define a dedicated `CalculatedEtag` field which considers changes on the local database table.
3. Include the additional discount field from `/DMO/TRVL_SCM_DC`.

```
discountpct : abap.dec( 3, 1 );
```

4. Add the element for the transient field that calculates the total price with discount.

```
totalpricewithdiscount : abap.dec(17,3);
```

5. Add an element for the calculated eTag.

```
calculatedetag : abap.sstring( 1024 );
```

6. Define the semantic relation between the amount and currency fields with the respective annotation.

```
@Semantics.amount.currencyCode: 'currency_code'
```

Results

The following codeblock displays the custom entity elements with the correct types and semantic annotations.

```
@EndUserText.label: 'CE for Service Consumption Scenario'
define custom entity /DMO/R_TRAVEL_SCM_TP
{
    key travel_id          : abap.numc( 8 );
    agency_id             : abap.numc( 6 );
    customer_id           : abap.numc( 6 );
    begin_date            : abap.dats;
    end_date              : abap.dats;
    @Semantics.amount.currencyCode: 'currency_code'
    booking_fee           : abap.dec( 17, 3 );
    @Semantics.amount.currencyCode: 'currency_code'
    total_price           : abap.dec( 17, 3 );
    currency_code         : abap.cuky( 5 );
    description           : abap.char( 1024 ); //renamed element
    status                : abap.char( 1 );
    last_changed_at       : timestamp;
    discountpct           : abap.dec( 3, 1 );
    @Semantics.amount.currencyCode: 'currency_code'
    totalpricewithdiscount : abap.dec(17,3);
    calculatedetag        : abap.sstring( 1024 );
}
```

Next Steps

Create an ABAP class that implements the interface IF_RAP_QUERY_PROVIDER. This class needs to be referenced in the entity annotation @ObjectModel.query.implementedBy.

Related Information

Creating the Query Implementation Class

Context

The runtime of a CDS custom entity must be implemented manually. Therefore, it requires an ABAP class that implements the select method of the interface IF_RAP_QUERY_PROVIDER to handle query requests by the client. This class is referenced by the custom entity and is dedicated to implementing the OData client data requests.

Procedure

1. In the **Project Explorer**, select the relevant package node, open the context menu, and choose **New > ABAP Class** to launch the creation wizard.
2. Follow the steps of the creation wizard and enter the necessary information. In the example scenario, we choose the name `/DMO/CL_R_TRAVELTP_SCM_QUERY` for the query implementation class.

Once you have completed the wizard, the initially generated source code is displayed and ready for editing.

3. Implement the `select` method of the query provider interface `IF_RAP_QUERY_PROVIDER`.

The custom entity can only be activated once the `select` method of the interface `IF_RAP_QUERY_PROVIDER` is implemented in the query implementation class.

```
CLASS /dmo/cl_r_traveltp_scm_query DEFINITION
  PUBLIC
  FINAL
  CREATE PUBLIC .

  PUBLIC SECTION.

    INTERFACES if_rap_query_provider .
  PROTECTED SECTION.
  PRIVATE SECTION.
ENDCLASS.

CLASS /dmo/cl_r_traveltp_scm_query IMPLEMENTATION.

  METHOD if_rap_query_provider~select.
  ENDMETHOD.
ENDCLASS.
```

4. In the custom entity `/DMO/R_TRAVEL_SCM_TP`, add the entity annotation `@ObjectModel.query.implementedBy` and reference the newly created query implementation class `/DMO/CL_R_TRAVELTP_SCM_QUERY`.

The reference to the ABAP class must start with `ABAP:`.

```
@ObjectModel.query.implementedBy: 'ABAP:/DMO/CL_R_TRAVELTP_SCM_QUERY'
@endUserText.label: 'CE for Service Consumption Scenario'
define custom entity /DMO/R_TRAVEL_SCM_TP
{ ...
```

5. Activate the custom entity `/DMO/R_TRAVEL_SCM_TP`.

Consuming the Remote OData Service

Address a remote service in your ABAP code and implement the query contract and the transactional behavior for the business object to build a new OData service.

Our service consumption scenario includes the creation of a new OData service in the consuming tenant. You have defined a data model for this service in a CDS custom entity, and now, the unmanaged query must be implemented in a related ABAP class as the query is not managed by a framework.

Additionally, this scenario also illustrates how transactional behavior can be included for the new OData service. To update the discount data, the business object (consisting only of the custom entity in this case) must be equipped with the implementation of the transactional runtime in a behavior pool. This includes the implementation of the interaction phase and save sequence.

Both, the query and the behavior implementation need to retrieve data from the remote service. With the help of a remote client proxy, the connection to the provisioning system is established and read requests are delegated to consume the remote service. The client proxy implementation is done in an auxiliary class for reuse reasons.

→ Remember

The consumption of a Web API requires the configuration of the involved systems (provisioning and consuming system) to allow the communication between them.

See [Prerequisites for the Consumption of a Remote Service](#).

i Note

If you face problems when setting up the OData client proxy connection based on the configured communication arrangements, you can use ABAP Cross Trace to analyze the URL and the payload requested respectively.

The following sections guide you through the implementation steps to instantiate a remote client proxy to reach the provisioning system, to implement the query for read access to the remote service and to implement transactional access to the discount persistence layer.

- [Creating a Remote Client Proxy](#)
- [Implementing the Query for Service Consumption](#)
- [Adding Transactional Behavior to the Business Object](#)

Creating a Remote Client Proxy

To enable data retrieval from a remote service, you must establish a connection to the provisioning system and instantiate a client proxy that passes the OData requests to the remote service.

Prerequisites

To be able to address a remote service, make sure that you meet the [Prerequisites for the Consumption of a Remote Service](#).

Context

A remote client proxy is used when an OData service is consumed remotely using an HTTP request. For reuse reasons, it is useful to instantiate the client proxy in a separate helper class.

Exception and Message Handling

In the service consumption scenario, several types of errors might occur. The execution of query or transactional requests is quite complex, as one request must pass not only the consuming service but also the provisioning service. In different parts of your coding, errors can happen. For example, while trying to connect to the provisioning system, while instantiating the client proxy, while executing the unmanaged query, during the execution of the application logic or during the modification of the discount values. Every kind of error must be handled with a suitable exception and message. Therefore, it is convenient to unite all exceptions and their related message in one exception class and one message class.

Before starting with the implementation of the consumption of the remote service, create an exception and a message class.

The exception class must inherit from the superclass `CX_RAP_QUERY_PROVIDER`.

In the example scenario, the name of the exception class is `CX_TRAVEL_SERV_CONS`. The name of the message class is `CM_SERV_CONS`.

Implementation Steps

Creating an Auxiliary Class for the Client Proxy Implementation

- In the [Project Explorer](#), select the relevant package node, open the context menu, and choose  **New**  **ABAP Class**  to launch the creation wizard.
- Follow the steps of the creation wizard and enter the necessary information. In the example scenario, we choose the name `/DMO/CL_TRAVEL_CP_AUX`. Once you have completed the wizard, the initially generated source code is displayed and ready for editing.
- Declare a static method in the public section of the auxiliary class for the creation of a client proxy:
`create_client_proxy` with a returning parameter `ro_client_proxy` and an exception to be raised if there are errors. .

```

CLASS /dmo/cl_travel_cp_aux DEFINITION
  PUBLIC
  FINAL
  CREATE PUBLIC .

  PUBLIC SECTION.
    CLASS-METHODS:
      get_client_proxy RETURNING VALUE(ro_client_proxy) TYPE REF TO /iwbep/if_cp_client_proxy
                        RAISING      /dmo/cx_travel_serv_cons.

  PROTECTED SECTION.
  PRIVATE SECTION.
ENDCLASS.

CLASS /dmo/cl_travel_cp_aux IMPLEMENTATION.
  METHOD get_client_proxy.

  ENDMETHOD.

ENDCLASS.

```

Creating an HTTP destination to the provisioning system

To be able to access the provisioning tenant, you need to create an HTTP destination using the communication arrangement configured beforehand. To do so, use a code sample offered by the Service Consumption Model artifact [/DMO/TRAVEL_SCM](#) as the basis and provide the respective values of your communication configuration.

- Call the static method `create_by_comm_arrangement` of the class `CL_HTTP_DESTINATION_PROVIDER` to create an HTTP destination using your communication configuration. In our scenario, the communication scenario and system is set up under the name `TRAVEL_BASIC` and the service ID is `OutboundCommunication`.
- Call the static method `create_by_http_destination` of the class `CL_WEB_HTTP_CLIENT_MANAGER` to create an HTTP client using the created HTTP destination.
- Include error handling. Use the exception class `/DMO/CX_TRAVEL_SERV_CONS` to raise an adequate exception message.

```

  TRY.
    " Create http client
    DATA(lo_destination) = cl_http_destination_provider=>create_by_comm_arrangement(
                        comm_scenario = 'TRAVEL_BASIC'

```

```

                                comm_system_id = 'TRAVEL_BASIC'
                                service_id      = 'OutboundCommunication' ).
lo_http_client = cl_web_http_client_manager=>create_by_http_destination( lo_destination ).

CATCH cx_http_dest_provider_error INTO DATA(lx_web_http_client_error).
  RAISE EXCEPTION TYPE /dmo/cx_travel_serv_cons
  EXPORTING
    textid      = /dmo/cx_travel_serv_cons=>remote_access_failed
    previous    = lx_web_http_client_error.

CATCH cx_web_http_client_error INTO DATA(lx_http_client_error).
  RAISE EXCEPTION TYPE /dmo/cx_travel_serv_cons
  EXPORTING
    textid      = /dmo/cx_travel_serv_cons=>remote_access_failed
    previous    = lx_web_http_client_error.

ENDTRY.

```

3. Instantiate the Client Proxy

To consume the remote service from the provisioning system, we need a client proxy that delegates the requests to the remote service. Since our scenario works with OData V4, we need a V4 client proxy.

The Service Consumption Model provides code templates for this.

- Call the static method `create_v4_remote_proxy` of the class `/IWBEP/CL_CP_FACTORY_REMOTE` to instantiate an OData V4 client proxy. Provide the name `/DMO/TRAVEL_SCM` of the Service Consumption Model artifact as proxy model ID. Provide the relative URL path to the remote service.
- Use the exception class `/DMO/CX_TRAVEL_SERV_CONS` to raise adequate exception message.

```

" Instantiation of client proxy
ro_client_proxy = /iwbsp/cl_cp_factory_remote=>create_v4_remote_proxy(
  EXPORTING
    is_proxy_model_key      = VALUE #( repository_id      = 'DEFAULT'
                                        proxy_model_id      = '/DMO/TRAVEL_SCM'
                                        proxy_model_version = '0001' )
    io_http_client          = lo_http_client
    iv_relative_service_root = '<service_root>' ).

...

CATCH /iwbsp/cx_gateway INTO DATA(lx_gateway_error).
  RAISE EXCEPTION TYPE /dmo/cx_travel_serv_cons
  EXPORTING
    textid      = /dmo/cx_travel_serv_cons=>client_proxy_failed
    previous    = lx_gateway_error.

ENDTRY.

```

The resulting method looks like this:

```

METHOD get_client_proxy.

DATA:
  lo_http_client TYPE REF TO if_web_http_client,
  lo_client_proxy TYPE REF TO /iwbsp/if_cp_client_proxy.

TRY.
" Create http client
DATA(lo_destination) = cl_http_destination_provider=>create_by_comm_arrangement(
                                comm_scenario = 'TRAVEL_BASIC'
                                comm_system_id = 'TRAVEL_BASIC'
                                service_id     = 'OutboundCommunication' ).
  lo_http_client = cl_web_http_client_manager=>create_by_http_destination( lo_destination ).

```



```

" Instantiation of client proxy
ro_client_proxy = /iwbep/cl_cp_factory_remote=>create_v4_remote_proxy(
EXPORTING
    is_proxy_model_key      = VALUE #( repository_id      = 'DEFAULT'
                                         proxy_model_id    = '/DMO/TRAVEL_SCM'
                                         proxy_model_version = '0001' )

    io_http_client          = lo_http_client
    iv_relative_service_root = '<service_root>' ).

CATCH cx_http_dest_provider_error INTO DATA(lx_web_http_client_error).
    RAISE EXCEPTION TYPE /dmo/cx_travel_serv_cons
    EXPORTING
        textid    = /dmo/cx_travel_serv_cons=>remote_access_failed
        previous = lx_web_http_client_error.

CATCH cx_web_http_client_error INTO DATA(lx_http_client_error).
    RAISE EXCEPTION TYPE /dmo/cx_travel_serv_cons
    EXPORTING
        textid    = /dmo/cx_travel_serv_cons=>remote_access_failed
        previous = lx_web_http_client_error.

CATCH /iwbep/cx_gateway INTO DATA(lx_gateway_error).
    RAISE EXCEPTION TYPE /dmo/cx_travel_serv_cons
    EXPORTING
        textid    = /dmo/cx_travel_serv_cons=>client_proxy_failed
        previous = lx_gateway_error.

ENDTRY.

ENDMETHOD.

```

Related Information

[Working with Messages and Message Classes](#)

[Creating ABAP Exception Classes](#)

[Adding Code Templates to an ABAP Exception Class](#)

[Creating ABAP Classes](#)

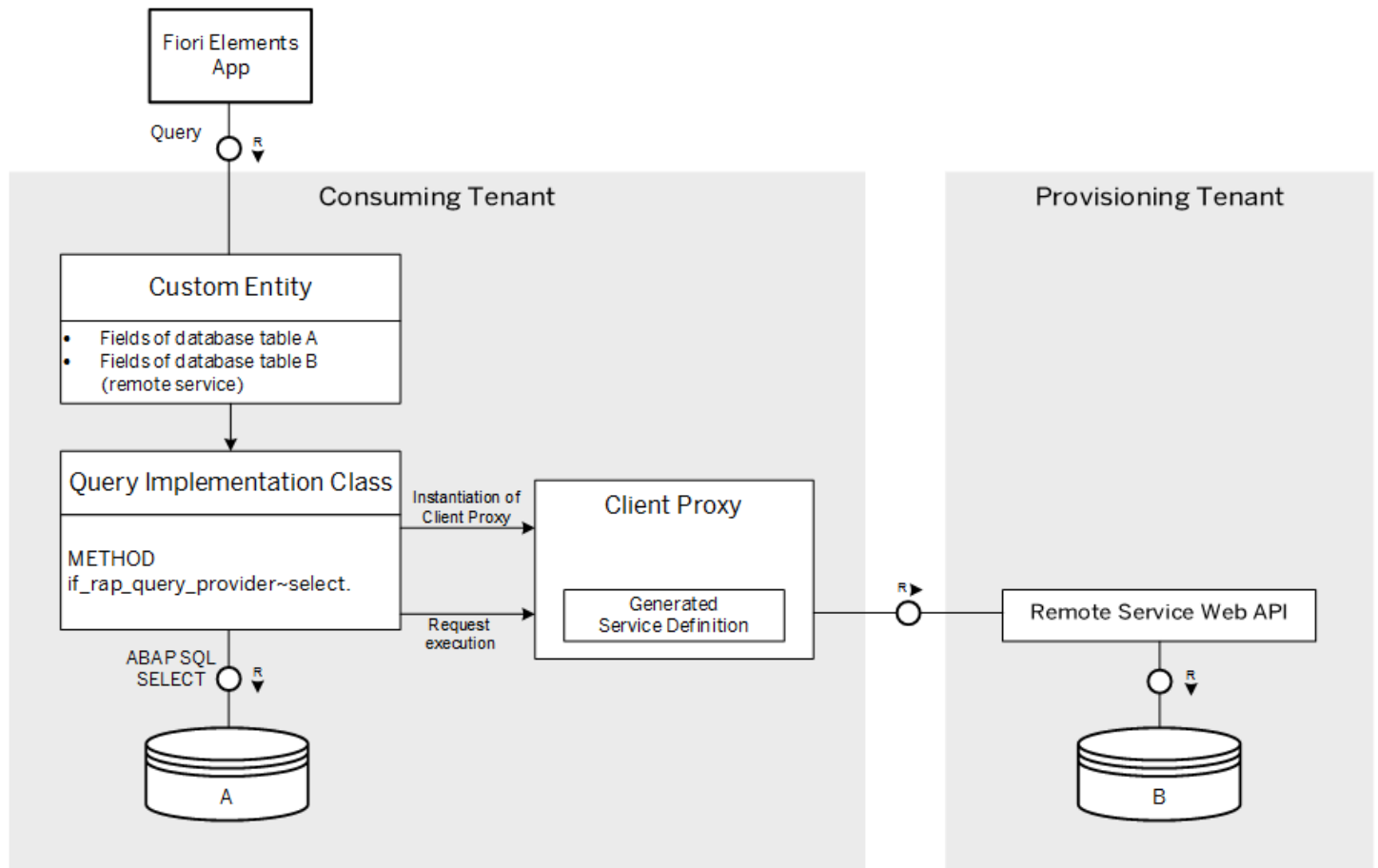
Implementing the Query for Service Consumption

The following topics guide you through the implementation steps required to execute a query request in a service consumption scenario.

Runtime Logic for the Query in the Service Consumption Scenario

Our scenario has two different data sources. One is a remote service from a possibly different system. To retrieve data from this service, you must have an appropriate connection to the provisioning tenant (see [Prerequisites for the Consumption of a Remote Service](#)) and the `select` method in the query implementation class must instantiate a client proxy that executes requests for the remote service. Additionally, it must include the implementation of every query option you want to support. The other data source is a database table in the consuming system that stores the additional discount field. To retrieve the data from this table, the `select` method includes an ABAP SQL `SELECT`.

The following figure illustrates how the query works in the service consumption scenario.



Query for Service Consumption Scenario

Apart from the simple data retrieval, the scenario also includes the query options **filtering**, **selecting**, **sorting**, and **paging**. All of these capabilities must be implemented in the query implementation class.

The client proxy is a powerful means to pass on OData requests to the provisioning tenant. It must be instantiated in the query implementation class. All the possible query options of the OData request in the consuming system must then be added to the request for the client proxy. This needs to be done for every query option separately. After the actual execution of the request and the data retrieval, the data set from the remote service is extended with its matching discount data.

i Note

The query options can only be delegated to the client proxy if the query options only affect the elements from the remote service. If the query options are operated on local database fields, they must be implemented manually after the retrieval of the data set from the provisioning system. As this might cause serious performance issues, it is not demonstrated in this example scenario. Consider performance aspects for your use case, if you decide to implement this.

Structure of the Query

For every request that is sent to the remote service you need to instantiate the client proxy. With this client proxy, you create a read request (as you only want to execute a query on the remote service). Before sending this read request to the remote service, it must be equipped with all the necessary query options. For example, whenever the count is requested by the UI, the query option for count must be added to the read request. The interface **IF_RAP_QUERY_PROVIDER** provides methods to find out which query options are requested by the UI and hence must be delegated to the remote service.

i Note

When choosing the **GO** button on the UI to retrieve data, it always requests the count and paging. Therefore, the implementation for data retrieval must include at least counting and paging. Otherwise you will get a runtime error.

When all necessary query options are added to the request, it can be executed and the result must be given back to the UI.

The procedure for implementing the query implementation is described in the following in detail:

- 1. [Implementing Data and Count Retrieval](#)
- 2. [Implementing Filtering](#)
- 3. [Implementing Column Selections](#)
- 4. [Implementing Sorting](#)

Related Information

[Query Implementation Types](#)

Implementing Data and Count Retrieval

To enable data retrieval from a remote service, you must establish a connection to the provisioning tenant and instantiate a client proxy that passes the OData requests to the remote service. The data sets from the remote service are then imported to the consuming tenant and can be merged with additional data.

Preview

Choosing the **Go** button on an SAP Fiori UI triggers the OData request for data and count retrieval. The UI automatically includes a paging query option with usually 20 data records per page.

Standard

Go Adapt Filters

Travel ID:Agency ID:Customer ID:

^

↻

Travel ID	Agency ID	Customer ID	Starting Date	End Date	Comment	Total Price	Total Price with Discount	Discount %	Status	
1	70041	594	Jun 3, 2023	Mar 31, 2024	Vacation	1,889.00 USD	1,851.22 USD	2.0	P	>
2	70007	608	Jun 3, 2023	Jun 3, 2023	Business Trip for Christine, Pierre	900.00 USD	630.00 USD	30.0	N	>
3	70046	93	Jun 3, 2023	Mar 31, 2024	Vacation	4,164.00 USD	4,164.00 USD	0.0	P	>
4	70042	665	Jun 3, 2023	Mar 31, 2024	Vacation	1,871.00 USD	1,871.00 USD	0.0	P	>
5	70007	225	Jun 3, 2023	Jun 3, 2023	Business Trip for Kurt, Ida	992.00 USD	992.00 USD	0.0	N	>
6	70049	72	Jun 3, 2023	Mar 31, 2024	Vacation	5,586.00 USD	5,586.00 USD	0.0	P	>
7	70046	138	Jun 3, 2023	Mar 31, 2024	Vacation	5,691.00 USD	5,691.00 USD	0.0	P	>
8	70012	705	Jun 3, 2023	Jun 5, 2023	Vacation for Lothar, Ulla, Thilo,	2,777.00 USD	2,777.00 USD	0.0	N	>
9	70032	115	Jun 3, 2023	Mar 31, 2024	Visiting Anna	5,792.00 USD	5,792.00 USD	0.0	B	>
10	70016	697	Jun 3, 2023	Jun 5, 2023	Business Trip to USA	959.00 USD	959.00 USD	0.0	N	>
11	70005	582	Jun 3, 2023	Jun 3, 2023	Vacation	959.00 USD	959.00 USD	0.0	N	>
12	70028	583	Jun 3, 2023	Mar 29, 2024	Sightseeing in Miami, Florida	4,304.00 USD	4,304.00 USD	0.0	B	>
13	70014	4	Jun 3, 2023	Jun 5, 2023	Vacation for Thomas, Jasmin,	2,014.00 USD	2,014.00 USD	0.0	N	>
14	70010	667	Jun 3, 2023	Jun 3, 2023	Business Trip for Anna	544.00 USD	544.00 USD	0.0	N	>
15	70028	466	Jun 3, 2023	Mar 29, 2024	Sightseeing in Miami, Florida	3,001.00 USD	3,001.00 USD	0.0	B	>
16	70050	640	Jun 3, 2023	Mar 31, 2024	Vacation	5,937.00 USD	5,937.00 USD	0.0	P	>
17	70048	478	Jun 3, 2023	Mar 31, 2024	Vacation	6,087.00 USD	6,087.00 USD	0.0	P	>
18	70050	264	Jun 3, 2023	Mar 31, 2024	Vacation	4,040.00 USD	4,040.00 USD	0.0	P	>
19	70049	155	Jun 3, 2023	Mar 31, 2024	Vacation	2,042.00 USD	2,042.00 USD	0.0	P	>
20	70049	680	Jun 3, 2023	Mar 31, 2024	Vacation	6,272.00 USD	6,272.00 USD	0.0	P	>

Implementation Steps

1. Instantiate the Client Proxy

To send requests to the provisioning system, you need a remote client proxy.

- In the query implementation class, call the method `get_client_proxy` of the auxiliary class `/DMO/CL_TRAVEL_CP_AUX` in the `select` method.

```
METHOD if_rap_query_provider~select.
```

```
" Instantiate Client Proxy
DATA(lo_client_proxy) = /dmo/cl_travel_cp_aux=>get_client_proxy( ).

ENDMETHOD.
```

2. Create a Read Request

To execute a query request on the remote service, a read request for the relevant entity set must be created. This read request will then be enhanced with query options.

- On the client proxy object, call the method `create_resource_for_entity_set` with the **TRAVEL** entity set name as importing parameter.

i Note

The OData entity set name can be found in the respective constant structure of the generated proxy class:

```
446      "!" Travel
447      "!" <br/> Collection of type 'TravelType'
448      travel          TYPE /iwbsp/if_cp_runtime_types=>ty_entity_set_name VALUE 'TRAVEL',
449      "!" TravelAgency
450      "!" <br/> Collection of type 'TravelAgencyType'
451      travel_agency    TYPE /iwbsp/if_cp_runtime_types=>ty_entity_set_name VALUE 'TRAVEL_AGENCY',
452      "!" TravelStatus
453      "!" <br/> Collection of type 'TravelStatusType'
454      travel_status     TYPE /iwbsp/if_cp_runtime_types=>ty_entity_set_name VALUE 'TRAVEL_STATUS',
455      END OF gcs_entity_set .
```

This method returns an object for the entity set resource.

- Create a read request by calling the method `create_request_for_read` on the entity set resource.
- Surround the method call with a TRY - CATCH block and include error handling with a suitable error message.

```
TRY.
" Create Read Request
DATA(lo_read_request) = lo_client_proxy->create_resource_for_entity_set( 'TRAVEL' )->create_request_for_read( ).

CATCH /iwbsp/cx_gateway INTO DATA(lx_gateway).
  RAISE EXCEPTION TYPE /dmo/cx_travel_serv_cons
    EXPORTING
      textid      = /dmo/cx_travel_serv_cons=>query_failed
      previous    = lx_gateway.
ENDTRY.
```

3. Request the Inline Count of the Remote Service

As a Fiori UI always requests the inline count of the service, this request must be delegated to the remote service to get the total number of records from there.

- Within the try-catch Block, check whether the total number of records is requested by the UI by calling the method `is_total_numb_of_rec_requested` of the interface `IF_RAP_QUERY_REQUEST`.

For more information about `is_total_num_of_rec_requested`, see [Method `is\_total\_num\_of\_rec\_requested`](#).

i Note

The same exception as previously defined can be used for all methods called on the read request.

- Call the method `request_count` on the read request.

```
" Request Count
IF io_request->is_total_num_of_rec_requested( ).
    lo_read_request->request_count( ).
ENDIF.
```

4. Implement Paging if Data is Requested

Whenever the UI requests paging, the query framework checks that the query does not return more data records than requested. Therefore, you need to set the paging information for the read request the client proxy sends to the remote service.

- Within the try-catch block, check whether data is requested. If data must be returned using the method `set_data`, the requested paging must be applied. If data does not have to be returned, if `is_data_requested` returns `abap_false`, paging is not relevant.
- Get the paging information of the UI request into a local variable (`ls_paging`) by calling the method `get_paging` of the interface `IF_RAP_QUERY_REQUEST`.

For more information about `get_paging`, see [Method `get\_paging`](#).

- If the offset is ≥ 0 , pass it to the read-request.
- Pass the page size to the read request if it is not unlimited.

i Note

If all data records are requested, the method returns the constant `page_size_unlimited`, which has the value `-1`. If this constant is returned, the query option `$skip` must not be passed to the read request.

```
" Request Data
IF io_request->is_data_requested( ).

    " Request Paging
    DATA(ls_paging) = io_request->get_paging( ).
    IF ls_paging->get_offset( ) >= 0.
        lo_read_request->set_skip( ls_paging->get_offset( ) ).
    ENDIF.
    IF ls_paging->get_page_size( ) <> if_rap_query_paging=>page_size_unlimited.
        lo_read_request->set_top( ls_paging->get_page_size( ) ).
    ENDIF.
ENDIF.
```

5. Execute the Request

The read request for the remote OData service is now equipped with the query options for count `$count=true` (if total number of records is requested) and for paging `$skip` and `$top` (if data is requested). At least, these two query options should be implemented in the query implementation class if the consuming service is a UI service.

→ Remember

By default, the UI sends the query options for counting and paging.

Now, the request can be sent. In other words, it is executed on the remote service.

- Execute the request and write it into a local variable `lo_response`.

```
" Execute the Request
DATA(lo_response) = lo_read_request->execute( ).
```

6. Provide the Count Result for the Response of the Unmanaged Query

The response parameter `io_response` of `IF_RAP_QUERY_PROVIDER~select` must be filled with the total number of records that the count request to the remote service returns.

- Check if the total number of records is requested.
- Get the count from the response of the remote service by calling the method `get_count` on the remote service response object.
- Set the returned number for the response to the unmanaged query by calling the method `set_total_number_of_records` on the response object of the unmanaged query.

```
" Set Count
IF io_request->is_total_numb_of_rec_requested( ).
    io_response->set_total_number_of_records( lo_response->get_count( ) ).
ENDIF.
```

7. Provide the Result Data for the Response of the Unmanaged Query

The response parameter `io_response` of `IF_RAP_QUERY_PROVIDER~select` must be filled with the requested data, which are returned by the remote service and enhanced with the local discount data.

- Check whether data is requested.
- Declare `lt_travel` with a table type of the structure type `tys_travel_type` defined in the generated proxy class `/DMO/CL_TRAVEL_SCM`. This type is also used for sample variable declarations in the code samples generated by the Service Consumption Model artifact. The data from the remote service will be written into the declared internal table `lt_travel`.
- Declare `lt_travel_ce` with the type of the custom entity. This table is used to fill the output parameter of the select method.
- Declare `lt_traveladd` with the type of the local database table. The data from the database table `/DMO/TRVL_SCM_DC` is read into this table.
- Get the data from the response object of remote service request by calling the method `get_business_data` and write it into `lt_travel`.
- If `lt_travel` returns entries, provide a mapping for those elements in the custom entity that are not identical with the names of the abstract entity elements. In our scenario, this concerns the element `Description`.
- Select the local data (discounts) based on the entity sets that are retrieved from the remote service.
- For retrieved instances with matching keys in the local database table, set the following data in the internal table `lt_travel_ce`:
 - Set the percentage discount from the local database table

- Calculate the amount for the field `TotalPriceWithDiscount`
- Take over the time stamp from the local database table
- Calculate the etag based on the local time stamp
- For retrieved instances without matching keys in the local database table, use the retrieved total price for the field `TotalPriceWithDiscount` and set the static value '20000101120000' for the field `lastchangedat`
- Set the returned data for the response to the unmanaged query by calling the method `set_data` on the response object of the unmanaged query.

" Set Data

```
IF io_request->is_data_requested( ).
  DATA: lt_travel      TYPE STANDARD TABLE OF /DMO/CL_TRAVEL_SCM=>tys_travel_type,
        lt_travel_ce   TYPE STANDARD TABLE OF /DMO/R_TRAVEL_SCM_TP,
        lt_traveladd   TYPE STANDARD TABLE OF /DMO/TRVL_SCM_DC.

  lo_response->get_business_data( IMPORTING et_business_data = lt_travel ).
  IF lt_travel IS NOT INITIAL.
    lt_travel_ce = CORRESPONDING #( lt_travel MAPPING description = memo ).
    SELECT * FROM /DMO/TRVL_SCM_DC FOR ALL ENTRIES IN @lt_travel_ce WHERE travel_id = @lt_travel

    LOOP AT lt_travel_ce ASSIGNING FIELD-SYMBOL(<fs_travel_ce>).
      IF line_exists( lt_traveladd[ travel_id = <fs_travel_ce>-travel_id ] ).
        <fs_travel_ce>-discountpct      = lt_traveladd[ travel_id = <fs_travel_ce>-travel
        <fs_travel_ce>-totalpricewithdiscount = <fs_travel_ce>-total_price * ( 1 - <fs_travel
        <fs_travel_ce>-last_changed_at      = lt_traveladd[ travel_id = <fs_travel_ce>-trav
        <fs_travel_ce>-calculatedetag      = <fs_travel_ce>-calculatedetag && '-' && lt_travel
      ELSE.
        <fs_travel_ce>-totalpricewithdiscount = <fs_travel_ce>-total_price.
        <fs_travel_ce>-last_changed_at      = '20000101120000' . "initial value Jan 1,
      ENDIF.
    ENDLLOOP.
  ENDIF.

  io_response->set_data( lt_travel_ce ).
ENDIF.
```

Results

Expand the following listing to view the source code of the query implementation for data retrieval and count:

Query Implementation Class

```
CLASS /dmo/cl_r_traveltp_scm_query DEFINITION
  PUBLIC
  FINAL
  CREATE PUBLIC .

  PUBLIC SECTION.

    INTERFACES if_rap_query_provider .
  PROTECTED SECTION.
  PRIVATE SECTION.
ENDCLASS.

CLASS /dmo/cl_r_traveltp_scm_query IMPLEMENTATION.

  METHOD if_rap_query_provider~select.

    " Instantiate Client Proxy
    DATA(lo_client_proxy) = /dmo/cl_travel_cp_aux=>get_client_proxy( ).
```

TRY.

" Create Read Request

DATA(lo_read_request) = lo_client_proxy->create_resource_for_entity_set('TRAVEL')->create_reque

" Request Count

IF io_request->is_total_numb_of_rec_requested().

lo_read_request->request_count().

ENDIF.

" Request Data

IF io_request->is_data_requested().

" Request Paging

DATA(ls_paging) = io_request->get_paging().

IF ls_paging->get_offset() >= 0.

lo_read_request->set_skip(ls_paging->get_offset()).

ENDIF.

IF ls_paging->get_page_size() <> if_rap_query_paging=>page_size_unlimited.

lo_read_request->set_top(ls_paging->get_page_size()).

ENDIF.

ENDIF.

" Execute the Request

DATA(lo_response) = lo_read_request->execute().

" Set Count

IF io_request->is_total_numb_of_rec_requested().

lo_response->set_total_number_of_records(lo_response->get_count()).

ENDIF.

" Set Data

IF io_request->is_data_requested().

DATA: lt_travel TYPE STANDARD TABLE OF /DMO/CL_TRAVEL_SCM=>tys_travel_type,

lt_travel_ce TYPE STANDARD TABLE OF /DMO/R_TRAVEL_SCM_TP,

lt_traveladd TYPE STANDARD TABLE OF /DMO/TRVL_SCM_DC.

lo_response->get_business_data(IMPORTING et_business_data = lt_travel).

IF lt_travel IS NOT INITIAL.

lt_travel_ce = CORRESPONDING #(lt_travel MAPPING description = memo).

SELECT * FROM /DMO/TRVL_SCM_DC FOR ALL ENTRIES IN @lt_travel_ce WHERE travel_id = @lt_travel

LOOP AT lt_travel_ce ASSIGNING FIELD-SYMBOL(<fs_travel_ce>).

IF line_exists(lt_traveladd[travel_id = <fs_travel_ce>-travel_id]).

<fs_travel_ce>-discountpct = lt_traveladd[travel_id = <fs_travel_ce>-travel

<fs_travel_ce>-totalpricewithdiscount = <fs_travel_ce>-total_price * (1 - <fs_trav

<fs_travel_ce>-last_changed_at = lt_traveladd[travel_id = <fs_travel_ce>-trav

<fs_travel_ce>-calculatedetag = <fs_travel_ce>-calculatedetag && '-' && lt_trav

ELSE.

<fs_travel_ce>-totalpricewithdiscount = <fs_travel_ce>-total_price.

<fs_travel_ce>-last_changed_at = '20000101120000' . "initial value Jan 1,

ENDIF.

ENDLOOP.

ENDIF.

io_response->set_data(lt_travel_ce).

ENDIF.

CATCH /iwbsp/cx_gateway INTO DATA(lx_gateway).

RAISE EXCEPTION TYPE /dmo/cx_travel_serv_cons

EXPORTING

textid = /dmo/cx_travel_serv_cons=>query_failed

previous = lx_gateway.

ENDTRY.

ENDMETHOD.

ENDCLASS.

If you have defined a UI service for the custom entity (see [Defining an OData Service](#)), you can test your implementation with the SAP Fiori Elements Preview. When you open the app and choose the **Go** button, you receive the data and the count from the remote service together with the matching discount data.

Related Information

- [Implementing an Unmanaged Query](#)
- [Implementing Paging in an Unmanaged Query](#)
- [Interface IF_RAP_QUERY_REQUEST](#)

Implementing Filtering

Filter options can be delegated to the remote service. Fill the request that is passed to the OData client proxy with the query option `$filter`.

Preview

Travel ID:

Agency ID:

Customer ID:

Status: =P x

Travel ID	Agency ID	Customer ID	Starting Date	End Date	Comment	Total Price	Total Price with Discount	Discount %	Status	
1	70041	594	Jun 3, 2023	Mar 31, 2024	Vacation	1,889.00 USD	1,851.22 USD	2.0	P	>
3	70046	93	Jun 3, 2023	Mar 31, 2024	Vacation	4,164.00 USD	4,164.00 USD	0.0	P	>

To retrieve the appropriate entries for the filter, you must implement a filter factory to get the filtered results from the remote service.

Implementation Steps

This scenario only implements filtering by fields of the remote service. The service proxy API provides a filter factory to delegate the filter to the remote service. Filtering by local fields is not supported in this scenario.

i Note

To filter by local fields, you would have to retrieve all available data sets of the remote service and merge the local fields to these first, to be able to filter by discount values then. In this case, paging could not be delegated to the remote service either and would have to be implemented after data retrieval manually. Hence filtering for discount values might cause serious performance issues and is therefore not exemplified in this scenario.

i Note

Filtering on an amount field only works if the end user also provides a filter on the unit field, in our case a currency code. In our scenario, the fields `total_price`, `totalpricewithdiscount` and `booking_fee` are amount fields. If a filter is entered for these fields, the currency code must be in the filter condition as well. Hence, the implementation must also include the currency code in these cases.

The filter information is relevant for data and count retrieval. Therefore the implementation for the filter must be independent of the checks for data or count requests. The filter information must be added to the read request for the remote service. So the filter implementation must be called before executing the read request.

- The unmanaged query API provides a method to get the filter conditions from the UI OData request into a local variable. Use `io_request->get_filter( )->get_as_ranges( )` to get the filter conditions as ranges into a local variable `lt_filter`. Handle the exception if the filter cannot be converted into a ranges table.

For more information about the unmanaged query filter API, see [Interface IF RAP_QUERY_FILTER](#).

- Loop over `lt_filter`.

→ Remember

Some filter requests require special handling before sending them to the remote service:

- Filtering on local fields is not allowed.
- For filtering on an amount field, the unit must be provided in the filter.
- Raise an exception with an adequate message, for example `Filtering on &l is not allowed` if filter is requested for a local field.
- Find out the currency code for filter requests on amount fields.
- Raise an exception if currency code is not provided, for example `Currency code is not supplied for &l`.
- Create the filter property to be passed to the remote service. Map the element `Description` to `Memo` for the remote service to match the data model of the remote service.
- Create a filter factory to provide the filter request for the client proxy.
- Create the filter ranges for the remote service by calling the method `create_by_range` of the filter factory and export the filter property, the filter range and, if necessary, the currency code.
- If a filter is defined for more than one element, concatenate the filter condition.
- Set the filter for the request.

i Expand the following code sample to view the source code of *Request Filtering*.

≡ Sample Code

```
" Request Filtering
TRY.
    DATA(lt_filter) = io_request->get_filter( )->get_as_ranges( ).
    CATCH cx_rap_query_filter_no_range INTO DATA(lx_no_range).
        RAISE EXCEPTION TYPE /dmo/cx_travel_serv_cons
            EXPORTING
                textid = /dmo/cx_travel_serv_cons=>no_ranges
                previous = lx_no_range.
    ENDTRY.

LOOP AT lt_filter ASSIGNING FIELD-SYMBOL(<fs_filter>).
    IF <fs_filter>-name = 'DISCOUNTPCT' OR
       <fs_filter>-name = 'TOTALPRICEWITHDISCOUNT' OR
       <fs_filter>-name = 'CALCULATEDETAG'.
        RAISE EXCEPTION TYPE /dmo/cx_travel_serv_cons
            EXPORTING
                textid = /dmo/cx_travel_serv_cons=>filtering_failed
                element = <fs_filter>-name.
    ENDIF.
    "provide currency code, if filtering on amount field
    IF <fs_filter>-name = 'TOTAL_PRICE' OR
       <fs_filter>-name = 'BOOKING_FEE'.
        IF line_exists( lt_filter[ name = 'CURRENCY_CODE' ] ).
            DATA(lv_currencycode) = VALUE waers_curc( lt_filter[ name = 'CURRENCY_CODE' ]-rang
        ELSE.
            RAISE EXCEPTION TYPE /dmo/cx_travel_serv_cons
```

```

EXPORTING
    textid = /dmo/cx_travel_serv_cons=>no_currencycode
    element = <fs_filter>-name.
ENDIF.
ENDIF.
"map element names
DATA(lv_filter_property) = COND /iwbsp/if_cp_runtime_types=>ty_property_path( WHEN <fs_
                                                                    THEN 'ME
                                                                    ELSE <fs_

"create filter factory for read request
DATA(lo_filter_factory) = lo_read_request->create_filter_factory( ).
"
DATA(lo_filter_for_current_field) = lo_filter_factory->create_by_range( iv_property_pa
                                                                    it_range
                                                                    iv_currency_co

"Concatenate filter if more than one filter element
DATA: lo_filter          TYPE REF TO /iwbsp/if_cp_filter_node.
IF lo_filter IS INITIAL.
    lo_filter = lo_filter_for_current_field.
ELSE.
    lo_filter = lo_filter->and( lo_filter_for_current_field ).
ENDIF.
ENDLOOP.

"set filter
IF lo_filter IS NOT INITIAL.
    lo_read_request->set_filter( lo_filter ).
ENDIF.

```

Results

If you have defined a UI service for the custom entity (see [Defining an OData Service](#)), you can test your implementation with the SAP Fiori Elements Preview. You can define filters and check that the UI only retrieves data that matches the filter request.

Related Information

[Implementing Filtering in an Unmanaged Query](#)

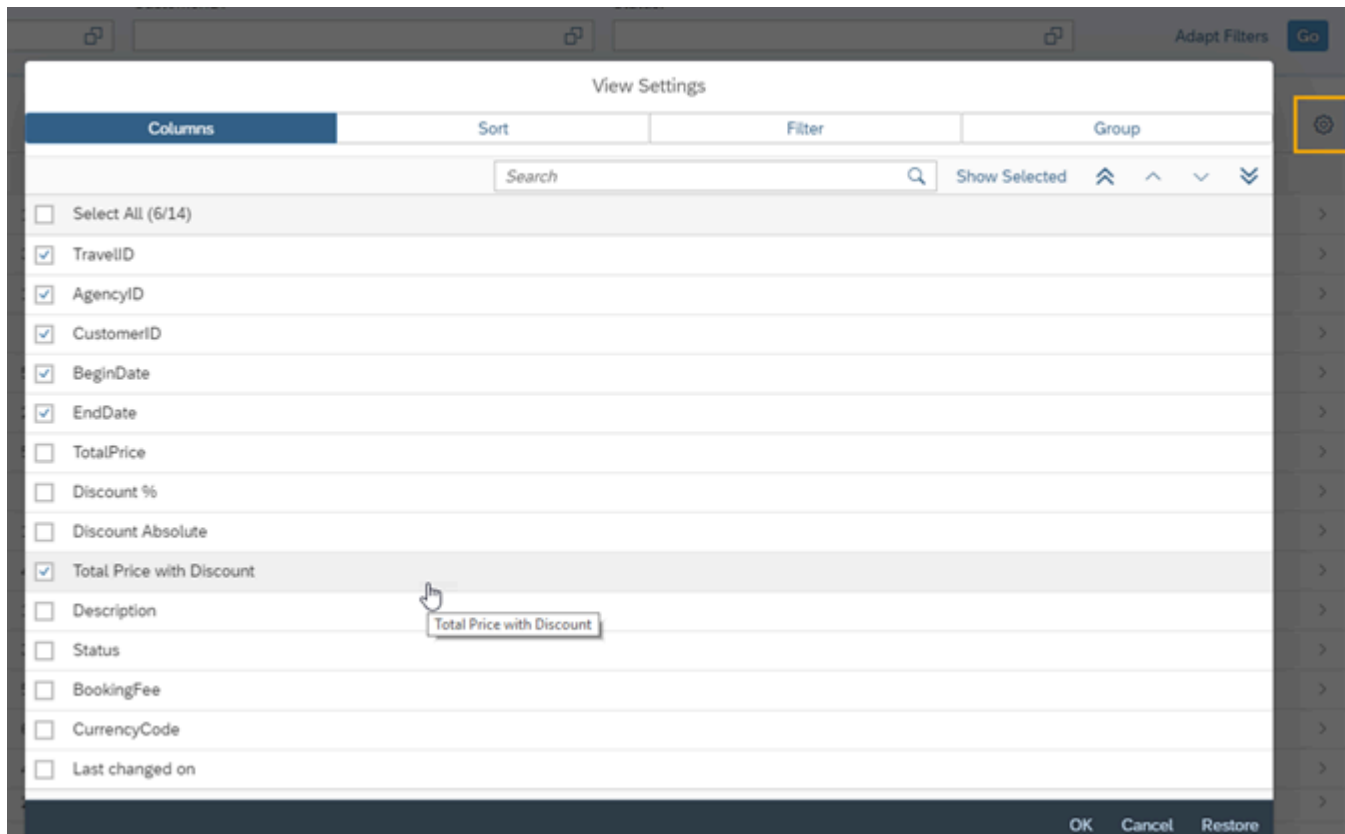
[Interface IF_RAP_QUERY_FILTER](#)

Implementing Column Selections

Select options can be delegated to the remote service. Fill the request that is passed to the OData client proxy with the query option `$select`.

Preview

The SAP Fiori UI allows you to select columns.



Implementation Steps

You can optimize the performance if you only select those elements from the remote service that are selected on the UI. The local fields are selected separately. Therefore, you must exclude the local fields from the requested elements as they are not delegated to the remote service. To retrieve the appropriate data, you must transfer the selected properties to the read request for the remote service.

Considering the requested elements from the UI when delegating the request to the remote service is only necessary when data is requested. Therefore the implementation must only be called if data is requested before the execution of the read request.

- The unmanaged query API provides a method to get those elements that are requested by the UI. Use `DATA(lt_req_elements) = io_request->get_requested_elements( ).` to get the elements into a local variable.

For more information about the unmanaged query API, see [Interface IF RAP_QUERY_REQUEST](#).

- Delete the local elements from the properties that are selected.

→ Remember

Local fields are not delegated to the remote service. They are selected manually.

- Loop over `lt_req_elements` and map the element `Description` to `MemoTo` to retrieve the appropriate data, you must transfer the selected properties to the for the remote service to match the data model of the remote service.
- Declare `lt_select_properties` and type it as a table of property paths. This table contains the properties that are given to the client proxy for creating the select query option.
- Fill the table `lt_select_properties` that is given to the client proxy with the elements to be selected from the remote service.
- Set the select properties for the OData request of the remote service.

```
" Request Elements
```

```
DATA(lt_req_elements) = io_request->get_requested_elements( ).
```

```
"delete local fields out of the fields to be selected via OData Client Proxy
```

```
DELETE lt_req_elements WHERE table_line = 'DISCOUNTPCT' OR
                                table_line = 'TOTALPRICEWITHDISCOUNT' OR
                                table_line = 'CALCULATEDETAG'.
```

```
"map differing names
```

```
LOOP AT lt_req_elements ASSIGNING FIELD-SYMBOL(<fs_req_elements>).
```

```
DATA(lv_select_property) = COND /iwbep/if_cp_runtime_types=>ty_property_path( WHEN <fs_
                                                                                     ELSE <fs_
```

```
DATA: lt_select_properties TYPE /iwbep/if_cp_runtime_types=>ty_t_property_path.
```

```
APPEND lv_select_property TO lt_select_properties.
```

```
ENDLOOP.
```

```
"set select properties
```

```
IF lt_select_properties IS NOT INITIAL.
```

```
lo_read_request->set_select_properties( lt_select_properties ).
```

```
ENDIF.
```

Results

If you have defined a UI service for the custom entity (see [Defining an OData Service](#)), you can test your implementation with the SAP Fiori Elements Preview. Trigger the selection by choosing columns on the SAP Fiori UI and check whether the request works.

Related Information

[Interface IF_RAP_QUERY_REQUEST](#)

[Considering Requested Elements in an Unmanaged Query](#)

Implementing Sorting

Sorting options can be delegated to the remote service. Fill the request that is passed to the OData client proxy with the relevant query option \$orderby.

Preview

Travel ID	Agency ID	Customer ID	Starting Date	End Date	Comment
1	70041	Sort By: Customer ID Ascending Descending ↓↑ Sort > [≡] Group > [≡] Columns >			ion
2	70007				ess Trip for Christine, Pierre
3	70046				ion
4	70042				ion
5	70007				ess Trip for Kurt, Ida
6	70049				ion
7	70046				ion
8	70012	705	Jun 3, 2023	Jun 5, 2023	Vacation for Lothar, Ulla, Thilo,
9	70032	115	Jun 3, 2023	Mar 31, 2024	Visiting Anna
10	70016	697	Jun 3, 2023	Jun 5, 2023	Business Trip to USA
11	70005	582	Jun 3, 2023	Jun 3, 2023	Vacation
12	70028	583	Jun 3, 2023	Mar 29, 2024	Sightseeing in Miami, Florida
13	70014	4	Jun 3, 2023	Jun 5, 2023	Vacation for Thomas, Jasmin,
14	70010	667	Jun 3, 2023	Jun 3, 2023	Business Trip for Anna
15	70028	466	Jun 3, 2023	Mar 29, 2024	Sightseeing in Miami, Florida
16	70050	640	Jun 3, 2023	Mar 31, 2024	Vacation
17	70048	478	Jun 3, 2023	Mar 31, 2024	Vacation
18	70050	264	Jun 3, 2023	Mar 31, 2024	Vacation
19	70049	155	Jun 3, 2023	Mar 31, 2024	Vacation
20	70049	680	Jun 3, 2023	Mar 31, 2024	Vacation

To retrieve the data in the appropriate order, you must add sorting information to the request for the OData client proxy.

Implementation Steps

This scenario only implements sorting by fields of the remote service. The service proxy API provides the method `set_orderby` to delegate the sorting order to the remote service.

i Note

To sort by local fields, you would have to retrieve all available data sets of the remote service and merge the local fields to these first, to be able to sort by discount values then. In this case, paging could not be delegated to the remote service either and would have to be implemented after data retrieval manually. Hence, sorting by discount values might cause serious performance issues and is therefore not exemplified in this scenario.

The sorting information is only relevant if you retrieve data. Therefore the implementation must only be called if data is requested to equip the read request for the remote service with this information.

- The unmanaged query API provides a method to get the sorting information. Use `DATA(lt_sort) = io_request->get_sort_elements( )` to get the sorting information into a local variable.

For more information about the unmanaged query API, see [Interface IF RAP_QUERY_REQUEST](#).

- Loop over `lt_sort`.

→ Remember

Sorting on local elements is not supported due to performance reasons.

- Raise an exception with an adequate message, for example `Sorting on &l is not allowed`.
- Declare `lt_sort_properties`. This table is passed to the client proxy with the elements to be sorted.
- Map the element `Description` to `Memo` for the remote service to match the data model of the remote service.
- Fill the table `lt_sort_properties` that is given to the client proxy with the elements to be selected from the remote service and map the columns.
- Set the sorting properties for the request by using the `set_orderby`.

" Request Sorting

```
DATA(lt_sort) = io_request->get_sort_elements( ).

LOOP AT lt_sort ASSIGNING FIELD-SYMBOL(<fs_sort>).
  IF <fs_sort>-element_name = 'DISCOUNTPCT' OR
    <fs_sort>-element_name = 'TOTALPRICEWITHDISCOUNT' OR
    <fs_sort>-element_name = 'CALCULATEDETAG'.
    RAISE EXCEPTION TYPE /dmo/cx_travel_serv_cons
      EXPORTING
        textid = /dmo/cx_travel_serv_cons=>sorting_failed
        element = <fs_sort>-element_name.
  ENDIF.
  "map differing names
  DATA: lt_sort_properties TYPE /iwbp/if_cp_runtime_types=>ty_t_sort_order.
  APPEND VALUE #( property_path = COND #( WHEN <fs_sort>-element_name = 'DESCRIPTION' THEN
                                          ELSE <fs_sort>-element_name )
    descending = <fs_sort>-descending )
    TO lt_sort_properties.
ENDLOOP.
"set sorting properties
IF lt_sort_properties IS NOT INITIAL.
  lo_read_request->set_orderby( lt_sort_properties ).
ENDIF.
```

Results

If you have defined a UI service for the custom entity (see [Defining an OData Service](#)), you can test your implementation with the SAP Fiori Elements Preview. Try out the sorting on different elements and check whether the results are sorted or an exception is thrown.

Related Information

[Interface IF RAP_QUERY_REQUEST](#)

Adding Transactional Behavior to the Business Object

The following steps guide you through the implementation steps to update a business object when a remote service is merged with local data.

Like in [Developing Unmanaged Transactional Apps](#), creating, updating, and deleting data requires a transactional runtime implementation. The service consumption scenario is also a use case for the unmanaged implementation type as the transactional logic is defined and implemented by the application developer. That means, the transactional behavior must be defined in a behavior definition and has to be implemented in ABAP classes of a behavior pool.

i Note

This scenario only supports transactional activities on the data that is persisted on a database table in the consuming tenant. Transactional operations on the remote service are not supported.

In this example scenario, only the discount data can be manipulated by the end user. The field on the database table `/DMO/TRVL_SCM_DC` that can be changed is `discount_pct`. The travel instance for which the discount is maintained is identified by the travel ID, which is retrieved from the remote service. Travel ID is a key field and therefore cannot be changed by the end user. For the eTag check, the distinct field `calculatedetag` is used. Its value is calculated using the timestamp of the field `lastchangedat` from the local database table.

The following tasks are relevant to include transactional behavior for the business object:

- [Defining a Behavior for the Business Object](#)
- [Implementing the Behavior Pool for the Business Object](#)

Defining a Behavior for the Business Object

The transactional behavior of the travel business object must be defined in a behavior definition.

To get transactional access to the local data, the data model must be turned into a business object. The custom entity `/DMO/R_TRAVEL_SCM_TP` must be defined as root entity.

1. Open the custom entity `/DMO/R_TRAVEL_SCM_TP` and insert the keyword `root`. A behavior definition can only be created for a root entity.

```
define root custom entity /DMO/R_TRAVEL_SCM_TP
```

2. Activate the root custom entity.

Creating a Behavior Definition `/DMO/R_TRAVEL_SCM_TP`

By creating a behavior definition, the referenced root entity gains a transactional character. In the behavior definition, you define the transactional capabilities of your business object. It is directly related to the root custom entity and must therefore have the same name `/DMO/R_TRAVEL_SCM_TP`.

1. In the **Project Explorer**, select the relevant node for the data definition that contains the CDS root entity `/DMO/R_TRAVEL_SCM_TP`, for which you want to create a behavior definition.
2. Open the context menu and select **New Behavior Definition** to launch the creation wizard.
3. Follow the steps of the creation wizard to create the behavior definition

i **Note**

The implementation type of a custom entity can only be unmanaged. That is why, the wizard only allows this option.

Result

The created behavior definition object represents the root node of a new business object in the ABAP RESTful Application Programming Model.

In the Project Explorer, the new behavior definition is added to the Core Data Services folder.

Defining the Behavior for the Travel Business Object with Discount Data

For the service consumption scenario, only the unmanaged implementation type is supported. That means, you as an application developer must implement the essential components of the business object yourself.

→ **Remember**

Our scenario is limited to modifying the data that is stored on the database table `/DMO/TRVL_SCM_DC`.

For a detailed description on the unmanaged implementation type, see [CDS BDL - implementation type \(ABAP Keyword Documentation\)](#).

Modifying the data that are exposed by the remote service is out of scope in this scenario. Nevertheless, each manipulation of the persistent data requires the retrieval of the remote data, as the discount data can only be stored in conjunction with the travel ID to match the discount to the relevant travel entry of the remote service. That is why, the creation of new discounts for a travel entry is considered as an update on a certain travel instance of the business object. In the case of creating a data set of discount data, you change the discount from the initial value but the key entry (`TRAVEL_ID`) is already there.

Compare the UI when retrieving the travel data merged with the discount data. The discount field is displayed with initial values if no discount data is maintained for the relevant travel instances.

Travel ID	Agency ID	Customer ID	Starting Date	End Date	Comment	Total Price		Total Price with Discount		Discount %	Status	
1	70041	594	Jun 3, 2023	Mar 31, 2024	Vacation	1,889.00	USD	1,851.22	USD	2.0	P	>
2	70007	608	Jun 3, 2023	Jun 3, 2023	Business Trip for Christine, Pierre	900.00	USD	630.00	USD	30.0	N	>
3	70046	93	Jun 3, 2023	Mar 31, 2024	Vacation	4,164.00	USD	4,164.00	USD	0.0	P	>
4	70042	665	Jun 3, 2023	Mar 31, 2024	Vacation	1,871.00	USD	1,871.00	USD	0.0	P	>
5	70007	225	Jun 3, 2023	Jun 3, 2023	Business Trip for Kurt, Ida	992.00	USD	992.00	USD	0.0	N	>

The modification of discount data is an **UPDATE**. Hence, we do not need to implement neither the **CREATE** nor the **DELETE** operation to maintain discount data.

A distinct field must be indicated as [ETag](#) in the behavior definition. The eTag checks that the data the end user sees that on the UI is consistent with the data on the database and has not been changed in the meantime. The field `calculatedetag` is used for this purpose and calculated with data from the database table `/DMO/TRVL_SCM_DC`.

The newly created behavior definition requires

- defining an update (delete the definitions of `create` and `delete` if they are generated)
- defining the field for the `etag master`

The following code block displays the behavior definition with the definitions that are relevant for the transactional behavior of the business object.

```

unmanaged implementation in class /dmo/bp_r_travel_scm_tp unique;
strict ( 2 );

define behavior for /DMO/R_TRAVEL_SCM_TP alias Travel_SCM

lock master
authorization master ( instance )
etag master calculatedetag
{
    update;
}

```

i Note

For better usability in the behavior implementation, you can define an alias for the business object in the behavior definition.

Related Information

[Adding Behavior to the Business Object](#)

Implementing the Behavior Pool for the Business Object

The implementation of behavior is done in the local types of an ABAP class.

Creating a Behavior Pool /dmo/bp_r_travel_scm_tp

A behavior pool for a behavior implementation is needed to implement the behavior that was defined in a behavior definition. A behavior pool is a special ABAP class that is equipped with the extension `FOR BEHAVIOR OF` and references the root entity of the relevant business object.

1. Use the quick fix offered in the line `unmanaged implementation in class /dmo/bp_r_travel_scm_tp unique;` of the behavior definition to launch the creation wizard.
2. Follow the steps of the creation wizard and check if the suggested entries are correct.

Result

The generated global class pool provides you with the addition `FOR BEHAVIOR OF` with reference to the corresponding entity.

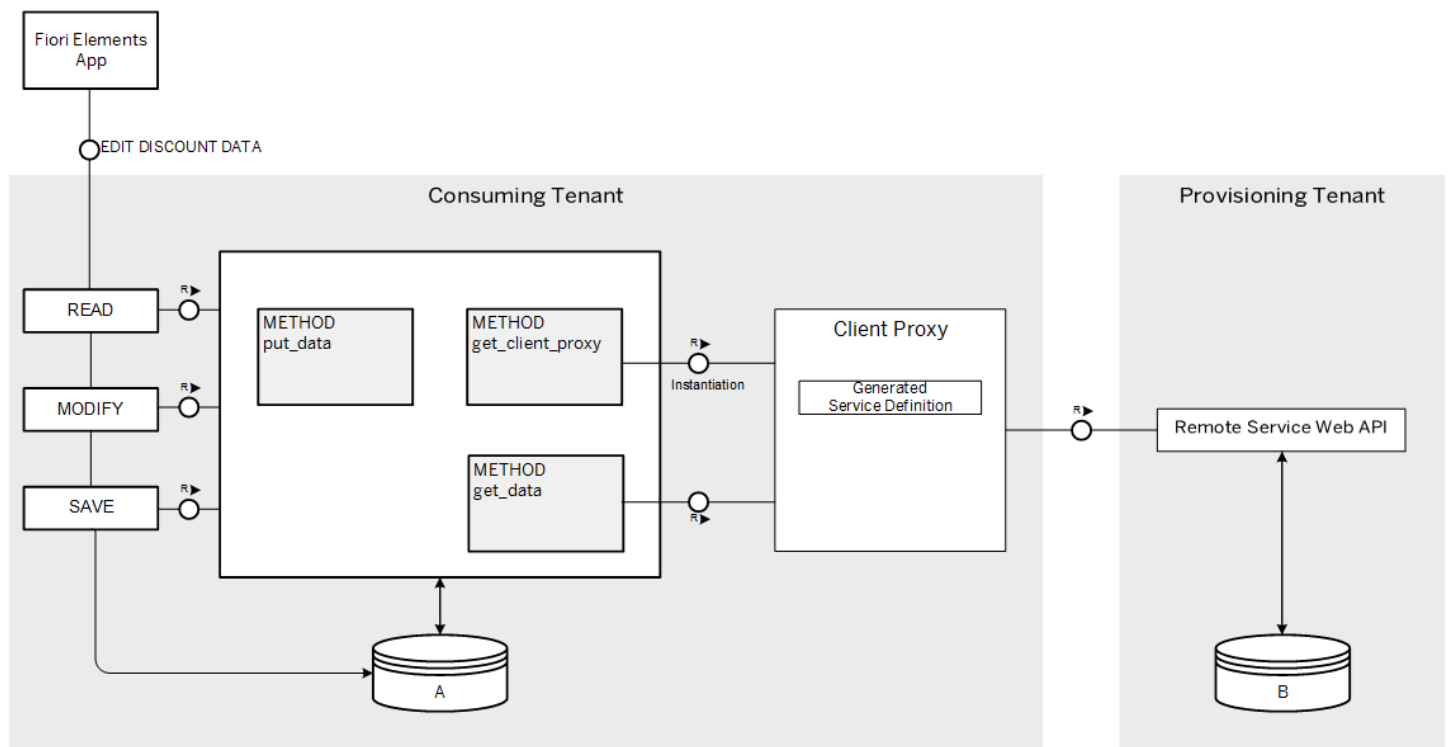
In the local types, a handler class with an [UPDATE method for modify](#) is generated. The generated saver class contains the methods `FINALIZE`, `CHECK_BEFORE_SAVE`, and `SAVE`, the predefined standard methods for the save sequence.

Runtime Logic for Transactional Behavior in the Service Consumption Scenario

In a service consumption scenario, the transactional behavior for the business object (the updating of discount data) must be implemented in the local types of a behavior pool. The implementation includes the data retrieval from the remote service, merging the remote data with the local discount data and finally updating the local discount data. Hence, the logic of how to access a foreign system also comes into play when updating local data. To keep performance expenses on a low level, it is indispensable to work with a transactional buffer and retrieve the remote data only once in one unit of work. Therefore we use a local buffer class to work actively with a transactional buffer.

The runtime of an update operation of the discount data is displayed in the following diagram. It illustrates how the methods for `READ`, `MODIFY`, and `SAVE` and the local buffer class interact with each other and how the OData client proxy is used to request the

necessary data from the remote service.



Transactional Runtime of the Travel BO

BUFFER

Due to performance reasons, it is useful to work with the transactional buffer in the service consumption scenario. Retrieving data from the remote service is very costly. Therefore, the data from the remote service should only be read once in one logical unit of work. We use the buffer as intermediate storage for data retrieval and processing.

The **READ**, **MODIFY**, and **SAVE** methods call dedicated methods from the buffer to process data. These buffer methods are:

- **get_data**

To receive data the Client Proxy is used. To instantiate the client proxy object, the method **get_client_proxy** is called. With this request object, you can define the resource object of the entity set and define the HTTP operation. After the execution, the data can be read from the response object returned by the execution.

In case of an error, the **FAILED** table is filled and can be used to raise an application error.

- **get_client_proxy**

This method establishes the connection to the provisioning tenant via an HTTP client and instantiates the client proxy. The connection is used at runtime for retrieving data from the remote service.

- **put_data**

This method calls **get_data** to retrieve the remote data. Additionally, it updates the local discount data for the retrieved entity instance and calculates the total price with discount. Finally, the method merges the remote and local data.

If there is an error, the **FAILED** table is filled and can be used to raise an application error.

A detailed step-by-step description for the implementation of the local buffer class is given in [Implementing the Buffer Class](#).

read_travel FOR READ

For the ETag check (which is necessary for the **UPDATE** operation), a method for the **READ** operation must be implemented. The latest timestamp of the field **lastchangedat** from **/DMO/TRVL_SCM_DC** is used to calculate the field for the ETag check. In our

example scenario, only the timestamp from the database table /DMO/TRVL_SCM_DC is relevant as only the field contained in this table can be updated.

i Note

The example implementation in this guide does not only retrieve the data from the relevant ETag field, but reads the whole data set when the READ operation is executed, including the merge with the persistent data. This is done for performance reasons.

To calculate the total price with discount, the value of the field `TotalPrice` from the remote instance is needed. Therefore it is more convenient to read the whole data set once, store it in the buffer and use this data set for the calculation on the update.

A detailed step-by-step description for the implementation of the READ is given in [Implementing the READ](#).

For general information about the READ method, see [<method> FOR READ](#).

update FOR MODIFY

The real update takes place during the MODIFY operation. The method `update` calls the buffer method `put_data`, which uses the method `get_data` to retrieve the data from the buffer (if available). The discount data is updated and the entry for the transient field `totalpricewithdiscount` is calculated.

The method `update` also calls the method `_map_messages`, which wraps the messages to write them in the REPORTED table.

A detailed step-by-step description for the implementation of the MODIFY is given in [Implementing the MODIFY](#).

For general information about the MODIFY method, see [<method> FOR MODIFY](#).

SAVE

The save method writes the updated data to the database. It uses the method `get_data` to retrieve the updated data from the buffer. It generates a new time stamp to fill the field `lastchangedat` on the database table /DMO/TRVL_SCM_DC. At last, it persists the updated data on the database table.

A detailed step-by-step description is given in [Implementing the SAVE](#).

For general information about the SAVE method, see [SAVE](#).

Implementation Steps

In this service consumption scenario, the READ and the MODIFY method, including a `_map_messages` method are treated in one handler class, as the retrieved data from the READ is reused in the MODIFY method. This relies to the [Best Practices for Modularization and Performance](#).

This guide starts with the implementation of the buffer and consequently describes calls of the buffer methods from the predefined methods READ, MODIFY, and SAVE.

- [Implementing the Buffer Class](#)
- [Implementing the READ](#)
- [Implementing the MODIFY](#)
- [Implementing the SAVE](#)

Implementing the Buffer Class

The local buffer class serves as intermediate storage for the data processing in the service consumption scenario. It is a separate local class in the behavior pool.

Creating a Buffer Class

The buffer must be accessed from the predefined methods READ, MODIFY, and SAVE.

1. Create a local buffer class `lcl_buffer` in the local types of the behavior implementation pool.
2. Set the local class to PRIVATE.

Listing: Local Buffer Class

```
CLASS lcl_buffer DEFINITION CREATE PRIVATE.

ENDCLASS.

CLASS lcl_buffer IMPLEMENTATION.
ENDCLASS.
```

Creating a Buffer Instance

When working with a transactional buffer to store and process data, an instance of the buffer must be created at first. The singleton pattern is used to ensure that there is only one existing buffer instance of the application buffer.

1. Declare the static method `get_instance` which returns a buffer instance in the public section of the buffer class
2. Implement `get_instance` to receive a buffer instance if no instance exists

Listing: Declaration and Implementation of `get_instance`

```
CLASS lcl_buffer DEFINITION CREATE PRIVATE.
  PUBLIC SECTION.
    CLASS-METHODS get_instance
      RETURNING VALUE(ro_instance) TYPE REF TO lcl_buffer.

  PRIVATE SECTION.
    CLASS-DATA: go_instance TYPE REF TO lcl_buffer.
ENDCLASS.

CLASS lcl_buffer IMPLEMENTATION.
  METHOD get_instance.
    IF go_instance IS NOT BOUND.
      go_instance = NEW #( ).
    ENDIF.
    ro_instance = go_instance.
  ENDMETHOD.
ENDCLASS.
```

Retrieving Data from the Remote Service

The following listing represents the signature and implementation of the method `get_data` that retrieves data from the remote service, if it is not already in the buffer. The implementation of the method includes

1. checking whether the travel instance for which discount data is to be updated is already in the buffer and writing it to the exporting parameter `et_travel`, if available in the buffer. If not in the buffer, the `travel_id` has to be collected to

retrieve the relevant data from the persistence.

2. the instantiation of a client proxy, by calling the method `get_client_proxy` of the auxiliary class `/DMO/CL_TRAVEL_CP_AUX`, see [Creating a Remote Client Proxy](#).
 3. a reading request for the client proxy with a filter for only requesting the data set for the specific travel IDs
 4. a select to get and calculate the matching discount data from the database table `/DMO/TRVL_SCM_DC` and to process the time stamp for the `calculatedetag` field.
 5. calculating the `TotalPriceWithDiscount` and setting an initial time stamp if no discount data has changed.
 6. filling the `FAILED` table if the travel id is not found.
- and
7. error handling if the access to the remote system fails.

i Note

If you want to display a suitable message, you need to create a message class. For example, here, the message `Accesss to remote system cannot be established` is convenient. It is retrieved from the message class `/DMO/CM_SERV_CONS`.

Listing: Declaration and Implementation of `get_data` *Expand the following code sample to view the source code of Declaration and Implementation of `lcl_buffer` (method `get_data`).*

Sample Code

```

CLASS lcl_buffer DEFINITION CREATE PRIVATE.
  PUBLIC SECTION.
    CLASS-METHODS get_instance
      RETURNING VALUE(ro_instance) TYPE REF TO lcl_buffer.

    "types used in get_data
    TYPES: BEGIN OF ts_message,
      travelid TYPE /DMO/R_TRAVEL_SCM_TP-travel_id,
      symsg     TYPE symsg,
      fields    TYPE string_table,
    END OF ts_message,
    tt_travel   TYPE STANDARD TABLE OF /DMO/R_TRAVEL_SCM_TP,
    tt_travel_in TYPE TABLE FOR READ IMPORT /DMO/R_TRAVEL_SCM_TP,
    tt_travel_out TYPE TABLE FOR READ RESULT /DMO/R_TRAVEL_SCM_TP,
    tt_travel_failed TYPE TABLE FOR FAILED /DMO/R_TRAVEL_SCM_TP,
    tt_message   TYPE STANDARD TABLE OF ts_message.

    METHODS get_data
      IMPORTING it_travel           TYPE tt_travel_in OPTIONAL
      EXPORTING et_travel           TYPE tt_travel_out
                et_travel_failed TYPE tt_travel_failed
                et_message        TYPE tt_message.

  PRIVATE SECTION.
    CLASS-DATA: go_instance TYPE REF TO lcl_buffer.
    DATA: mt_travel TYPE tt_travel.

ENDCLASS.

```

CLASS lcl_buffer IMPLEMENTATION.

METHOD get_data.

```

DATA: lt_travel      TYPE STANDARD TABLE OF /DMO/CL_TRAVEL_SCM=>tys_travel_type.
DATA: ls_result      LIKE LINE OF et_travel.
DATA: lt_travel_id   TYPE STANDARD TABLE OF /DMO/R_TRAVEL_SCM_TP-travel_id.

```

```

DATA: lt_filter      TYPE RANGE OF /DMO/R_TRAVEL_SCM_TP-travel_id.
DATA: ls_filter      LIKE LINE OF lt_filter.
DATA: lt_travel_ce   TYPE STANDARD TABLE OF /DMO/R_TRAVEL_SCM_TP.
DATA: lt_traveladd   TYPE STANDARD TABLE OF /DMO/TRVL_SCM_DC.
FIELD-SYMBOLS: <fs_travel_ce> LIKE LINE OF lt_travel_ce.

```

```

IF it_travel IS SUPPLIED.

```

```

  LOOP AT it_travel ASSIGNING FIELD-SYMBOL(<fs_travel>).
    IF line_exists( mt_travel[ travel_id = <fs_travel>-travel_id ] ).
      ls_result = CORRESPONDING #( mt_travel[ travel_id = <fs_travel>-travel_id ] ).
      " collect from buffer for result
      APPEND ls_result TO et_travel.
    ELSE.
      " collect to retrieve from persistence
      APPEND <fs_travel>-travel_id TO lt_travel_id.
    ENDIF.
  ENDLLOOP.

```

```

IF lt_travel_id IS NOT INITIAL.

```

```

  TRY.
    DATA(lo_client_proxy) = /dmo/cl_travel_cp_aux=>get_client_proxy( ).
    DATA(lo_request) = lo_client_proxy->create_resource_for_entity_set( 'TRAVEL' )->crea

    lt_filter = VALUE #( FOR travel_id IN lt_travel_id ( sign = 'I' option = 'EQ' low =
    DATA(lo_filter) = lo_request->create_filter_factory( )->create_by_range( iv_property,
                                                                it_range

    lo_request->set_filter( lo_filter ).
    DATA(lo_response) = lo_request->execute( ).
    " get relevant data sets
    lo_response->get_business_data( IMPORTING et_business_data = lt_travel ).

    " add local data
    IF lt_travel IS NOT INITIAL.
      " map OData service to custom entity
      lt_travel_ce = CORRESPONDING #( lt_travel MAPPING description = memo ).
      SELECT * FROM /DMO/TRVL_SCM_DC FOR ALL ENTRIES IN @lt_travel_ce WHERE travel_id =

    LOOP AT lt_travel_id ASSIGNING FIELD-SYMBOL(<fs_travel_id>).
      IF line_exists( lt_travel_ce[ travel_id = <fs_travel_id> ] ).
        ASSIGN lt_travel_ce[ travel_id = <fs_travel_id> ] TO <fs_travel_ce>.

        IF line_exists( lt_traveladd[ travel_id = <fs_travel_ce>-travel_id ] ).
          <fs_travel_ce>-discountpct      = lt_traveladd[ travel_id = <fs_travel_ce>
          <fs_travel_ce>-totalpricewithdiscount = <fs_travel_ce>-total_price * ( 1 - <
          <fs_travel_ce>-last_changed_at    = lt_traveladd[ travel_id = <fs_travel_c

```

```

        <fs_travel_ce>-calculatedettag          = <fs_travel_ce>-calculatedettag && '- '
    ELSE.
        <fs_travel_ce>-totalpricewithdiscount = <fs_travel_ce>-total_price.
        <fs_travel_ce>-last_changed_at = '20000101120000' . "initial value Jan 1, 2
    ENDIF.

    ls_result = CORRESPONDING #( <fs_travel_ce> ).
    APPEND <fs_travel_ce> TO mt_travel.
    APPEND ls_result      TO et_travel.
ELSE.
    APPEND VALUE #( travel_id = <fs_travel_id> ) TO et_travel_failed.
    APPEND VALUE #( travelid   = <fs_travel_id>
                    symsg-msgty = 'E'
                    symsg-msgid = '/DMO/CM_SERV_CONS'
                    symsg-msgno = '004'
                    symsg-msgv1 = <fs_travel_id> )
        TO et_message.
    ENDIF.
ENDLOOP.
ENDIF.

CATCH /iwbsp/cx_gateway /dmo/cx_travel_serv_cons.
    et_travel_failed = CORRESPONDING #( lt_travel_id MAPPING travel_id = table_line ).
    et_message = CORRESPONDING #( lt_travel_id MAPPING travelid = table_line ).
    LOOP AT et_message ASSIGNING FIELD-SYMBOL(<fs_message>).
        <fs_message>-symsg-msgty = 'E'.
        <fs_message>-symsg-msgid = '/DMO/CM_SERV_CONS'.
        <fs_message>-symsg-msgno = '001'.
    ENDLOOP.
ENDTRY.
ENDIF.
ELSE.
    et_travel = CORRESPONDING #( mt_travel ).

ENDIF.

ENDMETHOD.

METHOD get_instance.
    IF go_instance IS NOT BOUND.
        go_instance = NEW #( ).
    ENDIF.
    ro_instance = go_instance.
ENDMETHOD.

ENDCLASS.

```

Updating the Discount Data

With the preceding method, the data is retrieved from the remote service into the buffer. The essence of the **UPDATE** operation, however, is done with the method `put_data`. It uses the data in the buffer, which is already enriched with the additional discount data from the database table `/DMO/TRVL_SCM_DC`, and writes the new discount data in the buffer. This is done by

1. calling the method `get_data` to retrieve the data from the buffer
2. looping at `it_travel_upd` to identify the changed fields
3. overwriting the changed fields with the new discount data
4. calculating the data for `TotalPriceWithDiscount`
5. filling the `FAILED` table in case the total price with discount is negative after discount calculation.

i Note

If you want to display a suitable message, you need to add a message to the message class `/DMO/CM_SERV_CONS`. In this case, an exception needs to be thrown if the `Total Price with Discount` is negative. For example, here, the message `Total Price with Discount must be greater than 0` is convenient.

6. writing the updated data set to the buffer.

Listing: Declaration and Implementation of `put_data` *Expand the following code sample to view the source code of Declaration and Implementation of `lcl_buffer` (method `put_data`).*

Sample Code

```

CLASS lcl_buffer DEFINITION CREATE PRIVATE.
  PUBLIC SECTION.

  "types used in put_data
  TYPES:
    tt_travel_upd    TYPE TABLE FOR UPDATE /DMO/R_TRAVEL_SCM_TP,
    tt_travel_mapped TYPE TABLE FOR MAPPED /DMO/R_TRAVEL_SCM_TP.

  METHODS put_data
    IMPORTING it_travel_upd    TYPE tt_travel_upd
    EXPORTING et_travel_failed TYPE tt_travel_failed
              et_message       TYPE tt_message.

  ...
ENDCLASS.

CLASS lcl_buffer IMPLEMENTATION.

  ...

  METHOD put_data.

    get_data(
      EXPORTING it_travel      = CORRESPONDING #( it_travel_upd mapping %key = %key except * )
      IMPORTING et_travel      = data(lt_travel)
                et_travel_failed = DATA(lt_travel_failed)
                et_message       = DATA(lt_message)
    ).

    LOOP AT it_travel_upd ASSIGNING FIELD-SYMBOL(<fs_travel_upd>).
      CHECK line_exists( lt_travel[ KEY entity COMPONENTS travel_id = <fs_travel_upd>-travel_id

```

```

ASSIGN lt_travel[ KEY entity COMPONENTS travel_id = <fs_travel_upd>-travel_id ] TO FIELD-S

IF <fs_travel_upd>-%control-DiscountPct = if_abap_behv=>mk-on.
  <fs_travel>-DiscountPct = <fs_travel_upd>-DiscountPct.
ENDIF.
ENDLOOP.

" Postprocessing
LOOP AT lt_travel ASSIGNING FIELD-SYMBOL(<fs_traveladdinfo>).
  DATA(lv_totalpricediscount) = <fs_traveladdinfo>-total_price * ( 1 - <fs_traveladdinfo>-di
  IF lv_totalpricediscount >= 0.
    <fs_traveladdinfo>-totalpricediscount = lv_totalpricediscount.
  ELSE.
    APPEND VALUE #( travel_id      = <fs_traveladdinfo>-travel_id ) TO et_travel_failed.
    APPEND VALUE #( travelid      = <fs_traveladdinfo>-travel_id
                    symsg-msgty   = 'E'
                    symsg-msgid   = '/DMO/CM_SERV_CONS'
                    symsg-msgno   = '009'
                    symsg-msgv1   = <fs_traveladdinfo>-travel_id
                    symsg-msgv2   = |{ lv_totalpricediscount NUMBER = USER }|
                    fields        = VALUE #( ( |discountpct| )
                                             )
                    )
    TO et_message.
  ENDIF.
ENDLOOP.
"save data in buffer
mt_travel = CORRESPONDING #( lt_travel ) .

ENDMETHOD.
ENDCLASS.

```

Implementing the READ

The READ is necessary for ETag check.

The READ method processes reading requests. Hence, it is necessary to retrieve the data from the field `lastchangedat`.

This implementation does not only retrieve the time stamp, which is used for the eTag check, but the whole data set to store it in the buffer and use it in the actual UPDATE phase without retrieving the data again. The implementation must ensure that this data is retrieved and handed over to the framework, which handles the comparison with the time stamp of the UI. It is also the framework that prevents the update if the time stamps do not match.

See [<method> FOR READ](#) for information about parameters of the READ.

Declaring a method for READ

A code template for the READ method is generated automatically if an update is declared in the behavior definition.

1. Rename the method for read in the definition part of the handler class with the importing parameter `it_travel_read` and the result parameter `et_traveladdinfo`, which is used for the ETag check. The implementation.

```

CLASS lhc_Travel_SCM DEFINITION INHERITING FROM cl_abap_behavior_handler.
  PRIVATE SECTION.

    METHODS read FOR READ
      IMPORTING it_travel_read FOR READ Travel_SCM
      RESULT et_traveladdinfo.

    ...
ENDCLASS.

```

Implementing the method for READ

The actual READ is done by the method `get_data` in the buffer class. It either retrieves the data from the remote system into the buffer or reads the data from the buffer. The data is provided with the result parameter `et_traveladdinfo`. The method `read` instantiates the buffer and calls the method `get_data`.

1. Instantiate the buffer with the method `get_instance`.
2. Call the `get_data` method of the buffer class to get the data into `et_traveladdinfo`.

If the READ does not return the requested instance, the key of the instance to be read must be written in the `failed` table.

→ Remember

The `failed` table is an implicit changing parameter of the READ method.

```

METHOD read.
  DATA(lo_buffer) = lcl_buffer=>get_instance( ).

  lo_buffer->get_data(
    exporting
      it_travel          = it_travel_read
    importing
      et_travel          = et_traveladdinfo
      et_travel_failed   = failed-travel_scm
  ).

ENDMETHOD.

```

Implementing the MODIFY

The **MODIFY** updates data.

The **MODIFY** method is called by the framework when an **UPDATE** is defined in the behavior definition. It modifies the data in the application buffer. In this consumption scenario, the buffer method `put_data` is called to do this. To return adequate messages, the **MODIFY** method also includes a mapping method for messages.

Declaring a method for MODIFY

The declaration of the **MODIFY** method is generated by the template for the behavior implementation as the **UPDATE** was declared in the behavior definition.

1. Change the name of the importing parameter into `it_travel_update`.

```

CLASS lhc_Travel_SCM DEFINITION INHERITING FROM cl_abap_behavior_handler.
  PRIVATE SECTION.

  ...
  METHODS update FOR MODIFY
    IMPORTING it_travel_update FOR UPDATE Travel_SCM.

  ...
ENDCLASS.

```

Implementing the method for **MODIFY**

The real update is done by the method `put_data` in the buffer class. This method firstly calls the method `get_data` to retrieve the data from the buffer or get it from the remote service. Then it updates the discount data for the selected travel instance and calculates the amount for `totalpricewithdiscount`.

For the processing of the appropriate messages if there is a fault event, the method `_map_messages` is called within the method `update`.

1. Instantiate the buffer with the method `get_instance`.
2. Call the `put_data` method of the buffer class to update the discount data for the selected travel ID.

If the `UPDATE` does not work, the key of the failed travel instance must be written in the `failed` table.

→ Remember

The `failed` table is an implicit changing parameter of the method for **MODIFY**.

Write the messages of `put_data` into a local table `lt_message`.

3. Call the `_map_messages` method to map `lt_message` to the `reported` table.

→ Remember

The `reported` table is an implicit changing parameter of the method for **MODIFY**.

```

METHOD update.
  DATA(lo_buffer) = lcl_buffer=>get_instance( ).

  lo_buffer->put_data(
    EXPORTING
      it_travel_upd      = it_travel_update
    IMPORTING
      et_travel_failed   = failed-travel_scm
      et_message         = DATA(lt_message)
  ).

  _map_messages(
    EXPORTING
      it_message         = lt_message
    IMPORTING
      et_travel_reported = reported-travel_scm
  ).
ENDMETHOD.

```

Message Handling

In case of failure, the issue has to be transferred to the **REPORTED** table, which includes all instance-specific messages. For the processing of such messages, the method `_map_messages` is used. In the consumption scenario, it is defined in the local handler class, as it is only used in the **MODIFY** method.

Declare _map_messages

1. Declare the method `_map_messages` with an importing parameter `it_message` that is compatible to the type the method `put_data` returns for messages; and an exporting parameter `et_travel_reported` that is compatible to the reported table of the travel entity.

```

CLASS lhc_Travel_SCM DEFINITION INHERITING FROM cl_abap_behavior_handler.
  PRIVATE SECTION.
    ...
    TYPES:          tt_travel_reported    TYPE TABLE FOR REPORTED /DMO/R_TRAVEL_SCM_TP.

    METHODS: _map_messages
              IMPORTING it_message          TYPE lcl_buffer=>tt_message
              EXPORTING et_travel_reported TYPE tt_travel_reported.
    ...
ENDCLASS.

```

Implement _map_messages

To write messages to the REPORTED table, they have to align with a fixed structure. Messages for this structure are created via the method `new_message`, which is inherited from the interface `IF_ABAP_BEHV_MESSAGE`.

1. For each message in `it_message`, create a new message for the REPORTED table by calling the method `new_message`.
2. Fill the relevant fields in `ls_travel_reported` with the travel ID of the message and change the flag for the element which causes the message.

```

METHOD _map_messages.

  DATA: ls_travel_reported LIKE LINE OF et_travel_reported.
  FIELD-SYMBOLS: <fs_element> TYPE data.

  LOOP AT it_message ASSIGNING FIELD-SYMBOL(<fs_message>).
    CLEAR ls_travel_reported.
    ls_travel_reported-%msg = new_message( id          = <fs_message>-symsg-msgid
                                           number       = <fs_message>-symsg-msgno
                                           severity    = if_abap_behv_message=>severity-error
                                           v1          = <fs_message>-symsg-msgv1
                                           v2          = <fs_message>-symsg-msgv2
                                           v3          = <fs_message>-symsg-msgv3
                                           v4          = <fs_message>-symsg-msgv4 ).

    IF <fs_message>-travelid IS NOT INITIAL.
      ls_travel_reported-%key-travel_id = <fs_message>-travelid.
      ls_travel_reported-travel_id     = <fs_message>-travelid.
      LOOP AT <fs_message>-fields ASSIGNING FIELD-SYMBOL(<fs_field>).
        ASSIGN COMPONENT <fs_field> OF STRUCTURE ls_travel_reported-%element TO <fs_element>
        CHECK sy-subrc = 0.
        <fs_element> = if_abap_behv=>mk-on.
      ENDLOOP.
      APPEND ls_travel_reported TO et_travel_reported.
    ENDIF.
  ENDLOOP.
ENDMETHOD.

```

Implementing the SAVE

The template for the behavior implementation provides you with the method declarations for redefinitions of the save sequence.

```

CLASS lsc_R_TRAVEL_SCM_TP DEFINITION INHERITING FROM cl_abap_behavior_saver.
  PROTECTED SECTION.

```

```

  METHODS finalize REDEFINITION.

```

```
METHODS check_before_save REDEFINITION.
```

```
METHODS save REDEFINITION.
```

```
METHODS cleanup REDEFINITION.
```

```
METHODS cleanup_finalize REDEFINITION.
```

```
ENDCLASS.
```

i Note

In this consumption scenario, we only exemplify the implementation for the `save` method.

In the method `SAVE`, the transactional buffer is saved to the database. The following listing illustrates the procedure for the `SAVE` implementation. The saving action includes

1. instantiating of the buffer instance
2. calling the method `get_data` to get the updated data set
3. retrieving the latest time stamp
4. modifying the database table `/DMO/TRVL_SCM_DC`

```
CLASS lsc_R_TRAVEL_SCM_TP IMPLEMENTATION.
```

```
METHOD save.
```

```
DATA: ls_traveladd TYPE /DMO/TRVL_SCM_DC.
```

```
DATA(lo_buffer) = lcl_buffer=>get_instance( ).
```

```
lo_buffer->get_data(
  IMPORTING
    et_travel = DATA(lt_travel)
).
```

```
LOOP AT lt_travel ASSIGNING FIELD-SYMBOL(<fs_traveladdinfo>).
```

```
  ls_traveladd = CORRESPONDING #( <fs_traveladdinfo> MAPPING travel_id    = travel_id
                                discount_pct = discountpct ).
```

```
  GET TIME STAMP FIELD ls_traveladd-lastchangedat.
```

```
  MODIFY /DMO/TRVL_SCM_DC FROM @ls_traveladd.
```

```
ENDLOOP.
```

```
ENDMETHOD.
```

```
...
```

```
ENDCLASS.
```

Defining an OData Service

In order to be able to consume the service with an SAP Fiori UI, you need to create an OData service.

1. Follow the development steps as described in [Creating an OData Service](#).

Create a **service definition** and a **service binding** to expose the CDS custom entity including its query implementation for a service. You only need to include the custom entity in the service definition, its implementation is included implicitly.

You can then test the resulting UI service by using the **Fiori Elements Preview** in the service binding.

2. Follow the steps described in [Designing the User Interface for a Fiori Elements App](#)

Use @UI annotations to define the UI of the SAP Fiori app. UI annotations are maintained in the CDS custom entity in exactly the same way as in CDS views.

For this consumption scenario, it is convenient to label the element for line items and identification annotations as the data element information is not retrieved from the remote service.

i Note

For selection fields, it is not possible to maintain a label in the @UI annotations. Use the @EndUserText.label annotation instead for elements with selection fields.

i Expand the following listing to view the source code of the travel CDS view.

```
@ObjectModel.query.implementedBy: 'ABAP:/DM0/CL_R_TRAVELTP_SCM_QUERY'
@EndUserText.label: 'CE for Service Consumption Scenario'
```

```
define root custom entity /DM0/R_TRAVEL_SCM_TP
{
```

```
    @UI.facet
        { id
          purpose
          type
          label
          position
        : [
          : 'Travel',
          : #STANDARD,
          : #IDENTIFICATION_REFERENCE,
          : 'Travel',
          : 10 } ]
```

```
    @UI
        : {lineItem: [ { position: 10, label: 'Travel ID', importance: #
          selectionField: [ { position: 10 } ],
          identification: [{ position: 10 , label: 'Travel ID' }] }
    @EndUserText.label
        : 'Travel ID'
    key travel_id
        : abap.numc( 8 );
```

```
    @UI
        : {lineItem: [ { position: 20, label: 'Agency ID', importance:
          selectionField: [ { position: 20 } ],
          identification: [{ position: 20, label: 'Agency ID' }] }
    @EndUserText.label
        : 'Agency ID'
    agency_id
        : abap.numc( 6 );
```

```
    @UI
        : {lineItem: [ { position: 30, label: 'Customer ID', importance:
          selectionField: [ { position: 30 } ],
          identification:[ { position: 30 , label: 'Customer ID'} ] }
    @EndUserText.label
        : 'Customer ID'
    customer_id
        : abap.numc( 6 );
```

```

@UI          : {lineItem: [ { position: 40, label: 'Starting Date',  importan
                    identification:[ { position: 40, label: 'Starting Date' } ] }
begin_date   : abap.dats;

@UI          : {lineItem: [ { position: 45,   label: 'End Date', importance:
                    identification:[ { position: 45,   label: 'End Date' } ] }
end_date     : abap.dats;

@UI          : {identification:[ { position: 60, label: 'Booking Fee' } ] }
@Semantics.amount.currencyCode: 'currency_code'
booking_fee  : abap.dec( 17, 3 );

@UI          : {lineItem: [ { position: 70, label: 'Total Price',  importance
                    identification:[ { position: 70, label: 'Total Price' } ] }
@Semantics.amount.currencyCode: 'currency_code'
total_price  : abap.dec( 17, 3 );
currency_code : abap.cuky( 5 );

@UI          : {lineItem: [ { position: 50, label: 'Comment', importance: #ME
                    identification:[ { position: 50, label: 'Comment' } ] }
description  : abap.char( 1024 ); //renamed element

@UI          : {lineItem: [ { position: 90, label: 'Status', importance: #HIG
                    identification:[ { position: 90, label: 'Status' } ] }
status       : abap.char( 1 );

@EndUserText.label : 'Last changed on'
last_changed_at    : timestamp;

@UI          : {lineItem: [ { position: 80, label: 'Discount %', importance:
                    identification:[ { position: 80, label: 'Discount %' } ] }
discountpct      : abap.dec( 3, 1 );
@Semantics.amount.currencyCode: 'currency_code'
@UI              : {lineItem: [ { position: 72, label: 'Total Price with Discount
                    identification:[ { position: 72, label: 'Total Price with Discount' } ]
totalpricewithdiscount : abap.dec(17,3);
@UI.hidden: true
calculatedetag        : abap.sstring( 1024 );
}

```

3. Add static feature control for attributes to set them read-only or mandatory.

The only attribute that is modifiable is DISCOUNTPCT. It needs to be filled with values, hence set it as mandatory. Set all other attributes as readonly. For further information, see [Feature Control](#).

i

4. Define a RAP side effect that triggers a reload of the field totalpricewithdiscount by the UI when the field discountpct is changed.

Expand the following listing to view the source code of the behavior definition with the relevant feature control additions.


```

unmanaged implementation in class /dmo/bp_r_travel_scm_tp unique;
strict ( 2 );

define behavior for /DMO/R_TRAVEL_SCM_TP alias Travel_SCM

lock master
authorization master ( instance )
etag master calculatedetag
{
    update;

    field ( readonly ) TRAVEL_ID, AGENCY_ID, CUSTOMER_ID,
    BEGIN_DATE, END_DATE, DESCRIPTION,
    BOOKING_FEE, TOTAL_PRICE, TOTALPRICEWITHDISCOUNT,
    STATUS, CURRENCY_CODE, calculatedetag;

    field ( mandatory ) DISCOUNTPCT;

    side effects { field discountpct affects field totalpricewithdiscount; }

}

```

5. The defined RAP side effect adds the field `totalpricewithdiscount` to the list of requested elements in the query implementation class. Without the field `total_price` however, the value for the field `totalpricewithdiscount` cannot be calculated. Hence, add the field `total_price` to the list of requested elements before the deletion of the local and transient fields, if the field `totalpricewithdiscount` is requested and if the field `total_price` is not already part of the list.

```

...
" Request Elements
DATA(lt_req_elements) = io_request->get_requested_elements( ).

IF line_exists( lt_req_elements[ table_line = 'TOTALPRICEWITHDISCOUNT' ] ) AND NOT line_exist
    APPEND 'TOTAL_PRICE' TO lt_req_elements.
ENDIF.

"delete local fields out of the fields to be selected via OData Client Proxy
DELETE lt_req_elements WHERE table_line = 'DISCOUNTPCT' OR
    table_line = 'TOTALPRICEWITHDISCOUNT' OR
    table_line = 'CALCULATEDETAG'.
...

```

Related Information

[RAP Side Effects](#)

[Creating an OData Service](#)

[Designing the User Interface for a Fiori Elements App](#)

Develop Web APIs

An OData service can be published as a web API. This enables the consumption of the service by any client, independent from a particular UI.

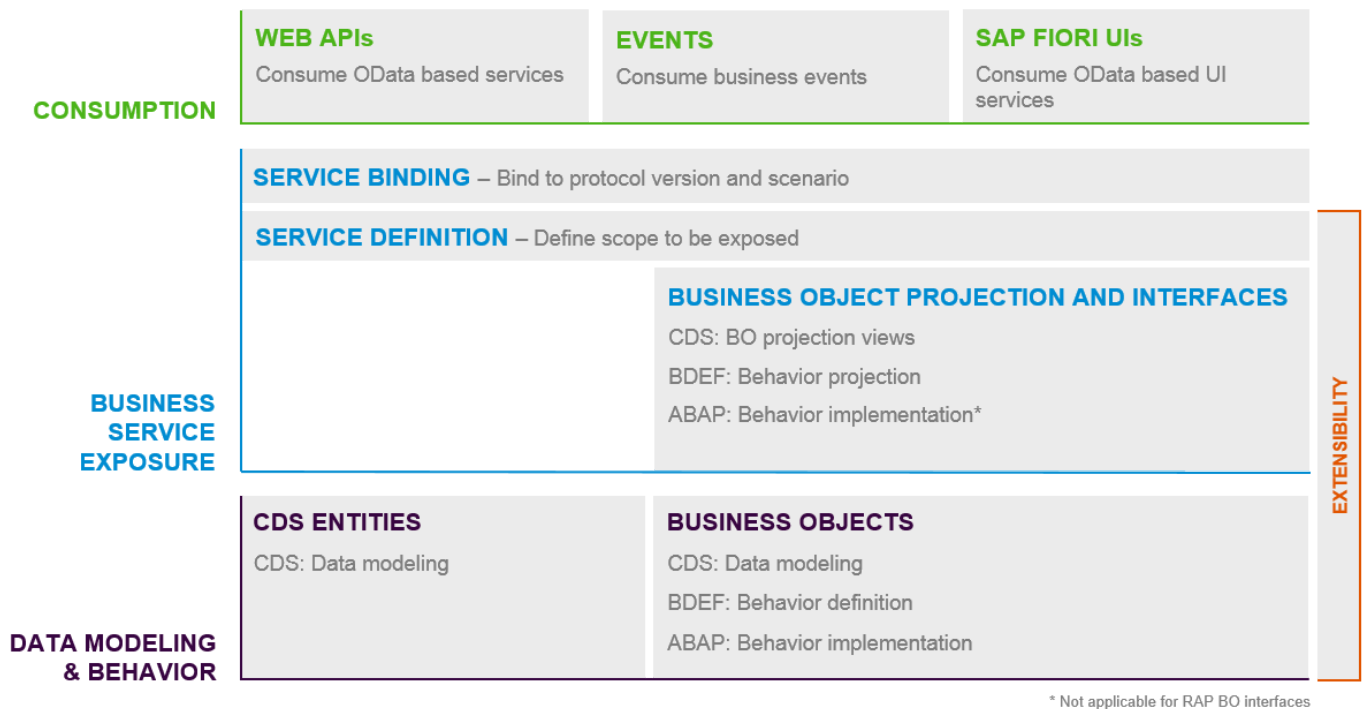
Introduction

A Web API is an OData service whose metadata does not entail any UI-specific annotations that are defined for the data model. It is published for the purpose of providing an API to access the service by an unspecified client. A Web API facilitates the exchange of business information between an application and any client, including from a different system or server.

In this development guide you will reuse the service that you created in [Developing Unmanaged Transactional Apps](#) and publish a Web API for it. Since a Web API is not being used directly in a UI context, the consumer of a service of this type only requires a reduced set of metadata. The metadata lacks any kind of UI-relevant information.

The basis for the service remains identical to an OData service exposed for a UI. It is just the binding type in the service binding that differs, which can be seen in the subsequent figure. It is even possible to expose a service that was created originally as a UI service for API consumption. The metadata are automatically reduced to the relevant information for Web API, which means without UI-related annotations or value helps.

RAP - The big picture



☁ The procedure of consuming a remote Web API service like this from a foreign system is described in the following development guide [Developing a UI Service with Access to a Remote Service](#).

Prerequisites

Developing the scenario that is described in the subsequent chapters requires the following:

- You fulfill the requirements described in [Prerequisites](#).
- You have installed ABAP Development Tools (ADT).

SAP recommends to use the latest version of the client installation. The ADT download is available on the update site <https://tools.hana.ondemand.com/>.

- To recreate the demo scenario, the **ABAP Flight Reference Scenario** must be available in your ABAP system. You can download the complete reference scenario from **GitHub**: [Downloading the ABAP Flight Reference Scenario](#).
- You have understood the development steps to create a transactional OData service for a UI as described in [Developing Unmanaged Transactional Apps](#).

In particular, you are able to reuse the data model including the behavior of the existing OData service `/DMO/TRAVEL_U` to expose it for a Web API.

Via ABAPGit You can import the service `/DMO/TRAVEL_U` including the related development objects into your development environment. So you do not have to build the service to test the publishing as Web API. You find the service in the package `/DMO/FLIGHT_UNMANAGED`.

For information about downloading the ABAP Flight Reference Scenario, see [Downloading the ABAP Flight Reference Scenario](#).

Example Scenario

As described above, the following guide reuses the data model and behavior of the service that was created in the transactional guide. This means that the following artifacts must be available in your system to follow the steps of this guide.

Artefact	Description
CDS Entities	
<code>/DMO/I_TRAVEL_U</code>	The <code>Travel</code> entity defines general travel data, such as the agency ID or customer ID, status of the travel booking, and the price of travel. The entity represents the root node of the business object
<code>/DMO/I_Booking_U</code>	The <code>Booking</code> entity manages data for a booked flight for a certain travel instance. It is a composition of the <code>Travel</code> entity and is therefore dependent on its root.
<code>/DMO/I_Agency</code>	The <code>Agency</code> defines general data about the responsible agency for a travel. It is associated with the <code>Travel</code> entity.
<code>/DMO/I_Customer</code>	The <code>Customer</code> defines personal data about the customers involved in travel and booking. It is associated with the <code>Travel</code> entity and the <code>Booking</code> entity.
Behavior Artifacts	
<code>/DMO/I_TRAVEL_U</code>	The behavior definition defines the capabilities of the business object involved.
<code>/DMO/CL_TRAVEL_U</code>	The behavior is implemented in the special ABAP classes.
Business Service Artifact	
<code>/DMO/TRAVEL_U</code>	The service definition defines all the entities that are exposed for the service.

Publishing a Web API

Prerequisites

You have an existing service definition for which you want to create a Web API service. In our example scenario we reuse the service definition `/DMO/TRAVEL_U`, which was already exposed as a UI service in the transactional scenario.

If no service definition is available, choose the entities that you want to expose as an API and create a service definition for these entities. For a description on how to create a service definition, refer to [Creating a Service Definition for the Read-Only Service](#).

Context

You have defined the scope of the service that you want to expose in a service definition. The service must now be bound to an OData protocol and published as a Web API.

Procedure

1. In your ABAP project, open the context menu for the existing service definition **/DMO/TRAVEL_U** and choose **New Service Binding** to launch the creation wizard.
2. In addition to the **Project** and **Package**, enter the **Name** and the **Description** for the service binding you want to create.
3. Select the **Binding Type** **ODATA V4 - Web API** to bind the service to a V4 protocol and expose it as a Web API.
4. Verify that the correct **Service Definition** is preset in the wizard to be used as a base for your service binding.
5. Choose **Next**.
6. Assign a transport request and choose **Finish**.

The ABAP back end creates a service binding and stores it in the ABAP Repository.

In the **Project Explorer**, the new service binding is added to the **Business Services** folder of the corresponding package node. As a result, the service binding form editor is opened and you can verify the information of the service.

7. Choose the button **Activate**.
8. To expose the service, click the button **Publish local service endpoint**.

The metadata document can be accessed by following the link **Service URL**. The UI preview is naturally not available.

Results

Except for UI features, the service including all implemented features is now exposed as an API. In particular, this means that the implemented behavior is also exposed in this API.

The difference to a service that is published for a UI client can best be seen in the service metadata. Whereas the metadata of a UI service carries information about the UI representation of the service, the Web API service does not contain any such information.

❖ Example

In our example scenario this difference can be seen when comparing the annotation section that refers to the implemented value help.

The UI service lists every field of a value help provider view as `ValueListParameter` in the annotation section. The Web API on the other hand lacks any UI specific information, although the value help annotation is defined in the respective CDS views.

Develop Business Events

In this chapter you learn how to define and raise business events in a RAP business object.

Use Case

You can use business events to notify BO consumers when the state of a business object has changed. The business events can then be consumed as described in chapter [Business Event Consumption](#).

For information on how to define business events, see [Business Events](#).

Related Information

[Business Events](#)

[Business Event Consumption](#)

[Creating Event Bindings](#)

Business Events

Example

The following example is based on the extensibility data model as described in [Data Modeling and Behavior](#). In this case, the event is raised in a behavior extension of the original Agency RAP BO.

You want to define an event that is raised every time a new review is created in order to inform the respective agency. Events have to be triggered once the point of no return has passed and the business process is finished, which is why the event is raised with an additional save in the late save phase. This ensures that the data will be saved to the database and that the state change of the BO is final. An event itself is defined with the keyword `event` in the behavior definition and can be defined with or without parameters. For this use case, a parameter is defined using an abstract entity that transmits an email address and the text to inform the agency via email. The raising of the event is implemented in the behavior pool within an additional save.

1. Create the abstract entity `/DMO/ZZ_D_AgencyReviewCreated` and define the required data types. The variable `EMailAddress` is defined as `strg(256)` and contains the email address of the respective Agency. The `ReviewID` contains the information about the review that triggered the event.

```
@EndUserText.label: 'Agency Review Infos for create event'
define abstract entity /DMO/ZZ_D_AgencyReviewCreated
{
  ReviewID      : /dmo/zz_review_id;
  Rating        : /dmo/zz_rating;
  FreeTextComment : /dmo/zz_free_text_comment;
  EMailAddress   : /dmo/email_address;
}
```

For details about modeling of parameters in RAP, see [Modeling Parameters for Non-Standard Operations](#).

2. Define the event in the behavior definition extension with the keyword `event`. The event in this example is named `/DMO/AgencyReviewCreated`. The fields of the abstract entity `/DMO/ZZ_D_AgencyReviewCreated` form the payload of the event.

```
extend behavior for /DMO/Agency
{
  event /DMO/AgencyReviewCreated parameter /DMO/ZZ_D_AgencyReviewCreated;
}
```

3. Define the additional save for the BO entity for which you want to raise the event. In this example, the event is raised for the extension node `/DMO/ZZ_R_Agency_ReviewTP`, and so with `additional` save is added to the header of the entity definition:

```
define behavior for /DMO/ZZ_R_AGENCY_REVIEWTP alias /DMO/ZZ_Review using /DMO/ZZ_I_AGENCY_REV
with additional save
persistent table /DMO/ZZ_AGN_REVA
draft table /DMO/ZZ_AGN_REVD
etag master LocalLastChangedAt
lock dependent
authorization dependent
```

```
late numbering
{
...
}
```

4. Implement the raising of the event in the `save_modified` method in the behavior pool.

This method checks whether a review was actually created by checking that `create-/dmo/zz_review` isn't empty. The `READ` returns the root node agency instance for which the review was created. The `emailaddress` information is the email address to which the email will be sent after the event has been triggered. Then the event is raised with `RAISE EVENT` on the RAP BO interface `/DMO/I_AgencyTP` that was extended with the event `/DMO/AgencyReviewCreated`. In the last step, the instance key and the parameter values for the event are filled.

For general information about the additional save, see [SAVE](#).

```
METHOD save_modified.
  IF create-/dmo/zz_review IS NOT INITIAL.
    READ ENTITIES OF /dmo/i_agencytp IN LOCAL MODE
      ENTITY /dmo/agency
        FIELDS ( emailaddress ) WITH CORRESPONDING #( create-/dmo/zz_review )
        RESULT DATA(agencies).

    RAISE ENTITY EVENT /dmo/i_agencytp~/dmo/agencyreviewcreated
    FROM VALUE #(
      FOR review IN create-/dmo/zz_review (
        agencyid      = review-agencyid
        reviewid      = review-reviewid
        rating        = review-rating
        freetextcomment = review-freetextcomment
        emailaddress   = agencies[ KEY entity agencyid = review-agencyid ]-emailaddre
      )
    ).
  ENDIF.
ENDMETHOD.
```

i Note

Before events can be consumed, they need to be exposed in a RAP BO interface using the syntax `use event`. Note that this is not needed in the example above, as the event is added via an extension. For the local consumption of exposed events, the RAP BO interface needs to be C1 released.

i Note

In case the event consumer is not in the same system as the event provider (remote consumption), you need to create an event binding and include the RAP BO interface with the exposed event in this event binding. The event binding represents the design time definition of the event and maps the event defined in a RAP BO to a namespace, a business object and a business object operation like `modify`.

Related Information

[Business Events](#)

[Business Event Consumption](#)

Derived Business Events

Context

Developing a RAP Business Event involves the definition of its payload and its implementation. In case you want to use an already implemented event with a different payload, you can make use of derived events. A derived event is defined with reference to an already implemented event and allows the definition of a payload that deviates from the one of the referenced event.

The payload is redefined with a CDS entity. To model a derived event, the input parameter must fulfill the following requirements:

- The input parameter must be a CDS view entity without any associations.
- The input parameter of a derived event must be flat.
- The key fields of the view entity used as the input parameter must match the key fields of the entity the referenced event is defined for.

When this referenced event is raised, the derived event is raised as well with its custom payload. Derived events can be defined in behavior definitions and in behavior definition extensions. Following this declarative approach, no additional implementation in the behavior pool is required.

For more information see [RAP Events \(ABAP Keyword Documentation\)](#)

Procedure

Developing a derived event comprises the definition of a CDS view for the event payload and the definition of the derived event itself in a behavior definition or in a behavior definition extension. The key fields of the payload for the derived event need to match the key fields of the entity the referenced event is defined for.

Prerequisites for derived events in extensions

If the derived event is defined in a behavior definition extension, the following prerequisites apply for the original business object:

- The business object containing the event that is referenced by the derived event needs to be extensibility-enabled as described in [RAP Extensibility-Enablement](#).
- The event referenced by the derived event needs to be exposed in the C0 and C1 released RAP BO interface. If the referenced event has been added in a behavior extension, it is implicitly exposed in the RAP BO interface.
- The CDS view entity which serves as the data source for the payload of the derived event needs to be C1 released.

Example

The basis for the following example is the extensibility data model as described in [Data Modeling and Behavior](#). In this example, a derived event with a custom payload is defined for the event `/DMO/AgencyReviewCreated` which is raised in the node extension of the `Agency` business object (package `/DMO/FLIGHT_REUSE_AGENCY_REV_D`).

1. Create a data definition for a CDS view entity called `/DMO/ZZ_P_AGENCYREVIEW_CONTACT` that selects from the CDS view entity `/DMO/I_Agency`. Add the key field `AgencyID` as well as the fields `Name` and `PhoneNumber` to the select list. The created CDS view entity will provide the payload for the derived event.

i Note

Keep in mind that this CDS view entity is meant for data selection. If no data is selected, no event is raised. This is important in case an event is raised after deletion of data that has the same source as the view.

```
@EndUserText.label: 'Agency Extension: Derived Event'
@AccessControl.authorizationCheck: #NOT_REQUIRED
define view entity /DMO/ZZ_P_AgencyReview_Contact
  as select from /DMO/I_Agency
```

```
{
  key AgencyID,
    Name,
    PhoneNumber
}
```

2. In the context menu of the behavior definition `/DMO/R_AGENCYTP`, choose the option **New Behavior Extension**.
3. In the BDEF extension creation dialogue, enter the name of the package where the BDEF extension is to be stored as well as the name `/DMO/ZZ_X_REV_CNTCT_R_AGENCYTP`, a description and the BO interface `/DMO/I_AGENCYTP`.
4. Remove the keywords `implementation in class...` from the generated behavior definition extension.

The resulting BDEF extension looks like this:

```
extension using interface /dmo/i_agencytp;
```

5. Add the keywords `extend behavior for /DMO/Agency { }`. The derived event can now be defined within the curly brackets.

```
extension using interface /dmo/i_agencytp;
```

```
extend behavior for /DMO/Agency
{

}
```

6. Define a derived event with the name `/DMO/AgencyReviewCreatedCntct` and with reference to the existing event `/DMO/AgencyReviewCreated`. Define the CDS view entity `/DMO/ZZ_P_AgencyReview_Contact`; as the payload for the derived event.

```
extension using interface /dmo/i_agencytp;
```

```
extend behavior for /DMO/Agency
{
  managed event /DMO/AgencyReviewCreatedCntct on /DMO/AgencyReviewCreated parameter /DMO/ZZ_P
}
```

Develop APIs

Context

This chapter guides you through the necessary steps to develop an API for an existing business object. This API decouples requirements for the stable consumption of your business object from its core data model and behavior. In RAP, an API is implemented by means of a RAP BO interface.

For further information on RAP BO interfaces, see [Business Object Interface](#).

Prerequisites

You can add a RAP BO interface to any existing business object. In this development guide, a RAP BO interface is developed for the business object resulting from the draft reference scenario. So in order to follow the steps described in this section, you should have completed the development guide [Draft Reference Scenario](#) beforehand.

To add an interface to the business object, the following steps are relevant:

- [Projecting the Draft BO Data Model](#)
- [Projecting the Draft BO Behavior](#)

Projecting the Draft BO Data Model

For the managed scenario with draft, create and implement CDS projection views for the draft BO entities.

Context and Procedure

Create data definitions for projection views for all three BO entities **Travel**, **Booking**, and **Booking Supplement**. In ADT, use the context menu on the underlying CDS view entities `/DMO/R_Travel_D`, `/DMO/R_Booking_D` and `/DMO/R_BookingSupplement_D` to open the wizard for creating CDS data definitions.

Select the template **Define Projection View**.

For more information about the syntax in projection views, see [Define View Entity as Projection \(ABAP Keyword Documentation\)](#).

Projection View `/DMO/I_Travel_D`

Use the annotation `@Metadata.ignorePropagatedAnnotations: true` in the header section. This prevents annotations from the underlying base data model from being propagated further through the CDS view stack, thus possibly impacting projections that are built on top of the RAP BO interface.

The projection view is a projection on `/DMO/R_Travel_D`. Include the keyword `root` in the projection view.

Use the provider contract `transactional_interface`. This defines the projection view as part of the RAP BO interface.

Use all elements of the underlying BO travel entity. You do not need to use the administrative data in the projection view, except for the element representing the ETag master.

As the projection view will not be used for OData service consumption, no UI specific annotations are required. The long texts for ID elements do not need to be denormalized either.

Define semantic annotations that establish the relation between the fields `BookingFee` and `TotalPrice` to their currency code field. In Addition, define the field `LocalLastChangedAt` as such. The defined semantic annotations will take effect on all projection views that are built on top of this interface view.

As soon as all projection views of the composition structure of the business object are available, redirect the to-child composition to the projection view of the booking entity.

i Expand the following code sample to view the source code of Projection View `/DMO/I_Travel_D`.

Sample Code

```
@Metadata.ignorePropagatedAnnotations: true
@EndUserText.label: 'Travel Interface Projection View'
@AccessControl.authorizationCheck: #NOT_REQUIRED
define root view entity /DMO/I_Travel_D
provider contract transactional_interface
as projection on /DMO/R_Travel_D
{
  key TravelUUID,
  TravelID,
  AgencyID,
```

```

    CustomerID,
    BeginDate,
    EndDate,
    @Semantics.amount.currencyCode: 'CurrencyCode'
    BookingFee,
    @Semantics.amount.currencyCode: 'CurrencyCode'
    TotalPrice,
    CurrencyCode,
    Description,
    OverallStatus,
    @Semantics.systemDateTime.localInstanceLastChangedAt: true
    LocalLastChangedAt,
    /* Associations */
    _Agency,
    _Booking: redirected to composition child /DMO/I_Booking_D,
    _Currency,
    _Customer,
    _OverallStatus
}

```

Projection View /DMO/I_Booking_D

Use the annotation `@Metadata.ignorePropagatedAnnotations: true` in the header section. This prevents annotations from the underlying base data model from being propagated further through the CDS view stack, thus possibly impacting projections that are built on top of the RAP BO interface.

The projection view is a projection on `/DMO/R_Booking_D`.

Use all elements of the underlying BO travel entity.

As the projection view will not be used for OData service consumption, no UI specific annotations are required. The long texts for ID elements do not need to be denormalized either.

Define a semantic annotation that establishes the relation between the field `FlightPrice` to its currency code field. In addition, define the field `LocalLastChangedAt` as such. The defined semantic annotations will take effect on all projection views that are built on top of this interface view.

As soon as all projection views of the composition structure of the business object are available, redirect the to-child composition to the projection view of the booking supplement entity and the association to the root entity to the respective projection view.

i *Expand the following code sample to view the source code of Projection View /DMO/I_Booking_D.*

Sample Code

```

@Metadata.ignorePropagatedAnnotations: true
@EndUserText.label: 'Booking Interface Projection View '
@AccessControl.authorizationCheck: #NOT_REQUIRED
define view entity /DMO/I_Booking_D
as projection on /DMO/R_Booking_D {
    key BookingUUID,
    TravelUUID,
    BookingID,
    BookingDate,
    CustomerID,
    AirlineID,
    ConnectionID,
    FlightDate,
    @Semantics.amount.currencyCode: 'CurrencyCode'
    FlightPrice,
    CurrencyCode,
    BookingStatus,
    @Semantics.systemDateTime.localInstanceLastChangedAt: true
    LocalLastChangedAt,

```

```

/* Associations */
_bookingStatus,
_bookingSupplement: redirected to composition child /DMO/I_BookingSupplement_D,
_carrier,
_connection,
_customer,
_travel: redirected to parent /DMO/I_Travel_D
}

```

Projection View /DMO/I_BookingSupplement_D

Use the annotation `@Metadata.ignorePropagatedAnnotations: true` in the header section. This prevents annotations from the underlying base data model from being propagated further through the CDS view stack, thus possibly impacting projections that are built on top of the RAP BO interface.

The projection view is a projection on `/DMO/R_BookingSupplement_D`.

Use all elements of the underlying BO travel entity.

As the projection view will not be used for OData service consumption, no UI specific annotations are required. The long texts for ID elements do not need to be denormalized either.

Define a semantic annotation that establishes the relation between the field `BookSupplPrice` to its currency code field. In addition, define the field `LocalLastChangedAt` as such. The defined semantic annotations will take effect on all projection views that are built on top of this interface view.

As soon as all projection views of the composition structure of the business object are available, redirect the to-parent association to the projection view of the booking entity and the association to the root entity to the respective projection view.

i *Expand the following code sample to view the source code of Projection View /DMO/I_BookingSupplement_D.*

Sample Code

```

@Metadata.ignorePropagatedAnnotations: true
@EndUserText.label: 'BookSupplement Interface Projection View'
@AccessControl.authorizationCheck: #NOT_REQUIRED
define view entity /DMO/I_BookingSupplement_D
as projection on /DMO/R_BookingSupplement_D {
  key BookSupplUUID,
  TravelUUID,
  BookingUUID,
  BookingSupplementID,
  SupplementID,
  @Semantics.amount.currencyCode: 'CurrencyCode'
  BookSupplPrice,
  CurrencyCode,
  @Semantics.systemDateTime.localInstanceLastChangedAt: true
  LocalLastChangedAt,
  /* Associations */
  _booking: redirected to parent /DMO/I_Booking_D,
  _product,
  _supplementText,
  _travel : redirected to /DMO/I_Travel_D
}

```

Projecting the Draft BO Behavior

For the managed scenario with draft, create an interface behavior definition to expose the behavior to be provided via the API.

Context and Procedure

Create an interface behavior definition for the travel business object. In ADT, use the context menu of the root interface projection view to open the wizard for creating behavior definitions. The implementation type is prefilled with Projection.

Interface Behavior Definition /DMO/I_Travel_D

Use the complete behavior that was defined in the underlying behavior definition for the interface projection. (The template does not include everything, some features must be added manually.)

i *Expand the following code sample to view the source code of Projection Behavior Definition /DMO/I_Travel_D.*

Sample Code

```
interface;
use draft;
define behavior for /DMO/I_Travel_D alias Travel
use etag
{
    use create;
    use update;
    use delete;

    use action acceptTravel;
    use action rejectTravel;
    use action deductDiscount;
    use function GetDefaultsFordeductDiscount;

    use action validateAgencyID;
    use action validateCustomerID;
    use action validateDateRange;
    use action validateBookingCharge;

    use action Edit;
    use action Activate;
    use action Discard;
    use action Prepare;
    use action Resume;

    use association _Booking { create; with draft; }
}

define behavior for /DMO/I_Booking_D alias Booking
use etag
{
    use update;
    use delete;

    use action validateFlightCosts;

    use association _BookingSupplement { create; with draft; }
    use association _Travel { with draft; }
}

define behavior for /DMO/I_BookingSupplement_D alias BookingSupplement
use etag
{
    use update;
    use delete;

    use action validateSupplementPrice;

    use association _Travel { with draft; }
    use association _Booking { with draft; }
}
```

Develop UI-Specifics

This chapter explains how to optimize you data model for UI consumption using CDS annotations. The chapter outlines the most important and most common UI layouting tasks to get you started.

- [Adding Field Labels and Descriptions](#)

End-user texts, such as field labels or field descriptions must be translated. Therefore, the CDS development infrastructure is able to extract them from the source code and transfer the extracted texts to the actual translation infrastructure of the corresponding delivery package.

- [Providing Value Help](#)

The implementation of a value help in CDS enables the end user to choose values from a predefined list for input fields on a user interface.

- [Develop UI-Specifics](#)

How to solve the most common UI layout tasks with UI annotations in CDS.

Adding Field Labels and Descriptions

End-user texts, such as field labels or field descriptions, are taken from ABAP Dictionary data elements to which the corresponding element is bound - unless you redefine the texts using CDS annotations. Unlike technical element names, the header texts, field labels and descriptions are **language-dependent**. For example, the field 'Airline' would have a language-dependent label 'Airline Code'.

Such texts must be translated. Therefore, the CDS development infrastructure is able to extract them from the source code and transfer the extracted texts to the actual translation infrastructure of the corresponding delivery package.

Relevant Annotations

Annotation	Effect
<code>@EndUserText.label: '<text>'</code>	This annotation is used to define translatable semantic short texts (with maximum 60 characters) for an element of the CDS view (label text).
<code>@EndUserText.quickInfo: '<text>'</code>	The annotation defines a human-readable <code><text></code> that provides additional information about an element. The quick info is used for accessibility hints or the mouse over function.

→ Remember

The `<text>` specified in the source code should consist of text in the original language of the CDS source code and will be translated into the required languages.

Example

The listing below demonstrates the usage of `@EndUserText` annotations at the view and element (field) level:

```
...
@EndUserText.label: 'Overview of available flights' -- Annotation at the view level

DEFINE VIEW <CDS_view> as Flights {
```

...

```
-- Annotation at the field level
@EndUserText: { label: 'Airline Code',
quickinfo: 'Code to identify which airline operates the flight' }
carrier_id as CarrierID;
...
```

→ Tip

Press **F1** in the CDS source code editor for extended help content on @EndUserText annotation!

i Note

If @UI labeling annotations are used, they will be evaluated primarily. That means, they will overwrite the text given with the @EndUserText annotations.

OData Metadata

To verify that the additional information of labels and descriptions is pushed correctly to the OData service, you can check the OData metadata document. This can also be helpful to find out which label information is used if you maintain @UI and @EndUserText in your CDS view.

If no UI annotations are used, the OData metadata document of the example above should contain the annotations that are marked in the following image:

```
<EntityType sap:content-version="1" sap:label="Overview of Available Flights" Name="FlightsType">
  <Key/>
  <Property sap:label="Airline Code" Name="CarrierID" sap:quickinfo="Code to identify which airline operates the flight" sap:display-format="UpperCase"
```

Adding Value Help

The implementation of a value help in CDS enables the end user to choose values from a predefined list for input fields on a user interface.

Why and When to Use Value Help

You can define value helps in the CDS data layer for any input field on the UI (for example, selection fields, parameters, or writable fields in transactional scenarios). A value help is useful when the values for the input field have to be taken from a predefined set of values. When you call the value help, it displays all the available values from the value help provider. It appears on the UI when you choose the button  in the input field or press the **F4** key. The end user can filter by related information to identify the correct value. This makes it easier to find the desired value, especially if the value itself contains little or no identifying information, for example, an ID number.

❖ Example

To help the end user to enter the right customer ID to create a new booking, the application developer defines a value help that enables the user to enter the name or any other element from the value help provider to help find the correct number. The value help provider view in this case is a CDS view that manages customer information. As shown below, the end user is searching for a particular customer ID. The value help offers to filter by the customer last name, so that the end user can choose from the available entries. The value of the customer ID field is then transferred to the respective input field.

Expand the following figure to view the procedure of calling the value help on a Fiori Elements UI.

The screenshot shows a SAP Fiori Elements UI for an 'Unnamed Object'. The header bar includes the SAP logo, a back arrow, the text 'Travel', and a search icon. The main content area is titled '<Unnamed Object>' and contains a 'General Information' section. This section has several input fields: 'Agency ID' with the value '70006', 'Customer ID' with a cursor, 'Starting Date' with a placeholder 'MMM d, y', 'End Date' with a placeholder 'MMM d, y', and 'Booking Fee' with two empty input boxes. At the bottom, there is a 'Total Price' label and a dark bar with 'Save' and 'Cancel' buttons.

Value Help on a Fiori Elements UI

How are Value Helps Implemented?

Value helps are defined in CDS with an annotation on the element or parameter for which the value help dialog is to appear on the UI. The annotation `@Consumption.valueHelpDefinition` allows you to reference the value help provider without implementing an association. You simply assign a CDS entity as the value help provider and provide an element for the mapping in the annotation. All fields of the value help provider are displayed on the UI. When you choose one of the entries of the value help provider, the value of the referenced element is transferred to the input field.

→ Tip

It is recommended to implement value help annotations in the data definition. Although this is a UI related feature, you shouldn't implement it in the metadata extension.

For the default implementation of the value help, you can use any CDS entity that contains the desired values of the element for the input field. You do not need to explicitly define a CDS entity as the value help provider. However, the value help provider must be exposed for the OData service to make use of the value help.

i Note

Any annotation that is maintained in the value help provider is evaluated. This means that associated entities, value helps, and text associations of the value help provider view are also displayed and can be used in the value help. This means that you can have a value help in the value help.

Related Information

[Providing Value Help for Selection Fields](#)

[CDS Annotations](#)

Simple Value Help

A simple value help is convenient if you want to display values from the value help provider for an input field.

Context

You want to provide a value help for an input field on the UI.

The following steps implement a value help for a customer ID field, using a booking CDS view as an example.

Procedure

1. Create a data definition for a CDS view that serves as a value help provider. It must contain a field with the available values for the input field in the source view.

❖ Example

The value help provider view contains the customer ID and fields to identify the customer ID, such as the customer's name or address. The end user can then filter by these fields to find the right customer ID.

```
define view entity /DMO/I_Customer_VH as select from /dmo/customer
{
  key customer_id,
  first_name,
  last_name,
  title,
  street,
  postal_code,
  city,
  country_code,
  phone_number,
  email_address
}
```

To retrieve the full value list when invoking the value help, annotate the value help view with the annotation `@Consumption.valueHelpDefault.fetchValues: #AUTOMATICALLY_WHEN_DISPLAYED`. To explicitly disable loading the full list, use `@Consumption.valueHelpDefault.fetchValues: #ON_EXPLICIT_REQUEST`.

2. In your CDS source view, add the following annotation to the element for which you want to provide the value help on the UI.

```
@Consumption.valueHelpDefinition: [{ entity: { name: 'entityRef' ,
                                              element: 'elementRef' } }]
```

The annotation defines the binding for the value help to the value help providing entity. You must specify the entity name and the element providing the possible values for the annotated element.

❖ Example

The following code example shows how an annotation is used on the element `CustomerID` in `/DMO/I_Booking`. It references the value help provider view (`/DMO/I_CUSTOMER_VH`) for the customer ID and the element providing the possible values (`customer_id`) in the value help provider view.

```
define view /DMO/I_Booking as select from /dmo/booking
{...
  @Consumption.valueHelpDefinition: [{entity: { name: '/DMO/I_CUSTOMER_VH',
                                              element: 'customer_id' }}]
  customer_id as CustomerID,
  ... }
```


Results

If you expose the source view in an OData service, the value help provider view is automatically exposed with it. You do not have to list value help provider views in the service definition.

On a Fiori UI, choosing F4 in the selection field opens a search mask and the end user can filter by any field in the value help provider view. Selecting an entry transfers the value of the element that is referenced in the annotation to the annotated element in the source view.

The metadata of the OData service displays the value help implementation for the following properties:

- The property in the CDS source view for which a value help is implemented (`sap:value-list="standard"`)
- The value help provider entity type (`sap:value-list="true"`)
- The value help provider entity is marked as Target in the Annotations property. The value list enumerates every property in the value help provider that is exposed for the value help (`Annotation Term="Common.ValueList"`).
- The element that is defined in the mapping condition is marked as an inbound and outbound parameter Record `Type="Common.ValueListParameterInOut"`.

Expand to view the extracts of the metadata document of a service exposing a booking CDS view and the value help provider for the element CustomerID.

```
- <EntityType sap:content-version="1" sap:label="Consumer CDS View - Booking" Name="BookingType">
  + <Key>
    <Property sap:label="Travel ID" Name="TravelID" sap:quickinfo="Flight Reference Scenario: Travel ID" sap:display-format="NonNegative" MaxLength="8" Nullable="false"
      Type="Edm.String"/>
    <Property sap:label="Booking Number" Name="BookingID" sap:quickinfo="Flight Reference Scenario: Booking ID" sap:display-format="NonNegative" MaxLength="4"
      Nullable="false" Type="Edm.String"/>
    <Property sap:label="Booking Date" Name="BookingDate" sap:quickinfo="Flight Reference Scenario: Booking Date" sap:display-format="Date" Type="Edm.DateTime"
      Precision="0"/>
    <Property sap:label="Customer ID" Name="CustomerID" sap:quickinfo="Flight Reference Scenario: Customer ID" sap:display-format="NonNegative" MaxLength="6"
      Type="Edm.String" sap:value-list="standard"/>
    <Property sap:label="Airline ID" Name="CarrierID" sap:quickinfo="Flight Reference Scenario: Carrier ID" sap:display-format="UpperCase" MaxLength="3"
      Type="Edm.String"/>
    <Property sap:label="Flight Number" Name="ConnectionID" sap:quickinfo="Flight Reference Scenario: Connection ID" sap:display-format="NonNegative" MaxLength="4"
      Type="Edm.String"/>
    <Property sap:label="Flight Date" Name="FlightDate" sap:quickinfo="Flight Reference Scenario: Flight Date" sap:display-format="Date" Type="Edm.DateTime" Precision="0"/>
    <Property sap:label="Flight Price" Name="FlightPrice" sap:quickinfo="Flight Reference Scenario: Flight Price" Type="Edm.Decimal" Precision="17" sap:unit="CurrencyCode"
      Scale="3"/>
    <Property sap:label="Currency Code" Name="CurrencyCode" sap:quickinfo="Flight Reference Scenario: Currency Code" MaxLength="5" Type="Edm.String"
      sap:semantics="currency-code"/>
  </EntityType>
- <EntityType sap:content-version="1" sap:label="CDS View Customer - Value Help Provider" Name="CustomerType" sap:value-list="true">
  + <Key>
    <Property sap:label="Customer ID" Name="customer_id" sap:quickinfo="Flight Reference Scenario: Customer ID" sap:display-format="NonNegative" MaxLength="6"
      Nullable="false" Type="Edm.String"/>
    <Property sap:label="First Name" Name="first_name" sap:quickinfo="Flight Reference Scenario: First Name" MaxLength="40" Type="Edm.String"/>
    <Property sap:label="Last Name" Name="last_name" sap:quickinfo="Flight Reference Scenario: Last Name" MaxLength="40" Type="Edm.String"/>
- <Annotations xmlns="http://docs.oasis-open.org/odata/ns/edm" Target="cds_xdmoxbooking_vh.BookingType/CustomerID">
  - <Annotation Term="Common.ValueList">
    - <Record>
      <PropertyValue String="CDS View Customer - Value Help Provider" Property="Label"/>
      <PropertyValue String="Customer" Property="CollectionPath"/>
      <PropertyValue Property="SearchSupported" Bool="true"/>
    - <Property Value="Parameters">
      - <Collection>
        - <Record Type="Common.ValueListParameterInOut">
          <PropertyValue Property="LocalDataProperty" PropertyPath="CustomerID"/>
          <PropertyValue String="customer_id" Property="ValueListProperty"/>
        </Record>
        - <Record Type="Common.ValueListParameterDisplayOnly">
          <PropertyValue String="first_name" Property="ValueListProperty"/>
        </Record>
        - <Record Type="Common.ValueListParameterDisplayOnly">
          <PropertyValue String="last_name" Property="ValueListProperty"/>
        </Record>
        + <Record Type="Common.ValueListParameterDisplayOnly">
          <PropertyValue String="street" Property="ValueListProperty"/>
        </Record>
        + <Record Type="Common.ValueListParameterDisplayOnly">
          <PropertyValue String="city" Property="ValueListProperty"/>
        </Record>
        - <Record Type="Common.ValueListParameterDisplayOnly">
          <PropertyValue String="country_code" Property="ValueListProperty"/>
        </Record>
        - <Record Type="Common.ValueListParameterDisplayOnly">
          <PropertyValue String="phone_number" Property="ValueListProperty"/>
        </Record>
        - <Record Type="Common.ValueListParameterDisplayOnly">
          <PropertyValue String="email_address" Property="ValueListProperty"/>
        </Record>
      </Collection>
    </Property>
  </Annotation>
```

! Restriction

If you associate the value help provider to your source view, the metadata of the value help provider view do not have `sap:value-list="true"`. As a result, the request `<service_url>/$metadata?sap-value-list=<service_namespace>.<entity>Type/<element>` (where `<element>` is the element that is referenced in the annotation `@Consumption.ValueHelpDefinition` in your CDS view) does not work. This can lead to problems when invoking value helps.

To avoid this clash, separate your value help information in value help provider views that do not coincide with other associated views, for example text provider views.

Other Value Help Options

Other Capabilities for the Value Help Provider

Any annotation that is maintained in the value help provider is evaluated. This means that the following capabilities are possible for the value help:

- **Associations:** If the target of the association is included in the OData service, the elements of entities associated with the value help provider are also displayed as additional input fields.
- **Search Capabilities:** Including search capabilities in your value help provider enables the end user to search for any detail in an input field.
- **Value Helps:** The value help can itself contain value helps.
- **Text:** Text that is provided for the value help provider is also displayed.

Value Help as Dropdown List

If you want to show the possible values for the input field as a dropdown list only, instead of a search mask with all other elements of the value help provider, annotate the value help provider with `@ObjectModel : { resultSet.sizeCategory: #XS }` on the view level.

For dropdown value lists, the OData \$metadata document includes the annotation `sap:value-list= "fixed-values"` instead of `standard` on the property in the source view for which the value help is implemented.

The UI displays the possible values for the input field in a dropdown list. The following image shows a value help list for the carrier ID field in the booking scenario.

Airline ID:



☐ AA
 ☐ AC
 ☐ AF
 ☐ AZ
 ☐ BA
 ☐ CO

Drop Down Value Help

Value Help Typeahead in the Selection Field

The entries of the value help provider view can be displayed while typing in the selection field. From the list that opens you can choose an entry to fill the selection field with the desired value:

The UI displays the possible values for the input field while typing values from the value help provider:

Customer ID:



First Name	Last Name	City
Andreas	Vrsic	Walldorf
Max	Vrsic	Strausberg
Uli	Vrsic	Elsenz
Volker	Gutenberg	Koenigsbrunn
Sophie	Neubasler	Vélizy

Show All Items

Value Help Typeahead

To enable the typeahead functionality enable the value help provider entity with search capabilities. This is done by using the annotation `@Search.searchable: true` at value help level. The value help typeahead only finds values from the value help provider that are annotated with `@Search.defaultSearchElement: true`.

❖ Example

If you want to display customer information of customers with first names, last names and cities starting with **V**, the corresponding elements in the value help provider entity must be annotated with `@Search.defaultSearchElement: true`.

To limit the value help list to certain elements, annotate the elements that you want to display with `@UI.LineItem.importance: #HIGH`. Then, the other elements with lower importance are not taken into consideration for the value help list.

🏠 Value Help Via Association

If the value help provider is already associated with the source entity, you can use the association

`@Consumption.valueHelpDefinition.association: 'Assoc'` and reference the association instead of the entity. The binding condition is then already given in the on-condition of the association.

❖ Example

The following code example shows how the annotation is used on the element `CustomerID` in `/DMO/I_Booking`. It references the association to the value help provider view (`_Customer`).

```
define view /DMO/I_Booking as select from /dmo/booking
association [0..1] to /DMO/I_Customer_VH as _Customer on $projection.CustomerID = _Customer.cust
{...
    @Consumption.valueHelpDefinition.association: '_Customer'
    customer_id as CustomerID,
    ...
    _Customer
}
```

i Note

When using the option of consuming value helps via an association, it is not ensured that the value help works properly, see [Restriction](#).

Value Help with Additional Binding

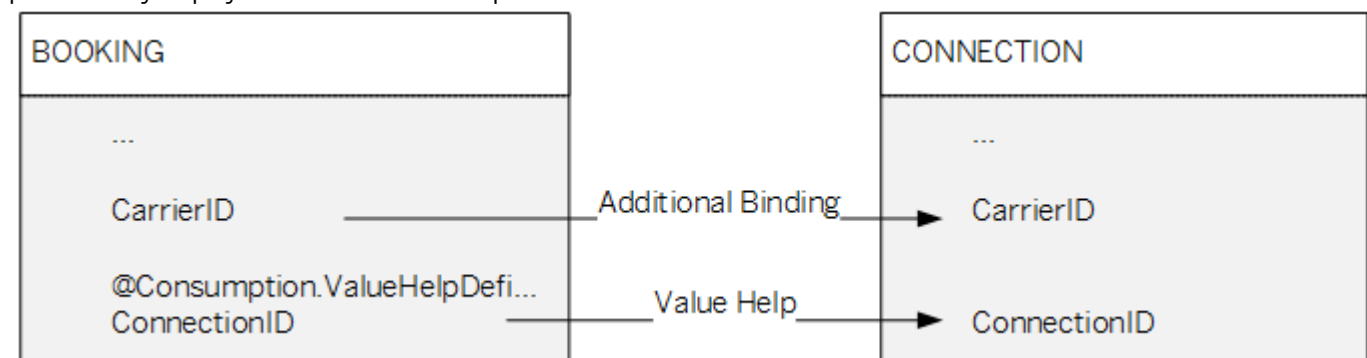
A preset filter condition can be established by an additional binding for the value help.

Context

You use an additional binding to define more than one binding condition between the source view and the value help provider. The value help is then automatically filtered by the additional binding. It proposes only entries that match the additional binding. This additional binding can either be another element or a parameter. These need to be present in the source view and in the value help provider view. When an entry is selected in the value help provider, the values of both binding elements are transferred to the input fields of the source CDS view.

❖ Example

In our booking scenario, we can apply the value help with an additional binding on the field `ConnectionID`. The value help provider is a view that manages connections. This value help provider contains not only the field for the connection IDs, but also a field for carrier IDs, which is also in the consumer view. We can establish a second binding condition so that the value help provider only displays connections with the prechosen carrier ID.



Value Help with Additional Binding on Booking Scenario

Procedure

1. Create a data definition for a CDS view that serves as a value help provider. It must contain a field with the available values for the input field in the source view. In addition, it must contain the field for which the additional binding is established.

The value help provider view contains the connection ID and the carrier ID, for which a mapping condition is defined.

```
define view /DMO/I_Connection_VH as select from /dmo/connection
{
  key carrier_id,
  key connection_id,
  airport_from_id,
  airport_to_id,
  departure_time,
  arrival_time,
  distance,
  distance_unit
}
```

2. In your CDS source view, add the following annotation to the element for which you want to provide the value help on the UI.

```
@Consumption.valueHelpDefinition: [{ entity: {name: 'entityRef', element: 'elementRef'} ,
  additionalBinding: [{ element: 'elementRef' | parameter: 'paramRef', localEle
```

The additional binding defines a second condition for the value help on the same target value help provider entity for filtering the value help result list and/or returning values from the selected value help record. The additional binding can be defined for an additional element or parameter. Depending on the value provided in `usage`, the additional binding works as a filter, result or filter and result condition:

- o **#Filter:** The value of the referenced element or parameter in `localElement` or `localParameter` is used as a filter for the value help. The value help only displays filtered results.

i Note

For numerical data types of local elements, the value help is always filtered for 0 (initial value) if the local elements are empty.

- o **#Result:** The referenced element or parameter in `localElement` or `localParameter` is filled with the value provided by the value help. When creating an instances, you can fill various fields with this option.
- o **#Filter_and_Result:** The value help works in both directions. It filters the value help and takes over the values for the corresponding fields.

`#Filter_and_Result` is the default if no `usage` type is specified.

The following code example shows the usage of the annotation on the element `ConnectionID` in `/DMO/I_Booking`. It references the value help provider view (`/DMO/I_Connection_VH`) and the element providing the possible values (`connection_id`) in the value help provider view, as well as the matching condition on the elements `CarrierID` and `carrier_id` in the consumer view and the value help provider view, respectively.

```
define view /DMO/I_Booking_VH as select from /dmo/booking
{...
  @Consumption.valueHelpDefinition: [{ entity: {name: '/DMO/I_Connection_VH' , element: 'co
    additionalBinding: [{ localElement: 'CarrierID', element: 'carrier_id', us
    }]
  }
  connection_id as ConnectionID,
  ... }
```

Results

Choosing **F4** in the selection field opens a search mask and the end user can display all available entries in the value help provider or filter by a field, for example, the destination airport. If the carrier ID is already filled in the consumer view, the value help provider is prefiltered by that value. Selecting an entry in the value help transfers the connection ID as well the carrier ID to the CDS consumer view.

The metadata of the OData service displays the value help implementation on the following properties:

- The property in the CDS source view for which a value help is implemented (`sap:value-list="standard"`)
- The value help provider entity type (`sap:value-list="true"`)
- The value help provider entity is marked as **Target** in the **Annotations** property. The value list enumerates every property in the value help provider that is exposed for the value help (`Annotation Term="Common.ValueList"`).
- The elements that are defined in the mapping conditions are marked as inbound and outbound parameters `Record Type="Common.ValueListParameterInOut"`.

! Restriction

If you associate the value help provider to your source view, the metadata of the value help provider view do not have `sap:value-list="true"`. As a result, the request `<service_url>/$metadata?sap-value-list=<service_namespace>.<entity>Type/<element>` (where `<element>` is the element that is referenced in the annotation `@Consumption.ValueHelpDefinition` in your CDS view) does not work. This can lead to problems when invoking value helps.

To avoid this clash, separate your value help information in value help provider views that do not coincide with other associated views, for example text provider views.

Value Help with Hierarchical Structure

A value help with hierarchical structure is convenient if you want to display values that have dependent child instances.

Context

If you want to provide a value help for an input field on the user interface (UI) that allows you to display not only a list of instances but also their hierarchically dependent child instances, you can apply treeviews in your value help.

Select: Value help for manager hierarchical

[Hide Filters](#)

Employee:
FirstName:
LastName:
Manager:

Items (10) 🔍 ↑ ↓ ⌵ ⌶

Employee	FirstName	LastName	Manager
▼ 1	Friedrich	Meier	–
10	Thilo	Rahn	1
▼ 2	Juan	Neubasler	–
20	Mathilde	Pratt	2
▼ 3	Hendrik	Mustermann	–
30	Anna	Becker	3
▼ 4	Hendrik	Babilon	–
40	Thomas	Hansmann	4
▼ 5	Amelie	Rahn	–
50	Thomas	Mustermann	5

With hierarchical value helps, it is possible to display the hierarchical relationship among instances. In the above value help example for the field `LastName`, we can see that Thilo Rahn reports directly to Friedrich Meier.

The following steps show you how you can implement a value help with a hierarchical structure.

Procedure

1. Create a view entity as the base value help provider view, which should have a recursive association to itself.

❖ Example

```
define root view entity /DMO/R_Manager_HRVH
as select from /dmo/manager_hr

association of many to one /DMO/R_Manager_HRVH as _ParentNode on $projection.Manager =
```

2. Create a CDS hierarchy and use the view entity created in the first step as source.

🔗 Sample Code

```
define hierarchy /DMO/R_Manager_HN_HRVH
as parent child hierarchy(
source /DMO/R_Manager_HRVH

child to parent association _ParentNode

start where
Manager is initial
siblings order by
LastName
)
```

3. Create a projection view based on the view entity you have created previously and add the two annotations below to enable the hierarchical view for your value help.

```
@UI.presentationVariant: [{ recursiveHierarchyQualifier: 'Your CDS Hierarchy artefact',
initialExpansionLevel: 1, qualifier: 'HIERARCHY' }]
```

```
@OData.hierarchy.recursiveHierarchy: [{ entity.name: 'Your CDS Hierarchy artefact' }
```

Sample Code

```
...

@UI.presentationVariant:[{ recursiveHierarchyQualifier: '/DMO/R_Manager_HN_HRVH', initialE>

@OData.hierarchy.recursiveHierarchy: [{ entity.name: '/DMO/R_Manager_HN_HRVH' }]

@Search.searchable: true
define root view entity /DMO/C_Manager_HRVH
  provider contract transactional_query
  as projection on /DMO/R_Manager_HRVH
{
  ...
```

4. Now, consume the value help for a field of your choice. In this example, we are using it for the view entity of employee.

Sample Code

```
define root view entity /DMO/C_Employee
  provider contract transactional_query
  as projection on /DMO/R_Employee
{
  @Consumption.valueHelpDefinition: [
    { entity: { name:'/DMO/C_Manager_HRVH', element: 'Employee' },
      presentationVariantQualifier:'HIERARCHY' } ]
  key EmployeeID,
    FirstName,

  ...
```

5. The value help is displayed as a treeview.

i Note

The value help provider view must be modeled correctly according to the guidelines for RAP hierarchies and provide all hierarchy capabilities. Therefore, the annotation `@OData.hierarchy.recursiveHierarchy.entity.name` is mandatory.

Hierarchies and the annotation `recursiveHierarchyQualifier` are supported for OData V4 only.

Related Information

[Adding Value Help](#)

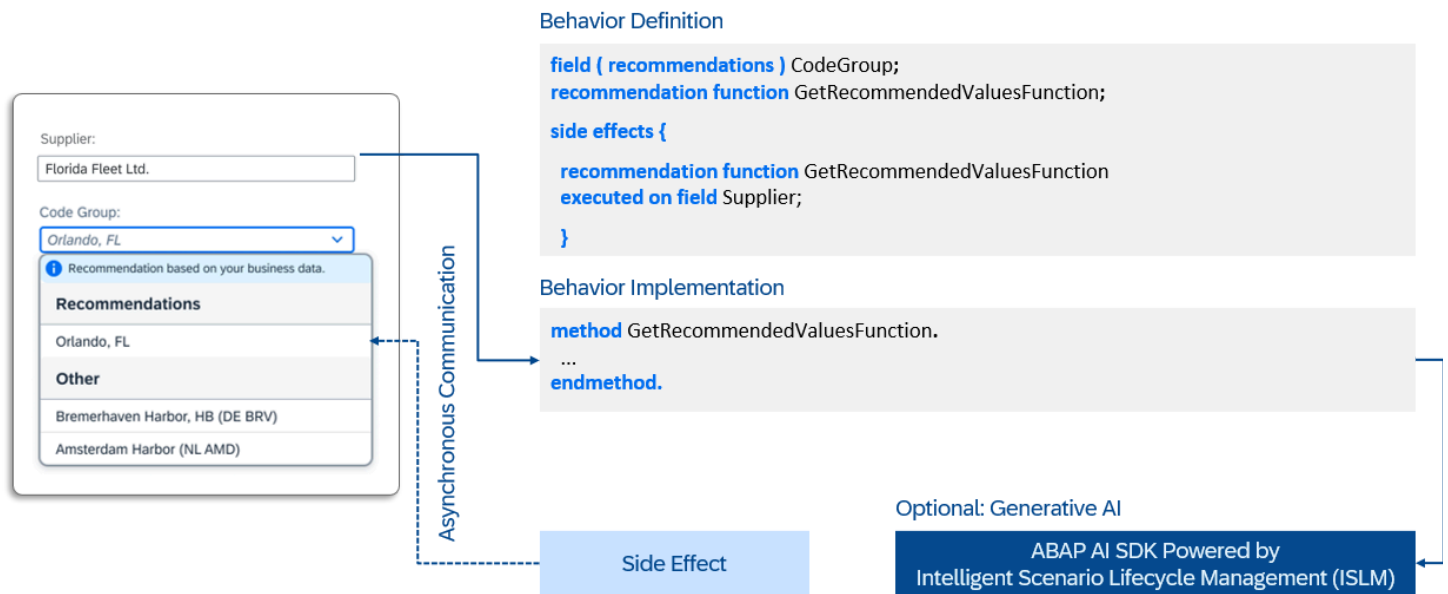
[Treeviews: Architecture Overview](#)

Adding RAP Recommendations

A RAP recommendation provides input assistance that proposes field values for input fields in RAP applications on SAP Fiori UIs, based on the user input and interaction history. You can implement RAP recommendations using generative AI or deterministic rules in the behavior pool of RAP business objects.

RAP Recommendations: Overview

You can implement RAP recommendations for an input field only if there's a value help available for this field. RAP recommendations enhance traditional value help with context-aware suggestions.



RAP Recommendations: Overview

In this example, RAP recommendations are calculated and suggested to users based on entries in the **Supplier** field. The system calculates RAP recommendations asynchronously. When the calculation on the back end is complete, the SAP Fiori UI is notified using a side effect. The UI then fetches the values from the back end and displays them on the UI.

Implementation Approaches

In the behavior pool of RAP business objects, you can implement the recommendation function by defining deterministic rules or by using generative AI. To implement RAP recommendations based on generative AI, use the ABAP AI SDK powered by Intelligent Scenario Lifecycle Management (ISLM).

The implementation approach depends on your required business logic. Consider the following before you decide:

- An implementation based on deterministic rules is recommended for rule-based inferences, for example, based on Customizing or other configuration settings.
- An implementation based on generative AI is recommended for categorizing content or for searching within a limited set of content, where large language models can deliver good results.
- Processing time can vary depending on the chosen implementation approach for the recommendation function. As a consequence, the calculated results might appear delayed on the UI.

Development Constraints

RAP recommendations are not supported by RAP extensibility. Recommended values can't be assigned to extension fields, and extension fields can't be used as triggers for calculating recommended values.

If recommended values have assigned descriptions (e.g. country codes), the UI also displays the description, only if the recommendation target field is the only key field of the value help. If the value help uses additional binding (see [Value Help with Additional Binding](#)), there may be cases where the bound field is also relevant for RAP recommendations. In such cases, the UI doesn't display a description for the bound field as part of the RAP recommendation.

Overview: Main Steps

To define and implement a RAP recommendation, follow these main steps:

1. [Defining RAP Recommendations in the Behavior Definition](#)

In the behavior definition of a RAP business object (RAP BO), you define input fields as recommendation target fields and a recommendation function. You must also add side effects to inform an SAP Fiori elements UI that data changes require the recalculation of recommended values.

2. [Implementing RAP Recommendations](#)

In the behavior pool of a RAP business object, you implement RAP recommendations using deterministic rules or generative AI features.

3. [Adding RAP Recommendations to the Projection Layer](#)

To expose the RAP recommendations on a Fiori Elements UI, you must reuse RAP recommendations and side effects in the projection behavior definition.

Related Information

[RAP Recommendations](#)

Defining RAP Recommendations in the Behavior Definition

In the behavior definition of a RAP business object (RAP BO), you define input fields as recommendation target fields and a recommendation function. You must also add side effects to inform an SAP Fiori elements UI that data changes require the recalculation of recommended values.

Prerequisites

You have a transactional SAP Fiori application based on a RAP BO. The RAP BO can be managed or unmanaged. In the reference scenario used in this documentation, the RAP BO is managed.

The fields for which you want to implement RAP recommendations must already have value help implemented (see [Adding Value Help](#)).

Procedure

1. In the behavior definition of the RAP BO, specify the input fields for which you want to provide RAP recommendations:

≡ **Sample Code**

```
field ( recommendations ) field1, field2, ...;
```

You can specify recommendation target fields for the root view entity of a RAP BO or for its child entities.

2. In the root view entity of the RAP BO, add a function to calculate RAP recommendations using the following syntax:

≡ **Sample Code**

```
recommendation function MyFunction;
```

Only one recommendation function is possible for each behavior definition. Recommendation functions can only be defined in root view entities, not in child entities. However, recommendation fields can be defined in all RAP BO entities.

You can choose any function name, including namespaces.

The signature of the recommendation function must include an importing parameter for keys and a tabular result parameter that includes the fields `value`, `score`, and `suggested`. You can use the available quick fix to generate the function declaration in the behavior pool of the RAP BO.

3. In the root view entity of the RAP BO, add a side effect so that the calculation of RAP recommendations is triggered when the current entity, defined entities, or defined fields change:

Sample Code

```
side effects {
  recommendation function MyFunction executed on
    $self | entity _MyEntity | field MyField ...;
}
```

These side effects can only be defined in the root view entity of the RAP BO, just like the recommendation function. However, you can also use fields from child entities as recommendation source fields. Define these fields using an association path: `_assoc.MyField`.

Sample Code

```
side effects {
  recommendation function MyFunction executed on field _assoc.MyField;
}
```

i Note

Don't confuse recommendation source fields with recommendation target fields. Source fields trigger the side effects. Recommendation target fields are the input fields for which RAP recommendations are provided.

`$self` triggers a complete reload of the screen whenever input changes on the screen. For performance reasons, we recommend that you use fields to trigger side effects where possible.

To support a flexible design, the definition of the recommendation function is separate from the definition of potential triggers, where RAP side effects are used. By adding a side effect definition to the definition of the recommendation function, you define the RAP recommendation source separately from the recommendation function.

Example

The following example is taken from the reference scenario for RAP recommendations (see [Reference Scenario for RAP Recommendations](#)). The example shows how you can define RAP recommendations in the behavior definition of the root view entity of a RAP BO:

Sample Code

```
define behavior for /dmo/r_traveltp_rec alias Travel
" ...
{
" ...

field ( recommendations ) Destination, Accommodation;
recommendation function GetRecommendedValuesFunction;

side effects
{
  recommendation function GetRecommendedValuesFunction executed on field Description, field De
}
" ...
```

In this behavior definition, the **Destination** and **Accommodation** fields are defined as recommendation target fields, meaning they are input fields for which RAP recommendations are to be provided. The recommendation function **GetRecommendedValuesFunction** is executed only when there is a change on the fields **Description** and **Destination**, using side effects.

In this example, the field **Destination** is a recommendation target field. It's populated with recommended values based on the user input in the **Description** field. At the same time, changes to the **Destination** field also trigger side effects for the recommendation function **GetRecommendedValuesFunction**, which calculates recommended values for the **Accommodation** field.

Task overview: [Adding RAP Recommendations](#)

Next task: [Implementing RAP Recommendations](#)

Implementing RAP Recommendations

In the behavior pool of a RAP business object, you implement RAP recommendations using deterministic rules or generative AI features.

Prerequisites

You have already defined the recommendation function in the behavior definition of the RAP business object (see [Defining RAP Recommendations in the Behavior Definition](#)).

Procedure

1. In the behavior definition of the RAP business object, use a quick fix to generate the method body.

If you use the quick fix, the `result` type of the implementation exit is typed to the list of recommendation target fields defined in the recommendation function definition.

2. Implement the retrieval of recommended values:

```
CLASS lhc_MyRAPBO DEFINITION INHERITING FROM cl_abap_behavior_handler.
  PRIVATE SECTION.
    ...

    METHODS getrecommendedvaluesfunction FOR READ
      IMPORTING keys FOR FUNCTION MyRAPBO~getrecommendedvaluesfunction RESULT result.
ENDCLASS.

CLASS lhc_MyRAPBO IMPLEMENTATION.
  ...

  METHOD getrecommendedvaluesfunction.
    " Retrieve values, for example, using ABAP AI SDK or deterministic rules
    " ...
    " Return result
    result = VALUE #( ( value = <rec_value>
                        score = <rec_score>
                        suggested = abap_true ) ).

  ENDMETHOD.
ENDCLASS.
```

The tabular `return` parameter of the recommendation function includes the following fields:

- `value`: It contains the recommended value.

- **score:** Use this field to assign a score to a recommended value. This field is useful if you want to implement multiple recommended values for one value help and want to order the values. The recommended values then appear at the top of the dropdown menu for the value help, ordered by their scores. A higher score results in a higher position of the recommended value in the value help on the UI.
- **suggested:** Set this field to `abap_true` or `abap_false`. This setting determines whether a recommended value is directly entered in the input field, ready for user confirmation. `abap_false` is the default.

You can define more than one recommended value.

How you combine the fields in the `return` parameter depends on the business logic you use. For example, in an implementation using ABAP AI SDK, generative AI can calculate a score for each possible recommended value. In addition, you can decide that the recommended value with the top score gets the setting `suggested = abap_true`.

The way how recommended values are calculated depends on your application. You can implement the retrieval of recommended values by using generative AI-based features using the ABAP AI SDK powered by ISLM. Alternatively, you can implement deterministic rules to propose recommended values.

Example

[Reference Scenario for RAP Recommendations](#)

For RAP recommendations, you also get demo content that you can use for testing. This content can also help you explore how to implement this feature in your own sample applications. The reference scenario forms the basis of the examples in this documentation.

[Example: Implementing RAP Recommendations](#)

This example shows how you can implement RAP recommendations using deterministic rules and the ABAP AI SDK powered by Intelligent Scenario Lifecycle Management (ISLM).

[Example: Implementing Generative AI Features for RAP Recommendations](#)

This example shows how you can implement generative AI features to provide RAP recommendations.

Task overview: [Adding RAP Recommendations](#)

Previous task: [Defining RAP Recommendations in the Behavior Definition](#)

Next task: [Adding RAP Recommendations to the Projection Layer](#)

Reference Scenario for RAP Recommendations

For RAP recommendations, you also get demo content that you can use for testing. This content can also help you explore how to implement this feature in your own sample applications. The reference scenario forms the basis of the examples in this documentation.

RAP Business Object

In the reference scenario, RAP recommendations propose accommodations based on the destination that business users select in a travel booking. To keep things simple, the scenario uses a RAP business object with only one entity, `Travel`, and with a limited number of fields.

i Note

This reference scenario for RAP recommendations is self-contained and works independently from other reference scenarios. The reference scenario also includes a generator class that you can use to fill the database tables of the reference scenario with sample data.

Downloading the Reference Scenario

The reference scenario for RAP recommendations is part of the ABAP Flight Reference Scenario, which you can download from GitHub. The ABAP Flight Reference Scenario helps you get started with development in the context of the ABAP RESTful Application Programming Model. You can find the reference scenario for RAP recommendations in the package /DMO/FLIGHT_REC.

Related Information

[Downloading the ABAP Flight Reference Scenario](#)

Example: Implementing RAP Recommendations

This example shows how you can implement RAP recommendations using deterministic rules and the ABAP AI SDK powered by Intelligent Scenario Lifecycle Management (ISLM).

Context

In this example of a simple RAP travel business object, you get an overview of how the method `GetRecommendedValuesFunction` is implemented. The method gets recommended values using the following approaches:

- Deterministic rules: Recommended values for accommodations, based on the destinations provided in the `Destination` field on the UI
- Generative AI: Recommended values for destinations, based on the descriptions users provide in the `Description` field on the UI

The example is based on the reference scenario for RAP, which is available on GitHub (see [Reference Scenario for RAP Recommendations](#)).

Sample Code

Expand the sample code to view the full source code:

Sample Code

```
CLASS lhc_r_traveltp_rec DEFINITION INHERITING FROM cl_abap_behavior_handler.
  PRIVATE SECTION.
    TYPES: BEGIN OF rec_struct,
              value      TYPE string,
              score      TYPE int1,
              suggested TYPE abap_bool,
            END OF rec_struct.
    TYPES rec_tab TYPE TABLE OF rec_struct WITH DEFAULT KEY.
    TYPES country_tab TYPE TABLE OF /dmo/c_dest_rec_h WITH DEFAULT KEY.
    TYPES business_user_travel TYPE TABLE FOR READ RESULT /dmo/r_traveltp_rec\\travel.
    METHODS GetRecommendedValuesFunction FOR READ
      IMPORTING keys FOR FUNCTION Travel~GetRecommendedValuesFunction RESULT result.
    METHODS get_accommodation_names
      IMPORTING business_user_travel TYPE business_user_travel
      RETURNING VALUE(result)        TYPE string_table.
    METHODS get_country_names
      RETURNING VALUE(result) TYPE country_tab.
    METHODS get_rec_values
      IMPORTING rec_values TYPE string_table
      RETURNING VALUE(result) TYPE rec_tab.
  ENDClass.
```

CLASS lhc_r_traveltp_rec IMPLEMENTATION.

METHOD GetRecommendedValuesFunction.

READ ENTITIES OF /dmo/r_traveltp_rec IN LOCAL MODE

ENTITY Travel

FIELDS (Description Destination) WITH CORRESPONDING #(keys)

RESULT DATA(business_user_travel)

FAILED failed.

"Selects all available country names from the value help

DATA(country_names) = get_country_names().

"Selects all accommodations that match the business user's input for the destination for the

DATA(accommodation_names) = get_accommodation_names(business_user_travel).

"Prepare recommendation values for accommodations

DATA(accommodation_rec_values) = get_rec_values(accommodation_names).

LOOP AT business_user_travel ASSIGNING FIELD-SYMBOL(<travel>).

"Based on the business user's input for the description, the AI should suggest destination

DATA(destinations) = /dmo/cl_ai_util_rec=>get_destination_rec_values(description = <trave

"Deletes destinations the AI suggested, but that are not actual countries

LOOP AT destinations ASSIGNING FIELD-SYMBOL(<dest>).

IF NOT line_exists(country_names[CountryName = <dest>]).

DELETE destinations FROM sy-tabix.

ENDIF.

ENDLOOP.

"Prepare recommendation values for destinations

DATA(destination_rec_values) = get_rec_values(destinations).

"Return recommended values for destination and accommodation

INSERT VALUE #(%tky = <travel>-%tky

%param-travel = VALUE #(destination = CORRESPONDING #(destination_rec_v
accommodation = CORRESPONDING #(accommodation_r

ENDLOOP.

ENDMETHOD.

METHOD get_country_names.

SELECT FROM /dmo/c_dest_rec_h

FIELDS CountryName

INTO CORRESPONDING FIELDS OF TABLE @result.

ENDMETHOD.

METHOD get_accommodation_names.

IF lines(business_user_travel) IS INITIAL.

RETURN.

ENDIF.

SELECT DISTINCT acc_name

FROM /dmo/acc_rec_h AS accommodation

INNER JOIN @business_user_travel AS destination

ON accommodation~acc_country = destination~Destination

INTO TABLE @result.

ENDMETHOD.

METHOD get_rec_values.

IF rec_values IS INITIAL.

RETURN.

ENDIF.

DATA(counter) = 0.

LOOP AT rec_values ASSIGNING FIELD-SYMBOL(<rec_values>) TO 3. "#EC CI_NOORDER
counter += 1.

INSERT VALUE #(value = <rec_values>

score = 100 - counter

suggested = COND #(WHEN counter = 1

THEN abap_true

ELSE abap_false)) INTO TABLE result.

ENDLOOP.

ENDMETHOD.

ENDCLASS.

The code sample is explained below step by step.

Helper Method for Retrieving Valid Countries

The method `get_country_names` reads country names using the CDS view entity `/dmo/c_dest_rec_h`, which provides value help for country names:

Sample Code

```
METHOD get_country_names.
  SELECT FROM /dmo/c_dest_rec_h
    FIELDS CountryName
  INTO CORRESPONDING FIELDS OF TABLE @result.
ENDMETHOD.
```

The list of country names is used later to match valid country names against the values returned by generative AI.

Helper Method for Determining Accommodations

In the following sample code, recommended values for accommodations are calculated using deterministic rules.

The method `get_accommodation_names` retrieves all distinct available accommodation names that match the country entered in the `Destination` field. It uses the database table `/dmo/acc_rec_h`, which provides the value help for accommodations:

Sample Code

```
METHOD get_accommodation_names.
  IF lines( business_user_travel ) IS INITIAL.
    RETURN.
  ENDIF.

  SELECT DISTINCT acc_name
    FROM /dmo/acc_rec_h AS accommodation
    INNER JOIN @business_user_travel AS destination
      ON accommodation~acc_country = destination~Destination
  INTO TABLE @result.
ENDMETHOD.
```

Helper Method for Preparing Recommended Values

The `get_rec_values` method prepares recommended values for further processing.

The recommendation function calls this method twice: for accommodations and destinations. The method loops over only the first three returned recommended values and fills the fields `score` and `suggested` for them. The values get descending scores: 99, 98, and 97. The recommended value with score 99 gets the setting `suggested = abap_true`.

As a result, the UI displays a maximum of three recommended values, even deterministic rules or generative AI return more. For the sake of simplicity, these three recommended values are selected randomly in this example. Depending on your use case, you might want to implement business logic to select the appropriate recommended values. For more information about `score` and `suggested`, see [Implementing RAP Recommendations](#).

Sample Code

```

METHOD get_rec_values.
  IF rec_values IS INITIAL.
    RETURN.
  ENDIF.

  DATA(counter) = 0.

  LOOP AT rec_values ASSIGNING FIELD-SYMBOL(<rec_values>) TO 3. "#EC CI_NOORDER
    counter += 1.

    INSERT VALUE #( value = <rec_values>
                    score = 100 - counter
                    suggested = COND #( WHEN counter = 1
                                        THEN abap_true
                                        ELSE abap_false ) ) INTO TABLE result.

  ENDLOOP.
ENDMETHOD.

```

Recommendation Function

1. In the recommendation function `GetRecommendedValuesFunction`, the `Description` and the `Destination` fields of the `Travel` entity of the `RAP` business object are read:

Sample Code

```

METHOD GetRecommendedValuesFunction.

  READ ENTITIES OF /dmo/r_traveltp_rec IN LOCAL MODE
    ENTITY Travel
    FIELDS ( Description Destination ) WITH CORRESPONDING #( keys )
    RESULT DATA(business_user_travel)
    FAILED failed.
  " ...

```

The result is written to the internal table `business_user_travel`.

2. The methods `get_country_names`, `get_accommodation_names`, and `get_rec_values` are called:

Sample Code

```

"...
  "Selects all available country names from the value help
  DATA(country_names) = get_country_names( ).

  "Selects all accommodations that match the business user's input for the destination fc
  DATA(accommodation_names) = get_accommodation_names( business_user_travel ).

  "Prepare recommendation values for accommodations
  DATA(accommodation_rec_values) = get_rec_values( accommodation_names ).
  "...

```

Based on the entry in the `Destination` field, the method `get_accommodation_names` selects matching accommodations using a deterministic approach. The result `accommodation_names` is then further processed by the `get_rec_values` method.

3. To get the recommended values for the destination based on the user's description using generative AI, the method `get_destination_rec_values` from the `/dmo/ai_util_rec` class is called:

Sample Code

```
"...
    LOOP AT business_user_travel ASSIGNING FIELD-SYMBOL(<travel>).

        "Based on the business user's input for the description, the AI should suggest destir
        DATA(destinations) = /dmo/cl_ai_util_rec=>get_destination_rec_values( description = <
"...
```

This class contains helper methods to prepare prompts, submit requests, and receive answers from large language models (see [Example: Implementing Generative AI Features for RAP Recommendations](#)).

The returned destinations are written to the internal table `destinations` for further processing.

4. A check verifies if destinations returned by the `get_destination_rec_values` method match countries in the value help. For this purpose, the internal table `countries` is used, which was filled with all country names from the value help using the method `get_country_names`:

Sample Code

```
"...
    "Deletes destinations the AI suggested, but that are not actual countries
    LOOP AT destinations ASSIGNING FIELD-SYMBOL(<dest>).
        IF NOT line_exists( country_names[ CountryName = <dest> ] ).
            DELETE destinations FROM sy-tabix.
        ENDIF.
    ENDLOOP.
"...
```

The recommendation function `GetRecommendedValuesFunction` must handle the responses from large language models (LLMs). These LLMs might return values that don't exist, so a mitigation is needed. In the example above, all returned destinations that aren't part of the value help are deleted.

5. The method `get_rec_values` is called to prepare the returned values for destinations for further processing:

Sample Code

```
" ...
    DATA(destination_rec_values) = get_rec_values( destinations ).
" ...
```

6. All recommended values for destinations and accommodations are inserted into the internal table `result`, which serves as a tabular return parameter of the recommendation function:

Sample Code

```
" ...
    "Return recommended values for destination and accommodation
    INSERT VALUE #( %tky = <travel>-%tky
                    %param-travel = VALUE #( destination = CORRESPONDING #( destination_
                                                accommodation = CORRESPONDING #( accommodat
    ENDLOOP.
ENDMETHOD.
```

The recommendation function returns the recommended values, which are then displayed on the UI.

Example: Implementing Generative AI Features for RAP Recommendations

This example shows how you can implement generative AI features to provide RAP recommendations.

Context

You can implement RAP recommendations using generative AI. In this example, the RAP business object is a simple travel business object. User input in a **Description** field is used to generate recommended values for countries in a **Destination** field.

The methods in this example process user input, settings, and other input to send a prompt and get a response from a large language model (LLM). All methods are called as part of a RAP recommendation implementation (see [Example: Implementing RAP Recommendations](#)).

This example is based on the reference scenario for RAP, which is available on GitHub (see [Reference Scenario for RAP Recommendations](#)).

i Note

Some code in the code sample is commented out because it only runs if you set up a generative AI scenario using ABAP AI SDK. For more information about ABAP AI SDK and its setup, see [Developing Your Own AI-Enabled Applications](#) in the *Generative AI in ABAP Cloud* documentation.

Sample Code

Expand the sample code to view the full source code:

≡ Sample Code

```

CLASS /dmo/cl_ai_util_rec DEFINITION
  PUBLIC
  FINAL
  CREATE PUBLIC.
  PUBLIC SECTION.
    CLASS-METHODS get_destination_rec_values
      IMPORTING description TYPE string
      RETURNING VALUE(result) TYPE string_table.
  PROTECTED SECTION.
  PRIVATE SECTION.
    CLASS-METHODS get_destinations
      RETURNING VALUE(result) TYPE string.
    CLASS-METHODS get_prompt_settings
      IMPORTING description TYPE string
      destinations TYPE string
      RETURNING VALUE(result) TYPE string.
    CLASS-METHODS get_completion_ai
      IMPORTING prompt_setting TYPE string
      RETURNING VALUE(result) TYPE string.
    CLASS-METHODS get_completion_api_instance
      RETURNING VALUE(result) TYPE REF TO if_aic_completion_api.
    CLASS-METHODS get_destinations_tab
      IMPORTING destination_string TYPE string
      RETURNING VALUE(result) TYPE string_table.
ENDCLASS.

CLASS /dmo/cl_ai_util_rec IMPLEMENTATION.

  METHOD get_destination_rec_values.
    DATA(destinations) = get_destinations( ).
    "DATA(prompt_setting) = get_prompt_settings( description = description destinations = destin
    "DATA(destination_string) = get_completion_ai( prompt_setting ).

    "result = get_destinations_tab( destination_string ).

```

```

    result = VALUE string_table( ( ) ).
ENDMETHOD.

METHOD get_destinations.
  "Using this explicit list of countries, the AI result is improved
  SELECT FROM /dmo/c_dest_rec_h
    FIELDS CountryName
  INTO TABLE @DATA(country_names).

  LOOP AT country_names ASSIGNING FIELD-SYMBOL(<country_name>).
    result &&= <country_name>-CountryName && ', '.    "#EC CI_NOORDER
  ENLOOP.
ENDMETHOD.

METHOD get_prompt_settings.
  IF description IS NOT INITIAL.
    "For security reasons you should extract this to, for example, your ISLM scenario
    result = ##NO TEXT
      | \{ "model": "gpt-4o",| &&
      | "messages": [| &&
      | \{ "role": "system", "content": "You are a travel assistant returning destinati
      | \{ "role": "user", "content": "Example 1: Analyze: `Vacation in Barcelona`. Exp
      | \{ "role": "assistant", "content": "1" \},| &&
      | \{ "role": "user", "content": "Example 2: Analyze: `Trip to Munich`. Expected:
      | \{ "role": "assistant", "content": "2" \},| &&
      | \{ "role": "user", "content": "Example 3: Analyze: `Business in Paris`. Expecte
      | \{ "role": "assistant", "content": "3" \},| &&
      | \{ "role": "user", "content": "Example 4: Analyze: `Sports in Sydney`. Expected
      | \{ "role": "assistant", "content": "4" \},| &&
      | \{ "role": "user", "content": "Example 5: Analyze: `Visit to Hanover`. Expected
      | \{ "role": "assistant", "content": "5" \},| &&
      | \{ "role": "user", "content": "Analyze the type of the following description. R
      | Do not suggest, infer, or create any location that is not explicitly present in
      | Your response must contain only the country names, without any explanations or
      | The list is: { destinations }. Description: `{ description }`.| &&
      | The result should not contain any decoration or quotation marks. Only up to thr
      | If there is more than one destination, separate them by commas." \}| &&
      | ],| &&
      | "max_tokens": 3,| &&
      | "temperature": 0.0 \}||.
  ENDIF.
ENDMETHOD.

METHOD get_completion_ai.
  DATA(completion_api_instance) = get_completion_api_instance( ).

  IF completion_api_instance IS BOUND.
    TRY.
      DATA(completion_api_result) = completion_api_instance->execute_for_string( prompt_sett
      CATCH cx_aic_completion_api.
      RETURN result.
    ENDTRY.

    result = completion_api_result->get_completion( ).
  ENDIF.
ENDMETHOD.

METHOD get_completion_api_instance.
  TRY.
    result = cl_aic_islm_compl_api_factory=>get( )->create_instance( 'MyIntelligentScen' ).
  CATCH cx_aic_api_factory.
  RETURN result.
ENDTRY.
ENDMETHOD.

METHOD get_destinations_tab.
  SPLIT destination_string AT ` , ` INTO TABLE result.
ENDMETHOD.

ENDCLASS.

```

The code sample is explained below step by step.

Getting Recommended Values

To get recommended values for destinations based on a description, the method `get_destination_rec_values` calls helper methods `get_destinations`, `get_prompt_settings`, and `get_completion_ai`:

Sample Code

```
METHOD get_destination_rec_values.
  DATA(destinations) = get_destinations( ).
  "DATA(prompt_setting) = get_prompt_settings( description = description destinations = destin
  "DATA(destination_string) = get_completion_ai( prompt_setting ).

  "result = get_destinations_tab( destination_string ).
  result = VALUE string_table( ( ) ).
ENDMETHOD.

METHOD get_destinations_tab.
  SPLIT destination_string AT ` , ` INTO TABLE result.
ENDMETHOD.
```

The method `get_destinations` returns the available destinations as a string. The method `get_prompt_settings` generates the prompt for the LLM using the importing parameters `description` and the string with the list of destinations. Then, the method `get_completion_ai` is called, which returns the LLM response. The method `get_destinations_tab` splits the destination string and converts it into an internal table before it's returned in the `result` table.

The response is further processed by the recommendation function (see [Example: Implementing RAP Recommendations](#)).

Helper Method for Getting Country Names from Value Help

To get all country names available in the value help for destinations, the following method is used:

Sample Code

```
METHOD get_destinations.
  SELECT FROM /dmo/c_dest_rec_h
    FIELDS CountryName
    INTO TABLE @DATA(country_names).

  LOOP AT country_names ASSIGNING FIELD-SYMBOL(<country_name>).
    result &&= <country_name>-CountryName && ', '.      "#EC CI_NOORDER
  ENDLOOP.
ENDMETHOD.
```

The method `get_destinations` selects all country names from the CDS view entity `/dmo/c_dest_rec_h`, which is used for value help. The method concatenates all country names, separated by commas. This format is needed as part of the prompt to the LLM.

This list of all country names available in the value help for destinations provides more context for the LLM. As a result, the prompt produces more precise results.

Helper Method for Preparing the Prompt

To prepare all available data and additional settings for a prompt, the following method is used:

Sample Code

```
METHOD get_prompt_settings.
  IF description IS NOT INITIAL.
```

"For security reasons you should extract this to, for example, your ISLM scenario

```
result = ##NO_TEXT
```

```
| \{"model": "gpt-4o",| &&
| "messages": [| &&
| \{"role": "system", "content": "You are a travel assistant returning destinati
| \{"role": "user", "content": "Example 1: Analyze: `Vacation in Barcelona`. Exp
| \{"role": "assistant", "content": "1" \},| &&
| \{"role": "user", "content": "Example 2: Analyze: `Trip to Munich`. Expected:
| \{"role": "assistant", "content": "2" \},| &&
| \{"role": "user", "content": "Example 3: Analyze: `Business in Paris`. Expecte
| \{"role": "assistant", "content": "3" \},| &&
| \{"role": "user", "content": "Example 4: Analyze: `Sports in Sydney`. Expected
| \{"role": "assistant", "content": "4" \},| &&
| \{"role": "user", "content": "Example 5: Analyze: `Visit to Hanover`. Expected
| \{"role": "assistant", "content": "5" \},| &&
| \{"role": "user", "content": "Analyze the type of the following description. R
| Do not suggest, infer, or create any location that is not explicitly present in
| Your response must contain only the country names, without any explanations or
| The list is: { destinations }. Description: `{ description }`.| &&
| The result should not contain any decoration or quotation marks. Only up to thr
| If there is more than one destination, separate them by commas." \},| &&
| ],| &&
| "max_tokens": 3,| &&
| "temperature": 0.0 \},|.
```

```
ENDIF.
ENDMETHOD.
```

The method concatenates information into one string. The string includes the selected large language model, system role, maximum number of tokens, and temperature for the LLM. The method also provides examples and additional instructions for the LLM. The list of possible destinations and the description entered by the user are provided using the `destination` and `description` parameters.

⚠ Caution

For demo purposes, all information for the prompt is stored as text in the `get_prompt_settings` method. For security reasons, you might want to use other means to transfer this type of information to an LLM. For example, use settings and templates in an intelligent scenario. For more information about creating intelligent scenarios, see [Developing Your Own AI-Enabled Applications](#) in the *Generative AI in ABAP Cloud* documentation.

Helper Methods for Generative AI Calls

To instantiate the completion API, the following method is used:

≡ Sample Code

```
METHOD get_completion_api_instance.
  TRY.
    result = cl_aic_islm_compl_api_factory=>get( )->create_instance( 'MyIntelligentScen' ).
  CATCH cx_aic_api_factory.
    RETURN result.
  ENDTRY.
ENDMETHOD.
```

The class `cl_aic_islm_compl_api_factory` is part of the ABAP AI SDK powered by ISLM. To create a completion API instance, an intelligent scenario must exist. For more information about setting up the ABAP AI SDK and about creating intelligent scenarios, see [Developing Your Own AI-Enabled Applications](#) in the *Generative AI in ABAP Cloud* documentation.

To call the configured LLM and get a response, the following method is used:

≡ Sample Code

```

METHOD get_completion_ai.
  DATA(completion_api_instance) = get_completion_api_instance( ).

  IF completion_api_instance IS BOUND.
    TRY.
      DATA(completion_api_result) = completion_api_instance->execute_for_string( prompt_sett
      CATCH cx_aic_completion_api.
        RETURN result.
    ENDTRY.

    result = completion_api_result->get_completion( ).
  ENDIF.
ENDMETHOD.

```

The method `get_completion_ai` calls the methods `execute_for_string` and `get_completion`, which are part of the ABAP AI SDK. With the `execute_for_string` method, you send a prompt to the LLM, using the string `prompt_setting` generated by method `get_prompt_settings` (see previous steps). The method `get_completion` returns the results that the generative AI calculated based on the prompt provided by the method `execute_for_string`.

Adding RAP Recommendations to the Projection Layer

To expose the RAP recommendations on a Fiori Elements UI, you must reuse RAP recommendations and side effects in the projection behavior definition.

Prerequisites

You have already defined and implemented RAP recommendations (see [Defining RAP Recommendations in the Behavior Definition](#) and [Implementing RAP Recommendations](#)).

Procedure

In the projection behavior definition of the RAP business object, add `use recommendations` and `use side effects` as shown in the following example:

≡ Sample Code

```

projection;
strict ( 2 );
use draft;
use recommendations;
use side effects;

define behavior for /dmo/c_traveltp_rec alias Travel
use etag
{
  use create;
  use update;
  use delete;

  use action Activate;
  use action Discard;
  use action Edit;
  use action Resume;
}

```

```

    use action Prepare;
}

```

i Note

Be sure to add `use side effects`, too, which is required because side effects must be used to trigger the recommendation function.

Task overview: [Adding RAP Recommendations](#)

Previous task: [Implementing RAP Recommendations](#)

Defining CDS Annotations for Metadata-Driven UIs

How to solve the most common UI layout tasks with UI annotations in CDS.

Prerequisites

In order to use this help chapter, you should be familiar with:

- CDS Views and CDS DDL (Data Definition Language)
- Setting up Data Definitions, Service Bindings and Service Definitions
- Using ABAP Development Tools (ADT)

In order to use the example code, download the [ABAP Flight Reference Scenario](#).

Context

For a UI to work, you do not need to implement everything yourself. The barebones of the UI will be taken care of automatically by RAP if you use the Fiori Elements App Preview (FEAP) in the service binding. For example the root page will have a header and a list report (table) facet with sorting and filtering capabilities, even without you implementing any UI annotations in CDS. Furthermore the UI will include a Share-Button and a basic view manager.

i Note

To view the entries of your database on the UI without implementing any UI annotations, click on the gear icon on the UI and select at least one column.

FAQ

This FAQ directs you to the correct topics for questions that often arise when working with Metadata-Driven UIs for the first time.

- [What are fields?](#)
- [How can I overwrite default labels of fields?](#)
- [How can I define the positioning of fields?](#)
- [How can I filter my data by specific fields?](#)
- [How can I manipulate the Object Page Layout?](#)

- [How can I hide fields on the Object Page?](#)
- [How can I visualize my data?](#)

Source Code Structure

You can use UI annotations both in Data Definition or Metadata Extension files.

i Note

We recommend that you use a Metadata Extension file for all UI annotations because this will improve the legibility of the source code in the Data Definition and also simplify the development and maintenance of the CDS view. Metadata Extensions also allow for layering of multiple Metadata Extensions, which is useful if your BO is used by customers, partners and industries at the same time with different customization needs.

Your code will have the following sections:

1. Header section
2. Annotate/define view section
3. Facet Definitions
4. Element List
 - a. Fields
 - b. Associations

See the code snippet below for a complete overview of all sections and the correct syntax.

Find the Metadata Extension files for the Travel Scenario in the [Complete UI Annotations Example Code](#).

i Expand the following code sample to view the source code

☰ Sample Code

```
// HEADER SECTION
@Metadata.layer: ...
@UI.headerInfo: ...

// ANNOTATE VIEW SECTION
annotate/define view ...

{
  // FACET DEFINITIONS
  // Header Facets (Object Page):
  @UI.facet: [ { purpose: #HEADER, ... } ],

  // Body Facets (Object Page)
  { ... } ]

  // ELEMENT LIST

  // FIELDS

  @UI: { lineItem:      [ { ... } ],
        identification: [ { ... } ],
        selectionField: [ { ... } ],
        fieldGroup:    [ { ... } ] }
  element_1;

  @UI: { lineItem:      [ { ... } ],
```

```

        identification: [ { ... } ],
        selectionField: [ { ... } ],
        fieldGroup:      [ { ... } ] }
    element_2;

    ...

    // ASSOCIATIONS
    _Association1

}

```

Related Information

[Example: Metadata-Driven User Interface](#)

Example: Metadata-Driven User Interface

Get an idea what your UI might look like and how changes in the code influence the UI.

The following screenshots from the Fiori Elements App Preview (FEAP) serve as examples of how your UI might look like and how changes in the code influence the UI. The Fiori Elements App Preview can be started from the service binding.

i Note

With the preview, you haven't created a complete Fiori app yet, but you must still perform some steps to implement the SAP Fiori UI. For more information about Fiori, see [Introduction to SAP Fiori Elements](#).

In the following documentation, the most common page layouts, also called Fiori floorplans, are explained in detail:

- List reports
- Object pages

List Report Page

The list report is used to display all the entries of a dataset in one list or table. You can specify which columns to display and which ones to hide or omit.

A

Standard * ▾

Search Travel ID: Agency ID: Customer ID: Adapt Filters

B

List Report - Table Header (7,462) Standard ▾

C

Travel ID	Agency	Customer	Start of Travel	End of Travel	Travel Status
1	Maxitrip (70041)	Ryan (594)	Nov 2, 2019	Aug 30, 2020	B
2	Hot Socks Travel (70007)	Prinz (608)	Nov 2, 2019	Nov 2, 2019	B
3	Hendrik's (70046)	Detemple (93)	Nov 2, 2019	Aug 30, 2020	P

D

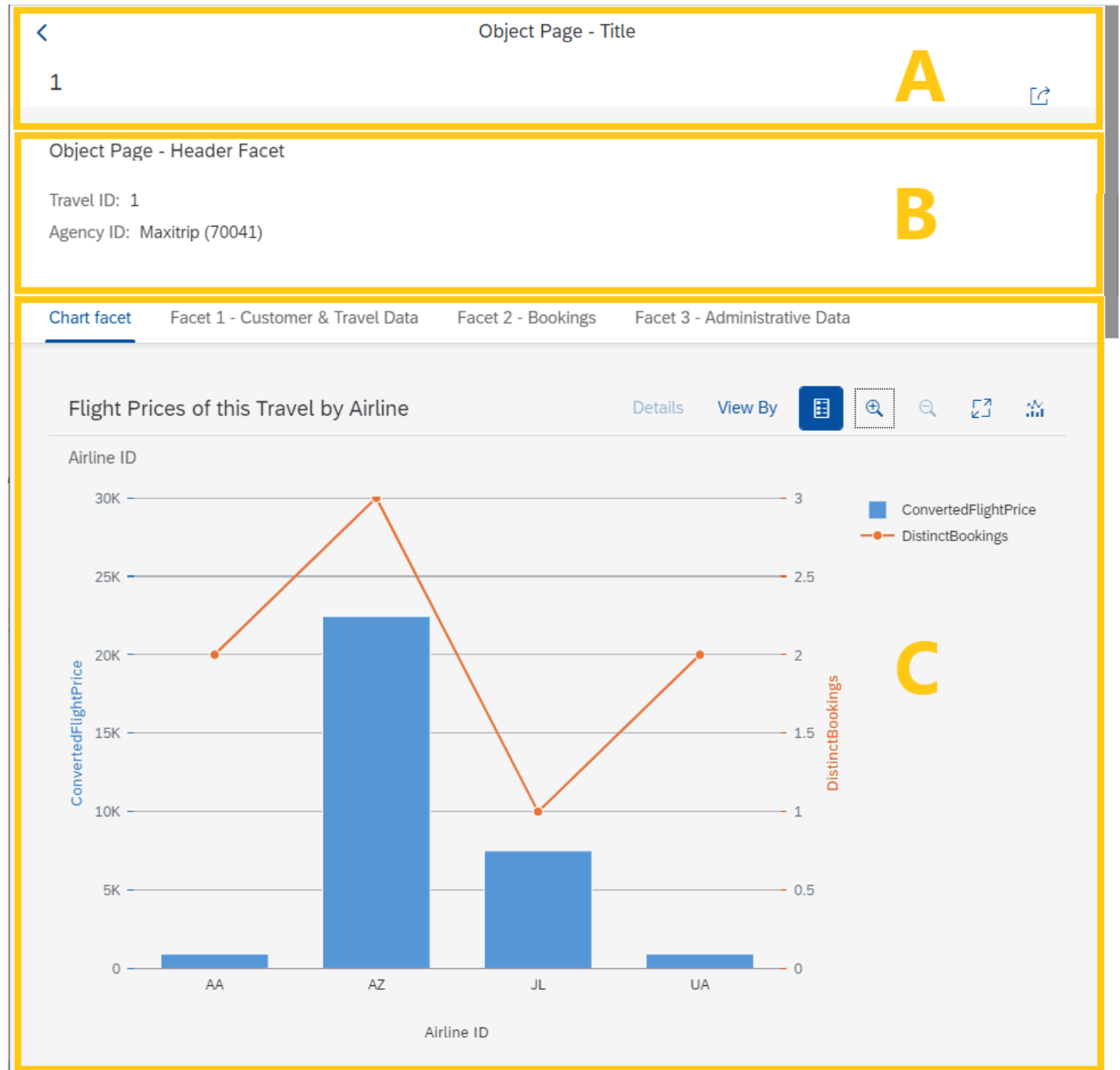
A – List Report Header

B – List Report Table Header

To navigate from the list report to the object page of an entry, click anywhere on the row of that entry.

Object Page

The object page is mostly used to show more details about one entry of the dataset. Again, you can choose which details shall be displayed on the object page.



A – Object Page Title Section

B – Header Facet

C – Facet Navigation Tab and Chart Facet

Related Information

[Fiori Element App Preview \(FEAP\)](#)

Fields on the UI

Use UI annotations to rearrange, label and manipulate fields on your UI.

Fields are elements from the CDS view that are displayed on the UI screen. Fields can be table columns of list reports or single items within facets on the object page, such as “Customer ID: ...”. In other words: whenever you annotate a CDS element to be displayed on the UI, it will appear on the UI as a field.

In this chapter, you will be guided through the following tasks:

- [Overwriting Default Field Labels](#)
- [Positioning Fields](#)
- [Grouping Fields](#)
- [Hiding Fields on the Object Page](#)
- [Adding Selection Fields](#)

Overwriting Default Field Labels

How to use UI annotations in CDS to overwrite default field labels on your UI.

Quick Info

Annotation Syntax	@UI.identification: [{label: '<new label name>'}]
Description	Defines the displayed field label. Overrules default labels of identification derived from CDS annotation <code>@EndUserText . label</code> or from data elements in the ABAP dictionary.
Use	To be placed in the element list section, preceding each respective element.
Related Topics	Fields on the UI CDS Annotations

Annotation Syntax	@UI.lineItem: [{label: '<new label name>'}]
Description	Defines the displayed field label. Overrules default labels of columns of a list report derived from CDS annotation <code>@EndUserText . label</code> or from data elements in the ABAP dictionary.
Use	To be placed in the element list section, preceding each respective element.
Related Topics	Fields on the UI CDS Annotations

Preview on Fiori UI

Facet 3 - Administrative Data

Travel ID:

1

Start of Travel:

Nov 2, 2019

Comment:

Vacation

Agency:

Maxitrip (70041)

End of Travel:

Aug 30, 2020

Customer:

Ryan (594)

Total Price:

30,096.41 USD

Labeled UI Identification Fields on the Object Page

List Report - Table Header (7,462) Standard ⌵ ⚙

Travel ID	Agency	Customer	Start of Travel	End of Travel	Travel Status	
☐ 1	Maxitrip (70041)	Ryan (594)	Nov 2, 2019	Aug 30, 2020	B	>
☐ 2	Hot Socks Travel (70007)	Prinz (608)	Nov 2, 2019	Nov 2, 2019	B	>

Labeled UI Line Item Fields on the List Report

Context and Use

The labels of exposed CDS elements are derived by default from the data element labels in the ABAP dictionary, or, if defined, from the CDS annotation `@EndUserText.label`. You can overwrite both, data element labels and labels defined by `@EndUserText.label`, with the subannotation `label` on the UI. Labels of UI annotations are only used in UI contexts, whereas labels from the ABAP dictionary or `@EndUserText.label` are used in the service document of any service. The subannotation `label` can be used with several UI annotations, e.g. `@UI.identification` or `@UI.LineItem` (see Quick Info above or [CDS Annotations](#)).

Place the annotation within the element list section, right before the respective element that you want to label.

Code Snippet

In this example, we use the annotation `@UI.identification.label` to label the fields that are displayed on the object page in the facet with type `#IDENTIFICATION_REFERENCE` as can be seen in the first figure of the preview above. Similarly we label the fields (i.e. columns) of the list report with the annotation `@UI.LineItem.label` as can be seen in the second figure of the preview above.

Find the Metadata Extension files for the Travel Scenario in the [Complete UI Annotations Example Code](#).

i Expand the following code sample to view the source code to annotate view `Z_C_TRAVEL_R`.

Sample Code

```
...
annotate view Z_C_TRAVEL_R with
{
  ...
  // Body Facets (Object Page)
  ...
  // Facet 3
  { id:          'Facet3-ID',
    purpose:     #STANDARD,
    type:        #IDENTIFICATION_REFERENCE, // Refers to elements annotated with
    label:       'Facet 3 - Overview and Comments',
    position:    30 }
```

```

    ]

    // ELEMENT LIST

    // No label defined.
    TravelID;

    @UI: { lineItem:      [ { label: 'Agency' } ],
          identification: [ { label: 'Agency' } ]
        }
    AgencyID;

    @UI: { lineItem:      [ { label: 'Customer' } ],
          identification: [ { label: 'Customer' } ]
        }
    CustomerID;

    @UI: { lineItem:      [ { label: 'Start of Travel' } ],
          identification: [ { label: 'Start of Travel' } ]
        }
    BeginDate;

    @UI: { lineItem:      [ { label: 'End of Travel' } ],
          identification: [ { label: 'End of Travel' } ]
        }
    EndDate;

    @UI: { hidden: true }
    BookingFee;

    // No label defined.
    TotalPrice;

    // Label only defined for identification field
    @UI: { identification: [ { label: 'Comment' } ]
    Memo;

    // Label only defined for line item field
    @UI: { lineItem:      [ { label: 'Set to Booked' } ]
    Status;
}

```

Positioning Fields

How to rearrange fields using UI annotations in CDS.

Quick Info

Annotation Syntax	@UI.identification: [{position: <value>}]
Description	Sorts fields in the order of <code>position</code> values.
Use	To be placed in the element list section of the code, preceding each respective element.
Related Topics	Fields on the UI CDS Annotations

Annotation Syntax	@UI.lineItem: [{ position: <value> }]
Description	Sorts fields (i.e. table columns) in the order of <code>'position'</code> values.
Use	To be placed in the element list section of the code, preceding each respective element.

Annotation Syntax	@UI.lineItem: [{ position: <value> }]
Related Topics	Fields on the UI CDS Annotations

Preview on Fiori UI

Facet 3 - Administrative Data		
Travel ID: 1	Start of Travel: Nov 2, 2019	Comment: Vacation
Agency: Maxitrip (70041)	End of Travel: Aug 30, 2020	
Customer: Ryan (594)	Total Price: 30,096.41 USD	

UI Identification Fields sorted in order of the position values on the Object Page

List Report - Table Header (7,462) Standard ▼						
Travel ID	Agency	Customer	Start of Travel	End of Travel	Travel Status	
1	Maxitrip (70041)	Ryan (594)	Nov 2, 2019	Aug 30, 2020	B	>
2	Hot Socks Travel (70007)	Prinz (608)	Nov 2, 2019	Nov 2, 2019	B	>

UI Line Item Fields sorted in order of the position values on the List Report

Context and Use

Place the annotation in the element list section of your CDS Data Definition or Metadata Extension, directly preceding the respective element. The value for `position` is relative, meaning that the order of the elements on the UI is defined by the relative order of the values for `position` of each element.

We recommend that you leave gaps between the respective values for `position`. This will make it easier to add an element between two other elements retroactively, as you will not need to change the values of `position` for all other elements.

The subannotation `position` can be used in a similar way with the following annotations:

UI.facet
 UI.fieldGroup
 UI.identification
 UI.lineItem
 UI.selectionField
 UI.statusInfo

You can find more details about this in [CDS Annotations](#).

Code Snippet

In this example we use `@UI.identification.position` to define the order of the identification fields in the facet with type `#IDENTIFICATION_REFERENCE` on the object page. Similarly we define the order of the fields (i.e. columns) of the list report with the annotation `@UI.lineItem.position`.

You can find the Metadata Extension files for the Travel Scenario in the [Complete UI Annotations Example Code](#).

i Expand the following code sample to view the source code for the annotations of the view **Z_C_TRAVEL_R**.

```
...
annotate view Z_C_TRAVEL_R with

{
  ...
  // Body Facets (Object Page)
  ...
    // Facet 3

    { id:          'Facet3-ID',
      purpose:      #STANDARD,
      type:         #IDENTIFICATION_REFERENCE, // Refers to elements annotated with '@
      label:        'Facet 3 - Overview and Comments',
      position:     30 }
    ]

  // ELEMENT LIST

  @UI: { listItem:   [ { position: 10 } ],
        identification: [ { position: 10 } ]
        }
  TravelID;

  @UI: { listItem:   [ { position: 20 } ],
        identification: [ { position: 20 } ]
        }
  AgencyID;

  @UI: { listItem:   [ { position: 30 } ],
        identification: [ { position: 30 } ]
        }
  CustomerID;

  @UI: { listItem:   [ { position: 40 } ],
        identification: [ { position: 40 } ]
        }
  BeginDate;

  @UI: { listItem:   [ { position: 50 } ],
        identification: [ { position: 50 } ]
        }
  EndDate;

  @UI: { hidden: true,
        identification: [ { position: 80 } ] }
  BookingFee;

  @UI: { identification: [ { position: 60 } ] }
  TotalPrice;
```



```
@UI: { identification: [ { position: 70,  
                        label: 'Comment' } ]  
  
  }  
  
Memo;  
  
@UI: { lineItem:      [ { position: 60,  
                        label: 'Set to Booked' } ]  
  
  Status;  
}
```

Grouping Fields

How to use fieldgroups to structure the content on your UI using UI annotations in CDS.

Quick Info

Annotation Syntax	@UI.fieldGroup: [{ qualifier: '<group name>', position: <value>}]
Description	Arranges fields in groups within other sections, like headers or facets.
Use	To be placed in the element section of the code, preceding the respective element. Use <code>qualifier</code> to assign a field to the <code>targetQualifier</code> of a facet. Use <code>position</code> to define the order of the fields.
Related Topics	Using Facets to change the Object Page Layout Fields on the UI CDS Annotations

Preview on Fiori UI

Facet 1 - Customer & Travel Data

Fieldgroup 1 - Customer Data

Street:
Wilhelminentr. 24

Customer ID:
Ryan (594)

First Name:
Illya

Last Name:
Ryan

Title:
Mr.

Postal Code:
69121

City:
Heidelberg

Country Key:
DE

Phone No.:
+49-713-208948

E-Mail Address:
illya.ryan@flight.example.de

Fieldgroup 2 - Travel Data

Travel Status:
B

Description:
Vacation

Total Price:
30,096.41 USD

End Date:
Aug 30, 2020

Customer ID:
Ryan (594)

Starting Date:
Nov 2, 2019

Two Fieldgroups, Collected in One Facet

Context and Use

You can arrange fields to be displayed as a group within a facet.

1. Create a facet type `#FIELDGROUP_REFERENCE` in the element section.
2. Define a `'targetQualifier'` for that facet. `'targetQualifier'` is used as a reference to this facet when defining elements.
3. Place the `UI.fieldGroup` annotation in the element section, directly preceding each element you want to include in the group.
4. Use `'qualifier'` to assign the element to the `'targetQualifier'` of the facet you created before.

Facets can also collect fieldgroup items from other data definitions. In the example below, `'Fieldgroup1-ID'` assigns items from `_Customer` to the parent `Facet1-ID` in the travel CDS view. The items are described in the Metadata Extension for `_Customer`. `'Fieldgroup2-ID'` assigns items from the travel CDS view to the same parent `Facet1-ID`.

You will find more details in the [UI Reference](#).

Code Snippet

Find the Metadata Extension files for the Travel Scenario in the [Complete UI Annotations Example Code](#).

1. Metadata Extension for TRAVEL (Data Definition for list report table)

i *Expand the following code sample to view the source code*

Sample Code

```

annotate view Z_C_TRAVEL_R with
{
  ...
  @UI.facet: [
    ...
    // Body Facets (Object Page)

    // Facet 1 - Parent (collection) for Fieldgroup 1 and Fieldgroup 2
    { id:          'Facet1-ID',
      type:        #COLLECTION,
      label:       'Facet 1 - Customer & Travel Data',
      position:    10 },

    // Facet for Fieldgroup 1 - nested inside Facet 1
    { id:          'Fieldgroup1-ID',
      type:        #FIELDGROUP_REFERENCE,
      label:       'Fieldgroup 1 - Customer Data',
      parentId:    'Facet1-ID', // Places this facet into 'Facet 1'
      targetElement: 'Customer', // Specifies where the data is retrieved from - U
      targetQualifier: 'Fieldgroup1',
      position:    10 },

    // Facet for Fieldgroup 2 - nested inside Facet 1
    { id:          'Fieldgroup2-ID',
      type:        #FIELDGROUP_REFERENCE,
      label:       'Fieldgroup 2 - Travel Data',
      parentId:    'Facet1-ID', // Places this facet into 'Facet 1'
      targetQualifier: 'Fieldgroup2', // No targetElement defined. Default target is th
      position:    20 }
  ]
}

// ELEMENT LIST

```

// FIELDS

```

@UI: { lineItem:      [ { position: 30,
                        importance: #HIGH } ],
      identification: [ { position: 30 } ],
      fieldGroup:     [ { qualifier: 'Fieldgroup2',
                        position: 10 } ] }
CustomerID;

@UI: { lineItem:      [ { position: 40,
                        qualifier: 'Facet2' } ],
      identification: [ { position: 40 } ],
      fieldGroup:     [ { qualifier: 'Fieldgroup2',
                        position: 10 } ] }
BeginDate;

@UI: { lineItem:      [ { position: 41,
                        importance: #MEDIUM,
                        qualifier: 'Facet2' } ],
      identification: [ { position: 41 } ],
      fieldGroup:     [ { qualifier: 'Fieldgroup2',
                        position: 10 } ] }
EndDate;

@UI: { identification: [ { position: 42 } ],
      fieldGroup:     [ { qualifier: 'Fieldgroup2',
                        position: 10 } ] }
BookingFee;

@UI: { identification: [ { position: 43 } ],
      fieldGroup:     [ { qualifier: 'Fieldgroup2',
                        position: 10 } ],
      dataPoint:      { title: 'Total Price',
                        targetValueElement: 'TotalPrice' } }
TotalPrice;

@UI: { identification:[ { position: 45,
                        label: 'Comment' } ],
      fieldGroup:     [ { qualifier: 'Fieldgroup2',
                        position: 10 } ] }
Memo;

@UI: { lineItem:      [ { position: 50,
                        importance: #HIGH },
                        { type: #FOR_ACTION,
                          dataAction: 'set_status_booked',
                          label: 'Set to Booked' } ],
      fieldGroup:     [ { qualifier: 'Fieldgroup2',
                        position: 10 } ] }
Status;
}

```

2. Metadata Extension for _Customer

i Expand the following code sample to view the source code

Sample Code

```

...
annotate view Z_C_CUSTOMER_R with

{
  @UI.fieldGroup: [{qualifier: 'Fieldgroup1', position: 05}]
  Street;

  @UI.fieldGroup: [{qualifier: 'Fieldgroup1', position: 10}]
  CustomerID;

  @UI.fieldGroup: [{qualifier: 'Fieldgroup1', position: 20}]
  FirstName;
}

```

```

@UI.fieldGroup: [{qualifier: 'Fieldgroup1', position: 30}]
LastName;

@UI.fieldGroup: [{qualifier: 'Fieldgroup1', position: 40}]
Title;

@UI.fieldGroup: [{qualifier: 'Fieldgroup1', position: 50}]
PostalCode;

@UI.fieldGroup: [{qualifier: 'Fieldgroup1', position: 60}]
City;

@UI.fieldGroup: [{qualifier: 'Fieldgroup1', position: 70}]
CountryCode;

@UI.fieldGroup: [{qualifier: 'Fieldgroup1', position: 80}]
PhoneNumber;

@UI.fieldGroup: [{qualifier: 'Fieldgroup1', position: 90}]
EmailAddress;

/* Associations */
//Z_UI_CUSTOMER
@UI.fieldGroup: [{qualifier: 'Fieldgroup1', position: 100}]
_Country;
}

```

Prioritizing Fields

How to prioritize fields on the UI

Quick Info

Annotation Syntax	@UI.lineItem: { importance: [#LOW, #MEDIUM, #HIGH] }
Description	Expresses importance of a field (i.e. table column). The value of importance is used to determine which fields are dynamically hidden if the UI is displayed on a small screen.
Use	To be placed in the element section of the code, preceding the respective element.
Related Topics	Fields on the UI CDS Annotations

Annotation Syntax	@UI.identification: { importance: [#LOW, #MEDIUM, #HIGH] }
Description	Expresses importance of an identification field. The value of importance is used to determine which fields are dynamically hidden if the UI is displayed on a small screen.
Use	To be placed in the element section of the code, preceding the respective element.
Related Topics	Fields on the UI CDS Annotations

Context and Use

To make sure that the most relevant info is displayed even on small screens, you can define the priority of fields on the UI. The assigned priority is recognized by Fiori Elements and used to decide which fields will be dynamically hidden in case the UI is displayed on a smaller screen, e.g. a smartphone.

Code Snippet

In this example we use the annotation `@UI.lineItem.importance` to make sure that the `TravelID`, `AgencyID` and `CustomerID` columns on the list report have high priority.

You can find the Metadata Extension files for the Travel Scenario in the [Complete UI Annotations Example Code](#).

```
...
annotate view Z_C_TRAVEL_R with

{
  ...
  @UI: { lineItem:      [ { importance: #HIGH } ]
    TravelID;

  @UI: { lineItem:      [ { importance: #HIGH } ]
    AgencyID;

  @UI: { lineItem:      [ { importance: #HIGH } ]

    CustomerID;
  ...
}
```

Adding Selection Fields

How to add selection fields for filtering lists or tables on your UI using UI annotations in CDS.

Quick Info

Annotation Syntax	<code>@UI.selectionField: [{ position: <value> }]</code>
Description	Adds selection fields for the corresponding columns in the list report. Useful for filtering long lists or tables for specific values. Not to be confused with search fields (see: Adding Search Capabilities).
Use	To be placed in the element list section of the code, preceding the respective element.
Related Topics	Adding Search Capabilities Fields on the UI CDS Annotations

Preview on Fiori UI

Standard * 

Travel ID: 	Agency ID: 	Customer ID: Ryan (594) x	Adapt Filters (1) 
------------------------------------	------------------------------------	---	---

List Report - Table Header (5) Standard 

Travel ID	Agency	Customer	Start of Travel	End of Travel	Travel Status	
☐ 1	Maxitrip (70041)	Ryan (594)	Nov 2, 2019	Aug 30, 2020	B	>
☐ 547	Rainy, Stormy, Cloudy (70039)	Ryan (594)	Aug 26, 2020	Aug 27, 2020	P	>
☐ 1427	Not Only By Bike (70050)	Ryan (594)	Nov 3, 2019	Nov 3, 2019	P	>
☐ 2209	All British Air Planes (70047)	Ryan (594)	Aug 28, 2020	Aug 28, 2020	P	>
☐ 3794	Bavarian Castle (70015)	Ryan (594)	Aug 28, 2020	Aug 28, 2020	N	>

Selection Field with Applied Filter "Customer ID = 594"

Context and Use

The annotation displays selection fields for the corresponding columns of the list report. The subannotation `@UI.selectionField.position` defines the order of the selection fields on the UI. Labels of selection fields are derived from the data elements in the ABAP Dictionary. If you want to use different labels on the UI, define labels with the `@EndUserText.label` annotation. Selection fields enable the user to filter a long list or table for specific values, such as Customer ID and/or Agency ID (see example).

Place the annotation in the element list section of your CDS Data Definition or Metadata Extension, preceding each respective element that you want to filter your table for.

You will find more details in the [CDS Annotations](#).

Code Snippet

Find the Metadata Extension files for the Travel Scenario in the [Complete UI Annotations Example Code](#).

```

annotate view Z_C_TRAVEL_R
{
  ...
  // FIELDS

  @UI.lineItem: [ { position: 10 } ]
  @UI.selectionField: [ { position: 5 } ]
  TravelID;

  @UI: { lineItem: [ { position: 20 } ],
    selectionField: [ { position: 10 } ] }
  AgencyID;

  @UI: { lineItem: [ { position: 30 } ],
    selectionField: [ { position: 15 } ] }
  CustomerID;
  ...
}

```

Tables

How to use UI annotations in CDS to design the layout of lists and tables on the UI.

The following topics apply to tables both on the list report and on the object page. Note that list reports always use tables, while object pages may integrate tables.

Find more information about Fiori UI floorplans on [Introduction to SAP Fiori Elements](#).

You will be guided through the following tasks:

- [Defining the Table Header of a List Report](#)
- [Labeling Table Columns](#)
- [Positioning Table Columns](#)

i Note

The table header of a table can only be defined on a list report. Tables on object pages do not have a table header.

Defining the Table Header of a List Report

How to define the table header of a list report of your UI using UI annotations in CDS.

Quick Info

Annotation Syntax	@UI.headerInfo: { typeNamePlural: '<List Report - Table Header>' }
Description	Defines the table header of a list report.
Use	To be placed in the header section of the code.
Related Topics	Defining the Title Section of an Object Page CDS Annotations

Preview on Fiori UI

Standard * ▾

Travel ID:

Agency ID:

Customer ID:

Adapt Filters

Go

List Report - Table Header (7,597) Standard ▾

Travel ID	Agency	Customer	Start of Travel	End of Travel	Travel Status	
1	Maxitrip (70041)	Ryan (594)	Nov 2, 2019	Aug 30, 2020	B	>
2	Hot Socks Travel (70007)	Prinz (608)	Nov 2, 2019	Nov 2, 2019	B	>
3	Hendrik's (70046)	Detemple (93)	Nov 2, 2019	Aug 30, 2020	P	>

Defined Table Header of the List Report Page

Context and Use

Place the annotation `UI.headerInfo.typeNamePlural` in the header section of your CDS Data Definition or Metadata Extension.

Code Snippet

Find the Metadata Extension files for the Travel Scenario in the [Complete UI Annotations Example Code](#).

```
// METADATA EXTENSION - HEADER SECTION
@Metadata.layer: #CORE
@UI: { headerInfo: { typeNamePlural: 'List Report - Table Header' } }

annotate view Z_C_TRAVEL_R
...
```

Object Page

Use UI annotations to influence the design of the object page on your UI.

The object page is mostly used to show details about one entry of a data table. On an object page you can gather information for one entry from different CDS views in different layouts. Click on a row of the list report page to navigate to the object page of the corresponding entry.

Find more information about the object page in [Defining CDS Annotations for Metadata-Driven UIs](#) or in the [Introduction to SAP Fiori Elements](#).

You will be guided through the following tasks:

- [Defining the Title Section of an Object Page](#)
- [Using Facets to change the Object Page Layout](#)
- [Hiding Fields on the Object Page](#)
- [Visualizing Data with Charts](#)
- [Labeling Fields on the Object Page](#)
- [Positioning Fields on the Object Page](#)

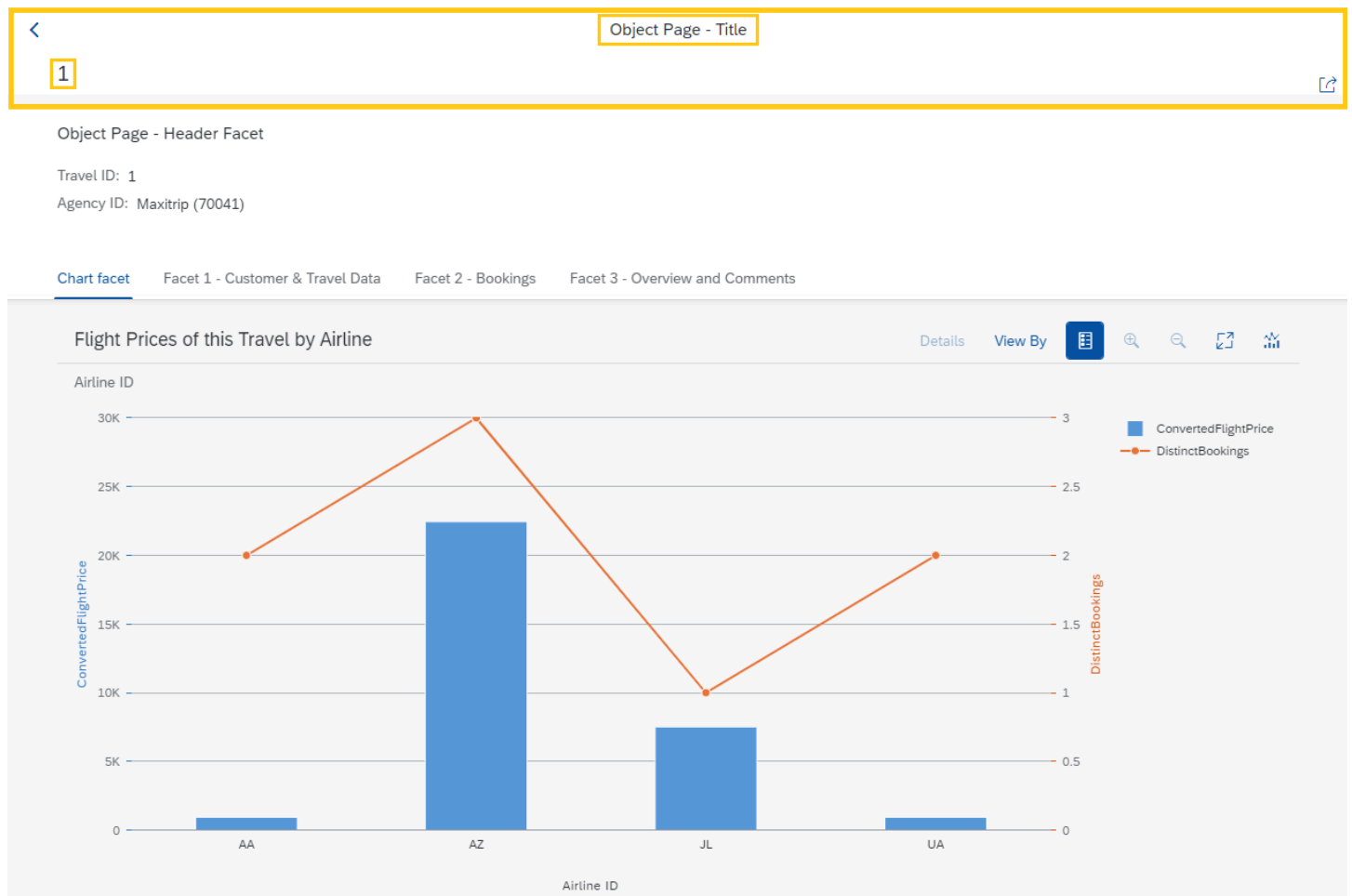
Defining the Title Section of an Object Page

How to use UI annotations in CDS to define the title of the object page of your UI.

Quick Info

Annotation Syntax	<code>@UI.headerInfo: { typeName: '<Object Page Title>', title: { value: '<field>' } }</code>
Description	Defines the title section of an object page.
Use	To be placed in the header section of the code.
Related Topics	Defining the Table Header of a List Report CDS Annotations

Preview on Fiori UI



Title Section of the Object page comprising of a title and a value

Context and Use

Place the annotations `@UI.headerInfo.typeName` to define the Object Page title and `@UI.headerInfo.title.value` to define the Object Page title value in the header section of your CDS Data Definition or Metadata Extension.

i Note

You can display another value in the title section with the annotation `@UI.headerInfo.description.value`.

Code Snippet

Find the Metadata Extension files for the Travel Scenario in the [Complete UI Annotations Example Code](#).

```
// METADATA EXTENSION - HEADER SECTION
@Metadata.layer: #CORE
@UI: { headerInfo: { typeName: 'Object Page - Title' },
        title: { value: 'TravelID' }
      }
}
annotate view Z_C_TRAVEL_R
...
```

Using Facets to change the Object Page Layout

Quick Info

Annotation Syntax	<code>@UI.facet: [{ id: '...', label: '...', parentId: '...', position: <value>, purpose: #..., qualifier: '...', targetElement: '...', targetQualifier: '...', type: #..., hidden: true }]</code>
Description	Defines facets for the object page.
Use	To be placed in the facet definition section of the code.
Related Topics	Defining CDS Annotations for Metadata-Driven UIs Complete UI Annotations Example Code CDS Annotations

Preview on Fiori UI

1



Object Page - Header Facet

Travel ID: 1

Agency ID: Maxitrip (70041)

1

Chart facet

Facet 1 - Customer & Travel Data

Facet 2 - Bookings

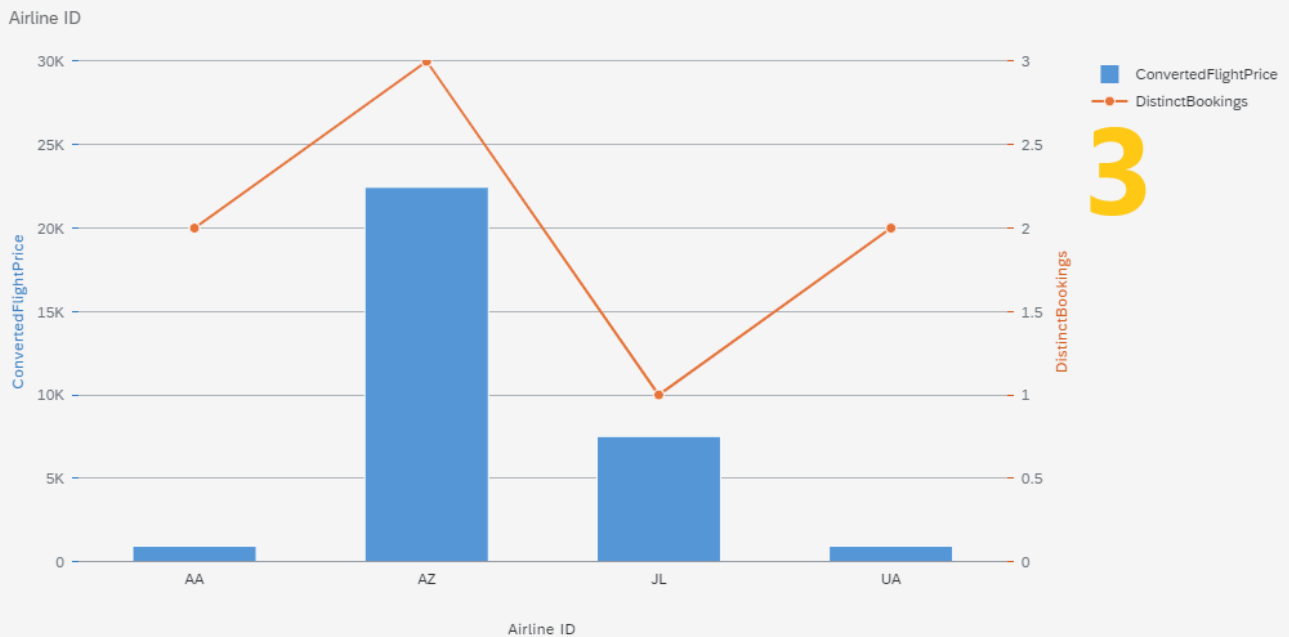
Facet 3 - Overview and Comments

2

Flight Prices of this Travel by Airline

Details

View By



3

Facet 1 - Customer & Travel Data

Fieldgroup 1 - Customer Data

Street:
Wilhelminenstr. 24Customer ID:
Ryan (594)First Name:
IllyaLast Name:
RyanTitle:
Mr.Postal Code:
69121City:
HeidelbergCountry Key:
DEPhone No.:
+49-713-208948E-Mail Address:
illya.ryan@flight.example.de

Fieldgroup 2 - Travel Data

Travel Status:
BDescription:
VacationTotal Price:
30,096.41 USDEnd Date:
Aug 30, 2020Customer ID:
Ryan (594)Starting Date:
Nov 2, 2019

4

Facet 2 - Bookings

	Booking No.	Date of Booking	Customer	Airline	Conne...	Date of flight	Flight Price	Currency		
	1	Oct 16, 2019	594	UA	1537	Nov 2, 2019	438.00 USD	USD	>	
	2	Nov 2, 2019	594	AA	322	Nov 4, 2019	438.00 USD	USD	>	

5

Facet 3 - Overview and Comments

6

Travel ID: 1	Start of Travel: Nov 2, 2019	Comment: Vacation
Agency: Maxitrip (70041)	End of Travel: Aug 30, 2020	
Customer: Ryan (594)	Total Price: 30,096.41 USD	

Different Types of Facets on the Object Page

- 1 - Header Facet (purpose: #HEADER)
- 2 - Facet Navigation Tabs
- 3 - Chart facet (type: #CHART_REFERENCE)
- 4 - Parent Facet (type: #COLLECTION), Containing Two Child Facets, namely Fieldgroup 1 and Fieldgroup 2 (type: #FIELDGROUP_REFERENCE)
- 5 - Facet with Table (type: #LINEITEM_REFERENCE)
- 6 - Facet with Array (type: #IDENTIFICATION_REFERENCE)

Context and Use

Facets are useful for organizing the basic layout of a page, on the object page for example. `@UI . facet` can be refined with a variety of subannotations. Place the annotation in the facet section of your CDS Data Definition or Metadata Extension.

Purpose and Type of UI Facets

The annotation `@UI . facet . purpose` defines whether the facet is a header or a standard facet. On the other hand, the annotation `@UI . facet . type` defines the format of a facet and the types of fields that will be displayed in a facet.

<code>purpose: #HEADER</code>	Displays the facet and its content in the header section of the object page.
<code>purpose: #STANDARD</code>	Use this for standard facets. Can be omitted in most cases.
<code>type: #COLLECTION</code>	The facet serves as parent (or container) for other facets. Child facets reference the parent facet by 'parentId'.
<code>type: #FIELDGROUP_REFERENCE</code>	Reference to elements assigned to a fieldgroup.
<code>type: #IDENTIFICATION_REFERENCE</code>	Reference to elements annotated with <code>UI . identification</code> . The facet will display the referenced elements in a list format.
<code>type: #LINEITEM_REFERENCE</code>	Reference to elements annotated with <code>UI . lineItem</code> . The facet will display the referenced elements in a table format.
<code>type: #CHART_REFERENCE</code>	Reference to charts annotated with <code>UI . chart</code> annotation. The facet will display the referenced charts. For more information, see Visualizing Data with Charts
<code>hidden: true</code>	Used to hide facets from a page view. The fields are not displayed but still available for calculations for example.

Connecting Parent and Child Facets

Parent facets are annotated with `type: #COLLECTION`. To nest another facet into a parent facet, you must use the annotations below.

id: '...'	Unique identifier. Needed when child facets are nested into #COLLECTION facets. The value of id is referenced by parentID.
parentID: '...'	Needed for child facets to be nested into #COLLECTION facets.

Connecting Facets to Fieldgroups and Other Targets

To retrieve data from a source, you must define the `targetElement` in the facet. If there is no `targetElement` defined, the default target is the entity in which the facet is defined. To nest field into a fieldgroup, you must define the `targetQualifier` of the facet. The value of `targetQualifier` is referenced by fields with the annotation `@UI.fieldgroup.qualifier`.

targetElement: '...'	Needed for retrieving data from associated Data Definitions, e.g. <code>_Customer</code>
targetQualifier: '...'	Unique identifier. Needed for elements to be nested into a fieldgroup.

Facet Header and Position

To further customize the layout, you can define the `label` and `position` of each facet. By default, the position of facets will be the order in which they are defined in the source code.

label: '...'	Label displayed in the facet header and navigation tabs.
position: <value>	Sorts facets in the order of the 'position' values. For more information, see Positioning Fields .

For more details, see [CDS Annotations](#).

Code Snippet

Find the Metadata Extension files for the Travel Scenario in the [Complete UI Annotations Example Code](#).

i Expand the following code sample to view the source code to annotate the view `Z_UI_TRAVEL`.

 Sample Code

```
...
annotate view Z_UI_TRAVEL with
{
  //FACET SECTION

  @UI.facet: [

    // Header Facet (Object Page):

    { id:           'HeaderFacet',
      purpose:      #HEADER,
      type:         #FIELDGROUP_REFERENCE,
      label:        'Object Page - Header Facet',
      targetQualifier: 'HeaderItems', // Refers to lineItems with @UI.fieldGroup: [{qua
      position:     10 },

    // Body Facets (Object Page)

    // Facet for chart - charts can only be displayed in facets
```

```

    { id:          'Chart1-ID', // Facet for our Chart with type "#CHART_REFERENCE"
      purpose:     #STANDARD,
      type:        #CHART_REFERENCE,
      label:       'Chart facet',
      targetQualifier: 'Chart1',
      targetElement: '_Booking', // Specifies where the data is retrieved from
      position:     1
    },

    // Facet 1 - Parent (collection) for Fieldgroup 1 and Fieldgroup 2

    { id:          'Facet1-ID',
      type:        #COLLECTION,
      label:       'Facet 1 - Customer & Travel Data',
      position:     10 },

    // Facet for Fieldgroup 1 - nested inside Facet 1

    { id:          'Fieldgroup1-ID',
      type:        #FIELDGROUP_REFERENCE,
      label:       'Fieldgroup 1 - Customer Data',
      parentId:    'Facet1-ID', // Places this facet into 'Facet 1'
      targetElement: '_Customer', // Specifies where the data is retrieved from - UI
      targetQualifier: 'Fieldgroup1',
      position:     10 },

    // Facet for Fieldgroup 2 - nested inside Facet 1

    { id:          'Fieldgroup2-ID',
      type:        #FIELDGROUP_REFERENCE,
      label:       'Fieldgroup 2 - Travel Data',
      parentId:    'Facet1-ID', // Places this facet into 'Facet 1'
      targetQualifier: 'Fieldgroup2', // No targetElement defined. Default target is th
      position:     20 },

    // Facet 2

    { id:          'Facet2-ID',
      type:        #LINEITEM_REFERENCE, // Facet shows the referenced items in a li
      label:       'Facet 2 - Bookings',
      targetQualifier: 'Facet2',
      position:     20,
      targetElement: '_Booking'},

    // Facet 3

    { id:          'Facet3-ID',
      purpose:     #STANDARD,
      type:        #IDENTIFICATION_REFERENCE, // Refers to elements annotated with
      label:       'Facet 3 - Administration Data',
      position:     30 }
  ]

  // ELEMENT LIST

  @UI: { identification: [ { position: 10 } ],
        fieldGroup:    [ { qualifier: 'HeaderItems',
                           position: 10 } ] }
  TravelID;

  @UI: { identification: [ { position: 20} ],
        fieldGroup:    [ { qualifier: 'HeaderItems',
                           position: 20 } ] }
  AgencyID;

  @UI: { identification: [ { position: 30 } ],
        fieldGroup:    [ { qualifier: 'Fieldgroup2',
                           position: 10 } ] }
  CustomerID;

  @UI: { lineItem:      [ { position: 40,
                           qualifier: 'Facet2' } ],
        identification: [ { position: 40 } ] ,

```

```

        fieldGroup:      [ { qualifier: 'Fieldgroup2',
                             position: 10 } ] }
BeginDate;

@UI: { lineItem:      [ { position: 41,
                          importance: #MEDIUM,
                          qualifier: 'Facet2' } ],
      identification: [ { position: 41 } ],
      fieldGroup:     [ { qualifier: 'Fieldgroup2',
                          position: 10 } ] }

EndDate;

@UI: { hidden: true,
      identification: [ { position: 42 } ] ,
      fieldGroup:     [ { qualifier: 'Fieldgroup2',
                          position: 10 } ] }

BookingFee;

@UI: { identification: [ { position: 43 } ] ,
      fieldGroup:     [ { qualifier: 'Fieldgroup2',
                          position: 10 } ] }

TotalPrice;

@UI: { identification:[ { position: 45,
                          label: 'Comment' } ] ,
      fieldGroup:     [ { qualifier: 'Fieldgroup2',
                          position: 10 } ] }

Memo;

@UI: { fieldGroup:     [ { qualifier: 'Fieldgroup2',
                          position: 10 } ] }

Status;
}

```

Hiding Fields on the Object Page

How to use UI annotations in CDS to hide fields from the UI without deleting them.

Quick Info

Annotation Syntax	@UI.hidden: true
Description	Hides fields from the object page without deleting them.
Use	To be placed in the element list section of the code, preceding the respective element.
Related Topics	Fields on the UI CDS Annotations

Context and Use

In general, all fields that are exposed by the OData service are available to the client, regardless of whether the fields are exposed explicitly using UI annotations. To enable end-user personalization, the client may offer the possibility to add fields that are hidden for UI consumption, for example in facets on the object page. Use the annotation `@UI.hidden` to prevent fields from being displayed on the UI and in the personalization dialog, but leaving the field available for a WebAPI client.

You can also use this annotation if, for example, a CDS view contains technical keys, for example GUIDs, that must be exposed to the OData service to work. These keys are usually not supposed to be displayed on the UI. `@UI.hidden: true` is also useful if fields are required in calculations, but not supposed to be displayed on the UI. Hiding fields is especially useful if you want to hide fields dynamically, e.g. if you want to show fields only when they have positive values.

Note that you can annotate fields of list reports with `@UI.hidden: true` as well. However, this will only deselect the annotated fields as columns, not hide them completely. It is effectively the same as if you hadn't annotated the element with `@UI.LineItem` in the first place.

Place the annotation in the element list section of your CDS Data Definition or Metadata Extension, directly preceding the respective element.

For more details, see [CDS Annotations](#).

Code Snippet

In this example, the Booking Fee shall not be displayed to the user, but might be necessary to calculate the total price.

Find the Metadata Extension files for the Travel Scenario in the [Complete UI Annotations Example Code](#).

```
annotate view ...

{
  ...
  // ELEMENT LIST
  ...
  @UI: { hidden: true }
  BookingFee;

  @UI: { identification: [ { position: 43 } ] ,
    fieldGroup:      [ { qualifier: 'Fieldgroup2',
                        position: 10 } ] }

  TotalPrice;
  ...
}
```

Hiding Fields Dynamically on the Object Page

How to use the `hidden` annotation to hide fields dynamically on the object page of your application.

Quick Info

Annotation Syntax	<code>@UI.identification: [{hidden:#('virtual Element')}]</code>
Description	You can use the <code>UI.identification</code> annotation in combination with a virtual element to enable or disable the visibility of a field on the object page.
Use	You can define a condition for the field visibility with a virtual element. Depending on the condition, the field is visible or not.
Related Topics	• Modeling Virtual Elements • Implementing the Calculation of Virtual Elements • CDS Annotations

Context and Use

You can use the `hidden` annotation in combination with a virtual element to show or hide fields dynamically on the UI depending on a boolean value. If the return value corresponds to `true`, the respective field is hidden and if the value corresponds to `false` the respective field is shown on the user interface. You require the following steps to enable dynamic field hiding with a virtual element:

1. Define a virtual element in the behavior definition projection. For more general information about how to implement a virtual element, see [Using Virtual Elements in CDS Projection Views](#).
2. Create an ABAP class to determine the return value for the virtual element.
3. Annotate the field you want to dynamically hide the respective field on the object page.

In the following you can see sample snippets of all components and below you can find an a sample implementation for a particular use case.

Code Snippet

In the following definition, the extracted field is checked for its numeric value, if the value of `FieldToCheck` is greater than 1000 then the method returns `abap_true` and the `Field` is hidden on the object page. If the contained value is less then 1000, the methord returns `abap_false` and the `Field` is displayed on the user interface. You can check any value against any condition and determine the visibility of the respective field dynamically.

Example Definition: /DMO/Projection_View

```
define root view entity /DMO/Projection_View
  as projection on /DMO/Root_View
{
    @UI: {
        lineItem:      [ { position: 40, importance: #MEDIUM } ],
        identification: [ { position: 40, hidden: #(testField)} ] }
    Field as Field,

    @UI: {
        identification:[ { position: 50 } ] }
        FieldToCheck as FieldToCheck

    @ObjectModel.virtualElementCalculatedBy: 'ABAP:Z_ShowField'
    virtual testField :abap_boolean,
}
```

Example Definition: CLASS Z_ShowField

```
CLASS Z_ShowField DEFINITION
  PUBLIC

  FINAL
  CREATE PUBLIC .

  PUBLIC SECTION.

  INTERFACES IF_SADL_EXIT_CALC_ELEMENT_READ.

  ENDCLASS.

CLASS Z_ShowField IMPLEMENTATION.
```

```
METHOD IF_SADL_EXIT_CALC_ELEMENT_READ~CALCULATE.
```

```
* Check the field for your condition and return either abap_true or abap_false for the
* virtual field.
```

```
DATA LT_ORIGINAL_DATA TYPE STANDARD TABLE OF Z_FD_C_Travel_Processor_M WITH DEFAULT KEY.
LT_ORIGINAL_DATA = CORRESPONDING #( IT_ORIGINAL_DATA ).
```

```
LOOP AT LT_ORIGINAL_DATA ASSIGNING FIELD-SYMBOL(<FS_ORIGINAL_DATA>).
  <FS_ORIGINAL_DATA>-testField = COND ABAP_BOOLEAN( WHEN <FS_ORIGINAL_DATA>-FieldToCheck > :
ENDLOOP.
```

```
CT_CALCULATED_DATA = CORRESPONDING #( LT_ORIGINAL_DATA ).
```

```
ENDMETHOD.
```

```
METHOD IF_SADL_EXIT_CALC_ELEMENT_READ~GET_CALCULATION_INFO.
```

```
* Extract the field you want the field visibility to depend on
```

```
*
```

```
IF IV_ENTITY <> '/DMO/Projection_View'.
  RAISE EXCEPTION TYPE /dmo/cx_virtual_elements
  EXPORTING
    TEXTID = /dmo/cx_virtual_elements=>ENTITY_NOT_KNOWN
    ENTITY = IV_ENTITY.
```

```
ENDIF.
```

```
LOOP AT IT_REQUESTED_CALC_ELEMENTS ASSIGNING FIELD-SYMBOL(<FS_CALC_ELEMENT>).
```

```
  CASE <FS_CALC_ELEMENT>.
```

```
    WHEN 'testField'.
```

```
      APPEND 'FieldToCheck' TO ET_REQUESTED_ORIG_ELEMENTS.
```

```
      ...
```

```
    ENDCASE.
```

```
  ENDLOOP.
```

```
ENDMETHOD.
```

```
ENDCLASS.
```

Visualizing Data with Charts

Use Charts to visualize data on object pages.

Quick Info

i Note

Modeling charts in CDS view entities is possible for OData V2 services. OData V4 is restricted to micro charts. For further information on micro charts, refer to [Micro Chart Facet](#) .

Annotation Syntax	@UI.facet: [{ id: '...', label: '...', position: <value>, purpose: #STANDARD, targetElement: '...', targetQualifier: '...', type: #CHART_REFERENCE, hidden: '...' }]
Description	Defines a facet for charts on the object page.
Use	To be placed in the facet definition section of the code.

Annotation Syntax	<code>@Aggregation.default: #...</code>
Description	Aggregates data of a given element.
Use	To be placed in the CDS view, above the element that you want to aggregate.

i Note

You must aggregate data, otherwise your chart won't display meaningful data, i.e. will only display data of one instance. For more information on aggregating data, see [RAP Analytical Tables](#).

Annotation Syntax	<code>@UI.Chart: [{ title: '...', qualifier: '...', chartType: '...', dimensions: '...', measures: '...', measureAttributes: [{ measure: '...', role: #... }], dimensionAttributes: [{ dimension: '...', role: #... }], description: '...' }]</code>
Description	Defines the chart itself.
Use	To be placed in the <code>targetElement</code> or its respective metadata extension of the <code>@UI.facet</code> annotation.

Preview on Fiori UI

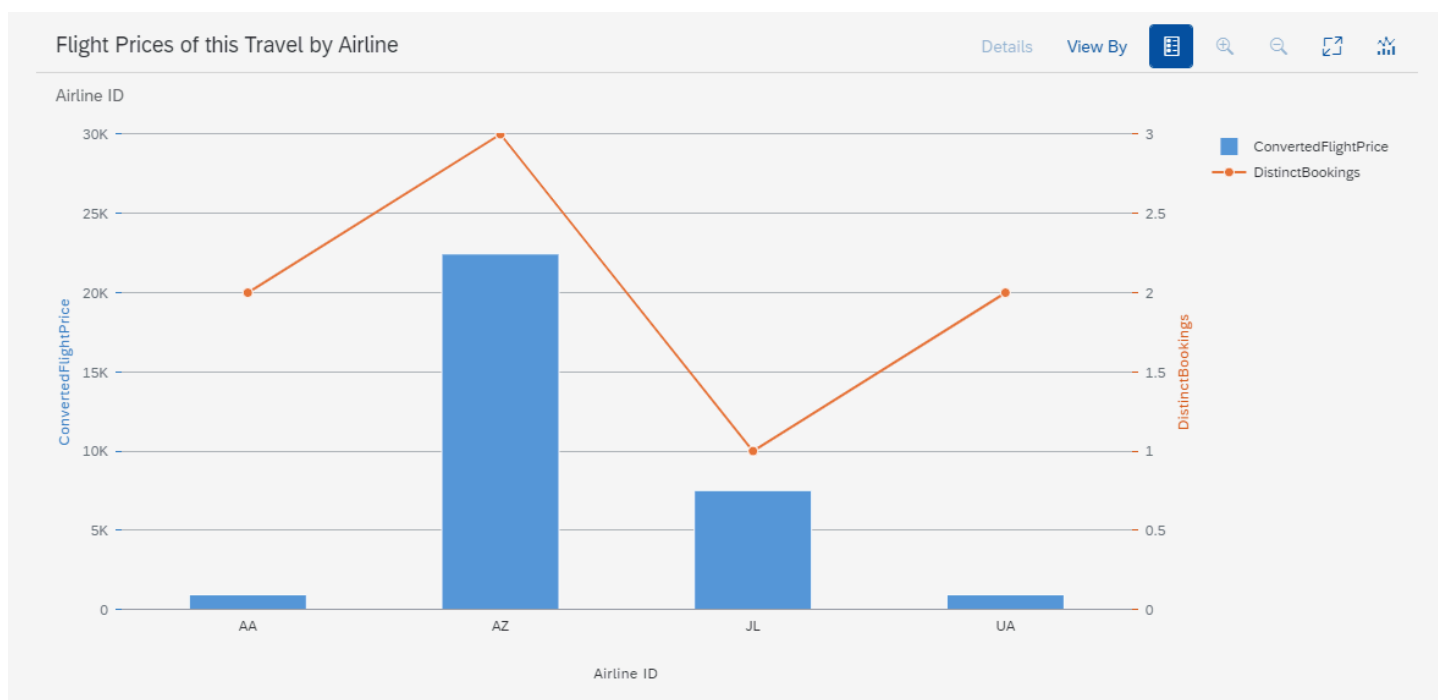


Chart on the Object Page

Use in context

To successfully implement a chart on your object page, you need to:

1. Define a facet for the chart in the parent CDS view
2. Aggregate data with `@Aggregate.default` annotation.
3. Define the chart itself with `@UI.chart` annotation.

Defining a facet for the chart

You must define a facet for your chart to define a spot for the chart on the object page and the data source of the chart. See the sample code below for a better look on how a facet for a chart is defined.

```

annotate view Z_UI_TRAVEL with
{
...
@UI.facet: [
    { id:          'Chart1-ID',
      label:       'Chart facet',
      position:    10,
      purpose:     #STANDARD,
      type:        #CHART_REFERENCE,
      targetQualifier: 'Chart1',
      targetElement:  '_Booking'
    }
]
...
}

```

Subannotation Syntax	Description
<code>id</code>	Defines a unique identifier for the facet, which you can freely choose.
<code>label</code>	Defines the name of the facet. The facet will not have a name on the UI if you don't define a label.
<code>position</code>	Defines the position of the facet.
<code>purpose: #STANDARD</code>	Defines the purpose of the facet. Can be omitted in most cases.
<code>type: #CHART_REFERENCE</code>	Defines the type of the facet. Charts can only be displayed in facets of type: <code>#CHART_REFERENCE</code> .
<code>targetQualifier</code>	Unique identifier. Referenced by subannotation <code>qualifier</code> of <code>@UI.chart</code> annotation in <code>targetElement</code> .
<code>targetElement</code>	Specifies where the data for the chart is retrieved from.

i Note

Charts can only be displayed in facets with `purpose: #STANDARD`. Charts are not supported in facets with `purpose: #HEADER`.

Aggregating Data for the Chart

To aggregate data, you have to use the `@Aggregation.default` annotation. In our example, we use this annotation to aggregate all flight prices and to convert them accordingly. We also use the annotation to aggregate the number of distinct bookings. For more information on aggregating data, see [RAP Analytical Tables](#).

```

define view Z_C_BOOKING_R
as select from /dmo/booking as Booking
...
@Semantics.amount.currencyCode: 'TargetCurrency'
@Aggregation.default: #SUM
CURRENCY_CONVERSION(
  amount          => Booking.flight_price,
  source_currency => Booking.currency_code,
  target_currency => cast( 'EUR' as abap.cuky ),
  exchange_rate_date => cast( '20200420' as abap.dats ),
  error_handling  => 'SET_TO_NULL' )
as ConvertedFlightPrice,

```

```

_Customer.CountryCode as CountryCode,

@Aggregation.referenceElement: ['BookingID']
@Aggregation.default: #COUNT_DISTINCT
cast( 1 as abap.int4 )                                as DistinctBookings,
...

```

i Note

@Aggregation annotations are not permitted in metadata extensions. Therefore you have to place these annotations directly into your CDS view.

Defining the Chart itself

See the sample code below for a better look on how the chart itself is defined. Note that you must place this annotation in the entity previously specified by `targetElement` or its respective metadata extension.

```

@UI.chart: [{
  qualifier: 'Chart1', //refers to targetQualifier defined in chart facet in Z_C_TRAVEL_UI
  title: 'Flight Prices of this Travel by Airline',
  chartType: #COMBINATION_DUAL,
  dimensions: [ 'AirlineID' ],
  measures: [ 'ConvertedFlightPrice', 'DistinctBookings' ],
  measureAttributes: [ {measure: 'ConvertedFlightPrice', role: #AXIS_1},
                       {measure: 'DistinctBookings', role: #AXIS_2} ],
  dimensionAttributes: [ {dimension: 'AirlineID', role: #SERIES} ],
  description: 'Chart shows flight prices of travel by the airlines used per each booking.'
}
]
...
annotate view Z_UI_BOOKING_U with {
...

```

Subannotation Syntax	Description
<code>title</code>	Defines the title of the chart.
<code>qualifier</code>	References the <code>targetQualifier</code> we set in the facet.
<code>chartType</code>	Defines the type of the chart. There is a wide variety of chart types available. For more information on chart types, see Chart Types .
<code>dimensions</code>	Defines the dimensions of the chart. Different chart types require different quantities of dimensions. For more information on dimensions and corresponding chart types, see Chart Types .
<code>measures</code>	Defines the measures of the chart. Different chart types require different quantities of measures. For more information on measures and corresponding chart types, see Chart Types .
<code>measureAttributes</code>	Defines which measure will be shown on which axis. <code>#AXIS_1</code> is the left axis, <code>#AXIS_2</code> is the right axis.
<code>dimensionAttributes</code>	Defines which dimension will be the primary on the UI. <code>#CATEGORY</code> is the primary dimension, <code>#CATEGORY2</code> is the secondary dimension. Note that we do not use <code>dimensionAttributes</code> in this example, because we only use one dimension.
<code>description</code>	Internal description of the chart.

Chart Types

Get an overview of all currently supported chart types within CDS and their required dimension/measures.

Chart Types for Data Visualization

Chart Type	Required Dimensions	Required Measures
COLUMN	Dimensions: 1+	Measures: 1+
COLUMN_STACKED	Axis: x	Axis: y
COLUMN_STACKED_100	Note: AREA, AREA_STACKED and AREA_STACKED_100 have the same result as COLUMN, COLUMN_STACKED and COLUMN_STACKED_100 on the UI.	
AREA		
AREA_STACKED		
AREA_STACKED_100		
LINE		
BAR	Dimensions: 1+	Measures: 1+
BAR_STACKED	Axis: y	Axis: x
BAR_STACKED_100	Note: HORIZONTAL_AREA, HORIZONTAL_AREA_STACKED and HORIZONTAL_AREA_STACKED_100 have the same result as BAR, BAR_STACKED and BAR_STACKED_100 on the UI.	
HORIZONTAL_AREA		
HORIZONTAL_AREA_STACKED		
HORIZONTAL_AREA_100		
PIE	Dimensions: 1+	Measures: 1+
DONUT	Note: Determines by what the sections are segmented	Note: Determines size of segment
SCATTER	Dimensions: 1+	Measures: 2+
BUBBLE	Dimensions: 1+	Measures: 3+
COMBINATION	Dimensions: 1+	Measures: 2+
COMBINATION_STACKED	Axis: x	Axis: y
COMBINATION_DUAL		
COLUMN_DUAL		
COLUMN_STACKED_DUAL		
COLUMN_STACKED_DUAL_100		
COMBINATION_STACKED_DUAL		
LINE_DUAL		
HORIZONTAL_COMBINATION	Dimensions: 1+	Measures: 2+
HORIZONTAL_COMBINATION_STACKED	Axis: y	Axis: x
HORIZONTAL_COMBINATION_DUAL		
BAR_DUAL		
BAR_STACKED_DUAL		

Chart Type	Required Dimensions	Required Measures
BAR_STACKED_DUAL_100		
HORIZONTAL_COMBINATION_STACKED_DUAL		
HEAT_MAP	Dimensions: 1+	Measures: 1+ Note: Determines the shade of color
BULLET	Dimensions: 1+ Axis: y	Measures: 1+ Axis: x
VERTICAL_BULLET	Dimensions: 1+ Axis: x	Measures: 1+ Axis: y
WATERFALL	Directions: 1+ Axis: x	Measures: 1+ Axis: y
HORIZONTAL_WATERFALL	Directions: 1+ Axis: y	Measures: 1+ Axis: x

Complete UI Annotations Example Code

Working Example Code for the use of UI annotations in CDS.

Metadata Extension for Main CDS View

Z_C_TRAVEL_R

i *Expand the following code sample to view the source code*

Sample Code

```
@Metadata.layer: #CORE

@UI: { headerInfo: { typeName: 'Object Page - Title',
                    title: { value: 'TravelID' }, // Defines the value shown in title section
                    typeNamePlural: 'List Report - Table Header'
                  }
      }

annotate view Z_C_TRAVEL_R with

{
  // FACET SECTION

  @UI.facet: [

    // Header Facet (Object Page):

    { id:           'HeaderFacet',
      purpose:      #HEADER,
      type:         #FIELDGROUP_REFERENCE,
      label:        'Object Page - Header Facet',
      targetQualifier: 'HeaderItems', // Refers to lineItems with @UI.fieldGroup: [{qua
      position:     10 },
```

// Body Facets (Object Page)

// Facet for chart - charts can only be displayed in facets

```

    { id:          'Chart1-ID', // Facet for our Chart with type "#CHART_REFERENCE"
      purpose:     #STANDARD,
      type:        #CHART_REFERENCE,
      label:       'Chart_facet',
      targetQualifier: 'Chart1',
      targetElement:  '_Booking', // Specifies where the data is retrieved from
      position:     1
    },

```

// Facet 1 - Parent (collection) for Fieldgroup 1 and Fieldgroup 2

```

    { id:          'Facet1-ID',
      type:        #COLLECTION,
      label:       'Facet 1 - Customer & Travel Data',
      position:    10 },

```

// Facet for Fieldgroup 1 - nested inside Facet 1

```

    { id:          'Fieldgroup1-ID',
      type:        #FIELDGROUP_REFERENCE,
      label:       'Fieldgroup 1 - Customer Data',
      parentId:    'Facet1-ID', // Places this facet into 'Facet 1'
      targetElement:  '_Customer', // Specifies where the data is retrieved from - UI
      targetQualifier: 'Fieldgroup1',
      position:    10 },

```

// Facet for Fieldgroup 2 - nested inside Facet 1

```

    { id:          'Fieldgroup2-ID',
      type:        #FIELDGROUP_REFERENCE,
      label:       'Fieldgroup 2 - Travel Data',
      parentId:    'Facet1-ID', // Places this facet into 'Facet 1'
      targetQualifier: 'Fieldgroup2', // No targetElement defined - Default target is t
      position:    20 },

```

// Facet 2

```

    { id:          'Facet2-ID',
      type:        #LINEITEM_REFERENCE, // Facet shows the referenced items in a li
      label:       'Facet 2 - Bookings',
      targetQualifier: 'Facet2',
      position:    20,
      targetElement:  '_Booking'},

```

// Facet 3

```

    { id:          'Facet3-ID',
      purpose:     #STANDARD,
      type:        #IDENTIFICATION_REFERENCE, // Refers to elements annotated with
      label:       'Facet 3 - Overview and Comments',
      position:    30 }
  ]

```

// ELEMENT LIST

```

@UI: { lineItem:    [ { position: 10,
                      importance: #HIGH } ],
      identification: [ { position: 10 } ],
      selectionField: [ { position: 10 } ],
      fieldGroup:    [ { qualifier: 'HeaderItems',
                        position: 10 } ] }

```

TravelID;

```

@UI: { lineItem:    [ { position: 20,
                      importance: #HIGH,
                      label: 'Agency' } ],
      identification: [ { position: 20,
                        label: 'Agency' } ],

```



```

        selectionField: [ { position: 20 } ],
        fieldGroup:      [ { qualifier: 'HeaderItems',
                             position: 20 } ] }
AgencyID;

@UI: { lineItem:          [ { position: 30,
                             importance: #HIGH,
                             label: 'Customer' } ],
      identification: [ { position: 30,
                             label: 'Customer' } ],
      selectionField: [ { position: 30 } ],
      fieldGroup:      [ { qualifier: 'Fieldgroup2',
                             position: 10 } ] }
CustomerID;

@UI: { lineItem:          [ { position: 40,
                             label: 'Start of Travel' } ],
      identification: [ { position: 40,
                             label: 'Start of Travel' } ],
      fieldGroup:      [ { qualifier: 'Fieldgroup2',
                             position: 20 } ] }
BeginDate;

@UI: { lineItem:          [ { position: 50,
                             label: 'End of Travel',
                             importance: #MEDIUM } ],
      identification: [ { position: 50,
                             label: 'End of Travel' } ],
      fieldGroup:      [ { qualifier: 'Fieldgroup2',
                             position: 10 } ] }
EndDate;

@UI: { hidden: true,
      identification: [ { position: 80 } ] ,
      fieldGroup:      [ { qualifier: 'Fieldgroup2',
                             position: 10 } ] }
BookingFee;

@UI: { identification: [ { position: 60 } ] ,
      fieldGroup:      [ { qualifier: 'Fieldgroup2',
                             position: 10 } ],
      dataPoint:        { title: 'Total Price',
                           targetValueElement: 'TotalPrice' } } }
TotalPrice;

@UI: { identification: [ { position: 70,
                             label: 'Comment' } ] ,
      fieldGroup:      [ { qualifier: 'Fieldgroup2',
                             position: 10 } ] }
Memo;

@UI: { lineItem:          [ { position: 60,
                             importance: #HIGH },
                           { type: #FOR_ACTION,
                             dataAction: 'set_status_booked',
                             label: 'Set to Booked' } ],
      fieldGroup:      [ { qualifier: 'Fieldgroup2',
                             position: 10 } ] }
Status;
}

```

Metadata Extension for _Customer

Z_C_CUSTOMER_R

i *Expand the following code sample to view the source code*

☰ **Sample Code**

```

@Metadata.layer: #CORE
annotate view Z_C_CUSTOMER_R
with
{
  @UI.fieldGroup: [{qualifier: 'Fieldgroup1', position: 05}]
  Street;

  @UI.fieldGroup: [{qualifier: 'Fieldgroup1', position: 10}] // Using gaps between position num
  CustomerID;

  @UI.fieldGroup: [{qualifier: 'Fieldgroup1', position: 20}]
  FirstName;

  @UI.fieldGroup: [{qualifier: 'Fieldgroup1', position: 30}]
  LastName;

  @UI.fieldGroup: [{qualifier: 'Fieldgroup1', position: 40}]
  Title;

  @UI.fieldGroup: [{qualifier: 'Fieldgroup1', position: 50}]
  PostalCode;

  @UI.fieldGroup: [{qualifier: 'Fieldgroup1', position: 60}]
  City;

  @UI.fieldGroup: [{qualifier: 'Fieldgroup1', position: 70}]
  CountryCode;

  @UI.fieldGroup: [{qualifier: 'Fieldgroup1', position: 80}]
  PhoneNumber;

  @UI.fieldGroup: [{qualifier: 'Fieldgroup1', position: 90}]
  EMailAddress;

  /* Associations */
  @UI.fieldGroup: [{qualifier: 'Fieldgroup1', position: 100}]
  _Country;
}

```

Metadata Extension for _Booking

Z_C_BOOKING_R

i Expand the following code sample to view the source code

Sample Code

```

@Metadata.layer: #CORE

@UI.chart: [{
  qualifier: 'Chart1', //refers to targetQualifier defined in chart facet in Z_C_TRAVEL_UI
  title: 'Flight Prices of this Travel by Airline',
  chartType: #COMBINATION_DUAL,
  dimensions: [ 'AirlineID'],
  measures: [ 'ConvertedFlightPrice', 'DistinctBookings'],
  measureAttributes: [ {measure: 'ConvertedFlightPrice', role: #AXIS_1},
    {measure: 'DistinctBookings', role: #AXIS_2} ],
  dimensionAttributes: [ {dimension: 'AirlineID', role: #SERIES} ],
  description: 'Chart shows flight prices of travel by the airlines used per each booking.'
}]

annotate view Z_C_BOOKING_R with
{
  @UI.lineItem:
  [{ qualifier: 'Facet2',
    position: 10,
    label: 'Travel No.',

```

```

        hidden: true }]
TravelID;

@UI.lineItem:
  [{ qualifier: 'Facet2',
    position: 20,
    label: 'Booking No.' }]
BookingID;

@UI.lineItem:
  [{ qualifier: 'Facet2',
    position: 30,
    label: 'Date of Booking' }]
BookingDate;

@UI.lineItem:
  [{ qualifier: 'Facet2',
    position: 40,
    label: 'Customer' }]
CustomerID;

@UI.lineItem:
  [{ qualifier: 'Facet2',
    position: 50,
    label: 'Airline' }]
AirlineID;

@UI.lineItem:
  [{ qualifier: 'Facet2',
    position: 60,
    label: 'Connection No.' }]
ConnectionID;

@UI.lineItem:
  [{ qualifier: 'Facet2',
    position: 70,
    label: 'Date of flight' }]
FlightDate;

@UI.lineItem:
  [{ qualifier: 'Facet2',
    position: 80,
    label: 'Flight Price' }]
FlightPrice;

@UI.lineItem:
  [{ qualifier: 'Facet2',
    position: 90,
    label: 'Currency' }]
CurrencyCode;

}

```

Field Manipulation

Get information about what UI annotations to use to manipulate fields for SAP Fiori UIs.

This chapter describes annotations that influence the appearance exposed fields. When a field is marked with these annotations, it is manipulated no matter in what other annotations the field is used. The reason for this is that annotations for manipulation are self-contained annotations and not properties of other annotations.

For example, when a field is marked with the *@UI.masked* annotation, the field is masked regardless if it is used in a *@UI.lineItem* annotation or a *@UI.identification* annotation.

To manipulate the appearance of fields on SAP Fiori UIs, you can use the annotations explained in the following sections:

- [Multi-Line Text](#)

Here you get information about what UI annotations to use to display fields as multi-line text on SAP Fiori UIs.

- [Field Masking](#)

Here you get information about what UI annotations to use to mask fields.

- [Interaction with Other Annotations](#)

Here you get information about how to provide interaction between annotations.

Multi-Line Text

Get information about what UI annotations to use to display fields as multi-line text on SAP Fiori UIs.

You can use the following annotation to mark a field to be displayed by a control that supports multi-line input, for example a text area:

- *@UI.multiLineText*

Sample Code

```
...
define view Product as select from ... {
  @UI.identification: [ { position: 10 } ]
  key ProductID,

  @UI.identification: [ { position: 20 } ]
  ProductName,

  @UI.identification: [ { position: 30 } ]
  @UI.multiLineText: true
  Description,

  ...
}
```

Related Information

[Field Manipulation](#)

[Field Masking](#)

[Interaction with Other Annotations](#)

[Hiding Fields on the Object Page](#)

Field Masking

Get information about what UI annotations to use to mask fields, for example for password input, on SAP Fiori UIs.

In some cases, data of fields need to be consumed by the client, but must not be visible on the UI. This field behavior is required when users need to enter passwords, for example.

You can use the following annotation to mark a field to not to be displayed in clear text by the client because, for example, it contains sensitive data. Field masking is only intended for input fields:

- *@UI.Masked*

This annotation does not influence how data is transferred. If a field is marked with the *@UI.masked* annotation, the data belonging to this field is still transferred to the client like any other property in clear text.

Sample Code

```
...
define view Destination as select from ... {
  @UI.identification: [ { position: 10 } ]
  key DestinationID,
  ...

  @UI.identification: [ { position: 20 } ]
  AuthType,          -- None, Basic, SSO, ...

  @UI.identification: [ { position: 30 } ]
  BasicAuthUserName,

  @UI.identification: [ { position: 40 } ]
  @UI.masked
  BasicAuthPassword,

  ...
}
```

Related Information

[Field Manipulation](#)

[Multi-Line Text](#)

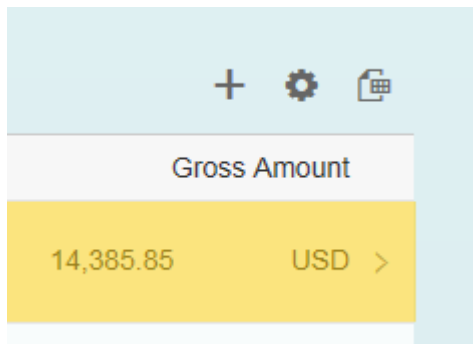
[Interaction with Other Annotations](#)

[Hiding Fields on the Object Page](#)

Interaction with Other Annotations

Get information about how to provide interaction between annotations.

In addition to UI annotations, you can use model-specific annotations that affect the desired client behavior. You can implement, for example, unit-currency-mappings or ID-text-mappings. These model-specific annotations can be evaluated by the client and **no** additional UI annotations are required.



Example of interaction between CDS annotations

❖ Example

In the following example, the field `CurrencyCode` is marked with a `@Semantics.currencyCode` and is referenced by field `GrossAmount`. This means that the field `GrossAmount` is always displayed with the corresponding currency. The field `CurrencyCode` does not need to be exposed explicitly.

☰ Sample Code

```
...
define view ZExample_SalesOrder as select from sepm_cds_sales_order as so
{
  @UI.identification: [ { position: 10 } ]
  key so.sales_order_id as SalesOrder,

  @Semantics.currencyCode: true
  so.currency_code as CurrencyCode,

  @Semantics.amount.currencyCode: 'CurrencyCode'
  @UI.identification: [ { position: '20' } ]
  so.gross_amount as GrossAmount,
```

Related Information

[Field Manipulation](#)

[Multi-Line Text](#)

[Field Masking](#)

[Hiding Fields on the Object Page](#)

Inheritance of Annotations

Get information about what property to use to prevent elements from being inherited from an underlying CDS view.

By default, all UI annotation elements are inherited from the underlying CDS view. You can explicitly disable this behavior. You can use the following property to prevent a UI annotation element from being inherited:

- *exclude*

The following sample code depicts the CDS view `ZP_SalesOrder` that inherits elements from the underlying CDS view `SEPM_CDS_SALES_ORDER`, and uses the UI annotation `@UI.identification`:

Sample Code

```

...
define view ZP_SalesOrder as select from sepm_cds_sales_order as so {
    @UI.identification: [ { position: 10 } ]
    key so.sales_order_id as SalesOrder,

    @UI.identification: [ { position: 20 } ]
    so.customer.company_name as CompanyName,
...
}

```

The following sample code depicts the CDS view `ZI_SalesOrder` that inherits elements from the underlying CDS view `ZP_SalesOrder`. In this view, the element `key SalesOrder` is inherited from the underlying CDS view as UI annotation `@UI.identification` by default. The element `so.customer.company_name as CompanyName`, however, is not inherited as UI annotation `@UI.identification` because of the property `exclude`:

Sample Code

```

...
define view ZI_SalesOrder as select from ZP_SalesOrder as so {

    key SalesOrder,

    @UI.identification: [ { exclude } ]
    so.customer.company_name as CompanyName,
...
}

```

Data Points

Get an overview of how to use data points to display criticality, trends, and references to people and time periods on SAP Fiori UIs.

In some cases, you want to visualize a single point of data that typically is a number that can be enriched with business-relevant data but may also be textual, for example a status value.

Gross Amount

Gross Amount per Customer

 **34.5 kEUR**

Reference Period: 2015 Q3

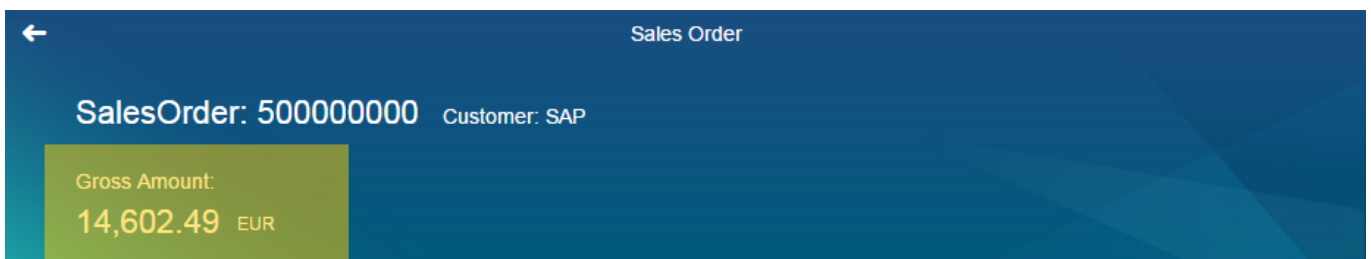
Responsible: John Doe

Example of data points

You can use the following UI annotation to define a single point of data:

- *@UI.dataPoint*

You can, for example, express if a high or a low value is desired, or if a value is increasing or decreasing. The simplest variant of the UI annotation *@UI.dataPoint* consists of the `title` property.



Example of simple variant of data point

In the following example, only the title is exposed to the UI.

Sample Code

```
...
define view ZExample_SalesOrdersByCustomer as select from ... as so {
  key so.buyer_guid as BuyerGuid,

  @Semantics.currencyCode: true
  so.currency_code as CurrencyCode,

  @UI.dataPoint: { title: 'Gross Amount' }
  @Semantics.amount.currencyCode: 'CurrencyCode'
```



```
so.actual_amount as ActualAmount
}
```

Related Information

[Criticality](#)

[Trends](#)

[Trend-Criticality Calculation](#)

[Person Responsible and Reference Period](#)

[DataField Type: #AS_DATAPPOINT](#)

[CDS Annotations](#)

Criticality

Get information about how to use data points to display criticality on SAP Fiori UIs.

A more usable variant of the UI annotation *@UI.dataPoint* also contains information about the criticality, the trend, and the name of a person responsible.

You can use the sub-annotation *@dataPoint.criticality* to express if a value is positive or negative, for example.

You can use the sub-annotation *@dataPoint.trend* to express if a value has decreased or increased, for example.

In this case, the properties *targetValue*, *criticality*, and *trend* are already evaluated in the CDS view. In the CDS view, the target value is already calculated, and if the current value thus is negative or positive, and if the current value has improved or declined, for example. These values are only referred to from the *@UI.dataPoint* annotation.

Data can be defined as being either positive, critical, or negative. These data can be statuses, for example.

You can use the following sub-annotation to highlight criticality:

- *UI.dataPoint.criticality*

You define this UI annotation at view level. It refers to the elements that are to be used in the chart. Additionally, you can provide a title and description.

The table below lists the values that are valid for the UI annotation *@UI.dataPoint.criticality*, and shows how these values are visualized on the UI:

Values and Visualization of Criticality

Value	Description	Visualization in Color
1	Negative	Red
2	Critical	Yellow
3	Positive	Green

Sample Code

```

...
define view ZExample_SalesOrdersByCustomer as select from ... as so {
    key so.buyer_guid as BuyerGuid,

    @Semantics.currencyCode: true
    so.currency_code as CurrencyCode,

    @UI.dataPoint: {
        title: 'Gross Amount',
        targetValueElement: 'TargetAmount',          -- Reference to element
        criticality: 'AmountCriticality',             -- Reference to element
        trend: 'AmountTrend',                        -- Reference to element
    }
    @Semantics.amount.currencyCode: 'CurrencyCode'
    so.actual_amount as ActualAmount,

    @Semantics.amount.currencyCode: 'CurrencyCode'
    so.target_amount as TargetAmount,

    so.criticality as AmountCriticality,

    so.trend as AmountTrend
}

```

Related Information

[Data Points](#)

[Trends](#)

[Trend-Criticality Calculation](#)

[Person Responsible and Reference Period](#)

[DataField Type: #AS_DATAPOINT](#)

[CDS Annotations](#)

Trends

Get information about how to use data points to display trends on SAP Fiori UIs.

Data can be defined as being either increasing, decreasing, or stable. These data can be measured over a certain period of time and visualized on the UI.

You can use the following sub-annotations to highlight trends:

- *@UI.dataPoint.trend*

❖ Example

For an example, see the example code in section *Criticality* linked below.

- *@UI.dataPoint.trendCalculation*

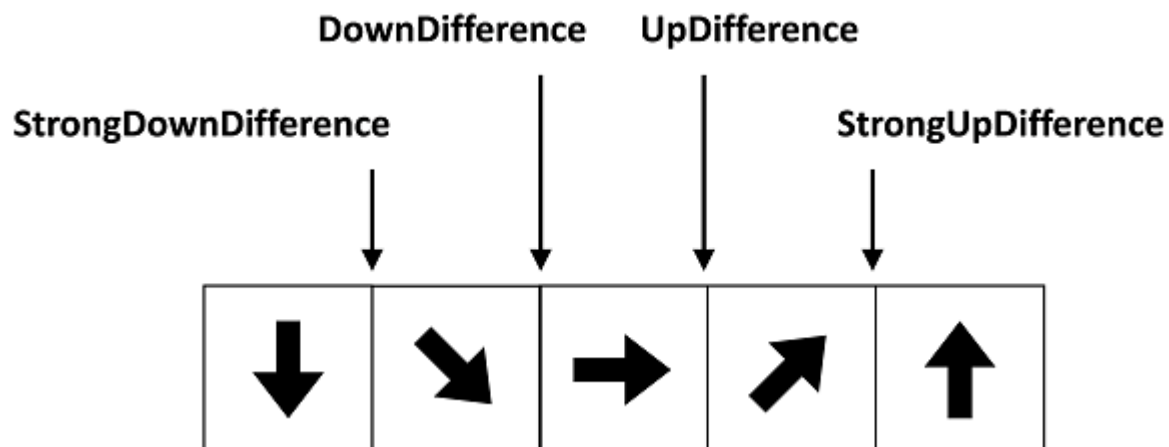
❖ Example

The table below lists the values that are valid for the UI annotation `@UI.dataPoint.trendCalculation`, and shows how these values are visualized on the UI:

Values and Visualization of Trend

Value	Description	Visualization
1	Strong up	↑
2	Up	↗
3	Sideways	→
4	Down	↘
5	Strong down	↓

For the trend calculation, the flag `isRelativeDifference` indicates whether the absolute or the relative difference between the actual value and the reference value is used to calculate the trend.



Visualization of trend calculation

Sample Code

```
...
define view ZExample_SalesOrdersByCustomer as select from ... as so {
  key so.buyer_guid as BuyerGuid,

  @Semantics.currencyCode: true
  so.currency_code as CurrencyCode,

  @UI.dataPoint: {
    title: 'Gross Amount',
    //...
    trendCalculation: {
      referenceValue: 'ReferenceAmount',      -- Reference to element
      isRelativeDifference: true,              -- Comparison of ratio
      strongUpDifference: 1.25,
      upDifference: 1.1,
      downDifference: 0.9,
      strongDownDifference: 0.75
    }
  }
}
```

```

    }
  }
  @Semantics.amount.currencyCode: 'CurrencyCode'
  so.target_amount as TargetAmount,

  @Semantics.amount.currencyCode: 'CurrencyCode'
  so.reference_amount as ReferenceAmount
}

```

Related Information

[Data Points](#)

[Criticality](#)

[Trend-Criticality Calculation](#)

[Person Responsible and Reference Period](#)

[DataField Type: #AS_DATAPOINT](#)

[CDS Annotations](#)

Trend-Criticality Calculation

Get information about how to use data points to calculate and display trend-criticality relations.

Another way to specify properties of criticality and trend is to define rules for criticality and trend within the UI annotation *@UI.dataPoint*.



Example of visualization of trend-criticality calculation

You can use the following sub-annotations to calculate trends and derive from these calculation the criticality of data:

- `@UI.dataPoint.trendCalculation`
- `@UI.dataPoint.criticalityCalculation`

Sample Code

```
...
define view ZExample_SalesOrdersByCustomer as select from ... as so {
  key so.buyer_guid as BuyerGuid,

  @Semantics.currencyCode: true
  so.currency_code as CurrencyCode,

  @UI.dataPoint: {
    title: 'Gross Amount',
    targetValue: 9216,
    criticalityCalculation: {
      improvementDirection: #TARGET,
      toleranceRangeLowValue: 9200,
      toleranceRangeHighValue: 9300,
      deviationRangeLowValue: 8800,
      deviationRangeHighValue: 9700
    },
    trendCalculation: {
      referenceValue: 'ReferenceAmount',      -- Reference to element
      isRelativeDifference: false,             -- Comparison of difference
      strongUpDifference: 100,
      upDifference: 10,
      downDifference: -10,
      strongDownDifference: -100
    }
  }
  @Semantics.amount.currencyCode: 'CurrencyCode'
  so.target_amount as TargetAmount,

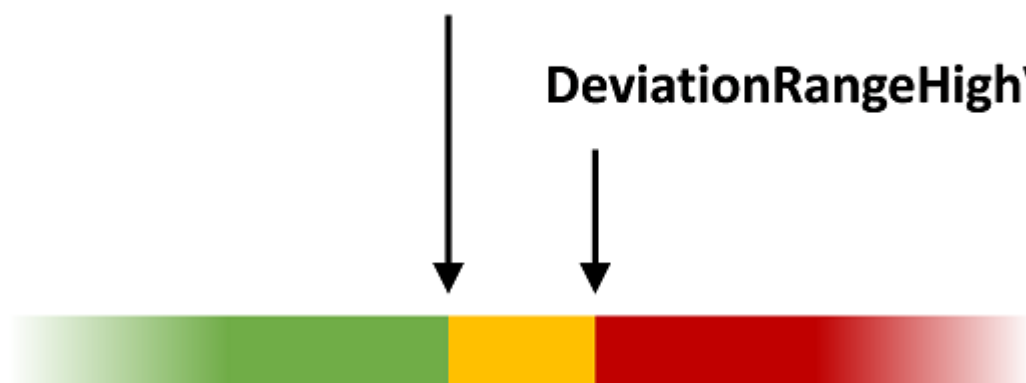
  @Semantics.amount.currencyCode: 'CurrencyCode'
  so.reference_amount as ReferenceAmount
}
```

For the criticality calculation, the value of the property `improvementDirection` is crucial because this value determines what further properties are needed. If, for example, the value is `#MINIMIZE`, the properties `ToleranceRangeHighValue` and `DeviationRangeHighValue` are relevant.



ToleranceRangeHighValue

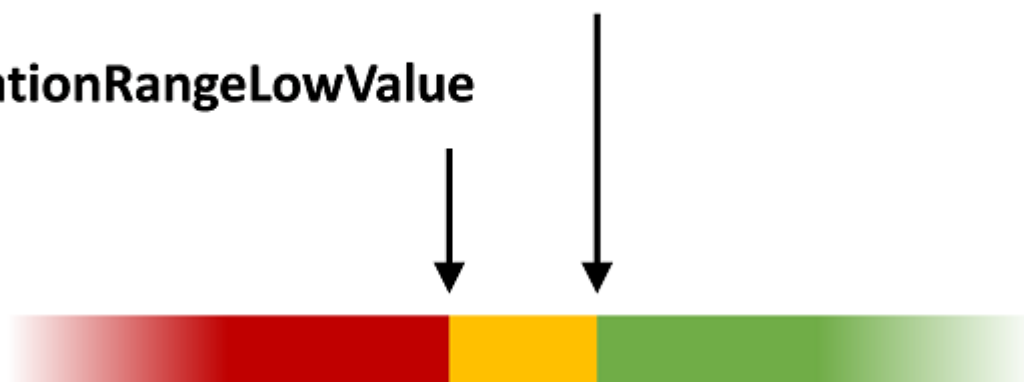
DeviationRangeHighValue



Example of improvementDirection: # MINIMIZE

ToleranceRangeLowValue

DeviationRangeLowValue



Example of improvementDirection: # MAXIMIZE

The properties of the sub-annotation *@UI.dataPoint.criticalityCalculation* can have either constant values or derive values from referencing to other elements. If a property references to another element, the suffix *Element* must be added to the name of the property.

i Note

This also applies to the properties of the sub-annotation *@UI.dataPoint.trendCalculation*, except for the property *referenceValue*. This property always references to another element.

❖ Example

toleranceRangeLowValue becomes *toleranceRangeLowValueElement*.

Related Information

[Data Points](#)

[Criticality](#)

[Trends](#)

[Person Responsible and Reference Period](#)

[DataField Type: #AS_DATAPOINT](#)

[CDS Annotations](#)

Person Responsible and Reference Period

Get information about how to use data points to display references to persons responsible and to reference periods on SAP Fiori UIs.

You can add the following properties to the UI annotation *@UI.dataPoint*:

- `referencePeriod`
- `responsibleName`

You can define both properties either in the UI annotation directly, or in another element and reference from the UI annotation to this element.

❖ Example

In the following example, the data point has a static reference period and a static person responsible. The value of the gross amount is formatted with the `valueFormat` property. The value is thus scaled with factor 1000 and is displayed with one decimal place, this is the value 34500 EUR would be displayed as **34.5 kEUR**.



Example of visualization of person responsible, reference period, and value format

≡ Sample Code

```
...
define view ZExample_SalesOrdersByCustomer as select from ... as so {
  key so.buyer_guid as BuyerGuid,

  @Semantics.currencyCode: true
  so.currency_code as CurrencyCode,
```

```

@UI.dataPoint: {
  title: 'Gross Amount',
  description: 'Gross Amount per Customer',
  longDescription: 'The gross amount per customer ...',
  valueFormat: {
    scaleFactor: 1000,
    numberOfFractionalDigits: 1
  },
  referencePeriod: { description: '2015 Q3' },
  responsibleName: 'John Doe'
}
@Semantics.amount.currencyCode: 'CurrencyCode'
so.actual_amount as ActualAmount
}

```

❖ Example

In the following example, a dynamic reference period is used that is supplied by the following parameters:

- start
- end

These parameters have to be aliased in the element list before they can be used in the *@UI.dataPoint* annotation. The responsible property must refer to a to-one-association. The target entity of this association should contain the contact data of the person responsible.

i *Expand the following code sample to view the source code*

☰ Sample Code

```

...
define view ZExample_SalesOrdersByCust
with parameters p_StartDate : abap.dats,
               p_EndDate   : abap.dats
as select from ... as so
association [0..1] to Employees as _PersonResponsible
on _PersonResponsible.EmployeeId = $projection.PersonResponsible
{
  ...

  $parameters.p_StartDate as StartDate, -- Alias is required for annotation
  $parameters.p_EndDate as EndDate,      -- Alias is required for annotation
  so.person_responsible as PersonResponsible,

  @Semantics.currencyCode: true
  so.currency_code as CurrencyCode,

  @UI.dataPoint: {
    title: 'Gross Amount',
    referencePeriod: {
      start: 'StartDate', -- Reference to element
      end: 'EndDate'      -- Reference to element
    },
  },
}

```



```

    responsible: '_PersonResponsible' -- Reference to association
  }
  @Semantics.amount.currencyCode: 'CurrencyCode'
  @DefaultAggregation: #SUM
  so.actual_amount as ActualAmount,

  _PersonResponsible
}
where so.validity_date >= $parameters.p_StartDate
and   so.validity_date <= $parameters.p_EndDate

```

Sample Code

For a definition of element list, see section *Glossary* linked below.

Related Information

[Data Points](#)

[Criticality](#)

[Trends](#)

[Trend-Criticality Calculation](#)

[DataField Type: #AS_DATAPOINT](#)

[CDS Annotations](#)

DataField Type: #AS_DATAPOINT

Get information about how to use the type #AS_DATAPOINT to see other annotations.

The type #AS_DATAPOINT maps to *DataFieldForAnnotation*. *DataFieldForAnnotation* is used to see other annotations using the *Edm.AnnotationPath* abstract type. The annotation path must end in *vCard.Address* or *UI.dataPoint*.

You can use the following type to reference an exposed data point from dataField-like annotations:

- #AS_DATAPOINT

You use this type to include a microchart in the UI annotation *@UI.lineItem*, for example.

Example

In this example, the UI annotation *@UI.lineItem* has to be defined at the same CDS element as the UI annotation *@UI.dataPoint* itself.

Sample Code

```

...
define view ZExample_SalesOrdersByCustomer as select from ... as so {
  key so.buyer_guid as BuyerGuid,
  ...

  @Semantics.currencyCode: true
  so.currency_code as CurrencyCode,

```

```
@UI.dataPoint: { title: 'Gross Amount' }  
@UI.lineItem: [ { type: #AS_DATAPoint } ]  
@Semantics.amount.currencyCode: 'CurrencyCode'  
so.actual_amount as ActualAmount  
}
```

Related Information

[Data Points](#)

[Criticality](#)

[Trends](#)

[Trend-Criticality Calculation](#)

[Person Responsible and Reference Period](#)

[CDS Annotations](#)

Actions

Get information about how to use *dataField* types to provide means of executing actions on SAP Fiori UIs.

Actions are directly related to items that you can see in a table on a master-detail floorplan, for example. Users can select items and execute certain actions on the selected items.

The screenshot shows the SAP Fiori 'AnnoDokuExmaple' application. At the top, there's a header bar with the SAP logo and a user profile 'Default User'. Below the header, the title 'AnnoDokuExmaple' is displayed. The main area features a filter bar with 'Standard *' and a search input field. Below the filter bar, there's a 'Company:' label and an input field. The main content area displays a table titled 'Sales Orders (1,216) | Demo'. The table has four columns: 'Sales Order ID', 'Company', 'Currency Code', and 'Gross Amount'. The first row is selected, and the 'Copy' button in the top right corner of the table is highlighted with a yellow box. The table lists 15 sales orders with IDs from 500000000 to 500000015, each with a company name, currency code (EUR), and gross amount.

Sales Order ID	Company	Currency Code	Gross Amount
500000000	SAP	EUR	25,867.03 EUR
500000001	DelBont Industries	EUR	14,602.49 EUR
500000002	TECUM	EUR	5,631.08 EUR
500000003	Asia High tech	EUR	1,704.04 EUR
500000004	Asia High tech	EUR	761.24 EUR
500000005	AVANTEL	EUR	101,299.22 EUR
500000006	Talpa	EUR	250.73 EUR
500000007	Panorama Studios	EUR	10,311.35 EUR
500000008	Telecomunicaciones Star	EUR	195.16 EUR
500000009	SAP	EUR	3,972.22 EUR
500000010	DelBont Industries	EUR	827.95 EUR
500000011	Panorama Studios	EUR	325.94 EUR
500000012	TECUM	EUR	24,704.40 EUR
500000013	Asia High tech	EUR	8,256.22 EUR
500000014	Asia High tech	EUR	3,459.33 EUR
500000015	AVANTEL	EUR	862.73 EUR

Example of action 'Copy' on master-detail floorplan

You can use the following *dataField* type to expose actions to the client:

- #FOR_ACTION

This property has to be assigned to some arbitrary element. It is thereby irrelevant if the property refers to the element to which the property is assigned.

Sample Code

```
...
define view ZExample_SalesOrder as select from sepm_cds_sales_order as so {
  @UI.lineItem: [
    -- Standard Lineitem
    { position: 10 },
    -- Action Lineitem
    { type: #FOR_ACTION, dataAction: 'Copy', label: 'Copy' }
  ]
}
```

```
key so.sales_order_id as SalesOrder,  
...  
}
```

Navigation

Get an overview of how to use *dataField* types to provide means of navigation on SAP Fiori UIs.

It often is not sufficient to stay on one screen. Users might need to navigate between screens or even to web sites outside an application. You can use the following *dataField* types to include navigation concepts:

- [#WITH_NAVIGATION_PATH](#)

Used for navigation within an application.

- [#WITH_URL](#)

Used for navigation from an application to an external web site.

- [#FOR_INTENT_BASED_NAVIGATION](#)

Used for navigation based on an action that is related to a semantic object.

With Navigation Path

Get information about how to provide navigation between UI screens and pages on SAP Fiori UIs.

This navigation type contains either a navigation property or a term cast. The term either is of type `Edm.EntityType`, a concrete entity type, or a collection of these types.

Standard *

Hide Filter Bar Filters Go

Company:

Sales Orders (1,216) | Demo

Business Partner

Sales Order ID	Company	Currency Code	Gross Amount
☒ 500000000	SAP	EUR	25,867.03 EUR >
☐ 500000001	DelBont Industries	EUR	14,602.49 EUR >
☐ 500000002	TECUM	EUR	5,631.08 EUR >
☐ 500000003	Asia High tech	EUR	1,704.04 EUR >
☐ 500000004	Asia High tech	EUR	761.24 EUR >
☐ 500000005	AVANTEL	EUR	101,299.22 EUR >
☐ 500000006	Talpa	EUR	250.73 EUR >
☐ 500000007	Panorama Studios	EUR	10,311.35 EUR >
☐ 500000008	Telecomunicaciones Star	EUR	195.16 EUR >
☐ 500000009	SAP	EUR	3,972.22 EUR >
☐ 500000010	DelBont Industries	EUR	827.95 EUR >
☐ 500000011	Panorama Studios	EUR	325.94 EUR >
☐ 500000012	TECUM	EUR	24,704.40 EUR >
☐ 500000013	Asia High tech	EUR	8,256.22 EUR >
☐ 500000014	Asia High tech	EUR	3,459.33 EUR >
☐ 500000015	AVANTEL	EUR	862.73 EUR >

Example of dataField of type #WITH_NAVIGATION_PATH

You can use the following dataField type to expose a link to other pages of a UI:

- #WITH_NAVIGATION_PATH

Example

In the following example, CompanyName is displayed as link referring to the association _BusinessPartner.

Sample Code

```
...
define view ZExample_SalesOrder as select from sepm_cds_sales_order as so
  association [0..1] to sepm_cds_business_partner as _BusinessPartner
  on $projection.buyer_guid = _BusinessPartner.business_partner_key
{
  key so.sales_order_id as SalesOrder,
  so.buyer_guid,
  ...

  @UI.lineItem: [ {
    position: 20,
    type: #WITH_NAVIGATION_PATH,
```

```

        targetElement: '_BusinessPartner' -- Reference to association
    } ]
    so.customer.company_name as CompanyName,
    ...
    _BusinessPartner
}

```

Related Information

[Navigation](#)

[With URL](#)

[Based on Intent](#)

With URL

Get information about how to provide navigation from SAP Fiori UIs to external web sites, for example.

This type navigation type contains a reference to a URL to navigate to specific web sites, for example.

Standard *

Hide Filter Bar

Filters

Go

Company:

Sales Orders (1,216) | Demo

Sales Order ID	Company	Currency Code	Gross Amount
500000000	SAP	EUR	25,867.03 EUR >
500000001	DelBont Industries	EUR	14,602.49 EUR >
500000002	TECUM	EUR	5,631.08 EUR >
500000003	Asia High tech	EUR	1,704.04 EUR >
500000004	Asia High tech	EUR	761.24 EUR >
500000005	AVANTEL	EUR	101,299.22 EUR >
500000006	Talpa	EUR	250.73 EUR >
500000007	Panorama Studios	EUR	10,311.35 EUR >
500000008	Telecomunicaciones Star	EUR	195.16 EUR >
500000009	SAP	EUR	3,972.22 EUR >
500000010	DelBont Industries	EUR	827.95 EUR >
500000011	Panorama Studios	EUR	325.94 EUR >
500000012	TECUM	EUR	24,704.40 EUR >
500000013	Asia High tech	EUR	8,256.22 EUR >
500000014	Asia High tech	EUR	3,459.33 EUR >

You can use the following *dataField* type to display links to external websites:

- #WITH_URL

❖ Example

In the following example, `CompanyName` is displayed as link referring to the CDS element `WebsiteUrl`.

≡ Sample Code

```
...
define view ZExample_SalesOrder as select from sepm_cds_sales_order as so
{
  key so.sales_order_id as SalesOrder,
  ...

  @UI.lineItem: [ {
    position: 20,
    type: #WITH_URL,
    url: 'WebsiteUrl' -- Reference to element
  } ]
  so.customer.company_name as CompanyName,

  so.customer.web_address as WebsiteUrl,
  ...
}
```

Related Information

[Navigation](#)

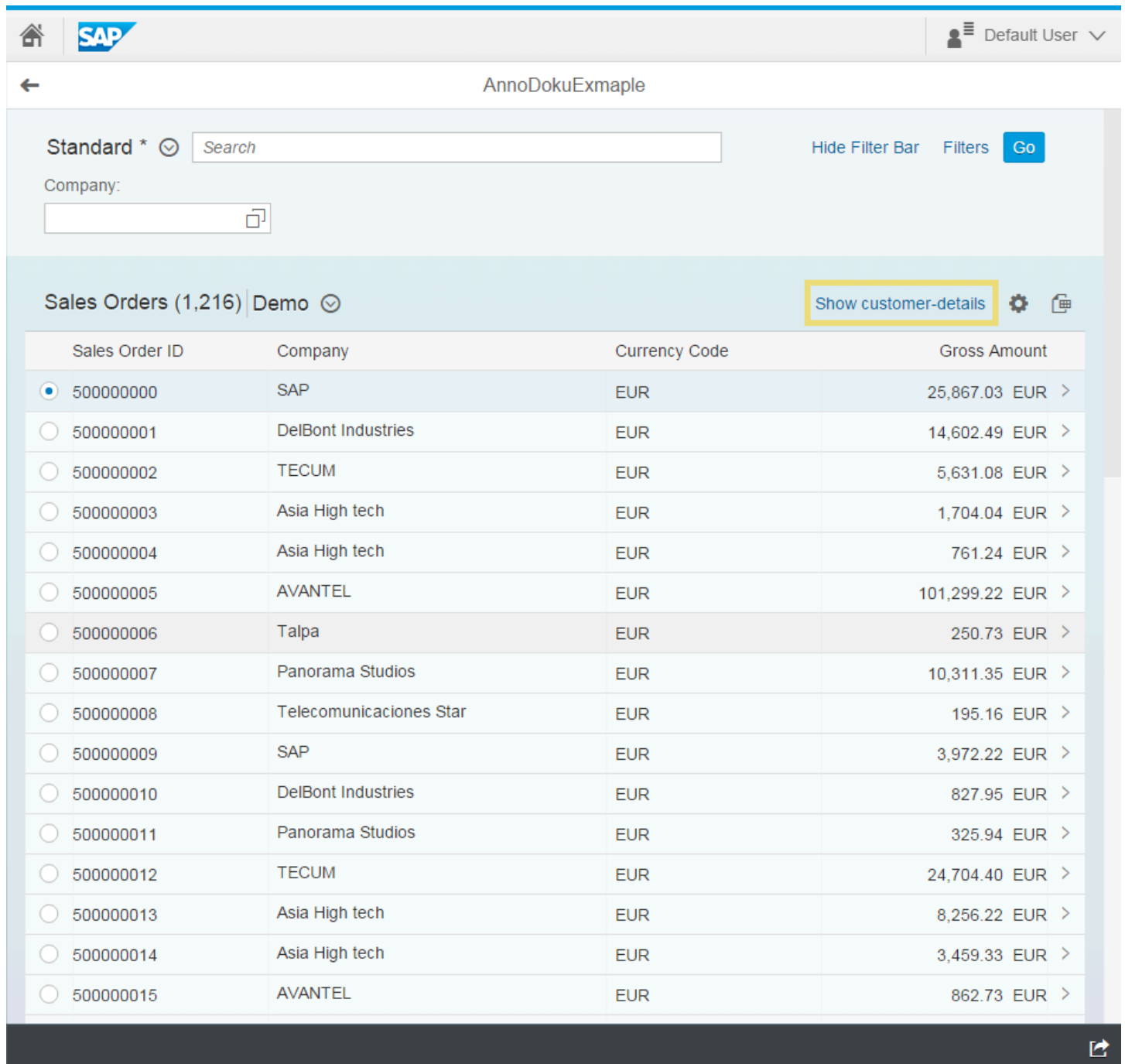
[With Navigation Path](#)

[Based on Intent](#)

Based on Intent

Get information about how to provide navigation related on actions that are executed on SAP Fiori UIs.

This navigation type contains an action that is related to a semantic object. This combination of action and semantic object is an **intent**. The annotation `@Consumption.semanticObject` is required for navigation based on intent. The client decides how to react when this navigation is triggered.



The screenshot shows the SAP Fiori 'AnnoDokuExmaple' application. At the top, there's a header with the SAP logo and a user profile 'Default User'. Below the header, the title 'AnnoDokuExmaple' is displayed. A filter bar contains 'Standard *' and a search input field. A 'Company:' label is followed by a selection box. On the right, there are buttons for 'Hide Filter Bar', 'Filters', and 'Go'. The main content area is titled 'Sales Orders (1,216) | Demo'. A button labeled 'Show customer-details' is highlighted with a yellow box. Below this is a table with columns: Sales Order ID, Company, Currency Code, and Gross Amount. The table lists 15 sales orders, with the first one selected. Each row has a radio button in the first column and a chevron icon in the last column.

Sales Order ID	Company	Currency Code	Gross Amount
☒ 500000000	SAP	EUR	25,867.03 EUR >
☐ 500000001	DelBont Industries	EUR	14,602.49 EUR >
☐ 500000002	TECUM	EUR	5,631.08 EUR >
☐ 500000003	Asia High tech	EUR	1,704.04 EUR >
☐ 500000004	Asia High tech	EUR	761.24 EUR >
☐ 500000005	AVANTEL	EUR	101,299.22 EUR >
☐ 500000006	Talpa	EUR	250.73 EUR >
☐ 500000007	Panorama Studios	EUR	10,311.35 EUR >
☐ 500000008	Telecomunicaciones Star	EUR	195.16 EUR >
☐ 500000009	SAP	EUR	3,972.22 EUR >
☐ 500000010	DelBont Industries	EUR	827.95 EUR >
☐ 500000011	Panorama Studios	EUR	325.94 EUR >
☐ 500000012	TECUM	EUR	24,704.40 EUR >
☐ 500000013	Asia High tech	EUR	8,256.22 EUR >
☐ 500000014	Asia High tech	EUR	3,459.33 EUR >
☐ 500000015	AVANTEL	EUR	862.73 EUR >

Example of dataField of type #FOR_INTENT_BASED_NAVIGATION

You can use the following *dataField* type to expose the intent to navigate without specifying how this navigation is to be resolved:

- #FOR_INTENT_BASED_NAVIGATION

❖ Example

In the following example, the intent 'Show' (action) 'BusinessPartner' (semantic object) is expressed. The client can, for example, open a separate application to display the details of the corresponding business partner.

≡ Sample Code

```
...
define view ZExample_SalesOrder as select from sepm_cds_sales_order as so
{
  key so.sales_order_id as SalesOrder,
  ...

  @UI.lineItem: [ {
```



```

        position: 20,
        label: 'Show customer-details',
        type: #FOR_INTENT_BASED_NAVIGATION,
        semanticObjectAction: 'Show' -- Action
    } ]
@Consumption.semanticObject: 'BusinessPartner' -- Semantic Object
so.customer.company_name as CompanyName,
...
}

```

Related Information

[Navigation](#)

[With Navigation Path](#)

[With URL](#)

[CDS Annotations](#)

Contact Data

Get information about what UI annotations to use to display contact data on SAP Fiori UIs.

In some cases users of an application need to see contact data, for example, of business partners, customers, or employees.

You can use the following annotation set to inform a client that an entity contains contact information and map the CDS elements to the corresponding address field:

- *@Semantics*

This annotation set contains annotations to inform about telephone numbers, email addresses, names, addresses, and contacts.

❖ Example

The following example contains sub-annotations belonging to the annotation set *@Semantics*. For a complete list, see section *Semantics Annotations* linked below.

i *Expand the following code sample to view the source code*

☰ Sample Code

```

...
define view Employees as select from ...
{
    key EmployeeId,

    @Semantics.name.givenName
    FirstName,

    @Semantics.name.additionalName
    MiddleName,

    @Semantics.name.familyName

```

```

    LastName,

    GenderCode,

    @Semantics.telephone.type: [#WORK, #PREF]
    PhoneNumber,

    @Semantics.telephone.type: [#FAX]
    FaxNumber,

    @Semantics.telephone.type: [#CELL]
    MobilePhoneNumber,

    @Semantics.eMail.address
    EmailAddress,

    PreferredLanguage,

    @Semantics.contact.birthDate
    BirthDate
}

```

Related Information

[CDS Annotations](#)

Develop Individual BO Capabilities

This chapter outlines how to develop individual RAP BO capabilities like for example numbering scenarios or message exposure.

- [Using Type and Control Mapping](#)

Whenever existing legacy code and data types are to be reused in behavior pools of business objects, then you need to perform a mapping between CDS field names and types and corresponding legacy field names and types.

- [Exposing Messages on the UI and Message Mapping](#)

Well-designed messages help to recognize, diagnose, and resolve issues. That's why it's important to always use messages consistently and optimize the interaction as a whole.

[Editing Language-Dependent Fields](#)

You can behavior-enable denormalized text elements in business objects by augmenting incoming requests with modify requests for the denormalized elements. This topic guides you through the steps to enable end users to maintain language-dependent texts in their log-on language.

[Using Type and Control Mapping](#)

Whenever existing legacy code and data types are to be reused in behavior pools of business objects, then you need to perform a mapping between CDS field names and types and corresponding legacy field names and types.

[Exposing Messages on the UI and Message Mapping](#)

[Working with Large Objects](#)

[Modeling Parameters for Non-Standard Operations](#)

Automatically Drawing Primary Key Values in Managed BOs

In **managed** scenarios with UUID keys, the values for primary key fields can be automatically generated by the managed runtime framework.

Context

Values for primary key fields can be generated automatically for managed business objects. This so-called [Managed Internal Early Numbering](#) is defined in the behavior definition of the business object. During the CREATE operation, the managed runtime framework draws the UUID automatically for the defined key fields. The newly created BO instance is immediately uniquely identifiable after the CREATE operation and saved with the given key during the save sequence.

For more information, see [Numbering](#).

Prerequisites

- The primary key field has the ABAP type `raw(16)` (UUID).
- The implementation type of the RAP BO is **managed**.

For details about the syntax for defining managed early numbering for primary key fields in the behavior definition, see [CDS BDL - field numbering \(ABAP Keyword Documentation\)](#).

The key field must be read-only if you intend the key fields to be filled internally only. If you do not set the key field to read-only, the key value can also be given by the consumer. However, use cases for this [optional external numbering](#) are rare, as it is untypical for the consumer to fill in a UUID value.

Example

The following simplified BO exemplifies the use case to have the framework generate a UUID key for a managed BO.

1. Database table `/dmo/travel_uuid`

The basis for the data model is a database table similar to `/DMO/travel` (that is included in the [ABAP Flight Reference Scenario](#)). To demonstrate managed numbering, we need a key field with UUID type (`raw(16)`).

```
@EndUserText.label : 'Flight Reference Scenario: Managing Travels'
@AbapCatalog.enhancementCategory : #NOT_EXTENSIBLE
@AbapCatalog.tableCategory : #TRANSPARENT
@AbapCatalog.deliveryClass : #A
@AbapCatalog.dataMaintenance : #LIMITED
define table /dmo/travel_uuid {
  key client      : abap.clnt not null;
  key travel_uuid : sysuuid_x16 not null;
  travel_id       : /dmo/travel_id;
  agency_id       : /dmo/agency_id;
  customer_id     : /dmo/customer_id;
  begin_date      : /dmo/begin_date;
  end_date        : /dmo/end_date;
```

```

    @Semantics.amount.currencyCode : '/dmo/travel_uuid.currency_code'
    booking_fee      : /dmo/booking_fee;
    @Semantics.amount.currencyCode : '/dmo/travel_uuid.currency_code'
    total_price      : /dmo/total_price;
    currency_code     : /dmo/currency_code;
    description       : /dmo/description;
    overall_status    : /dmo/overall_status;
    created_by        : syuname;
    created_at        : timestamp;
    last_changed_by   : syuname;
    last_changed_at   : timestamp;
}

```

2. CDS root view /DMO/I_TRAVEL_UUID

The data model for the simplified travel scenario is defined in a CDS view that is similar to /DMO/I_TRAVEL_M. The difference in the UUID scenario is that the UUID key field is obligatory. In addition to the UUID key field, the travel CDS view also defines the non-key field `travel_id`, which assigns an additional semantic identifier. This semantic key is not a key element in the scenario. It simply serves as an additional, more simple, identifier that is easier to read by consumers. The value for this field is assigned via a determination in the behavior pool. **Expand the following code sample to view the source code**

Sample Code

```

@AbapCatalog.sqlViewName: '/dmo/itraveluuid'
@AbapCatalog.compiler.compareFilter: true
@AbapCatalog.preserveKey: true
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'Root View with Travel UUID'

define view /dmo/i_travel_uuid as select from /dmo/travel_uuid

{ ///dmo/travel_uuid
  key travel_uuid,
  travel_id,
  agency_id,
  customer_id,
  begin_date,
  end_date,
  @Semantics.amount.currencyCode: 'currency_code'
  booking_fee,
  @Semantics.amount.currencyCode: 'currency_code'
  total_price,
  @semantics.currencyCode: true
  currency_code,
  description,
  overall_status,

  @Semantics.user.createdBy: true
  created_by,
  @Semantics.systemDateTime.createdAt: true
  created_at,
  @Semantics.user.lastChangedBy: true
  last_changed_by,

```

```
@Semantics.systemDateTime.lastChangedAt: true
  last_changed_at
}
```

3. Behavior Definition /DMO/I_TRAVEL_UUID

For the root CDS view /DMO/I_TRAVEL_UUID, the following behavior definition defines managed numbering for the UUID key field `travel_uuid`.

The key UUID field is defined as `read only`. Not setting the key field to `read only` causes [optional external numbering](#), in which it is also possible for the consumer to set the value manually.

In this scenario, we want to set the value for the additional identifier `travel_id` by a determination. Therefore, this field is also set to read-only.

i *Expand the following code sample to view the source code*

Sample Code

```
managed;

define behavior for /dmo/i_travel_uuid alias Travel
implementation in class /dmo/bp_i_travel_uuid unique
persistent table /DMO/TRAVEL_UUID
lock master
etag master last_changed_at
{
  field ( read only, numbering : managed ) travel_uuid;
  field ( read only ) travel_id;
  create;
  update;
  delete;
  /** optional determination to determine the travel ID
  determination CreateKeys on modify { create; }
}
```

4. Optional: Determination to determine the value for semantic travel identifier

The value for the `travel_id` field is assigned via a determination. In this way, the consumer does not have to enter the semantic identifier manually. It is assigned on CREATE.

The determination is defined in the behavior definition and implemented in the behavior pool. For more information, see [Determinations](#).

For the implementation, get a new travel id and execute an EML `modify` on the newly created instance to assign the new travel ID to the instance. To make sure that the determination result does not change in case it is executed twice under the same circumstances, we have to make sure that a new travel id is only assigned if the the travel id field is initial.

i Note

To ensure that one travel ID is only assigned once, you can use a number range object.

The following code example does not show how to get this new travel ID. It is just suggested by the method call of `get_number`.

i Expand the following code sample to view the source code

Sample Code

```

CLASS lhc_Travel_UUID DEFINITION INHERITING FROM cl_abap_behavior_handler.
  PRIVATE SECTION.

    METHODS CreateKeys FOR DETERMINATION Travel~CreateKeys
      IMPORTING keys FOR Travel.
ENDCLASS.

CLASS lhc_Travel IMPLEMENTATION.

  METHOD CreateKeys.

    " Read entity to check if travel id is initial
    READ ENTITIES OF zmv_i_travel_uuid IN LOCAL MODE
    ENTITY travel
      FIELDS ( travel_id )
      WITH CORRESPONDING #( keys )
    RESULT DATA(lt_travel_result).

    " only modify entities where no travel_id is given
    DELETE lt_travel_result WHERE travel_id IS NOT INITIAL.

    LOOP AT lt_travel_result ASSIGNING FIELD-SYMBOL(<fs_travel>).
      " get a new travel ID
      DATA(lv_travel_id_new) = get_number( ).
      <fs_travel>-travel_id = lv_travel_id_new.
    ENDLLOOP.

    MODIFY ENTITIES OF zmv_i_travel_uuid IN LOCAL MODE
    ENTITY travel
      UPDATE FROM VALUE #( FOR travel IN lt_travel_result
        ( travel_uuid = travel-%key-travel_uuid
          travel_id   = travel-travel_id
          %control-travel_id = if_abap_behv=>mk-on ) ).
  ENDMETHOD.
ENDCLASS.

```

5. UI Preview

The following figure shows the creation of a new travel instance when using managed numbering for the UUID key field and a determination to assign the semantic identifier on CREATE.

For this example to run like it is shown below, you must add UI annotations to the elements in the CDS view.

i Note

This example scenario to automatically draw UUID values for primary key fields is highly simplified. Basic important elements of a travel scenario, such as eTag or BO projection, are left aside to focus on the principle of managed early numbering.

Managed Early Numbering in UUID Scenario

Managed Numbering in UUID Scenario

Related Information

[Numbering](#)

Editing Language-Dependent Fields

You can behavior-enable denormalized text elements in business objects by augmenting incoming requests with modify requests for the denormalized elements. This topic guides you through the steps to enable end users to maintain language-dependent texts in their log-on language.

Context

Language-dependent text is usually stored in a separate text table that contains the same text in different languages. One data set is only uniquely identifiable if the field that determines the language is part of the key. The ABAP RESTful Application Programming Model offers an option to expose language-dependent text elements for a Fiori UI in the log-on language of the browser.

The exposure of such a language-dependent fields is usually done by denormalizing the text field in the projection view and adding the syntax element `localized`. This reduces the cardinality of the one-to-many association to a one-to-one relationship, as the text provider view is filtered by the log-on language and the result set of the association is reduced to data sets of one language. The structure of such a scenario is illustrated in [Getting Language-Dependent Text in Projection Views](#).

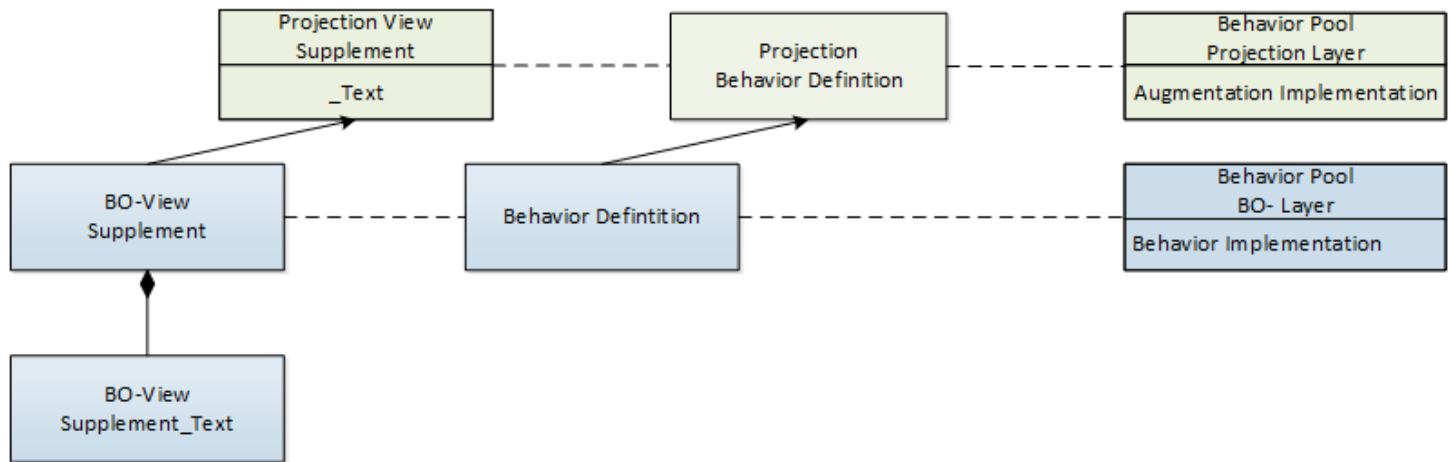
For business objects that maintain entities with language-dependent text fields, you can behavior-enable denormalized text elements to enable an end user to create and update the corresponding text in the log-on language; even if this text is taken from a separate text view.

To include denormalized fields, for example language-dependent text fields, into transactional handling, you must modify-enable them in the projection behavior definition. Such fields can then also be defined with field characteristics and dynamic field control.

For more information about the syntax, see [CDS BDL - field characteristics, projection BDEF \(ABAP - Keyword Documentation\)](#).

Example Scenario

To illustrate the functionality of editing language-dependent text by using an augmentation implementation, we use an example from the ABAP Flight Reference Scenario. We use a business object that maintains supplements that an end user can book for a flight booking in a travel business object. The following diagram shows the structure of the supplement business object.



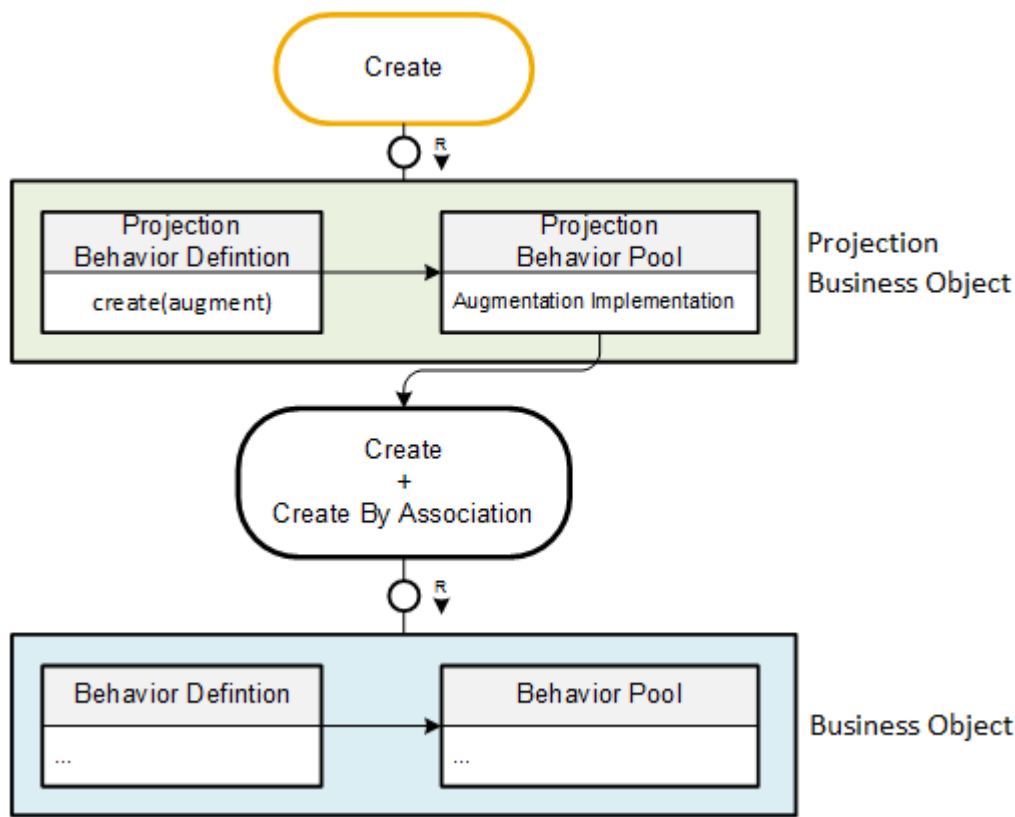
Structure of Supplement BO for Editing Text Fields

Runtime Aspects

Text elements are stored in a separate database table and managed by a separate CDS view to be able to store texts in multiple languages. Due to the filtering of `localized` in the projection view, the one-to-n-relationship is reduced to a one-to-one relationship; and as the text field is denormalized in the main CDS view, it can be displayed together with the fields of the main view on the UI. Thus, the end user cannot differentiate between fields of the main view and text elements that derive from a separate CDS view. Consequently, the end user must be able to create and update fields from a text provider view in the same manner as other text fields.

During runtime, this means, that a create request for an instance that has a denormalized text field must be transformed to two requests: A `CREATE` request for the main view and a `CREATE_BY_ASSOCIATION` for the associated text provider view.

The ABAP RESTful Application Programming Model offers the option to implement such a transformation before the `CREATE` request reaches the application buffer. The relevant operations in the projection business object are augmented. The logic of the augmentation must be implemented in the corresponding behavior pool in the method for augmentation. The following diagram illustrates the runtime process for the above mentioned example scenario.



Runtime for Editing Denormalized Text Fields

When an **UPDATE** request is sent, the augmentation method must transform the request in an **UPDATE** request for the main entity and in an **UPDATE** request for the text provider entity.

i Note

The behavior pool for a projection behavior definition generates two separate methods, if both, the **CREATE** and the **UPDATE** operation, are augmented in the behavior definition. However, it is best practice to implement the logic for both operations in one single method for augmented operations. The reason for this is to support requests that create and update an instance in one logical unit of work. In such cases, the `%cid_ref` must be used to refer to a newly created instance in the implementation.

A step-by-step description for such an augmentation implementation is provided in [Editing Text for Supplements](#).

Editing Text for Supplements

By using an example of the ABAP Flight Reference Scenario, this topic guides you through the steps to enable end users to maintain language-dependent product descriptions for supplements in their log-on language.

The example in this topic is taken from the ABAP Flight Reference Scenario, which you can download from GitHub, see [ABAP Flight Reference Scenario](#).

The supplement business object in package `/DM0/FLIGHT_REUSE` is modeled with the CDS root view entity `/DM0/I_Supplement` and its projection view `/DM0/C_Supplement`. The text provider view is in a compositional relationship and contains the text element and the language field. The text element is denormalized in the projection view and the logic on how to edit this denormalized is implemented in the behavior pool for the projection behavior definition.

Prerequisites

- You have modeled language-dependent text elements as described in [Getting Language-Dependent Text in Projection Views](#).

- The view with the denormalized text and the text provider view are in a parent-child relationship in your business object.
- The business object is create- and update-enabled.

Procedure

1. In your projection behavior definition, define augmentation for the create and the update operation.

```
projection implementation in class /dmo/bp_c_supplement unique;
strict(2);
use draft;

define behavior for /DMO/C_Supplement alias Supplement
{
    use create ( augment );
    use update ( augment );
    use delete;
}
```

2. To behavior-enable the denormalized text field, add it to the projection behavior definition as a modifiable field.

```
projection implementation in class /dmo/bp_c_supplement unique;
strict(2);
use draft;

define behavior for /DMO/C_Supplement alias Supplement
{
    use create ( augment );
    use update ( augment );
    use delete;

    field ( modify ) SupplementDescription;
}
```

3. In the behavior pool for the projection business object, add a method for the augmentation implementation with an importing parameter for CREATE and one for UPDATE.

i Note

If you don't have an implementation class for the projection behavior definition yet, you will receive a warning that prompts you to create one. The behavior pool for the projection BO must be indicated in the projection behavior definition with `implementation in class ProjectionBehaviorPool unique;`. Once you add this to the projection behavior definition, you can create the behavior pool by using the quick fix that is provided in the projection behavior definition.

The behavior pool is generated including two methods to implement the augmentation logic for CREATE and UPDATE separately. To be able to reference a newly created instance in an UPDATE logic, the implementation for CREATE and UPDATE must be done in one method.

```
CLASS lhc_Supplement DEFINITION INHERITING FROM cl_abap_behavior_handler.
    PRIVATE SECTION.
```

```
    METHODS augment FOR MODIFY
        IMPORTING entities_create FOR CREATE Supplement
                entities_update FOR UPDATE Supplement.
```

```
ENDCLASS.
```

```
CLASS lhc_Supplement IMPLEMENTATION.
```

```
    METHOD augment.
    ENDMETHOD.
```

ENDCLASS.

4. The augmentation implementation must handle all possible incoming request options for the behavior-enabled field:

Incoming Request	Augmentation for Supplement Text Entity
CREATE of a new supplement instance including new product name.	CREATE_BY_ASSOCIATION of a new supplement text instance. i Note The CBA uses the incoming %CID as %CID_REF to reference the parent instance.
UPDATE of a supplement instance with already existing product name.	UPDATE of an already existing supplement text instance. i Note The link table of a READ_BY_ASSOCIATION is used to check if entry for product name already exists.
UPDATE of a newly created supplement instance that has not yet been saved to the database.	UPDATE of a supplement text instance that only exists in the transactional buffer. i Note The %CID_REF of the incoming UPDATE refers to the %CID of the possibly newly created supplement instance.
UPDATE of an already existing supplement instance without entry for product name.	CREATE_BY_ASSOCIATION of a supplement text instance for already existing supplement instance.

Implement the logic for augmenting the operations CREATE and UPDATE:

- Handle the importing parameters of CREATE and UPDATE separately.
- Loop at the importing parameter for CREATE to check for incoming create requests. Fill the relating table for modify augmenting requests and create an internal table with the values for the CREATE_BY_ASSOCIATION of supplement text. Augment the incoming CREATE request with an EML MODIFY AUGMENTING statement.
- If the incoming request is an UPDATE, read the associated supplement text instances for the requested supplement instances. Loop at all imported instances where the field SupplementDescription is to be updated and handle the previously described cases for augmenting the incoming UPDATE request. Exclude non-existing supplement instances from this processing. Depending on the use case an internal table for CREATE BY ASSOCIATION or an internal table for UPDATE must be filled. Augment the incoming UPDATE request with an EML MODIFY AUGMENTING statement for the supplement and the supplement text entity.

i Expand the following code sample to view the source code of method augment.

Sample Code

METHOD augment.

```
DATA: suppltxt_for_new_suppl TYPE TABLE FOR CREATE /DM0/I_Supplement\Supplement1
      suppltxt_for_existing_suppl TYPE TABLE FOR CREATE /DM0/I_Supplement\Supplement1
      supplementtext_update TYPE TABLE FOR UPDATE /DM0/I_SupplementText.
DATA: relates_create TYPE abp_behv_relating_tab,
```

```

        relates_update TYPE abp_behv_relating_tab,
        relates_cba    TYPE abp_behv_relating_tab.
DATA: suppl_text_tky_link TYPE STRUCTURE FOR READ LINK /DMO/I_Supplement\\Supplement\\_s
      suppl_text_tky      LIKE suppl_text_tky_link-target.

"Handle create requests including SupplementDescription
LOOP AT entities_create INTO DATA(supplement_create).
  "Count Table index for uniquely identifiably %cid on creating supplementtext
  APPEND sy-tabix TO relates_create.

  "Direct the Supplement Create to the corresponding SupplementText Create-By-Ass
  APPEND VALUE #( %cid_ref      = supplement_create-%cid
                  %is_draft     = supplement_create-%is_draft
                  %key-supplementid = supplement_create-%key-SupplementID
                  %target       = VALUE #( ( %cid      = |CREATETEXT|
                                           %is_draft   = supplement_
                                           %key-supplementid = supplement_
                                           languagecode = sy-langu
                                           description  = supplement_
                                           %control    = VALUE #( si
                                                                la
                                                                de
                                                                ) )
                  ) TO suppltext_for_new_suppl.
ENDLOOP.

MODIFY AUGMENTING ENTITIES OF /DMO/I_Supplement
  ENTITY Supplement
    CREATE BY \_SupplementText
    FROM suppltext_for_new_suppl
    RELATING TO entities_create BY relates_create.

IF entities_update IS NOT INITIAL.
  READ ENTITIES OF /DMO/I_Supplement
    ENTITY Supplement BY \_SupplementText
    FROM CORRESPONDING #( entities_update )
    LINK DATA(link)
    FAILED DATA(link_failed).

  "Handle update requests
  LOOP AT entities_update INTO DATA(supplement_update) WHERE %control-SupplementDescrip

    CHECK NOT line_exists( link_failed-supplement[ KEY draft %tky = CORRESPONDING #( s
      OR line_exists( suppltext_for_new_suppl[ KEY cid %cid_ref = supplement_update-%c
      DATA(tabix) = sy-tabix.

    "Create variable for %tky for target entity instances
    suppl_text_tky = CORRESPONDING #( supplement_update-%tky ) .
    suppl_text_tky-LanguageCode = sy-langu.

    "If a suppl_text with sy-langu already exists, perform an update. Else perform a cr
    IF line_exists( link[ KEY draft source-%tky = CORRESPONDING #( supplement_update-%
      target-%tky = CORRESPONDING #( suppl_text_tky ) ]

      APPEND tabix TO relates_update.

      APPEND VALUE #( %tky      = suppl_text_tky
                     %cid_ref   = supplement_update-%cid_ref
                     description = supplement_update-SupplementDescription
                     %control   = VALUE #( description = supplement_update-%cont
                     ) TO supplementtext_update.

      "If suppl_text was created in the current MODIFY, perform an update based on %cid
      ELSEIF line_exists( suppltext_for_new_suppl[ KEY cid %is_draft = supplement_update
        %cid_ref = supplement_update-%ci

        APPEND tabix TO relates_update.

        APPEND VALUE #( %tky      = suppl_text_tky
                       %cid_ref   = suppltext_for_new_suppl[ %is_draft = supplemer
                       %cid_ref   = supplement_up
                       description = supplement_update-SupplementDescription
                       %control   = VALUE #( description = supplement_update-%cont

```

```

) TO supplementtext_update.

"If suppl_text with sy-langu does not exist yet for corresponding supplement
ELSE.
  APPEND tabix TO relates_cba.

  "Direct the Supplement Update to the corresponding SupplementText Create-By-Assoc
  APPEND VALUE #( %tky      = CORRESPONDING #( supplement_update-%tky )
                  %cid_ref = supplement_update-%cid_ref
                  %target  = VALUE #( ( %cid      = |UPDATETEXTCID{ tabix }|
                                       languagecode = sy-langu
                                       %is_draft    = suppl_text_tky-%is_draft
                                       description  = supplement_update-Supplemer
                                       %control     = VALUE #( languagecode = if_
                                                                description  = si
                                                                ) )
                  ) )
) TO suppltext_for_existing_suppl.

ENDIF.

ENDLOOP.
ENDIF.

MODIFY AUGMENTING ENTITIES OF /DMO/I_Supplement
ENTITY SupplementText
  UPDATE FROM supplementtext_update
  RELATING TO entities_update BY relates_update
ENTITY Supplement
  CREATE BY \_SupplementText
  FROM suppltext_for_existing_suppl
  RELATING TO entities_update BY relates_cba.

ENDMETHOD.

```

Using Type and Control Mapping

Whenever existing legacy code and data types are to be reused in behavior pools of business objects, then you need to perform a mapping between CDS field names and types and corresponding legacy field names and types.

Use Case

You can use this kind of mapping in applications that include unmanaged or managed business objects based on CDS entities on the one hand, and legacy data types (ABAP Dictionary types) (and functionality) that are generally older.

This is particularly significant for the unmanaged implementation type, which essentially represents a kind of wrapper for existing legacy functionality.

Also with the managed implementation type, it can happen that, for example, the code for a determination or validation already exists, but is based on "old" (legacy) data types.

When accessing the legacy code, the developer would normally have to use "corresponding with mapping" in many places to map input derived types (type table for create, type table for action import) to legacy types and vice versa to map legacy results to output derived types (type table for action result, type table for read result, failed).

In addition, the types used to represent the control information also can differ significantly in the legacy code and the current ABAP programming model:

In some legacy scenarios (especially the ones using BAPIs), in addition to the dictionary type directly corresponding to the entity, there is another one that contains (ideally) the same fields as that type, but these all have the type C (1) (like the control

structures in BAPIs). This type has the same function as the [%CONTROL](#) structure in derived types, which indicates that fields in the main structure that are accessed by an operation (update, read, and so on) Such type pairs are often used in BAPIs, for example BAPIAD2VD/BAPIAD2VDX. The control data element is BAPIUPDATE with the type C(1).

The solution is now to introduce a central, declarative mapping within the behavior definition instead of many individual corresponding statements. This improves maintainability of the application's source code.

Defining Type and Control Mapping in the Behavior Definition

A mapping sets the entity in relation to any other structured ABAP Dictionary type (legacy type). It is introduced in the behavior definition for an unmanaged or managed implementation type with the keyword `mapping for`.

Syntax: Mapping statement in the behavior definition

```
[implementation] unmanaged|managed [implementation in class ABAP_CLASS [unique]];
define behavior for Entity [alias AliasedName]
...

{
  mapping for ...

  ...
}
```

Variant 1: Field Mapping

For details about the syntax for field mapping with corresponding, see [CDS BDL - type mapping \(ABAP Keyword Documentation\)](#)

The addition `corresponding` automatically maps fields of the same name to each another. The corresponding fields of different names can be specified through explicitly listed field pairs.

The `corresponding` addition can also be omitted, which is not recommended in general, because then the automatic mapping of fields with the same name is lost.

If no renaming of the fields is required, the short form can be used:

Syntax (short form):

```
mapping for LegacyType corresponding;
```

i Note

Mapping can be partial (legacy type contains fields that do not match any fields in CDS).

Variant 2: Field and Control Type Mapping

Using the following syntax, a mapping definition can be made simultaneously for a main field type `LegacyType` and a control type `ControlType`:

Syntax: Field type and control mapping with corresponding

```
mapping for LegacyType control ControlType corresponding
{
  EntityField1 = LegacyField1;
```

```

    EntityField2 = LegacyField2;
    ...
}

```

If no renaming of the fields is required, the short form can be used:

Syntax (short form):

```
mapping for LegacyType control ControlType corresponding;
```

Usually, it is assumed that the fields in the main and control type have the same name.

A different field mapping in { ... } is specified as:

```
EntityField = LegacyField control ControlField;
```

i Note

For all control type fields, the type C(1) or X(1) is expected.

Using Type Mapping in ABAP

The reference to mappings that are defined in a behavior definition is done by some special variants of the ABAP `corresponding` operator.

Syntax: Input mapping (entity type to legacy type)

```

data:
    legacy_var type LegacyType,
    entity_var type structure|table for create|update|delete entity.
...

legacy_var = corresponding #( entity_var mapping from entity ).

```

The addition `from entity` assigns `entity_var` field-wise to data object `legacy_var` according to the mapping definition for `LegacyType` in the behavior definition.

This mapping works fine if the type of `entity_var` is an input derived type, or the entity type itself, or the table type of the entity.

Syntax: Output mapping (from legacy type to entity type)

```

data:
    legacy_var type LegacyType,
    entity_var type table for read result entity.
...

entity_var = corresponding #( legacy_var mapping to entity ).

```

The addition `to entity` assigns `legacy_var` field-wise to data object `entity_var` according to the mapping definition for `LegacyType` in the behavior definition.

This mapping works fine if the type of `entity_var` is an output derived type, or the entity type itself, or the table type of the entity.

Using Control Type Mapping in ABAP

Control mapping is about supporting the actual semantics of the `%control` fields: They indicate which of the fields should be, for example, changed by an update or read by a read operation.

For the move between an input derived type (type table for create, type table for action import) and a target structure, this means that only the fields for which the corresponding control field is set should be moved.

Syntax: Using control

```
target = corresponding #( source using control ).
```

The structure or table `source` must be a derived type that also includes the `%control` structure.

For example, if the source has a field `amount`, then `%control-amount` decides whether to move the value of `amount` to the corresponding field of `target` – only if the value of the corresponding `control` field is non-initial.

By combining with the additional mapping `from entity` described above, also field mapping specified in the behavior definition is effective:

Syntax: Using control with field mapping

```
target = corresponding #( source mapping from entity using control ).
```

Conversely, there is an addition `changing control` that allows the `%control` fields of a derived type to be set from a `%control-less` source structure – based on the criterion of whether the corresponding field is non-initial.

Syntax: Changing control

```
target = corresponding #( source changing control ).
```

Related Information

[Derived Data Types](#)

[Implicit Response Parameters](#)

Exposing Messages on the UI and Message Mapping

Messages

Messages offer an important way to guide and validate consumer and user actions, and help to avoid and resolve problems. Thus, messages are important to communicate problems to a consumer or user. Well-designed messages help to recognize, diagnose, and resolve issues. That's why it's important to always use messages consistently and optimize the interaction as a whole. Consequently, errors and warnings that require action should be clearly stated and described in a way that helps to resolve the issue quickly and efficiently. It's recommended to provide a message for each entry in the fail structure to provide additional information.

This topic explains end-to-end how to receive and adapt messages from a different business object.

Related Information

[Transition Messages](#)

[State Messages](#)

[Example Business Object Structure](#)

[Creating a Message Exception Class](#)

[Exposing Messages for a Sample Business Object with Draft Capabilities](#)

[Mapping Messages Between Business Objects](#)

Example Business Object Structure

Example

The data model for the following message implementation examples is based on the data model for business objects with draft capabilities, but was reduced to highlight the important points regarding messages. The business object model for this example contains the two entities **Travel** and **Booking**. For more contextual information about annotations and syntax, refer to [Creating CDS View Entities](#). This example focuses on the relevant points for messages.

Data Definition: /DMO/I_Travel_Messages

The data source for the travel root view is the database table /DMO/A_TRAVEL_D. All fields from the database table are used for the data model.

i *Expand the following code sample to view the source code Of /DMO/I_Travel_Messages.*

≡ Sample Code

```
@EndUserText.label: 'Travel View Entity for Messages.'
@AccessControl.authorizationCheck: #NOT_REQUIRED

define root view entity /DMO/I_Travel_Messages
as select from /dmo/a_travel_d

composition [0..*] of /DMO/I_BOOKING_Messages as _Booking

association [0..1] to /DMO/I_Agency      as _Agency on $projection.AgencyID      = _Agency.Agen
association [0..1] to /DMO/I_Customer  as _Customer on $projection.CustomerID  = _Customer.Cu
association [0..1] to I_Currency        as _Currency on $projection.CurrencyCode = _Currency.Cu

{ ///dmo/a_travel_d
  key travel_uuid          as TravelUUID,

    travel_id              as TravelID,
    agency_id              as AgencyID,
    customer_id            as CustomerID,
    begin_date             as BeginDate,
    end_date               as EndDate,
    @Semantics.amount.currencyCode: 'CurrencyCode'
    booking_fee            as BookingFee,
    @Semantics.amount.currencyCode: 'CurrencyCode'
    total_price            as TotalPrice,
    currency_code          as CurrencyCode,
    description            as Description,
    overall_status         as OverallStatus,
```

```

@Semantics.user.createdBy: true
local_created_by      as LocalCreatedBy,
@Semantics.systemDateTime.createdAt: true
local_created_at      as LocalCreatedAt,
@Semantics.user.lastChangedBy: true
local_last_changed_by as LocalLastChangedBy,
//local ETag field --> OData ETag
@Semantics.systemDateTime.localInstanceLastChangedAt: true
local_last_changed_at as LocalLastChangedAt,

//Associations
_booking,

_customer,
_currency
}

```

Data Definition: /DMO/I_Booking_Messages

The data source for the booking view is the database table /DMO/A_BOOKING_D. All fields from the database table are used for the data model.

i Expand the following code sample to view the source code of /DMO/I_Booking_D.

Sample Code

```

@EndUserText.label: 'Booking View Entity for Messages'
@AccessControl.authorizationCheck: #NOT_REQUIRED

define view entity /DMO/I_Booking_Messages
  as select from /dmo/a_booking_d

  association          to parent /DMO/I_Travel_Messages          as _Travel          on $projection.Trav

  association [1..1] to      /DMO/I_Customer                    as _Customer        on $projection.CustomerID
  association [1..1] to      /DMO/I_Carrier                     as _Carrier          on $projection.AirlineID
  association [1..1] to      /DMO/I_Connection                  as _Connection       on $projection.AirlineID
                                                              and $projection.ConnectionI

{ ///dmo/a_booking_d
  key booking_uuid          as BookingUUID,
  parent_uuid               as TravelUUID,

  booking_id                as BookingID,
  booking_date              as BookingDate,
  customer_id               as CustomerID,
  carrier_id                as AirlineID,
  connection_id             as ConnectionID,
  flight_date               as FlightDate,
  @Semantics.amount.currencyCode: 'CurrencyCode'
  flight_price              as FlightPrice,
  currency_code             as CurrencyCode,
  booking_status            as BookingStatus,

  //local ETag field --> OData ETag
  @Semantics.systemDateTime.localInstanceLastChangedAt: true
  local_last_changed_at    as LocalLastChangedAt,

  //Associations
  _Travel,
  _BookingSupplement,

  _Customer,
  _Carrier,
  _Connection
}

```

}

Behavior Definition: /DMO/I_Travel_Messages

This data model focuses on illustrating how messages work in different contexts. Consequently, the behavior definition uses a reduced function scope for both entities. Each entity implements an action and a validation to illustrate state messages and transition message behavior for each of these cases.

Travel

The travel entity implements:

- Validation `validateCustomer`: Checks if the customer ID that is entered by the consumer is valid
- Action `setToBooked`: Changes the booking status from **N**(new) to **B** (booked)

Booking

The booking entity implements:

- Validation `validateDates`: Checks if the entered **BookingDate** is valid
- Determination `validateBookingDate`: Checks whether the selected booking date is between the `StartDate` and `EndDate` of the parent travel entity.

i Expand the following code sample to view the source code of **/DMO/I_Travel_Messages**.

Sample Code

```
managed;
with draft;

define behavior for /DMO/I_Travel_Messages alias Travel
implementation in class dmo_travel_messages_demo unique
persistent table /dmo/a_travel_d
draft table zfdtravelmessage
lock master
total etag LocalLastChangedAt
//authorization master ( instance )
etag master LocalLastChangedAt
{
  create;
  update;
  delete;
  association _Booking { create; with draft; }

  field ( numbering : managed, readonly ) TravelUUID;
  validation validateCustomer on save { field CustomerID; }
  validation validateDates on save { create; field BeginDate, EndDate; }
  action setToBooked result [1] $self;

  draft determine action Prepare
  {
    validation validateCustomer;
    validation validateDates;
    validation booking~validateDates ;
    determination booking~setBookingDate;
  }

  mapping for /dmo/a_travel_d
  { AgencyID = agency_id;
```

```

    BeginDate = begin_date;
    BookingFee = booking_fee;
    CurrencyCode = currency_code;
    CustomerID = customer_id;
    Description = description;
    EndDate = end_date;
    LocalCreatedAt = local_created_at;
    LocalCreatedBy = local_created_by;
    LocalLastChangedAt = local_last_changed_at;
    LocalLastChangedBy = local_last_changed_by;
    OverallStatus = overall_status;
    TotalPrice = total_price;
    TravelID = travel_id;
    TravelUUID = travel_uuid; }
}

define behavior for /DMO/I_Booking_Messages_DEMO alias Booking
implementation in class DMO_i_booking_messages_demo unique
persistent table /dmo/a_booking_d
draft table zfdbookinmessage
lock dependent by _Travel
//authorization dependent by <association>
etag master LocalLastChangedAt
{
    update;
    delete;

    association _Travel { with draft; }

    field ( readonly ) TravelUUID;
    field ( numbering : managed, readonly ) BookingUUID;

    validation validateDates on save { field BookingDate; create; }

    determination setBookingNumber on save { create; }

    mapping for /dmo/a_booking_d
    { AirlineID = carrier_id;
      BookingDate = booking_date;
      BookingID = booking_id;
      BookingStatus = booking_status;
      BookingUUID = booking_uuid;
      ConnectionID = connection_id;
      CurrencyCode = currency_code;
      CustomerID = customer_id;
      FlightDate = flight_date;
      FlightPrice = flight_price;
      LocalLastChangedAt = local_last_changed_at;
      TravelUUID = parent_uuid; }
}

```

Creating a Message Exception Class

This topics explains end-to-end how to create a message-wrapper class using a specific implementation example.

Context

You want to provide and expose messages on the UI. The procedure is identical for business objects of all types, however the message behavior of state messages may vary if your business object doesn't have draft capabilities. For more information about how state messages behave in different contexts, refer to [State Messages](#).

For more information about the example business object structure, refer to [Example Business Object Structure](#).

Procedure

1. Create your messages in a T100-message class. This example uses the following T100-messages from the message class /DMO/CM_TRAVEL_MESSAGES:

Locked	Number	Short Text	Self Expl	Last Changed By	Changed On
	001	Enter Customer ID for Travel&1.	☒		2020-10-05
	002	Customer &1 doesn't exist.	☒		2020-10-05
	003	Enter a begin date for travel &1.	☒		2020-10-05
	004	Enter an end date for Travel &1.	☒		2020-10-05
	005	Select a begin date before end date &2 for Travel &2.	☒		2020-10-05
	006	Begin date &1 must be on or after system date.	☒		2020-10-05
	007	Enter an AirlineID for Booking &1.	☒		2020-10-05
	008	Enter a ConnectionID for Booking &1.	☒		2020-10-05
	009	No Flight for FlightDate &3.	☒		2020-10-05
	010	Action was successful.	☒		2020-10-05
	011	Select a BookingDate between Start and EndDate for Travel &1..	☒		2020-10-07

2. Create an ABAP-class as message-wrapper to wrap the T100-messages as message constants. This wrapper-class takes care of formatting message variables in accordance with their data type, especially date and time types. The date and time formatting is derived from backend user settings. If front- and backend user settings differ, messages always reflect the backend user settings. The wrapper-class inherits from CX_STATIC_CHECK which takes care of the type formatting. For more information about CX_STATIC_CHECK, refer to [Exception Categories \(ABAP Keyword Documentation\)](#). Furthermore, the class implements the relevant message interfaces IF_ABAP_BEHV_MESSAGE, which implements the interfaces IF_T100_DYN_MSG and IF_T100_MESSAGE. For more details about the interfaces, refer to [Messages - System Interfaces \(ABAP Keyword Documentation\)](#).

```

CLASS DMO_MESSAGEWRAPPER DEFINITION
  PUBLIC
    INHERITING FROM CX_STATIC_CHECK
  FINAL
  CREATE PUBLIC .

  PUBLIC SECTION.

    INTERFACES IF_ABAP_BEHV_MESSAGE .

  PROTECTED SECTION.
  PRIVATE SECTION.
ENDCLASS.

CLASS DMO_MESSAGEWRAPPER IMPLEMENTATION.
ENDCLASS.
```

3. Create variables for all fields of the data definition you want to pass as message variables with the same type as the fields in the data definitions and add them as optional importing parameters to the message constructor. For this example, the fields travel_id, end_date, begin_date, flight_date, Customer_ID and booking_id are relevant for the message texts. The variable types are derived from the data definition the fields were contained in. The message severity can be set with the severity variable severity TYPE if_abap_behv_message=>t_severity in the message constructor.

```

//Relevant fields from the data definitions
DATA:
  mv_attr1          TYPE string,
  mv_attr2          TYPE string,
  mv_attr3          TYPE string,
  mv_attr4          TYPE string,
  mv_travel_id      TYPE /dmo/travel_id,
  mv_booking_id     TYPE /dmo/booking_id,
  mv_customer_id    TYPE /dmo/customer_id,
  mv_connection_id  TYPE /dmo/connection-connection_id,
```

```

mv_begin_date      TYPE /dmo/begin_date,
mv_end_date        TYPE /dmo/end_date,
mv_booking_date    TYPE /dmo/booking_date,
mv_flight_date     TYPE /dmo/flight_date.

METHODS constructor
IMPORTING
    textid          LIKE if_t100_message=>t100key OPTIONAL
    attr1           TYPE string OPTIONAL
    attr2           TYPE string OPTIONAL
    attr3           TYPE string OPTIONAL
    attr4           TYPE string OPTIONAL
    previous        LIKE previous OPTIONAL
    travel_id       TYPE /dmo/travel_id OPTIONAL
    booking_id      TYPE /dmo/booking_id OPTIONAL
    customer_id     TYPE /dmo/customer_id OPTIONAL
    connection_id   TYPE /dmo/connection-connection_id OPTIONAL
    begin_date      TYPE /dmo/begin_date OPTIONAL
    end_date        TYPE /dmo/end_date OPTIONAL
    booking_date    TYPE /dmo/booking_date OPTIONAL
    flight_date     TYPE /dmo/flight_date OPTIONAL
    severity        TYPE if_abap_behv_message=>t_severity OPTIONAL.

```

4. Implement the constructor and map the importing parameters to the variables. Set the severity and the textID.

CLASS DMO_MESSAGEWRAPPER IMPLEMENTATION.

METHOD CONSTRUCTOR.

SUPER->CONSTRUCTOR(PREVIOUS = PREVIOUS).

```

MV_ATTR1          = ATTR1.
MV_ATTR2          = ATTR2.
MV_ATTR3          = ATTR3.
MV_ATTR4          = ATTR4.
MV_TRAVEL_ID      = TRAVEL_ID.
MV_BOOKING_ID     = BOOKING_ID.
MV_CUSTOMER_ID    = CUSTOMER_ID.
MV_CONNECTION_ID  = CONNECTION_ID.
MV_BEGIN_DATE     = BEGIN_DATE.
MV_END_DATE       = END_DATE.
MV_BOOKING_DATE   = BOOKING_DATE.
MV_FLIGHT_DATE    = FLIGHT_DATE.

```

IF_ABAP_BEHV_MESSAGE~M_SEVERITY = SEVERITY.

CLEAR ME->TEXTID.

IF TEXTID IS INITIAL.

IF_T100_MESSAGE~T100KEY = IF_T100_MESSAGE=>DEFAULT_TEXTID.

ELSE.

IF_T100_MESSAGE~T100KEY = TEXTID.

ENDIF.

ENDMETHOD.

ENDCLASS.

5. Create message constant for each T100-message text you have created in the T100-message class.

A generic message constant consists of the following elements::

```

CONSTANTS:
GC_MSGID TYPE SYMSGID VALUE 'Message_Class',

BEGIN OF Message_Constant,
  MSGID TYPE SYMSGID VALUE 'Message_Class',
  MSGNO TYPE SYMSGNO VALUE 'Message_Number',
  ATTR1 TYPE SCX_ATTRNAME VALUE 'Message_Variable',
  ATTR2 TYPE SCX_ATTRNAME VALUE 'Message_Variable',
  ATTR3 TYPE SCX_ATTRNAME VALUE 'Message_Variable',
  ATTR4 TYPE SCX_ATTRNAME VALUE 'Message_Variable',
END OF Message_Constant,
```

A message constant begins and end with the constant name. Ideally, the name reflects the message content to make the message usage easier. MSGID TYPE SYMSGID has the message class assigned as value and MSGNO TYPE SYMSGNO has the message number assigned as value. You can pass message variables with the ATTR 1-4.

A specific implementation for message 001 ("Enter Customer ID for Travel&1.") with the message variable TravelID is implemented analogously to the generic message constant implementation:

```

CONSTANTS:
GC_MSGID TYPE SYMSGID VALUE 'DMO_TRAVEL_MESSAGES',

BEGIN OF CUSTOMERID_NOT_INITIAL,
  MSGID TYPE SYMSGID VALUE 'DMO_TRAVEL_MESSAGES',
  MSGNO TYPE SYMSGNO VALUE '001',
  ATTR1 TYPE SCX_ATTRNAME VALUE 'TRAVEL_ID',
  ATTR2 TYPE SCX_ATTRNAME VALUE '',
  ATTR3 TYPE SCX_ATTRNAME VALUE '',
  ATTR4 TYPE SCX_ATTRNAME VALUE '',
END OF CUSTOMERID_NOT_INITIAL,
```

Results

You have created message-wrapper class for your T100-Messages. The following source code shows the sample implementation of a message-wrapper.

i *Expand the following code sample to view the source code of DMO_MESSAGEWRAPPER.*

Sample Code

```

CLASS DMO_MESSAGEWRAPPER DEFINITION
  PUBLIC
    INHERITING FROM CX_STATIC_CHECK
  FINAL
  CREATE PUBLIC .

  PUBLIC SECTION.

    INTERFACES IF_ABAP_BEHV_MESSAGE .
```

CONSTANTS:

```
GC_MSGID TYPE SYMSGID VALUE 'DMO_TRAVEL_MESSAGES',
```

```
BEGIN OF CUSTOMER_ID_NOT_INITIAL,
  MSGID TYPE SYMSGID VALUE 'DMO_TRAVEL_MESSAGES',
  MSGNO TYPE SYMSGNO VALUE '001',
  ATTR1 TYPE SCX_ATTRNAME VALUE 'TRAVEL_ID',
  ATTR2 TYPE SCX_ATTRNAME VALUE 'Message_Variable',
  ATTR3 TYPE SCX_ATTRNAME VALUE 'Message_Variable',
  ATTR4 TYPE SCX_ATTRNAME VALUE 'Message_Variable',
END OF CUSTOMER_ID_NOT_INITIAL,
```

```
BEGIN OF CUSTOMERID_NOT_EXIST,
  MSGID TYPE SYMSGID VALUE 'DMO_TRAVEL_MESSAGES',
  MSGNO TYPE SYMSGNO VALUE '002',
  ATTR1 TYPE SCX_ATTRNAME VALUE 'Customer_id',
  ATTR2 TYPE SCX_ATTRNAME VALUE '',
  ATTR3 TYPE SCX_ATTRNAME VALUE '',
  ATTR4 TYPE SCX_ATTRNAME VALUE '',
END OF CUSTOMERID_NOT_EXIST,
```

```
BEGIN OF BEGIN_DATE_MISSING,
  MSGID TYPE SYMSGID VALUE 'DMO_TRAVEL_MESSAGES',
  MSGNO TYPE SYMSGNO VALUE '003',
  ATTR1 TYPE SCX_ATTRNAME VALUE 'TRAVEL_ID',
  ATTR2 TYPE SCX_ATTRNAME VALUE '',
  ATTR3 TYPE SCX_ATTRNAME VALUE '',
  ATTR4 TYPE SCX_ATTRNAME VALUE '',
END OF Begin_DATE_MISSING,
```

```
BEGIN OF END_DATE_MISSING,
  MSGID TYPE SYMSGID VALUE 'DMO_TRAVEL_MESSAGES',
  MSGNO TYPE SYMSGNO VALUE '004',
  ATTR1 TYPE SCX_ATTRNAME VALUE 'TRAVEL_ID',
  ATTR2 TYPE SCX_ATTRNAME VALUE '',
  ATTR3 TYPE SCX_ATTRNAME VALUE '',
  ATTR4 TYPE SCX_ATTRNAME VALUE '',
END OF END_DATE_MISSING,
```

```
BEGIN OF END_DATE_BEf_BEGIN_DATE,
  MSGID TYPE SYMSGID VALUE 'DMO_TRAVEL_MESSAGES',
  MSGNO TYPE SYMSGNO VALUE '005',
  ATTR1 TYPE SCX_ATTRNAME VALUE 'BEGIN_DATE',
  ATTR2 TYPE SCX_ATTRNAME VALUE 'TRAVEL_ID',
  ATTR3 TYPE SCX_ATTRNAME VALUE '',
  ATTR4 TYPE SCX_ATTRNAME VALUE '',
END OF END_DATE_BEf_BEGIN_DATE,
```

```
BEGIN OF BEGIN_DATE_WRONG,
  MSGID TYPE SYMSGID VALUE 'DMO_TRAVEL_MESSAGES',
  MSGNO TYPE SYMSGNO VALUE '006',
  ATTR1 TYPE SCX_ATTRNAME VALUE 'BEGIN_DATE',
  ATTR2 TYPE SCX_ATTRNAME VALUE '',
  ATTR3 TYPE SCX_ATTRNAME VALUE '',
  ATTR4 TYPE SCX_ATTRNAME VALUE '',
END OF BEGIN_DATE_WRONG,
```

```
BEGIN OF AIRLINE_ID_IS_INITIAL,
  MSGID TYPE SYMSGID VALUE 'DMO_TRAVEL_MESSAGES',
  MSGNO TYPE SYMSGNO VALUE '007',
  ATTR1 TYPE SCX_ATTRNAME VALUE 'BOOKING_ID',
  ATTR2 TYPE SCX_ATTRNAME VALUE '',
  ATTR3 TYPE SCX_ATTRNAME VALUE '',
  ATTR4 TYPE SCX_ATTRNAME VALUE '',
END OF AIRLINE_ID_IS_INITIAL,
```

```
BEGIN OF CONNECTION_ID_IS_INITIAL,
  MSGID TYPE SYMSGID VALUE 'DMO_TRAVEL_MESSAGES',
  MSGNO TYPE SYMSGNO VALUE '008',
  ATTR1 TYPE SCX_ATTRNAME VALUE 'BOOKING_ID',
  ATTR2 TYPE SCX_ATTRNAME VALUE '',
```



```

    ATTR3 TYPE SCX_ATTRNAME VALUE '',
    ATTR4 TYPE SCX_ATTRNAME VALUE '',
END OF CONNECTION_ID_IS_INITIAL,

```

```

BEGIN OF FLIGHT_NOT_EXIST,
    MSGID TYPE SYMSGID VALUE 'DMO_TRAVEL_MESSAGES',
    MSGNO TYPE SYMSGNO VALUE '009',
    ATTR1 TYPE SCX_ATTRNAME VALUE 'FLIGHT_DATE',
    ATTR2 TYPE SCX_ATTRNAME VALUE '',
    ATTR3 TYPE SCX_ATTRNAME VALUE '',
    ATTR4 TYPE SCX_ATTRNAME VALUE '',
END OF FLIGHT_NOT_EXIST,

```

```

BEGIN OF ACTION_SUCCESSFUL,
    MSGID TYPE SYMSGID VALUE 'DMO_TRAVEL_MESSAGES',
    MSGNO TYPE SYMSGNO VALUE '010',
    ATTR1 TYPE SCX_ATTRNAME VALUE '',
    ATTR2 TYPE SCX_ATTRNAME VALUE '',
    ATTR3 TYPE SCX_ATTRNAME VALUE '',
    ATTR4 TYPE SCX_ATTRNAME VALUE '',
END OF ACTION_SUCCESSFUL,

```

```

BEGIN OF INCORRECT_BOOKING_DATE,
    MSGID TYPE SYMSGID VALUE 'DMO_TRAVEL_MESSAGES',
    MSGNO TYPE SYMSGNO VALUE '011',
    ATTR1 TYPE SCX_ATTRNAME VALUE 'TRAVEL_ID',
    ATTR2 TYPE SCX_ATTRNAME VALUE '',
    ATTR3 TYPE SCX_ATTRNAME VALUE '',
    ATTR4 TYPE SCX_ATTRNAME VALUE '',
END OF INCORRECT_BOOKING_DATE.

```

DATA:

```

MV_ATTR1      TYPE STRING,
MV_ATTR2      TYPE STRING,
MV_ATTR3      TYPE STRING,
MV_ATTR4      TYPE STRING,
MV_TRAVEL_ID   TYPE DMO_I_Travel_Messages_DEMO-travelUUID OPTIONAL,
MV_BOOKING_ID  TYPE DMO_I_Booking_Messages_DEMO-BookingUUID OPTIONAL,
MV_CUSTOMER_ID TYPE /DMO/CUSTOMER_ID,
MV_CONNECTION_ID TYPE /DMO/CONNECTION-CONNECTION_ID,
MV_BEGIN_DATE  TYPE /DMO/BEGIN_DATE,
MV_END_DATE    TYPE /DMO/END_DATE,
MV_BOOKING_DATE TYPE /DMO/BOOKING_DATE,
MV_FLIGHT_DATE TYPE /DMO/FLIGHT_DATE.

```

METHODS CONSTRUCTOR

```

IMPORTING
    TEXTID      LIKE IF_T100_MESSAGE=>T100KEY OPTIONAL
    ATTR1       TYPE STRING OPTIONAL
    ATTR2       TYPE STRING OPTIONAL
    ATTR3       TYPE STRING OPTIONAL
    ATTR4       TYPE STRING OPTIONAL
    PREVIOUS    LIKE PREVIOUS OPTIONAL
    TRAVEL_ID   TYPE DMO_I_Travel_Messages_DEMO-travelUUID OPTIONAL
    BOOKING_ID  TYPE DMO_I_Booking_Messages_DEMO-BookingUUID OPTIONAL
    CUSTOMER_ID TYPE /DMO/CUSTOMER_ID OPTIONAL
    CONNECTION_ID TYPE /DMO/CONNECTION-CONNECTION_ID OPTIONAL
    BEGIN_DATE  TYPE /DMO/BEGIN_DATE OPTIONAL
    END_DATE    TYPE /DMO/END_DATE OPTIONAL
    BOOKING_DATE TYPE /DMO/BOOKING_DATE OPTIONAL
    FLIGHT_DATE TYPE /DMO/FLIGHT_DATE OPTIONAL
    SEVERITY    TYPE IF_ABAP_BEHV_MESSAGE=>T_SEVERITY OPTIONAL.

```

```

PROTECTED SECTION.
PRIVATE SECTION.
ENDCLASS.

```

```

CLASS DMO_MESSAGEWRAPPER IMPLEMENTATION.
METHOD CONSTRUCTOR ##ADT_SUPPRESS_GENERATION.

```

```

    SUPER->CONSTRUCTOR( PREVIOUS = PREVIOUS ).

```

```

    MV_ATTR1          = ATTR1.
    MV_ATTR2          = ATTR2.
    MV_ATTR3          = ATTR3.
    MV_ATTR4          = ATTR4.
    MV_TRAVEL_ID      = TRAVEL_ID.
    MV_BOOKING_ID     = BOOKING_ID.
    MV_CUSTOMER_ID    = CUSTOMER_ID.
    MV_CONNECTION_ID  = CONNECTION_ID.
    MV_BEGIN_DATE     = BEGIN_DATE.
    MV_END_DATE       = END_DATE.
    MV_BOOKING_DATE   = BOOKING_DATE.
    MV_FLIGHT_DATE    = FLIGHT_DATE.

```

```

    IF ABAP_BEHV_MESSAGE~M_SEVERITY = SEVERITY.

```

```

        CLEAR ME->TEXTID.
        IF TEXTID IS INITIAL.
            IF T100_MESSAGE~T100KEY = IF_T100_MESSAGE=>DEFAULT_TEXTID.
                ELSE.
                    IF T100_MESSAGE~T100KEY = TEXTID.
                ENDIF.

```

```

    ENDMETHOD.

```

```

ENDCLASS.

```

Exposing Messages for a Sample Business Object with Draft Capabilities

This topic explains how to integrate messages in your behavior implementation.

Prerequisites

You've created a message-wrapper class for all of your T100-messages relevant for the business object. This example uses the message texts and the message-wrapper class created in [Creating a Message Exception Class](#).

For more information about the example business object structure, refer to [Example Business Object Structure](#).

The implementation examples list the message implementation - and at the bottom of each section you can find the complete source code for each method to see how the message implementation is integrated in the context of the source code.

i Note

It's strongly recommended to add all validations to the PREPARE action. Since validation return state messages, the state messages are only rendered according to their properties when triggered during the PREPARE and before the ACTIVATE. Even though validations are triggered a second time during the ACTIVATE, the state message handling differs: Validation included in the PREPARE run on the draft instance and render state messages as such, because no rollback is triggered if the validation fails. However, if a validation fails later during the ACTIVATE, a rollback is triggered and all state messages triggered until this point are converted to transition messages by the framework. For more information about the PREPARE , see [Preparing Draft Instances for Activation](#).

Sample Implementation: `validateCustomer` (Travel)

This example shows the message implementation for `validateCustomer` of the `Travel` entity.

This validation checks whether a `CustomerID` entered by the user exists in the `/DMO/CUSTOMER` table. Since the validation checks the consistency of business object state, it returns state messages in the `reported` structure of the travel entity. To implement a state message, the component `%state_area` must be filled in with a string. `ValidateCustomer` essentially checks for two conditions of which only one can be true at a time - either the user entered a `CustomerID` that is unknown or the user didn't enter any value. Consequently, both messages can have the same `%state_area`, so that only one invalidation statement is required at the beginning of the implementation. The `%state_area` has the same name as the validation. The target field in both cases is the field `CustomerID`. As the messages are returned after an entry in the `failed` structure, all messages are categorized with the severity **error**.

i Note

It's recommended to use the same `%state_area` within the implementation of a validation. If the `%state_area` differs, each `%state_area` must be invalidated separately.

```
" Invalidate state messages
APPEND VALUE #( %tky          = ls_travel-%tky
                %state_area   = 'VALIDATE_CUSTOMER' ) TO reported-travel.

[...]

"Return state message when no CustomerID was entered
APPEND VALUE #( %tky = ls_travel-%tky
                %msg = NEW dmo_messagewrapper(
                        textid      = dmo_messagewrapper=>customer_id_not_initial
                        severity    = if_abap_behv_message=>severity-error )
                %element-customerid = if_abap_behv=>mk-on
                %state_area         = 'VALIDATE_CUSTOMER'
                ) TO reported-travel.

[...]

"Return state message if incorrect CustomerID was entered
APPEND VALUE #( %tky          = ls_travel-%tky
                %msg          = NEW dmo_messagewrapper(
                        textid = dmo_messagewrapper=>customerid_not_exist
                        severity = if_abap_behv_message=>severity-error )
                %element-customerid = if_abap_behv=>mk-on
                %state_area         = 'VALIDATE_CUSTOMER'
                ) TO reported-travel.
```

i Expand the following code sample to view the source code of `validateCustomer`.

Sample Code

```
METHOD validatecustomer.

  READ ENTITIES OF dmo_i_travel_messages IN LOCAL MODE
    ENTITY travel
      FIELDS ( customerid )
      WITH CORRESPONDING #( keys )
    RESULT DATA(lt_travel)
```

```

FAILED DATA(lt_failed).

failed = CORRESPONDING #( DEEP lt_failed ).

DATA lt_customer TYPE SORTED TABLE OF /dmo/customer WITH UNIQUE KEY customer_id.

" Optimization of DB select: extract distinct non-initial customer IDs
lt_customer = CORRESPONDING #( lt_travel DISCARDING DUPLICATES MAPPING customer_id = custo
DELETE lt_customer WHERE customer_id IS INITIAL.

IF lt_customer IS NOT INITIAL.
  " Check if customer ID exists
  SELECT FROM /dmo/customer FIELDS customer_id
        FOR ALL ENTRIES IN @lt_customer
        WHERE customer_id = @lt_customer-customer_id
  INTO TABLE @DATA(lt_customer_db).
ENDIF.

" Raise message for non existing customer id
LOOP AT lt_travel INTO DATA(ls_travel).

" Invalidate state messages
  APPEND VALUE #( %tky = ls_travel-%tky
                  %state_area = 'VALIDATE_CUSTOMER' ) TO reported-travel.

IF ls_travel-customerid IS INITIAL.
  APPEND VALUE #( %tky = ls_travel-%tky ) TO failed-travel.
  APPEND VALUE #( %tky = ls_travel-%tky
                  %msg = NEW dmo_messagewrapper(
                      textid = dmo_messagewrapper=>customer_id_not_initial
                      severity = if_abap_behv_message=>severity-error )
                  %element-customerid = if_abap_behv=>mk-on
                  %state_area = 'VALIDATE_CUSTOMER'
                  ) TO reported-travel.

ELSEIF ls_travel-customerid IS NOT INITIAL AND NOT line_exists( lt_customer_db[ custom
APPEND VALUE #( %tky = ls_travel-%tky ) TO failed-travel.

" Message if customerID is unknown
APPEND VALUE #( %tky = ls_travel-%tky
                %msg = NEW dmo_messagewrapper(
                    textid = dmo_messagewrapper=>customerid_not_exist
                    severity = if_abap_behv_message=>severity-error )
                %element-customerid = if_abap_behv=>mk-on
                %state_area = 'VALIDATE_CUSTOMER'
                ) TO reported-travel.
ENDIF.

```

Sample Implementation: setToBooked (Travel)

This example shows the message implementation for setToBooked of the Travel entity.

This action sets the overall_status field to **B** (booked). Usually, actions return transition messages, because they trigger a change in the business object. Consequently, this action returns a transition message: This message to confirm the successful transition of the overallStatus. The transition message is categorized as **success** and it's allocated to the reported-travel since it's semantically related to a request triggered on this entity. The target field is overallStatus. For transitions messages the corresponding operation must be specified in %op. In this case, the message belongs to the action setToBooked.

```

"Return transition message in case of successful request
APPEND VALUE #( %tky = result[ 1 ]-%tky
                %msg = NEW dmo_messagewrapper(
                    textid = dmo_messagewrapper=>action_successful
                    severity = if_abap_behv_message=>severity-success )
                %element-overallstatus = if_abap_behv=>mk-on

```

```

        %action-setToBooked      = if_abap_behv=>mk-on
    ) TO reported-travel.

```

i Expand the following code sample to view the source code of **setToBooked**.

Sample Code

```

" Modify travel instance
MODIFY ENTITIES OF z_fd_i_travel_messages_demo IN LOCAL MODE
    ENTITY travel
        UPDATE FIELDS ( overallstatus )
        WITH VALUE #( FOR key IN keys ( %tky = key-%tky
                                         overallstatus = 'B' ) ) "Booked

FAILED    failed.

" read changed data for action result
READ ENTITIES OF dmo_travel_messages IN LOCAL MODE
    ENTITY travel ALL FIELDS WITH
    CORRESPONDING #( keys )
RESULT DATA(lt_travel).

result = VALUE #( FOR travel IN lt_travel ( %tky      = travel-%tky
                                             %param    = travel ) ).

*add success MESSAGE at the END of the action
APPEND VALUE #( %tky = result[ 1 ]-%tky
                %msg = NEW dmo_messagewrapper(
                    textid      = dmo_messagewrapper=>action_successful
                    severity     = if_abap_behv_message=>severity-success )
                %element-overallstatus = if_abap_behv=>mk-on
                %action-setToBooked   = if_abap_behv=>mk-on
                ) TO reported-travel.

```

Sample Implementation: validateBookingDate (Booking)

This example shows the message implementation for **validateBookingDate** of the **Booking** entity.

The validation checks whether the selected booking date is between the **StartDate** and **EndDate** of the parent travel entity. In this example, the validation is included in the determine action **PREPARE**, so that the validation is triggered before the **ACTIVATE**. This validation returns a state message in case the selected1 **BookingDate** doesn't lie in between **Start**-and **End**date. The message has the severity error and the target field **BookingDate**. Since the booking entity is a direct child of the travel entity, it's required to fill in the **%path** component with the path to the travel entity by mapping both draft instances. This state message is bound to the **REPORTED-booking** structure.

i Note

If you have external numbering in your business object and pass the ID as a message variable, the message-wrapper class doesn't take care of omitting leading zeros.

"Return state message when bookingdate is not in of selected time frame

```

APPEND VALUE #( %tky          = <ls_booking>-%tky
                %state_area   = lv_state_area
                %path         = VALUE #( travel-%is_draft = <ls_booking>-%is_draft
                                         travel-traveluuid = <ls_booking>-traveluuid )
                %msg          = NEW dmo_messagewrapper(
                    textid      = dmo_messagewrapper=>incorrect_booking_date

```

```

        travel_id = <ls_booking>-traveluuid
        severity = if_abap_behv_message=>severity-error )
%element-bookingdate = if_abap_behv=>mk-on ) T0 reported-booking.

```

i Expand the following code sample to view the source code of **validateDates**.

Sample Code

```

METHOD validateDates.
  CHECK keys IS NOT INITIAL.

  READ ENTITIES OF z_fd_i_travel_messages_demo IN LOCAL MODE
    ENTITY booking
      FIELDS ( bookinguuid traveluuid bookingid bookingdate )
      WITH CORRESPONDING #( keys )
  RESULT DATA(lt_booking).

  READ ENTITIES OF z_fd_i_travel_messages_demo IN LOCAL MODE
    ENTITY booking BY \_travel
      FIELDS ( traveluuid travelid begindate enddate )
      WITH CORRESPONDING #( lt_booking )
  RESULT DATA(lt_travel).

  LOOP AT lt_booking ASSIGNING FIELD-SYMBOL(<ls_booking>) WHERE bookingdate IS NOT INITIAL.
    ASSIGN lt_travel[ KEY entity COMPONENTS traveluuid = <ls_booking>-traveluuid ] TO FIELD-
    <ls_booking>.

    IF <ls_booking>-begindate IS NOT INITIAL AND
       <ls_booking>-enddate IS NOT INITIAL.

      DATA(lv_state_area) = 'VAL_BOOKING_DATE'.

      IF <ls_booking>-%tky-%is_draft = if_abap_behv=>mk-on.
        APPEND VALUE #( %tky = <ls_booking>-%tky
                        %state_area = lv_state_area ) TO reported-booking.
      ENDIF.

      IF <ls_booking>-bookingdate < <ls_booking>-begindate OR
         <ls_booking>-bookingdate > <ls_booking>-enddate.

        APPEND VALUE #( %tky = <ls_booking>-%tky ) TO failed-booking.
        APPEND VALUE #( %tky = <ls_booking>-%tky
                        %state_area = lv_state_area
                        %path = VALUE #( travel-%is_draft = <ls_booking>-%is_
                                         travel-traveluuid = <ls_booking>-tra
                                         %msg = NEW dmo_messagewrapper(
                                             textid = dmo_messagewrapper=>incorrect_
                                             travel_id = <ls_booking>-traveluuid
                                             severity = if_abap_behv_message=>severity
                                             %element-bookingdate = if_abap_behv=>mk-on ) T0 reported-booking.
                        ) TO failed-booking.
      ENDIF.
    ENDIF.
  ENDLOOP.
ENDMETHOD.

```

Sample Implementation: Messages in Global Context

Messages that are returned from global authorization or global feature control methods don't belong to a specific instance, because global methods are called before an instance is created. That's why global methods always return entity bound transition messages. For more information, refer to [Transition Messages](#).

As global authorization is independent of the state of the entity and just checks the authorization of the user that has executed the request, the global authorization handler is used to forbid certain operations for certain user groups in general. A typical use case is to only allow specific user groups to create new instances.

Mapping Messages Between Business Objects

This topic explains end-to-end how to receive and adapt messages from a different business object.

Message Mapping

A business object determination may trigger an operation of another business object that then returns messages. To receive and adapt the messages from a foreign business object, you register the foreign business object to extend the `reported` structure of the receiving business object. You can adapt the received messages to your requirements and the messages are then mapped during the `SAVE` sequence.

1. To handle messages of other business objects, you reference the foreign business object as foreign entity in the behavior definition of your business object with the following syntax:

Sample Code

```
implementation {unmanaged | managed | abstract};
[with draft];
foreign entity EntityName alias Alias;
```

The foreign entity reference requires a redefinition of the `MAP_MESSAGES` method in the behavior implementation of the root entity. This method has changing parameter `reported` of type `response` for `reported late`. The `reported structure` contains all entities of the source business object.

Example

In the following example, a [Travel](#) business object with the child entity [Booking](#) references the foreign business object `/DMO/I_Flight_M` in the behavior definition:

Sample Code

```
implementation managed;
with draft;
foreign entity /DMO/I_Flight_M alias Flight;

define behavior for /DMO/I_Travel alias travel
implementation in class /DMO/TRAVEL_MESSAGES unique
...
```

This triggers the redefinition of the `MAP_MESSAGES` method in the behavior definition of the root view if not specified otherwise in the behavior definition - in this example, the behavior implementation of `travel`. The saver class inherits from the `CL_ABAP_BEHAVIOR_SAVER` class. For more information about `CL_ABAP_BEHAVIOR_SAVER`, refer to [Saver Classes](#).

Sample Code

```
CLASS LSC_DMO/I_Travel DEFINITION INHERITING FROM CL_ABAP_BEHAVIOR_SAVER.

PROTECTED SECTION.
METHODS MAP_MESSAGES REDEFINITION.
```

```
ENDCLASS.
```

```
CLASS LSC_DMO/I_Travel IMPLEMENTATION.
```

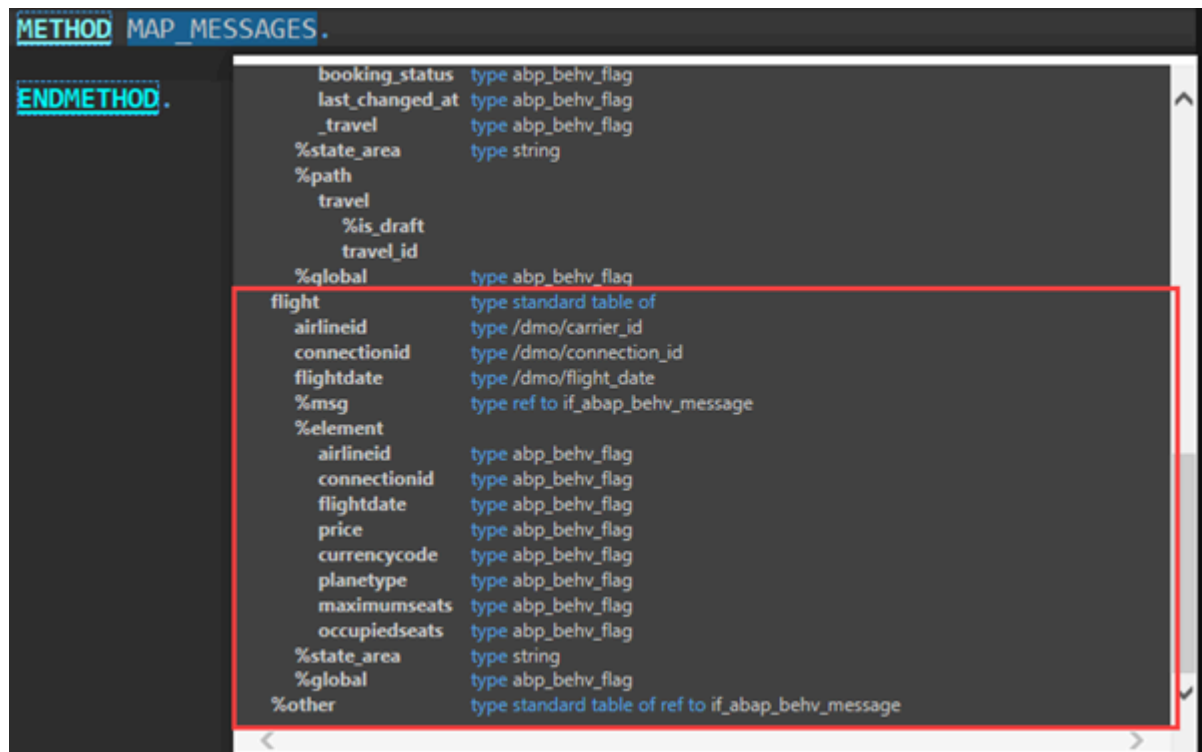
```
METHOD MAP_MESSAGES.
```

```
//Message mapping implementation
```

```
ENDMETHOD.
```

```
ENDCLASS.
```

The reported structure is extended with the entities of the referenced foreign business object. In this example, the reported structure is extended with the `flight` entity:



This gives you access to the messages that are returned in the reported structure of the `flight` entity.

2. You can map the returned messages to your business object according to your requirements, for example by:

- Mapping messages to your business object instance key (`%tky`)
- Adapting the path for messages on child instances (`%path`)
- Adapting the targets of your messages (`%element`)
- Omitting messages that do not fit certain criteria (for example: Success messages are omitted)

Working with Large Objects

Context

You can enable your RAP application for maintaining large objects (LOBs). By doing so, you provide end users the option to incorporate external binary files or text files when editing entity instances.

The following video shows how an end user performs an image upload using a RAP application, which has been enabled for the maintenance of LOBs.

The screenshot shows a Fiori Elements UI for creating a new agency. The header includes a 'New: Agency' button and tabs for 'Header' and 'Agency'. The 'Agency' tab is active, displaying 'Object Information' and a form with fields for Agency Name, Agency ID, Country/Region Key, Attachment, Street, Phone No., Postal Code, and E-Mail Address. A large orange text overlay reads 'Adding Image Attachments on a Fiori Elements UI'. At the bottom right, there are 'Create' and 'Discard Draft' buttons.

Adding Image Attachments on a Fiori Elements UI

Data Model Enhancements

Large objects are modeled by means of the following fields:

- Attachment
- Mimetype
- Filename

The field **Attachment** contains the LOB itself and is technically bound to the field **Mimetype**, which represents the content type of the attachment. No attachment can exist without its mimetype and vice versa. The filename is an optional field.

When consuming large objects via a Fiori Elements UI, the values for the fields **Mimetype** and **Filename** are derived from the field **Attachment** by the framework based on the maintained CDS annotations, so UI consumers only need to interact with the field **Attachment** to maintain large objects.

Example Scenario

We use an example from the ABAP Flight Reference Scenario to show how to enable the maintenance of large objects in a business object. In this example, we work with the business object from the package `/DMO/FLIGHT_REUSE_AGENCY` that can be used for the administration of agency master data like agency names and addresses. This guide shows how to enable your existing business object for the maintenance of file attachments such as agency flyers.

Prerequisites

To be able to follow the instructions of this guide and test the result of your enhancements, your business object should meet the following requirements:

- Your business object is create- and update-enabled.
- Your business object contains a projection layer.
- The projection layer has a metadata extension for providing CDS annotations .
- Your business object is exposed as an OData UI service.

Procedure

1. In your active and draft database tables, add the following fields for the large object:

```
attachment: /dmo/attachment;
mimetype: /dmo/mime_type;
filename: /dmo/filename;
```

The fields are typed with the corresponding base types from the /dmo/ domain. Whereas the base type /dmo/attachment uses the data type rawstring, /dmo/mime_type and /dmo/filename use the data type char128.

2. In the CDS view /DMO/I_Agency, add the fields defined for the database tables, define field aliases and use CDS annotations to semantically relate the fields to each other. The annotation contentDispositionPreference can be used to define whether, depending on the browser settings, the file attachment is either displayed in the browser (setting #INLINE) or downloaded when selected (option #ATTACHMENT). Use the CDS annotation @Semantics.mimeType: true to define the field MimeType as such.

```
@Semantics.largeObject:
{ mimeType: 'MimeType',
  fileName: 'Filename',
  contentDispositionPreference: #INLINE }
Agency.attachment          as Attachment,

@Semantics mimeType: true
Agency.mime_type          as MimeType,

Agency.filename           as Filename
```

3. In the base CDS view /DMO/R_AGENCYTP, add the fields Attachment, MimeType and Filename from the CDS view /DMO/I_Agency.
4. Also add the fields to the CDS projection views /DMO/C_AgencyTP and /DMO/I_AgencyTP.
5. In the metadata extension for the CDS projection view /DMO/C_AgencyTP, you can set the header information to be displayed on the object page. In this example scenario, the content of the Attachment field is displayed as an avatar in the header, in case the file attachment is an image. This is achieved by providing the UI annotation imageUrl: 'Attachment' in the title section.

```
@UI: { headerInfo: {
  typeName: 'Agency',
  typeNamePlural: 'Agencies',
  title: { type: #STANDARD, value: 'Name' },
          imageUrl: 'Attachment',
          description: { type: #STANDARD, value: 'AgencyID' } },
  presentationVariant: [{
    sortOrder: [{ by: 'AgencyID', direction: #ASC }],
    visualizations: [{type: #AS_LINEITEM}] } ] }
```

6. Additionally, use the CDS annotation @UI.hidden: true for the fields Mimetype and Filename to ensure that only the field Attachment is displayed for the large object on the Fiori Elements UI. As mentioned, a UI consumer only needs to

interact with the field Attachment to maintain large objects, the value handling for the fields Mimetype and Filename is taken care of by the framework.

```
@UI:
{ fieldGroup:      [ { position: 100, qualifier: 'General_FG' } ],
  identification: [ { position: 100, label: 'Attachment' } ] }
Attachment;

@UI.hidden: true
MimeType;

@UI.hidden: true
Filename;
```

Consuming Large Objects via EML

When consuming Large Objects via EML, the fields related to Large Objects need to be provided correctly by the consumer. The BO developer needs to ensure by e.g. a validation that only consistent values are persisted. For Large Objects with inconsistent field values, a correct OData orchestration with Fiori Elements is not possible.

An example for such a validation is included in the local class `lhc_agency` of the behavior pool `/dmo/bp_r_agencytp` of the agency reference scenario. The respective method `validateLargeObject` considers the following criteria:

- No attachment may exist without its mimetype.
- The attachment must not be initial for an existing mimetype. This includes both empty and non-existing attachments, since there is no way to differentiate between these two cases in ABAP.
- The mimetype must be in the list of supported mimetypes. This list is provided in the class constructor for the sake of simplicity.
- The file extension of the file name must match the file extension in the list of supported mimetypes.
- Filenames are optional. If a filename is provided, however, there must be a corresponding attachment and mimetype.

METHOD `validateLargeObject`.

```
READ ENTITIES OF /dmo/r_agencytp IN LOCAL MODE
  ENTITY /dmo/agency
  FIELDS ( attachment mimetype filename )
  WITH CORRESPONDING #( keys )
  RESULT DATA(agencies).

LOOP AT agencies INTO DATA(agency).

  APPEND VALUE #( %tky          = agency-%tky
                  %state_area = state_area_validate_lob ) TO reported-/dmo/agency.

  " No mimetype for attachment
  IF agency-attachment IS NOT INITIAL AND agency-mimetype IS INITIAL.
    APPEND VALUE #( %tky = agency-%tky ) TO failed-/dmo/agency.
    APPEND VALUE #( %tky          = agency-%tky
                    %state_area   = state_area_validate_lob
                    %msg           = NEW /dmo/cx_agency( textid      = /dmo/cx_agency=>mimety
                                                          attachment = agency-attachment )
                    %element-mimetype = if_abap_behv=>mk-on )
    TO reported-/dmo/agency.

  ENDIF.

  IF agency-mimetype IS NOT INITIAL.

    " No (empty) attachment for mimetype
    IF agency-attachment IS INITIAL.
      APPEND VALUE #( %tky = agency-%tky ) TO failed-/dmo/agency.
      APPEND VALUE #(
```

```

%tky          = agency-%tky
%state_area   = state_area_validate_lob
%msg          = NEW /dmo/cx_agency( textid      = /dmo/cx_agency=>attachment_emj
                                     attachment = agency-attachment )
%element-attachment = if_abap_behv=>mk-on )
    T0 reported-/dmo/agency.
ENDIF.

" Mimetype is not supported
IF NOT line_exists( allowed_mimetypes[ mimetype = agency-mimetype ] ).
    APPEND VALUE #( %tky = agency-%tky ) T0 failed-/dmo/agency.
    APPEND VALUE #( %tky          = agency-%tky
                    %state_area   = state_area_validate_lob
                    %msg          = NEW /dmo/cx_agency( textid      = /dmo/cx_agency=>mimety
                                                         mimetype = agency-mimetype )
                    %element-mimetype = if_abap_behv=>mk-on )
    T0 reported-/dmo/agency.
ELSE.

    IF agency-filename IS NOT INITIAL.
        DATA(file_extension) = substring_from( val = agency-filename pcre = '([^\.]*)$' ).

        " File extension does not match mimetype
        IF file_extension <> allowed_mimetypes[ mimetype = agency-mimetype ]-file_extension.
            APPEND VALUE #( %tky = agency-%tky ) T0 failed-/dmo/agency.
            APPEND VALUE #(
                %tky          = agency-%tky
                %state_area   = state_area_validate_lob
                %msg          = NEW /dmo/cx_agency( textid      = /dmo/cx_agency=>extension_mime
                                                         mimetype = agency-mimetype )
                %element-mimetype = if_abap_behv=>mk-on
                %element-filename = if_abap_behv=>mk-on )
            T0 reported-/dmo/agency.
        ENDIF.
    ENDIF.

ENDIF.

ENDIF.

ELSEIF agency-attachment IS INITIAL AND agency-filename IS NOT INITIAL.
    APPEND VALUE #( %tky = agency-%tky ) T0 failed-/dmo/agency.
    APPEND VALUE #( %tky          = agency-%tky
                    %state_area   = state_area_validate_lob
                    %msg          = NEW /dmo/cx_agency( textid      = /dmo/cx_agency=>only_1
                                                         filename = agency-filename )
                    %element-attachment = if_abap_behv=>mk-on
                    %element-mimetype   = if_abap_behv=>mk-on )
    T0 reported-/dmo/agency.

ENDIF.

ENDLOOP.

ENDMETHOD.

```

Modeling Parameters for Non-Standard Operations

This topic explains how you model parameters and use their representations in ABAP.

Usage of Parameters

Parameters in RAP are understood as input or output parameters to transmit values that are not inherently part of the BO data model. You can use parameters for non-standard operations (actions, functions, business events) to request extra information values from the operation consumer in input parameters. This information must be provided by the operation consumer, so that the operation can use the provided values for its calculations. In case of actions and functions, parameters can also be used to return results. This is realized as output parameters. When working with business events, parameters are needed to pass the relevant values to perform the event.

Depending on the use case, you can model flat or deep parameters for your non-standard operation. With flat parameters, scalar values or flat structures are handed over. With deep parameters you can use nested structure- or table-typed elements as parameter values.

For more information about actions, see [Actions](#). An example for a parameter action is given in [Implementing Actions for the Travel Entity](#).

For more information about functions, see [Functions](#). Deep parameters are not supported as input or output parameters of functions when the service is exposed with OData.

For more information about business events, see [Business Events](#). An example for the parameter usage in business events is given in [Develop Business Events](#).

Modeling Parameters

The values for parameters must be typed properly. Since parameters are not described in the BO data model, they don't need a database object to define their data model. The typing is done in a CDS abstract entity, which is defined in a CDS data definition.

For more information about abstract entities, see [ABAP CDS - Abstract Entities \(ABAP Keyword Documentation\)](#).

Flat Parameters

Flat parameters are parameters that do not reflect a hierarchical structure. They can be defined in an abstract entity. Each value of the parameter must be typed properly as an element in the abstract entity. The implementation of the non-standard action then imports a flat structure with which you can work as a regular flat ABAP structure.

The definition of a non-standard operation with a flat parameter does not include the keyword `deep`.

❖ Example

Action Definition with Flat Parameter

```
*flat input parameter (flat structure)
action MyAction parameter /dmo/a_flat_param

*flat output parameter (nested structure)
action MyAction result [1] /dmo/a_flat_param
```

Flat Parameter Modeling in CDS

```
D /DMO/A_FLAT_PARAM X
1 @EndUserText.label: 'Flat Parameter Demo'
2 define abstract entity /DMO/a_flat_param
3
4 {
5     element1 : abap.int1;
6     element2 : abap.char( 10 );
7 }
8
9
```

Representation in ABAP

```
%param          type /dmo/a_flat_param
```

Flat Parameter Modeling in CDS and its Representation in ABAP

Just like in regular CDS view entities, you can set labels, headers or quickinfos for the readable representation of the elements in an abstract entity on the UI. This is done via element annotations, see [DEFINE ABSTRACT ENTITY, element_annot \(ABAP Keyword Documentation\)](#).

Deep Parameters

Deep parameters are parameters that reflect a hierarchical structure, meaning the parameter contains elements that are in a parent-child relationship. They are defined in a composition hierarchy of abstract entities. The hierarchy is represented in ABAP as nested structures for compositions with cardinality [0..1] or [1]. It is represented as nested tables for compositions with cardinality [0..*] or [1..*].

Like in a database-represented data model, the compositional relationship must be declared in each abstract entity with the keywords `composition` or `association` to `parent` to express their relationship to other composition components. In addition, an abstract behavior definition, that includes the composition hierarchy of abstract entities must be defined to construct the hierarchical derived type for the abstract entity. Each associated abstract entity must be included in the behavior definition, as well as the related association. Beginning at the root entity, a type derived from the behavior definition structure contains all entity fields, plus a component for every composition. For a composition with cardinality [0..1] or [1], this component is a structure; otherwise with cardinality [0..*] or [1..*], the component is a table. The keyword to define such an abstract behavior definition as hierarchy model is with `hierarchy`. For more information, see [CDS BDL - CDS Abstract Behavior Definitions \(ABAP Keyword Documentation\)](#).

The definition of a non-standard operation with a deep parameter must include the keyword `deep` or `deep table` for input parameters. Output parameters of actions or functions declare their structure or table type via the cardinality. Deep result parameters with cardinality [1] represent a structure, with cardinality [1..*] they represent a table in ABAP.

❖ Example

Action Definition with Deep Parameter

```
*deep input parameter (nested structure)
action MyAction deep parameter /dmo/a_deep_param_root
*deep input parameter (nested table)
action MyAction deep table parameter /dmo/a_deep_param_root

*deep output parameter (nested structure)
action MyAction deep result [1] /dmo/a_deep_param_root
*deep output parameter (nested table)
action MyAction deep result [1..*] /dmo/a_deep_param_root
```

Deep Parameter Modeling in CDS

```

D /DMO/A_DEEP_PARAM_ROOT X
1 @EndUserText.label: 'Deep Parameter Demo Root'
2 define root abstract entity /DMO/a_deep_param_root
3
4 {
5     element1 : abap.int1;
6     element2 : abap.int1;
7     element3 : abap.int1;
8
9     _child : composition [0..*] of /DMO/a_deep_param_child ;
10 }
11

D /DMO/A_DEEP_PARAM_CHILD X
1 @EndUserText.label: 'Deep Parameter DEMP Child'
2 define abstract entity /DMO/a_deep_param_child
3
4 {
5     element1 : abap.int1;
6     element2 : abap.int1;
7     element3 : abap.int1;
8
9     _parent : association to parent /DMO/a_deep_param_root ;
10 }
11
12

B /DMO/A_DEEP_PARAM_ROOT X
1 abstract;
2 strict ( 2 );
3 with hierarchy;
4
5 define behavior for /DMO/a_deep_param_root alias Root
6 {
7     association _child;
8 }
9
10 define behavior for /DMO/a_deep_param_child alias Child
11 {
12 }

```

Deep Parameter Modeling in CDS and its Representation in ABAP

Representation in ABAP

Deep Parameter

```

%param
element1      type int1
element2      type int1
element3      type int1
_child        type standard table of
              element1
              element2
              element3

```

Deep Table Parameter

```

%param
element1      type standard table of
              element1
              element2
              element3
_child        type standard table of
              element1
              element2
              element3

```

You can use an abstract entity model for different parameters of different non-standard operations, as long as the derived types match.

⚠ Caution

You cannot use the same abstract entity model once as flat parameter and once as deep (table) parameter in the same service. The derived types of such parameters would not match and the representation in ABAP would be unclear. An error message in the service binding prevents such usage.

The representation of a parameter as flat or deep parameter depends on its definition in the non-standard operation. If the keyword `deep` or `deep table` is not used before the parameter, the parameter is represented as a flat parameter even if the abstract entities are modeled as a composition hierarchy.

→ Remember

A parameter is only represented as hierarchical structure in ABAP if it is explicitly defined as deep in the non-standard operation. For example, the definition of an action with a deep parameter must look like this in the behavior definition:

```
action myAction deep parameter myParam
```

The definition of an action with a deep table parameter must look like this:

```
action myAction deep table parameter myParam
```

Optional and Mandatory Parameters

By default, the importing parameter fields of actions and functions are implicitly mandatory. Even if a consumer does not provide any value to, for instance, an action, the default value (initial) is passed to this operation. Hence, it is not possible for the consumer to not provide any value. For the provider of the operation, on the other hand, it is not possible to distinguish between initial and NULL values.

Optional Parameters

In order to define parameter fields as optional, you can use the addition `with control` on entity level in abstract behavior definitions:

```
abstract;
strict;
with hierarchy;

define behavior for /dmo/a_deep_param_root
with control
{
}
```

This clause adds a `%CONTROL` structure to the derived type of the parameter and enables EML consumers to choose whether they want to pass a parameter field value to the operation or not by setting the `%CONTROL` flag respectively when calling the operation.

```
* Pass a parameter field value
MODIFY ENTITIES OF MyBO
  ENTITY MyEntity
  EXECUTE MyAction
    FROM VALUE #( ( ...
      %param-myfield = myvalue
      %param-%control-myfield = if_abap_behv=>mk-on
    ) )
...
.

* Pass no parameter field (NULL) value
MODIFY ENTITIES OF MyBO
  ENTITY MyEntity
  EXECUTE MyAction

    FROM VALUE #( ( ...
      %param-%control-myfield = if_abap_behv=>mk-off
    ) )
...
.
```

Mandatory Parameters

If the addition `with control` is used for an entity in abstract behavior definitions, it is possible to explicitly define its fields as mandatory by using the syntax `field ( mandatory:execute )`.


```
...
define behavior for /dmo/a_deep_param_root
with control
{
    field ( mandatory:execute ) myfield;
}
```

This requires EML consumers to pass a value for this parameter field and to set the respective %CONTROL flag to `if_abap_behv=>mk-on` as shown above. EML requests without this field setting in the %CONTROL structure result in a runtime error. On a Fiori Elements UI, consumers face an error message that prevents the execution of the operation, if no value is passed for the parameter field.

In order to define an associated entity as mandatory, you can use the syntax analogously for the respective association:

```
...
define behavior for /dmo/a_deep_param_root
with control
{
    ...
    association ( mandatory:execute ) _myassociation;
}
```

For more information, see [RAP - Field Characteristics, Abstract BDEF \(ABAP Keyword Documentation\)](#).

Redefining Parameters in Projections

Parameters that are defined for a non-standard operation (actions, functions or business events) on the base layer of a business object can be redefined on its projection layer (`transactional_query` or `transactional_interface`). To do so, the redefining parameter is specified in the projection behavior definition for the operation or the event instead of the base parameter.

```
// Base behavior definition
...
define behavior for /dmo/r_roottp
...
{
    ...
    action myaction deep parameter /dmo/a_myparam_base;
}

// Projection behavior definition
...
define behavior for /dmo/c_roottp
...
{
    ...
    action myaction deep parameter /dmo/a_myparam_proj;
}
```

The parameter on the projection layer must be a subset of the respective parameter on the base layer, meaning it can have the same or less components as the one on the base layer. Fields or associations that are defined as mandatory on the base layer must be part of the parameter on the projection layer.

Components of parameters on the projection layer can have a different name than the ones on the base layer. Their type, however, needs to be compatible. The relation to the corresponding components on the base layer must be specified using respective mapping instructions in the abstract behavior definition projection. This is necessary for every component of the projection layer parameter, even if their name does not differ from the base layer.

```
// Abstract behavior definition of redefining parameter on projection layer
abstract;
...
with hierarchy;
define behavior for /dmo/a_myparam_proj
{
mapping for /dmo/a_myparam_base
{
field_on_projection = field_on_base;

}
}
```

Only parameters from the base layer of a business object can be redefined on its projections. Parameters from a projection cannot be redefined on further projections.

For more information, see [RAP - RedefinedParameter \(ABAP Keyword Documentation\)](#).

Extending Deep Parameters

A parameter for a non-standard operation can be extended. Use cases can be:

- extending a business event signature by adding a field in an extension.
- extend an action signature by adding a field in an extension and enriching the mapping that is used in the BO's behavior definition.

For both use cases, the abstract behavior definition must be enabled for extensibility. This is done just like in standard behavior definitions with the keyword `extensible`, see [Extensibility-Enablement for Behavior Extensions](#).

Exposure and Consumption of Operations with Parameters in RAP Services

The following restrictions apply on exposing operations with deep parameters:

OData V2 only allows non-standard operations with flat parameters. The service binding shows an error message if a non-standard operation with a deep parameter is included in the service.

The following types are not supported for OData exposure:

- scalar types
- deep table parameters
- deep parameters as input or output parameters of functions.

If such parameters are used, the service binding reports an error. Operations with these parameters can still be consumed via EML.

Fiori Elements UIs can't represent hierarchical structures by default. So deep parameters can't be consumed by these UIs.

Defaulting Input Parameters for Operations

Context

When triggering an action or a function with input parameters on a Fiori Elements UI, a popup window is displayed where the end user can enter values for these parameters. In case you want to provide input assistance for the end user by presetting default values for the input parameters in the popup window, you can make use of default values functions. A default values function calculates default input parameters and provides them as its result to the respective action or function. On a Fiori Elements UI, the calculated default values are then displayed in the popup window for the input parameters when the end user triggers the respective operation.

Default values functions can also be used to default fields for create and create-by-association operations. On a Fiori Elements UI, these fields are then shown with the defaulted values when the end user triggers the create or create-by-association operation.

If you don't use a Fiori Elements UI, please note the [requirements for consuming OData apps](#).

For conceptual information on default values functions, see [Operation Defaulting](#).

Procedure

Developing a default values function comprises its definition for the related operation in the behavior definition and its implementation. The defined default values function needs to be exposed on the projection layer in addition to the related operation.

Example

The basis for this example is the action `deductDiscount` from the `/DM0/FLIGHT_DRAFT` reference scenario package as described in [Developing Transactional Apps with Exclusive Draft Capabilities](#). In this example, a default values function is developed which calculates default values for the parameter field `discount_percent` based on the data of the instance the action `deductDiscount` is executed on.

1. Define the default values function `GetDefaultsForedeductDiscount` in the behavior definition by adding the keywords `{ default function GetDefaultsForedeductDiscount ; }` to the definition of the action `deductDiscount`.

```
managed;
strict ( 2 );
with draft;

define behavior for /DM0/R_Travel_D alias Travel
...
{
  action ( features : instance, authorization : update ) deductDiscount parameter /dmo/a_trav
  ... }
```

2. Use the quick fix offered for the default values function to generate the declaration of its method in the behavior implementation.

```
CLASS lhc_travel DEFINITION INHERITING FROM cl_abap_behavior_handler
FRIENDS ltc_travel.
...
METHODS GetDefaultsForedeductDiscount FOR READ
  IMPORTING keys FOR FUNCTION Travel~GetDefaultsForedeductDiscount RESULT result.
...
```

3. Implement the default values function as follows:

- a. Read the total prices for the imported instance keys. Pass failed entries resulting from non-existing keys to the failed response structure of the method.
- b. For each read travel instance, check the total price. If the total price of an instance is greater than or equal to 5000, append a row containing its primary key field and the draft indicator as well as the value 20 as output parameter for the field `discount_percent` to the result table. If the total price is lower than 5000, use the value 10.

```
METHOD GetDefaultsForedeductDiscount.
```

```
  READ ENTITIES OF /DMO/R_TRAVEL_D
  IN LOCAL MODE
  ENTITY Travel
    FIELDS ( TotalPrice )
    WITH CORRESPONDING #( keys )
  RESULT DATA(travels)
  FAILED failed.
```

```
  LOOP AT travels INTO DATA(travel).
    IF travel-TotalPrice >= 5000.
      APPEND VALUE #( %tky                = travel-%tky
                     %param-discount_percent = 20 ) TO result.
    ELSE.
      APPEND VALUE #( %tky                = travel-%tky
                     %param-discount_percent = 10 ) TO result.
    ENDIF.
  ENDLOOP.
```

```
ENDMETHOD.
```

Develop Common Capabilities

This chapter outlines how to develop individual capabilities both for business objects and for queries.

[Working with Text Elements](#)

This section describes how to provide related text for a CDS view element in business service based on the ABAP RESTful Programming Model.

[Enabling Text and Fuzzy Searches in SAP Fiori Apps](#)

The descriptions in this topic refer to the range of functions for text and fuzzy searches that are provided in the context of SAP HANA.

[Using Virtual Elements in CDS Projection Views](#)

With virtual elements, you define additional CDS elements that are not persisted on the database, but calculated during runtime using ABAP classes that implement the virtual element interface.

Working with Text Elements

This section describes how to provide related text for a CDS view element in business service based on the ABAP RESTful Programming Model.

Text elements are used to make technical names consumable for end users. Instead of only displaying an ID element, or a technical name, the corresponding description is shown in combination with the ID elements on a Fiori UI.

ID elements are needed to uniquely identify real-life artifacts in a technical environment as readable names might not be unique, or are distinct for different languages or might change at some point or another. However, these technical names are hard to work with for end user

Examples are

- product, company, or customer names
- descriptions

Example

The product ID for supplements that can be booked for a flight are, for example, BV - 0008 or ML - 0027. These technical names are useful for uniquely identifying the products, but are not easy to remember or work with by end users. It is helpful if the readable text is displayed along with the identifier element, as can be seen in the following illustration of a travel app.

Contents

You have different options to provide text for CDS view elements:

- [Providing Text by Text Elements in the Same Entity](#)

Use this option if the identifier element and the text element are part of the same entity. You can establish a direct link between the two elements.

- [Getting Text Through Text Associations](#)

Use this option if the identifier element and the text element are not part of the same entity, or if the text element is language-dependent and shall be displayed in the system log-on language in scenarios without projection layer.

- [Getting Language-Dependent Text in Projection Views](#)

Use this option if you are working with a projection layer.

i Note

You cannot combine text associations with projection scenarios.

- [Editing Language-Dependent Fields](#)

Use this option if you want to enable editing for the denormalized text field.

Providing Text by Text Elements in the Same Entity

Language independent text elements can be maintained in the same entity as the identifier element.

Within the context of CDS views, the text elements establish the link between identifier elements (code values) of the view and its descriptive language-independent texts. For example, you can define a link between a company code and the (descriptive) company name, or between currency code and the currency name if these elements are part of the one CDS view. These kinds of descriptive texts are **language-independent**. That means, they do not contain text that is to be translated.

Relevant Annotation

Annotation	Effect
<code>@ObjectModel.text.element[]</code>	Establishes the link between the annotated element (that defines an identifier element) and its descriptive language-independent texts i Note The usage of this annotation excludes the usage of <code>@ObjectModel.text.association</code> in the same CDS entity.

More on this: [CDS Annotations](#)

i **Note**

Runtime Behavior: In scenarios with exposure via OData, only the **first text element** listed in the annotation array will be handled as a descriptive text of the annotated field.

Example

In the listing below, the CDS view `/DMO/I_SupplementText` defines the fields `Description` that serves as language-independent descriptions for the view field `SupplementID`.

```
define view /DMO/I_SupplementText
as select from /dmo/suppl_text as SupplementText

{
    @ObjectModel.text.element: ['Description']
    key SupplementText.supplement_id as SupplementID,
    ...
    SupplementText.description    as Description
    ...
}
```

Related Information

- [Getting Text Through Text Associations](#)
- [Getting Language-Dependent Text in Projection Views](#)

Getting Text Through Text Associations

Context

Using the CDS text association, you can define the relationship between an element (field) and its corresponding texts or descriptions. The texts are usually language-dependent and are displayed in the end user's language. If you annotate an element with a text association (as described below), the associated text or description field is added as a field to the referencing entity. At runtime, this field is read from the database and filtered by the logon language of the OData consumer automatically. It is not necessary to use session properties or view parameters in your CDS view.

To retrieve texts by direct use of text associations, proceed as follows:

Procedure

1. Create a data definition with a CDS view that serves as text provider
- The following annotations are required at **element level** in the text provider view to annotate the language key element and the text elements of the view's element list:

Annotation and Values	Effect
@Semantics.language: true	The annotated element identifies the language field.
@Semantics.text: true	Identifies view elements as text elements (fields pointing to a textual description) i Note In general, you can annotate more than one view field as a text field. However, only the first annotated field will be considered in the text consumer view for OData

Annotation and Values	Effect
	exposure.

More on this: [CDS Annotations](#)

2. Create a text association from your consumer CDS view to the text provider view .

The following CDS annotation is relevant when creating text associations:

Annotation and Values	Effect
@ObjectModel.text.association: '<_AssocToTextProvider>'	Name of an association with a text view that provides descriptive texts for the annotated element. In other words: the annotation indicates that the description for the annotated element is available using the text association <_AssocToTextProvider>.

More on this: [CDS Annotations](#)

The view /DM0/I_BookSuppl_T serves as a consumer for the text provider view /DM0/I_SupplText_T. For this purpose, the association _SupplementText with the text provider view as target is defined. To indicate the field for which a text should be made available through the association _SupplementText, the annotation @ObjectModel.text.association is added. Note that only the first text element (Description) from the text provider, which is annotated with @Semantics.text: true, will be considered for OData exposure. In Fiori Elements apps, the supplement ID will then be displayed together with the long text in description of the text provider view.

Text Provider View /DM0/I_SupplText_T

```
define view /DM0/I_SupplText_T as select from /dmo/suppl_text as SupplementText
{
  key SupplementText.supplement_id as SupplementID,
    @Semantics.language: true
  key SupplementText.language_code as LanguageCode,
    @Semantics.text: true
  SupplementText.description as Description
    @Semantics.text: true
  SupplementText.alt_Description as AlternativeDescription
}
```

Text Consuming View /DM0/I_BookSuppl_T

```
define root view /DM0/I_BookSuppl_T as select from /dmo/book_suppl as BookingSupplement
  association [1..*] to /DM0/I_SupplText_T as _SupplementText on $projection.supplement_id =
{
  key travel_id,
  key booking_id,
  key booking_supplement_id,
    @ObjectModel.text.association: '_SupplementText'
    supplement_id,

  /* Associations */
  _SupplementText
}
```

Related Information

[Providing Text by Text Elements in the Same Entity](#)

[Getting Language-Dependent Text in Projection Views](#)

Getting Language-Dependent Text in Projection Views

The denormalization of language-dependent text in projection views is done via the keyword `localized` for the text elements, which are included in the projection view and referenced with the annotation `@ObjectModel.text.element`: `['<text_element>']`.

Context

You have a text provider view, in which text for unreadable elements is maintained. To get the text from there you do not need any preparation in the text provider view.

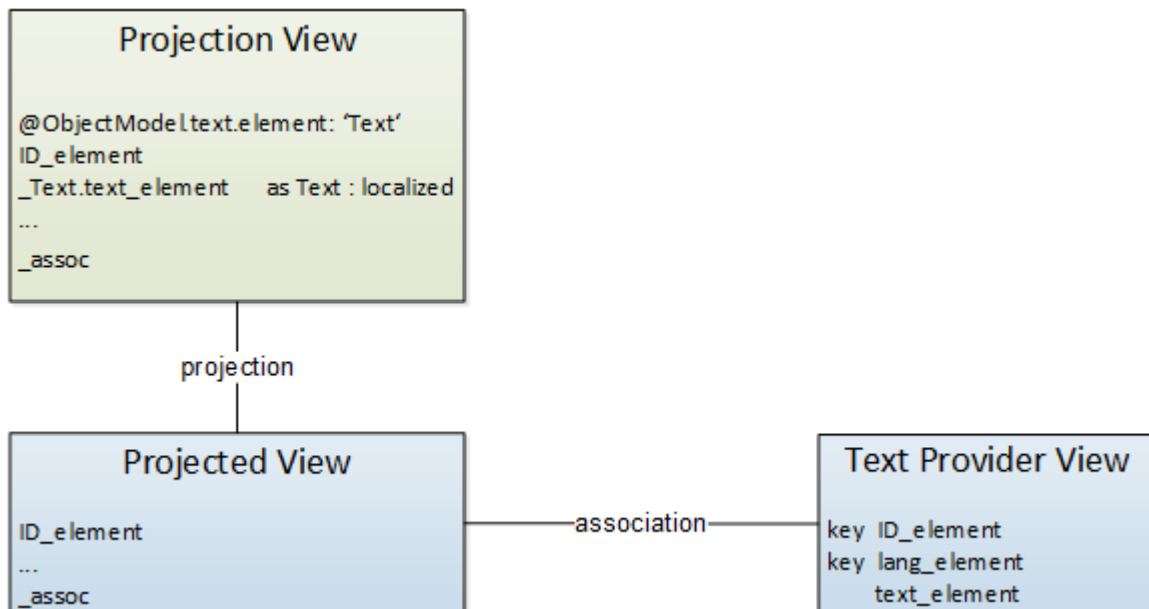
To establish a connection to this text provider view, the projected view must have an association to the text provider view.

i Note

To use the denormalization in projection views, you must not use the indicator for text associations `@ObjectModel.text.association`: `'<text_association>` in the projected view.

You can then include the text element via the association in the projection view. The relationship between identifier elements and the respective text is defined in the projection view via the annotation `@ObjectModel.text.element`: `['<text_element>']`, see [CDS Projection View](#). If the keyword `localized` is used, the text in the system log-on language is drawn.

The following diagram illustrates the modeling of text denormalization in projection views.



Prerequisites

- The text provider view has a key element indicating the language of the text.
- The projected CDS view has an association to the text provider view, but does not use the annotation `ObjectModel.text.association`.

Procedure

The example code snippets are taken from the ABAP Flight Scenario, which can be downloaded from GitHub, see [ABAP Flight Reference Scenario](#).

1. Include the text element from the text provider view into the projection view.
2. Annotate the element in the projection view, for which you want to provide the text with `@ObjectModel.text.element: [<text_element>]` and reference the text element.
3. To get language-dependent texts use the keyword `localized` on the text element.

i *Expand the following code sample to view the source code*

Sample Code

```
//Text Provider View
//The text provider view selects from a database table, in which the description texts in differ
define view entity /DM0/I_SupplementText
  as select from /dmo/suppl_text as SupplementText

{
  @ObjectModel.text.element: ['Description']
  key SupplementText.supplement_id as SupplementID,

  @Semantics.language: true
  key SupplementText.language_code as LanguageCode,

  @Search.defaultSearchElement: true
  @Search.fuzzinessThreshold: 0.8
  @Semantics.text: true
  SupplementText.description    as Description
}
Projected View
define view entity /DM0/I_BookSuppl_M
  as select from /dmo/booksuppl_m as BookingSupplement

  association          to parent /DM0/I_Booking_M as _Booking      on $projection.travel_id    = .
                                                                and $projection.booking_id = .

...
  association [1..*] to /DM0/I_SupplementText as _SupplementText on $projection.supplement_id = .
{
  key travel_id,
  key booking_id,
  key booking_supplement_id,
  supplement_id,
...
  _SupplementText
}
//Projection View
//The projection view denormalizes the supplement description and filters the relevant values ba
define view entity /DM0/C_BookSuppl_Processor_M
  as projection on /DM0/I_BookSuppl_M
{
```

```

key travel_id                as TravelID,
key booking_id              as BookingID,
key booking_supplement_id   as BookingSupplementID,
@ObjectModel.text.element: ['SupplementDescription']
supplement_id              as SupplementID,
_SupplementText.Description as SupplementDescription : localized,
...
_SupplementText
}

```

If you create a UI service for this example, you can check the results when sending the request in different languages, given that texts for different languages are provided on the database table.

Book. Supp. Number	Product ID	Product Price
4	Apple Juice (BV-0005)	3.50 EUR >
1	Lemon Limonade (BV-0008)	3.50 EUR >
5	Pear Juice (BV-0006)	3.50 EUR >
1	Mango Juice (BV-0007)	3.50 EUR >
2	Hot Chocolate (BV-0001)	2.30 EUR >
1	Alcohol free Champagne (BV-0002)	7.50 EUR >

Book. Supp. Number	Product ID	Product Price
4	Apfelsaft (BV-0005)	3,50 EUR >
1	Zitronenlimonade (BV-0008)	3,50 EUR >
5	Birnensaft (BV-0006)	3,50 EUR >
1	Mangosaft (BV-0007)	3,50 EUR >
2	Heiße Schoki (BV-0001)	2,30 EUR >
1	Alkoholfreier Champagner (BV-0002)	7,50 EUR >

Supplement Descriptions in English and German

Related Information

[Getting Text Through Text Associations](#)

[Providing Text by Text Elements in the Same Entity](#)

Enabling Text and Fuzzy Searches in SAP Fiori Apps

The descriptions in this topic refer to the range of functions for text and fuzzy searches that are provided in the context of SAP HANA.

Text and Fuzzy Searches

The full text searching (or just text search) provides you with the capability to identify natural language terms that satisfy a query and, optionally, to sort them by relevance (ranking) to the query. The most common type of search is to find texts that contain the term specified and return them in the order of their similarity to these terms.

Fuzzy search is a fast and fault-tolerant search feature of SAP HANA. The basic concept behind the **fault-tolerant search** is that a database query returns records even if the search term (user input) contains additional or missing characters, or even spelling errors. Fuzzy search can be used in various applications -- for example, to trigger a fault-tolerant search in a structured database content, like a search for a product called 'coffe krisp biscuit' and you find 'Toffee Crisp Biscuits'.

Providing Freestyle Search Capabilities in SAP Fiori UI screen

Within the context of the ABAP RESTful programming model, you only need to enable the text and fuzzy search functionality in your data model definitions. For this purpose, you implement it in designated CDS views using appropriate text and fuzzy search annotations (listed below).

i Note

As an application developer however, you must ensure that your CDS views are suitable for text and fuzzy search enabling. For more information take a look at the corresponding topics in the [SAP HANA Search Developer Guide](#).

Annotations for Text- and Fuzzy Search

As the name suggests, search annotations enable the search feature on the CDS view elements.

First of all, you need the following CDS annotation at the **view level**:

Annotation and Value	Effect
@Search.searchable: true/false	Defines whether a CDS view is generally relevant for search scenarios. This annotation provides a general switch and a means to quickly detect whether a view is search-relevant or not. Set to value true in order to enable search support by means of @Search annotations. Here, at least one view field must be defined as @defaultSearchElement at element level.

The annotations (required) at the **element level** are:

Annotation and Values	Effect
@Search.defaultSearchElement: true/false	Specifies that the annotated element is to be considered in a full-text search i Note At least one element has to be defined for the default full-text search. Searching in views without default full-text search elements is not supported! All view elements that are annotated for the default search define the search scope. (The search will be performed on all elements that have this annotation.). ⚠ Caution Such a search must not operate on all elements – for performance reasons and because not all elements qualify for this kind of access.
@Search.fuzzinessThreshold: <value>	This annotation specifies the least level of fuzziness the element has to have in order to be considered in a fuzzy search at all. The <value> defines the threshold for a fuzzy search (how fuzzy scores are calculated when comparing two strings or two terms). Possible values are: 0 . . 1 The default value is 1. The fuzzy search algorithm calculates a fuzzy score for each string comparison. The higher the score, the more similar the strings are. A score of 1 . 0 means the strings are identical. A score of 0 . 0 means the strings have nothing in common.
@Search.ranking: <value>	This annotation specifies how relevant the values of an element (view field) are for ranking, should the freestyle search terms match the element's value.

Annotation and Values	Effect
	The ranking can have the following values: • HIGH - The element is of high relevance; typically, this is useful for IDs and their descriptions. • MEDIUM - The element is of medium relevance; designated usually for important elements. • LOW - Although the element is relevant for a freestyle search, a hit for this element has no real significance for the ranking of a result item.

→ Tip

For the fuzzy search threshold, we recommend using the default value `0.7` to start with. Later on, you can fine-tune the value based on your experiences with the search. You can also fine-tune the search using feedback collected from your users.

Example

The listing below implements a search model for searching products. The model definition results from the data source `db_pd` that already specifies the persistence layer for searching. The data source provides product data and text reference fields.

The annotation `@Search.searchable: true` marks the view as searchable. In addition, the elements `Name` and `Category` are annotated with `@Search.defaultSearchElement: true`. This means that a freestyle search is enabled on the search UI where it is possible to search for the annotated elements. The annotation `@Search.fuzzinessThreshold: 0.7 (0.8)` defines that the text search should be applied to the element `Category` with a similarity value of 70% and to the element `Name` with a similarity value of 80%.

```
...

@Search.searchable : true

define view SearchForPuducts as select from db_pd

    key pd_id as ID,

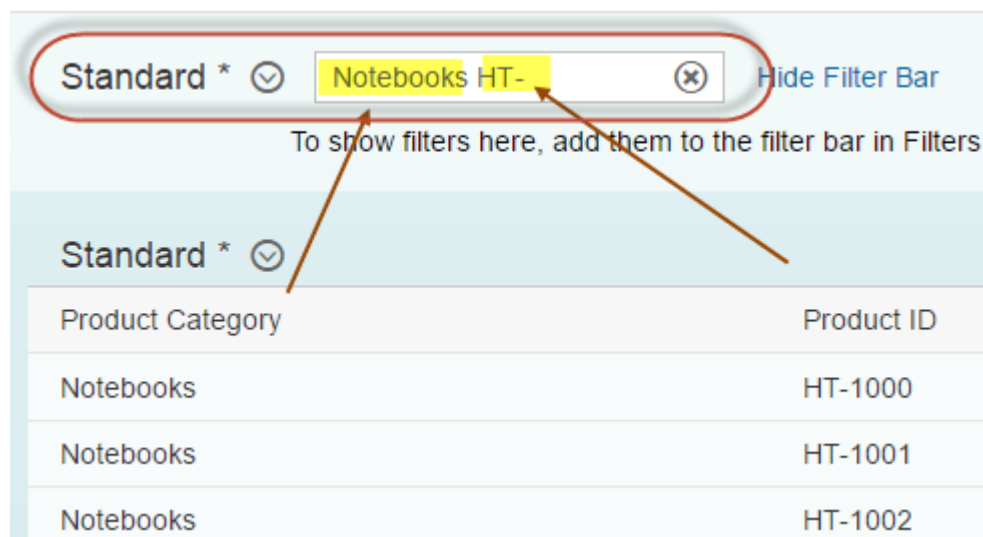
    @Search.defaultSearchElement : true
    @Search.fuzzinessThreshold : 0.8
    @Search.ranking : #HIGH
    pd_text as Name,

    @Search.defaultSearchElement : true
    @Search.fuzzinessThreshold : 0.7
    @Search.ranking : #LOW
    pd_category as Category
}
```

Results

If you expose a CDS view with search annotations for an OData service, the OData entity set receives the annotation `sap:searchable: true`.

The following image display a Fiori UI that consumes an OData service with the example CDS view.



Standard filter allows to search for product category and product name

Related Information

[CDS Annotations](#)

Using Virtual Elements in CDS Projection Views

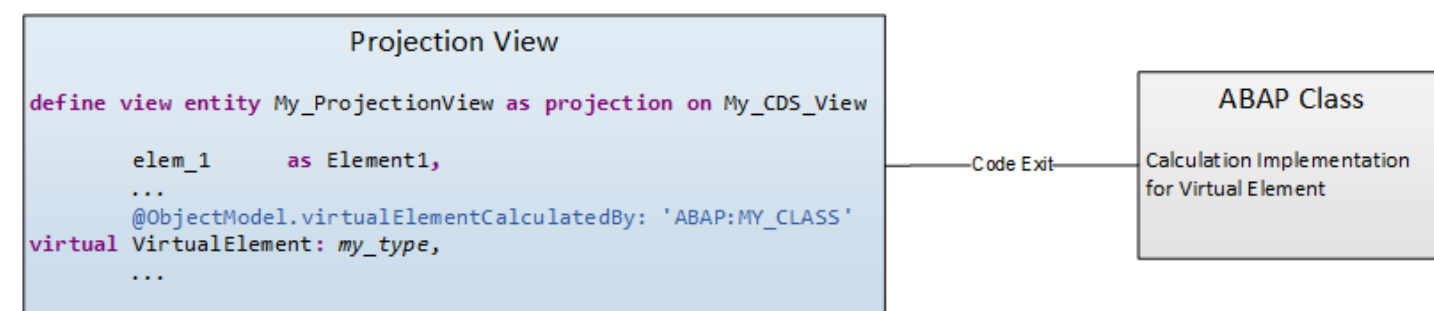
With virtual elements, you define additional CDS elements that are not persisted on the database, but calculated during runtime using ABAP classes that implement the virtual element interface.

What Are Virtual Elements, and Why Do I Need Them?

For some business use cases, it may be necessary to add new elements to the data model of an application whose values cannot be fetched from the database. Virtual elements are used if these elements are not provided as part of the original persistence data model but can be calculated using ABAP resources during runtime.

Virtual elements represent transient fields in business applications. They are defined at the level of CDS projection views as additional elements within the SELECT list. However, the calculation of their values is carried out by means of ABAP classes that implement the specific virtual element interface provided for this purpose. The ABAP implementation class is referenced by annotating the virtual element in the CDS projection view with `@ObjectModel.virtualElementCalculatedBy: 'ABAP:<CLASS_NAME>'`. With data retrieval via OData, the query framework calls the ABAP implementation class of the virtual element to retrieve the value for it. The instance of the ABAP implementation class instance is reused for all requests related to the same entity within the ABAP session.

The OData service metadata do not differentiate between regular CDS elements with database persistence and virtual elements. Consequently, the virtual element appears in an application UI equal to any other element.



Definition of Virtual Element in CDS Projection View and Implementation in Referenced ABAP Class

i Note

- You can use virtual elements only in CDS projection views.
- ☁ In ABAP Cloud, you cannot use the annotation `@ObjectModel.virtualElementCalculatedBy` in CDS extensions.
- ☁ In the current version of the ABAP RESTful Programming Model, you cannot add implementations for filtering by virtual elements.
- For RAP analytical tables, it is not possible to annotate calculation, filter or sort exits.
- Virtual elements cannot be keys of the CDS projection view.
- Virtual elements cannot be used together with the grouping or the aggregation function.
- Data from virtual elements can only be retrieved via the query framework. In particular this means that the following options to retrieve data from CDS are **not** possible for virtual elements:
 - ABAP SQL SELECT on CDS views return initial values for the virtual element,
 - EML READ on BO entities is not possible as EML does not know virtual elements.

Example

In an application that processes flight bookings, we want to include a field that calculates how many days are left until the flight date, or how many days have passed since the flight date. The following booking app UI provides the booking information with an additional field **Days to Flight** that calculates the days that have passed since the flight or the days that are left until the flight. The value is calculated by comparing the flight date with the system date.

Travel ID	Booking Number	Booking Date	Customer ID	Airline ID	Flight Number	Flight Date	Flight Price	Days to Flight
1	1	Nov 27, 2019	Ryan (594)	United Airlines, Inc. (UA)	1537	Dec 14, 2019	438.00 USD	-79 >
1	2	Dec 14, 2019	Ryan (594)	American Airlines Inc. (AA)	322	Dec 16, 2019	438.00 USD	-77 >
1	3	Sep 20, 2020	Ryan (594)	United Airlines, Inc. (UA)	1537	Oct 9, 2020	438.00 USD	221 >
1	4	Sep 24, 2020	Ryan (594)	American Airlines Inc. (AA)	322	Oct 11, 2020	438.00 USD	223 >
2	1	Dec 12, 2019	Detemple (99)	United Airlines, Inc. (UA)	1537	Dec 14, 2019	438.00 USD	-79 >
2	2	Dec 12, 2019	Sisko (660)	United Airlines, Inc. (UA)	1537	Dec 14, 2019	438.00 USD	-79 >

Flight Bookings List with Virtual Element to Calculate Days to Flight

Developer Activities

1. Adding the virtual element and corresponding annotations to the relevant CDS projection view.

For a more detailed description, see [Modeling Virtual Elements](#).

2. Creating and implementing an ABAP class that implements the virtual element calculation.

For more detailed information, see [Implementing the Calculation of Virtual Elements](#).

Modeling Virtual Elements

Virtual elements are defined in the `select` list of CDS projection views. Their values are calculated in a referenced ABAP class. This topic explains how you can model virtual elements in CDS.

Context

A virtual element is declared in the CDS projection view. It enables you to have additional fields that are not persisted on the database in your application scenario. You can also have different virtual elements in different BO projections. With those, you are flexible in providing an accurately tailored projection layer for one specific service without affecting the basic business object.

You can use virtual elements for read-only, as well as for transactional scenarios.

Virtual elements can also be used in CDS projection views that are defined as custom queries by the annotation `ObjectModel.query.implementedBy`. In this case, the custom query is evaluated first. Then, the virtual element is calculated and can use the values returned by the custom query. This can be useful, for example, in extensibility scenarios.

The following steps model a virtual element in a CDS projection view using the CDS view `/DMO/I_BOOKING_U` as an underlying CDS view. The projection CDS view defines the subset of the CDS view that is relevant to the respective service and is extended with a virtual field for this service.

For details about the syntax for defining a Virtual Element in CDS Projection View, see [CDS DDL - CDS Projection View, VIRTUAL \(ABAP Keyword Documentation\)](#).

❖ Example

```
define view entity /DMO/C_Booking_VE
  as projection on /DMO/I_Booking_U
{ ...
    @ObjectModel.virtualElementCalculatedBy: 'ABAP:/DMO/CL_DAYS_TO_FLIGHT'
    @EndUserText.label: 'Days to Flight'
    virtual DaysToFlight : abap.int2,
    ...}
```

Next Steps

1. Creating and implementing an ABAP class that implements the virtual element contract.

More on this: [Implementing the Calculation of Virtual Elements](#).

Implementing the Calculation of Virtual Elements

The values of virtual elements are calculated in a referenced ABAP class. This topic explains how you calculate their values by means of an example implementation.

Context

By using a virtual element in a CDS projection view, you define an additional field for your OData service. As this additional field is not persisted in the database layer, its value must be calculated during runtime. The calculation is implemented in an ABAP class that is referenced in an annotation on the virtual element in the CDS projection view.

The calculation class must implement the interface `IF_SADL_EXIT_CALC_ELEMENT_READ`. This interface provides two methods for the calculation implementation:

- `get_calculation_info`: This method is called before the actual data retrieval. It ensures that all the relevant elements that are needed for the calculation of the virtual element are selected.
- `calculate`: This method is called after the data retrieval. It uses the values of the relevant elements to calculate the value for the virtual element.

Prerequisites

- The virtual element is modeled in the CDS projection view as explained in [Modeling Virtual Elements](#).
- The virtual element uses the annotation `@ObjectModel.virtualElementCalculatedBy` which references an existing ABAP class.

Procedure

1. Add the interface `IF_SADL_EXIT_CALC_ELEMENT_READ` to the public section of your calculation class and add the two method implementations.

❖ Example

```

CLASS /dmo/cl_days_to_flight DEFINITION
  PUBLIC
  FINAL
  CREATE PUBLIC .

  PUBLIC SECTION.
    INTERFACES if_sadl_exit_calc_element_read.
  PROTECTED SECTION.
  PRIVATE SECTION.
ENDCLASS.

CLASS /dmo/cl_days_to_flight IMPLEMENTATION.
  METHOD if_sadl_exit_calc_element_read~get_calculation_info.
  ENDMETHOD.

  METHOD if_sadl_exit_calc_element_read~calculate.
  ENDMETHOD.

ENDCLASS.

```

2. Implement the method `get_calculation_info`.

With this method, you can check that the calculation for the virtual element is only executed if the relevant element is requested. In addition, you must provide a list of elements that are required for the calculation. You can only add elements of the requested CDS entity.

For more information, see the method description of [get_calculation_info](#).

If an entity or a virtual element other than the intended is requested, raise adequate exceptions. .

❖ Example

At first, we check if the requested entity is the entity that contains the virtual element for which the calculation is intended. To calculate the remaining time or the passed time compared to the flight date, we need the element

FlightDate. This element is added to the requested elements list to ensure that the value is available to calculate the virtual element.

It is possible to calculate the values for more than one virtual element in the same ABAP class. Depending on the virtual element, you might have to insert different elements into the list of requested elements.

i Expand the following code sample to view the source code of method `get_calculation_info`.

Sample Code

```
CLASS /dmo/cl_days_to_flight IMPLEMENTATION.
  METHOD if_sadl_exit_calc_element_read~get_calculation_info.
    IF iv_entity <> '/DMO/C_BOOKING_VE'.
      RAISE EXCEPTION TYPE /dmo/cx_virtual_elements
        EXPORTING
          textid = /dmo/cx_virtual_elements=>entity_not_known
          entity = iv_entity.
    ENDIF.

    LOOP AT it_requested_calc_elements ASSIGNING FIELD-SYMBOL(<fs_calc_element>).
      CASE <fs_calc_element>.
        WHEN 'DAYSTOFLIGHT'.
          INSERT `FLIGHTDATE` INTO TABLE et_requested_orig_elements.

*         WHEN 'ANOTHERELEMENT'.
*           INSERT `` ...

        WHEN OTHERS.
          RAISE EXCEPTION TYPE /dmo/cx_virtual_elements
            EXPORTING
              textid = /dmo/cx_virtual_elements=>virtual_element_not_known
              element = <fs_calc_element>
              entity = iv_entity.
      ENDCASE.
    ENDLOOP.
  ENDMETHOD.
ENDCLASS.
```

i Note

The methods of the interface `IF_SADL_CAL_ELEMENT_READ` import and export their parameters as strings in upper case. The CDS entity and element names must therefore be in upper case as well, so they can be mapped correctly.

3. Implement the method `calculate`.

With this method, you calculate the value for the virtual element. The parameter `it_original_data` contains the result table of the retrieved values for the elements requested by the client, in addition to the elements that are needed for calculation, which are requested for data retrieval by the method `get_calculation_info`. The parameter `it_requested_calc_elements` contains one or more virtual elements. In the changing parameter `ct_calculated_data`, you must provide the calculated value for the virtual element.

For more information, see the method description of [calculate](#).

Example

In our example, we want to calculate how many days are left before the flight or how many days have passed since the flight, so we compare the value of FLIGHTDATE with today's date to calculate the value for the virtual element.

The system date can be retrieved via `cl_abap_context_info=>get_system_date( )`.

ABAP is able to calculate with date types as long as the date context is clear. The actual calculation is therefore quite easy: We just subtract the today's date from the flight date. The returning parameter can then be filled with the calculated days to flight.

```
METHOD if_sadl_exit_calc_element_read~calculate.
  DATA(lv_today) = cl_abap_context_info=>get_system_date( ).

  DATA lt_original_data TYPE STANDARD TABLE OF /dmo/c_booking_proc_ve WITH DEFAULT KEY.
  lt_original_data = CORRESPONDING #( it_original_data ).

  LOOP AT lt_original_data ASSIGNING FIELD-SYMBOL(<fs_original_data>).
    <fs_original_data>-DaysToFlight = <fs_original_data>-FlightDate - lv_today.
  ENDLOOP.

  ct_calculated_data = CORRESPONDING #( lt_original_data ).
ENDMETHOD.
```

Related Information

[Interface IF_SADL_EXIT_CALC_ELEMENT_READ](#)

Implementing Sorting of Virtual Elements

Sorting on virtual elements is implemented in the ABAP class that is referenced in the sorting transformation for virtual elements. This topic explains how to transform the sorting condition for virtual elements.

Context

Since virtual elements are not persisted on the database, incoming sorting requests cannot be delegated to the database to get the requested sorted results. The sorting must be implemented in the related ABAP class.

The class for the sort transformation must implement the interface `IF_SADL_EXIT_SORT_TRANSFORM`. This interface provides the method `map_element`. It replaces the sorting on virtual elements with other CDS elements. The sorting direction is taken from the incoming request, but can be reverted if needed. This method is called before data retrieval whenever an OData request requests sorted responses.

Prerequisites

- The virtual element is modeled in the CDS projection view according to the instructions of [Modeling Virtual Elements](#).
- The virtual element uses the annotation `@ObjectModel.sort.transformedBy` which references an existing ABAP class.

Procedure

1. Add the interface `IF_SADL_EXIT_SORT_TRANSFORM` to the public section of your sort transformation class and add the method implementation for `map_element`.

❖ Example

```

CLASS /dmo/cl_days_to_flight_sort DEFINITION
  PUBLIC
  FINAL
  CREATE PUBLIC .

  PUBLIC SECTION.
    INTERFACES if_sadl_exit_sort_transform.

  PROTECTED SECTION.

  PRIVATE SECTION.
ENDCLASS.

CLASS /dmo/cl_days_to_flight_sort IMPLEMENTATION.
  METHOD if_sadl_exit_sort_transform~map_element.
    " Implementation of the method goes here
  ENDMETHOD.
ENDCLASS.

```

2. Implement the method map_element.

With this method, you transform the sorting condition on virtual elements to a sorting condition for other CDS elements.

❖ Example

The sorting transformation of the incoming sorting condition is defined in the method `map_element`. The incoming sort request on the virtual element must be transformed to a sort request on another CDS element.

At first, we check if the entity is the CDS in question and if the sort element is the virtual element that we want to sort. Exceptions are raised if this is not the case. Then the exporting parameter `et_sort_elements` must be filled with a regular CDS element, on which the sorting condition is transformed.

```

CLASS /dmo/cl_days_to_flight_sort IMPLEMENTATION.

  METHOD if_sadl_exit_sort_transform~map_element.
    IF iv_entity <> '/DMO/C_BOOKING_VE'.
      RAISE EXCEPTION TYPE /dmo/cx_virtual_elements
      EXPORTING
        textid = /dmo/cx_virtual_elements=>entity_not_known
        entity = iv_entity.
    ENDIF.

    IF iv_element <> 'DAYSTOFLIGHT'.
      RAISE EXCEPTION TYPE /dmo/cx_virtual_elements
      EXPORTING
        textid = /dmo/cx_virtual_elements=>sort_element_not_expected
        entity = iv_entity
        element = iv_element.
    ENDIF.
  ENDMETHOD.

```

```

    APPEND VALUE #( name = `FLIGHTDATE` reverse = abap_false ) TO et_sort_elements.
ENDMETHOD.

```

```

ENDCLASS.

```

Implementing Filtering for Virtual Elements

Filtering for virtual elements is implemented in the ABAP class that is referenced in the filter annotation for virtual elements. This topic explains how you can transform the filter for virtual elements.

Context

As virtual elements are not persisted on the database, incoming filter requests cannot be delegated to the database to get the requested filter results. The filter for virtual elements must be implemented in the related ABAP class. For each filter operator, the filtering must be transformed to a filter for the elements from which the virtual element is calculated.

The class for the filter transformation must implement the interface `IF_SADL_EXIT_FILTER_TRANSFORM`. This interface provides the method `map_atom`. This method is called before data retrieval whenever an OData requests filters for the virtual element. The interface provides constants for the following filter operators:

- equals
- less than
- greater than
- is null
- covers pattern.

Prerequisites

- The virtual element is modeled in the CDS projection view as explained in [Modeling Virtual Elements](#).
- The virtual element uses the annotation `@ObjectModel.filter.transformedBy` which references an existing ABAP class.

Procedure

1. Add the interface `IF_SADL_EXIT_FILTER_TRANSFORM` to the public section of your filter transformation class and add the method implementation for `map_atom`.

Example

```

CLASS /dmo/cl_days_to_flight_filter DEFINITION

```

```

    PUBLIC
    FINAL
    CREATE PUBLIC .

```

```

    PUBLIC SECTION.

```

```

    INTERFACES if_sadl_exit_filter_trar
    PROTECTED SECTION.
    PRIVATE SECTION.
ENDCLASS.

```

```

CLASS /dmo/cl_days_to_flight_filter
METHOD if_sadl_exit_filter_transfor

ENDMETHOD.

ENDCLASS.

```

2. Implement the method `map_atom`.

With this method, you must transform the filter for the virtual element to a filter that can be applied to other elements of the CDS entity. For each possible filter operator, define a filter condition that uses only regular CDS elements. The substitution of the filter condition with the method `map_atom` must not include virtual elements.

The simple condition factory provides the option to create such a filter condition. For more information about the interfaces used in the condition factory, see [Creating Conditions with the Simple Condition Factory](#).

❖ Example

The filter transformation of the incoming filter condition is defined in the method `map_atom`. The OData condition contains a filter URI option on the virtual element, so in this example for `DAYSTOFLIGHT`. This filter has to be transformed to a filter that can be applied on the regular CDS elements `FlightDate`.

At first, we check if the entity is the CDS in question and if the filter element is the virtual element, which we want to filter. Exceptions are raised if this is not the case.

To create a filter factory, call the public method `create_simple_cond_factory` of the class `CL_SADL_COND_PROV_PUB`. Then create a condition element for the reference element of the filter transformation. In this example, the reference element is `FlightDate`. To get the filter condition for `FlightDate`, you need to add the incoming filter value to the system date. With this date value in `lv_date`, you can compare `FlightDate` for all filter operators.

❖ Note

ABAP can handle calculations with date formats in contexts where it is clear that the value is a date type.

```

CLASS /dmo/cl_days_to_flight_filter IMPLEMENTATION.

```

```

METHOD if_sadl_exit
IF iv_entity <> '/I
RAISE EXCEPTION TYPE
EXPORTING
textid = /dmo/cx_
element = iv_entity
ENDIF.

```

```

IF iv_element <> 'I
RAISE EXCEPTION TYPE
EXPORTING
textid = /dmo/cx_
entity = iv_entit
element = iv_eleme
ENDIF.

```

```

DATA(lo_cf) = cl_
DATA(flightdate) =
DATA(lv_date) = cl_

```

```

lv_date += iv_value

CASE iv_operator.

WHEN if_sadl_exit_1
ro_condition = flic

WHEN if_sadl_exit_1
ro_condition = flic

WHEN if_sadl_exit_1
ro_condition = flic

WHEN if_sadl_exit_1
ro_condition = flic

WHEN OTHERS.
"Operator not expected
RAISE EXCEPTION TYPE
EXPORTING
textid    = /dmo/cx_
entity    = iv_entit
element   = iv_eleme
operator   = iv_operat
ENDCASE.
ENDMETHOD.

ENDCLASS.

```

Related Information

Creating Conditions with the Simple Condition Factory

Complex conditions can be built with the simple condition factory. This factory concatenates expressions with relational and logical operators; both of them represented by methods.

Context

Filtering on virtual elements requires the transformation of the given filter condition into a corresponding condition that does not use any virtual elements for the filter. The implementation of the substituting condition is done with the method `IF_SADL_EXIT_FILTER_TRANSFORM~MAP_ATOM`.

The construction of this condition is most easily done with the simple condition factory created by the method `CREATE_SIMPLE_COND_FACTORY`. This static method is called by the class `CL_SADL_COND_PROV_FACTORY_PUB`. You can build complex conditions with the simple condition factory by concatenating them with relational and logical operators.

Procedure

1. Create a simple condition factory by calling the static method `CREATE_SIMPLE_COND_FACTORY` on the public condition provider class `CL_SADL_COND_PROV_FACTORY_PUB`.

```
DATA(lo_cfacs) = cl_sadl_cond_prov_factory_pub=>create_simple_cond_factory( ).
```

The method returns a condition factory (a reference variable of type `IF_SADL_SIMPLE_COND_FACTORY` which defines the methods `TRUE`, `FALSE` and `ELEMENT`).

2. Use the condition factory to call one of the methods of the interface `IF_SADL_SIMPLE_COND_FACTORY`.

Method	Description
element	Use this method to initialize a condition. It defines the first operand represented by a non-virtual element of the CDS view. Methods of relational operators are executed on its returning value <code>ro_element</code> by using methods of relational operators. For the importing parameter <code>iv_element</code>, enter the name of the element that represents the operand 'FIELD1' in the condition. <pre>ro_element = lo_cfacs->element('FIELD1').</pre> Continue with step 3.
true	Use this method to set the condition to be always true. <pre>ro_condition = lo_cfacs->>true().</pre> To concatenate this condition with other conditions continue with step 4.
false	Use this method to set the condition to be always false. <pre>ro_condition = lo_cfacs->>false().</pre> To concatenate this condition with other conditions continue with step 4.

3. In case you used the method `ELEMENT` in the previous step, choose one of the methods for relational operators to set the operands **FIELD1**(`iv_element`) and **X**(`iv_value`) into relation with each other.

Relational Operator	Description
equals	Use this method to build an expression like FIELD1 = 72. For the importing parameter <code>iv_value</code>, enter the reference value for the relation (for example a r <pre>ro_condition = lo_cfacs->element('FIELD1')->>equals(72).</pre>
not_equals	Use this method to build a condition like FIELD1 ≠ 72. For the importing parameter <code>iv_value</code>, enter the reference value for the relation (for example a r <pre>ro_condition = lo_cfacs->element('FIELD1')->not_equals(72).</pre>
less_than	Use this method to build a condition like FIELD1 < 72. For the importing parameter <code>iv_value</code>, enter the reference value for the relation (for example a r <pre>ro_condition = lo_cfacs->element('FIELD1')->less_than(72).</pre>
greater_than	Use this method to build a condition like FIELD1 > 72.

Relational Operator	Description
	For the importing parameter <code>iv_value</code>, enter the reference value for the relation (for example a r <code>ro_condition = lo_cfac->element('FIELD1')->greater_than(72).</code>
less_than_or_equals	Use this method to build a condition like <i>FIELD1 ≤ 72.</i> For the importing parameter <code>iv_value</code>, enter the reference value for the relation (for example a r <code>ro_condition = lo_cfac->element('FIELD1')->less_than_or_equals(72).</code>
greater_than_or_equals	Use this method to build a condition like <i>FIELD1 ≥ 72.</i> For the importing parameter <code>iv_value</code>, enter the reference value for the relation (for example a r <code>ro_condition = lo_cfac->element('FIELD1')->greater_than_or_equals(72).</code>
covers_pattern	Use this method to build a condition like <i>FIELD1 covers the pattern of ABCDEF.</i> For the importing parameter <code>iv_value</code>, enter the reference value for the relation (for example a s <i>ABC*</i>). <code>ro_condition = lo_cfac->element('FIELD1')->covers_pattern('ABC*').</code>
not_covers_pattern	Use this method to build a condition like <i>FIELD1 does not cover the pattern of ABCDEF.</i> For the importing parameter <code>iv_value</code>, enter the reference value for the relation (for example a s <i>*DEF</i>). <code>ro_condition = lo_cfac->element('FIELD1')->not_covers_pattern('*DEF').</code>
is_null	Use this method to build a condition like <i>FIELD1 is null.</i> This method does not have an importing parameter. <code>ro_condition = lo_cfac->element('FIELD1')->is_null.</code>
is_not_null	Use this method to build a condition like <i>FIELD1 is not null.</i> This method does not have an importing parameter. <code>ro_condition = lo_cfac->element('FIELD1')->is_not_null.</code>
is_between	Use this method to build a condition like <i>72 ≤ FIELD1 ≤ 120.</i> For the importing parameter <code>iv_low</code> and <code>iv_high</code>, enter the values that provide the lower limit a example <i>120</i>) of the range (for example numeric values like 72 and 120). <code>ro_condition = lo_cfac->element('FIELD1')->is_between(iv_low = 72 iv_h:</code>
is_not_between	Use this method to build a condition like <i>not 72 ≤ FIELD1 ≤ 120.</i> For the importing parameter <code>iv_low</code> and <code>iv_high</code>, enter the values that provide the lower limit a range (for example numeric values like 72 and 120). <code>ro_condition = lo_cfac->element('FIELD1')->is_not_between(iv_low = 72 :</code>

4. You can concatenate the conditions built by the previous steps with the methods of the logical operators **AND**, **OR**, **NEGATED**, and **EXISTS**.

Logical Operators	Description
and	Use this method to concatenate two conditions if you want the concatenated condition to be true if both expressions (the preceding expression and the succeeding expression) are true. For the importing parameter enter a condition as built in step 1 to step 3. <pre>ro_condition = lo_condition1->and(lo_condition2).</pre>
or	Use this method to concatenate two conditions if you want the concatenated condition to be true if either of the expressions (the preceding expression and the succeeding expression) is true. For the importing parameter enter a condition as built in step 1 to step 3. <pre>ro_condition = lo_condition1->or(lo_condition2).</pre>
negated	Use this method to negate the preceding expression. This method does not have an importing parameter. <pre>ro_condition = lo_condition1->negated().</pre> This condition expresses the same content as a condition with the opposite relational operator.
exists	Use this method if the preceding condition refers to an associated CDS entity with cardinality "ONE TO MANY". The condition is true if there are entries in the source entity that have associated entries that satisfy the condition. For the importing parameter (iv_name) enter an alias for the target entity that you have chosen in the preceding condition. For the importing parameter (iv_path) enter the association path that is declared in the CDS entity definition. <pre>ro_condition = lo_cfacs->element('alias.element')->>equals(72)->exists(iv_name = 'alias')</pre>

Results

You can continuously combine conditions with relational and logical operators:

```
ro_condition = (lo_cfacs->element( 'FIELD1' )->>equals( 'xyz' )
               ->and(lo_cfacs->element( 'FIELD2' )->less_than( 65 ) ) )->or( lo_cfacs->less_than( 100 ) )
```

→ Tip

Improve the readability of your condition by predefining the first operand before you use it in the condition:

```
DATA(field1) = lo_cfacs->element( 'FIELD1' ).
```

The condition then looks like this:

```
ro_condition = field1->less_than( 100 ).
```

This condition is true for all field values of **FIELD1** that are less than 100.

API for Virtual Elements

Virtual elements are defined in CDS projection views and implemented in related ABAP classes that calculate their values and transform filtering and sorting requests. For this calculation and transformation, the generic RAP runtime engine calls the ABAP classes that implement the methods of the virtual elements API.

- Calculation: [Interface IF_SADL_EXIT_CALC_ELEMENT_READ](#)

For more information about virtual elements, see [Using Virtual Elements in CDS Projection Views](#).

Interface IF_SADL_EXIT_CALC_ELEMENT_READ

Interface that must be implemented by calculation classes for virtual elements that are referenced at the level of the CDS projection view element with the annotation `@ObjectModel.virtualElementCalculatedBy`.

This interface defines the following methods for implementing the field calculation:

Method `get_calculation_info`

This method provides a list of all elements that are required for calculating the values of the virtual elements in the requested entity.

This method is called during runtime before the retrieval of data from the database to ensure that all necessary elements for calculation are filled with data.

Signature

```
TYPES tt_elements TYPE SORTED TABLE OF string WITH UNIQUE DEFAULT KEY .

METHODS get_calculation_info IMPORTING !it_requested_calc_elements TYPE tt_elements
                                !iv_entity TYPE string
                                EXPORTING !et_requested_orig_elements TYPE tt_elements
                                RAISING   cx_sadl_exit.
```

Parameter

importing	it_requested_calc_elements	List of the virtual elements that are requested by the client and intended for calculation. The names of the virtual elements are listed as upper case strings.
	iv_entity	Name of the CDS entity that contains the requested virtual elements in <code>it_requested_calc_elements</code>. The name of the CDS entity is a string in upper case.
exporting	et_requested_orig_elements	List of the CDS elements that are needed for the calculation. The CDS elements are listed as upper case strings. i Note Only elements of the requested entity that are needed for the calculation of the virtual elements. <code>iv_entity</code> can be appended for data retrieval.

raising	cx_sadl_exit	Abstract exception class that can be used to raise exceptions and return messages related to the processing of the virtual element calculation.
---------	--------------	---

 **Example**
See [Implementing the Calculation of Virtual Elements](#).

Method calculate

Executes the value calculation for the virtual element.

This method is called during runtime after data is retrieved from the database. The elements needed for the calculation of the virtual elements are already inside the data table passed to this method. The method returns a table that contains the values of the requested virtual elements.

Signature

```
TYPES tt_elements TYPE SORTED TABLE OF string WITH UNIQUE DEFAULT KEY .

METHODS calculate IMPORTING !it_original_data          TYPE STANDARD TABLE
                     !it_requested_calc_elements TYPE tt_elements
                     CHANGING !ct_calculated_data      TYPE STANDARD TABLE
                     RAISING  cx_sadl_exit.
```

importing	it_original_data	Result table of the retrieved values for the elements requested by the client, in addition to the elements that are needed for calculation, which are requested for data retrieval by the method get_calculation_info.
	it_requested_calc_elements	List of virtual elements that are requested by the client and are intended for calculation. The names of the virtual elements are listed as upper case strings.
exporting	ct_calculated_data	Table of the virtual element for each entity instance corresponding to the table it_original_data by index.
raising	cx_sadl_exit	Abstract exception class that can be used to raise exceptions and return messages related to the processing of virtual element calculation.

With this method you ensure that the virtual element is calculated and that its values are passed to the runtime engine to process the data in the requested entity.

 **Example**
See [Implementing the Calculation of Virtual Elements](#).

Interface IF_SADL_EXIT_SORT_TRANSFORM

Interface that must be implemented by sort transformation classes annotated at the level of the CDS view element with the @ObjectModel.sort.transformedBy annotation

This interface defines the following method for implementing the sort transformation:

Method MAP_ELEMENT

Transforms the sort criteria specified for the annotated view element to sort criteria of other view elements

This method is called by the SADL framework before data is retrieved from the database.

i Note

You can also use multiple elements to replace the sort criteria. In addition, you can change the sort order if necessary.

i Note

When using this sort transformation along with the grouping or aggregation function, you must ensure that elements to be replaced are part of the requested elements (requesteset by a client). In addition, you must annotate the element to be replaced with `@Consumption.groupWith: 'replacement_element'`.

Signature

<i>importing</i>	<i>IV_ENTITY</i>	<i>type</i>	<i>STRING</i>
	<i>IV_ELEMENT</i>	<i>type</i>	<i>SADL_ENTITY_ELEMENT</i>
<i>exporting</i>	<i>ET_SORT_ELEMENTS</i>	<i>type</i>	<i>IF_SADL_EXIT_SORT_TRANSFORM=>TT_SORT_ELEMENTS</i>
<i>raising</i>	<i>CX_SADL_EXIT</i>		

Parameters

Parameter	Description
IV_ENTITY	Name of the CDS entity, such as the CDS view that defines the virtual elements in question
IV_ELEMENT	Name of the sort element
ET_SORT_ELEMENTS	Table with replacement elements that will be used for sorting instead of the annotated element For each element, it specifies whether the sort order must be kept or reversed.
CX_SADL_EXIT	Abstract exception class that can be used to return exceptions and messages related to the processing of virtual elements

Implementing an Unmanaged Query

An unmanaged query uses an ABAP interface to implement read-only access to persistent or non-persistent data. It enables a more flexible data retrieval than using the SQL push down by the query framework to retrieve data from a database table.

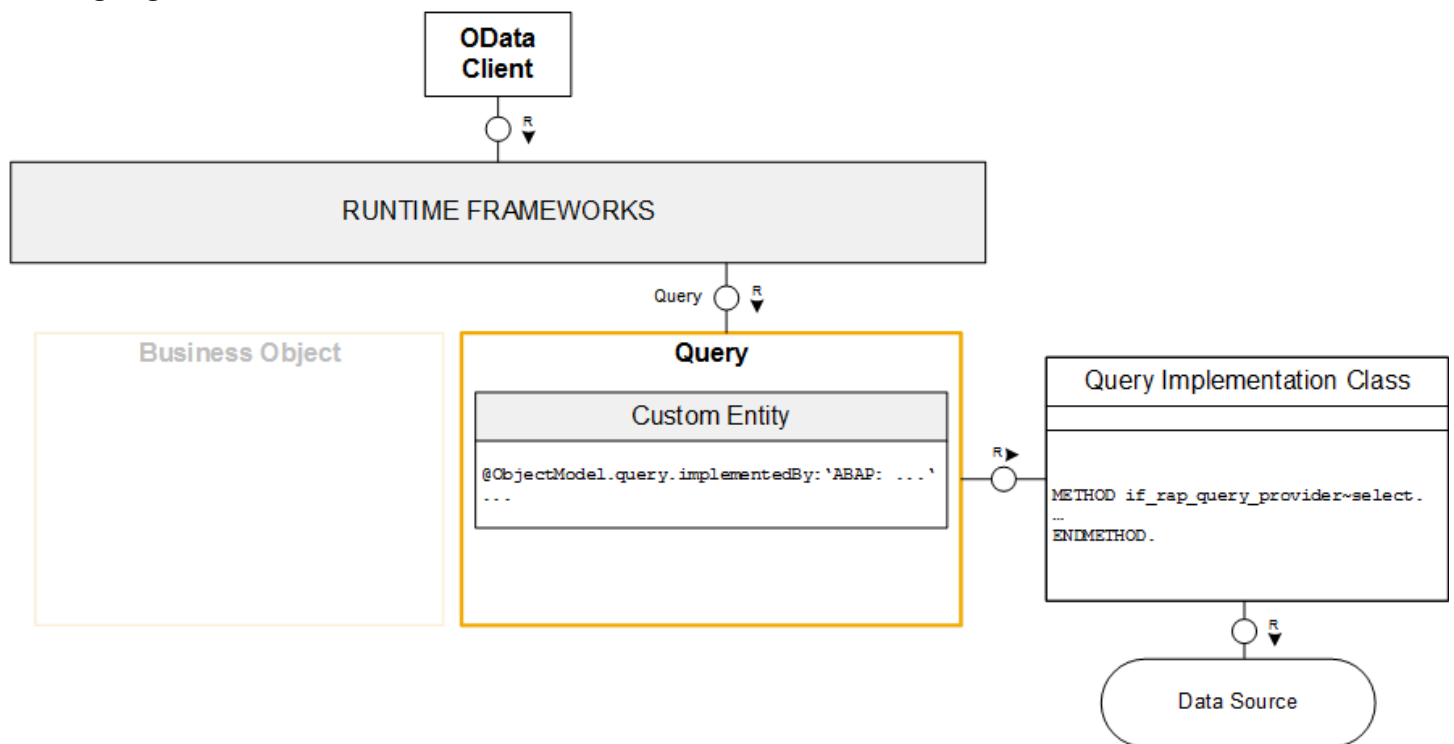
Context

An unmanaged query is implemented for read-only access to a data source whenever the standard SQL push-down by the query framework is not sufficient or not usable at all; or if there is no persistent data source at all.

Use cases for unmanaged queries are

- the data source for an OData request is not a database table, but, for example another OData service, which is reached by an OData client proxy,
- performance optimization with application specific handling,
- using AMDPs with some query push-down parameters in the SQL script implementation,
- forwarding the call to the analytical engines, or
- enrichment of query result data on property or row level, for example when splitting rows for intermediate sums or condensing the filter result.

In these cases, the data model is defined in a CDS view, for example, in a custom entity which references a query implementation class, where the query is implemented using a query provider interface. The runtime of an unmanaged query is illustrated in the following diagram.



For more background information about the unmanaged query and the custom entity, see [Unmanaged Query](#).

Example Scenario

The following sections offer an example for the implementation of the query provider interface. It is aimed to give an understanding on how to work with the interface `IF_RAP_QUERY_PROVIDER`. The example query implementation retrieves data from a database table, which is not a typical use case.

i Note

Do not use the custom query in the straight-forward case of retrieving data from a database table. The example is only used for demonstration purposes, as no background information about another technology (for example in AMDP implementations) is necessary to understand the example. The recommended implementation for such a scenario is to use a CDS view and the underlying query implementation of the SQL select by the RAP runtime engine.

The data model is defined in a custom entity. Expand the following codeblock to view the data model of the example scenario. For simplification reasons, the same element names as in the data source are used. (If you use differing names, you must map the elements of the custom entity to the corresponding table fields in the query implementation. Make sure that the types are compatible.)

For information about the syntax in custom entities, see [CDS DDL - DEFINE CUSTOM ENTITY \(ABAP Keyword Documentation\)](#). **i**

Expand the following code sample to view the source code

Custom Entity /DMO/I_TRAVEL_U

```
@EndUserText.label: 'Custom entity for unmanaged travel query'
@ObjectModel.query.implementedBy: 'ABAP:/dmo/cl_travel_uq'

define custom entity /DMO/I_TRAVEL_UQ
{
  key Travel_ID      : abap.numc( 8 );
    Agency_ID       : abap.numc( 6 );
    Customer_ID      : abap.numc( 6 );
    Begin_Date       : abap.dats;
    End_Date         : abap.dats;
    Booking_Fee      : abap.dec( 17, 3 );
    Total_Price      : abap.dec( 17, 3 );
    Currency_Code    : abap.cuky;
    Status           : abap.char( 1 );
    LastChangedAt    : timestamp;
}
```

The annotation `@ObjectModel.query.implementedBy` references the query implementation class. Learn how you implement the unmanaged query for this example scenario in the following section.

The data source in the example scenario is the database table `/dmo/travel`, see [ABAP Flight Reference Scenario](#).

Implementation

Every method provided by `IF_RAP_QUERY_PROVIDER` is used and explained in this implementation. The complete source code of the query implementation with every method is displayed after the implementation steps.

i Note

For every query request on the CDS custom entity, the implementation class is reinstantiated. The class instance is never cached.

Prerequisites

- The custom entity references the query implementation class in the annotation `@ObjectModel.query.implementedBy`.
- The query implementation class implements the interface `IF_RAP_QUERY_PROVIDER` with its `select` method.

Caution

In scenarios, in which you expose your custom entity for a Fiori UI, you have to include at least the implementation for counting and paging as the UI always requests the count and sets the query options `$top` and `$skip` for paging.

To receive consistent results on the pages, it is also indispensable to implement the method for sorting to support the query option `$orderby`. When paging is requested the RAP query engine adds the primary key to the query option for stable paging results.

If the corresponding methods are not implemented and the unmanaged query does not return the respective information, there will be an error during runtime.

i Note

To avoid high resource consumption when calling an OData service without using `$top` each OData service, based on the ABAP RESTful Programming Model uses a default paging. This includes

- Setting a default paging if the client does not provide `$top`
- Reducing the response to the default limit if `$top` exceeds this limit
- Adding a `__next` link with a `skiptoken` to the response if the response represents a partial listing, which is a subset of all available data records. See <https://www.odata.org/documentation/odata-version-2-0/json-format/> ➔ 6. Representing Collections of Entries.

i Note

The RAP runtime engine can only handle associations to or from a custom entity with attribute bindings ($A1 = A2$ and $B1 = B2$), but no:

- OR, NOT
- Other operators than '='
- Using something else than CDS elements as operands (e.g. no literals or variables)

Implementation Steps

- Check that the query is only executed when the requested entity set matches the custom entity.

For implementation details, see [Returning Requested Entity in an Unmanaged Query](#).

- Separate your implementation for data retrieval and count.

For implementation details, see [Requesting and Setting Data or Count in an Unmanaged Query](#).

- Implement filter conditions according to

- a requested filter,

For implementation details, see [Implementing Filtering in an Unmanaged Query](#).

- a requested search term,

For implementation details, see [Implementing Search in an Unmanaged Query](#).

- requested parameters.

For implementation details, see [Using Parameters in an Unmanaged Query](#).

- Get the paging information and retrieve data according to the requested page.

For implementation details, see [Implementing Paging in an Unmanaged Query](#).

- Get the sorting information and order the retrieved data according to the sort elements and direction.

For implementation details, see [Implementing Sorting in an Unmanaged Query](#).

- Get the requested elements and select only the relevant elements from the data source.

For implementation details, see [Considering Requested Elements in an Unmanaged Query](#).

- Get the aggregated and the grouped elements and aggregated and group the records accordingly.

For implementation details, see [Implementing Aggregations in an Unmanaged Query](#).

i Expand the following code sample to view the source code

Query Implementation Class /dmo/cl_travel_uq c

```

CLASS /dmo/cl_travel_uq DEFINITION
  PUBLIC
  FINAL
  CREATE PUBLIC .

  PUBLIC SECTION.
    INTERFACES if_rap_query_provider.
  PROTECTED SECTION.
  PRIVATE SECTION.
ENDCLASS.

CLASS /dmo/cl_travel_uq IMPLEMENTATION.

  METHOD if_rap_query_provider~select.
    TRY.
      CASE io_request->get_entity_id( ).

        WHEN '/DMO/I_TRAVEL_UQ' .
          **query implementation for travel entity

          **filter
          DATA(lv_sql_filter) = io_request->get_filter( )->get_as_sql_string( ).
          TRY.
            DATA(lt_filter) = io_request->get_filter( )->get_as_ranges( ).
            CATCH cx_rap_query_filter_no_range.
              "handle exception
            ENDTRY.
          **parameters
          DATA(lt_parameters) = io_request->get_parameters( ).
          DATA(lv_next_year) = CONV /dmo/end_date( cl_abap_context_info=>get_system_date( ) + 36
          DATA(lv_par_filter) = | BEGIN_DATE >= '{ cl_abap_dyn_prg=>escape_quotes( VALUE #( lt_pa
                                DEFAULT
                                | AND | &&
                                | END_DATE <= '{ cl_abap_dyn_prg=>escape_quotes( VALUE #( lt_para
                                DEFAULT
          IF lv_sql_filter IS INITIAL.
            lv_sql_filter = lv_par_filter.
          ELSE.
            lv_sql_filter = |( { lv_sql_filter } AND { lv_par_filter } )|.
          ENDIF.
          **search
          DATA(lv_search_string) = io_request->get_search_expression( ).
          DATA(lv_search_sql) = |DESCRIPTION LIKE '%{ cl_abap_dyn_prg=>escape_quotes( lv_search_s
          IF lv_sql_filter IS INITIAL.
            lv_sql_filter = lv_search_sql.
          ELSE.
            lv_sql_filter = |( { lv_sql_filter } AND { lv_search_sql } )|.
          ENDIF.
          **request data
          IF io_request->is_data_requested( ).
          **paging
            DATA(lv_offset) = io_request->get_paging( )->get_offset( ).
            DATA(lv_page_size) = io_request->get_paging( )->get_page_size( ).
            DATA(lv_max_rows) = COND #( WHEN lv_page_size = if_rap_query_paging=>page_size_unlimi
                                  THEN 0 ELSE lv_page_size ).
          **sorting
            DATA(sort_elements) = io_request->get_sort_elements( ).
            DATA(lt_sort_criteria) = VALUE string_table( FOR sort_element IN sort_elements
              ( sort_element-element_name && COND #( WHI

```



```

DATA(lv_sort_string) = COND #( WHEN lt_sort_criteria IS INITIAL THEN `primary key`
                                ELSE concat_lines_of( lt_req_elements, lv_sort_string ) );

**requested elements
DATA(lt_req_elements) = io_request->get_requested_elements( ).

**aggregate
DATA(lt_aggr_element) = io_request->get_aggregation( )->get_aggregated_elements( ).

IF lt_aggr_element IS NOT INITIAL.
  LOOP AT lt_aggr_element ASSIGNING FIELD-SYMBOL(<fs_aggr_element>).
    DELETE lt_req_elements WHERE table_line = <fs_aggr_element>-result_element.
    DATA(lv_aggregation) = |{ <fs_aggr_element>-aggregation_method }( { <fs_aggr_element>-result_element } | );
    APPEND lv_aggregation TO lt_req_elements.
  ENLOOP.
ENDIF.
DATA(lv_req_elements) = concat_lines_of( table = lt_req_elements sep = `, ` ).

****grouping
DATA(lt_grouped_element) = io_request->get_aggregation( )->get_grouped_elements( ).
DATA(lv_grouping) = concat_lines_of( table = lt_grouped_element sep = `, ` ).

**select data
DATA lt_travel_response TYPE STANDARD TABLE OF /dmo/i_travel_uq.
SELECT (lv_req_elements) FROM /dmo/travel
      WHERE (lv_sql_filter)
      GROUP BY (lv_grouping)
      ORDER BY (lv_sort_string)
      INTO CORRESPONDING FIELDS OF TABLE @lt_travel_response
      OFFSET @lv_offset UP TO @lv_max_rows ROWS.

**fill response
io_response->set_data( lt_travel_response ).
ENDIF.

**request count
IF io_request->is_total_numb_of_rec_requested( ).
**select count
SELECT COUNT( * ) FROM /dmo/travel
      WHERE (lv_sql_filter)
      INTO @DATA(lv_travel_count).

**fill response
io_response->set_total_number_of_records( lv_travel_count ).
ENDIF.

WHEN `/DMO/I_BOOKING_UQ`.
**query implementation for booking entity
ENDCASE.
CATCH cx_rap_query_provider.

ENDTRY.

ENDMETHOD.
ENDCLASS.

```

Returning Requested Entity in an Unmanaged Query

Getting the requested entity ID into your query implementation class can be helpful to ensure that the query is only executed if a specific entity is queried. You can also differentiate between query implementations of different custom entities in one query implementation class. The interface `IF_RAP_QUERY_REQUEST` provides a method to get the entity ID, which is requested.

Prerequisites

For general prerequisites of an unmanaged query implementation, see [Prerequisites Unmanaged Query](#).

Implementation Steps

The following steps provide an example on how to use the method `get_entity_id` in your query implementation class.

1. Call the method `get_entity_id` of the interface `IF_RAP_QUERY_REQUEST`, which returns the requested CDS entity name.

2. Use the returned value to compare to the custom entity the query implementation is aimed at, or to define query implementation for different custom entities in one query implementation class.

The following codeblock illustrates the implementation within the `SELECT` method of `IF_RAP_QUERY_PROVIDER` in the query implementation class.

```
CASE io_request->get_entity_id( ).
  WHEN `/DMO/I_TRAVEL_UQ` .
****query implementation for travel entity
  WHEN `/DMO/I_BOOKING_UQ` .
****query implementation for booking entity
ENDCASE.
```

For API information, see [Method `get\_entity\_id`](#).

Requesting and Setting Data or Count in an Unmanaged Query

The interface `IF_RAP_QUERY_REQUEST` provides methods to indicate whether data or the count is requested. These methods can be used to separate the implementations for data retrieval and count or to ensure that the query implementation is only executed if the respective request is made.

The interface `IF_RAP_QUERY_RESPONSE` provides methods to return the requested data or the count as a response for the request.

i Note

If data is requested, the method `set_data` must be called. If the total number of records is requested, the method `set_total_number_of_records` must be called. Otherwise there will be an error during runtime.

The `io_response->set_data( )` method is only allowed to fill elements listed by `io_request->get_requested_elements( )`. System may react with a failure or a short dump if these rules aren't followed.

In UI scenarios with Fiori Elements the total number of records is always requested by the UI.

Prerequisites

For general prerequisites of an unmanaged query implementation, see [Prerequisites Unmanaged Query](#).

Implementation Steps

Requesting and Setting Data

The following steps provide an example on how to use the method `is_data_requested` in your query implementation class.

1. Call the method `is_data_requested` of the interface `IF_RAP_QUERY_REQUEST`, which returns a boolean value.
2. Create the SQL `SELECT` to retrieve the requested data into a local variable.
3. Call the method `set_data` of the interface `IF_RAP_QUERY_RESPONSE` and use the retrieved data as importing parameter. The result is then returned to the OData client, for example the SAP Fiori Elements UI.

The following codeblock illustrates the implementation for data requests within the `select` method of `IF_RAP_QUERY_PROVIDER` in the query implementation class.

```
IF io_request->is_data_requested( ).
    DATA lt_travel_response TYPE STANDARD TABLE OF /dmo/i_travel_uq.
    SELECT * FROM /dmo/travel
           INTO CORRESPONDING FIELDS OF TABLE @lt_travel_response.
    io_response->set_data( lt_travel_response ).
ENDIF.
```

For API information, see [Method `is\_data\_requested`](#) and [Method `set\_data`](#).

Requesting and Setting Count

The following steps provide an example on how to use the method `is_total_numb_of_rec_requested` in your query implementation class.

1. Call the method `is_total_numb_of_rec_requested` of the interface `IF_RAP_QUERY_REQUEST`, which returns a boolean value.
2. Create the SQL `SELECT` to retrieve the requested count into a local variable.
3. Call the method `set_total_number_of_records` of the interface `IF_RAP_QUERY_RESPONSE` and use the retrieved data as importing parameter. The count is then returned to the OData client, for example the SAP Fiori Elements UI.

The following codeblock illustrates the implementation for data requests within the `select` method of `IF_RAP_QUERY_PROVIDER` in the query implementation class. .

```
IF io_request->is_total_numb_of_rec_requested( ).
    SELECT COUNT( * ) FROM /dmo/travel
           INTO @DATA(lv_travel_count).
    io_response->set_total_number_of_records( lv_travel_count ).
ENDIF.
```

For API information, see [Method `is\_total\_numb\_of\_rec\_requested`](#) and [Method `set\_total\_number\_of\_records`](#).

Implementing Filtering in an Unmanaged Query

To retrieve filtered data or a filtered number of records in an unmanaged query, you need to implement a filter in the query implementation class. The interface `IF_RAP_QUERY_REQUEST` provides a method to get the filter for the request with three options to retrieve the filter conditions:

- Get the filter condition as an SQL string.
- Get the filter condition in a ranges table.
- Get the filter condition in a filter tree.

Choose one of these options depending on your use case. If you retrieve your data from the data source with an SQL `SELECT`, you can include the SQL filter string directly in the `WHERE` clause of the `SELECT` statement. If you want to manipulate the filter conditions before executing the filter, a ranges table can be the better choice. Ranges tables cannot express all possible filter conditions. In cases where you want to include complex filter conditions in an easily parsable format, the filter tree is a suitable option. It also enables you to manipulate the filter conditions beforehand.

i Note

If the filter is not feasible as a ranges table, an exception is raised. You then have to handle the error appropriately.

Prerequisites

For general prerequisites of an unmanaged query implementation, see [Prerequisites Unmanaged Query](#).

Implementation Steps

Using the Filter as an SQL String

The following steps provide an example on how to use the method `get_as_sql_string` in your query implementation class.

1. Call the method `get_filter` of the interface `IF_RAP_QUERY_REQUEST`, which returns an interface instance of `IF_RAP_QUERY_FILTER`. Use the method `get_as_sql_string` to use the filter directly in an SQL string.
2. Check if data is requested.
3. Use the filter condition in the **WHERE** clause of the SQL statement to retrieve the filtered data from the data source.
4. Call the method `set_data` of the interface `IF_RAP_QUERY_RESPONSE` to respond to the OData request with the filtered data. The filtered result is then returned to the OData client, for example the SAP Fiori Elements UI.
5. Check if count is requested.
6. Use the filter condition in the **WHERE** clause of the SQL statement to retrieve the count for the data records that match the request.
7. Call the method `set_total_number_of_records` of the interface `IF_RAP_QUERY_RESPONSE` to respond to the OData request with the filtered count. The filtered result is then returned to the OData client, for example the SAP Fiori Elements UI.

The following codeblock illustrates the filter implementation within the `select` method of `IF_RAP_QUERY_PROVIDER` in the query implementation class.

```
DATA(lv_sql_filter) = io_request->get_filter( )->get_as_sql_string( ).

IF io_request->is_data_requested( ).
  DATA lt_travel_response TYPE STANDARD TABLE OF /dmo/i_travel_uq.
  SELECT * FROM /dmo/travel
        WHERE (lv_sql_filter)
        ORDER BY ('primary key')
        INTO CORRESPONDING FIELDS OF TABLE @lt_travel_response.
  io_response->set_data( lt_travel_response ).
ENDIF.

IF io_request->is_total_numb_of_rec_requested( ).
  DATA(lv_travel_count) = conv int8( lines( lt_travel_response ) ).
  io_response->set_total_number_of_records( lv_travel_count ).
ENDIF.
```

i Note

It is recommended to implement a default sort order to return consistent results from the data source.

For API information, see [Method get_filter](#).

Getting the Filter as Ranges Table

The following steps provide an example on how to use the method `get_as_ranges` in your query implementation class.

1. In a TRY - CATCH block, call the method `get_filter` of the interface `IF_RAP_QUERY_REQUEST`, which returns an interface instance of `IF_RAP_QUERY_FILTER`. Use the method `get_as_ranges` to get the filter as a ranges table. The format of the returned ranges table is described in [Method get_as_ranges](#).
2. Use the filter condition of the ranges table in your implementation.
3. Catch the exception `CX_RAP_QUERY_FILTER_NO_RANGE`, which is raised if the filter cannot be expressed as a ranges table.
4. Handle the exception appropriately. For example:
 - a. Throw an error.
 - b. Use the filter SQL string as a fall back, see [Method get_as_sql_string](#)

```
TRY.  
    DATA(lt_ranges) = io_request->get_filter( )->get_as_ranges( ).  
****filter manipulation  
    CATCH cx_rap_query_filter_no_range.  
        "error handling  
    ENDTRY.
```

For API information, see [Method get_filter](#).

Getting the Filter as Filter Tree

The following steps provide an example on how to use the method `get_as_tree` in your query implementation class.

1. Call the method `get_filter` of the interface `IF_RAP_QUERY_REQUEST`, which returns an interface instance of `IF_RAP_QUERY_FILTER`.
2. Call the method `get_as_tree` for this instance. The format of the returned filter tree is described in [Method get_as_tree](#).
3. Check if the returned filter tree reference is bound. If no filter condition is supplied, the reference is unbound and method calls using the filter tree reference variable lead to a dump.
4. If the filter tree reference is bound, you can use the methods `get_root_node`, `get_children`, `get_type` and `get_value` of the interface to parse the structure of the filter tree and use the retrieved filter condition in your implementation.

```
DATA(filter) = io_request->get_filter( ).  
DATA(filter_tree) = filter->get_as_tree( ).  
IF filter_tree IS NOT INITIAL.  
    "parse the filter tree using the methods get_root_node, get_children, get_type and get_value  
    ...  
ENDIF.
```

For API information, see [Method get_filter](#).

Using Parameters in an Unmanaged Query

To retrieve data dependent on an entity parameter that is set in the custom entity, you need to implement a handling for the parameters in the query implementation class. The interface `IF_RAP_QUERY_REQUEST` provides a method to get the parameters. It returns a string table with the parameter names and values.

It is up to the application developer how to implement the parameter logic. One option is to implement the parameters as filter criteria. To use the parameter values as a filter in the `WHERE` clause of an SQL `SELECT` statement, you need to create a filter string from the parameter values.

The interface `IF_RAP_QUERY_RESPONSE` provides a method to set the filtered data or the filtered total number of records for the query response after retrieving data from the data source.

Prerequisites

- The custom entity has one or more entity parameters.

❖ Example

```
define custom entity /DMO/I_Travel_UQ
with parameters p_start_date : /dmo/begin_date,
               p_end_date    : /dmo/end_date
```

- For general prerequisites of an unmanaged query implementation, see [Prerequisites Unmanaged Query](#).

Implementation Steps

The following steps describe the procedure of using parameters as additional filter criteria. The parameters `p_start_date` and `p_end_date` are used as a filter on the elements `Begin_Date` and `End_Date`.

1. Call the method `get_parameters` of the interface `IF_RAP_QUERY_REQUEST`, which returns a string table of the parameters and their values.
2. Define default parameter values in case the parameters are not given in the request. The implementation example uses the system date for `Begin_Date` and the system date of the following year for `End_Date`.
3. Define a variable for the filter string and fill it with the filter condition for the SQL `WHERE` clause on the element `Begin_Date` and `End_Date` and integrate the default values.

i Note

To avoid security risks via SQL string injections, use the method `escape_quotes` of the public class `CL_ABAP_DYN_PRG`.

4. If there are other filter conditions (from filter requests or parameters), concatenate the filter strings with `AND`.
5. Check if data is requested.
6. Use the filter condition in the `WHERE` clause of the SQL statement to retrieve the filtered data from the data source.
7. Call the method `set_data` of the interface `IF_RAP_QUERY_RESPONSE` to respond to the OData request with the filtered data. The filtered result is then returned to the OData client, for example the SAP Fiori Elements UI.
8. Check if count is requested.

9. Use the filter condition in the **WHERE** clause of the SQL statement to retrieve the count for the data records that match the request.

10. Call the method `set_total_number_of_records` of the interface `IF_RAP_QUERY_RESPONSE` to respond to the OData request with the filtered count. The filtered result is then returned to the OData client, for example the SAP Fiori Elements UI.

The following codeblock illustrates an implementation for parameters within the `select` method of `IF_RAP_QUERY_PROVIDER` in the query implementation class. **Expand the following code sample to view the source code**

Sample Code

```
DATA(lt_parameters) = io_request->get_parameters( ).
DATA(lv_next_year) = CONV /dmo/end_date( cl_abap_context_info=>get_system_date( ) + 365 )
DATA(lv_par_filter) = |( BEGIN_DATE >= '{ cl_abap_dyn_prg=>escape_quotes( VALUE #( lt_parame
                                                                    DEFAULT cl_
                                                                    | AND | &&
                                                                    | END_DATE <= '{ cl_abap_dyn_prg=>escape_quotes( VALUE #( lt_parameter
                                                                    DEFAULT lv_n

IF lv_sql_filter IS INITIAL.
  lv_sql_filter = lv_par_filter.
ELSE.
  lv_sql_filter = |( { lv_sql_filter } AND { lv_par_filter } )| .
ENDIF.

IF io_request->is_data_requested( ).
  DATA lt_travel_response TYPE STANDARD TABLE OF /dmo/i_travel_uq.
  SELECT * FROM /dmo/travel
        WHERE (lv_sql_filter)
        ORDER BY ('primary key')
        INTO CORRESPONDING FIELDS OF TABLE @lt_travel_response
  io_response->set_data( lt_travel_response ).
ENDIF.

IF io_request->is_total_numb_of_rec_requested( ).
  SELECT COUNT( * ) FROM /dmo/travel
        WHERE (lv_sql_filter)
        INTO @DATA(lv_travel_count).
  io_response->set_total_number_of_records( lv_travel_count ).
ENDIF.
```

i Note

It is recommended to implement a default sort order to return consistent results from the data source.

For API information, see [Method `get\_parameters`](#).

Implementing Search in an Unmanaged Query

To retrieve data according to the search term in the OData request, you need to implement a search logic in your query implementation class. The interface `IF_RAP_QUERY_REQUEST` provides a method to get the search expression from the request.

It is up to the application developer how the search logic is implemented. One option is to use the search expression as (additional) filter criteria for one or more elements. To use the search expression as a filter in the WHERE clause of an SQL SELECT statement, you need to create a filter string from the search expression and combine it with other possible filter strings.

The interface `IF_RAP_QUERY_RESPONSE` provides a method to set the filtered data for the query response after retrieving data from the data source.

Prerequisites

- For general prerequisites of an unmanaged query implementation, see [Prerequisites Unmanaged Query](#).
- To send query requests with search conditions from a Fiori Elements UI, you need to annotate the custom entity with `@Search.searchable:true`.

i Note

The annotation `@Search.searchable: true` requires using the annotation `@Search.defaultSearchElement: true` on at least one element in the custom entity. This element annotation does not have any influence on the search, as it is up to the application developer on which element(s) the search logic is implemented.

Implementation Steps

The following steps provide an example on how to implement the search as a filter for the element `Description`. If a filter is also requested, you need to ensure that the SQL filter string has the right syntax for the SQL SELECT statement.

1. Check if data is requested.
2. Call the method `get_search_expression` of the interface `IF_RAP_QUERY_REQUEST`, which returns a string of the requested search expression.
3. Create the filter string for the WHERE clause of SQL SELECT statement with the filter for the element `Description`.

i Note

To avoid security risks via SQL string injections use the method `escape_quotes` of the public class `CL_ABAP_DYN_PRG`.

4. If there are other filter conditions (from filter requests or parameters), concatenate the filter strings with `AND`.
5. Check if data is requested.
6. Use the filter condition in the WHERE clause of the SQL statement to retrieve the filtered data from the data source.
7. Call the method `set_data` of the interface `IF_RAP_QUERY_RESPONSE` to respond to the OData request with the filtered data. The filtered result is then returned to the OData client, for example the SAP Fiori Elements UI.
8. Check if count is requested.
9. Use the filter condition in the WHERE clause of the SQL statement to retrieve the count for the data records that match the request.
10. Call the method `set_total_number_of_records` of the interface `IF_RAP_QUERY_RESPONSE` to respond to the OData request with the filtered count. The filtered result is then returned to the OData client, for example the SAP Fiori Elements UI.

The following codeblock illustrates the search implementation within the `select` method of `IF_RAP_QUERY_PROVIDER` in the query implementation class.

```
DATA(lv_search_string) = io_request->get_search_expression( ).
DATA(lv_search_sql) = |DESCRIPTION LIKE '%{ cl_abap_dyn_prg=>escape_quotes( lv_search_string )

IF lv_sql_filter IS INITIAL.
    lv_sql_filter = lv_search_sql.
ELSE.
    lv_sql_filter = |( { lv_sql_filter } AND { lv_search_sql } )|.
ENDIF.

IF io_request->is_data_requested( ).
    DATA lt_travel_response TYPE STANDARD TABLE OF /dmo/i_travel_uq.
    SELECT * FROM /dmo/travel
        WHERE (lv_sql_filter)
        ORDER BY ('primary key')
        INTO CORRESPONDING FIELDS OF TABLE @lt_travel_response.
    io_response->set_data( lt_travel_response ).
ENDIF.

IF io_request->is_total_numb_of_rec_requested( ).
    SELECT COUNT( * ) FROM /dmo/travel
        WHERE (lv_sql_filter)
        INTO @DATA(lv_travel_count).
    io_response->set_total_number_of_records( lv_travel_count ).
ENDIF.
```

i Note

It is recommended to implement a default sort order to return consistent results from the data source.

For API information, see [Method `get\_search\_expression`](#).

Implementing Paging in an Unmanaged Query

To retrieve data in packages, you need to implement paging in the query implementation class. The interface `IF_RAP_QUERY_REQUEST` provides a method to get the paging information. It returns an interface instance of `IF_RAP_QUERY_PAGING` with two methods for the beginning and the number of records to be retrieved.

The method `get_offset` defines the number of records that are dropped.

i Note

In accordance to the OData query option `$skip`, the method `get_offset` does not return the position of the first data record to retrieve, but the number of records that are not taken into account before the retrieval starts. That means, the first line of the data source that is retrieved is the returning value of `get_offset` plus 1.

i Note

For retrieving all available data records, the method `get_page_size` returns the constant `page_size_unlimited` of the interface `IF_RAP_QUERY_PRAGING`. This has to be converted when using the paging information in an SQL string.

The method `get_page_size` defines the number of records that are retrieved.

The paging information can then be used in the `OFFSET` and the `UP TO n ROWS` clause of the SQL `SELECT` statement.

The interface `IF_RAP_QUERY_RESPONSE` provides a method to set the reduced data records for the query response after retrieving data from the data source.

Prerequisites

For general prerequisites of an unmanaged query implementation, see [Prerequisites Unmanaged Query](#).

Implementation Steps

The following steps provide an example on how to implement paging.

1. Call the method `get_paging` of the interface `IF_RAP_QUERY_REQUEST`, which returns an interface instance of `IF_RAP_QUERY_PAGING`. Use the method `get_offset` to get the number of records to drop into a local variable.
2. Call the method `get_paging` of the interface `IF_RAP_QUERY_REQUEST`, which returns an interface instance of `IF_RAP_QUERY_PAGING`. Use the method `get_page_size` to get the number of records to retrieve.
3. Convert the number for infinite numbers of records to be compatible with the definition of the SQL `SELECT` statement to retrieve an infinite number.

i Note

The SQL `SELECT` addition `UP TO 0 ROWS` retrieves all available data sets.

4. Use the additions `OFFSET` and `UP TO n ROWS` in the SQL `SELECT` to retrieve the data records in packages.
5. Call the method `set_data` of the interface `IF_RAP_QUERY_RESPONSE` to respond to the OData request with the reduced data records. The filtered result is then returned to the OData client, for example the SAP Fiori Elements UI.

The following codeblock illustrates the paging implementation within the `select` method of `IF_RAP_QUERY_PROVIDER` in the query implementation class.

```
IF io_request->is_data_requested( ).

    DATA(lv_offset) = io_request->get_paging( )->get_offset( ).
    DATA(lv_page_size) = io_request->get_paging( )->get_page_size( ).
    DATA(lv_max_rows) = COND #( WHEN lv_page_size = if_rap_query_paging=>page_size_unlimited THEN
                                ELSE lv_page_size ).

    DATA lt_travel_response TYPE STANDARD TABLE OF /dmo/i_travel_uq.
    SELECT * FROM /dmo/travel
            ORDER BY ('primary key')
            INTO CORRESPONDING FIELDS OF TABLE @lt_travel_response
            OFFSET @lv_offset UP TO @lv_max_rows ROWS.
    io_response->set_data( lt_travel_response ).
ENDIF.
```

i Note

It is required to implement a default sort order to return consistent results from the data source. Sorted results are essential in combination with paging. If you do not provide a default order, the data records for a certain page might not be consistent.

For API information, see [Method `get\_paging`](#).

Implementing Sorting in an Unmanaged Query

To retrieve sorted data in an unmanaged query, you need to add sorting criteria to the SQL SELECT statement. The interface `IF_RAP_QUERY_REQUEST` provides a method to get the sort element and the sort order for the request.

The method returns an ordered list of elements with their sort order. If there is more than one sort element, the sorting priority is in order of appearance. To use the sorting criteria in an SQL SELECT clause, the sorting criteria has to be transformed into a string that has comma-separated pairs of sort element and sort order.

Whereas `abap.bool` indicates the sort order for the element descending in the sorted table that you get from the query request, the sort order must be indicated with the string `ascending` or `descending` in the `ORDER BY` clause of the SQL statement.

❖ Example

The string for the SQL select statement must look like `element1 ascending , element2 descending, ...`.

The interface `IF_RAP_QUERY_RESPONSE` provides a method to set the sorted data for the query response after retrieving the data records from the data source.

Prerequisites

For general prerequisites of an unmanaged query implementation, see [Prerequisites Unmanaged Query](#).

Implementation Steps

The following steps provide an example on how to implement sorting criteria.

1. Call the method `get_sort_elements` of the interface `IF_RAP_QUERY_REQUEST`, which returns an ordered list of sort elements with the respective sort direction.
2. Write the elements of the returning value into a string table and map `abap.bool` to ' `ascending` ' or ' `descending` ' respectively.
3. Concatenate the lines of the string table with comma separation into a string variable.
4. To achieve a consistent result if there is a query request with initial sort order, provide a default sort order, for example ' `primary key` '.
5. Use the sort condition in the `ORDER BY` clause of the SQL select.
6. Call the method `set_data` of the interface `IF_RAP_QUERY_RESPONSE` to respond to the OData request with the sorted data. The sorted result is then returned to the OData client, for example the SAP Fiori Elements UI.

The following codeblock illustrates the sort implementation within the `select` method of `IF_RAP_QUERY_PROVIDER` in the query implementation class.

```

IF io_request->is_data_requested( ).

DATA(sort_elements) = io_request->get_sort_elements( ).
DATA(lt_sort_criteria) = VALUE string_table( FOR sort_element IN sort_elements
                                             ( sort_element-element_name && COND #( WHEN sort_e
                                                                                       THEN ` desc
                                                                                       ELSE ` asc

DATA(lv_sort_string) = COND #( WHEN lt_sort_criteria IS INITIAL THEN `primary key`
                               ELSE concat_lines_of( table

DATA lt_travel_response TYPE STANDARD TABLE OF /dmo/i_travel_uq.
SELECT * FROM /dmo/travel
       ORDER BY (lv_sort_string)
       INTO CORRESPONDING FIELDS OF TABLE @lt_travel_response.
io_response->set_data( lt_travel_response ).
ENDIF.

```

For API information, see [Method get_sort_elements](#).

Considering Requested Elements in an Unmanaged Query

You can optimize the performance for your unmanaged query you can add an implementation to retrieve only the elements that are requested in the OData request. If you do not specify requested element, the query retrieves the value for every element in the custom entity.

To retrieve only the elements that are requested in the OData request, you need to implement an element restriction in the SQL SELECT statement. The interface `IF_RAP_QUERY_PROVIDER` provides a method to get the requested elements for the request.

To select only the requested element in the SQL SELECT clause, you must transform the requested elements into a string with comma separations.

The interface `IF_RAP_QUERY_RESPONSE` provides a method to set the data for the query response after retrieving the relevant data from the data source.

Prerequisites

For general prerequisites of an unmanaged query implementation, see [Prerequisites Unmanaged Query](#).

Implementation Steps

The following steps provide an example on how to select only the requested elements from a data source.

1. Check if data is requested.
2. Call the method `get_requested_elements` of the interface `IF_RAP_QUERY_REQUEST`, which returns a table with the requested elements.
3. Concatenate the lines of the table with comma separation into a string variable.
4. Include the string variable in the SQL SELECT clause to retrieve only the requested elements from the data source.
5. Call the method `set_data` of the interface `IF_RAP_QUERY_RESPONSE` to respond to the OData request with the requested elements. The result is then returned to the OData client, for example the SAP Fiori Elements UI.

The following codeblock illustrates the implementation to retrieve only the requested elements within the `select` method of `IF_RAP_QUERY_PROVIDER` in the query implementation class.

```

IF io_request->is_data_requested( ).

    DATA(lt_req_elements) = io_request->get_requested_elements( ).
    DATA(lv_req_elements) = concat_lines_of( table = lt_req_elements sep = ` , ` ).

    DATA lt_travel_response TYPE STANDARD TABLE OF /dmo/i_travel_uq.
    SELECT (lv_req_elements) FROM /dmo/travel
            ORDER BY ('primary key')
            INTO CORRESPONDING FIELDS OF TABLE @lt_travel_response.
    io_response->set_data( lt_travel_response ).
ENDIF.

```

i Note

It is recommended to implement a default sort order to return consistent results from the data source.

For API information, see [Method `get\_requested\_elements`](#).

Implementing Aggregations in an Unmanaged Query

To retrieve aggregate data, you need to implement a logic to retrieve and group data according to the requested aggregations in the OData request. The interface `IF_RAP_QUERY_REQUEST` provides a method to get the aggregation information. It returns an interface instance of `IF_RAP_QUERY_AGGREGATION` with two methods to get the elements to be aggregated or grouped.

To select the requested elements according to the requested aggregation, you need to transform the requested aggregation elements into an aggregation string for the SQL SELECT. The aggregation string must look as follows:

```
<AGGR_METHOD> ( <aggr_element> ) as <result_element>
```

For more information about aggregate functions in SQL expressions, see [ABAP SQL -Aggregate Expressions `agg\_exp` \(ABAP Keyword Documentation\)](#).

To avoid double selecting the result element, you need to delete it from the list of elements that are requested as the aggregation string is used in the SQL SELECT together with the other requested elements.

To group the response data by the requested grouped elements, you need to create an SQL string for the SQL GROUP BY clause.

The interface `IF_RAP_QUERY_RESPONSE` provides a method to set the aggregated and grouped data for the query response after retrieving data from the data source.

Prerequisites

- For general prerequisites of an unmanaged query implementation, see [Prerequisites Unmanaged Query](#).
- To use aggregation in a Fiori Elements UI and send requests for aggregate values, you need to annotate the related elements with `@Aggregation.default: <aggr_method>`.

Implementation Steps

This procedure combines the implementation of requested elements and aggregation, as both are used in the select list of the SQL SELECT. The steps also provide an example on how to use the grouped elements in the SQL SELECT.

1. Check if data is requested.
2. Call the method `get_requested_elements` of the interface `IF_RAP_QUERY_REQUEST`, which returns a table with the requested elements.
3. Call the method `get_aggregation` of the interface `IF_RAP_QUERY_REQUEST`, which returns an interface instance of `IF_RAP_QUERY_AGGREGATION`. Use the method `get_aggregated_elements`, which returns a list of the elements to be aggregated, their respective aggregation method and the result element for the aggregated value.
4. Delete the result element from the requested elements.
5. Create the aggregation string for the SQL SELECT with the aggregation method, the element to be aggregated and the result element.
6. Append the aggregation string to the string of requested elements.
7. Call the method `get_aggregation` of the interface `IF_RAP_QUERY_REQUEST`, which returns an interface instance of `IF_RAP_QUERY_AGGREGATION`. Use the method `get_grouped_elements`, which returns a list of the elements that are requested as group reference.
8. Concatenate the lines of the string table with comma separation into a string variable.
9. Use the requested elements in the select list of the SQL SELECT.
10. Use the string with the grouped elements in the GROUP BY clause of the SELECT statement.

The following codeblock illustrates the implementation for aggregation and grouping select method of `IF_RAP_QUERY_PROVIDER` in the query implementation class.

```

IF io_request->is_data_requested( ).

    DATA(lt_req_elements) = io_request->get_requested_elements( ).

    DATA(lt_aggr_element) = io_request->get_aggregation( )->get_aggregated_elements( ).
    IF lt_aggr_element IS NOT INITIAL.
        LOOP AT lt_aggr_element ASSIGNING FIELD-SYMBOL(<fs_aggr_element>).
            DELETE lt_req_elements WHERE table_line = <fs_aggr_element>-result_element.
            DATA(lv_aggregation) = |{ <fs_aggr_element>-aggregation_method }| { <fs_aggr_element>-in
            APPEND lv_aggregation TO lt_req_elements.
        ENDLOOP.
    ENDIF.

    DATA(lv_req_elements) = concat_lines_of( table = lt_req_elements sep = `, ` ).

    DATA(lt_grouped_element) = io_request->get_aggregation( )->get_grouped_elements( ).
    DATA(lv_grouping) = concat_lines_of( table = lt_grouped_element sep = `, ` ).

    DATA lt_travel_response TYPE STANDARD TABLE OF /dmo/i_travel_uq.
    SELECT (lv_req_elements) FROM /dmo/travel
        GROUP BY (lv_grouping)
        ORDER BY ('primary key')
        INTO CORRESPONDING FIELDS OF TABLE @lt_travel_response.
    io_response->set_data( lt_travel_response ).
ENDIF.

```

i Note

It is recommended to implement a default sort order to return consistent results from the data source.

i Note

For API information, see [Method get_aggregation](#).